



ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΠΡΟΓΡΑΜΜΑ ΠΡΟΠΤΥΧΙΑΚΩΝ ΣΠΟΥΔΩΝ
ΤΜΗΜΑΤΟΣ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΑΛΛΗΛΕΠΙΔΡΑΣΗ ΚΑΙ ΑΜΦΙΔΡΟΜΗ ΕΠΙΔΡΑΣΗ ΠΟΙΟΤΗΤΑΣ ΚΑΙ ΑΣΦΑΛΕΙΑΣ
ΛΟΓΙΣΜΙΚΟΥ ΚΑΙ ΚΩΔΙΚΑ ΣΕ ΤΡΟΠΟΠΟΙΗΣΕΙΣ ΤΟΥ ΠΗΓΑΙΟΥ ΚΩΔΙΚΑ ΤΟΥ
ΛΟΓΙΣΜΙΚΟΥ

Πτυχιακή Εργασία

του

Αναστάσιου Τσάλμα

A.M.: ics23008

Επιβλέπων Καθηγητής: Απόστολος Αμπατζόγλου

Θεσσαλονίκη, Σεπτέμβριος 2026

Περίληψη

Η ποιότητα και η ασφάλεια του πηγαίου κώδικα του λογισμικού καθώς και του ίδιου του λογισμικού αποτελούν δύο θεμελιώδεις έννοιες της σύγχρονης ανάπτυξης λογισμικού. Παρότι συχνά εξετάζονται ανεξάρτητα, ενδέχεται να παρουσιάζουν σημαντικές μεταξύ τους σχέσεις και αλληλεπιδράσεις κατά την εξέλιξη ενός συστήματος λογισμικού. Οι συχνές τροποποιήσεις του πηγαίου κώδικα μπορούν να μεταβάλλουν χαρακτηριστικά τα οποία σχετίζονται τόσο με την ποιότητα όσο και με την ασφάλεια, χωρίς όμως η κατεύθυνση και η ένταση των μεταξύ τους σχέσεων να είναι πάντα προφανείς σε επίπεδο επιμέρους αλλαγών. Το ζήτημα αποκτά ιδιαίτερο ενδιαφέρον όταν μία τροποποίηση αποσκοπεί στη βελτίωση μιας διάστασης ενδέχεται να συνδέεται με μεταβολές σε χαρακτηριστικά της άλλης. Παρότι η ποιότητα και η ασφάλεια έχουν εκτενώς μελετηθεί ως αυτοτελείς διαστάσεις της αξιοπιστίας και της ορθής λειτουργίας των συστημάτων λογισμικού, η κοινή εμπειρική εξέτασή τους σε πραγματικά Pull Requests εξακολουθεί να αποτελεί σύνθετο ερευνητικό αντικείμενο.

Σκοπός της παρούσας εργασίας τίθεται η εμπειρική διερεύνηση της σχέσης και της πιθανής αλληλεπίδρασης μεταξύ της ποιότητας και της ασφάλειας του λογισμικού και του πηγαίου κώδικα που το υλοποιεί. Παράλληλα, εξετάζεται η σχέση χαρακτηριστικών της τελικής κατάστασης του αναλυόμενου κώδικα, όπως το μέγεθος και η πολυπλοκότητά του, καθώς και χαρακτηριστικών του Pull Request, όπως η κατηγορία στην οποία ανήκει και το project, με τις παρατηρούμενες μετρικές και πυκνότητες προβλημάτων ποιότητας και ασφάλειας.

Για την πραγματοποίηση της έρευνας συγκροτήθηκε σύνολο δεδομένων από Pull Requests (PR) πραγματικών έργων λογισμικού, τα οποία χρησιμοποιεί η κοινότητα καθώς και μεγάλοι οργανισμοί για την επίτευξη των στόχων τους και βρίσκονται στην λεγόμενη κατάσταση παραγωγής (Production), δηλαδή είναι διαθέσιμα για χρήση από τους ενδιαφερόμενους συνεργάτες ή πελάτες ενός οργανισμού. Η συλλογή και ο εμπλουτισμός των δεδομένων βασίστηκαν σε δύο συμπληρωματικές πηγές πληροφορίας. Αρχικά, αξιοποιήθηκαν δεδομένα του GitHub/GitHub API σχετικά με τα Pull Requests και το περιεχόμενο των αντίστοιχων αλλαγών, όπως ο τίτλος, το body/description, τα labels, τα changed files, τα comments και τα reviews, σε συνδυασμό με το πλήρες .diff/.patch. Οι πληροφορίες αυτές χρησιμοποιήθηκαν σε αυτοματοποιημένη multi-label διαδικασία ταξινόμησης με το μοντέλο gpt-oss-120b, από την οποία προέκυψαν οι κατηγορίες τεχνικού προσανατολισμού των Pull Requests. Παράλληλα, πραγματοποιήθηκε στατική ανάλυση του πηγαίου κώδικα με το εργαλείο SonarQube, αρχικά τόσο για την κατάσταση pr_base όσο και για την κατάσταση pr_closed, με στόχο να εξαχθούν μετρικές σχετικές με το μέγεθος και την πολυπλοκότητα του αναλυόμενου κώδικα, καθώς και indicators ποιότητας και ασφάλειας, όπως bugs, code smells, vulnerabilities, security hotspots και ζητήματα αξιοπιστίας, συντηρησιμότητας και ασφάλειας.

Για την ανάλυση των δεδομένων εφαρμόστηκαν περιγραφικές και πολυμεταβλητές στατιστικές τεχνικές, καθώς και μοντέλα παλινδρόμησης και μικτών επιδράσεων προκειμένου να διερευνηθούν οι σχέσεις μεταξύ των χαρακτηριστικών των Pull Requests και των μετρικών ποιότητας και ασφάλειας, λαμβάνοντας όμως

ταυτόχρονα υπόψη τη διαφοροποίηση μεταξύ διαφορετικών έργων λογισμικού. Τα Ordinary Least Squares models χρησιμοποιήθηκαν ως baseline προσεγγίσεις, ενώ τα Linear Mixed-Effects Models αποτέλεσαν την κύρια inferential προσέγγιση, επιτρέποντας τη ρητή μοντελοποίηση της ομαδοποίησης των Pull Requests στα αντίστοιχα projects και της σημαντικής μεταξύ-project ετερογένειας. Παράλληλα, η Principal Component Analysis και η Factor Analysis χρησιμοποιήθηκαν για τη διερεύνηση της πολυμεταβλητής δομής των εξεταζόμενων μετρικών και της δυνατότητας αναπαράστασής τους μέσω μικρότερου αριθμού ευρύτερων διαστάσεων. Συμπληρωματικά εφαρμόστηκαν robustness και sensitivity analyses για την αξιολόγηση της σταθερότητας των κύριων αποτελεσμάτων υπό εναλλακτικές αναλυτικές προδιαγραφές.

Τα αποτελέσματα της έρευνας δείχνουν ότι η πολυπλοκότητα του αναλυόμενου κώδικα παρουσιάζει σταθερή σχέση με την πυκνότητα προβλημάτων συντηρησιμότητας, ενώ οι αντίστοιχες σχέσεις με τους δείκτες ασφάλειας εμφανίζονται ασθενέστερες και περισσότερο εξαρτώμενες από τη χρησιμοποιούμενη στατιστική προσέγγιση. Τα μοντέλα, παράλληλα, μικτών επιδράσεων ανέδειξαν έντονη διαφοροποίηση μεταξύ των επιμέρους έργων λογισμικού, υποδεικνύοντας ότι το project-specific context αποτελεί σημαντικό παράγοντα για την ερμηνεία των μετρικών τόσο ποιότητας όσο και ασφάλειας. Η κατηγορία του Pull Request δεν παρουσίασε σταθερή προβλεπτική αξία στα κύρια μοντέλα, ενώ η Ανάλυση Κύριων Συνιστωσών και η Παραγοντική Ανάλυση ανέδειξαν μία πολυδιάστατη δομή, στην οποία οι δείκτες πολυπλοκότητας και συντηρησιμότητας διακρίνονται σε σημαντικό βαθμό από τη διάσταση της ασφάλειας.

Η αρχικά σχεδιασμένη σύγκριση των μεταβολών μεταξύ pr_base και pr_closed δεν κατέστη δυνατό να χρησιμοποιηθεί ως η κύρια αναλυτική προσέγγιση, καθώς αρκετά μεγάλο ποσοστό των εξεταζόμενων μετρικών παρέμενε αμετάβλητο μεταξύ των δύο καταστάσεων, με αποτέλεσμα οι αντίστοιχες διαφορές να παρουσιάζουν ανεπαρκή μεταβλητότητα για αξιόπιστη longitudinal ανάλυση. Διά τούτου, τα κύρια συμπεράσματα της εργασίας αφορούν τις cross-sectional σχέσεις που παρατηρούνται στην τελική κατάσταση των Pull Requests και όχι την εμπειρική τεκμηρίωση χρονικών συνεργειών, αντισταθμίσεων ή αμφίδρομων επιδράσεων μεταξύ μεταβολών ποιότητας και ασφάλειας.

Τα ευρήματα της παρούσας ερευνητικής εργασίας αναδεικνύουν τη σημαντικότητα της συνδυαστικής αξιολόγησης και context-aware αξιολόγησης της ποιότητας και της ασφάλειας κατά την εξελικτική πορεία του λογισμικού. Παράλληλα, μπορούν να συμβάλουν στην ανάπτυξη μεθόδων και εργαλείων που αξιοποιούν πολλαπλές μετρικές και το ιδιαίτερο πλαίσιο κάθε έργου για την πιο αποτελεσματική αξιολόγηση των Pull Requests και την έγκαιρη αναγνώριση περιπτώσεων που απαιτούν αυξημένη προσοχή κατά το code review και τους αυτοματοποιημένους ελέγχους ποιότητας και ασφάλειας.

Λέξεις Κλειδιά: Λογισμικό, Πηγαίος Κώδικας, Ποιότητα Λογισμικού, Ασφάλεια Λογισμικού, Ποιότητα Πηγαίου Κώδικα, Ασφάλεια Πηγαίου Κώδικα, Pull

Requests, Ευπάθειες, Στατιστική Ανάλυση, Μοντέλο Μικτών Επιδράσεων, Μικτό Μοντέλο, Πολυεπίπεδο Μοντέλο, Σθεναρές/Σταθερές Επιδράσεις, Τυχαίες Επιδράσεις, Ανάλυση Κύριων Συνιστωσών, Ανάλυση Παραγόντων, Μετρικές Λογισμικού, Μετρικές Κώδικα, Μηχανική Λογισμικού, Εμπειρική Μηχανική Λογισμικού.

Abstract

The quality and security of software source code, as well as of the software itself, constitute two fundamental concepts in modern software development. Although they are often examined independently, they may exhibit significant relationships and interactions throughout the evolution of a software system. Frequent modifications to source code can alter characteristics associated with both quality and security; however, the direction and magnitude of the relationships between these dimensions are not always evident at the level of individual changes. This issue becomes particularly relevant when a modification intended to improve one dimension may also be associated with changes in characteristics of the other. Although quality and security have been extensively studied as distinct dimensions of software reliability and correct system operation, their joint empirical investigation in real-world Pull Requests remains a complex research problem.

The aim of the present study is to empirically investigate the relationship and potential interaction between software quality and security, including the source code through which the software is implemented. In parallel, the study examines the relationship between characteristics of the final state of the analyzed code, such as its size and complexity, as well as characteristics of the Pull Request, including its category and the project to which it belongs, and the observed metrics and densities of quality- and security-related issues.

To conduct the research, a dataset was constructed from Pull Requests (PRs) originating from real-world software projects that are used by the broader community as well as by large organizations to support their operational objectives and are deployed in what is commonly referred to as a production environment. In other words, these systems are available for use by relevant stakeholders, collaborators, or customers of an organization. Data collection and enrichment were based on two complementary sources of information. First, data obtained from GitHub and the GitHub API were used to capture information relating to Pull Requests and the content of the corresponding changes, including the title, body/description, labels, changed files, comments, and reviews, together with the complete .diff/.patch representation. This information was subsequently used in an automated multi-label classification process based on the gpt-oss-120b model, from which the technical-orientation categories of the Pull Requests were derived. In parallel, static analysis of the source code was performed using SonarQube, initially for both the pr_base and pr_closed states, with the objective of extracting metrics related to the size and complexity of the analyzed code, as well as quality and security indicators such as bugs, code smells, vulnerabilities, security hotspots, and issues related to reliability, maintainability, and security.

The data were analyzed using descriptive and multivariate statistical techniques, together with regression and mixed-effects models, to investigate the relationships between Pull Request characteristics and quality and security metrics while simultaneously accounting for differences across software projects. Ordinary Least Squares models were employed as baseline approaches, whereas Linear Mixed-Effects Models constituted the primary inferential framework, allowing the explicit modeling of the clustering of Pull Requests within their respective projects

and of the substantial between-project heterogeneity. In addition, Principal Component Analysis and Factor Analysis were used to investigate the multivariate structure of the examined metrics and the extent to which they could be represented by a smaller number of broader underlying dimensions. Robustness and sensitivity analyses were also conducted to assess the stability of the main findings under alternative analytical specifications.

The results of the study indicate that the complexity of the analyzed code exhibits a consistent relationship with the density of maintainability-related issues, whereas the corresponding relationships with security indicators appear weaker and more dependent on the statistical approach employed. At the same time, the mixed-effects models revealed substantial variation across individual software projects, indicating that project-specific context constitutes an important factor in the interpretation of both quality and security metrics. The Pull Request category did not demonstrate consistent predictive value in the principal models, while Principal Component Analysis and Factor Analysis revealed a multidimensional structure in which complexity and maintainability indicators were, to a considerable extent, distinguishable from the security dimension.

The originally planned comparison of changes between the `pr_base` and `pr_closed` states could not be used as the primary analytical approach because a relatively large proportion of the examined metrics remained unchanged between the two states. Consequently, the corresponding differences exhibited insufficient variability to support a reliable longitudinal analysis. Therefore, the main conclusions of the study concern the cross-sectional relationships observed in the final state of the Pull Requests rather than the empirical demonstration of temporal synergies, trade-offs, or bidirectional effects between changes in software quality and security.

The findings of the present research highlight the importance of the joint and context-aware evaluation of quality and security throughout the evolution of software systems. They may also contribute to the development of methods and tools that incorporate multiple metrics together with the specific context of each software project to support more effective Pull Request evaluation and the early identification of cases requiring increased attention during code review and automated quality and security assessment processes.

Keywords: Software, Source Code, Software Quality, Software Security, Source Code Quality, Source Code Security, Pull Requests, Vulnerabilities, Statistical Analysis, Mixed-Effects Model, Mixed Model, Multilevel Model, Fixed Effects, Random Effects, Principal Component Analysis, Factor Analysis, Software Metrics, Source Code Metrics, Software Engineering, Empirical Software Engineering.

Πρόλογος – Ευχαριστίες

Η καθημερινή προστριβή με την ανάπτυξη, συντήρηση και μελέτη διάφορων τύπων λογισμικών μικρού ή μεγάλου βεληνεκούς γεννάει συνεχώς την ανάγκη για εύρεση μεθόδων και θεωριών όπου θα αποσκοπούν στην βέλτιστη ανάπτυξη και διαχείριση ζωής ενός λογισμικού. Αν και συνειδητά, κάθε μηχανικός ή αρχιτέκτονας λογισμικού γνωρίζει πως καμία υλοποίηση δεν δύναται να είναι βέλτιστη, διότι τα πρότυπα της βιομηχανίας αλλάζουν και μεταβάλλονται συνεχώς και δυναμικά, η έρευνα για το βέλτιστο είναι συνεχής. Η προσωπική μου επαφή με την ελαχιστοποίηση προβλημάτων ποιότητας του κώδικα ενός λογισμικού καθώς και με την ανάπτυξη μεθόδων ασφάλειας του κώδικα και του λογισμικού έδωσαν το έναυσμα να λάβω υπόψη τις δύο αυτές παραμέτρους ως δύο οντότητες που εξελίσσονται καθ' όλη την διάρκεια ζωής και ανάπτυξης ενός λογισμικού και αλληλοεπιδρούν μεταξύ τους με άμεση επίδραση της μίας στην άλλη. Η παρατήρηση αυτή οδήγησε στην περαιτέρω έρευνα που η συγκεκριμένη εργασία προσπαθεί να πλαισιώσει και να αναλύσει.

Με την ολοκλήρωση της παρούσας πτυχιακής εργασίας, αισθάνομαι την ανάγκη να εκφράσω την ειλικρινή μου ευγνωμοσύνη προς όλους εκείνους που με τη στήριξη, την καθοδήγηση και την ενθάρρυνσή τους συνέβαλαν στην πραγματοποίησή της. Αρχικά, θα ήθελα να ευχαριστήσω θερμά τους επιβλέποντές μου, κ. Απόστολο Αμπατζόγλου και κ. Νικόλαο Νικολαΐδη, για την πολύτιμη καθοδήγηση, τις συμβουλές και την έμπρακτη υποστήριξή του καθ' όλη τη διάρκεια εκπόνησης της παρούσας εργασίας. Ιδιαίτερες ευχαριστίες θα ήθελα να εκφράσω στον Παναγιώτη Ασλανίδη, τον άνθρωπο που με μύησε στην μαγεία του λογισμικού, για το έναυσμα που μου έδωσε να αναλύσω και να δημιουργήσω ένα αρκετά πρώιμο επίπεδο ενός λογισμικού, δίνοντάς μου την ευκαιρία να δω μία πραγματική αλληλεπίδραση χρήστη-λογισμικού και να κατανοήσω την σημασία της ποιότητας και της ασφάλειας του κώδικα και κατ' επέκταση του ίδιου του λογισμικού. Τέλος, θα ήθελα να ευχαριστήσω ιδιαίτερα την οικογένειά μου και όλους τους κοντινούς μου ανθρώπους για την αδιάκοπη στήριξη, την υπομονή και την ενθάρρυνσή τους καθ' όλη τη διάρκεια των σπουδών μου.

Περιεχόμενα

Συμβολισμοί	xxi
Εισαγωγή	1
1.1 Σύγχρονη ανάπτυξη λογισμικού	1
1.2 Η σημασία της ποιότητας λογισμικού	4
1.3 Η σημασία της ασφάλειας λογισμικού	6
1.4 Η παραδοσιακή απομόνωση της ποιότητας και της ασφάλειας του λογισμικού	7
1.5 Η ποιότητα και η ασφάλεια μπορούν να αλληλοεπιδρούν	10
1.6 Το ερευνητικό πρόβλημα	11
1.7 Ερευνητικό κενό	12
1.8 Σκοπός της εργασίας	14
1.9 Επιμέρους στόχοι	14
1.10 Ερευνητικά ερωτήματα	15
1.11 Ερευνητικές υποθέσεις	16
1.12 Συνεισφορά της εργασίας	17
1.12.1 Θεωρητική συνεισφορά	18
1.12.2 Εμπειρική συνεισφορά	18
1.12.3 Μεθοδολογική συνεισφορά	18
1.12.4 Πρακτική συνεισφορά	19
1.13 Δομή της εργασίας	19
Σύγχρονη ανάπτυξη λογισμικού και Pull Request	20
2.1 Συστήματα ελέγχου εκδόσεων και Git	20
2.1.1 Αποθετήριο κώδικα (Repository)	23
2.1.2 Υποβολή/Καταγραφή τροποποιήσεων (Commit)	24
2.1.3 Διακλάδωση αντιγράφου πηγαίου κώδικα (Branch)	25
2.1.4 Συγχώνευση κώδικα (merge) και μηχανισμός διαφορών (diff)	26
2.2 Αίτημα συγχώνευσης/έλξης κώδικα (Pull Request)	27
2.2.1 Base και Head κατάσταση	28

2.2.2 Κύκλος ζωής (Lifecycle) ενός Pull Request	29
2.2.3 Αξιολόγηση κώδικα (Code Review)	30
2.2.4 Merge και Closed Pull Requests	31
2.3 Μεταβολές πηγαίου κώδικα	31
2.3.1 Προστιθέμενες (Added), Διαγραφμένες (Deleted) και Τροποποιημένες (Modified) γραμμές πηγαίου κώδικα (Lines)	33
2.3.2 Μετρικές LOC και NCLOC	34
2.4 Μετρικές Λογισμικού – Software Metrics	35
2.5 Αυτοματοποιημένη ανάλυση λογισμικού	37
2.5.1 Στατική Ανάλυση (Static Analysis)	38
2.5.2 Δυναμική ανάλυση (Dynamic Analysis)	40
2.5.3 Αυτοματοποιημένες Δοκιμές (Automated Testing)	41
2.6 Συνεχής Ενοποίηση/Ενσωμάτωση και Συνεχής Παράδοση (CI/CD), DevOps, DevSecOps	42
Ποιότητα Λογισμικού – Software Quality	45
3.1 Έννοια και μοντέλα ποιότητας	45
3.1.1 Ποιότητα Προϊόντος (Λογισμικού) και Ποιότητα Κώδικα	46
3.1.2 Internal και External quality	48
3.2 Συντηρησιμότητα (Maintainability) και Αξιοπιστία (Reliability)	49
3.3 Bugs, Defects, Faults και Failures	50
3.4 Code smells και Συντηρησιμότητα (Maintainability)	53
3.5 Πολυπλοκότητα (Complexity) λογισμικού και πηγαίου κώδικα	54
3.5.1 Κυκλωματική Πολυπλοκότητα (Cyclomatic Complexity)	56
3.5.2 Γνωστική Πολυπλοκότητα (Cognitive Complexity)	59
3.6 Διπλότυπος Κώδικα (Code Duplication)	60
3.7 Κάλυψη Δοκιμών (Test Coverage)	62
3.8 Τεχνικό Χρέος (Technical Debt)	64
3.9 Αναδιάρθρωση Κώδικα (Code Refactoring)	66
3.10 Αυτοματοποιημένες μετρικές ποιότητας και Quality Gates	67
3.11 Περιορισμοί αυτοματοποιημένων μετρικών ποιότητας	70
Ασφάλεια Λογισμικού – Software Security	73

4.1 Θεμελιώδεις έννοιες της Ασφάλειας Λογισμικού.....	73
4.1.1 Εμπιστευτικότητα (Confidentiality) – Ακεραιότητα (Integrity) – Διαθεσιμότητα (Availability)	74
4.1.2 Αυθεντικοποίηση (Authentication) και Εξουσιοδότηση (Authorization)	76
4.1.3 Threat, Vulnerability, Weakness και Exploit	78
4.1.4 Ρίσκο (Risk) και Επιφάνεια Επίθεσης (Attack Surface)	80
4.2 Αδυναμίες ασφαλείας στον πηγαίο κώδικα	82
4.3 CWE, CVE, CVSS	85
4.4 Κατηγορίες Ευπαθειών (Vulnerabilities)	87
4.5 Static Application Security Testing	89
4.5.1 Τρόπος λειτουργίας.....	90
4.5.2 Vulnerabilities και Security Hotspots	92
4.5.3 False positives και False negatives	94
4.5.4 Περιορισμοί ανάλυσης SAST	96
4.6 Συμπληρωματικές Τεχνικές Ασφάλειας	98
4.6.1 Dynamic Application Security Testing (DAST)	99
4.6.2 Software Composition Analysis και Dependency Scanning	101
4.6.3 Secret, IaC (Infrastructure as Code) και Configuration Scanning	102
4.7 Secure Software Development Lifecycle	103
4.8 DevSecOps και CI/CD pipelines	105
Αλληλεπίδραση συνιστωσών Ποιότητας και Ασφάλειας πηγαίου κώδικα και λογισμικού.....	108
5.1 Θεωρητική σχέση ποιότητας και ασφάλειας	108
5.2 Κοινές πηγές Quality και Security Defects	109
5.3 Πολυπλοκότητα, Συντηρησιμότητα και Security Defects	110
5.4 Code Smells, Duplication και Vulnerabilities	112
5.5 Reliability, Error handling και Security	113
5.6 Refactoring και Security	115
5.6.1 Πιθανές βελτιώσεις	115
5.6.2 Πιθανές regressions.....	117
5.7 Technical και Security Debt	117

5.8 Synergies και trade-offs.....	118
5.8.1 Quality improvement – Security improvement	119
5.8.2 Quality improvement – Security degradation	120
5.8.3 Security improvement – Quality improvement	120
5.8.4 Security improvement – Quality degradation	121
5.9 Confounding variables και περιορισμοί αιτιώδους ερμηνείας.....	122
5.10 Προηγούμενες εμπειρικές μελέτες	123
5.10.1 Pull Request και repository mining	124
5.10.2 Μελέτες Quality-Security interactions	125
5.10.3 Static-analysis-based studies.....	126
5.10.4 Complexity, defects και vulnerabilities	126
5.11 Στατιστικές Προσεγγίσεις Υπαρχουσών Εργασιών	127
5.11.1 Regression και Mixed Effects Models	128
5.11.2 PCA και Factor Analysis σε μετρικές λογισμικού	129
5.12 Σύνθεση της βιβλιογραφίας και τοποθέτηση της παρούσας εργασίας	130
5.13 Conceptual model της παρούσας έρευνας	132
Ερευνητικός σχεδιασμός, συλλογή και προετοιμασία δεδομένων.....	134
6.1 Ερευνητική προσέγγιση.....	134
6.1.1 Ποσοτική εμπειρική έρευνα.....	135
6.1.2 Observational study.....	136
6.2 Μονάδα ανάλυσης: Pull Request	137
6.3 Ιεραρχική δομή των δεδομένων	137
6.3.1 Pull Requests εντός projects.....	138
6.3.2 Project-specific effects	139
6.4 Population, sample και διαδικασία επιλογής	140
6.4.1 Repository selection	140
6.4.2 PR inclusion/exclusion criteria	141
6.5 Ερευνητικό pipeline	142
6.6 Συλλογή δεδομένων Pull Requests.....	143
6.6.1 Κατηγοριοποίηση των Pull Requests	144
6.7 Ανακατασκευή των δύο καταστάσεων των εξεταζόμενων PR commits	146
6.7.1 pr_base	147

6.7.2 pr_closed	147
6.8 Static analysis και εξαγωγή μετρικών.....	147
6.9 Μετρικές της εμπειρικής ανάλυσης	148
6.9.1 NCLOC	149
6.9.2 Bugs	149
6.9.3 Vulnerabilities	150
6.9.4 Code smells	150
6.9.5 Security hotspots	151
6.9.6 Duplicated lines density	151
6.9.7 Complexity	152
6.9.8 Cognitive complexity	152
6.9.9 Reliability issues.....	152
6.9.10 Maintainability issues	153
6.9.11 Security issues	153
6.10 Καθορισμός και έλεγχος ποιότητας δεδομένων.....	154
6.10.1 Missing και invalid observations	154
6.10.2 Duplicates	155
6.10.3 NCLOC = 0	155
6.10.4 Failed scans.....	155
6.10.5 Base/closed similarity	156
6.10.6 Outliers και ακραίες περιπτώσεις	156
6.11 Κατασκευή αναλυτικών μεταβλητών.....	156
6.11.1 Raw counts και normalization	157
6.11.2 Issue densities	157
6.11.3 Complexity και Έλεγχος ως προς NCLOC.....	158
6.11.4 Αρχικός ορισμός μεταβολών και αξιολόγηση της delta προσέγγισης	159
6.11.5 Relative και percentage changes.....	160
6.12 Τελικό analytical dataset	161
6.13 Δυνατότητα αναπαραγωγής και ερευνητική διαφάνεια	161
Στατιστική μεθοδολογία.....	163
7.1 Στρατηγική στατιστικής ανάλυσης	163
7.2 Περιγραφική στατιστική και Exploratory Data Analysis (EDA)	165
7.2.1 Κατανομές και ακραίες τιμές	166

7.2.2 Base-closed comparisons	167
7.2.3 Project-level distributions	168
7.3 Συσχετίσεις	168
7.3.1 Quality-Security Relationships.....	169
7.4 Ordinary Least Squares	169
7.4.1 Μοντέλο και συντελεστές	170
7.4.2 Παραδοχές	171
7.4.3 Multicollinearity και VIF	173
7.4.4 Residual diagnostics	174
7.4.5 Περιορισμός ανεξαρτησίας των PRs	174
7.5 Linear Mixed-Effects Models (LMM)	175
7.5.1 Fixed και Random effects	176
7.5.2 Random-intercept model	177
7.5.3 Variance components	178
7.5.4 Intraclass Correlation Coefficient (ICC)	179
7.5.5 ML και REML.....	180
7.5.6 AIC, BIC και model comparison	181
7.5.7 Diagnostics και model convergence.....	182
7.6 Σύγκριση OLS και LMM.....	184
7.7 Principal Component Analysis	186
7.7.1 Στόχος και standardization	186
7.7.2 Eigenvalues και explained variance.....	187
7.7.3 Loadings και component scores	189
7.7.4 Επιλογή και ερμηνεία components.....	191
7.8 Factor Analysis	193
7.8.1 Latent factors.....	194
7.8.2 KMO και Bartlett.....	195
7.8.3 Parallel analysis	196
7.8.4 Extraction.....	197
7.8.5 Rotation	198
7.8.6 Loadings, communalities και uniqueness	198
7.8.7 Factor scores και ερμηνεία	200
7.9 PCA έναντι Factor Analysis	202
7.10 Robustness και sensitivity analyses.....	204

7.10.1 Negative Binomial GLM	207
7.10.2 Generalized Estimating Equations (GEE)	208
7.10.3 Sensitivity analysis ως προς το μέγεθος των project clusters.....	209
7.10.4 Size-adjusted sensitivity analysis των PCA και Factor Analysis	209
Αποτελέσματα	211
8.1 Τελικό δείγμα	211
8.1.1 Filtering flow	212
8.1.2 Αριθμός Pull Requests και projects.....	213
8.1.3 PRs ανά project.....	214
8.1.4 Κατηγορίες των Pull Requests	215
8.2 Περιγραφικά αποτελέσματα	216
8.2.1 Quality metrics	217
8.2.2 Security metrics	218
8.2.3 Base έναντι closed	220
8.2.4 Project-level variability	222
8.3 Exploratory analysis και συσχετίσεις.....	223
8.4 Αποτελέσματα OLS	223
8.5 Αποτελέσματα LMM.....	225
8.5.1 Null models και ICC.....	226
8.5.2 Fixed effects.....	228
8.5.3 Random effects.....	229
8.5.4 Σύγκριση LMM με OLS.....	231
8.6 Αποτελέσματα PCA.....	233
8.7 Αποτελέσματα Factor Analysis	236
8.8 Robustness analyses	238
8.9 Ενοποίηση αποτελεσμάτων	240
8.9.1 Συμφωνίες μεταξύ μεθόδων	241
8.9.2 Διαφορές μεταξύ μεθόδων	243
8.9.3 Ισχυρότερα ευρήματα	245
8.9.4 Αποτελέσματα ανά ερευνητικό ερώτημα και υπόθεση	247
Συζήτηση.....	253
9.1 Επιστροφή στο ερευνητικό πρόβλημα.....	253

9.2 Ερμηνεία της σχέσης ποιότητας και ασφάλειας	254
9.3 Ρόλος της πολυπλοκότητας, της συντηρησιμότητας και των χαρακτηριστικών των Pull Requests	256
9.4 Project-specific effects και ερμηνεία ICC	257
9.5 Ερμηνεία των PCA components και latent factors.....	258
9.6 Σύγκριση με τη βιβλιογραφία και μη αναμενόμενα αποτελέσματα	259
9.7 Πρακτικές επιπτώσεις	261
9.7.1 Code review και context-aware προτεραιοποίηση Pull Request	261
9.7.2 DevSecOps και CI/CD	262
9.7.3 Automated quality και security gates	263
9.8 Threats to Validity και περιορισμοί.....	264
9.8.1 Construct validity και measurement limitations	264
9.8.2 Internal validity και causal inference	267
9.8.3 External validity και selection bias	267
9.8.4 Conclusion validity και statistical limitations	268
Συμπεράσματα και μελλοντική έρευνα	271
10.1 Ανακεφαλαίωση της έρευνας και βασικά ευρήματα	271
10.2 Απάντηση στο κύριο ερευνητικό ερώτημα	273
10.3 Συνεισφορά της έρευνας.....	274
10.4 Κατευθύνσεις μελλοντικής έρευνας	276
10.5 Τελικό συμπέρασμα	277
Βιβλιογραφία.....	278
Παράρτημα Α – Data Dictionary	284
Παράρτημα Β – Data Collection, Snapshot Reconstruction, Data Cleaning και Reproduction	290
Παράρτημα Γ – Πλήρεις περιγραφικές αναλύσεις και correlation matrices	293
Παράρτημα Δ – Πλήρη στατιστικά μοντέλα, diagnostics και robustness analyses	294
Παράρτημα Ε – Πλήρη PCA, Factor Analysis και συμπληρωματικά γραφήματα	297

Κατάλογος Εικόνων

Εικόνες οι οποίες φέρουν το αναγνωριστικό αστεράκι () στο τέλος της ετικέτας τους υποδηλώνουν δημιουργία με υποβοήθηση συστήματος Τεχνητής Νοημοσύνης.*

Εικόνα 1.3.1 - Το τρίγωνο CIA *	6
Εικόνα 3.3.1 – Εννοιολογική ακολουθία από ανθρώπινο λάθος έως την αποτυχία λογισμικού. *	52
Εικόνα 4.1.1 – Σχέση μεταξύ Weakness, SEC, Vulnerability, Exploit και επιπτώσεων στο σύστημα *	79
Εικόνα 4.3.1 – Σχέση μεταξύ των εννοιών CWE, CVE, CVSS και Risk *	86
Εικόνα 4.5.1 – Αναπαράσταση βημάτων SAST ανάλυσης *	92
Εικόνα 4.5.2 – Διαφορά μεταξύ Vulnerability (Ευπάθειας) και Security Hotspot *	93
Εικόνα 4.7.1 – Φιλοσοφία ενός Secure SDLC *	105
Εικόνα 4.8.1 – Εννοιολογική ακολουθία διασφάλισης ασφάλειας σε DevSecOps και CI/CD pipelines *	107
Εικόνα 6.3.1 – Ιεραρχική δομή δεδομένων έρευνας *	138
Εικόνα 6.5.1 – Ερευνητικό Pipeline *	143
Εικόνα 7.1.1 - Σύγκριση των καταστάσεων pr_base και pr_closed ως προς (α) την πυκνότητα Maintainability Issues και (β) την πυκνότητα Security Issues	164
Εικόνα 7.7.1 – Ποσοστό και σωρευτικό ποσοστό της συνολικής διακύμανσης που εξηγείται μέσω των κύριων συνιστωσών της PCA	188
Εικόνα 7.7.2 – Correlation loadings των επτά (7) αρχικών μεταβλητών στις τρεις κύριες συνιστώσες της PCA	190
Εικόνα 7.7.3 – Parallel Analysis για την επιλογή του αριθμού των κύριων συνιστωσών	191
Εικόνα 7.7.4 – Κατανομή των Pull Requests στον χώρο των δύο πρώτων κύριων συνιστωσών ανά κατηγορία	193
Εικόνα 7.8.1 – Parallel Analysis για τον προσδιορισμό του αριθμού των παραγόντων της κύριας Factor Analysis	196
Εικόνα 7.8.2 – Περιστραμμένα factor loadings των επτά αρχικών μεταβλητών στους τρεις λανθάνοντες παράγοντες της κύριας Factor Analysis έπειτα της εφαρμογής της Varimax rotation	199
Εικόνα 7.10.1 – Αποτελέσματα της Parallel Analysis στις size-adjusted sensitivity αναλύσεις, (α) PCA και (β) Factor Analysis	206
Εικόνα 8.1.1 – Ροή φιλτραρίσματος δεδομένων και διαμόρφωσης τελικού ερευνητικού δείγματος *	213
Εικόνα 8.1.2 – Κατανομή του αριθμού των Pull Requests ανά project στο τελικό ερευνητικό δείγμα	214
Εικόνα 8.1.3 – Κατανομή των Pull Requests του τελικού δείγματος στις κατηγορίες quality, security, both *	215

Εικόνα 8.2.1 – Σύγκριση των τιμών pr_base και pr_closed για τις κανονικοποιημένες μετρικές maintainability και security issues	221
Εικόνα 8.6.1 – Προβολή των PRs στον χώρο των δύο πρώτων principal components ανά κατηγορία PR (quality, security, both).....	235

Κατάλογος Πινάκων

Πίνακας 7.9.1 – Συγκριτική αντιστοίχιση των κύριων πολυμεταβλητών διαστάσεων που προέκυψαν από την PCA και τη Factor Analysis	203
Πίνακας 8.2.1 – Βασικά περιγραφικά στατιστικά των μετρικών ποιότητας στο τελικό ερευνητικό δείγμα	218
Πίνακας 8.2.2 – Βασικά περιγραφικά στατιστικά των μετρικών ασφάλειας στο τελικό ερευνητικό δείγμα	219
Πίνακας 8.4.1 – Εκτιμήσεις OLS μοντέλων για τη λογαριθμικά μετασχηματισμένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας, με cluster-robust standard errors σε επίπεδο project	225
Πίνακας 8.5.1 – Συνιστώσες διακύμανσης και ICC των null LMM για τις πυκνότητες προβλημάτων ποιότητας και ασφάλειας	227
Πίνακας 8.5.2 – Εκτιμήσεις των fixed effects των LMM για τη λογαριθμικά μετασχηματισμένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας	229
Πίνακας 8.5.3 – Εκτιμώμενες συνιστώσες διακύμανσης και ICC των πλήρων random-intercept Linear Mixed Models	230
Πίνακας 8.5.4 – Σύγκριση των εκτιμώμενων συντελεστών των OLS και LMM για τις πυκνότητες προβλημάτων ποιότητας και ασφάλειας	233
Πίνακας 8.6.1 – Eigenvalues, ποσοστά εξηγούμενης διακύμανσης και συνοπτική ερμηνεία των τριών principal components της κύριας διενεργηθείσας PCA	234
Πίνακας 8.7.1 – Περιστραμμένα factor loadings των επτά βασικών μεταβλητών στους τρεις factors της κύριας Factor Analysis	237
Πίνακας 8.9.1 – Συνοπτική αποτύπωση των βασικών σημείων συμφωνίας μεταξύ των στατιστικών και πολυμεταβλητών μεθόδων της έρευνας	243
Πίνακας 8.9.2 – Κύριες διαφοροποιήσεις μεταξύ των στατιστικών και πολυμεταβλητών μεθόδων και η ερμηνευτική τους σημασία	245
Πίνακας 8.9.3 – Ιεράρχηση των κυριότερων εμπειρικών ευρημάτων με βάση τη συνέπεια και τη σταθερότητά τους μεταξύ των διαφορετικών αναλυτικών προσεγγίσεων	247

Συμβολισμοί

PR	<i>Pull Request</i> Ένα Pull Request αποτελεί μία διαδικασία (ή μέθοδο) η οποία προτείνει αλλαγές στον πηγαίο κώδικα. Οι μηχανικοί λογισμικού δημιουργούν διακλαδώσεις του κύριου αποθετηρίου του πηγαίου κώδικα (repository) σε ξεχωριστές διακλαδώσεις (branches), επεξεργάζονται και υποβάλουν τον κώδικα στην συγκεκριμένη διακλάδωση που επεξεργάζονται και δημιουργούν ένα αίτημα ενσωμάτωσης για να επισημάνουν την πρόταση αλλαγής τους για έλεγχο του κώδικα πριν από την ενσωμάτωση/συγχώνευση των αλλαγών από την διακλάδωση στην κύρια βάση κώδικα (κύρια διακλάδωση).
LOC	<i>Lines of Code</i> Συνολικές γραμμές κώδικα στο σύνολο του πηγαίου κώδικα. Μπορεί να περιλαμβάνει σχόλια καθώς και κενές γραμμές, σημεία που δεν αποτελούν εκτελέσιμο κώδικα.
NLOC	<i>Non-Comment Lines of Code</i> Συνολικές γραμμές κώδικα στο σύνολο του πηγαίου κώδικα. Εξ' ορισμού δεν μπορεί να περιλαμβάνει γραμμές σχολίων ή κενές γραμμές, καθώς δεν αποτελούν τμήματα εκτελέσιμου κώδικα.
SAST	<i>Static Application Security Testing</i> Static Application Security Testing (SAST) είναι μια τεχνική στατικής ανάλυσης που εξετάζει τον πηγαίο ή μεταγλωττισμένο κώδικα μιας εφαρμογής χωρίς να απαιτείται η εκτέλεση της, με στόχο έγκαιρο εντοπισμό και αποκατάσταση ευπαθειών κατά την ανάπτυξη.
DAST	<i>Dynamic Application Security Testing</i> Dynamic Application Security Testing (DAST) είναι μια μέθοδος ελέγχου ασφάλειας «από έξω προς τα μέσα» (outside-in). Εντοπίζει ευπάθειες σε εκτελούμενες διαδικτυακές εφαρμογές και APIs προσομοιώνοντας αυτοματοποιημένες, πραγματικές

	κυβερνοεπιθέσεις. Είναι γνωστή και ως black-box testing, αφού δεν απαιτεί πρόσβαση στον εσωτερικό πηγαίο κώδικα και αξιολογεί το λογισμικό με τρόπο παρόμοιο με αυτόν ενός επιτιθέμενου.
PCA	Principal Component Analysis Principal Component Analysis (Ανάλυση Κύριων Συνιστωσών) είναι μια γραμμική στατιστική τεχνική μείωσης της διαστατικότητας των δεδομένων, η οποία μετασχηματίζει ένα σύνολο συσχετιζόμενων μεταβλητών σε ένα μικρότερο σύνολο νέων, ασυσχέτιστων μεταβλητών, τις κύριες συνιστώσες. Η μέθοδος ανάλυσης χρησιμοποιείται κυρίως για την απλοποίηση πολύπλοκων δεδομένων, την οπτικοποίηση, την αναγνώριση προτύπων και την προεπεξεργασία δεδομένων.
FA	Factor Analysis Factor Analysis (Παραγοντική Ανάλυση) είναι μια στατιστική μέθοδος που χρησιμοποιείται για να εξηγήσει τις συσχετίσεις μεταξύ πολλών παρατηρούμενων μεταβλητών μέσω ενός μικρότερου αριθμού μη παρατηρούμενων, υποκείμενων μεταβλητών, οι οποίες ονομάζονται παράγοντες (factor). Στόχος της είναι να εντοπίσει κοινά πρότυπα και λανθάνουσες δομές στα δεδομένα, μειώνοντας έτσι την πολυπλοκότητά τους και διευκολύνοντας την κατανόηση των σχέσεων μεταξύ των μεταβλητών.
OLS	Ordinary Least Squares Ordinary Least Squares (Μέθοδος Ελαχίστων Τετραγώνων) είναι μια στατιστική μέθοδος που χρησιμοποιείται κυρίως στη γραμμική παλινδρόμηση για την εκτίμηση της σχέσης μεταξύ μιας εξαρτημένης μεταβλητής και μίας ή περισσότερων ανεξάρτητων μεταβλητών. Η μέθοδος επιλέγει τις παραμέτρους του μοντέλου έτσι ώστε να ελαχιστοποιείται το άθροισμα των τετραγώνων των διαφορών μεταξύ των πραγματικών παρατηρήσεων και των τιμών που προβλέπει το μοντέλο, επιτυγχάνοντας έτσι την καλύτερη δυνατή προσαρμογή στα δεδομένα.
LMM	Linear Mixed Model

	Linear Mixed Models (Γραμμικά Μικτά Μοντέλα) αποτελούν επέκταση των κλασικών γραμμικών μοντέλων και συνδυάζουν σταθερές επιδράσεις με τυχαίες επιδράσεις.
ICC	<i>Intraclass Correlation Coefficient</i> Intraclass Correlation Coefficient (Ενδοταξικός Συντελεστής Συσχέτισης) είναι ένα στατιστικό μέτρο που χρησιμοποιείται για να εκφράσει πόσο όμοιες είναι μεταξύ τους οι παρατηρήσεις που ανήκουν στην ίδια ομάδα.
CI	<i>Continuous Integration</i> Continuous Integration (Συνεχής Ενσωμάτωση) είναι πρακτική ανάπτυξης λογισμικού κατά την οποία οι προγραμματιστές ενσωματώνουν συχνά τις αλλαγές τους σε ένα κοινό αποθετήριο κώδικα. Κάθε ενσωμάτωση συνοδεύεται συνήθως από αυτοματοποιημένο build και εκτέλεση ενός pipeline (διαδικασίας ελέγχων), προκειμένου να εντοπίζονται έγκαιρα σφάλματα, προβλήματα συμβατότητας ή ποιότητας και να διατηρείται η κύρια έκδοση του λογισμικού σε λειτουργική κατάσταση.
CD	<i>Continuous Deployment</i> Continuous Deployment (Συνεχής Ανάπτυξη/Εγκατάσταση) είναι πρακτική ανάπτυξης λογισμικού κατά την οποία οι αλλαγές στον κώδικα, αφού περάσουν επιτυχώς από αυτοματοποιημένους ελέγχους προωθούνται αυτόματα στο περιβάλλον παραγωγής χωρίς να απαιτείται χειροκίνητη έγκριση. Αποτελεί εξέλιξη της Συνεχούς Ενσωμάτωσης και της Συνεχούς Παράδοσης και επιτρέπει συχνότερες, μικρότερες και ταχύτερες εκδόσεις λογισμικού, απαιτώντας όμως αξιόπιστες αυτοματοποιημένες διαδικασίες ελέγχου και ισχυρούς μηχανισμούς παρακολούθησης ώστε να περιορίζεται ο κίνδυνος εισαγωγής σφαλμάτων στην παραγωγή.
CI / CD	<i>Continuous Integration / Continuous Deployment</i> Αποτελεί συνδυασμό των διαδικασιών Continuous Integration και Continuous Deployment.

IaC	<i>Infrastructure as Code</i> Infrastructure as Code (IaC) είναι μια πρακτική DevOps κατά την οποία η υποδομή ενός συστήματος, όπως servers, δίκτυα, βάσεις δεδομένων και ρυθμίσεις ασφαλείας, ορίζεται και διαχειρίζεται μέσω κώδικα ή αρχείων διαμόρφωσης αντί για χειροκίνητες διαδικασίες. Με αυτόν τον τρόπο, η δημιουργία και η διαχείριση υποδομών αυτοματοποιείται, γίνεται επαναλήψιμη και μπορεί να ελέγχεται μέσω version control, μειώνοντας ανθρώπινα σφάλματα και ασυνέπειες μεταξύ διαφορετικών περιβαλλόντων.
-------------------	--

ΜΕΡΟΣ Α

ΕΙΣΑΓΩΓΗ ΚΑΙ ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ

Εισαγωγή

1.1 Σύγχρονη ανάπτυξη λογισμικού

Η ανάπτυξη σύγχρονων συστημάτων λογισμικού αποτελεί μία δυναμική, επαναληπτική και εξελικτική διαδικασία, στην οποία ο πηγαίος κώδικας του λογισμικού επεξεργάζεται και μεταβάλλεται συνεχώς μέσω της προσθήκης νέων λειτουργιών, επεξεργασίας υπαρχόντων, της διόρθωσης σφαλμάτων, της αναδιάρθρωσης υφιστάμενου κώδικα και της αντιμετώπισης διαφόρων προβλημάτων λειτουργίας, ποιότητας ή ασφάλειας. Αντίθετα με παλαιότερες και περισσότερο σειριακές προσεγγίσεις ανάπτυξης και διαχείρισης ενός λογισμικού, οι σύγχρονες πρακτικές βασίζονται σε συχνές και σχετικά μικρές αλλαγές, στη συνεργασία μεταξύ πολλών προγραμματιστών και στην εκτεταμένη αυτοματοποίηση διαδικασιών ελέγχου, ενσωμάτωσης και διάθεσης λογισμικού. Πρακτικές συνεχούς ενσωμάτωσης, παράδοσης και ανάπτυξης έχουν καθιερώσει διαδικασίες στις οποίες η δημιουργία, ο έλεγχος και η ενσωμάτωση του κώδικα πραγματοποιούνται επαναληπτικά και επανειλημμένα κατά τη διάρκεια της εξέλιξης ενός συστήματος.

Βασικό στοιχείο της συγκεκριμένης διαδικασίας αποτελεί η διαχείριση εκδόσεων του πηγαίου κώδικα (Version Control – VC). Ένα από τα σημαντικότερα και πλέον ευρέως χρησιμοποιούμενα ως Industry Standard -αλλά όχι το μοναδικό- σύστημα καταμετρημένου ελέγχου εκδόσεων που χρησιμοποιούνται για τον σκοπό αυτό είναι το Git (Git Version Control System). Το Git επιτρέπει την καταγραφή της ιστορικής εξέλιξης ενός έργου ως μία ακολουθία διακριτών καταστάσεων και αλλαγών, διευκολύνοντας και ενισχύοντας την παράλληλη ανάπτυξη, τη συνεργασία μεταξύ προγραμματιστών (μίας ή περισσότερων ομάδων) καθώς και την εύκολη και γρήγορη επαναφορά σε προηγούμενες εκδόσεις του λογισμικού. Ο πηγαίος κώδικας και το ιστορικό των αλλαγών οργανώνονται σε ένα repository (αποθετήριο), στο οποίο διατηρούνται τα αρχεία του έργου, οι εκδόσεις τους και οι αντίστοιχες μεταβολές τους, ως ένα δέντρο (tree) με ρίζα την πρώτη έκδοση του λογισμικού.

Οι επιμέρους αλλαγές που πραγματοποιούνται στον κώδικα καταγράφονται μέσω των commits, δηλαδή «υποβολή» ή «καταγραφή» ή «στιγμιότυπο» και λειτουργεί ως μια μόνιμη αποθήκευση των αλλαγών του κώδικα στο ιστορικό του repository. Ένα commit αποτελεί μία καταγεγραμμένη κατάσταση του έργου και περιλαμβάνει ένα σύνολο αλλαγών αρχείων ή τμημάτων αρχείων μαζί με μετά-πληροφορίες (ή metadata) που περιγράφουν την προέλευσή τους, όπως τον δημιουργό, τον χρόνο δημιουργίας και ένα μήνυμα που συνήθως εξηγεί τον σκοπό της αλλαγής ή τις αλλαγές που διαπράχθηκαν στα αρχεία ενός έργου μέσα σε ένα αποθετήριο. Με τον τρόπο αυτό δημιουργείται ένα ιστορικό εξέλιξης του λογισμικού, μέσω του οποίου είναι δυνατό να προσδιοριστεί πότε και με ποιον τρόπο τροποποιήθηκε ένα συγκεκριμένο τμήμα του πηγαίου κώδικα.

Παράλληλα, τα σύγχρονα συστήματα ελέγχου εκδόσεων υποστηρίζουν τη δημιουργία διαφορετικών branches (διακλαδώσεων), δηλαδή ανεξάρτητων γραμμών ανάπτυξης του ίδιου έργου (ή αποθετηρίου). Ένα branch επιτρέπει σε έναν προγραμματιστή ή ένα σύνολο προγραμματιστών να αναπτύξει μία νέα λειτουργία, να αλλάξει μία υφιστάμενη, να διορθώσει ένα σφάλμα ή να πραγματοποιήσει μία πειραματική αλλαγή χωρίς να τροποποιήσει άμεσα την κύρια έκδοση του συστήματος. Μετά την ολοκλήρωση και τον έλεγχο των αλλαγών, το branch μπορεί να ενσωματωθεί στην κύρια γραμμή (κύριο branch) ανάπτυξης, είτε μέσω ενός Pull Request είτε με απευθείας συγχώνευση. Η χρήση branches επιτρέπει την παράλληλη εξέλιξη διαφορετικών τμημάτων του λογισμικού, ενώ ταυτόχρονα περιορίζει την άμεση επίδραση μη ολοκληρωμένων ή εσφαλμένων ή μη ελεγμένων αλλαγών στον βασικό κώδικα του λογισμικού.

Η διαδικασία ενσωμάτωσης μιας αλλαγής πραγματοποιείται συχνά μέσω ενός Pull Request (PR) -σε διάφορες πλατφόρμες πιθανώς να αναφέρεται ως Merge Request- το οποίο αποτελεί πρόταση ενσωμάτωσης ενός συνόλου αλλαγών από ένα branch σε κάποιο άλλο, συνήθως στην κύρια γραμμή ανάπτυξης, ή στην λεγόμενη development γραμμή ανάπτυξης η οποία χρησιμοποιείται ως δοκιμαστική πριν ο συνολικός κώδικας ενσωματωθεί με την κύρια γραμμή ανάπτυξης. Ένα Pull Request συγκεντρώνει σε ένα ενιαίο αντικείμενο τις αλλαγές των αρχείων, το σύνολο των commits, τις συζητήσεις για προτεινόμενες αλλαγές μεταξύ των «αξιολογητών» και του προγραμματιστή που δημιούργησε το συγκεκριμένο commit και ορίζεται ως «αξιολογούμενος», τα αποτελέσματα αυτοματοποιημένων ελέγχων και την απόφαση

σχετικά με την τελική αποδοχή ή απόρριψη της αλλαγής. Μελέτη των Gousios, Pinzger και van Deursen (2014) περιγράφει το pull-based development ως ένα σημαντικό μοντέλο κατανεμημένης ανάπτυξης, όπου διαχωρίζεται η διαδικασία δημιουργίας μιας αλλαγής από την απόφαση ενσωμάτωσής της στον βασικό κώδικα του έργου.

Πριν από την συγχώνευση ενός Pull Request με την κύρια γραμμή ανάπτυξης πραγματοποιείται συχνά η διαδικασία του code review, όπου ένας ή περισσότεροι προγραμματιστές εξετάζουν τις προτεινόμενες αλλαγές. Σκοπός της διαδικασίας δεν περιορίζεται αποκλειστικά στον εντοπισμό σφαλμάτων. Η μελέτη του Bacchelli και Bird (2013) έδειξε ότι το σύγχρονο code review συμβάλλει επίσης στην ασφάλεια, στη βελτίωση του κώδικα, στην ανταλλαγή γνώσης μεταξύ των μελών της ομάδας, στην καλύτερη κατανόηση των αλλαγών και στην εξεύρεση εναλλακτικών λύσεων. Κατά συνέπεια, το code review λειτουργεί ως σημαντικός μηχανισμός ελέγχου πριν μία μεταβολή καταστεί μέρος της κύριας έκδοσης του συστήματος.

Η ανθρώπινη αξιολόγηση του κώδικα συμπληρώνεται σήμερα από ένα σύνολο αυτοματοποιημένων ελέγχων. Τα automated pipelines (μπορούν να περιλαμβάνουν automated tests, αυτοματοποιημένοι έλεγχοι ποιότητας και ασφάλειας) μπορούν να εκτελούνται αυτόματα μετά την πραγματοποίηση μιας αλλαγής ή κατά τη δημιουργία και ενημέρωση ενός Pull Request, επιτρέποντας τον επαναλαμβανόμενο έλεγχο της αναμενόμενης συμπεριφοράς του λογισμικού. Η αυτοματοποίηση καθιστά εφικτή τη συνεπή επανάληψη μεγάλου αριθμού ελέγχων, περιορίζοντας την ανάγκη αποκλειστικά χειροκίνητης επαλήθευσης.

Οι αυτοματοποιημένοι έλεγχοι αποτελούν συνήθως μέρος μιας ευρύτερης διαδικασίας Continuous Integration και Continuous Delivery/Deployment (CI / CD). Η CI διαδικασία βασίζεται στην συνεχή ενσωμάτωση αλλαγών και στην αυτόματη εκτέλεση διαδικασιών όπως η μεταγλώττιση, η δημιουργία του λογισμικού και η εκτέλεση των tests. Τα συστήματα CI περιγράφονται από την εμπειρική μελέτη των (2016). ως μηχανισμοί που αυτοματοποιούν τις ροές των compilation, building και testing και παρουσιάζει στοιχεία που συνδέουν την υιοθέτηση τους με συχνότερες εκδόσεις λογισμικού.

Η έννοια του Continuous Delivery επεκτείνει τη διαδικασία αυτή, μέσω της επιδίωξης το λογισμικό να βρίσκεται συνεχώς σε κατάσταση κατάλληλη για διάθεση στον τελικό χρήστη, ενώ στην διαδικασία του Continuous Development οι αλλαγές που ικανοποιούν τις απαραίτητες προϋποθέσεις μπορούν να μεταφέρονται αυτόματα στο περιβάλλον παραγωγής. Τα CI/CD pipelines λειτουργούν ως αυτοματοποιημένες ακολουθίες σταδίων, μέσα στις οποίες μπορούν να πραγματοποιούνται build processes, tests, quality checks, security checks και τελικά διαδικασίες διάθεσης ή εγκατάστασης του λογισμικού.

Εκτός από τη δυναμική εκτέλεση tests, σημαντικό ρόλο στη σύγχρονη ανάπτυξη λογισμικού έχει η στατική ανάλυση κώδικα (static code analysis). Αντίθετα με τεχνικές που απαιτούν την εκτέλεση του προγράμματος, η στατική ανάλυση εξετάζει τον πηγαίο κώδικα ή άλλες αναπαραστάσεις του χωρίς να απαιτείται η εκτέλεσή του, αναζητώντας πιθανά σφάλματα, παραβιάσεις κανόνων συγγραφής, προβλήματα συντηρησιμότητας, μη ασφαλείς κατασκευές και άλλα χαρακτηριστικά. Τα εργαλεία static analysis επιτρέπουν την αυτοματοποίηση μέρους της

επιθεώρησης του κώδικα και μπορούν να χρησιμοποιούνται είτε απευθείας κατά την ανάπτυξη είτε ως μέρος ενός CI/CD pipeline. Η συγκεκριμένη μέθοδος είναι αρκετά ευπαθής σε παράγοντες όπως false positives, η κατανοησιμότητα των αποτελεσμάτων και ο βαθμός ενσωμάτωσής τους στην καθημερινή ροή εργασίας των προγραμματιστών.

Αντίστοιχα, έχει επεκταθεί και στον τομέα της ασφάλειας του λογισμικού. Στο πλαίσιο πρακτικών DevSecOps, οι έλεγχοι ασφάλειας ενσωματώνονται όλο και περισσότερο στα στάδια ανάπτυξης και στα CI/CD pipelines, αντί να πραγματοποιούνται αποκλειστικά μετά την ολοκλήρωση του συστήματος. Τεχνικές όπως το Static Application Security Testing (SAST) επιτρέπουν την ανάλυση του πηγαίου κώδικα για την ανίχνευση πιθανών αδυναμιών ασφάλειας, ενώ μπορούν να χρησιμοποιηθούν και άλλες τεχνικές. Πλέον τέτοιες τεχνικές ορίζονται ως βασικό στοιχείο ενσωμάτωσης σε CI/CD pipelines.

Υπό το μοντέλο ανάπτυξης, μία μεταβολή στον πηγαίο κώδικα δεν αποτελεί απλώς μία απομονωμένη ενέργεια προγραμματισμού. Αντίθετα, διαμορφώνεται μέσα από μία αλληλουχία ενεργειών που περιλαμβάνει τη δημιουργία ενός branch, την πραγματοποίηση ενός ή περισσότερων commits, τη δημιουργία ενός branch, την ανθρώπινη επιθεώρηση μέσω code review και την εκτέλεση αυτοματοποιημένων ελέγχων ποιότητας και ασφάλειας πριν από την τελική ενσωμάτωση της αλλαγής.

Αποτέλεσμα αυτού, είναι η ιδιαίτερη σημασία που αποκτά το Pull Request ως διακριτή και παρατηρήσιμη μονάδα αλλαγής του λογισμικού, καθώς συνδέει τις ίδιες τις τροποποιήσεις του κώδικα με τη διαδικασία αξιολόγησης, τα αποτελέσματα των αυτοματοποιημένων αναλύσεων και την τελική απόφαση ενσωμάτωσής τους. Παράλληλα, δημιουργεί τις προϋποθέσεις για τη συστηματική μελέτη της εξέλιξης της ποιότητας και της ασφάλειας του λογισμικού. Καθώς διαφορετικοί έλεγχοι μπορούν να εκτελούνται πριν και μετά από μία τροποποίηση του πηγαίου κώδικα, είναι δυνατό να καταγραφεί πώς μεταβάλλονται χαρακτηριστικά ποιότητας και ασφάλειας ως αποτέλεσμα συγκεκριμένων αλλαγών. Η παρατήρηση αυτών των μεταβολών αποτελεί τη βάση για τη διερεύνηση όχι μόνο της ανεξάρτητης εξέλιξης των δύο διαστάσεων αλλά, κυρίως της πιθανής μεταξύ τους επίδρασης και αλληλεπίδρασης.

1.2 Η σημασία της ποιότητας λογισμικού

Η ποιότητα του λογισμικού αποτελεί βασική προϋπόθεση για την ομαλή λειτουργία, τη συντήρηση και τη μακροχρόνια εξέλιξη ενός συστήματος λογισμικού. Αν και έχει άμεση σχέση με την παρουσία ή απουσία σφαλμάτων στον κώδικα και στο ίδιο το λογισμικό, δεν περιορίζεται μόνο εκεί αλλά περιλαμβάνει χαρακτηριστικά όπως η αξιοπιστία, η συντηρησιμότητα, η κατανοησιμότητα, η δυνατότητα τροποποίησης και η δυνατότητα αποτελεσματικού ελέγχου του ίδιου του λογισμικού. Ένα σύστημα υψηλής ποιότητας μπορεί να λειτουργεί με σταθερό, προβλέψιμο και πιο ελεγχόμενο τρόπο καθώς επίσης παράλληλα επιτρέπει στους προγραμματιστές να πραγματοποιούν αλλαγές χωρίς να προκαλούνται εκτεταμένες και απρόβλεπτες

επιπτώσεις σε άλλα τμήματα του κώδικα (ή του λογισμικού). Αντίθετα, η χαμηλή ποιότητα του κώδικα του λογισμικού δύναται να αυξήσει την πολυπλοκότητα, δυσχεραίνει την κατανόηση του κώδικα και περιορίζει την ικανότητα των προγραμματιστών να εντοπίζουν και να διορθώνουν προβλήματα.

Η ποιότητα συνδέεται άμεσα με τη συντηρησιμότητα, την αξιοπιστία και την πιθανότητα εισαγωγής νέων σφαλμάτων και bugs. Ένας ορθά οργανωμένος, αρθρωτός και επαρκώς -ή πλήρως- ελεγμένος κώδικας επιτρέπει οι αλλαγές να περιορίζονται σε συγκεκριμένα τμήματα του συστήματος και μειώνει τον κίνδυνο παλινδρομήσεων. Αντίθετα, η χαμηλή ποιότητα -η οποία συχνά χαρακτηρίζεται από υψηλή πολυπλοκότητα, ισχυρές εξαρτήσεις, επανάληψη κώδικα, έλλειψη αυτοματοποιημένων tests, ύπαρξη μη χρησιμοποιούμενων τμημάτων κώδικα, κ.α.- μπορεί να οδηγήσει σε απρόβλεπτες συμπεριφορές και σε μεγαλύτερη πιθανότητα εμφάνισης σφαλμάτων. Η κατάσταση αυτή επηρεάζει με την σειρά του και την αξιοπιστία του λογισμικού, καθώς δυσχεραίνει την πρόληψη, την ανίχνευση και την αποκατάσταση λειτουργικών αστοχιών.

Η χαμηλή ποιότητα του κώδικα του λογισμικού ή του ίδιου του λογισμικού μπορεί να οδηγήσει σε ύπαρξη τεχνικού χρέους. Το τεχνικό χρέος δημιουργείται όταν επιλέγονται βραχυπρόθεσμες ή πρόχειρες ή εφήμερες τεχνικές λύσεις, οι οποίες μπορεί να επιταχύνουν προσωρινά την ανάπτυξη και εξέλιξη του λογισμικού, αλλά υποβαθμίζουν σημαντικά τη δομή και τη μελλοντική δυνατότητα εξέλιξης του συστήματος. Μπορεί να προκύψει από ανεπαρκή σχεδιασμό, ελλιπείς δοκιμαστικές σουίτες (tests), παρωχημένες εξαρτήσεις, επαναλαμβανόμενο ή μη χρησιμοποιούμενο κώδικα, ανεπαρκή τεκμηρίωση ή αναβολή αναγκαιών αναδιαρθρώσεων. Συμπερασματικά, η συσσώρευση του μειώνει την ποιότητα, επειδή καθιστά τον κώδικα περισσότερο πολύπλοκο, δυσνόητο και δύσκολο στη διαχείριση και συντήρηση. Παράλληλα, η χαμηλή ποιότητα ευνοεί τη δημιουργία νέου τεχνικού χρέους, καθώς κάθε νέα αλλαγή πραγματοποιείται πάνω σε μία ήδη προβληματική βάση κώδικα. Με τον τρόπο αυτό δημιουργείται ένας αυτοτροφοδοτούμενος κύκλος, στον οποίο το τεχνικό χρέος αυξάνει το κόστος και τη δυσκολία των αλλαγών, ενώ η πίεση για γρήγορη παράδοση οδηγεί σε όλο και περισσότερες παραχωρήσεις στην ποιότητα του λογισμικού.

Οι συνέπειες αυτές επεκτείνονται στο κόστος ανάπτυξης και στην παραγωγικότητα των προγραμματιστών που αναπτύσσουν ή συντηρούν ένα λογισμικό. Παρόλο που οι πρακτικές διασφάλισης ποιότητας, όπως για παράδειγμα οι αναθεωρήσεις κώδικα (code review), οι αυτοματοποιημένες σουίτες δοκιμών (automated tests), η στατική ανάλυση και η αναδιάρθρωση του κώδικα, απαιτούν αρχικά χρόνο και πόρους, μακροπρόθεσμα αποσκοπούν στην μείωση της επανεργασίας σε ίδια τμήματα κώδικα, τη διάρκεια επίλυσης προβλημάτων καθώς και το συνολικό κόστος συντήρησης κατά τη διάρκεια ζωής ενός λογισμικού. Η χαμηλή ποιότητα ενός λογισμικού οδηγεί σε κατανάλωση μεγάλου μέρους του χρόνου των προγραμματιστών στην κατανόηση δύσκολων, πολύπλοκων και δυσνόητων υλοποιήσεων, στη διάγνωση σφαλμάτων και στην αντιμετώπιση παρενεργειών προηγούμενων αλλαγών. Εν κατακλείδι, η ποιότητα δεν αποτελεί απλώς ένα τεχνικό χαρακτηριστικό, αλλά συνιστώσα που επηρεάζει άμεσα τη βιώσιμη παραγωγικότητα, το συνολικό κόστος ανάπτυξης, την αξιοπιστία και την

ικανότητα του λογισμικού να εξελίσσεται χωρίς συνεχή αύξηση του τεχνικού χρέους και του κινδύνου εισαγωγής νέων σφαλμάτων.

1.3 Η σημασία της ασφάλειας λογισμικού

Η ασφάλεια του λογισμικού αποτελεί θεμελιώδη προϋπόθεση για την αξιόπιστη λειτουργία των σύγχρονων πληροφοριακών συστημάτων, καθώς αποσκοπεί στην προστασία των δεδομένων, των λειτουργιών και των διαθέσιμων πόρων τους από μη εξουσιοδοτημένη πρόσβαση, χρήση, αποκάλυψη, τροποποίηση ή καταστροφή. Κεντρικές διαστάσεις της ασφάλειας αποτελεί το τρίγωνο CIA (Confidentiality / Εμπιστευτικότητα – Integrity / Ακεραιότητα – Availability / Διαθεσιμότητα). Η εμπιστευτικότητα διασφαλίζει ότι οι πληροφορίες είναι προσβάσιμες μόνο από εξουσιοδοτημένες οντότητες, η ακεραιότητα ότι τα δεδομένα και οι λειτουργίες δεν τροποποιούνται με μη επιτρεπτό τρόπο, ενώ η διαθεσιμότητα ότι το σύστημα και οι πληροφορίες του παραμένουν προσβάσιμα στους εξουσιοδοτημένους χρήστες όταν αυτό κρίνεται απαραίτητο ή απαιτείται. Η παραβίαση οποιασδήποτε από αυτές τις ιδιότητες μπορεί να προκαλέσει απώλεια δεδομένων, διακοπή παροχής υπηρεσιών, πιθανή οικονομική ζημία, νομικές κυρώσεις/συνέπειες και μείωση της εμπιστοσύνης των χρηστών προς το λογισμικό και τον οργανισμό που του ανήκει ή/και το διαχειρίζεται.

Σημαντική θέση κατέχουν οι ευπάθειες, δηλαδή αδυναμίες στον συνολικό σχεδιασμό του λογισμικού, στην υλοποίηση, στη διαμόρφωση ή στις εξαρτήσεις και εξωτερικές διασυνδέσεις ενός συστήματος λογισμικού, οι οποίες μπορούν να αποτελέσουν αντικείμενο μη εξουσιοδοτημένης πρόσβασης και εκμετάλλευσης. Η ύπαρξη μίας ευπάθειας στον πηγαίο κώδικα που δομεί το λογισμικό δεν συνεπάγεται αυτομάτως μία ήδη πραγματοποιημένη επίθεση. Η εκμετάλλευση του συστήματος πραγματοποιείται όταν ένας επιτιθέμενος αξιοποιεί την ευπάθεια αυτή για να ασκήσει μία κακόβουλη ενέργεια (malicious action) με αποτέλεσμα μία μη επιτρεπτή συμπεριφορά του λογισμικού. Τέτοιες ενέργειες μπορεί να είναι η προσπάθεια απόκτησης πρόσβασης σε προστατευμένα ή προσωπικά ή ευαίσθητα δεδομένα, την

εκτέλεση αυθαίρετων και μη εξουσιοδοτημένων από τον διαχειριστή του συστήματος ενεργειών, την αλλοίωση πληροφοριών ή την παρεμπόδιση της ορθής λειτουργίας μιας υπηρεσίας. Συνήθως, οι ευπάθειες προκύπτουν από προγραμματιστικά σφάλματα αλλά και από σύνολο άλλων αιτιών όπως είναι ο ανεπαρκής έλεγχος των δεδομένων εισόδων - χαρακτηριστικό παράδειγμα αποτελούν τα λεγόμενα SQL Injections όπου ο χρήστης τροφοδοτεί σε σημεία



Εικόνα 1.3.1 - Το τρίγωνο CIA *

εισόδου δεδομένων του συστήματος εκτελέσιμο κώδικα που φτάνει στο επίπεδο της βάσης δεδομένων και μπορεί να τροποποιήσει την δομή και την λειτουργία της σε περιπτώσεις που η είσοδος αυτή δεν ελέγχεται από το σύστημα-, λανθασμένη διαχείριση συνεδριών και συνδεδεμένων χρηστών στο σύστημα, επισφαλείς προεπιλογές, ανεπαρκή κρυπτογραφική προστασία ή χρήση εξωτερικών βιβλιοθηκών και λογισμικού τρίτων παρόχων το οποίο είναι ευάλωτο σε επιθέσεις. Για τον λόγο αυτό, η ασφάλεια πρέπει να ενσωματώνεται σε ολόκληρο τον κύκλο ζωής ανάπτυξης ενός λογισμικού με στόχο τη μείωση του αριθμού ευπαθειών, τον περιορισμό των συνεπειών από την πιθανή εκμετάλλευσή τους και την αντιμετώπιση των αιτιών που τις δημιουργούν. Στο σημείο αυτό, κρίνεται αναγκαία η διευκρίνιση της αδυναμίας της συνολικής εξάλειψης των ευπαθειών ενός σύγχρονου λογισμικού, λόγω του όγκου των εξωτερικών βιβλιοθηκών που αυτό συνήθως χρησιμοποιεί καθώς και η ταχύτητα ανάπτυξης των τεχνολογιών που αυτό βασίζεται.

Η συνολική σημασία της ασφάλειας αποτυπώνεται μέσα από την έννοια του κινδύνου, ο οποίος εκφράζει τον βαθμό στον οποίο ένα σύστημα απειλείται από ένα πιθανό περιστατικό και αξιολογείται συνήθως σε συνάρτηση με την πιθανότητα πραγματοποίησής του και τη σοβαρότητα των επιπτώσεών του. Η συνολική πιθανότητα να τεθεί ένα σύστημα σε κίνδυνο δίνεται συναρτήσει της ύπαρξης μίας ή περισσοτέρων ευπαθειών καθώς και από τις απειλές μέσω των οποίων μπορούν κακόβουλοι χρήστες να εκμεταλλευτούν μία ευπάθεια, την έκθεση του συστήματος, την αξία των προστατευόμενων πόρων και τα ήδη εφαρμοζόμενα μέτρα ασφαλείας. Συμπερασματικά, δύο παρόμοιες ευπάθειες ενδέχεται να παρουσιάζουν διαφορετικό επίπεδο κινδύνου ανάλογα με το περιβάλλον, τον χρήστη και τις πιθανές συνέπειες της εκμετάλλευσής τους. Η ασφάλεια του λογισμικού δεν αποτελεί μία απόλυτη κατάσταση πλήρους απουσίας ευπαθειών και κενών ασφαλείας, αλλά μία συνεχή διαδικασία αναγνώρισης, αξιολόγησης, περιορισμού και παρακολούθησης των κινδύνων. Όπως και η συνιστώσα της ποιότητας του λογισμικού, δύναται να μεταβληθεί με κάθε αλλαγή του πηγαίου κώδικα, καθώς ένα commit ή ένα Pull Request (συνήθως ένα σύνολο από σειριακά ιστορικά commits) μπορεί είτε να διορθώσει υπάρχουσες αδυναμίες είτε να εισάγει νέες. Η συστημική παρατήρηση αυτών των μεταβολών δημιουργεί τη βάση για τη μελέτη της αλληλεπίδρασης και αμφίδρομης επίδρασης μεταξύ της ποιότητας και της ασφάλειας του κώδικα και του λογισμικού κατά την εκτέλεση του λογισμικού.

1.4 Η παραδοσιακή απομόνωση της ποιότητας και της ασφάλειας του λογισμικού

Η ποιότητα καθώς και η ασφάλεια του πηγαίου κώδικα του λογισμικού καθώς και του ίδιου του λογισμικού αν και αποτελούν την ρίζα του ίδιου προβλήματος, δηλαδή της υποβάθμισης της συνολικής αξιοπιστίας και απόδοσης ενός συστήματος λογισμικού συχνά αντιμετωπίζονται ως διαφορετικές και ανεξάρτητες πηγές σφαλμάτων. Η ποιότητα συνδέεται παραδοσιακά με χαρακτηριστικά του λογισμικού όπως η λειτουργική ορθότητα και σταθερότητα, η αξιοπιστία, η συντηρησιμότητα, η

αποδοτικότητα και η δυνατότητα ελέγχου ενός συστήματος. Αντίστοιχα, η ασφάλεια επικεντρώνεται στην προστασία του λογισμικού και του πηγαίου του κώδικα και των δεδομένων του από εξερχόμενες, κυρίως, απειλές, μη εξουσιοδοτημένη πρόσβαση, κακόβουλες ενέργειες και εκμετάλλευση ευπαθειών. Ωστόσο, ο διαχωρισμός μεταξύ της ποιότητας και της ασφάλειας δεν είναι απόλυτος σε θεωρητικό επίπεδο. Το ISO/IEC 25010:2023 πρότυπο εντάσσει την ασφάλεια στο συνολικό μοντέλο ποιότητας ενός προϊόντος λογισμικού, γεγονός που δείχνει ότι η ασφάλεια μπορεί να θεωρηθεί εξαρτημένο και αναπόσπαστο τμήμα της συνολικής ποιότητάς του. Στην πρακτική ανάπτυξη του λογισμικού όμως, οι δύο αυτές συνιστώσες εξακολουθούν να αξιολογούνται μέσω διαφορετικών διαδικασιών, εργαλείων, ομάδων και μετρικών.

Ένας βασικός λόγος που προκαλεί αυτή την απομόνωση είναι ότι η ποιότητα και η ασφάλεια του λογισμικού εξετάζουν παραδοσιακά διαφορετικές κατηγορίες προβλημάτων του. Οι πρακτικές διασφάλισης ποιότητας εστιάζουν συνήθως σε σφάλματα που προκύπτουν από ακούσιες αστοχίες, λανθασμένη υλοποίηση απαιτήσεων, αυξημένη πολυπλοκότητα ή ανεπαρκή σχεδιασμό. Σκοπός αυτών των πρακτικών είναι να επιβεβαιώσουν ότι το λογισμικό λειτουργεί όπως οι απαιτήσεις του προβλέπουν καθώς επίσης και ότι μπορεί να συντηρείται και να εξελίσσεται αποτελεσματικά. Αντίθετα, η ασφάλεια εξετάζει το σύστημα υπό την παρουσία αντιπάλου ή κακόβουλου χρήστη, ο οποίος ενδέχεται να αναζητά σκόπιμα τρόπους παράκαμψης των προβλεπόμενων λειτουργιών του λογισμικού. Ένα χαρακτηριστικό μπορεί να λειτουργεί ορθά για τις αναμενόμενες εισόδους και χρήστες, αλλά παράλληλα να παραμένει ευάλωτο σε πιθανές ειδικά κατασκευασμένες εισόδους, κατάχρηση δικαιωμάτων ή κακόβουλη αλληλουχία ενεργειών. Έτσι, τα συμβατικά functional tests (λειτουργικές δοκιμές) μπορεί να επιβεβαιώνουν ότι μία λειτουργία παράγει το αναμενόμενο αποτέλεσμα, χωρίς όπως απαραίτητα να εξετάζουν εάν η λειτουργία μπορεί να χρησιμοποιηθεί από μη εξουσιοδοτημένο χρήστη ή να παρακαμφθεί υπό διάφορες συνθήκες επίθεσης.

Ο διαχωρισμός μεταξύ των δύο αυτών εννοιών ενισχύθηκε επίσης και από τον τρόπο οργάνωσης του κύκλου ζωής ανάπτυξης ο οποίος χρησιμοποιείται. Οι δραστηριότητες και έλεγχοι ποιότητας, όπως είναι για παράδειγμα οι αυτοματοποιημένες δοκιμές, η αξιολόγηση κώδικα, η στατική ανάλυση και ο έλεγχος της κάλυψης του κώδικα από διάφορες δοκιμές, ενσωματώθηκαν σταδιακά στην καθημερινή ροή και εργασία των ομάδων ανάπτυξης καθώς και των ιδίων των προγραμματιστών. Αντίθετα, οι έλεγχοι ασφάλειας πραγματοποιούνταν συχνά σε μεταγενέστερο μη αρχικό στάδιο, αφού είχε ολοκληρωθεί μεγάλο μέρος της υλοποίησης ή λίγο πριν από τη διάθεση του συστήματος. Σε αυτό το μοντέλο ανάπτυξης, η ασφάλεια λειτουργούσε ως ένας τελικός έλεγχος ή ως γενική εξωτερική διαδικασία έγκρισης και όχι ως συνεχής δραστηριότητα της διαδικασίας της ανάπτυξης. Έχει επισημανθεί πως είναι λίγα τα παραδοσιακά μοντέλα κύκλου ζωής λογισμικού αντιμετωπίζουν αναλυτικά την ασφάλεια και ότι οι πρακτικές ασφαλούς ανάπτυξης πρέπει να προστίθενται και να ενσωματώνονται στις υπάρχουσες διαδικασίες. Ένας καθυστερημένος έλεγχος της ασφάλειας του λογισμικού μπορεί να οδηγήσει στον εντοπισμό προβλημάτων όταν σημαντικές σχεδιαστικές και αρχιτεκτονικές αποφάσεις έχουν ήδη ληφθεί, καθιστώντας τη διόρθωσή τους περισσότερο σύνθετη ή/και δαπανηρή.

Αυτή η απομόνωση εμφανίζεται ακόμη και σε οργανωτικό επίπεδο ενός οργανισμού. Οι προγραμματιστές και οι ομάδες Quality Assurance είναι συνήθως υπεύθυνοι για τη λειτουργικότητα, τα tests, τα σφάλματα και την ποιότητα του πηγαίου κώδικα, ενώ η ανάληψη του ελέγχου ευπαθειών, της συμμόρφωσης και της αξιολόγησης κινδύνων αναλαμβάνεται από άλλη ομάδα. Οι διαφορετικές ομάδες συνήθως κάνουν χρήση διαφορετικών εργαλείων, συστημάτων αναφοράς και κριτηρίων προτεραιοποίησης. Η έλλειψη επαρκούς επικοινωνίας μεταξύ των ομάδων οδηγεί σε λανθασμένη αντιμετώπιση από την κάθε ομάδα με αποτέλεσμα την πιθανή καθυστέρηση της παράδοσης ή και την αύξηση του συνολικού κόστους υλοποίησης, συνήθως λόγω λανθασμένης οπτικής και επίλυσης. Επίσης, έχουν καταγραφεί δυσκολίες ενσωμάτωσης της ασφάλειας στις συνεχείς διαδικασίες ανάπτυξης με σχετικές προκλήσεις: ανθρώπους, πρακτικές, εργαλεία και υποδομές.

Ο συγκεκριμένος τρόπος αντιμετώπισης και απομόνωσης των δύο αυτών διαστάσεων είναι προβληματική αφού η ποιότητα και η ασφάλεια επηρεάζονται συχνά από κοινά χαρακτηριστικά του πηγαίου κώδικα. Η υψηλή πολυπλοκότητα, οι ισχυρές εξαρτήσεις, η επανάληψη κώδικα, η ύπαρξη μη χρησιμοποιούμενου κώδικα και η δυσκολία κατανόησης συμβάλλουν στην αύξηση τόσο της πιθανότητας εμφάνισης λειτουργικών σφαλμάτων όσο και της δυσκολίας εντοπισμού αδυναμιών ασφάλειας. Αντίστοιχα, μία αλλαγή που πραγματοποιείται με στόχο τη βελτίωση της ποιότητας μπορεί να έχει θετική, ουδέτερη ή αρνητική επίδραση στην ασφάλεια. Έρευνες σχετικά με την επίδραση του οποιουδήποτε refactoring έχουν δείξει ότι οι παρεμβάσεις της ποιότητας μπορούν να μεταβάλουν τις ιδιότητες ασφάλειας του συστήματος και ότι η επίλυση προβλημάτων ποιότητας μπορεί να διορθώνει ορισμένα προβλήματα ασφάλειας, χωρίς όμως να εξαλείφει αναγκαστικά το σύνολό τους. Αντίστροφα, μία αλλαγή ασφάλειας μπορεί να επηρεάσει άλλες ιδιότητες ποιότητας. Η προσθήκη αυστηρότερων ελέγχων πρόσβασης, επικύρωσης συνόδων και εισόδων, κρυπτογράφησης ή καταγραφής συμβάντων μπορεί να αυξήσει την πολυπλοκότητα, να μειώσει την απόδοση ή να δυσχεράνει την συντήρηση του συστήματος, εάν δεν ενσωματωθεί με κατάλληλο σχεδιασμό. Η άκριτη βελτιστοποίηση αποκλειστικά για μία διάσταση, πιθανώς να μεταφέρει το πρόβλημα σε κάποια άλλη. Η ποιότητα και η ασφάλεια δεν είναι ταυτόσημες έννοιες εξ ορισμού και θα πρέπει να αποφεύγεται η πλήρης συγχώνευση, καθώς διαθέτουν διαφορετικούς στόχους και μεθόδους αξιολόγησης. Όμως, η οπτική της πλήρους ανεξαρτησίας μεταξύ των δύο αυτών όρων είναι λανθασμένη -όπως και η έρευνα δείχνει-, επειδή εφαρμόζονται πάνω στον ίδιο πηγαίο κώδικα, επηρεάζονται από τις ίδιες τεχνικές αποφάσεις και μεταβάλλονται μέσα από τις ίδιες διαδικασίες εξέλιξης του λογισμικού.

Η αναγνώριση αυτών των αλληλεπιδράσεων και αλληλεξαρτήσεων έχει οδηγήσει στην ανάπτυξη περισσότερο ενοποιημένων προσεγγίσεων, οι οποίες δεν αναλύονται πλήρως αλλά μόνο κατά περίπτωση στην παρούσα εργασία. Οι προσεγγίσεις αυτές επιδιώκουν την ενσωμάτωση των απαιτήσεων και των ελέγχων ασφάλειας από τα πρώτα στάδια του κύκλου ζωής και τη συνεχή εκτέλεσή τους παράλληλα με τους προαναφερθέντες ελέγχους ποιότητας. Παράλληλα, η ασφάλεια μετατρέπεται σταδιακά από αποκλειστική αρμοδιότητα μιας εξειδικευμένης ομάδας

σε κοινή ευθύνη των προγραμματιστών, των μηχανικών ποιότητας, των ομάδων λειτουργίας και των ειδικών ασφάλειας.

Στο πλαίσιο της παρούσας εργασίας, η παραδοσιακή απομόνωση ποιότητας και ασφάλειας δημιουργεί ένα σημαντικό ερευνητικό κενό. Εφόσον οι διαστάσεις αυτές αξιολογούνται συνήθως μέσω διαφορετικών εργαλείων και μετρικών, υπάρχει κίνδυνος να μην παρατηρούνται πάντα (ή και γενικά) οι ταυτόχρονες μεταβολές τους, κοινά πρότυπα, συσχετίσεις ή διαφοροποιήσεις που εμφανίζονται κατά την εξέλιξη του ίδιου λογισμικού. Θεωρητικά, μία τροποποίηση αποσκοπεί στη βελτίωση της συντηρησιμότητας μπορεί να συνοδεύεται από μεταβολή security-related χαρακτηριστικών και, αντίστροφα, μία παρέμβαση ασφάλειας μπορεί να επηρεάζει χαρακτηριστικά που σχετίζονται με την ποιότητα του κώδικα.

Η συγκριτική παρατήρηση καταστάσεων πριν και μετά από μία αλλαγή αποτελεί μία πιθανή προσέγγιση για τη μελέτη αντίστοιχων φαινομένων, υπό την προϋπόθεση ότι οι χρησιμοποιούμενες μετρικές παρουσιάζουν επαρκή μεταβλητότητα μεταξύ των δύο καταστάσεων. Λόγω των παραπάνω, η συγκεκριμένη λογική αποτέλεσε μέρος του αρχικού σχεδιασμού της παρούσας έρευνας. Η τελική εμπειρική στρατηγική, όμως, καθορίστηκε από τα χαρακτηριστικά των διαθέσιμων δεδομένων και εστιάζει κυρίως στην κοινή και πολυμεταβλητή εξέταση των quality και security indicators στην τελική κατάσταση των Pull Requests.

1.5 Η ποιότητα και η ασφάλεια μπορούν να αλληλοεπιδρούν

Η ποιότητα και η ασφάλεια του λογισμικού μπορούν να αλληλοεπιδρούν επειδή επηρεάζονται από κοινά χαρακτηριστικά του πηγαίου κώδικα και μεταβάλλονται από τις ίδιες διαδικασίες ανάπτυξης και συντήρησης. Για παράδειγμα, ένας περίπλοκος κώδικας συνήθως περιλαμβάνει περισσότερες διαδρομές εκτέλεσης (branching/διακλαδώσεις), συνθήκες και εξαρτήσεις, με αποτέλεσμα να είναι δυσκολότερο να κατανοηθεί, να ελεγχθεί και να δοκιμαστεί ολοκληρωμένα. Η δυσκολία αυτή έχει ως αποτέλεσμα την υποβάθμιση της συντηρησιμότητας, αλλά μπορεί παράλληλα να επιτρέψει σε αδυναμίες ασφαλείας να παραμείνουν απαρατήρητες. Εμπειρικές μελέτες της κοινότητας έχουν εξετάσει μετρικές πολυπλοκότητας, μεταβολών του κώδικα και δραστηριότητας των προγραμματιστών ως ενδείξεις για τον εντοπισμό τμημάτων λογισμικού που παρουσιάζουν αυξημένη πιθανότητα να περιέχουν ή να εμφανίζουν ευπάθειες, χωρίς αυτό να συνεπάγεται ότι η πολυπλοκότητα προκαλεί από μόνη της μία συγκεκριμένη ευπάθεια.

Αντίστοιχη αλληλεπίδραση εμφανίζεται στις περιπτώσεις του διπλότυπου κώδικα (duplicated code). Όταν το ίδιο ή παρόμοιο τμήμα πηγαίου κώδικα υπάρχει σε περισσότερα σημεία, ένα προγραμματιστικό σφάλμα ή μία αδυναμία ασφάλειας μπορεί να αντιγραφεί μαζί του αυξάνοντας με αριθμητική πρόοδο τα σφάλματα και τις ανεξέλεγκτες συμπεριφορές του λογισμικού. Επιπλέον, η διόρθωση ενός προβλήματος σε ένα αντίγραφο δεν εγγυάται ότι θα εφαρμοστεί σε κάθε άλλο από τα

ταυτόσημα τμήματα κώδικα, με αποτέλεσμα η ευπάθεια να παραμένει ενεργή ή να επανεμφανίζεται σε διαφορετικό σημείο του συστήματος. Η κατάλληλη αναδιάρθρωση μπορεί επομένως να αφαιρέσει διπλότυπο κώδικα, να συγκεντρώσει τη σχετική λογική σε ένα σημείο και να διευκολύνει τόσο στην συντήρηση όσο και στη συνεπή εφαρμογή μίας διόρθωσης ασφαλείας.

Ωστόσο, το *refactoring* (αναδιάρθρωση) δεν οδηγεί αυτομάτως σε ταυτόχρονη βελτίωση όλων των χαρακτηριστικών του λογισμικού. Παρότι έχει ως στόχο τη βελτίωση της εσωτερικής δομής χωρίς αλλαγή της εξωτερικής συμπεριφοράς, λανθασμένη ή ανεπαρκής αναδιάρθρωση μπορεί να εισάγει νέα επίπεδα αφαίρεσης και νέα λειτουργικά προβλήματα ή σφάλματα ασφαλείας. Για παράδειγμα, μία αλλαγή στον κώδικα αυθεντικοποίησης που προσθέτει πολυπαραγοντικό έλεγχο, αυστηρότερη διαχείριση συνεδριών (*sessions*) ή περισσότερους ελέγχους πρόσβασης μπορεί να βελτιώσει σημαντικά την προσφερόμενη ασφάλεια του λογισμικού, αλλά να εισάγει νέες συνθήκες και εξαρτήσεις με αποτέλεσμα την αύξηση της συνολικής πολυπλοκότητας του κώδικα άρα και την μείωση της υπολογιζόμενης ποιότητάς του.

Αντίθετα, υπάρχουν πρακτικές οι οποίες δύναται να επηρεάζουν ταυτόχρονα και θετικά (ή αντίστοιχα αρνητικά) και τις δύο συνιστώσες, την ποιότητα και την ασφάλεια. Χαρακτηριστικό παράδειγμα αποτελεί η πρακτική του *error handling*. Η συνεπής επικύρωση εισόδων, η ορθή διαχείριση εξαιρέσεων, η ελεγχόμενη αποτυχία και η κατάλληλη ανάκαμψη του ίδιου του λογισμικού έπειτα από την ύπαρξη σφάλματος, ενισχύουν την αξιοπιστία, αφού περιορίζουν τις απρόβλεπτες διακοπές και τις μη ελεγχόμενες καταστάσεις του συστήματος. Παράλληλα, προστατεύουν την ασφάλεια του λογισμικού, αποτρέποντας τη διαρροή ευαίσθητων ή άλλων πληροφοριών μέσω μηνυμάτων σφάλματος, την παράκαμψη ελέγχων ή την εκμετάλλευση μη αναμενόμενων καταστάσεων. Συμπερασματικά, η σχέση ποιότητας και ασφάλειας μπορεί να υπάρξει είτε ως συνεργατική είτε ως ανταγωνιστική. Μία αλλαγή μπορεί να βελτιώνει και τις δύο διαστάσεις, να βελτιώνει την μία με αποτέλεσμα την υποβάθμιση της άλλης ή να δημιουργεί νέες παρενέργειες. Η δυνατότητα αυτή καθιστά αναγκαία την αναθεώρηση κώδικα και την κοινή εξέτασή τους σε επίπεδο αλλαγών του πηγαίου κώδικα και συνήθως και των αντίστοιχων *Pull Requests*.

1.6 Το ερευνητικό πρόβλημα

Η ποιότητα και η ασφάλεια του λογισμικού συχνά αξιολογούνται ως δύο ανεξάρτητες και διακριτές διαστάσεις του ίδιου προβλήματος, της συνολικής απόδοσης και αξιοπιστίας του λογισμικού και του πηγαίου κώδικα. Οι αναλύσεις ποιότητας επικεντρώνονται συνήθως σε χαρακτηριστικά όπως είναι η συντηρησιμότητα, η πολυπλοκότητα, η αξιοπιστία, ο διπλότυπος κώδικας (*duplicated code*) και οι «μυρωδιές κώδικα» (*code smells*), ενώ οι αναλύσεις ασφάλειας εξετάζουν ευπάθειες, *security hotspots* και άλλες πιθανές αδυναμίες του κώδικα. Η ανεξάρτητη εξέτασή τους επιτρέπει την αξιολόγηση της κατάστασης κάθε

συνιστώσας, δεν αποκαλύπτει όμως απαραίτητα εάν οι μεταβολές που πραγματοποιούνται στη μία διάσταση συνδέονται με αντίστοιχες μεταβολές στην άλλη.

Το ερευνητικό πρόβλημα προκύπτει από το γεγονός ότι η ποιότητα και η ασφάλεια εφαρμόζονται και αξιολογούνται πάνω στον ίδιο πηγαίο κώδικα, επηρεάζονται από κοινά τεχνικά χαρακτηριστικά και διαμορφώνονται μέσα στο ίδιο project-specific περιβάλλον. Μετρικές όπως η πολυπλοκότητα, η συντηρησιμότητα, η αξιοπιστία, η διπλοτυπία και οι security-related indicators δεν είναι απαραίτητα ανεξάρτητες μεταξύ τους, ενώ η σημασία και η συμπεριφορά τους ενδέχεται να διαφοροποιούνται σημαντικά μεταξύ διαφορετικών έργων λογισμικού. Εν συνεχεία, η ανεξάρτητη εξέταση μεμονωμένων μετρικών ποιότητας ή/και ασφάλειας δεν επαρκεί για την πλήρη διερεύνηση της μεταξύ τους σχέσης.

Ο αρχικός σχεδιασμός της έρευνας περιλάμβανε τη σύγκριση των καταστάσεων στα στάδια πριν και μετά την ολοκλήρωση ενός Pull Request, ώστε να διερευνηθεί η περίπτωση οι μεταβολές στη μία διάσταση να συνοδεύονται από αντίστοιχες μεταβολές στην άλλη. Η δυνατότητα αυτή αποτελεί θεωρητικά έναν άμεσο τρόπο εξέτασης πιθανών συνεργατικών ή ανταγωνιστικών μεταβολών. Η καταλληλότητά της, εν τέλει, εξαρτάται από την ύπαρξη επαρκούς within-PR μεταβλητότητας στις εξεταζόμενες μετρικές, προϋπόθεση η οποία αξιολογήθηκε εμπειρικά κατά την φάση προετοιμασίας των δεδομένων.

Το βασικό ερευνητικό πρόβλημα της εργασίας καταλήγει στο να διαμορφωθεί ευρύτερα ως η διερεύνηση της σχέσης μεταξύ των διαστάσεων ποιότητας και ασφάλειας σε πραγματικά Pull Requests, των χαρακτηριστικών του αναλυόμενου κώδικα που συνδέονται με τα αντίστοιχα outcomes, της ετερογένειας που εμφανίζεται μεταξύ διαφορετικών projects και της πιθανής ύπαρξης ευρύτερων πολυμεταβλητών ή λανθανουσών δομών. Η διερεύνηση των pr_base-pr_closed μεταβολών διατηρείται ως μέρος του αρχικού ερευνητικού προβλήματος, χωρίς να προϋποτίθεται ότι τα διαθέσιμα δεδομένα επιτρέπουν αξιόπιστη εμπειρική τεκμηρίωση χρονικών συνεργειών ή αντισταθμίσεων.

1.7 Ερευνητικό κενό

Παρόλο που η υπάρχουσα βιβλιογραφία έχει ήδη μελετήσει εκτενώς την ποιότητα και την ασφάλεια του λογισμικού ως δύο ξεχωριστές μη εξαρτώμενες και αλληλοεπιδρώμενες διαστάσεις, δεν έχει μελετηθεί σε ικανοποιητικό επίπεδο η μεταξύ τους σχέση και επίδραση. Έχουν αντίστοιχα πραγματοποιηθεί μελέτες σε επίπεδο προτάσεων τροποποίησης (Pull Requests) που εξετάζουν χαρακτηριστικά όπως η ποιότητα του κώδικα, το τεχνικό χρέος, τα code smells, οι ευπάθειες ή παράγοντες που επηρεάζουν την αποδοχή και την επιτυχία ενός Pull Request. Στην έρευνα των Lenarduzzi et al. (2021) αναλύθηκαν 36.000 περίπου Pull Requests από 28 έργα ως προς την παρουσία διαφορετικών προβλημάτων ποιότητας και ασφάλειας κώδικα και λογισμικού. Ο βασικός, όμως, στόχος της συγκεκριμένης έρευνας ήταν η διερεύνηση της επίδρασης αυτών των προβλημάτων στην

πιθανότητα αποδοχής από την προγραμματιστική και την ομάδα ανάπτυξης και συντήρησης του λογισμικού ενός Pull Request και όχι η μελέτη του τρόπου αλληλοεπίδρασης της ποιότητας και της ασφάλειας του κώδικα και του λογισμικού πριν και μετά την τροποποίηση του κώδικα. Συνεπώς, η εμπειρική διερεύνηση της αλληλοεπίδρασης αυτών των δύο συνιστωσών σε επίπεδο μεμονωμένης τροποποίησης και Pull Request παραμένει συγκριτικά περιορισμένη.

Ένα δεύτερο σημαντικό ζήτημα αφορά τη διαφοροποίηση μεταξύ διαφορετικών έργων και repositories. Τα λογισμικά διαφέρουν ως προς το μέγεθος, την αρχιτεκτονική, τις χρησιμοποιούμενες τεχνολογίες, τις πρακτικές ανάπτυξης και το ιστορικό τους, με αποτέλεσμα μία σχέση που παρατηρείται σε ένα αποθετήριο πηγαίου κώδικα λογισμικού (repository) να μην είναι απαραίτητα ίδια σε κάποιο άλλο. Η βιβλιογραφία του cross-project defect prediction έχει αναδείξει τη σημαντική ετερογένεια των δεδομένων μεταξύ έργων, ενώ επίσης έχει διαπιστωθεί ότι η αποτελεσματικότητα των μοντέλων και των χρησιμοποιούμενων μετρικών δύναται να μεταβληθεί ανάλογα με το εκάστοτε έργο λογισμικού. Αντίστοιχα, μεγάλης κλίμακας μελέτες Pull Requests χρησιμοποιούν mixed-effects models προκειμένου να λάβουν υπόψη ότι οι παρατηρήσεις είναι ομαδοποιημένες μέσα σε διαφορετικά repositories και ότι οι επιδράσεις ορισμένων παραγόντων μεταβάλλονται ανάλογα με το πλαίσιο του έργου ή/και αποθετηρίου. Συμπερασματικά, μία ανάλυση η οποία αντιμετωπίζει όλα τα Pull Requests ως πλήρως ανεξάρτητες παρατηρήσεις ενδέχεται να αγνοήσει ουσιαστικές project-specific effects.

Η ποιότητα και η ασφάλεια του κώδικα και του λογισμικού δεν περιγράφονται από μία μόνο μεταβλητή αλλά από ένα σύνολο διαφορετικών και συχνά συσχετισμένων software metrics (μετρικών λογισμικού), όπως είναι η πολυπλοκότητα (complexity), η γνωστική πολυπλοκότητα (cognitive complexity), τα code smells, ο διπλότυπος κώδικας (duplicated code), τα προβλήματα αξιοπιστίας (reliability issues), τα προβλήματα ασφαλείας (security issues) και οι ευπάθειες (vulnerabilities). Η ταυτόχρονη χρήση μεγάλου αριθμού αντίστοιχων μεταβλητών οδηγεί στην δημιουργία προβλημάτων ερμηνείας και πιθανής πολυσυγγραμμικότητας, ενώ διαφορετικές μετρικές μπορεί στην πραγματικότητα να εκφράζουν κοινές, υποκείμενες διαστάσεις του λογισμικού. Για τον προαναφερθέντα λόγο, τεχνικές όπως η Principal Component Analysis (PCA) και η Factor Analysis έχουν χρησιμοποιηθεί στη βιβλιογραφία των μετρικών λογισμικού για τη μείωση της διάστασης και την αποκάλυψη ευρύτερων δομών που βρίσκονται πίσω από ομάδες συσχετισμένων μετρικών. Παρόλο που οι τεχνικές αυτές έχουν εφαρμοστεί ήδη σε προβλήματα ποιότητας κώδικα και λογισμικού καθώς και defect prediction, η διερεύνηση αντίστοιχων latent structures μέσα σε ένα κοινό σύνολο μετρικών ποιότητας και ασφάλειας και ειδικότερα στις μεταβολές τους σε επίπεδο PR αποτελεί ένα λιγότερο διερευνημένο πεδίο.

Η παρούσα εργασία επιχειρεί να συμβάλει στην κάλυψη των παραπάνω κενών μέσω της κοινής εξέτασης πολλαπλών quality και security μετρικών σε επίπεδο Pull Request, λαμβάνοντας ταυτόχρονα υπόψη την ιεραρχική οργάνωση των παρατηρήσεων μέσα σε διαφορετικά projects και τη μεταξύ τους πολυμεταβλητή δομή. Ο αρχικός ερευνητικός σχεδιασμός περιλάμβανε επιπλέον τη διερεύνηση των μεταβολών μεταξύ των καταστάσεων pr_base και pr_closed. Η συγκεκριμένη

διάσταση διατηρείται ως μέρος του αρχικού ερευνητικού ενδιαφέροντος και αξιολογείται ως προς τη μεθοδολογική της καταλληλότητα, ενώ η κύρια τελική ανάλυση επικεντρώνεται στις cross-sectional σχέσεις οι οποίες και παρατηρούνται στην τελική κατάσταση των Pull Requests.

1.8 Σκοπός της εργασίας

Η παρούσα εργασία έχει ως σκοπό την εμπειρική διερεύνηση της σχέσης μεταξύ των διαστάσεων ποιότητας και ασφάλειας του πηγαίου κώδικα και του λογισμικού στο πλαίσιο πραγματικών Pull Requests. Η διερεύνηση εστιάζει στο κατά πόσο διαφορετικοί δείκτες ποιότητας και ασφάλειας παρουσιάζουν μεταξύ τους συσχετίσεις, καθώς και στον τρόπο κατά τον οποίο χαρακτηριστικά του αναλυόμενου κώδικα και του εκάστοτε project συνδέονται με την παρουσία προβλημάτων συντηρησιμότητας και ασφάλειας.

Για τον σκοπό αυτό εξετάζονται μετρικές που σχετίζονται με το μέγεθος, την πολυπλοκότητα, τη γνωστική πολυπλοκότητα, τη διπλοτυπία, την αξιοπιστία, τη συντηρησιμότητα και τη ασφάλεια. Ιδιαίτερη έμφαση δίνεται στην πυκνότητα προβλημάτων ποιότητας και ασφάλειας, στην κατηγορία του Pull Request και στη διαφοροποίηση μεταξύ των επιμέρους projects. Παράλληλα, διερευνώνται οι πιθανότητες πολλαπλές μετρικές να οργανώνονται σε ευρύτερες κοινές ή λανθάνουσες διαστάσεις, με στόχο η ποιότητα και η ασφάλεια να μην αντιμετωπίζονται αποκλειστικά μέσω μεμονωμένων δεικτών.

Ο αρχικός ερευνητικός σχεδιασμός περιλάμβανε επίσης τη συγκριτική εξέταση των καταστάσεων `pr_base` και `pr_closed`, με στόχο τη διερεύνηση πιθανών μεταβολών ποιότητας και ασφάλειας μέσα στο ίδιο Pull Request. Ο έλεγχος των διαθέσιμων δεδομένων έδειξε εξαιρετικά περιορισμένη μεταβλητότητα μεταξύ των δύο αυτών καταστάσεων. Συνεπώς, η κύρια εμπειρική στρατηγική διαμορφώθηκε εν τέλει ως cross-sectional ανάλυση, με μία τελική `pr_closed` παρατήρηση ανά μονάδα Pull Request και με έμφαση στις σχέσεις μεταξύ των μετρικών και στην ιεραρχική δομή των δεδομένων. η δυνατότητα εξέτασης χρονικών συνεργειών ή αντισταθμίσεων διατηρείται ως μέρος του αρχικού ερευνητικού προβλήματος, αλλά δεν αντιμετωπίζεται ως εμπειρικά τεκμηριωμένο αποτέλεσμα της παρούσας εργασίας.

1.9 Επιμέρους στόχοι

Για την επίτευξη του γενικού σκοπού της εργασίας, διαμορφώνονται οι ακόλουθοι επιμέρους στόχοι:

- Η ποσοτική αποτύπωση των βασικών δεικτών ποιότητας και ασφάλειας στην τελική κατάσταση των εξεταζόμενων Pull Requests.

- Η διερεύνηση των σχέσεων μεταξύ χαρακτηριστικών του αναλυόμενου κώδικα, όπως το μέγεθος και η πολυπλοκότητα, και της πυκνότητας προβλημάτων συντηρησιμότητας και ασφάλειας.
- Η εξέταση της πιθανής διαφοροποίησης των quality και security outcomes σύμφωνα με την κατηγορία στην οποία το εκάστοτε Pull Request ανήκει (quality, security, both).
- Η εκτίμηση του βαθμού στον οποίο οι διαφορές μεταξύ των projects συνδέονται με τη συνολική μεταβλητότητα των μετρικών ποιότητας και ασφάλειας.
- Η ταυτόχρονη αξιοποίηση πολλαπλών μετρικών λογισμικού, ώστε η ποιότητα και η ασφάλεια να εξετάζονται ως πολυδιάστατα χαρακτηριστικά και όχι αποκλειστικά μέσω μεμονωμένων δεικτών.
- Η διερεύνηση πιθανών κοινών ή λανθανουσών δομών μεταξύ των μετρικών μέσω Principal Component Analysis και Factor Analysis.
- Η εφαρμογή και σύγκριση διαφορετικών στατιστικών προσεγγίσεων, με ιδιαίτερη έμφαση στη χρήση ιεραρχικών μοντέλων που λαμβάνουν υπόψη την ομαδοποίηση των Pull Requests ανά project και σε συμπληρωματικούς ελέγχους robustness και sensitivity.
- Η αξιολόγηση της καταλληλότητας της αρχικά προβλεπόμενης pr_base-pr_closed ανάλυσης και η τεκμηρίωση των περιορισμών που προκύπτουν από την περιορισμένη within-PR μεταβλητότητα των μετρικών.

Μέσα από τους παραπάνω στόχους, η εργασία επιδιώκει να προσεγγίσει τη σχέση μεταξύ ποιότητας και ασφάλειας τόσο με την χρήση επιμέρους παρατηρούμενων μετρικών όσο και μέσα από την ευρύτερη project-level και πολυμεταβλητή δομή των δεδομένων.

1.10 Ερευνητικά ερωτήματα

Η παρούσα εργασία οργανώνεται σε έξι (6) βασικά ερευνητικά ερωτήματα (Research Questions – RQs), για τα οποία προσπαθεί να προσφέρει μία λύση καθώς και να απαντήσει σε αυτά:

RQ-1

Υπάρχει στατιστικά σημαντική και ισχυρή σχέση μεταξύ δεικτών ποιότητας και δεικτών ασφάλειας λογισμικού σε Pull Requests;

RQ-2

Ποια χαρακτηριστικά του Pull Request, της τελικής κατάστασης του αναλυόμενου κώδικα και του project -όπως η κατηγορία του PR, το μέγεθος του αναλυόμενου

κώδικα, η πολυπλοκότητα, η γνωστική πολυπλοκότητα και η διπλοτυπία- σχετίζονται με αυξημένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας;

RQ-3

Σε ποιο βαθμό οι διαφορές μεταξύ projects εξηγούν τη διακύμανση των προβλημάτων ποιότητας και ασφάλειας;

RQ-4

Μπορούν οι πολλές επιμέρους μετρικές ποιότητας και ασφάλειας να αναχθούν σε μικρότερο αριθμό υποκείμενων διαστάσεων;

RQ-5

Μεταβολές της ποιότητας από την αρχική στην τελική κατάσταση ενός Pull Request συνδέονται με αντίστοιχες μεταβολές της ασφάλειας;

RQ-6

Υπάρχουν κατηγορίες Pull Requests στις οποίες η σχέση ποιότητας-ασφάλειας διαφοροποιείται;

Μεθοδολογική επισήμανση: Τα παραπάνω ερευνητικά ερωτήματα αποτυπώνουν τον αρχικό σχεδιασμό της έρευνας. Κατά τον προκαταρκτικό έλεγχο των δεδομένων διαπιστώθηκε ότι οι μετρικές των καταστάσεων `pr_base` και `pr_closed` παρέμεναν αμετάβλητες στη μεγάλη πλειονότητα των Pull Requests. Η εξαιρετικά περιορισμένη αυτή within-PR μεταβλητότητα δεν επέτρεψε την αξιόπιστη διερεύνηση του RQ-5 μέσω των αρχικά προβλεπόμενων δεικτών μεταβολής. Το ερώτημα διατηρείται ως μέρος του αρχικού ερευνητικού πλαισίου, αλλά τα αποτελέσματα της παρούσας εργασίας δεν ερμηνεύονται ως επιβεβαίωση ή απόρριψη της ύπαρξης σχέσης μεταξύ των μεταβολών ποιότητας και ασφάλειας.

1.11 Ερευνητικές υποθέσεις

Με βάση το θεωρητικό υπόβαθρο, το ερευνητικό πρόβλημα όπως παρουσιάστηκε και τους επιμέρους στόχους της παρούσας εργασίας, διατυπώνονται οι παρακάτω ερευνητικές υποθέσεις:

H1 – Σχέση μεταβολών ποιότητας και ασφάλειας

Οι μεταβολές στις μετρικές ποιότητας του λογισμικού σε επίπεδο PR σχετίζονται στατιστικά σημαντικά με τις αντίστοιχες μεταβολές στις μετρικές ασφάλειας.

H2 – Επίδραση χαρακτηριστικών του Pull Request

Χαρακτηριστικά της τελικής κατάστασης του αναλυόμενου κώδικα, όπως το μέγεθος και η πολυπλοκότητα, καθώς και χαρακτηριστικά του Pull Request, όπως η κατηγορία του, σχετίζονται με την πυκνότητα προβλημάτων ποιότητας και ασφάλειας.

H3 – Διαφοροποίηση μεταξύ έργων (projects)

Οι πυκνότητες των προβλημάτων ποιότητας και ασφάλειας παρουσιάζουν ουσιαστική μεταβλητότητα μεταξύ διαφορετικών projects, με σημαντικό μέρος της συνολικής διακύμανσής τους να εντοπίζεται στο επίπεδο του project ακόμη και μετά τον έλεγχο των παρατηρούμενων χαρακτηριστικών της τελικής κατάστασης του αναλυόμενου κώδικα και της κατηγορίας του Pull Request.

H4 – Ύπαρξη λανθανουσών δομών

Οι επιμέρους μετρικές ποιότητας και ασφάλειας δεν λειτουργούν αποκλειστικά ως ανεξάρτητες μεταβλητές, αλλά παρουσιάζουν συσχετίσεις που μπορούν να αποδοθούν σε μικρότερο αριθμό κοινών υποκείμενων ή λανθανουσών διαστάσεων.

H5 – Υπεροχή ιεραρχικής μοντελοποίησης

Τα στατιστικά μοντέλα που λαμβάνουν υπόψη την ομαδοποίηση των PRs ανά έργο (project) αναμένεται να περιγράφουν καλύτερα τη δομή των δεδομένων σε σχέση με απλούστερα μοντέλα που αντιμετωπίζουν όλες τις παρατηρήσεις ως πλήρως ανεξάρτητες.

H6 – Ύπαρξη συνεργατικών και ανταγωνιστικών μεταβολών

Οι αλλαγές στον πηγαίο κώδικα δεν αναμένεται να επηρεάζουν πάντα την ποιότητα καθώς και την ασφάλεια προς την ίδια κατεύθυνση (θετικά/αρνητικά). Αναμένονται τόσο περιπτώσεις ταυτόχρονης βελτίωσης ή υποβάθμισης των δύο διαστάσεων όσο και περιπτώσεις αντιστάθμισης, όπου η βελτίωση της μίας συνοδεύεται από την υποβάθμιση της άλλης καθώς επίσης και το αντίστροφο.

Οι παραπάνω υποθέσεις αποτυπώνουν τις διαφορετικές οπτικές σύμφωνα με τις οποίες σχεδιάστηκε η εμπειρική διερεύνηση της σχέσης μεταξύ ποιότητας και ασφάλειας. Η δυνατότητα ελέγχου καθεμιάς εξαρτάται, όμως, από την πληροφορία και τη μεταβλητότητα που παρέχουν τα διαθέσιμα δεδομένα. Πιο ειδικά, οι H1 και H6 προϋποθέτουν επαρκείς μεταβολές των αντίστοιχων μετρικών μεταξύ των καταστάσεων pr_base και pr_closed. Ο προκαταρκτικός έλεγχος έδειξε ότι η προϋπόθεση αυτή δεν ικανοποιείται στον αναμενόμενο επαρκή βαθμό, καθώς η μεγάλη πλειονότητα των PRs παρουσίαζε μηδενική μεταβολή στις εξεταζόμενες μετρικές. Ως αποτέλεσμα, οι H1 και H6 δεν μπορούν να αξιολογηθούν αξιόπιστα με τα συγκεκριμένα δεδομένα και δεν θεωρούνται ούτε εμπειρικά υποστηριζόμενες καθώς και ούτε απορριφθείσες. Για τις λοιπές υποθέσεις (H2-H5), η αξιολόγησή τους πραγματοποιείται βάσει των αντίστοιχων στατιστικών και πολυμεταβλητών αναλύσεων που παρουσιάζονται στα επόμενα κεφάλαια της βιβλιογραφίας.

1.12 Συνεισφορά της εργασίας

Η παρούσα εργασία επιδιώκει να συνεισφέρει στη μελέτη της σχέσης επίδρασης και αλληλεπίδρασης μεταξύ των συνιστωσών της ποιότητας και

ασφάλειας του πηγαίου κώδικα του λογισμικού και του ίδιου του λογισμικού σε θεωρητικό, εμπειρικό, μεθοδολογικό και πρακτικό επίπεδο. Η συνεισφορά της βασίζεται στη συνδυασμένη εξέταση πολλαπλών quality και security metrics σε πραγματικά Pull Requests, στην αναγνώριση της σημασίας του project-specific context και στη διερεύνηση της πολυδιάστατης δομής των σχετικών χαρακτηριστικών.

1.12.1 Θεωρητική συνεισφορά

Η εργασία συμβάλλει στην αντιμετώπιση της ποιότητας και της ασφάλειας ως σχετιζόμενων αλλά διακριτών διαστάσεων του λογισμικού. Αντί να προϋποθέτει μία ενιαία θετική ή αρνητική σχέση μεταξύ τους, εξετάζει κατά πόσο διαφορετικές quality και security metrics συνδέονται μεταξύ τους, επηρεάζονται διαφορετικά από χαρακτηριστικά του κώδικα και οργανώνονται σε ευρύτερες υποκείμενες διαστάσεις. Με τον τρόπο αυτό υποστηρίζεται μία περισσότερο πολυδιάστατη και context-aware θεώρηση της σχέσης ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού.

1.12.2 Εμπειρική συνεισφορά

Η εμπειρική συνεισφορά της εργασίας βασίζεται στην ανάλυση πραγματικών δεδομένων Pull Requests από πολλαπλά και διαφορετικά έργα λογισμικού. Η εξέταση πολλαπλών μετρικών επιτρέπει την αποτύπωση των σχέσεων μεταξύ χαρακτηριστικών του κώδικα και δεικτών ποιότητας και ασφάλειας, ενώ η παρουσία μεγάλου αριθμού διαφορετικών repositories καθιστά δυνατή την εκτίμηση της project-level ετερογένειας. Παράλληλα, η διαπίστωση ότι οι pr_base και pr_closed μετρικές παρουσίαζαν εξαιρετικά περιορισμένη μεταβλητότητα παρέχει εμπειρική τεκμηρίωση για τα όρια της συγκεκριμένης snapshot-based προσέγγισης ως μέσου μελέτης των μεταβολών εσωτερικά του PR.

1.12.3 Μεθοδολογική συνεισφορά

Σε μεθοδολογικό επίπεδο, η εργασία συνδυάζει διαφορετικές στατιστικές και συμπληρωματικές τεχνικές και προσεγγίσεις, με στόχο την πληρέστερη διερεύνηση της σχέσης ποιότητας και ασφάλειας. Τα μοντέλα Ordinary Least Squares (OLS) χρησιμοποιούνται ως αρχικό σημείο αναφοράς, ενώ τα Linear Mixed-Effects Models επιτρέπουν τη μοντελοποίηση της ιεραρχικής δομής των δεδομένων και της διαφοροποίησης μεταξύ projects. Συμπληρωματικά εφαρμόζονται robustness και sensitivity analyses ενώ η Principal Component Analysis και Factor Analysis αξιοποιούνται για τη διερεύνηση της πολυδιάστατης και λανθάνουσας δομής των μετρικών λογισμικού.

Ιδιαίτερη μεθοδολογική σημασία έχει η προσαρμογή της αναλυτικής στρατηγικής στα πραγματικά χαρακτηριστικά των δεδομένων. Η αρχικά προβλεπόμενη και σχεδιασμένη delta προσέγγιση δεν διατηρήθηκε όταν

διαπιστώθηκε ότι δεν παρείχε επαρκή μεταβλητότητα και αντικαταστάθηκε από cross-sectional σχεδιασμό μίας τελικής παρατήρησης ανά Pull Request. Η επιλογή αυτή περιορίζει την εξαγωγή μη υποστηριζόμενων συμπερασμάτων και συνδέει τη στατιστική μεθοδολογία με την πραγματική πληροφορία που περιέχεται στο τελικά διαθέσιμο dataset.

1.12.4 Πρακτική συνεισφορά

Τα αποτελέσματα της εργασίας μπορούν να έχουν πρακτική αξία για τις σύγχρονες διαδικασίες ανάπτυξης λογισμικού. Ειδικότερα, μπορούν να υποστηρίξουν περισσότερο context-aware και πολυμετρικές διαδικασίες αξιολόγησης Pull Requests, στις οποίες η προτεραιοποίηση του ελέγχου δεν βασίζεται αποκλειστικά σε μία μεμονωμένη μετρική ή στη δηλωμένη κατηγορία του Pull Request, αλλά στη συνδυαστική εξέταση των quality και security related indicators, χαρακτηριστικών της τελικής κατάστασης του αναλυόμενου κώδικα και του ιδιαίτερου πλαισίου του αντίστοιχου project.

Οι πληροφορίες αυτές μπορούν να λειτουργήσουν ως συμπληρωματικά signals για την κατανομή της προσοχής κατά το code review και για την παραμετροποίηση αυτοματοποιημένων quality και security checks σε περιβάλλοντα CI/CD και DevSecOps. Η χρήση τους δεν συνιστά άμεση εκτίμηση της πιθανότητας ένα Pull Request να εισαγάγει συγκεκριμένο πρόβλημα, vulnerability ή περιστατικό ασφάλειας. Η παρούσα εργασία δεν προτείνει και δεν επικυρώνει μοντέλο πρόβλεψης πραγματικού security risk, αλλά παρέχει εμπειρικά ευρήματα που μπορούν να αποτελέσουν βάση για την ανάπτυξη μελλοντικών risk-informed, project-aware εργαλείων, εφόσον προηγηθεί ανεξάρτητη κατασκευή και επικύρωσή τους.

1.13 Δομή της εργασίας

Η παρούσα εργασία οργανώνεται σε διαδοχικά κεφάλαια, τα οποία καλύπτουν το θεωρητικό υπόβαθρο για την πλήρη κατανόηση του επιστημονικού πεδίου που αφορά καθώς επίσης και το θεωρητικό, μεθοδολογικό και εμπειρικό μέρος της έρευνας. Στο πρώτο κεφάλαιο παρουσιάζεται το γενικό πλαίσιο της έρευνας, η σημασία της ποιότητας και της ασφάλειας του λογισμικού, η πιθανή αλληλεπίδραση που δημιουργείται μεταξύ τους, το ερευνητικό πρόβλημα και κενό, καθώς επίσης και ο σκοπός, οι στόχοι, τα ερευνητικά ερωτήματα, οι υποθέσεις και η συνεισφορά της εργασίας στον επιστημονικό κλάδο που απευθύνεται. Στο δεύτερο κεφάλαιο αναπτύσσονται οι βασικές έννοιες της σύγχρονης ανάπτυξης λογισμικού, με έμφαση στη διαχείριση των εκδόσεων μέσω του εργαλείου του Git, στα αποθετήρια κώδικα, στα Pull Requests, στις μεταβολές του πηγαίου κώδικα, στη στατιστική ανάλυση και στις πρακτικές CI/CD, DevOps και DevSecOps. Το τρίτο καθώς και τέταρτο κεφάλαιο παρουσιάζουν αντίστοιχα το θεωρητικό υπόβαθρο της

ποιότητας και της ασφάλειας λογισμικού, ενώ στο πέμπτο κεφάλαιο εξετάζεται η μεταξύ τους αλληλεπίδραση, παρουσιάζεται η σχετική βιβλιογραφία και προσδιορίζεται η θέση της παρούσας έρευνας σε σχέση με υπάρχουσες και προηγούμενες μελέτες.

Στο έκτο κεφάλαιο περιγράφεται ο ερευνητικός σχεδιασμός, η συλλογή και προετοιμασία των δεδομένων, η επιλογή και το φιλτράρισμα των Pull Requests, η δημιουργία των καταστάσεων `pr_base` και `pr_closed`, η εξαγωγή των μετρικών και η κατασκευή του τελικού αναλυτικού συνόλου δεδομένων. Στο έβδομο κεφάλαιο παρουσιάζεται η στατιστική μεθοδολογία που η εργασία ακολουθεί, η οποία περιλαμβάνει περιγραφική και διερευνητική ανάλυση, μοντέλα Ordinary Least Squares και Linear Mixed-Effects Models, καθώς και Principal Component Analysis και Factor Analysis. Στο όγδοο κεφάλαιο παρουσιάζονται και ενοποιούνται τα αποτελέσματα των παραπάνω αναλύσεων, όπως αυτές περιεγράφηκαν.

Στο ένατο κεφάλαιο πραγματοποιείται η συζήτηση και ερμηνεία των αποτελεσμάτων σε σχέση με τα ερευνητικά ερωτήματα (όπως αυτά έχουν τεθεί στο 1.10 *Ερευνητικά ερωτήματα*), τις υποθέσεις (όπως αυτές έχουν δοθεί στο 1.11 *Ερευνητικές υποθέσεις*) και την υπάρχουσα βιβλιογραφία, ενώ εξετάζονται επίσης οι πρακτικές επιπτώσεις, οι περιορισμοί και οι απειλές εγκυρότητας της έρευνας. Τέλος, στο δέκατο κεφάλαιο συνοψίζονται τα βασικά συμπεράσματα, παρουσιάζεται η συνολική συνεισφορά της εργασίας και διατυπώνονται προτάσεις για μελλοντική έρευνα δράττοντας του θέματος της παρούσας. Συμπληρωματικό τεχνικό, μαθηματικό και στατιστικό υλικό, καθώς και αναλυτικά αποτελέσματα και πληροφορίες αναπαραγωγής της έρευνας, παρατίθενται στα παραρτήματα.

Σύγχρονη ανάπτυξη λογισμικού και Pull Request

2.1 Συστήματα ελέγχου εκδόσεων και Git

Η διαδικασία της ανάπτυξης λογισμικού αποτελεί μία διαρκώς εξελισσόμενη διαδικασία, κατά την οποία ο πηγαίος κώδικας ενός συστήματος τροποποιείται επανειλημμένα. Υλοποιούνται καινούριες λειτουργίες, μεταβάλλονται υπάρχοντα χαρακτηριστικά, διορθώνονται σφάλματα και τμήματα του κώδικα αναδιαρθρώνονται ή/και αφαιρούνται, καθιστώντας έτσι την ανάπτυξή του μία διαρκή και δυναμική διαδικασία. Καθώς ένα σύστημα αυξάνεται σε μέγεθος και περισσότεροι προγραμματιστές συμμετέχουν στην ανάπτυξή του, προκύπτει η ανάγκη συστημικής, γραμμικής συνήθως, καταγραφής και διαχείρισης των μεταβολών που ο πηγαίος κώδικας του λογισμικού δέχεται. Η απλή διατήρηση διαφορετικών αντιγράφων δεν επαρκεί, καθώς δυσχεραίνει τον προσδιορισμό του ατόμου (ή συστήματος) που μετέβαλε τον πηγαίο κώδικα σε ένα συγκεκριμένο σημείο του, πότε πραγματοποιήθηκε η αλλαγή αυτή, ποια ήταν η προηγούμενη κατάσταση του συστήματος και με ποιες άλλες αλλαγές συνδέεται. Στο σημείο αυτό αξίζει να σημειωθεί ότι πριν την δημιουργία των συστημάτων που περιγράφονται

παρακάτω, μία ομάδα προγραμματιστών συνήθιζε να έχει τον έλεγχο των εκδόσεων της βάσης του κώδικα με την χρήση πολλαπλών αντιγράφων του ίδιου του κώδικα. Η συγκεκριμένη μέθοδος αντικαταστάθηκε από τα Συστήματα Ελέγχου Εκδόσεων (Version Control Systems – VCS), αφού σύντομα αποδείχθηκε ελλιπής, πολύπλοκη στην διαχείρισή της και επαχθής σε ανάγκες πόρων αφού η διατήρηση πολλαπλών εκδόσεων του ίδιου κώδικα ήταν ουσιαστικά η διατήρηση πολλαπλών αντιγράφων του. Τα συστήματα ελέγχου εκδόσεων οργανώνουν το ιστορικό ενός συνόλου αρχείων, ώστε οι παλαιότερες καταστάσεις τους να μπορούν να ανακτηθούν, να συγκριθούν και, όταν απαιτείται, να επαναφερθούν. Επομένως, δεν λειτουργούν απλώς ως αποθήκες αντιγράφων, αλλά ως μηχανισμοί ιχνηλάτησης της εξέλιξης του έργου (Chacon & Straub, 2014).

Η σημασία των συστημάτων ελέγχου εκδόσεων δεν περιορίζεται στην αποθήκευση παλαιότερων αντιγράφων του πηγαίου κώδικα. Το ιστορικό των μεταβολών που δημιουργείται αποτελεί μία οργανωμένη αναπαράσταση της συνεχούς εξέλιξης του λογισμικού. Οι μεταβολές μπορούν να αποδοθούν σε συγκεκριμένα χρονικά σημεία, να συγκριθούν μεταξύ τους και να συσχετιστούν με προηγούμενες ή ακόμα και επόμενες καταστάσεις του πηγαίου κώδικα που αντιπροσωπεύει το σύστημα. Με τον τρόπο αυτό, το version control υποστηρίζει τόσο τη συνεργασία μεταξύ προγραμματιστών της ίδιας ή και διαφορετικής ομάδας όσο και την ιχνηλασιμότητα -traceability- της διαδικασίας ανάπτυξης και δυναμικής εξέλιξης του λογισμικού. Σε θεωρητικές διατυπώσεις, ως βασικός σκοπός ενός VCS περιγράφεται η διατήρηση του ιστορικού ενός συνόλου αρχείων, έτσι ώστε να είναι δυνατή η σύγκριση και διαχείριση ιστορικών εκδόσεων, η επιστροφή σε προηγούμενη κατάσταση και η εξέταση των αλλαγών μεταξύ διαφορετικών καταστάσεων/εκδόσεων. Η ιδιότητα αυτή είναι ιδιαίτερα σημαντική και για την εμπειρική έρευνα λογισμικού, καθώς το καταγεγραμμένο ιστορικό παρέχει δεδομένα μέσω των οποίων μπορεί να ανακατασκευαστεί και να μελετηθεί εις βάθος η εξέλιξη ενός έργου.

Τα VCS διακρίνονται σε τρεις κατηγορίες: τοπικά, κεντριοποιημένα και κατανεμημένα. Στα πρώτα, το ιστορικό διατηρούνταν τοπικά στον υπολογιστή ενός χρήστη, του χρήστη ο οποίος προέβαινε στις τροποποιήσεις. Η ανάγκη συνεργασίας πολλών προγραμματιστών οδήγησε στην ανάπτυξη των Centralized VCS (CVCS), όπως τα CVS και Subversion. Στα CVCS υπάρχει ένας κεντρικός εξυπηρετητής που περιέχει το ιστορικό του έργου και οι χρήστες συνδέονται σε αυτόν για να ανακτήσουν ή να καταχωρίσουν αλλαγές. Το μοντέλο αυτό διευκολύνει τον κεντρικό έλεγχο αι τη διαχείριση δικαιωμάτων ενώ παράλληλα παρουσιάζει ισχυρή εξάρτηση από τον έναν κεντρικό εξυπηρετητή. Η μη διαθεσιμότητα ή η απώλειά του μπορεί να περιορίσει τη δυνατότητα συνεργασίας ή, ελλείψει κατάλληλων αντιγράφων ασφαλείας, να θέσει σε κίνδυνο το ιστορικό του έργου ως την στιγμή εκείνη.

Τα Distributed (Κατανεμημένα) VCS (DVCS) ακολουθούν μία εντελώς διαφορετική προσέγγιση από αυτές που ήδη περιεγράφηκαν. Σε ένα DVCS, κάθε προγραμματιστής μπορεί να διαθέτει ένα πλήρες τοπικό αντίγραφο του συνολικού repository και του ιστορικού του και όχι την τελευταία έκδοση του λογισμικού. Αυτό σημαίνει ότι βασικές λειτουργίες όπως η καταγραφή νέων εκδόσεων, η εξέταση του ιστορικού και η σύγκριση διαφορετικών καταστάσεων, μπορούν να

πραγματοποιούνται τοπικά χωρίς συνεχή επικοινωνία με έναν κεντρικό εξυπηρετητή. Πρακτικά, ενώ οι ομάδες χρησιμοποιούν έναν κοινό απομακρυσμένο εξυπηρετητή ως τελικό και αρχικό σημείο συνεργασίας, αυτός τεχνικά δεν αποτελεί τη μοναδική πλήρη πηγή του ιστορικού, όπως θα συνέβαινε σε ένα CVCS. Αυτή η κατανομημένη αρχιτεκτονική επιτρέπει περισσότερο ευέλικτες ροές εργασίας και διευκολύνει την παράλληλη ανάπτυξη διαφορετικών μεταβολών.

Λόγω της δημοφιλίας και της πρακτικότητας των DVCS, η κοινότητα έχει στρέψει την προσοχή για ανάπτυξη και χρήση σε εργαλεία που υλοποιούν την λογική αυτή. Το Git αναπτύχθηκε το 2005 για τις ανάγκες του Linux kernel, μετά τη διακοπή της χρήσης του BitKeeper. Ο σχεδιασμός του έδωσε έμφαση στην ταχύτητα, στην κατανομημένη λειτουργία, στην υποστήριξη μη γραμμικών ροών ανάπτυξης και στη διαχείριση έργων μεγάλης κλίμακας (Chacon & Straub, 2014). Τα προαναφερθέντα χαρακτηριστικά επηρέασαν σημαντικά την αρχιτεκτονική του Git και ιδιαίτερα τον τρόπο με τον οποίο αναπαριστά το ιστορικό του λογισμικού και υποστηρίζει την παράλληλη ανάπτυξη και παρέχει σε κάθε προγραμματιστή μία πλήρη ιδιωτική αναπαράσταση του repository και του ιστορικού του, όπως η βιβλιογραφία αναφέρει.

Στο μοντέλο δεδομένων του Git, κάθε commit παραπέμπει σε στιγμιότυπο της κατάστασης του έργου. Όταν ένα αρχείο παραμένει αμετάβλητο, το Git μπορεί να αναφερθεί στο ήδη αποθηκευμένο περιεχόμενό του αντί να το αποθηκεύσει εκ νέου, οπότε το ιστορικό σχηματίζεται ως ακολουθία συνδεδεμένων snapshots (Chacon & Straub, 2014).

Η προσέγγιση αυτή είναι ιδιαίτερα σημαντική για την πλήρη κατανόηση του Git καθώς επίσης και για την παρούσα εργασία. Καθώς διαφορετικές καταστάσεις του repository μπορούν να προσδιοριστούν και να ανακατασκευαστούν από το ιστορικό του, είναι δυνατό να συγκριθεί ο πηγαίος κώδικας σε δύο διαφορετικά χρονικά σημεία. Εάν, για παράδειγμα, μία τροποποίηση μεταφέρει το έργο από μία κατάσταση (S_{before}) σε μία κατάσταση (S_{after}), τότε μπορεί να θεωρηθεί ότι η μεταβολή του πηγαίου κώδικα αποτελεί τη διαφορά μεταξύ των δύο καταστάσεων. Η ίδια λογική χρησιμοποιείται στην εμπειρική ανάλυση της παρούσας εργασίας, όπου η κατάσταση του κώδικα πριν από μία τροποποίηση συγκρίνεται με την κατάσταση μετά την ολοκλήρωσή της, ώστε να εξεταστούν οι αντίστοιχες μεταβολές στις μετρικές ποιότητας και ασφάλειας. Η δυνατότητα ακριβούς αναφοράς σε διαφορετικές καταστάσεις του ιστορικού αποτελεί επομένως βασική τεχνική προϋπόθεση για την πραγματοποίηση ορθής ανάλυσης σε επίπεδο Pull Request.

Σημαντικό χαρακτηριστικό του Git, επίσης, αποτελεί η δυνατότητα μεγάλου μέρους λειτουργιών/ενεργειών να μπορεί να πραγματοποιηθεί τοπικά. Επειδή ένας κλώνος του πηγαίου κώδικα (clone) περιλαμβάνει το πλήρες ιστορικό του repository, λειτουργίες όπως η εξέταση προηγούμενων εκδόσεων ή ο υπολογισμός διαφορών μεταξύ καταστάσεων ή ο υπολογισμός διαφορών μεταξύ καταστάσεων του πηγαίου κώδικα δεν απαιτούν πρόσβαση σε κάποιον απομακρυσμένο εξυπηρετητή (server). Η συνεργασία πραγματοποιείται μέσω της ανταλλαγής δεδομένων μεταξύ repositories, χρησιμοποιώντας λειτουργίες όπως η ανάκτηση απομακρυσμένων αλλαγών και η δημοσίευση τοπικών αλλαγών. Αυτό επιτρέπει στον εκάστοτε ενδιαφερόμενο να πραγματοποιεί την εργασία του τοπικά και στη συνέχεια να συγχρονίζει το αντίγραφό του με ένα ή περισσότερα απομακρυσμένα αποθετήρια.

Η συνήθης ροή εργασίας του Git διακρίνει τρεις περιοχές: το working tree, όπου τροποποιούνται τα αρχεία, το staging area, όπου επιλέγεται το περιεχόμενο του επόμενου commit, και το τοπικό repository, όπου αποθηκεύεται οριστικά το commit (Chacon & Straub, 2014). Όταν πραγματοποιούνται αλλαγές, τα αρχεία βρίσκονται σε τροποποιημένη κατάσταση. Ο προγραμματιστής μπορεί να επιλέγει ποιες από αυτές τις μεταβολές θα τοποθετηθούν στο λεγόμενο staging area, δηλαδή ποιες θα συμπεριληφθούν στην επόμενη καταγεγραμμένη κατάσταση. Τελικά, με την καταχώριση της αλλαγής δημιουργείται μία μόνιμη εγγραφή στο τοπικό αποθετήριο. Η ενδιάμεση περιοχή του staging κρίνεται σημαντική, επειδή επιτρέπει στον developer να διαχωρίζει τις αλλαγές που υπάρχουν προσωρινά στον χώρο εργασίας από εκείνες που επιθυμεί να αποτελέσουν μία συγκεκριμένη λογική μονάδα του ιστορικού.

Το Git αναγνωρίζει τα αποθηκευμένα αντικείμενα από κατακερματισμούς που παράγονται από το περιεχόμενό τους. Κατά συνέπεια, η αλλαγή των δεδομένων μεταβάλλει και το αναγνωριστικό του αντικειμένου, παρέχοντας μηχανισμό ελέγχου της ακεραιότητας του ιστορικού (Chacon & Straub, 2014). Τα βασικά αντικείμενα που χρησιμοποιούνται εσωτερικά περιλαμβάνουν αντικείμενα που αναπαριστούν περιεχόμενο αρχείων, δομές καταλόγων και καταγεγραμμένες καταστάσεις του έργου. Οι τελευταίες συνδέονται με προηγούμενες καταστάσεις, με αποτέλεσμα το ιστορικό να σχηματίζει μία κατευθυνόμενη μη κυκλική δομή και όχι απλώς μία ανεξάρτητη συλλογή αντιγράφων αρχείων. Η συγκεκριμένη αρχιτεκτονική επιτρέπει στο Git να αναπαριστά τόσο γραμμικές ακολουθίες αλλαγών όσο και μη γραμμική εξέλιξη, όπου διαφορετικές γραμμές ανάπτυξης μπορούν να αποκλίνουν και αργότερα να ενωθούν εκ νέου.

Συνοψίζοντας τα παραπάνω, τα συστήματα ελέγχου εκδόσεων αποτελούν βασική υποδομή της σύγχρονης ανάπτυξης λογισμικού, καθώς μετατρέπουν την εξέλιξη του πηγαίου κώδικα σε μία συστηματικά καταγεγραμμένη και ανακτήσιμη ακολουθία καταστάσεων και αλλαγών. Το Git, μέσω της κατανεμημένης αρχιτεκτονικής που διαθέτει καθώς επίσης του snapshot-oriented μοντέλου, της τοπικής διαχείρισης του ιστορικού και της αποτελεσματικής υποστήριξης παράλληλων γραμμών ανάπτυξης, παρέχει το τεχνικό υπόβαθρο πάνω στο οποίο βασίζονται πολλές σύγχρονες συνεργατικές ροές ανάπτυξης.

2.1.1 Αποθετήριο κώδικα (Repository)

Στο πλαίσιο του Git, ένα αποθετήριο κώδικα (repository) αποτελεί τη βασική δομή μέσα στην οποία αποθηκεύεται και οργανώνεται ένα έργο λογισμικού συμπεριλαμβανομένου του ιστορικού μεταβολών του πηγαίου κώδικά του ή/και διάφορων αρχείων του έργου. Δεν πρόκειται απλώς για έναν φάκελο που περιέχει την τρέχουσα έκδοση του πηγαίου κώδικα, αλλά για μία δομή δεδομένων που διατηρεί πληροφορίες σχετικά με προηγούμενες καταστάσεις του έργου, τις καταγεγραμμένες τροποποιήσεις του, τις διαφορετικές γραμμές ανάπτυξης του πηγαίου κώδικα καθώς και τα μεταδεδομένα (δεδομένα που χαρακτηρίζουν τα κύρια δεδομένα ή πληροφορίες) που απαιτούνται για την ανακατασκευή του ιστορικού

εξέλιξής του. Σε ένα τυπικό Git repository διακρίνονται το working tree (δέντρο εργασίας), δηλαδή τα αρχεία πάνω στα οποία εργάζεται ένας προγραμματιστής και ο φάκελος/κατάλογος/directory με ονομασία .git, στον οποίο αποθηκεύεται η εσωτερική βάση δεδομένων του Git, με τα αντικείμενα, τις αναφορές και το ιστορικό του έργου. Ένα repository μπορεί είτε να δημιουργηθεί τοπικά με την χρήση της αντίστοιχης εντολής (*git init*) είτε να προϋπάρχει και να προκύψει ως αντίγραφο ενός υπάρχοντος repository μέσω της εντολής *git clone*.

Λόγω της κατανεμημένης αρχιτεκτονικής που ακολουθεί, καθίσταται δυνατή η διάκριση μεταξύ local (τοπικού) και remote (απομακρυσμένου) repository. Το local repository βρίσκεται στο περιβάλλον που ο προγραμματιστής διαθέτει, για παράδειγμα κάποιο λειτουργικό σύστημα που χρησιμοποιεί, και επιτρέπει την τοπική καταγραφή και εξέταση των αλλαγών, ενώ ένα remote repository αποτελεί ένα άλλο repository με το οποίο μπορούν να ανταλλάσσονται δεδομένα. Είναι σημαντικό να σημειωθεί ότι διάφορες παρεχόμενες υπηρεσίες -διαδικτυακές πλατφόρμες-, μεταξύ των οποίων το GitHub, το GitLab ή το Bitbucket δεν αποτελούν μέρος του εργαλείου του Git αλλά χρησιμοποιούνται συχνά ως κοινό σημείο φιλοξενίας και συνεργασίας. Επίσης, κρίνεται αναγκαία η αποσαφήνιση της λειτουργίας του Pull Request, της αξιολόγησης κώδικα (code review), issue tracking και automated workflows πάνω στην υποδομή του VCS, ως μέρος και έννοια αυτών και αντίστοιχων πλατφορμών και όχι ως έμφυτες λειτουργίες του εκάστοτε VCS.

Στην παρούσα έρευνα, το repository αποκτά ιδιαίτερη σημασία επειδή αποτελεί την πηγή μέσω της οποίας μπορούν να ανακατασκευαστούν διαφορετικές καταστάσεις του πηγαίου κώδικα και να μελετηθεί η εξέλιξη ενός project λογισμικού. Η δυνατότητα αυτή επιτρέπει τη σύγκριση του κώδικα πριν και μετά από μία συγκεκριμένη αλλαγή και, κατά συνέπεια, τη μέτρηση των αντίστοιχων μεταβολών στις μετρικές ποιότητας και ασφάλειας. Παρότι το repository παρέχει το ευρύτερο πλαίσιο της ανάλυσης, στην παρούσα εργασία η κύρια μονάδα ανάλυσης είναι το Pull Request, ενώ το repository λαμβάνεται ως το project-level πλαίσιο στο οποίο ανήκουν πολλαπλά Pull Request.

2.1.2 Υποβολή/Καταγραφή τροποποιήσεων (Commit)

Ένα commit καταγράφει μία συγκεκριμένη κατάσταση του έργου και περιλαμβάνει αναφορά στο snapshot που είχε προετοιμαστεί στο staging area, μαζί με μεταδεδομένα όπως ο δημιουργός, το μήνυμα και οι γονικές καταχωρίσεις του (Chacon & Straub, 2014). Κάθε commit επιπλέον διαθέτει ένα μοναδικό αναγνωριστικό (identifier – id) το οποίο προκύπτει από μηχανισμό κατακερματισμού και χρησιμοποιείται για τη σαφή αναφορά σε αυτό μέσα στο ιστορικό των μεταβολών που κρατείται. Έτσι, τα commits επιτρέπουν την ανακατασκευή προηγούμενων καταστάσεων του λογισμικού και τη διερεύνηση του χρονικού και τροπικού προσδιορισμού καθώς και του ατόμου το οποίο πρόβηκε στις τροποποιήσεις και πραγματοποίησε μία συγκεκριμένη μεταβολή.

Τα commits συνδέονται μεταξύ τους μέσω αναφορών στα προηγούμενα commits -ακολουθώντας την δομή δεδομένων λίστα και τον τρόπο με τον οποίο αυτή

υλοποιείται-, σχηματίζοντας το ιστορικό εξέλιξης του repository. Ένα τυπικό commit αναφέρεται σε ένα προηγούμενο commit ως parent, ενώ ένα merge commit (commit συνένωσης δύο διακλαδώσεων/αντιγράφων του κώδικα) μπορεί να διαθέτει περισσότερους από έναν γονείς, επειδή προκύπτει από την ένωση διαφορετικών γραμμών ανάπτυξης. Η σχέση αυτή επιτρέπει στο Git να αναπαριστά το ιστορικό όχι απλώς ως μία σειρά ανεξάρτητων αντιγράφων, αλλά ως μία συνδεδεμένη δομή καταστάσεων. Παράλληλα, τα commits είναι σκόπιμο να αντιπροσωπεύουν όσο το δυνατόν περισσότερο συνεκτικές και λογικά ολοκληρωμένες αλλαγές, ώστε το ιστορικό να παραμένει κατανοητό και να διευκολύνει διαδικασίες όπως η αναθεώρηση κώδικα, η ανίχνευση σφαλμάτων και η επαναφορά προηγούμενων αλλαγών. Κρίνεται σκόπιμη η μη ανάλυση των καλών πρακτικών ενός commit στην συγκεκριμένη περιγραφή της έρευνας λόγω της μη συμβολής στην αρτιότερη κατανόηση του θεωρητικού υποβάθρου που επιδιώκει.

Ο όρος αυτός έχει ιδιαίτερη σημασία επειδή αποτελεί τα επιμέρους βήματα μέσω των οποίων μεταβάλλεται ο πηγαίος κώδικας που περιλαμβάνεται σε ένα PR. Ένα PR μπορεί να περιλαμβάνει ένα ή περισσότερα commits, τα οποία συνολικά μεταφέρουν το repository από μία αρχική κατάσταση σε μία νέα. Η εμπειρική, ωστόσο, ανάλυση δεν αντιμετωπίζει κάθε commit ως ανεξάρτητη μονάδα παρατήρησης, αλλά εξετάζει τη συνολική μεταβολή που αντιστοιχεί στο PR, συγκρίνοντας την κατάσταση του πηγαίου κώδικα πριν και μετά την εφαρμογή του συνόλου των σχετικών τροποποιήσεων.

2.1.3 Διακλάδωση αντιγράφου πηγαίου κώδικα (Branch)

Στο Git, το branch δεν αποτελεί πλήρες αντίγραφο του έργου, αλλά μία ελαφριά και μετακινούμενη αναφορά προς commit. Με κάθε νέο commit στη συγκεκριμένη γραμμή ανάπτυξης, η αναφορά ενημερώνεται ώστε να δείχνει στη νεότερη κατάσταση (Chacon & Straub, 2014). Με τον συγκεκριμένο τρόπο, διαφορετικές γραμμές ανάπτυξης μπορούν να συνυπάρχουν μέσα στο ίδιο repository, καθιστώντας δυνατή την παράλληλη εργασία των προγραμματιστών σε νέες λειτουργίες, διορθώσεις σφαλμάτων, refactoring (αναδιάρθρωση υπάρχοντος κώδικα) ή άλλες τροποποιήσεις. Συνήθως υπάρχει ένα κύριο branch -είναι ευρύτερα γνωστό με την ονομασία main ή παλαιότερα master-, από το οποίο μπορούν να δημιουργούνται νέα branches για συγκεκριμένες εργασίες.

Η χρήση branches αποτελεί βασικό στοιχείο της συνεργατικής διαδικασίας ανάπτυξης λογισμικού, καθώς προσωρινά απομονώνει τις αλλαγές έως ότου αυτές ολοκληρωθούν πλήρως και στην συνέχεια ελεγχθούν. Για παράδειγμα, ένας προγραμματιστής μπορεί να δημιουργήσει ένα νέο branch από το κύριο, να πραγματοποιήσει μία σειρά commits και να εξελίξει ανεξάρτητα μία συγκεκριμένη λειτουργικότητα χωρίς να επηρεάζει την κύρια λειτουργία του συστήματος. Άλλοι προγραμματιστές έχουν την δυνατότητα, παράλληλα, να εργάζονται σε διαφορετικά τοπικά ή απομακρυσμένα branches χωρίς οι αλλαγές τους να τροποποιούν άμεσα την ίδια γραμμή ανάπτυξης του λογισμικού. Όταν η εργασία ολοκληρωθεί, οι μεταβολές μπορούν να εξεταστούν μέσω code review και στη συνέχεια να ενσωματωθούν στο

επιθυμητό branch. Η δυνατότητα αυτή υποστηρίζει τη μη γραμμική ανάπτυξη του λογισμικού και επιτρέπει την ταυτόχρονη ανάπτυξη και εξέλιξη πολλών χαρακτηριστικών ή διορθώσεων με περισσότερο ελεγχόμενο τρόπο.

Η έννοια του branch στην παρούσα εργασία είναι ιδιαίτερα σημαντική λόγω της λειτουργίας και λειτουργικότητας των Pull Requests. Ένα PR συνήθως δημιουργείται με σκοπό την ενσωμάτωση των αλλαγών ενός branch, του λεγόμενου ως head branch, σε ένα άλλο branch, το base branch (συνήθως είναι το κύριο branch του repository). Η διαφορά μεταξύ των δύο branches είναι αυτή που προσδιορίζει σε σημαντικό βαθμό το σύνολο των αλλαγών που προτείνεται να ενσωματωθούν. Η δυνατότητα ανακατασκευής της κατάστασης του κώδικα πριν και μετά από αυτές τις αλλαγές επιτρέπει στην παρούσα έρευνα να εξετάσει ενδελεχώς πώς ένα PR μεταβάλλει τις μετρικές ποιότητας και ασφάλειας του πηγαίου κώδικα και του λογισμικού.

2.1.4 Συγχώνευση κώδικα (merge) και μηχανισμός διαφορών (diff)

Η συγχώνευση ενσωματώνει την εξέλιξη ενός branch σε άλλο. Όταν δεν αρκεί μία απλή μετακίνηση της αναφοράς, το Git μπορεί να εκτελέσει three-way merge, συνδυάζοντας τις δύο τελικές καταστάσεις με τον κοινό πρόγονό τους (Chacon & Straub, 2014). Όταν οι αλλαγές μπορούν να συνδυαστούν χωρίς ασάφεια, η διαδικασία ολοκληρώνεται αυτομάτως. Εάν όμως διαφορετικές αλλαγές επηρεάζουν ασύμβατα τα ίδια τμήματα του κώδικα, προκύπτει σφάλμα συγχώνευσης (merge conflict), το οποίο πρέπει να επιλυθεί από τον developer πριν ολοκληρωθεί η ενσωμάτωση.

Το diff αποτελεί τον μηχανισμό μέσω του οποίου παρουσιάζονται οι διαφορές μεταξύ δύο καταστάσεων ενός έργου. Η εντολή σύγκρισης (*git diff*) μπορεί να χρησιμοποιηθεί για τη σύγκριση του working tree με το staging area ή του staging area με ένα commit ή δύο διαφορετικών commits ή branches μεταξύ τους. Το αποτέλεσμα συνήθως είναι οι γραμμές κώδικα που τροποποιήθηκαν, αφαιρέθηκαν ή προστέθηκαν, οργανωμένες σε επίπεδο αρχείου και επιμέρους τμήματα αλλαγών. Το diff δεν αποτελεί μία νέα κατάσταση για το repository, αλλά μία αναπαράσταση των διαφορών μεταξύ δύο καταστάσεων. Για αυτό τον λόγο αποτελεί βασικό εργαλείο τόσο για την καθημερινή ανάπτυξη όσο και για διαδικασίες αναθεώρησης κώδικα (code review) και Pull Requests, καθώς επιτρέπει στον reviewer να εξετάζει συγκεκριμένα τι μεταβάλλεται από μία προτεινόμενη αλλαγή.

Για την παρούσα βιβλιογραφία, οι παραπάνω έννοιες είναι ιδιαίτερα σημαντικές, καθώς ένα Pull Request περιγράφει μία οριοθετημένη διαδικασία τροποποίησης του πηγαίου κώδικα, ενώ το diff αποτυπώνει συγκεκριμένα το περιεχόμενο των προτεινόμενων στο Pull Request αλλαγών. Η δυνατότητα ανακατασκευής και σύγκρισης διαφορετικών καταστάσεων του repository αξιοποιήθηκε στον αρχικό ερευνητικό σχεδιασμό, ο οποίος προέβλεπε τη διερεύνηση διαφορών στις quality-security related μετρικές πριν και έπειτα της ολοκλήρωσης ενός Pull Request. Η εμπειρική αξιολόγηση των δεδομένων έδειξε ιδιαίτερα περιορισμένη μεταβλητότητα μεταξύ των αντίστοιχων snapshots. Για τον

λόγο αυτό, η σύγκρισή τους διατηρήθηκε ως σημαντικός διαγνωστικός έλεγχος της καταλληλότητας της αρχικής before/after προσέγγισης, ενώ η κύρια στατιστική ανάλυση βασίστηκε στην pr_closed κατάσταση κάθε Pull Request.

2.2 Αίτημα συγχώνευσης/έλξης κώδικα (Pull Request)

Το Pull Request αποτελεί αντικείμενο συνεργασίας των πλατφορμών φιλοξενίας Git και όχι εγγενή εντολή του ίδιου του Git. Συγκεντρώνει την προτεινόμενη αλλαγή, τη συζήτηση και την αξιολόγησή της πριν από την απόφαση συγχώνευσης (Gousios, Pinzger & van Deursen, 2014). Σε πλατφόρμες όπως το GitHub δημιουργεί έναν οργανωμένο χώρο μέσα στον οποίο οι συμμετέχοντες μπορούν να εξετάσουν τις αλλαγές πριν αυτές ενσωματωθούν στον κώδικα.

Η σημασία των Pull Requests προκύπτει κυρίως από τον διαχωρισμό μεταξύ της δημιουργίας μιας αλλαγής και της απόφασης αποδοχής της στο έργο. Σε ένα κατακευματισμένο περιβάλλον ανάπτυξης, διαφορετικοί developers έχουν τη δυνατότητα να δημιουργούν τροποποιήσεις ανεξάρτητα, χωρίς αυτές να εισάγονται απευθείας στην κύρια εξέλιξη του λογισμικού. Το Pull Request παρέχει έναν τυποποιημένο μηχανισμό μέσω του οποίου η αλλαγή παρουσιάζεται ως μία διακριτή πρόταση προς το έργο. Η προσέγγιση αυτή, βιβλιογραφικά, περιγράφεται ως προσέγγιση pull-based software development model, στο οποίο ο προγραμματιστής που πραγματοποιεί μία αλλαγή δεν χρειάζεται απαραίτητα να διαθέτει δικαίωμα άμεσης πρόσβασης τροποποίησης του κύριου repository. Αντίθετα, μπορεί να υποβάλει τις αλλαγές του προς αξιολόγηση και οι υπεύθυνοι του έργου διατηρούν τον έλεγχο σχετικά με το ποιες συνεισφορές θα γίνουν αποδεκτές. Με τον τρόπο αυτό, το μοντέλο υποστηρίζει τόσο τη συνεργασία με το εσωτερικό περιβάλλον της ομάδας όσο και τη συμμετοχή εξωτερικών contributors σε κατακευματισμένα και έργα ανοιχτού κώδικα.

Ένα Pull Request δεν αναπαριστά απαραίτητα μία μόνο τροποποιημένη γραμμή ή ένα μεμονωμένο commit. Συνήθως λειτουργεί ως λογική μονάδα αλλαγής, η οποία μπορεί να συγκεντρώνει ένα σύνολο συσχετισμένων τροποποιήσεων που εξυπηρετούν έναν συγκεκριμένο σκοπό, όπως την προσθήκη μιας νέας λειτουργίας, τη διόρθωση ενός σφάλματος, την πραγματοποίηση αναπροσαρμογών (refactoring), την αντιμετώπιση μιας ευπάθειας ή τη βελτίωση κάποιου άλλου χαρακτηριστικού του πηγαίου κώδικα και του λογισμικού που αυτός υλοποιεί. Η συγκέντρωση των σχετικών αλλαγών κάτω από μία κοινή οντότητα επιτρέπει να συνδεθεί ο τροποποιημένος κώδικας με πληροφορίες που εξηγούν τι επιχειρεί να αλλάξει η συνεισφορά και γιατί πραγματοποιείται. Στις σύγχρονες πλατφόρμες, ένα Pull Request μπορεί επίσης να αποτελέσει σημείο συγκέντρωσης μεταδεδομένων, συζητήσεων και αποτελεσμάτων αυτοματοποιημένων ελέγχων, δημιουργώντας έτσι ένα πλουσιότερο αντικείμενο από μία απλή αναφορά μεταξύ δύο διαφορετικών εκδόσεων του κώδικα.

Ο ευρύτερος σκοπός των Pull Request περιλαμβάνει παραπάνω δεδομένα από την απλή τεχνική μεταφορά κώδικα που η δυνατότητα του merging, που περιλαμβάνεται στο Git, προσφέρει. Παρέχουν ένα ελεγχόμενο όριο μεταξύ της ανάπτυξης μιας αλλαγής και της ένταξής της στο έργο, διευκολύνουν τον συντονισμό μεταξύ των συμμετεχόντων και δημιουργούν ένα καταγεγραμμένο ίχνος της διαδικασίας μέσω της οποίας εξελίσσεται το λογισμικό. Η σχετική εμπειρική βιβλιογραφία έχει αναδείξει το pull-based development ως σημαντικό μοντέλο αριθμού ανεξάρτητων συνεισφορών και αυξημένων ευκαιριών συμμετοχής της κοινότητας. Παράλληλα, επόμενες μελέτες επιβεβαιώνουν ότι τα Pull Requests αποτελούν κεντρικό στοιχείο των σύγχρονων collaborative development workflows και χρησιμοποιούνται ευρέως ως αντικείμενα πάνω στα οποία λαμβάνονται αποφάσεις σχετικά με την εξέλιξη ενός έργου, αυξάνοντας έτσι και τον καίριο χαρακτήρα που καταλαμβάνουν στην σύγχρονη ανάπτυξη λογισμικού.

Ιδιαίτερη σημασία αποκτά ο συγκεκριμένος όρος στα πλαίσια της παρούσας έρευνας καθώς χρησιμοποιείται και ως βασική μονάδα ανάλυσης της τροποποίησης του λογισμικού. Η επιλογή αυτή επιτρέπει μία αλλαγή να αντιμετωπίζεται όχι ως μεμονωμένη ή ανεξάρτητο commit, αλλά ως ένα συνεκτικό σύνολο τροποποιήσεων που υποβλήθηκε με συγκεκριμένο αναπτυξιακό σκοπό. Μέσω της σύγκρισης της κατάστασης του κώδικα προτού συμβούν οι αλλαγές και στην τελική μορφή του μεταβλημένου κώδικα από το σύνολο αυτών των αλλαγών δύναται να ποσοτικοποιηθούν οι αντίστοιχες μεταβολές στις μετρικές των συνιστωσών ποιότητας και ασφάλειας. Επομένως, το Pull Request αποτελεί κατάλληλο επίπεδο ανάλυσης για τη διερεύνηση του βασικού ερευνητικού ζητήματος της εργασίας: εάν και με ποιον τρόπο μία πραγματική τροποποίηση του πηγαίου κώδικα συνδέεται με ταυτόχρονες μεταβολές στην ποιότητα και στην ασφάλεια του λογισμικού.

2.2.1 Base και Head κατάσταση

Στην ορολογία των Pull Requests, το base δηλώνει τη γραμμή ανάπτυξης που πρόκειται να δεχθεί την αλλαγή, ενώ το head δηλώνει τη γραμμή από την οποία προέρχονται τα προτεινόμενα commits. Η μεταξύ τους σύγκριση ορίζει το περιεχόμενο που αξιολογείται πριν από το merge (Gousios, Pinzger & van Deursen, 2014). Αναλυτικότερα, το base branch λειτουργεί ως σημείο αναφοράς της σύγκρισης, ενώ το head branch αντιπροσωπεύει την κατάσταση του κώδικα συμπεριλαμβανομένων των προτεινόμενων τροποποιήσεων. Η σύγκριση των δύο επιτρέπει να προσδιοριστεί το σύνολο των commits, των αρχείων και των γραμμών κώδικα που διαφοροποιούνται και συνεπώς οι αλλαγές που προτείνονται μέσω του Pull Request που δημιουργείται.

Η διάκριση μεταξύ base και head branch (ή καταστάσεων) κρίνεται ιδιαίτερα καίρια για την παρούσα εργασία, καθώς επιτρέπει να οριστούν με σαφήνεια οι καταστάσεις του πηγαίου κώδικα που χρησιμοποιούνται για τη μελέτη μιας αλλαγής. Η κατάσταση αναφοράς αντιστοιχεί στον κώδικα πριν από την εφαρμογή των προτεινόμενων τροποποιήσεων, ενώ η κατάσταση που περιλαμβάνει τις αλλαγές του head αντιπροσωπεύει τη νέα μορφή του πηγαίου κώδικα. Η σύγκριση δύο τέτοιων

καταστάσεων μπορεί, αρχικά, να χρησιμοποιηθεί για την ποσοτικοποίηση μεταβολών σε μετρικές λογισμικού και αποτέλεσε τη βάση της αρχικά σχεδιασμένης before/after προσέγγισης της παρούσας έρευνας. Όμως, η περιορισμένη within-PR μεταβλητότητα η οποία διαπιστώθηκε στα πραγματικά δεδομένα δεν επέτρεψε τη χρήση των διαφορών αυτών ως κύριας αναλυτικής στρατηγικής. Η σύγκριση των δύο καταστάσεων διατηρήθηκε για τον έλεγχο της αρχικής υπόθεσης και της συνέπειας των δεδομένων, ενώ η κύρια στατιστική ανάλυση πραγματοποιήθηκε πάνω στην τελική pr_closed κατάσταση.

2.2.2 Κύκλος ζωής (Lifecycle) ενός Pull Request

Ο κύκλος ζωής (lifecycle) ενός Pull Request περιγράφει την πορεία μιας προτεινόμενης αλλαγής λαμβάνοντας ως αφετηρία την στιγμή που αυτή υποβάλλεται μέχρι την τελική ολοκλήρωση της διαδικασίας. Συνήθως ένα Pull Request δημιουργείται όταν ο developer έχει πραγματοποιήσει ένα σύνολο αλλαγών που επιθυμεί να παρουσιάσει στο έργο. Αν η εργασία δεν έχει ολοκληρωθεί, υπάρχει η δυνατότητα αρχικής δημιουργίας ως draft Pull Request, ώστε να είναι ορατή στους λοιπούς συμμετέχοντες χωρίς ακόμη να θεωρείται έτοιμη για τελική αξιολόγηση. Όταν οι αλλαγές έχουν φτάσει σε κατάλληλη κατάσταση, το Pull Request μπορεί να χαρακτηριστεί ως έτοιμο για εξέταση. Από το χρονικό σημείο αυτού του χαρακτηρισμού και έπειτα αποτελεί ενεργή πρόταση αλλαγής στην οποία συγκεντρώνονται τα σχετικά commits, οι μεταβολές των αρχείων, οι συζητήσεις και οι αυτοματοποιημένοι έλεγχοι που σχετίζονται με την αλλαγή.

Κατά τη διάρκεια που το Pull Request παραμένει open (κατάσταση:ανοιχτό), δεν αποτελεί στατική οντότητα. Ο δημιουργός του μπορεί να προσθέσει νέα commits και να τροποποιήσει τον κώδικα ως απάντηση σε προβλήματα ή ασυνέπειες που εντοπίστηκαν σε παρατηρήσεις που προέκυψαν κατά την αξιολόγηση. Παράλληλα, μπορούν να εκτελούνται αυτοματοποιημένοι έλεγχοι, όπως tests, builds και άλλα CI workflows, ενώ η πλατφόρμα καταγράφει την εξέλιξη του Pull Request και την κατάσταση των απαιτήσεων που πρέπει να ικανοποιηθούν. Το Pull Request, με τον τρόπο αυτό, περνάει επανειλημμένα από κύκλους τροποποίησης και επανελέγχου μέχρι να θεωρηθεί κατάλληλο για ολοκλήρωση. Το GitHub διαχωρίζει χαρακτηριστικά τις σχετικές πληροφορίες με προβολές για τη συζήτηση, τα commits, τους αυτοματοποιημένους ελέγχους και τις μεταβολές των αρχείων, επιτρέποντας την παρακολούθηση της εξέλιξης της αλλαγής κατά τη διάρκεια ζωής τους.

Το lifecycle ολοκληρώνεται επιτυχώς ή μη όταν ληφθεί τελική απόφαση για το Pull Request. Εφόσον οι απαιτήσεις του repository ικανοποιούνται, η προτεινόμενη αλλαγή μπορεί να γίνει αποδεκτή και η διαδικασία να ολοκληρωθεί επιτυχώς. Εναλλακτικά, ένα Pull Request μπορεί να κλείσει χωρίς οι αλλαγές του να γίνουν αποδεκτές και να ενσωματωθούν, για παράδειγμα επειδή η πρόταση εγκαταλείφθηκε, αντικαταστάθηκε ή κρίθηκε ότι δεν πρέπει να προχωρήσει. Συνεπώς, το lifecycle ενός Pull Request δεν περιγράφει μόνο μία τεχνική μεταφορά κώδικα, αλλά μία επαναληπτική διαδικασία εξέλιξης, αξιολόγησης και λήψης απόφασης γύρω από μία συγκεκριμένη αλλαγή. Η συγκεκριμένη ιδιότητα το καθιστά

ιδιαίτερα χρήσιμο και ως μονάδα εμπειρικής μελέτης, καθώς συγκεντρώνει το ιστορικό μιας αλλαγής από την αρχική της υποβολή μέχρις ότου την τελική της κατάσταση.

2.2.3 Αξιολόγηση κώδικα (Code Review)

Το modern code review είναι διαδικασία αξιολόγησης αλλαγών από άλλους συμμετέχοντες πριν ή γύρω από τη συγχώνευσή τους. Μπορεί να βελτιώσει τη μεταφορά γνώσης και να συμβάλει στον εντοπισμό προβλημάτων, χωρίς όμως να εγγυάται από μόνο του την απουσία defects (Bacchelli & Bird, 2013; McIntosh et al., 2016). Σε πλατφόρμες όπως το GitHub, μία ανασκόπηση μπορεί να ολοκληρωθεί με απλό σχολιασμό, με έγκριση (approve) ή με αίτημα αλλαγών (request changes). Με τον τρόπο αυτό, το code review λειτουργεί ως ένα οργανωμένο στάδιο ελέγχου μεταξύ της συγγραφής του κώδικα και της τελικής αποδοχής της αλλαγής.

Ο σκοπός της συγκεκριμένης διαδικασίας δεν περιορίζεται αποκλειστικά στον εντοπισμό bugs. Οι reviewers μπορούν να εξετάσουν την ορθότητα και την κατανοησιμότητα της υλοποίησης, τη συνέπεια με τις αρχιτεκτονικές και προγραμματιστικές πρακτικές του έργου, πιθανές περιπτώσεις περιττής πολυπλοκότητας, ανεπαρκούς χειρισμού σφαλμάτων ή προβλημάτων που μπορεί να επηρεάσουν την ποιότητα και την ασφάλεια του λογισμικού. Εμπειρική μελέτη έχει αναδείξει ότι, παρότι η εύρεση defects αποτελεί σημαντικό κίνητρο για τη διεξαγωγή reviews, στην πράξη το modern code review προσφέρει και άλλα οφέλη, όπως βελτίωση του κώδικα, μεταφορά γνώσης μεταξύ developers, καλύτερη κατανόηση των αλλαγών και αναζήτηση εναλλακτικών λύσεων. Κατά συνέπεια, το code review αποτελεί ταυτόχρονα μηχανισμό διασφάλισης ποιότητας και διαδικασία συνεργασίας και διάχυσης γνώσης στο εσωτερικό περιβάλλον της ομάδας ανάπτυξης.

Είναι επίσης σημαντικό να οριστεί η διάκριση μεταξύ του ανθρώπινου code review και των αυτοματοποιημένων ελέγχων που μπορεί να εκτελούνται στο ίδιο Pull Request. Εργαλεία testing, static analysis ή security scanning μπορούν να εντοπίζουν συγκεκριμένες κατηγορίες προβλημάτων με αυτοματοποιημένο τρόπο, όμως ο reviewer μπορεί να αξιολογήσει ευρύτερα το πλαίσιο και τον σκοπό μιας αλλαγής, τις σχεδιαστικές αποφάσεις και το κατά πόσο η υλοποίηση είναι κατάλληλη για το εκάστοτε έργο. Οι δύο διαδικασίες λειτουργούν συμπληρωματικά και δεν πρέπει να αντικαθιστά η μία την άλλη. Σχετική εμπειρική έρευνα έχει συνδέσει την έκταση και την ποιότητα της διαδικασίας review με την ποιότητα του λογισμικού, αναδεικνύοντας τη σημασία του modern code review ως ουσιαστικού μηχανισμού ελέγχου των μεταβολών πριν αυτές αποτελέσουν μέρος της δυναμικά εξελισσόμενης βάσης πηγαίου κώδικα.

2.2.4 Merge και Closed Pull Requests

Ένα Pull Request μπορεί να εισέλθει στην τελική του κατάσταση και ολοκλήρωση με δύο βασικούς τρόπους, είτε μέσω merge -ενσωμάτωση των προτεινόμενων αλλαγών-, είτε με κλείσιμο χωρίς ενσωμάτωση. Στην πρώτη περίπτωση, οι αλλαγές του Pull Request εντάσσονται στο branch προορισμού, υπό την προϋπόθεση ότι ικανοποιούνται οι κανόνες και οι απαιτήσεις του repository, όπως ενδεχόμενα required reviews ή επιτυχείς αυτοματοποιημένοι έλεγχοι. Η πλατφόρμα του GitHub επεκτείνεται μέσω της υποστήριξης διάφορων μεθόδων ενσωμάτωσης, όπως merge commit, squash and merge και rebase and merge, οι οποίες διαφοροποιούνται κυρίως ως προς τον τρόπο με τον οποίο διατηρείται ή αναδιαμορφώνεται το ιστορικό των commits. Οι προαναφερθέντες όροι δεν αποτελούν αντικείμενο μελέτης της παρούσας εργασίας και για τον λόγο αυτό η αναφορά σε αυτούς ήταν επιγραμματική χωρίς περαιτέρω επεξήγηση στον τρόπο λειτουργίας τους. Η επιλογή της κατάλληλης μεθόδου εξαρτάται από τις πρακτικές και την πολιτική διαχείρισης ιστορικού του εκάστοτε έργου.

Στην περίπτωση του merge μέσω της διαδικασίας Git merging, όταν οι δύο γραμμές ανάπτυξης έχουν αποκλίσει αρκετά, το Git μπορεί να χρησιμοποιήσει διαδικασία three-way merge, λαμβάνοντας υπόψη τις δύο τελικές καταστάσεις και έναν κοινό πρόγονό τους. Όταν δεν υπάρχει πραγματική απόκλιση και η γραμμή προορισμού μπορεί απλώς να μετακινηθεί προς ένα νεότερο commit, είναι δυνατό να πραγματοποιηθεί fast-forward, χωρίς τη δημιουργία ξεχωριστού merge commit. Αντίθετα, όταν οι αλλαγές δεν μπορούν να συνδυαστούν αυτόματα, μπορεί να προκύψουν merge conflicts που απαιτούν επίλυση πριν ολοκληρωθεί η διαδικασία.

Ένα Pull Request μπορεί επίσης να γίνει closed χωρίς merge, όταν η προτεινόμενη αλλαγή δεν πρόκειται τελικά να ενσωματωθεί. Αυτό μπορεί να συμβεί επειδή η πρόταση απορρίφθηκε, εγκαταλείφθηκε, αντικαταστάθηκε από διαφορετική υλοποίηση ή δεν θεωρήθηκε πλέον απαραίτητη. Η ένδειξη closed δεν συνεπάγεται αυτομάτως την ενσωμάτωση των αλλαγών στον κώδικα, απαιτείται διάκριση μεταξύ merged Pull Requests και closed-unmerged Pull Requests. Η διάκριση αυτή είναι σημαντική και για την εμπειρική ανάλυση της παρούσας εργασίας, καθώς η τελική κατάσταση ενός Pull Request επηρεάζει το πώς πρέπει να ερμηνεύεται η σχέση μεταξύ των προτεινόμενων αλλαγών και της πραγματικής εξέλιξης του πηγαίου κώδικα και κατ' επέκταση του ίδιου του repository.

2.3 Μεταβολές πηγαίου κώδικα

Οι μεταβολές του πηγαίου κώδικα (source code changes) αποτελούν βασικό μηχανισμό μέσω του οποίου ένα σύστημα λογισμικού εξελίσσεται κατά τη διάρκεια του κύκλου ζωής του. Ένα σύστημα είναι σχεδόν αδύνατο να παραμείνει στατικό μετά την αρχική του ανάπτυξη, τροποποιείται συνεχώς προκειμένου να ανταποκριθεί σε νέες απαιτήσεις των χρηστών (ή πελατών), να διορθώσει υπάρχοντα προβλήματα

στον τρόπο λειτουργίας και στην ίδια την λειτουργία του συστήματος, να προσαρμοστεί σε αλλαγές του τεχνολογικού ή λειτουργικού περιβάλλοντος και να βελτιώσει την εσωτερική του δομή. Η βιβλιογραφία της εξέλιξης του λογισμικού αντιμετωπίζει τη συνεχή μεταβολή ως ένα εγγενές χαρακτηριστικό των μακροχρόνια χρησιμοποιούμενων συστημάτων. Αντίστοιχα, σύγχρονες μελέτες των προσθέσεων πίσω από τις αλλαγές λογισμικού διακρίνουν μεταξύ σκοπών όπως η διόρθωση υφιστάμενων προβλημάτων, η προσαρμογή σε μεταβαλλόμενο φυσικό/ψηφιακό/άλλο περιβάλλον, η βελτίωση του συστήματος και η πρόληψη μελλοντικών προβλημάτων.

Μία μεταβολή πηγαίου κώδικα μπορεί επομένως να εξυπηρετεί διαφορετικό σκοπό και να κατέχει διαφορετικό εύρος. Για παράδειγμα, μπορεί να αφορά την προσθήκη ή τροποποίηση μιας λειτουργίας, τη διόρθωση ενός σφάλματος ή μιας ευπάθειας, την προσαρμογή του συστήματος σε μία νέα σύνδεση/εξάρτηση ή πλατφόρμα, καθώς και την αναδιάρθρωση της εσωτερικής υλοποίησης με στόχο τη βελτίωση της συντηρησιμότητας. Από την οπτική του Git, τέτοιες μεταβολές οδηγούν το repository από μία καταγεγραμμένη κατάσταση σε μία διαφορετική κατάσταση καθώς το Git οργανώνει το ιστορικό ως διαδοχή snapshots του έργου. Η ίδια λογική επιτρέπει τη σύγκριση διαφορετικών καταστάσεων του συστήματος και την ανακατασκευή της εξέλιξής του.

Ωστόσο, η επίδραση μιας αλλαγής δεν περιορίζεται απαραίτητα στο τμήμα του κώδικα το οποίο τροποποιείται για την επίτευξή της. Τα διαφορετικά στοιχεία ενός συστήματος συνδέονται μέσω σχέσεων ελέγχου, ροής δεδομένων, κλήσεων, εξαρτήσεων και κοινών δομών. Συνεπώς, μία τοπική τροποποίηση μπορεί να προκαλέσει άμεσες μεταβολές και επιπτώσεις σε διάφορα άλλα σημεία του λογισμικού. Η έννοια της Change Impact Analysis (Ανάλυση Επιπτώσεων Αλλαγής) αναφέρεται ακριβώς στη διερεύνηση των πιθανών συνεπειών μιας προτεινόμενης αλλαγής ή στον προσδιορισμό των επιπλέον στοιχείων που ενδέχεται να χρειαστεί να τροποποιηθούν εξαιτίας της. Η κατανόηση αυτών των επιδράσεων είναι σημαντική για δραστηριότητες όπως η συντήρηση, η επιλογή κατάλληλων ελέγχων και η αποφυγή ανεπιθύμητων παρενεργειών μετά από μία τροποποίηση.

Η έννοια της μεταβολής έχει ιδιαίτερη σημασία για την παρούσα εργασία, καθώς το ενδιαφέρον δεν περιορίζεται μόνο στην τεχνική ύπαρξη μιας αλλαγής, αλλά κυρίως στις συνέπειες που αυτή μπορεί να προκαλέσει στις ιδιότητες του λογισμικού. Μία τροποποίηση μπορεί να διορθώνει ένα πρόβλημα και ταυτόχρονα να επηρεάζει την πολυπλοκότητα, τη συντηρησιμότητα, την αξιοπιστία ή την ασφάλεια του συνόλου του συστήματος. Έτσι, κάθε Pull Request αντιμετωπίζεται ως μία διακριτή αυτόνομη μονάδα εξέλιξης, μέσω της οποίας το λογισμικό μεταβαίνει από μία αρχική σε μία νέα κατάσταση. Η σύγκριση των αντίστοιχων snapshots επιτρέπει στην συνέχεια να διερευνηθεί εάν η συγκεκριμένη αλλαγή συνοδεύεται από βελτίωση, υποβάθμιση ή διαφορετικού τύπου μεταβολή στις μετρικές ποιότητας και ασφάλειας του πηγαίου κώδικα ή/και του ίδιου του λογισμικού.

2.3.1 Προστιθέμενες (Added), Διαγραμμένες (Deleted) και Τροποποιημένες (Modified) γραμμές πηγαίου κώδικα (Lines)

Κατά τη διάρκεια σύγκρισης δύο διαφορετικών καταστάσεων του πηγαίου κώδικα, μία από τις απλούστερες και συχνότερα χρησιμοποιούμενες προσεγγίσεις για την περιγραφή του μεγέθους μίας μεταβολής του πηγαίου κώδικα αποτελεί η καταμέτρηση των γραμμών οι οποίες προστέθηκαν (added lines) και διαγράφηκαν (deleted lines). Οι προστιθέμενες γραμμές αντιπροσωπεύουν περιεχόμενο που υπάρχει στη νεότερη (committed) κατάσταση του κώδικα χωρίς να προϋπάρχει στην κατάσταση αναφοράς -κατάσταση από την οποία αφορμίσθηκαν οι τροποποιήσεις του κώδικα- και αντίθετα, οι διαγραμμένες γραμμές αντιπροσωπεύουν περιεχόμενο που υπήρχε στην κατάσταση αναφοράς αλλά αφαιρέθηκε με αποτέλεσμα να μην υπάρχει πλέον στην νέα κατάσταση. Στην αναπαράσταση ενός diff, οι προσθήκες συνηθίζεται να εμφανίζονται με το σύμβολο + (συν) και οι διαγραφές με το σύμβολο - (πλην). Το Git μπορεί επίσης μέσω μορφών όπως το -numstat να παρέχει για κάθε αρχείο τον συνολικό αριθμό των προστιθέμενων και διαγραμμένων γραμμών, γεγονός που διευκολύνει την αυτοματοποιημένη ποσοτικοποίηση των αλλαγών.

Η έννοια των τροποποιημένων (modified) lines απαιτεί μεγαλύτερη προσοχή τόσο σε εννοιολογικό όσο και σε επίπεδο τυπικού line-based diff του Git. Σε εννοιολογικό επίπεδο, μία γραμμή θεωρείται τροποποιημένη όταν μέρος του περιεχομένου της μεταβάλλεται μεταξύ δύο καταστάσεων του κώδικα. Σε επίπεδο τυπικού line-based diff του Git μία τέτοια μεταβολή δεν καταγράφεται απαραίτητα ως ξεχωριστή τρίτη κατηγορία. Συνήθως αναπαρίσταται ως διαγραφή της παλαιάς και προσθήκη της νέας γραμμής κώδικα, με αποτέλεσμα συνηθισμένα στατιστικά του Git να αναφέρουν καθαρά αριθμούς additions και deletions. Συνεπώς, όταν σε μία εμπειρική ανάλυση χρησιμοποιείται ξεχωριστή μεταβλητή για τροποποιημένες γραμμές, πρέπει να διευκρινίζεται ο ακριβής τρόπος με τον οποίο αυτή υπολογίζεται από το εργαλείο ή το dataset, καθώς δεν είναι κατ' ανάγκη ισοδύναμη με μία εγγενή τρίτη κατηγορία του git diff.

Οι μετρήσεις αυτές συχνά χρησιμοποιούνται ως απλοί δείκτες του μεγέθους ή της έκτασης μίας αλλαγής. Ένα Pull Request με μεγάλο αριθμό προσθηκών και διαγραφών μεταβάλλει ποσοτικά περισσότερο περιεχόμενο από ένα PR με ελάχιστες αλλαγές. Η συγκεκριμένη πληροφορία δύναται να χρησιμοποιηθεί για την περιγραφή και σύγκριση διαφορετικών Pull Requests και ως χαρακτηριστικό σε εμπειρικές αναλύσεις λογισμικού. Ο αριθμός των μεταβαλλόμενων γραμμών, όμως, δεν αποτυπώνει από μόνος του τη σημασία ή την πολυπλοκότητα μίας οποιασδήποτε αλλαγής στον πηγαίο κώδικα. Μία μικρή αλλαγή σε κρίσιμη λογική αυθεντικοποίησης μπορεί να έχει επίδραση στην ασφάλεια αρκετά μεγαλύτερη από εκατοντάδες αλλαγές σε επαναλαμβανόμενο ή μη λειτουργικό περιεχόμενο. Μετακινήσεις ή αναδιατάξεις κώδικα μπορούν, αντίστοιχα, να επηρεάσουν τα line-based statistics χωρίς η σημασιολογική συμπεριφορά του συστήματος να μεταβάλλεται στον ίδιο βαθμό.

Οι προστιθέμενες, διαγραμμένες και οι ορισμένες από το διαθέσιμο dataset τροποποιημένες γραμμές μπορούν να χρησιμοποιηθούν ως χαρακτηριστικά του μεγέθους τροποποίησης ενός Pull Request και όχι ως άμεσοι δείκτες ποιότητας ή

ασφάλειας, στο πλαίσιο της παρούσας εργασίας. Η διάκριση αυτή εξακολουθεί να λαμβάνει σημαντικότητα: οι συγκεκριμένες μετρήσεις περιγράφουν κυρίως το εύρος της αλλαγής, ενώ η επίδρασή της στην ποιότητα και στην ασφάλεια αξιολογείται μέσω των αντίστοιχων μετρικών λογισμικού. Με τον τρόπο αυτό μπορούν να εξεταστούν ερευνητικά ερωτήματα όπως είναι για παράδειγμα το ερώτημα εάν μεγαλύτερες αλλαγές παρουσιάζουν διαφορετική συμπεριφορά ως προς τις μεταβολές ποιότητας και ασφάλειας, χωρίς να θεωρείται εκ των προτέρων ότι η μεγαλύτερη ποσότητα τροποποιημένου κώδικα συνεπάγεται χαμηλότερη ποιότητα ή υψηλότερο κίνδυνο.

2.3.2 Μετρικές LOC και NCLOC

Οι Γραμμές του Κώδικα (Lines of Code – LOC) αποτελούν μία από τις περισσότερο διαδεδομένες μετρικές μεγέθους λογισμικού και χρησιμοποιούνται για την ποσοτική αποτύπωση της έκτασης του πηγαίου κώδικα ενός συστήματος, αρχείου ή άλλης μονάδας λογισμικού. Η έννοια LOC δεν διαθέτει έναν καθορισμένο ενιαίο τρόπο υπολογισμού και εξαρτάται άμεσα από την δομή και τη λειτουργία του εκάστοτε εργαλείου, αφού μπορεί είτε να περιλαμβάνει όλες τις φυσικές γραμμές του αρχείου είτε αυτές που περιέχουν αποκλειστικά ουσιαστικό κώδικα, εξαιρώντας για παράδειγμα κενές γραμμές ή γραμμές σχολίων. Για τον λόγο αυτό, στις εμπειρικές έρευνες κρίνεται απαραίτητη η δήλωση του ακριβούς ορισμού που χρησιμοποιείται. Η LOC αποτελεί κυρίως μετρική μεγέθους και όχι άμεσο δείκτη ποιότητας ή ασφάλειας, ενώ οι συγκρίσεις της επηρεάζονται από παράγοντες όπως η γλώσσα προγραμματισμού και το προγραμματιστικό ύφος.

Η μετρική της NCLOC ή Non-Comment Lines of Code αποτελεί συγκεκριμένη εκδοχή της προηγουμένως περιγραφόμενης εκδοχής της μέτρησης, καθώς επιχειρεί να αποτυπώσει τις γραμμές που περιέχουν πραγματικό πηγαίο κώδικα, εξαιρώντας γραμμές που αποτελούνται αποκλειστικά από σχόλια ή κενό περιεχόμενο. Η μετρική ncloc στο εργαλείο του SonarQube, υπολογίζει τις φυσικές γραμμές που περιλαμβάνουν τουλάχιστον έναν χαρακτήρα διαφορετικό από whitespace, tab ή περιεχόμενο σχολίου. Με τον τρόπο αυτό, η NCLOC μπορεί να προσφέρει πιο συνεπή εκτίμηση του ενεργού μεγέθους του κώδικα σε σχέση με μία γενική καταμέτρηση όλων των φυσικών γραμμών. Η LOC και η NCLOC ωστόσο δεν αποτυπώνουν από μόνες τους την πολυπλοκότητα, την συντηρησιμότητα ή την ασφάλεια του συνολικού συστήματος.

Η NCLOC χρησιμοποιείται εδώ κυρίως ως μεταβλητή μεγέθους και ως βάση κανονικοποίησης άλλων μετρικών. Η άμεση σύγκριση του απόλυτου αριθμού σφαλμάτων, ευπαθειών, code smells ή άλλων προβλημάτων (issues) μεταξύ των Pull Requests διαφορετικού μεγέθους μπορεί να είναι παραπλανητική, καθώς μεγαλύτερες ποσότητες κώδικα παρέχουν μεγαλύτερη έκθεση για την εμφάνιση περισσότερων ευρημάτων. Η χρήση της NCLOC ως παρονομαστή επιτρέπει την κατασκευή μετρικών πυκνότητας και διευκολύνει τη συγκρισιμότητα μεταξύ διαφορετικών PRs. Συμπερασματικά, στην εμπειρική ανάλυση η NCLOC δεν αντιμετωπίζεται ως άμεσος δείκτης ποιότητας ή ασφάλειας, αλλά ως σημαντικό

μέτρο μεγέθους που υποστηρίζει την ορθότερη ερμηνεία των υπόλοιπων μετρικών λογισμικού.

2.4 Μετρικές Λογισμικού – Software Metrics

Οι μετρικές λογισμικού μετατρέπουν ιδιότητες προϊόντων, διεργασιών ή έργων σε τυποποιημένη ποσοτική πληροφορία. Η ερμηνεία τους απαιτεί σαφή ορισμό του μετρούμενου χαρακτηριστικού, της διαδικασίας μέτρησης και του σκοπού αξιολόγησης (Flater et al., 2016; ISO/IEC/IEEE 15939:2017). Με τον τρόπο αυτό, οι μετρικές μπορούν να υποστηρίξουν την παρακολούθηση της κατάστασης ενός συστήματος, την αναγνώριση τάσεων, τη σύγκριση διαφορετικών εκδόσεων ή τμημάτων του λογισμικού και τη λήψη αποφάσεων κατά τη διαδικασία ανάπτυξης και συντήρησης. Η ποσοτικοποίηση και η μέτρηση, όπως επισημαίνεται από το Software Engineering Institute, παρέχουν τη βάση για τη δημιουργία και αξιολόγηση μοντέλων της διαδικασίας ανάπτυξης λογισμικού, ενώ από το ISO/IEC/IEEE 15939 η μέτρηση αντιμετωπίζεται ως οργανωμένη διαδικασία που ξεκινάει από συγκεκριμένες πληροφοριακές ανάγκες και καταλήγει στην παραγωγή και ερμηνεία αποτελεσμάτων μέτρησης.

Παρεμφερείς όροι, *measure* και *metric*, χρησιμοποιούνται συχνά με ελαφρώς διαφορετικό τρόπο στη βιβλιογραφία και στη λειτουργία και δόμηση των εργαλείων λογισμικού. Σε μία χρήσιμη διάκριση, μία άμεση μέτρηση (*measure*) αφορά ένα χαρακτηριστικό που μπορεί να παρατηρηθεί ή να καταμετρηθεί σχετικά άμεσα, ενώ μία μετρική (*metric*) μπορεί να αποτελεί επεξεργασμένη ή υψηλότερου επιπέδου αναπαράσταση που χρησιμοποιείται για την αξιολόγηση μιας ιδιότητας. Μία πρωτογενής, για παράδειγμα, καταμέτρηση μπορεί να χρησιμοποιηθεί μόνη της ή να συνδυαστεί με άλλες μετρήσεις ώστε να προκύψει στην συνέχεια ένας κανονικοποιημένος δείκτης. Η ορολογία δεν είναι απόλυτα ομοιόμορφη καθώς υπάρχει επικάλυψη μεταξύ των δύο εννοιών. Για τον λόγο αυτό, στην πράξη ο όρος *software metric* χρησιμοποιείται συχνά ευρύτερα για να περιγράψει τόσο άμεσες μετρήσεις όσο και παράγωγους δείκτες που χρησιμοποιούνται για την ποσοτική αξιολόγηση του λογισμικού.

Οι μετρικές μπορούν να αφορούν διάφορα και διαφορετικά επίπεδα της ανάπτυξης λογισμικού. Ο όρος *Product metrics* περιγράφει χαρακτηριστικά του ίδιου του προϊόντος ή του πηγαίου κώδικα, ο όρος *Process metrics* χρησιμοποιούνται για την παρακολούθηση της διαδικασίας μέσω της οποίας αναπτύσσεται και συντηρείται το λογισμικό, ενώ μετρήσεις επιπέδου έργου μπορούν να χρησιμοποιηθούν για την παρακολούθηση παραμέτρων είναι η προσπάθεια, η πρόοδος και η απόδοση μιας αναπτυξιακής διαδικασίας. Στην παρούσα εργασία το κύριο ενδιαφέρον επικεντρώνεται στις μετρικές του προϊόντος και ιδιαίτερα σε μετρήσεις οι οποίες μπορούν να εξαχθούν από τον πηγαίο κώδικα και από αποτελέσματα αυτοματοποιημένων αναλύσεων.

Μία σημαντική διάκριση αφορά επίσης τις άμεσες και παράγωγες μετρικές. Μία άμεση μέτρηση μπορεί να προκύπτει απευθείας από την παρατήρηση ενός χαρακτηριστικού του λογισμικού, ενώ μία παράγωγη μετρική υπολογίζεται μέσω συνδυασμού ή μετασχηματισμού άλλων μετρήσεων. Οι παράγωγες μετρικές κρίνονται ιδιαίτερα χρήσιμες όταν απαιτείται σύγκριση μεταξύ συστημάτων ή αλλαγών διαφορετικού μεγέθους. Για παράδειγμα, ένας απόλυτος αριθμός ευρημάτων μπορεί να εξαρτάται σε σημαντικό βαθμό από την ποσότητα κώδικα που εξετάζεται, ενώ ένας δείκτης πυκνότητας μπορεί να εκφράζει τα ίδια ευρήματα σε σχέση με μία κοινή βάση μεγέθους. Σύμφωνα με το πρότυπο ISO/IEC/IEEE 15939, μία διαδικασία μέτρησης πρέπει να προσδιορίζει πρώτα τις πληροφοριακές ανάγκες, να επιλέγει κατάλληλα μέτρα και να καθορίζει πώς τα αποτελέσματα θα αναλυθούν και θα ερμηνευθούν, καθιστώντας την επιλογή και τον τρόπο κατασκευής τέτοιων μετρικών απαραίτητα μη αυθαίρετο.

Οι μετρικές λογισμικού αποκτούν και κατέχουν μεγαλύτερη σημασία και αξία όταν αυτές χρησιμοποιούνται διαχρονικά ή συγκριτικά μεταξύ τους ή με άλλες αντίστοιχες μετρικές. Μία μεμονωμένη τιμή παρέχει πληροφορία για μία συγκεκριμένη κατάσταση του συστήματος -συνήθως χρονικά προσδιορισμένη-, όμως η σύγκριση της ίδιας μετρικής μεταξύ διαφορετικών εκδόσεων μπορεί να αποκαλύψει την κατεύθυνση και την ένταση μιας μεταβολής. Σε περιβάλλοντα συνεχούς ανάπτυξης όπου ο πηγαίος κώδικας μεταβάλλεται επανειλημμένα, είναι ιδιαίτερα σημαντικό. Η εξέταση των μετρικών πριν και μετά από μία προτεινόμενη αλλαγή στον πηγαίο κώδικα δύναται να βοηθήσει να διαπιστωθεί εάν η συγκεκριμένη τροποποίηση συνοδεύεται από βελτίωση, υποβάθμιση ή σταθερότητα ως προς το χαρακτηριστικό που επιχειρεί να αποτυπώσει η μετρική. Η αυτοματοποίηση της συλλογής πολλών μετρικών επιτρέπει την ενσωμάτωσή τους σε διαδικασίες συνεχούς παρακολούθησης και αξιολόγησης του λογισμικού. Η σημασία των μεθόδων και των μετρικών έχει αναγνωριστεί από οργανισμούς, για την αξιολόγηση χαρακτηριστικών που σχετίζονται με την ποιότητα και την αξιοπιστία του λογισμικού.

Οι μετρικές, παρά τη χρησιμότητά τους, δεν πρέπει να αντιμετωπίζονται ως απόλυτες ή πλήρεις αναπαραστάσεις σύνθετων εννοιών, όπως η ποιότητα ή η ασφάλεια πηγαίου κώδικα και λογισμικού. Μία μετρική αποτυπώνει ένα συγκεκριμένο παρατηρήσιμο χαρακτηριστικό ή λειτουργεί ως δείκτης μίας ευρύτερης ιδιότητας, χωρίς να την περιγράφει απαραίτητα στο σύνολό της. Εξετάζοντας ένα παράδειγμα, δύο τμήματα κώδικα μπορούν να έχουν παρόμοιες τιμές σε μία μετρική αλλά να διαφέρουν ουσιαστικά ως προς μία υλοποιημένη λειτουργία, το πλαίσιο χρήσης καθώς και τον κίνδυνο τον οποίο παρουσιάζουν. Η ερμηνεία μιας μέτρησης εξαρτάται επίσης από το εκάστοτε εργαλείο, τον ορισμό της μετρικής, τη γλώσσα προγραμματισμού του συστήματος, το μέγεθος του συστήματος και το πλαίσιο μέσα στο οποίο χρησιμοποιείται. Έχει επισημανθεί από το NIST ότι η μέτρηση χαρακτηριστικών λογισμικού παρουσιάζει σημαντικές θεωρητικές και πρακτικές δυσκολίες καθώς και πως οι διαθέσιμες μετρικές πρέπει να αξιοποιούνται με επίγνωση των περιορισμών τους και όχι ως απόλυτη «αλήθεια» για την συνολική και γενική κατάσταση ενός συστήματος.

Στην περίπτωση χρήσης πολλαπλών μετρικών ταυτόχρονων μετρικών απαιτείται ιδιαίτερη προσοχή. Διαφορετικές μετρήσεις δύναται να παρουσιάζουν

ισχυρές μεταξύ τους συσχετίσεις ή να αποτυπώνουν διαφορετικές πτυχές μιας κοινής υποκείμενης ιδιότητας. Η ανεξάρτητη ερμηνεία κάθε μετρικής, χωρίς συσχέτιση με το σύνολο, ενδέχεται να οδηγήσει σε πλεονάζουσα πληροφορία ή σε υπεραπλούστευση της πραγματικής δομής των δεδομένων. Στην εμπειρική Μηχανική Λογισμικού, οι μετρικές μπορούν να αναλύονται όχι μόνο μεμονωμένα, αλλά και ως πολυδιάστατο σύνολο μέσω στατιστικών τεχνικών που εξετάζουν τις μεταξύ τους σχέσεις. Για την παρούσα εργασία, η αρχή αυτή κρίνεται ιδιαίτερα σημαντική καθώς η ποιότητα και η ασφάλεια δεν επιχειρείται να αναπαρασταθούν από έναν μοναδικό γενικό δείκτη, αλλά από ένα σύνολο διαφορετικών παρατηρήσιμων χαρακτηριστικών.

Οι μετρικές εδώ αποτελούν τον βασικό μηχανισμό μέσω του οποίου ορίζονται οι έννοιες της ποιότητας και της ασφάλειας και μετατρέπονται σε εμπειρικά παρατηρήσιμες μεταβλητές. Για κάθε Pull Request εξετάζονται μετρήσεις που προέρχονται από συγκεκριμένες καταστάσεις του πηγαίου κώδικα ώστε να καθίσταται δυνατός ο υπολογισμός της μεταβολής τους μετά την εφαρμογή των αντίστοιχων αλλαγών. Η συγκεκριμένη προσέγγιση επιτρέπει την εξέταση της συσχέτισης των μεταβολών σε χαρακτηριστικά που σχετίζονται με την ποιότητα με μεταβολές σε χαρακτηριστικά που σχετίζονται με την ασφάλεια συνδυαστικά με την εξέταση της κατάστασης ενός έργου σε μία δεδομένη χρονική στιγμή. Οι επιμέρους μετρικές, ο ακριβής τρόπος υπολογισμού τους και οι ειδικοί περιορισμοί καθεμιάς παρουσιάζονται αναλυτικά σε επόμενη ενότητα της παρούσας βιβλιογραφίας.

2.5 Αυτοματοποιημένη ανάλυση λογισμικού

Η διαδικασία της αυτοματοποιημένης ανάλυσης λογισμικού (Automated Software Analysis) αναφέρεται στη χρήση διάφορων εργαλείων και αυτοματοποιημένων διαδικασιών για τη συστηματική εξέταση χαρακτηριστικών ενός συστήματος χωρίς να απαιτείται χειροκίνητη αξιολόγηση κάθε αλλαγής. Στόχος της συγκεκριμένης διαδικασίας είναι η έγκαιρη συλλογή πληροφοριών σχετικά με την κατάσταση στην οποία βρίσκεται το λογισμικό και ο εντοπισμός πιθανών προβλημάτων κατά τη διάρκεια της ανάπτυξής του. Τα εργαλεία μπορούν να αναλύουν -σε ορισμένες περιπτώσεις να επεξεργάζονται επίσης- πηγαίο κώδικα και άλλα σχετικά αντικείμενα/τεχνουργήματα (artifacts), να εφαρμόζουν προκαθορισμένους κανόνες ή κριτήρια και να παράγουν ευρήματα και μετρικές που χρησιμοποιούνται από τις ομάδες ανάπτυξης ή και ατομικά από προγραμματιστές. Η αυτοματοποίηση είναι ιδιαίτερα σημαντική σε μεγάλης κλίμακας έργα, όπου ο όγκος και η συχνότητα των αλλαγών καθιστούν δύσκολη την αποκλειστικά χειροκίνητη αξιολόγηση.

Πλεονεκτήματα της αυτοματοποιημένης ανάλυσης λογισμικού είναι η συνέπεια, η επαναληψιμότητα και η δυνατότητα συχνής εκτέλεσης των ίδιων ελέγχων. Οι ίδιοι ορισμένοι κανόνες μπορούν να εφαρμόζονται σε διαδοχικές καταστάσεις του λογισμικού, επιτρέποντας την έγκαιρη ανίχνευση νέων

προβλημάτων και τη σύγκριση της εξέλιξής του στον χρόνο. Με την ενσωμάτωση των αυτοματοποιημένων ελέγχων στην ροή ανάπτυξης, δύνανται να ενεργοποιηθούν μετά από αλλαγές και να παρέχουν γρήγορη ανατροφοδότηση και ενημέρωση κατάστασης καλής λειτουργίας στους προγραμματιστές. Η αξιολόγηση, με τον τρόπο αυτό, μετατρέπεται από μία περιοδική δραστηριότητα σε μία συνεχή διαδικασία που συνοδεύει την εξέλιξη του κώδικα και υποστηρίζει σύγχρονες πρακτικές DevOps και DevSecOps.

Τα αποτελέσματα αντίστοιχων αναλύσεων περιλαμβάνουν τόσο συγκεκριμένα ευρήματα όσο και ποσοτικές μετρήσεις. Παράλληλα, μπορούν να χρησιμοποιηθούν για την υποστήριξη αποφάσεων και για την εφαρμογή κανόνων πριν μία αλλαγή προχωρήσει σε επόμενο στάδιο της διαδικασίας ανάπτυξης, και πιθανώς ενσωματωθεί στην κύρια ροή ανάπτυξης. Η αυτοματοποιημένη ανάλυση, όμως, δεν υποκαθιστά πλήρως την ανθρώπινη αξιολόγηση, καθώς τα εργαλεία λειτουργούν βάσει συγκεκριμένων κανόνων, μοντέλων και παραδοχών και δεν διαθέτουν πάντοτε -στις περισσότερες περιπτώσεις- το πλήρες λειτουργικό ή επιχειρησιακό πλαίσιο μιας αλλαγής. Επομένως, τα αποτελέσματά τους πρέπει να αντιμετωπίζονται ως μηχανισμός υποστήριξης της αξιολόγησης και όχι ως απόλυτη απόφαση για την συνολική ποιότητα ή την ασφάλεια ενός συστήματος.

Η αυτοματοποιημένη ανάλυση αποτελεί βασικό μέρος της εμπειρικής μεθοδολογίας όπως αυτή αναπτύσσεται στην παρούσα εργασία, καθώς επιτρέπει την εφαρμογή της ίδιας διαδικασίας αξιολόγησης σε μεγάλο αριθμό καταστάσεων πηγαιού κώδικα. Με αυτόν τον τρόπο παράγονται συγκρίσιμες μετρικές λογισμικού για τις καταστάσεις που σχετίζονται άμεσα ή έμμεσα με τα εξεταζόμενα Pull Requests και καθίσταται δυνατή η ποσοτική μελέτη των μεταβολών ποιότητας και ασφάλειας. Οι επιμέρους τεχνικές αυτοματοποιημένης ανάλυσης παρουσιάζονται αναλυτικά στην συνέχεια.

2.5.1 Στατική Ανάλυση (Static Analysis)

Η μεθοδολογία της Στατικής Ανάλυσης Λογισμικού (Static Analysis) αποτελεί διαδικασία εξέτασης του πηγαιού (ή μεταγλωττισμένου κώδικα) χωρίς να απαιτείται η εκτέλεση του προγράμματος. Ως στόχο έχει την εξαγωγή πληροφοριών σχετικά με τη δομή και τις ιδιότητες του κώδικα και ο εντοπισμός πιθανών προβλημάτων κατά τη διαδικασία ανάπτυξης. Ένας static code analyzer εξετάζει τον κώδικα χωρίς να τον εκτελεί, με σκοπό την αναγνώριση προγραμματιστικών πρακτικών και πιθανών αδυναμιών ή συντακτικών λαθών. Η στατική ανάλυση μπορεί να εφαρμοστεί ακόμη και πριν το λογισμικό βρίσκεται σε πλήρως λειτουργική κατάσταση και να αποτελέσει μηχανισμό πρώιμης ανατροφοδότησης προς του προγραμματιστές.

Η στατική ανάλυση εξετάζει κώδικα ή συναφή artifacts χωρίς να απαιτεί την εκτέλεση της εφαρμογής. Ανάλογα με την τεχνική, μπορεί να εντοπίζει παραβιάσεις κανόνων, πιθανά defects ή security weaknesses, αλλά τα ευρήματα χρειάζονται αξιολόγηση στο πραγματικό τους context (Scarfone et al., 2008; Lenarduzzi et al., 2023). Περισσότερο προηγμένες τεχνικές είναι σε θέση να εξετάζουν τις σχέσεις μεταξύ μεταβλητών και συναρτήσεων ή να ακολουθούν τη διάδοση δεδομένων από

την είσοδό τους μέχρι τα σημεία στα οποία χρησιμοποιούνται. Επιτρέπουν επίσης τον εντοπισμό προβλημάτων τα οποία δεν είναι πάντοτε εμφανή από μία απλή εξέταση μεμονωμένων γραμμών κώδικα. Η αναλύσεις control flow και data flow αποτελούν χαρακτηριστικές τεχνικές του συγκεκριμένου πεδίου, ενώ σε εφαρμογές ασφάλειας χρησιμοποιούνται προσεγγίσεις όπως το taint analysis.

Η στατική ανάλυση δεν περιορίζεται αποκλειστικά στην ασφάλεια. Ανάλογα με το εργαλείο και τους κανόνες οι οποίοι έχουν προσδιοριστεί και εφαρμοστεί, μπορεί να χρησιμοποιηθεί για τον εντοπισμό πιθανών σφαλμάτων/bugs, παραβιάσεων προγραμματιστικών και συγγραφικών κανόνων, προβλημάτων συντηρησιμότητας, μη επιθυμητών μοτίβων κώδικα και πιθανών αδυναμιών ασφάλειας, καθώς και για την παραγωγή ποσοτικών μετρικών λογισμικού. Το εργαλείο του SonarQube, το οποίο χρησιμοποιήθηκε εκτενώς στην εξαγωγή δεδομένων για το dataset που αναλύθηκε στην παρούσα εργασία, εφαρμόζει ένα σύνολο κανόνων στον πηγαίο κώδικα και ταξινομεί τα αντίστοιχα ευρήματα σε διαστάσεις που σχετίζονται με reliability, maintainability και security. Η δυνατότητα εφαρμογής των ίδιων κανόνων σε διαφορετικές εκδόσεις λογισμικού καθιστά τη στατική ανάλυση ιδιαίτερα κατάλληλη για τη συστηματική παρακολούθηση της εξέλιξης του πηγαίου κώδικα.

Ένα σημαντικό πλεονέκτημα της συγκεκριμένης μεθοδολογίας ανάλυσης είναι η αυτοματοποίηση και η επαναλαμβανόμενη εκτέλεση μεγάλης κλίμακας την οποία και παρέχει. Οι αναλυτές μπορούν να εφαρμοστούν κατά τη συγγραφή του κώδικα, σε κάθε νέα αλλαγή ή ως μέρος μιας ευρύτερης διαδικασίας συνεχούς επαλήθευσης, συνήθως πριν την συγχώνευση των τροποποιήσεων με την κύρια ροή ανάπτυξης. Το NIST περιλαμβάνει static code scanning στις προτεινόμενες τεχνικές επαλήθευσης λογισμικού και επισημαίνει ότι μπορεί να χρησιμοποιείται για την αναζήτηση σημαντικών κατηγοριών bugs, αδυναμιών και παραβιάσεων προγραμματιστικών κανόνων και προτύπων. Η επαναληπτική εφαρμογή της ίδιας διαδικασίας επιτρέπει επίσης τη σύγκριση διαφορετικών καταστάσεων του ίδιου συστήματος με συνεπή κριτήρια.

Η στατική ανάλυση παρουσιάζει σημαντικούς περιορισμούς, παρά τα πλεονεκτήματα τα οποία παρέχει. Η πλήρης και απολύτως ακριβής ανάλυση της συμπεριφοράς πολύπλοκων προγραμμάτων δεν είναι πρακτικά εφικτή για όλες τις κατηγορίες προβλημάτων, με αποτέλεσμα τα εργαλεία να χρησιμοποιούν συχνά προσεγγίσεις και ευρετικές μεθόδους. Η διαδικασία αυτή μπορεί να οδηγήσει στα λεγόμενα false positives, όπου αναφέρεται ένα μη πραγματικό και ουσιαστικό πρόβλημα, καθώς επίσης και σε false negatives, όπου ένα πραγματικό πρόβλημα δεν ανιχνεύεται. Ορισμένα προβλήματα, επιπλέον, εξαρτώνται από πληροφορίες οι οποίες γίνονται διαθέσιμες μόνο στο περιβάλλον πραγματικής λειτουργίας του συστήματος και συνεπώς είναι δύσκολο να εντοπιστούν αποκλειστικά μέσω εξέτασης του πηγαίου κώδικα. Σε πρότυπα της βιομηχανίας αναφέρεται ότι πολλοί στατικοί αναλυτές χρησιμοποιούν heuristics ή προσεγγίσεις ώστε να μπορούν να εξετάζουν μεγάλα προγράμματα και πολλές κατηγορίες προβλημάτων, ενώ επίσης έχει επισημανθεί ότι το αποτέλεσμα που εξάγεται από μία στατική ανάλυση συχνά χρειάζεται ανθρώπινη αξιολόγηση για να προσδιοριστεί η πραγματική τους σημασία.

Στην παρούσα εργασία, η στατική ανάλυση αποτελεί τη βασική τεχνική μέσω της οποίας παράγονται οι μετρικές ποιότητας και ασφάλειας για τις διαφορετικές

καταστάσεις του πηγαίου κώδικα. η ίδια διαδικασία ανάλυσης εφαρμόζεται στις καταστάσεις που αντιστοιχούν στα Pull Requests, ώστε τα παραγόμενα αποτελέσματα να είναι συγκρίσιμα και να μπορεί να υπολογιστεί η μεταβολή των αντίστοιχων μετρικών. Η στατική ανάλυση δεν αντιμετωπίζεται ως απόλυτη και μοναδική μέτρηση της συνολικής ποιότητας και ασφάλειας ενός συστήματος, αλλά ως μία συστηματική και επαναλήψιμη μέθοδος παραγωγής παρατηρήσιμων δεικτών, οι οποίοι χρησιμοποιούνται στη συνέχεια ως δεδομένα για την εμπειρική και στατιστική διερεύνηση της μεταξύ τους αλληλεπίδρασης.

2.5.2 Δυναμική ανάλυση (Dynamic Analysis)

Η δυναμική ανάλυση λογισμικού (Dynamic Analysis) αναφέρεται στην εξέταση ενός προγράμματος κατά τη διάρκεια της πραγματικής ή προσομοιωμένης εκτέλεσής του, σε απομονωμένο συνθήτως περιβάλλον. Σε αντίθεση με τη στατική ανάλυση, η οποία εξετάζει τον κώδικα χωρίς να τον εκτελεί, η δυναμική ανάλυση βασίζεται στην παρατήρηση της συμπεριφοράς του λογισμικού καθώς αυτό δέχεται συγκεκριμένες εισόδους και αλληλοεπιδρά με το περιβάλλον εκτέλεσής του. Ο NIST οργανισμός ορίζει τη dynamic analysis ως διαδικασία κατά την οποία ένα πρόγραμμα εκτελείται με ένα σύνολο περιπτώσεων εισόδου και στη συνέχεια αναλύεται η συμπεριφορά του κατά τον χρόνο εκτέλεσης. Αντίστοιχα, ένας dynamic code analyzer μπορεί να εκτελεί το λογισμικό σε πραγματικό ή εικονικό επεξεργαστή, να παρατηρεί τη λειτουργία του και να εξετάζει τις αποκρίσεις που παράγει.

Η δυναμική ανάλυση βασίζεται στην παρατήρηση της συμπεριφοράς του λογισμικού κατά την εκτέλεσή του. Εξετάζει πραγματικές αποκρίσεις και καταστάσεις λειτουργίας, γι' αυτό συμπληρώνει τη στατική ανάλυση χωρίς να την υποκαθιστά (Scarfione et al., 2008; Software Engineering Institute, χ.χ.). Το Software Engineering Institute επισημαίνει σε αντίστοιχη αναφορά του ότι η δυναμική ανάλυση μπορεί να αποκαλύψει runtime dependencies και άλλες πληροφορίες που δεν είναι διαθέσιμες μέσω αποκλειστικά στατικών, parsing-based προσεγγίσεων.

Βασικό πλεονέκτημα της δυναμικής ανάλυσης είναι ότι εξετάζει την πραγματική συμπεριφορά του λογισμικού σε συγκεκριμένες συνθήκες εκτέλεσης. Όταν ένα πρόβλημα εκδηλώνεται κατά την εκτέλεση και παρατηρείται άμεσα, το αντίστοιχο αποτέλεσμα παρέχει ισχυρή ένδειξη ότι η συμπεριφορά είναι πραγματική και όχι απλώς πιθανή θεωρητικά -συνήθως το θεωρητικό αυτό συμπέρασμα εξάγεται από την δομή και συγγραφή του κώδικα-. Το NIST έχει επισημάνει ότι η δυναμική δοκιμή παρουσιάζει γενικά μικρότερο πρόβλημα false positives σε σχέση με προσεγγίσεις οι οποίες προσπαθούν να προβλέψουν πιθανές συμπεριφορές χωρίς εκτέλεση. Επιπλέον, η ανάλυση μπορεί να εφαρμοστεί σε λογισμικό για το οποίο δεν κρίνεται απαραίτητη η διαθεσιμότητα του πλήρους πηγαίου κώδικα, εφόσον είναι δυνατή η εκτέλεση και παρατήρησή του.

Αντίθετα, η δυναμική ανάλυση παρουσιάζει και έναν αρκετά σημαντικό περιορισμό ο οποίος υπόκειται στην εξέταση μόνο των συμπεριφορών που ενεργοποιούνται από τα συγκεκριμένα σενάρια εκτέλεσης και τις εισόδους οι οποίες χρησιμοποιούνται. Ένα πρόγραμμα μπορεί να διαθέτει πολύ μεγάλο αριθμό πιθανών

εκτελούμενων ροών και διαδρομών εκτέλεσης (branching), γεγονός το οποίο καθιστά πρακτικά αδύνατη την πλήρη εξέτασή τους. Η απουσία ενός προβλήματος από τα αποτελέσματα μιας δυναμικής ανάλυσης δεν αποδεικνύει, επομένως, ότι το πρόβλημα δεν υφίσταται, αλλά μόνο ότι δεν εκδηλώθηκε στα συγκεκριμένα σενάρια τα οποία τέθηκαν προς εξέταση. Σε πρότυπα της βιομηχανίας αναφέρεται ότι η πλήρης κάλυψη όλων των πιθανών συμπεριφορών εκτέλεσης μέσω δυναμικής ανάλυσης είναι μη ρεαλιστική, ενώ η αποτελεσματικότητα της διαδικασίας εξαρτάται σημαντικά από την επιλογή κατάλληλων test cases και εισόδων.

Η δυναμική και η στατική ανάλυση πρέπει συνεπώς να αντιμετωπίζονται ως συμπληρωματικές μεταξύ τους προσεγγίσεις και όχι ως εναλλακτικές που αλληλοαποκλείονται ή ανεξάρτητες. Η στατική ανάλυση μπορεί να εξετάσει μεγάλο μέρος της δομής του κώδικα και να εντοπίσει πιθανά προβλήματα πριν από την εκτέλεσή του, ενώ η δυναμική ανάλυση δύναται να επιβεβαιώσει ή να αποκαλύψει προβλήματα που εκδηλώνονται μόνο κατά τη λειτουργία του συστήματος. Η OWASP έχει επισημάνει ότι η ανάλυση κατά τον χρόνο εκτέλεσης μπορεί να εντοπίσει χαρακτηριστικά και αδυναμίες που δεν είναι διαθέσιμα σε μία αποκλειστικά στατική προσέγγιση. Στο πλαίσιο της παρούσας έρευνας, η διάκριση αυτή είναι σημαντική για την κατανόηση του ευρύτερου οικοσυστήματος αυτοματοποιημένης ανάλυσης, παρότι η ίδια η εμπειρική ανάλυση βασίζεται κυρίως σε μετρικές οι οποίες έχουν προκύψει με την χρήση στατικής ανάλυσης του πηγαίου κώδικα.

2.5.3 Αυτοματοποιημένες Δοκιμές (Automated Testing)

Οι Αυτοματοποιημένες Δοκιμές (Automated Testing) αναφέρονται στη χρήση λογισμικού για την εκτέλεση ή την υποστήριξη δραστηριοτήτων ελέγχου ενός λογισμικού/συστήματος, όπως η προετοιμασία των δοκιμών, η εκτέλεση προκαθορισμένων περιπτώσεων δοκιμών (test cases) και η σύγκριση των πραγματικών αποτελεσμάτων με τα αναμενόμενα αποτελέσματα και συμπεριφορά του. Η αυτοματοποίηση μπορεί να αφορά την εκτέλεση των tests, δραστηριότητες όπως ο σχεδιασμός, η διαχείριση και ο έλεγχος των αποτελεσμάτων τους. Στην πιο συνηθισμένη μορφή της, ένα αυτοματοποιημένο test καθορίζει μία αρχική κατάσταση και συγκεκριμένες εισόδους, εκτελεί το υπό εξέταση τμήμα του λογισμικού και ελέγχει εάν η πραγματική συμπεριφορά συμφωνεί με την αναμενόμενη. Με τον τρόπο αυτό, αποδεκτές ή μη συμπεριφορές οι οποίες θα έπρεπε να επαληθεύονται με χειροκίνητο τρόπο μπορούν να ελέγχονται συστηματικά και με τα ίδια κριτήρια.

Τα Automated Tests μπορούν να εφαρμοστούν σε διαφορετικά επίπεδα του λογισμικού. Σε χαμηλότερο επίπεδο μπορούν να εξετάζουν μεμονωμένες μονάδες λειτουργικότητας, ενώ σε υψηλότερα επίπεδα μπορούν να αξιολογούν τη συνεργασία πολλών συστατικών (components) ή τη συμπεριφορά ενός ολοκληρωμένου συστήματος. Ιδιαίτερα σημαντικός κρίνεται επίσης ο ρόλος τους στο regression testing (δοκιμές παλινδρόμησης). Στην περίπτωση αυτή, επανεκτελούνται τα ήδη υπάρχοντα tests μετά από μία τροποποίηση με σκοπό την διαπίστωση της αναμενόμενης συμπεριφοράς των λειτουργιών οι οποίες πριν την

τροποποίηση λειτουργούσαν ορθά. Η δυνατότητα αυτή είναι κρίσιμη σε συστήματα τα οποία εξελίσσονται συνεχώς, καθώς ακόμη και μία περιορισμένη αλλαγή μπορεί να προκαλέσει ανεπιθύμητες παρενέργειες σε άλλα τμήματα του λογισμικού.

Η επαναληψιμότητα αποτελεί ένα από τα βασικότερα πλεονεκτήματα του automated testing. Το ίδιο σύνολο δοκιμών μπορεί να εκτελείται πολλαπλές φορές και με συνεπή τρόπο χωρίς να απαιτείται αντίστοιχη επανάληψη ανθρώπινης εργασίας. Η αυτοματοποίηση καθιστά ιδιαίτερα πρακτική την εκτέλεση tests σε κάθε σημαντική αλλαγή του κώδικα, όπως μετά από ένα commit ή στο γενικότερο πλαίσιο ενός Pull Request, παρέχοντας γρήγορη ανατροφοδότηση σχετικά με την ικανοποίηση των υπάρχουσών προσδοκιών από τη νέα κατάσταση λογισμικού.

Παρά τα σημαντικά πλεονεκτήματα που μπορούν να προσδώσουν, οι αυτοματοποιημένες δοκιμές δεν αποτελούν αυτόνομα εγγύηση ορθότητας ή υψηλής ποιότητας του λογισμικού. Η αποτελεσματικότητά τους εξαρτάται από την ποιότητα των test cases, την έκταση των συμπεριφορών που καλύπτουν, την ορθότητα των αναμενόμενων αποτελεσμάτων και τη συνεχή συντήρησή τους καθώς το σύστημα τροποποιείται, αναπτύσσεται και εξελίσσεται. Μία σουίτα δοκιμών μπορεί να εκτελείται επιτυχώς αλλά χωρίς να εξετάζει κρίσιμες περιπτώσεις του συστήματος ή μη αναμενόμενες διαδρομές εκτέλεσης. Επιπλέον, η αυτοματοποίηση δημιουργεί με την σειρά της τεχνικό κόστος που να αφορά σε αυτή, καθώς τα scripts, τα δεδομένα δοκιμών και η σχετική υποδομή πρέπει να προσαρμόζονται στις αλλαγές του λογισμικού. Για τον λόγο αυτό, η αυτοματοποίηση δεν εξαλείφει πλήρως την ανάγκη για κατάλληλο σχεδιασμό των δοκιμών και ανθρώπινη κρίση, λειτουργεί ως μέσο για την αποτελεσματικότερη και συχνότερη εφαρμογή τους.

Στη σύγχρονη ανάπτυξη λογισμικού, τα automated tests ενσωματώνονται συχνά σε αυτοματοποιημένες ροές ανάπτυξης και λειτουργούν ως μηχανισμός ελέγχου πριν μία αλλαγή προχωρήσει σε επόμενο στάδιο. Η πρακτική αυτή επιτρέπει τον γρήγορο εντοπισμό παλινδρομήσεων και υποστηρίζει τη συνεχή επαλήθευση του συστήματος καθώς ο πηγαίος κώδικας μεταβάλλεται. Στο πλαίσιο το οποίο η περιγραφόμενη έρευνα δημιουργεί, οι αυτοματοποιημένες δοκιμές αποτελούν σημαντικό στοιχείο του ευρύτερου περιβάλλοντος μέσα στο οποίο πραγματοποιούνται και αξιολογούνται τα PRs. Διακρίνονται από τις μετρικές λογισμικού που χρησιμοποιούνται στην εμπειρική ανάλυση, καθώς οι πρώτες εξετάζουν κυρίως εάν συγκεκριμένες αναμενόμενες συμπεριφορές εξακολουθούν να ικανοποιούνται, ενώ οι δεύτερες χρησιμοποιούνται για την ποσοτική αποτύπωση χαρακτηριστικών των συνιστωσών της ποιότητας και της ασφάλειας του πηγαίου κώδικα και του λογισμικού.

2.6 Συνεχής Ενοποίηση/Ενσωμάτωση και Συνεχής Παράδοση (CI/CD), DevOps, DevSecOps

Η Continuous Integration οργανώνει τη συχνή ενσωμάτωση αλλαγών και την αυτοματοποιημένη επαλήθευσή τους, ενώ οι πρακτικές Continuous Delivery και

Continuous Deployment επεκτείνουν τη ροή προς την προετοιμασία ή και την αυτόματη παραγωγική διάθεση εκδόσεων (Shahin, Babar & Zhu, 2017; Hilton et al., 2016). Η συχνή ενσωμάτωση μικρότερων αλλαγών περιορίζει τη συσσώρευση διαφορών μεταξύ των branches ενός repository, διευκολύνει τον έγκαιρο εντοπισμό προβλημάτων ενοποίησης και παρέχει γρηγορότερη ανατροφοδότηση σχετικά με την επίδραση μιας αλλαγής στο γενικότερο υπόλοιπο σύστημα.

Η Συνεχής Παράδοση (Continuous Delivery – CD) επεκτείνει τη λογική που περιεγράφηκε προηγουμένως, του CI, πέρα από την ενσωμάτωση του κώδικα. Οι αλλαγές που περνούν επιτυχώς από τα προηγούμενα στάδια προετοιμάζονται αυτοματοποιημένα για πιθανή διάθεση σε κατάσταση παραγωγής (production), με στόχο να υπάρχει ανά πάσα στιγμή μία έκδοση του λογισμικού που μπορεί να διανεμηθεί με αξιόπιστο και επαναλαμβανόμενο τρόπο. Η διάκριση μεταξύ των Continuous Delivery και Continuous Deployment κρίνεται αναγκαία. Στην πρώτη περίπτωση η εφαρμογή διατηρείται σε κατάσταση κατάλληλη για release, αλλά η τελική απόφαση διάθεσης μπορεί να παραμένει ανθρώπινη. Αντίθετα, στη διαδικασία της Continuous Deployment, μία αλλαγή που ικανοποιεί όλα τα κριτήρια του pipeline μπορεί να προωθείται αυτοματοποιημένα μέχρι το production χωρίς υποχρεωτική χειροκίνητη έγκριση ή εκ νέου ρύθμιση. Ο όρος CI/CD, συνεπώς, χρησιμοποιείται συχνά ως γενικός όρος για ένα σύνολο αυτοματοποιημένων διαδικασιών που συνδέουν την ενσωμάτωση του κώδικα με την προετοιμασία και, σύμφωνα με το περιβάλλον, την τελική ανάπτυξη του.

Η πρακτική εφαρμογή των τεχνικών CI/CD πραγματοποιείται συνήθως μέσω ενός pipeline, δηλαδή μιας οργανωμένης ακολουθίας σταδίων μέσα από τα οποία διέρχεται μία αλλαγή. Ανάλογα με τη φύση του εκάστοτε συστήματος, ένα pipeline μπορεί να περιλαμβάνει στάδια δημιουργίας λογισμικού, επαλήθευσης, δημιουργίας, artifacts, προετοιμασίας μιας έκδοσης και ανάπτυξης της σε διαφορετικά περιβάλλοντα. Κάθε στάδιο μπορεί να λειτουργεί ως σημείο ελέγχου, προκειμένου μία αποτυχία να εμποδίζει την αλλαγή να προχωρήσει και να επιστρέφει σχετική πληροφορία στον προγραμματιστή. Τα CI/CD pipelines περιγράφονται ως αυτοματοποιημένα συστήματα που ενορχηστρώνουν δραστηριότητες build, ενσωμάτωσης, delivery και deployment και παράγουν παράλληλα στοιχεία που χρησιμοποιούνται ως συνεχής ανατροφοδότηση κατά τη διάρκεια του κύκλου ανάπτυξης του λογισμικού.

Οι πρακτικές αυτές αποτελούν βασικό στοιχείο του ευρύτερου μοντέλου DevOps (Development and Operations – Ανάπτυξη και Διαδικασίες). Το DevOps δεν αποτελεί ένα συγκεκριμένο εργαλείο και δεν ταυτίζεται αποκλειστικά με την αυτοματοποίηση ή την διαχείριση του CI/CD. Πρόκειται για ένα σύνολο οργανωτικών και τεχνικών πρακτικών που επιδιώκουν να περιορίσουν τον παραδοσιακό διαχωρισμό μεταξύ των ομάδων ανάπτυξης και λειτουργίας του λογισμικού. Κεντρικές αρχές του είναι η συνεργασία, η αυτοματοποίηση επαναλαμβανόμενων διαδικασιών, η συνεχής ανατροφοδότηση και η δυνατότητα συχνότερης και πιο αξιόπιστης παράδοσης μεταβολών στον πηγαίο κώδικα του λογισμικού. Βιβλιογραφικά, το DevOps ορίζεται ως σύνολο πρακτικών που αυτοματοποιούν και συνδέουν τις διαδικασίες μεταξύ της ανάπτυξης λογισμικού και των διαδικασιών IT, με τελικό και συνεχή στόχο τη συντόμευση του κύκλου ανάπτυξης και την αξιόπιστη

και συχνή παράδοση νέων -συνήθως μικρότερων- λειτουργιών, διορθώσεων και ενημερώσεων.

Η επέκταση της φιλοσοφίας αυτής με στόχο η ασφάλεια να αποτελέσει ενσωματωμένο μέρος της διαδικασίας ανάπτυξης έχει οδηγήσει στην δημιουργία του DevSecOps (Development, Security and Operations – Ανάπτυξη, Ασφάλεια και Διαδικασίες). Η βασική διαφοροποίηση είναι ότι οι δραστηριότητες ασφάλειας δεν αντιμετωπίζονται ως ανεξάρτητος τελικός έλεγχος πριν από τη διάθεση του λογισμικού, αλλά ενσωματώνονται σε ολόκληρο τον κύκλο ζωής και στις αυτοματοποιημένες ροές ανάπτυξης. Έτσι η ασφάλεια ανάγεται σε κοινή ευθύνη των ομάδων ανάπτυξης, security και operations. Τα CI/CD pipelines μπορούν να λειτουργήσουν σαν βασικό σημείο ενσωμάτωσης των συγκεκριμένων ελέγχων, επιτρέποντας την εφαρμογή security policies και την αναγνώριση πιθανών προβλημάτων σε πιο πρώιμα στάδια της ανάπτυξης. Η λογική αυτή συνδέεται με την προσέγγιση shift-left, κατά την οποία η διερεύνηση και αντιμετώπιση των προβλημάτων ασφαλείας μεταφέρεται όσο το δυνατόν σε πιο αρχικό στάδιο στον κύκλο ζωής του λογισμικού.

Μολονότι η αυτοματοποίηση αποτελεί κεντρικό χαρακτηριστικό των CI/CD, DevOps και DevSecOps, η αποτελεσματικότητά τους εξαρτάται άμεσα από τον τρόπο με τον οποίο σχεδιάζονται και υλοποιούνται τα pipelines και από την ποιότητα των κριτηρίων που εφαρμόζονται. Ένα πλήρως αυτοματοποιημένο pipeline δεν αποτελεί από μόνο του εγγύηση ότι μία αλλαγή είναι ποιοτική ή ασφαλής. Ανεπαρκείς έλεγχοι ή αποτελέσματά τους που δεν μπορούν να αξιολογηθούν ορθά, δύναται να μεταφέρουν τα προβλήματα σε επόμενα στάδια, συνήθως πιο κρίσιμα για τον κύκλο ζωής του λογισμικού. Κατά συνέπεια, η αξία αυτών των πρακτικών δεν βρίσκεται αποκλειστικά στην ταχύτητα της αυτοματοποίησης, αλλά στη δυνατότητα εφαρμογής συνεπών επαναλαμβανόμενων και έγκαιρων μηχανισμών ελέγχου και ανατροφοδότησης σε ολόκληρη τη διαδικασία ανάπτυξης.

Οι παραπάνω έννοιες αποκτούν ιδιαίτερη σημασία στο πλαίσιο της παρούσας εργασίας, αφού τα PRs αποτελούν σημεία στα οποία μπορούν να ενεργοποιούνται αυτοματοποιημένοι έλεγχοι πριν από την ενσωμάτωση μίας αλλαγής -στην κύρια, συνήθως, ροή ανάπτυξης-. Η σύνδεση των εννοιών του Version Control, των Pull Requests, των CI/CD pipelines και των αυτοματοποιημένων εργαλείων ανάλυσης δημιουργεί ένα περιβάλλον στο οποίο οι μεταβολές του πηγαίου κώδικα είναι σε θέση να αξιολογούνται συστηματικά πριν την αποδοχή τους. Οι τεχνικές του DevSecOps επεκτείνουν αυτή τη λογική ώστε η ποιότητα και η ασφάλεια να μην αποτελούν ανεξάρτητες δραστηριότητες στο τέλος της ανάπτυξης, αλλά στοιχεία που μπορούν να παρακολουθούνται παράλληλα σε όλη τη διαδικασία εξέλιξης του λογισμικού. Η συγκεκριμένη σύνδεση, επίσης, σημαντικό θεωρητικό υπόβαθρο για τη μεταγενέστερη διερεύνηση της αλληλεπίδρασης της ποιότητας και ασφάλειας στο επίπεδο των τροποποιήσεων του πηγαίου κώδικα.

Ποιότητα Λογισμικού – Software Quality

Η Ποιότητα Λογισμικού αποτελεί μία από τις βασικότερες διαστάσεις της ανάπτυξης και της συνεχούς συντήρησης ενός συστήματος λογισμικού και αφορά τον βαθμό στον οποίο αυτό ικανοποιεί τις απαιτήσεις, τις ανάγκες των χρηστών και τα αναμενόμενα χαρακτηριστικά και τρόπο λειτουργίας του. Η έννοιά της δεν περιορίζεται στην απουσία σφαλμάτων καθώς περιλαμβάνει ένα ευρύτερο σύνολο ιδιοτήτων οι οποίες σχετίζονται κατά κύριο λόγο με την αξιοπιστία, τη συντηρησιμότητα, την αποδοτικότητα και τη δυνατότητα εξέλιξης του λογισμικού.

Η αξιολόγηση της ποιότητας δύναται να πραγματοποιηθεί τόσο μέσω ποιοτικών κριτηρίων όσο και μέσω ποσοτικών μετρικών που επιτρέπουν τη συστηματική παρακολούθηση και καταγραφή της κατάστασης και της εξέλιξης του πηγαίου κώδικα. Στο παρόν κεφάλαιο παρουσιάζονται οι βασικές έννοιες, τα χαρακτηριστικά και οι παράγοντες που συνδέονται με την ποιότητα λογισμικού, δημιουργώντας το θεωρητικό υπόβαθρο για τη μετέπειτα διερεύνηση της σχέσης και της αλληλεπίδρασής της με την ασφάλεια.

3.1 Έννοια και μοντέλα ποιότητας

Η Ποιότητα Λογισμικού (Software Quality) αποτελεί μία πολυδιάστατη έννοια και δεν μπορεί να περιγραφεί επαρκώς μόνο μέσα από την απουσία σφαλμάτων ή την ορθή λειτουργία ενός συστήματος. Στο σύνολό της αφορά τα χαρακτηριστικά τα οποία προσδιορίζουν κατά πόσο ένα λογισμικό ανταποκρίνεται στις απαιτήσεις και στους στόχους για τους οποίους σχεδιάστηκε και αναπτύχθηκε. Ιδιότητες όπως είναι η αξιοπιστία, η συντηρησιμότητα, η αποδοτικότητα, η χρηστικότητα και η ασφάλεια αποτελούν διαφορετικές διαστάσεις/μετρήσεις της συνολικής ποιότητας. Ως εκ τούτου, δύο συστήματα τα οποία παρέχουν την ίδια λειτουργικότητα μπορούν να διαφέρουν σημαντικά ως προς την ποιότητά τους.

Η ανάγκη συστηματικής περιγραφής και αξιολόγησης της έννοιας της ποιότητας έχει οδηγήσει στην ανάπτυξη των μοντέλων ποιότητας (quality models). Τα μοντέλα αυτά οργανώνουν την ποιότητα σε επιμέρους χαρακτηριστικά και υποχαρακτηριστικά, προκειμένου μία αφηρημένη να μπορεί να συνδεθεί με περισσότερο συγκεκριμένες και δυνητικά μετρήσιμες ιδιότητες. Ιστορικά, σημαντικές προσεγγίσεις μοντέλων ποιότητας αποτέλεσαν τα μοντέλα των McCall και Boehm, ενώ στην συνέχεια ακολούθησαν μοντέλα όπως τα FURPS και Dromey. Κοινή παραδοχή των μοντέλων αποτελεί η πεποίθηση ότι η συνολική ποιότητα δεν είναι δυνατόν να αποτυπωθεί επαρκώς και ολοκληρωμένα από μία μοναδική μετρική, αλλά απαιτεί την εξέταση και τον συνδυασμών πολλαπλών χαρακτηριστικών.

Το ISO/IEC 25010:2023 περιγράφει την ποιότητα προϊόντος μέσω εννέα χαρακτηριστικών: functional suitability, performance efficiency, compatibility,

interaction capability, reliability, security, maintainability, flexibility και safety (ISO/IEC, 2023). Τα χαρακτηριστικά αυτά αναλύονται περαιτέρω σε υποχαρακτηριστικά και παρέχουν κοινό πλαίσιο για τον καθορισμό απαιτήσεων και την αξιολόγηση της ποιότητας ενός προϊόντος λογισμικού.

Τα μοντέλα ποιότητας δεν αποσκοπούν στη δημιουργία μίας απόλυτης και καθολικής βαθμολογίας ποιότητας. Λειτουργούν ως εννοιολογικά πλαίσια που καθορίζουν ποιες διαστάσεις ενός συστήματος πρέπει να εξετάζονται. Η σημασία κάθε χαρακτηριστικού είναι δυνατόν να διαφοροποιείται ανάλογα με το είδος, τον σκοπό και το περιβάλλον λειτουργίας του εκάστοτε λογισμικού. Στην παρούσα εργασία, η ποιότητα αντιμετωπίζεται ως πολυδιάστατη έννοια και προσδιορίζεται και εξετάζεται μέσω διαφορετικών μετρικών, ώστε να καταστεί δυνατή η μελέτη των μεταβολών της και της σχέσης τους με αντίστοιχες μεταβολές στην ασφάλεια.

3.1.1 Ποιότητα Προϊόντος (Λογισμικού) και Ποιότητα Κώδικα

Ο όρος Ποιότητα Προϊόντος Λογισμικού (Software Product Quality) αναφέρεται στο σύνολο των χαρακτηριστικών που καθορίζουν τον βαθμό ικανοποίησης των απαιτήσεων και των στόχων για τους οποίους έχει αναπτυχθεί ένα λογισμικό από το ίδιο το προϊόν λογισμικού. Πρόκειται για μία ευρεία και πολυδιάστατη έννοια, η οποία δεν περιορίζεται αποκλειστικά στην ορθότητα του πηγαίου κώδικα ή στην απουσία σφαλμάτων, αλλά περιλαμβάνει διαφορετικές ιδιότητες του συστήματος. Το πρότυπο ISO/IEC 25010:2023 ορίζει ένα μοντέλο ποιότητας προϊόντος αποτελούμενο από εννέα βασικά χαρακτηριστικά και τα αντίστοιχα υποχαρακτηριστικά τους, όπως αυτό προαναφέρθηκε και επεξηγήθηκε, παρέχοντας ένα κοινό πλαίσιο για τον καθορισμό, τη μέτρηση και την αξιολόγηση του συνόλου της ποιότητας προϊόντων λογισμικού και γενικότερα ICT προϊόντων.

Στις διαστάσεις αυτές περιλαμβάνεται, μεταξύ άλλων, η αποδοτικότητα επίδοσης (Performance Efficiency), η οποία αφορά την ικανότητα του συστήματος να εκτελεί τις λειτουργίες του με κατάλληλη χρονική συμπεριφορά, χωρητικότητα και χρήση διαθέσιμων πόρων, καθώς επίσης και η συμβατότητα (Compatibility), που σχετίζεται άμεσα με τη δυνατότητα συνύπαρξης και ανταλλαγής πληροφοριών με άλλα συστήματα. Σημαντική κρίνεται επίσης και η δυνατότητα αποτελεσματικής αλληλεπίδρασης του ίδιου του χρήστη με το προϊόν λογισμικού. Η συγκεκριμένη διάσταση πλέον έχει διευρυνθεί και ονομασθεί ως Interaction Capability (η ορολογία αυτή προέρχεται από το ενημερωμένο πρότυπο ISO/IEC 25010:2023), ενώ σε προηγούμενα πρότυπα, συγκεκριμένα στο ISO/IEC 25010:2011, είχε οριστεί ως Usability. Αντίστοιχα, η έννοια της δυνατότητας μεταφοράς και προσαρμογής του λογισμικού σε διάφορα και διαφορετικά περιβάλλοντα, η επονομαζόμενη ως Φορητότητα (Portability), έχει πλέον ενσωματωθεί στην ευρύτερη έννοια της Flexibility, η οποία σήμερα περιλαμβάνει επίσης και χαρακτηριστικά όπως είναι η προσαρμοστικότητα και η δυνατότητα τόσο οριζόντιας όσο και κάθετης κλιμάκωσης του λογισμικού. Οι μεταβολές αυτές στην ορολογία αντανakλούν άμεσα την δυναμική και συνεχή εξέλιξη του μοντέλου ώστε αυτό να καλύπτει πληρέστερα τις απαιτήσεις των σύγχρονων συστημάτων λογισμικού.

Αντίθετα, η έννοια της Ποιότητας Κώδικα (Code Quality ή Source Code Quality) εστιάζει κυρίως στην τεχνική υλοποίηση του συστήματος και στις ιδιότητες του πηγαίου κώδικα. Αφορά χαρακτηριστικά τα οποία επηρεάζουν τον βαθμό στον οποίο ο κώδικας μπορεί να γίνει κατανοητός, να αναλυθεί (με ή χωρίς την χρήση βοηθητικών εργαλείων), να ελεγχθεί, να τροποποιηθεί και να συντηρηθεί. Η συγκεκριμένη έννοια είναι επίσης πολυδιάστατη και δεν καθίσταται δυνατό να αποτυπωθεί πλήρως μέσω μιας μοναδικής μετρικής. Εμπειρική έρευνα σχετικά με τον τρόπο με τον οποίο οι προγραμματιστές αντιλαμβάνονται την ποιότητα κώδικα έχει εξάγει αποτελέσματα ότι χαρακτηριστικά όπως είναι η δομή, η αναγνωσιμότητα και η κατανοησιμότητα βρίσκονται μεταξύ των σημαντικότερων ιδιοτήτων που συνδέονται με κώδικα υψηλής ποιότητας. Παράλληλα, έχει επισημανθεί στη βιβλιογραφία ότι δεν υπάρχει πλήρης και ξεκάθαρη αντιστοίχιση μεταξύ των διαθέσιμων source-code metrics και όλων των αφηρημένων χαρακτηριστικών που οι προγραμματιστές αντιλαμβάνονται ως ποιότητα.

Η διάκριση μεταξύ των εννοιών της Ποιότητας Προϊόντος και Ποιότητας Κώδικα αποτυπώνεται στα διεθνή πρότυπα μέτρησης. Σε αντίθεση με το ISO/IEC 5055:2021 το οποίο επικεντρώνεται ειδικότερα στην αυτοματοποιημένη μέτρηση της ποιότητας σε επίπεδο πηγαίου κώδικα, το ISO/IEC 25010:2023 παρέχει ένα ευρύτερο μοντέλο για την ποιότητα του προϊόντος. Το πρώτο εξ αυτών πρότυπο βασίζεται στον εντοπισμό και την καταμέτρηση παραβιάσεων αρχιτεκτονικών και προγραμματιστικών τεχνικών, οι οποίες είναι πιθανό να οδηγήσουν σε λειτουργικούς κινδύνους ή αυξημένο κόστος. Το ίδιο το ISO/IEC 5055 επισημαίνει ότι τα ευρύτερα πρότυπα της οικογένειας ISO/IEC 25000 παρέχουν σχετικά περιορισμένο αριθμό μετρήσεων σε επίπεδο source code, γεγονός το οποίο δικαιολογεί την ανάγκη ύπαρξης ειδικότερων προτύπων για την αξιολόγηση της τεχνικής υλοποίησης.

Ως εκ τούτου, οι δύο παραπάνω έννοιες είναι στενά συνδεδεμένες, χωρίς να ταυτίζονται και να αλληλοκαλύπτονται. Η ποιότητα του πηγαίου κώδικα μπορεί να επηρεάσει σημαντικά χαρακτηριστικά του συνολικού προϊόντος, όμως δεν κρίνεται επαρκής από μόνη της να αξιολογήσει τα πλήρεις χαρακτηριστικά όλων των διαστάσεων της ποιότητας. Για παράδειγμα, ένα σύστημα μπορεί να διαθέτει πολύ οργανωμένο πηγαίο κώδικα ο οποίος είναι άμεσα συντηρήσιμος, αλλά να παρουσιάζει ανεπαρκή απόδοση, προβλήματα συμβατότητας ή δυσκολίες αλληλεπίδρασης με τους χρήστες του. Αντίστροφα, ένα προϊόν μπορεί να εμφανίζει ικανοποιητική συμπεριφορά προς τον τελικό χρήστη, ενώ ο πηγαίος κώδικας στον οποίο το λογισμικό βασίζεται παρουσιάζει αυξημένη πολυπλοκότητα ή άλλες τεχνικές αδυναμίες οι οποίες δυσχεραίνουν τη μελλοντική εξέλιξη και συντήρησή του.

Η συγκεκριμένη διάκριση κρίνεται ιδιαίτερα σημαντική στο πλαίσιο της παρούσας εργασίας, καθώς η εμπειρική αξιολόγηση βασίζεται σε μεγάλο βαθμό σε μετρικές οι οποίες εξάγονται από τον πηγαίο κώδικα και από την αυτοματοποιημένη ανάλυσή του. Οι μετρικές αυτές παρέχουν ποσοτικές ενδείξεις και πληροφορίες για συγκεκριμένα χαρακτηριστικά της τεχνικής ποιότητας και επιτρέπουν τη μελέτη της εξέλιξής τους μετά από μεταβολές στον πηγαίο κώδικα. Η ερμηνεία τους, όμως, δεν θα πρέπει να αντιστοιχεί σε μία πλήρης και απόλυτη μέτρηση της συνολικής ποιότητας του προϊόντος, καθώς χαρακτηριστικά όπως η απόδοση, η συμβατότητα ή

η αλληλεπίδραση με τον χρήστη μπορεί να απαιτούν διαφορετικούς τρόπους αξιολόγησης.

3.1.2 Internal και External quality

Η ποιότητα ενός συστήματος λογισμικού είναι δυνατόν να εξεταστεί από διαφορετικές οπτικές, σύμφωνα με το κριτήριο εάν η αξιολόγηση επικεντρώνεται στις ιδιότητες του ίδιου του λογισμικού ή στη συμπεριφορά που αυτό παρουσιάζει κατά την εκτέλεσή του. Στο πλαίσιο αυτό, στη βιβλιογραφία της Software Engineering έχει καθιερωθεί η διάκριση μεταξύ εσωτερικής ποιότητας (Internal Quality) και εξωτερικής ποιότητας (External Quality). Η διάκριση μεταξύ αυτών παρουσιάστηκε συστηματικά στο πλαίσιο του προτύπου ISO/IEC 9126, όπου οι εσωτερικές μετρικές χρησιμοποιούνται για τη μέτρηση ιδιοτήτων του ίδιου του λογισμικού, ενώ οι εξωτερικές μετρικές αφορούν τη συμπεριφορά του συστήματος κατά την εκτέλεσή τους.

Η εσωτερική ποιότητα αφορά χαρακτηριστικά τα οποία μπορούν να εξεταστούν χωρίς να απαιτείται απαραίτητα η ενεργητική εκτέλεση του συστήματος. Συνδέεται κυρίως με στατικές ιδιότητες της υλοποίησης, όπως η δομή και η οργάνωση του πηγαίου κώδικα, οι εξαρτήσεις μεταξύ των επιμέρους στοιχείων του πηγαίου κώδικα (ή άλλων υπηρεσιών) και γενικότερα χαρακτηριστικά που μπορούν να παρατηρηθούν ή να μετρηθούν μέσω της εξέτασης των artifacts του λογισμικού. Οι μετρικές, άρα, που προκύπτουν από τον πηγαίο κώδικα μπορούν να χρησιμοποιηθούν ως ενδείξεις της εσωτερικής ποιότητας. Θα πρέπει, όμως, να αποφευχθεί η απόλυτη ταύτιση της έννοιας αυτής με την συνολική ποιότητα του κώδικα, καθώς η εσωτερική ποιότητα αποτελεί ευρύτερη οπτική πάνω στις στατικές ιδιότητες του προϊόντος, ενώ η ποιότητα κώδικα επικεντρώνεται ειδικότερα στην τεχνική υλοποίησή του.

Η εξωτερική ποιότητα, αντίθετα με την προαναφερθείσα, αφορά τις ιδιότητες και τη συμπεριφορά που παρατηρούνται όταν το λογισμικό εκτελείται. Η αξιολόγησή της απαιτεί την παρατήρηση του εκτελούντος συστήματος υπό συγκεκριμένες συνθήκες και μπορεί να σχετίζεται με στοιχεία όπως η πραγματική λειτουργική συμπεριφορά, η απόδοση, η αξιοπιστία, ή άλλες ιδιότητες που εκδηλώνονται κατά τον χρόνο εκτέλεσης. Στο παλαιότερο πρότυπο ISO/IEC 9126, οι εξωτερικές μετρικές ορίζονταν ακριβώς ως μετρήσεις συμπεριφοράς του computer-based system που περιλαμβάνει το λογισμικό. Η διάκριση αυτή εξακολουθεί να κατέχει χρήσιμη εννοιολογική σημασία, ακόμη και αν τα νεότερα πρότυπα SQuaRE οργανώνουν πλέον τις μετρήσεις μέσα σε ένα ευρύτερο product quality framework.

Οι δύο αυτές μορφές ποιότητας δεν είναι ανεξάρτητες, χωρίς όμως να ταυτίζονται. Οι εσωτερικές ιδιότητες υλοποίησης είναι ικανές να επηρεάσουν τη συμπεριφορά που παρουσιάζει τελικά το λογισμικό, χωρίς όμως να υπάρχει πάντα μία απλή ή μονοσήμαντη αντιστοίχιση μεταξύ μιας εξωτερικής μετρικής και ενός εξωτερικού αποτελέσματος. Εμπειρικές μελέτες με εφαρμογή στην Σύγχρονη Ανάπτυξη Λογισμικού έχουν εξετάσει ακριβώς αυτή τη σχέση, χρησιμοποιώντας source-code metrics για την εσωτερική ποιότητα και παρατηρήσιμα αποτελέσματα

ή άλλα εξωτερικά δεδομένα για την εξωτερική ποιότητα. Τα αποτελέσματά τους καταδεικνύουν ότι ορισμένες σχέσεις μπορούν να εντοπιστούν, αλλά η διαθέσιμη βιβλιογραφία δεν υποστηρίζει ότι κάθε εξωτερικό χαρακτηριστικό προβλέπει με συνέπεια κάποιο αντίστοιχο εξωτερικό χαρακτηριστικό. Σε πρόσφατη tertiary study έχει επισημανθεί ότι τα ευρήματα σχετικά με τις σχέσεις source-code metrics και external quality attributes είναι σε ορισμένες περιπτώσεις αντιφατικά.

Η διάκριση που αναπτύχθηκε παραπάνω, κρίνεται αναγκαία για την πλήρη κατανόηση του πλαισίου της παρούσας έρευνας. Οι περισσότερες μετρικές που χρησιμοποιούνται στην εμπειρική ανάλυση προκύπτουν από στατική ανάλυση του πηγαίου κώδικα και επομένως αποτυπώνουν κατά κύριο λόγο εσωτερικά, τεχνικά χαρακτηριστικά των λογισμικών που εξετάστηκαν. Επομένως, τα αποτελέσματά τους δεν πρέπει να ερμηνεύονται αυτόματα ως πλήρης μέτρηση της συμπεριφοράς ή της συνολικής ποιότητας του εκτελούμενου συστήματος. Η διάκριση αυτή επιτρέπει ακριβέστερη ερμηνεία των ευρημάτων και αποφεύγει την εξαγωγή συμπερασμάτων για εξωτερικές ιδιότητες που δεν έχουν μετρηθεί άμεσα.

3.2 Συντηρησιμότητα (Maintainability) και Αξιοπιστία (Reliability)

Στο μοντέλο ISO/IEC 25010, η maintainability αφορά την αποτελεσματικότητα με την οποία ένα προϊόν μπορεί να τροποποιηθεί, ενώ η reliability αφορά τη διατήρηση καθορισμένων λειτουργιών υπό καθορισμένες συνθήκες και για καθορισμένη χρονική περίοδο (ISO/IEC, 2023). Οι τροποποιήσεις που συνδέονται με τη maintainability μπορεί να περιλαμβάνουν διορθώσεις, βελτιώσεις και προσαρμογές σε νέες απαιτήσεις ή μεταβολές του περιβάλλοντος λειτουργίας. Η έννοια είναι ιδιαίτερα σημαντική, επειδή το λογισμικό εξελίσσεται διαρκώς και σημαντικό μέρος του κύκλου ζωής του αφορά την κατανόηση και την τροποποίηση υπάρχοντος κώδικα.

Στο πρότυπο ISO/IEC 25010:2023 διακρίνονται πέντε υποχαρακτηριστικά της συντηρησιμότητας: modularity, reusability, analyzability, modifiability και testability. Η modularity περιγράφει τον διαχωρισμό του συστήματος λογισμικού σε διακριτά στοιχεία ώστε μία αλλαγή να έχει όσο το δυνατόν μικρότερη επίδραση σε άλλα μέρη του, ενώ η reusability αφορά τη δυνατότητα επαναχρησιμοποίησης του λογισμικού. Η analyzability σχετίζεται άμεσα με την ευκολία αξιολόγησης των συνεπειών μιας αλλαγής ή εντοπισμού της αιτίας ενός προβλήματος. Η έννοια της modifiability συναρτάται με τη δυνατότητα πραγματοποίησης αλλαγών χωρίς ανεπιθύμητη υποβάθμιση της ποιότητας του πηγαίου κώδικα και του ίδιου του λογισμικού, ενώ η testability με την ευκολία καθορισμού και εκτέλεσης κατάλληλων δοκιμών. Τα υποχαρακτηριστικά αυτά δείχνουν ότι η συντηρησιμότητα δεν αφορά απλώς και αποκλειστικά την δυνατότητα του κώδικα να αλλάξει, αλλά επίσης στο πόσο ελεγχόμενα, κατανοητά και αποτελεσματικά μπορεί να πραγματοποιηθεί η αλλαγή.

Αντίθετα, η Αξιοπιστία (Reliability) σχετίζεται περισσότερο με την ικανότητα του προϊόντος να εκτελεί τις προβλεπόμενες λειτουργίες του υπό καθορισμένες

συνθήκες και για συγκεκριμένο χρονικό διάστημα χωρίς ανεπιθύμητες διακοπές, αποτυχίες ή εμφάνιση σφαλμάτων. Στην έκδοση 25010:2023 του προτύπου ISO/IEC η αξιοπιστία αναλύεται στις διαστάσεις faultlessness, availability, fault tolerance και recoverability. η faultlessness αφορά την ορθή λειτουργία του λογισμικού υπό κανονικές συνθήκες, η availability τη δυνατότητα του συστήματος να είναι διαθέσιμο όταν αυτό απαιτείται, η fault tolerance τη συνέχιση της προβλεπόμενης λειτουργίας παρά την ύπαρξη σφαλμάτων και η recoverability την ικανότητα αποκατάστασης μετά από μία αποτυχία.

Αν και πρόκειται για διαφορετικά χαρακτηριστικά, η συντηρησιμότητα και η αξιοπιστία δύναται να αλληλοεπιδρούν κατά την εξέλιξη του λογισμικού. Ένα σύστημα που είναι δύσκολο να αναλυθεί και να τροποποιηθεί μπορεί να καταστήσει περισσότερο επισφαλείς τις μελλοντικές αλλαγές και να αυξήσει την πιθανότητα εισαγωγής ανεπιθύμητων προβλημάτων. Επαναλαμβανόμενες διορθώσεις, αντίστοιχα, σε ένα δύσκολα συντηρήσιμο σύστημα μπορούν να αυξήσουν περαιτέρω την πολυπλοκότητα της συντήρησης. Η σχέση αυτή δεν είναι άμεση και καθολική. Η υψηλή συντηρησιμότητα δεν εγγυάται αυτούσια υψηλή αξιοπιστία, και αντίστροφα. Εμπειρική έρευνα έχει δείξει ότι οι σχέσεις μεταξύ χαρακτηριστικών που χρησιμοποιούνται ως ενδείξεις maintainability και reliability μπορούν να διαφοροποιούνται σημαντικά μεταξύ έργων και να εμφανίζουν ακόμη και trade-offs.

Η σημασία των δύο αυτών χαρακτηριστικών είναι ιδιαίτερη σε συστήματα που εξελίσσονται μέσω συνεχών αλλαγών στον πηγαίο κώδικα. Η παρουσία ή μη συντηρησιμότητας επηρεάζει την ευκολία και την δυνατότητα ύπαρξης ελεγχόμενου τρόπου για την πραγματοποίηση των αλλαγών, ενώ η αξιοπιστία αφορά την εξακολουθητική ορθή συμπεριφορά του συστήματος μετά την εφαρμογή των τροποποιήσεων σε αυτό. Σύγχρονες εμπειρικές μελέτες οι οποίες χρησιμοποιούν αυτοματοποιημένη στατική ανάλυση εξετάζουν τα maintainability και reliability issues ως διαφορετικές κατηγορίες προβλημάτων και δείχνουν ότι η εξέλιξή τους μπορεί να διαφέρει σημαντικά μεταξύ διαφορετικών έργων. Έτσι, συνήθως είναι προτιμότερη η μελέτη τους ως διακριτές αλλά δυνητικά σχετιζόμενες διαστάσεις της ποιότητας, αντί να συνδυάζονται σε έναν μοναδικό γενικό δείκτη.

Ως επί τω πλείστων, η διάκρισή τους εδώ είναι ουσιώδης, καθώς η συντηρησιμότητα και η αξιοπιστία αποτελούν διαφορετικές διαστάσεις της ποιότητας που μπορούν να μεταβάλλονται μετά από μία τροποποίηση του πηγαίου κώδικα. Η παρακολούθησή τους μέσω αντίστοιχων μετρικών λογισμικού επιτρέπει να εξεταστεί όχι μόνο εάν η συνολική τεχνική κατάσταση του κώδικα μεταβάλλεται, αλλά και ποια επιμέρους διάσταση της ποιότητας επηρεάζεται και προς ποια κατεύθυνση. Οι συγκεκριμένες μετρικές και οι τεχνικοί παράγοντες που σχετίζονται με αυτές πρόκειται να αναλυθούν παρακάτω.

3.3 Bugs, Defects, Faults και Failures

Οι όροι error, fault και failure δεν είναι πλήρως ισοδύναμοι. Στην ταξινόμια dependability, το fault αποτελεί την υποκείμενη αιτία, το error την εσφαλμένη εσωτερική κατάσταση και το failure την παρατηρήσιμη απόκλιση της παρεχόμενης υπηρεσίας από την ορθή λειτουργία (Avizienis et al., 2004).

Ο όρος defect περιγράφει μία ατέλεια ή απόκλιση ενός προϊόντος λογισμικού από τις απαιτήσεις ή τις προδιαγραφές του. Η ύπαρξη ενός defect μέσα στον πηγαίο κώδικα δεν είναι απαραίτητη, μπορεί να προκύπτει σε απαιτήσεις, σχεδιασμό ή άλλα artifacts της διαδικασίας ανάπτυξης και αργότερα να μεταφερθεί στην τελική υλοποίηση. Στην ορολογία του software testing, οι όροι bug και fault χρησιμοποιούνται συχνά ως συνώνυμα του defect, δηλαδή ως ατέλειες που μπορούν, όταν υπό κατάλληλες συνθήκες ενεργοποιηθούν, να οδηγήσουν σε λανθασμένη συμπεριφορά.

Εντούτοις, ο όρος fault έχει και περισσότερο εξειδικευμένη χρήση στη βιβλιογραφία της αξιοπιστίας και του fault tolerance. Το ιστορικό πρότυπο IEEE Std 610.12-1990 ορίζει ένα fault ως ένα λανθασμένο βήμα, διαδικασία ή ορισμό δεδομένων σε ένα πρόγραμμα, δηλαδή ως την τεχνική εκδήλωση ενός προβλήματος μέσα στο αναπτυγμένο περιβάλλον του λογισμικού. Σημειώνει, επίσης, ότι στην καθημερινή χρήση οι όροι error και bug χρησιμοποιούνται συχνά για να περιγράψουν την ίδια έννοια. Σε μία αυστηρή τεχνική χρήση μπορεί να γίνει διάκριση μεταξύ defect και fault, ενώ σε πολλά σύγχρονα testing frameworks οι δύο όροι αντιμετωπίζονται πρακτικά ως ισοδύναμοι.

Το bug, από την άλλη, αποτελεί τον περισσότερο καθιερωμένο και χρησιμοποιούμενο όρο στην καθημερινή ανάπτυξη λογισμικού και χρησιμοποιείται για να περιγράψει ένα πρόβλημα στην υλοποίηση το οποίο μπορεί να οδηγήσει σε μη αναμενόμενη συμπεριφορά. Από αυστηρά ακαδημαϊκή άποψη, είναι προτιμότερο να αντιμετωπίζεται ως κοινός όρος για ένα defect ή fault και όχι ως συνώνυμο της έννοιας failure. Η συγκεκριμένη διάκριση είναι ουσιώδης, καθώς ένα bug μπορεί να υπάρχει για μεγάλο χρονικό διάστημα στον κώδικα χωρίς να προκαλέσει ποτέ εμφανές πρόβλημα, εάν δεν εκτελεστεί η συγκεκριμένη διαδρομή ή δεν προκύψουν οι συνθήκες που απαιτούνται για την ενεργοποίηση και εμφάνισή του.

Αντίθετα με τα παραπάνω, το failure αφορά σε πραγματική εκδήλωση λανθασμένης ή μη αναμενόμενης συμπεριφοράς κατά τη λειτουργία του συστήματος. Από τον οργανισμό IEEE περιγράφεται ως αδυναμία ενός συστήματος ή συστατικού (component) να εκτελέσει τις απαιτούμενες λειτουργίες του εντός των καθορισμένων απαιτήσεων, ενώ το NIST το συνδέει περισσότερο με τη διαφορά μεταξύ των πραγματικών αποτελεσμάτων της λειτουργίας και των απαιτούμενων, βάσει προδιαγραφών, αποτελεσμάτων. Ένα defect ή fault μπορεί να βρίσκεται λανθάνον στο λογισμικό και να οδηγήσει σε failure μόνο όταν ενεργοποιηθεί υπό συγκεκριμένες συνθήκες εκτέλεσης. Η παραγωγή ενός λανθασμένου αποτελέσματος, η μη εκτέλεση μιας απαιτούμενης λειτουργίας ή η μη αναμενόμενη διακοπή του προγράμματος αποτελούν άμεσα παραδείγματα εμφάνισης ή ύπαρξης ενός failure.

Οι παραπάνω έννοιες δεν παρουσιάζονται ως αυτόνομες. Η σχέση μεταξύ τους μπορεί να αποδοθεί σχηματικά ως μία πιθανή ακολουθία:



Εικόνα 3.3.1 – Εννοιολογική ακολουθία από ανθρώπινο λάθος έως την αποτυχία λογισμικού. *

Η συγκεκριμένη διατύπωση χρήζει προσοχής, επειδή ο όρος *error* χρησιμοποιείται διαφορετικά σε διαφορετικές πηγές. Σε διάφορα πρότυπα αναφέρεται στην ανθρώπινη ενέργεια ή σε λανθασμένη απόφαση που, με την σειρά της, δημιουργεί ένα *defect*, ενώ αλλού χρησιμοποιείται προκειμένου να περιγράψει μία λανθασμένη εσωτερική κατάσταση του αναπτυσσόμενου συστήματος. Έτσι, στην παρούσα εργασία δίνεται περισσότερη έμφαση στη σαφέστερη διάκριση μεταξύ *defect/fault/bug* ως ατέλειας ενός *artifact* λογισμικού και *failure* ως εκδηλωμένης αποτυχίας κατά την δυναμική εκτέλεση του λογισμικού. Σημειώνεται ότι η παραπάνω ακολουθία (Εικόνα 3.3-1) δεν θα πρέπει να θεωρείται απολύτως μηχανιστική. Ένα *defect* υπάρχει περίπτωση να μην ενεργοποιηθεί ποτέ σε αντίθεση με μια παρατηρούμενη αποτυχία η οποία μπορεί να επηρεάζεται και από διάφορες άλλες εξωτερικές συνθήκες.

Η διάκριση αυτή συνδέεται άμεσα με την έννοια της *Reliability*. Η αξιοπιστία αφορά την ικανότητα του λογισμικού να λειτουργεί χωρίς ανεπιθύμητες αποτυχίες υπό καθορισμένες συνθήκες, ενώ η ύπαρξη *faults* ή *defects* αποτελεί έναν από τους μηχανισμούς μέσω των οποίων είναι δυνατόν να προκύψει κάποιο *failure*. Για τον προαναφερθέντα λόγο, η καταμέτρηση προβλημάτων στατικά στον πηγαίο κώδικα και η παρατήρηση πραγματικών *failures* -συνήθως σε περιβάλλον δυναμικής εκτέλεσης του λογισμικού- παρέχουν διαφορετικού τύπου πληροφορία, αφού η πρώτη αφορά πιθανές αιτίες ή τεχνικές αδυναμίες, ενώ η δεύτερη αφορά την εκδηλωμένη συμπεριφορά του συστήματος που εκτελείται και βρίσκεται σε κατάσταση λειτουργίας.

Η διάκριση είναι ιδιαίτερα σημαντική επειδή οι μετρικές που προκύπτουν από στατική ανάλυση δεν περιγράφουν πραγματικές *failures* που εκδηλώνονται κατά τη διάρκεια εκτέλεσης ενός λογισμικού, αλλά ευρήματα που σχετίζονται με πιθανές αδυναμίες του κώδικα. Ειδικότερα, στο SonarQube ένας κανόνας χαρακτηρίζεται *bug/reliability issue* όταν αφορά κώδικα ο οποίος αποδεδειγμένα θεωρείται λανθασμένος ή έχει περισσότερες πιθανότητες να είναι λανθασμένος και όχι σωστός. Η μετρική *bugs* ή *reliability issues*, άρα, πρέπει να ερμηνεύεται ως αριθμός στατικά εντοπισμένων προβλημάτων που σχετίζονται με την αξιοπιστία και όχι ως αριθμός πραγματικών *failures* που έχουν παρατηρηθεί κατά τη λειτουργία του συστήματος.

3.4 Code smells και Συντηρησιμότητα (Maintainability)

Ο όρος `code smell` περιγράφει μία ένδειξη ότι η εσωτερική σχεδίαση ενδέχεται να χρειάζεται περαιτέρω εξέταση. Δεν ταυτίζεται κατ' ανάγκη με `defect`, αλλά μπορεί να σηματοδοτεί δομή που δυσχεραίνει την κατανόηση ή τη συντήρηση του κώδικα (Fowler, 2018). Για να διαπιστωθεί εάν υπάρχει πραγματικό πρόβλημα σχεδίασης ή συντηρησιμότητας, απαιτείται εξέταση του συγκεκριμένου πλαισίου στο οποίο εμφανίζεται.

Τα `code smells` συνδέονται κυρίως με την έννοια της Συντηρησιμότητας (Maintainability), επειδή χαρακτηρίζουν δομές και επιλογές υλοποίησης που μπορούν να καταστήσουν δυσκολότερη τη μελλοντική εργασία πάνω στον κώδικα του λογισμικού. Για παράδειγμα, ενδεικτικές περιπτώσεις υψηλής πιθανότητας ύπαρξης `code smells` στον κώδικα μπορούν να αποτελέσουν οι υπερβολικά μεγάλες ή σύνθετες μονάδες κώδικα, επαναλαμβανόμενες δομές, υπερβολικές εξαρτήσεις μεταξύ `components` ή καταστάσεις στις οποίες μία λειτουργικότητα κατανέμεται με τρόπο που δυσχεραίνει την κατανόηση και την αλλαγή της. Τα `smells` δεν αποτελούν μία ενιαία κατηγορία προβλημάτων και μπορούν να εμφανίζονται σε διαφορετικά επίπεδα της υλοποίησης. Ουσιαστικά, ως κοινή βάση έχουν την έννοια των προειδοποιητικών ενδείξεων για πιθανά προβλήματα σχεδίασης και συντήρησης, και όχι ως άμεση απόδειξη ελαττωματικής λειτουργίας.

Η άμεση σχέση που αναπτύσσουν με την έννοια της συντηρησιμότητας μπορεί να γίνει περισσότερο κατανοητή μέσα από τις δραστηριότητες και διαδικασίες οι οποίες απαιτούνται κατά την εξέλιξη ενός συστήματος. Για να πραγματοποιήσει μία αλλαγή ένας προγραμματιστής, θα πρέπει να κατανοήσει τον υπάρχοντα κώδικα, να εντοπίσει όλες τις εξαρτήσεις και τα στοιχεία τα οποία επηρεάζονται άμεσα ή έμμεσα από την αλλαγή αυτή, να εκτιμήσει τις συνέπειες της τροποποίησης και να την εφαρμόσει χωρίς να εισάγει ανεπιθύμητες επιπτώσεις στο σύστημα. Δομές που αυξάνουν τη δυσκολία κατανόησης ή δημιουργούν ισχυρές και δυσδιάκριτες εξαρτήσεις μπορούν να αυξήσουν την προσπάθεια που απαιτείται για τη συντήρηση.

Η εμπειρική βιβλιογραφία παρέχει ενδείξεις για τη σχέση αυτή, αλλά τα αποτελέσματα απαιτούν προσεκτική ερμηνεία. Εξετάζοντας προβλήματα που αντιμετώπισαν οι προγραμματιστές κατά τη συντήρηση πραγματικών συστημάτων, διαφορετικού βεληνικού και μεγέθους, οι Yamashita και Moonen (2013) διαπίστωσαν ότι μέρος των προβλημάτων σχετιζόταν με αρχεία που περιείχαν `code smells`. Παράλληλα, έδειξαν ότι ιδιαίτερη σημασία μπορεί να έχουν οι αλληλεπιδράσεις μεταξύ πολλαπλών `code smells` και άλλων χαρακτηριστικών του κώδικα. Παραταύτα, η ίδια η μελέτη κατέληξε, ότι τα `code smells` από μόνα τους εξηγούσαν σχετικά περιορισμένο μέρος των συνολικών προβλημάτων συντήρησης και ότι απαιτούνται συμπληρωματικές προσεγγίσεις για μία πληρέστερη αξιολόγηση της συντηρησιμότητας.

Μελέτες μεγαλύτερης κλίμακας, αντίστοιχα, έχουν εντοπίσει ότι κλάσεις που περιέχουν `smells` μπορούν να εμφανίζουν αυξημένη `change-proneness` και `fault-proneness` σε σχέση με κλάσεις χωρίς `code smells`. Ωστόσο, δεν είναι αποδεκτό το συμπέρασμα ότι κάθε `code smell` προκαλεί απαραίτητα μεγαλύτερο κόστος

συντήρησης ή περισσότερα σφάλματα. Συστηματικές ανασκοπήσεις της βιβλιογραφίας έχουν επισημάνει ότι η εμπειρική τεκμηρίωση δεν είναι πάντα συνεπής όπως επίσης και ότι παράγοντες όπως το είδος του smell, το μέγεθος του υπό ανάπτυξη συστήματος, το πλαίσιο ανάπτυξης και ο τρόπος ανίχνευσης του μπορούν να επηρεάσουν αισθητά και σημαντικά τα εξαγόμενα αποτελέσματα. Τα code smells, επομένως, εννοιολογικά είναι προτιμότερη η απόδοσή τους σε δείκτες για την ύπαρξη πιθανού κινδύνου για τη συντηρησιμότητα και όχι απόλυτες μετρήσεις.

Ιδιαίτερη αξία κατέχει επίσης ο τρόπος με τον οποίο τα code smells μετατρέπονται σε μετρήσιμα ευρήματα από εργαλεία στατικής ανάλυσης. Στο SonarQube, ο όρος code smell χρησιμοποιείται για issue που επηρεάζει τη Συντηρησιμότητα, ενώ στη νεότερη ορολογία του εργαλείου τα αντίστοιχα ευρήματα εντάσσονται άμεσα στη διάσταση Maintainability. Ακολουθώντας αυτή τη κατηγοριοποίηση, συγκεκριμένοι κανόνες εφαρμόζονται αυτοματοποιημένα στον πηγαίο κώδικα και οι παραβιάσεις τους μπορούν να καταγραφούν και να χρησιμοποιηθούν ως ποσοτικές ενδείξεις προβλημάτων συντηρησιμότητας. Μολαταύτα, επειδή πρόκειται για rule-based αυτοματοποιημένη αξιολόγηση, το πλήθος των smells δεν πρέπει να αντιμετωπίζεται ως πλήρης μέτρηση της συνολικής συντηρησιμότητας ενός λογισμικού/συστήματος.

Τα code smells και τα maintainability-related issues αντιμετωπίζονται, στην παρούσα εργασία, ως παρατηρήσιμες ενδείξεις συγκεκριμένων προβλημάτων της τεχνικής ποιότητας του πηγαίου κώδικα. Η μεταβολή τους μεταξύ διαφορετικών καταστάσεων ενός Pull Request μπορεί να χρησιμοποιηθεί για να εξεταστεί η πιθανότητα μίας αλλαγής να συνοδεύεται από βελτίωση ή επιδείνωση συγκεκριμένων πτυχών της συντηρησιμότητας. Η ερμηνεία του γίνεται σε συνδυασμό με τις υπόλοιπες μετρικές και όχι ως ανεξάρτητη ή απόλυτη μέτρηση της συνολικής ποιότητας.

3.5 Πολυπλοκότητα (Complexity) λογισμικού και πηγαίου κώδικα

Η Πολυπλοκότητα λογισμικού (Software Complexity) αποτελεί μία από τις σημαντικότερες ιδιότητες της τεχνικής δομής ενός παραγόμενου συστήματος και συνδέεται στενά με τη δυσκολία κατανόησης, επαλήθευσης, τροποποίησης, συντήρησης και επέκτασής του. Στην ορολογία η οποία έχει πλέον καθιερωθεί για την Μηχανική Λογισμικού (Software Engineering), η πολυπλοκότητα έχει περιγραφεί ως ο βαθμός στον οποίο ο σχεδιασμός ή η υλοποίηση ενός συστήματος ή ενός component είναι δύσκολο να γίνει κατανοητός και να επαληθευτεί. Καθολικά, δεν έχει βρεθεί και δεν υπάρχει αποδεκτός τρόπος με τον οποίο η πολυπλοκότητα μπορεί να ερμηνευθεί/περιγραφεί και να υπολογισθεί/μετρηθεί πλήρως, καθώς η πολυπλοκότητα είναι πολυδιάστατη και μπορεί να αφορά διαφορετικά χαρακτηριστικά της δομής και των αλληλεπιδράσεων του λογισμικού. Η σχετική βιβλιογραφία επιβεβαιώνει ότι οι διαφορετικοί ορισμοί της πολυπλοκότητας δίνουν

έμφαση είτε στις εσωτερικές δομές του λογισμικού είτε στη δυσκολία που αυτές δημιουργούν κατά την ανθρώπινη κατανόηση. Κρίνεται επίσης σκόπιμο να σημειωθεί ως εγκυκλοπαιδική προσθήκη ότι το πρότυπο ISO/IEC/IEEE 24765:2017 παρέχει το τυποποιημένο λεξιλόγιο της Systems and Software Engineering.

Η πολυπλοκότητα μπορεί να προκύπτει από τον αριθμό των επιμέρους στοιχείων ενός συστήματος, τις μεταξύ τους σχέσεις και εξαρτήσεις, τον τρόπο οργάνωσής τους και τη σαφήνεια με την οποία αναπαρίστανται στον κώδικα. Καθώς ένα σύστημα εξελίσσεται και προστίθενται νέες λειτουργίες, components και αλληλεπιδράσεις, οι σχέσεις μεταξύ των στοιχείων του μπορούν να γίνουν περισσότερο δυσνόητες και δυσκολότερες στην διαχείρισή τους. Η πολυπλοκότητα επομένως δεν θα πρέπει να ταυτίζεται με το μέγεθος του κώδικα. Ένα μεγαλύτερο σύστημα στην πραγματικότητα περιλαμβάνει περισσότερα στοιχεία στο σύνολό του, όμως δύο τμήματα κώδικα παρόμοιου μεγέθους μπορούν να διαφέρουν σημαντικά ως προς τη δομή, τις εξαρτήσεις και τη δυσκολία κατανόησής τους. Εμπειρική έρευνα αντιμετωπίζει τα στοιχεία του κώδικα, τις μεταξύ τους διασυνδέσεις και τη σαφήνεια της αναπαράστασής τους ως βασικές πηγές πολυπλοκότητας.

Στην συγκεκριμένη περίπτωση, κρίνεται χρήσιμη η θεωρητική διάκριση μεταξύ των εννοιών της εγγενούς (essential) και της πρόσθετης ή συμπτωματικής (accidental) πολυπλοκότητας. Έχει υποστηριχθεί από τον (1987) ότι μέρος της πολυπλοκότητας είναι αναπόφευκτο, καθώς προκύπτει από τη φύση του ίδιου του προβλήματος το οποίο το λογισμικό καλείται να επιλύσει και από τις σχέσεις μεταξύ των εννοιών που πρέπει να αναπαραστήσει. Ταυτόχρονα, πρόσθετη πολυπλοκότητα είναι δυνατόν να δημιουργηθεί από τεχνολογικές και σχεδιαστικές επιλογές που χρησιμοποιούνται για την υλοποίηση της λύσης. Η δεύτερη κατηγορία μπορεί να περιοριστεί σε μεγαλύτερη κλίμακα μέσω καλύτερης σχεδίασης, κατάλληλων αφαιρέσεων πηγαίου κώδικα και αποτελεσματικότερης οργάνωσης του λογισμικού και των συστατικών του μερών. Η διάκριση όπως περιεγράφηκε, κρίνεται σημαντική, διότι η θεώρηση ότι κάθε μορφή πολυπλοκότητας θα πρέπει να εξαλειφθεί πλήρως είναι μη ρεαλιστική. Το ζητούμενο κυρίως είναι η αποφυγή κάθε μορφής περιττής πολυπλοκότητας η οποία δυσχεραίνει τη διαχείριση του συστήματος χωρίς να εξυπηρετεί ουσιαστική λειτουργική ανάγκη.

Για την ποιότητα λογισμικού, η σημασία της πολυπλοκότητας προκύπτει κυρίως από την επίδρασή της στις δραστηριότητες που απαιτούν ανθρώπινη κατανόηση του συστήματος. Κατά την διαδικασία τροποποίησης του κώδικα, συνήθως διόρθωση ενός προβλήματος ή προσθήκη μιας νέας λειτουργίας, ο προγραμματιστής είναι αναγκαίο να κατανοήσει πλήρως το σχετικό τμήμα του κώδικα, τις εξαρτήσεις του και τις πιθανές συνέπειες μιας αλλαγής. Όσο δυσκολότερη είναι η συγκεκριμένη διαδικασία, τόσο περισσότερη προσπάθεια μπορεί να απαιτείται για τη συντήρηση και τόσο δυσκολότερη, με την σειρά της, γίνεται η ασφαλής πραγματοποίηση μιας τροποποίησης. Σε εμπειρική μελέτη 100 συμμετεχόντων από πανεπιστημιακούς και εταιρικούς οργανισμούς, οι Antinyan et al. (2017) εντόπισαν ότι οι προγραμματιστές αντιλαμβάνονταν την αυξημένη πολυπλοκότητα ως σημαντικό παράγοντα ο οποίος επιβαρύνει την αναγνωσιμότητα, την κατανοησιμότητα και τη δυνατότητα τροποποίησης του κώδικα, ενώ την συνέδεαν και με σημαντικά αυξημένο χρόνο συντήρησης. Κρίνεται αναγκαίο να

σημειωθεί ότι τα συγκεκριμένα αποτελέσματα θα πρέπει να ερμηνεύονται ως εμπειρικές ενδείξεις και όχι ως καθολικός αιτιώδης κανόνας για κάθε σύστημα.

Η πολυπλοκότητα δεν αποτελεί ξεχωριστό χαρακτηριστικό του μοντέλου ποιότητας που ορίστηκε στο πρότυπο ISO/IEC 25010:2023, αλλά χρησιμοποιείται συχνά ως εσωτερική ιδιότητα ή ως τεχνικός δείκτης που μπορεί να σχετίζεται με ευρύτερα χαρακτηριστικά ποιότητας, συνήθως με τη συντηρησιμότητα του λογισμικού. Αυτή η σχέση δεν σημαίνει ότι μία υψηλότερη τιμή οποιασδήποτε μετρικής πολυπλοκότητας αυτομάτως συνεπάγεται χαμηλή ποιότητα. Διαφορετικές μετρικές αποτυπώνουν διαφορετικές πτυχές της πολυπλοκότητας, ενώ τα αποτελέσματα επηρεάζονται από το είδος του συστήματος, τη δομή, τη γλώσσα προγραμματισμού και το επίπεδο στο οποίο πραγματοποιείται η μέτρηση. Υπογραμμίζεται, από το πρότυπο ISO/IEC 5055:2021, η σημασία παραβιάσεων αρχιτεκτονικών και προγραμματιστικών τεχνικών οι οποίες ενδέχεται να συνδέονται με λειτουργικούς κινδύνους ή αυξημένο κόστος.

Η μέτρηση της πολυπλοκότητας πρέπει να αντιμετωπίζεται ως προσέγγιση συγκεκριμένων όψεων ενός σύνθετου φαινομένου και όχι ως απόλυτη μέτρηση της συνολικής δυσκολίας ενός συστήματος. Πρόσφατες εμπειρικές έρευνες με θεματικό επίκεντρο την κατανοησιμότητα του πηγαίου κώδικα δείχνουν ότι οι διαθέσιμες μετρικές πηγαίου κώδικα είναι ικανές να παρέχουν χρήσιμη πληροφορία, αλλά δεν εξηγούν πλήρως τη δυσκολία κατανόησης την οποία ένας προγραμματιστής καλείται να αντιμετωπίσει. Αυτό ενισχύει την ανάγκη η πολυπλοκότητα να αξιολογείται σε συνδυασμό με άλλες ιδιότητες και μετρικές του λογισμικού και όχι μεμονωμένα.

Η πολυπλοκότητα, εδώ, αντιμετωπίζεται ως σημαντικό τεχνικό χαρακτηριστικό ανάλυσης του πηγαίου κώδικα που μπορεί να μεταβάλλεται κατά την εξέλιξή του. Η σύγκριση των αντίστοιχων μετρήσεων στο στάδιο πριν τις τροποποιήσεις και μετά τις τροποποιήσεις από μία αλλαγή επιτρέπει να εξεταστεί εάν ένα Pull Request συνοδεύεται από αύξηση ή μείωση συγκεκριμένων πτυχών της πολυπλοκότητας και τον τρόπο με τον οποίο οι μεταβολές αυτές σχετίζονται με άλλες μετρικές ποιότητας και ασφάλειας. Η πολυπλοκότητα, βέβαια, δεν δύναται να ερμηνευθεί μεμονωμένα ούτε θεωρείται αυτομάτως αιτία μεταβολών σε άλλα χαρακτηριστικά. Οι συγκεκριμένες προσεγγίσεις μέτρησής της αναλύονται ξεχωριστά παρακάτω.

3.5.1 Κυκλωματική Πολυπλοκότητα (Cyclomatic Complexity)

Η Cyclomatic Complexity εισήχθη από τον McCabe ως γράφο-θεωρητική μέτρηση της δομής ελέγχου ενός προγράμματος. Εκφράζει τον αριθμό γραμμικά ανεξάρτητων διαδρομών στον control-flow graph και δεν πρέπει να παρουσιάζεται ως άμεσο μέτρο συνολικής ποιότητας ή ασφάλειας (McCabe, 1976). Η Cyclomatic Complexity επικεντρώνεται στη δομή των αποφάσεων και των διακλαδώσεων του πηγαίου κώδικα, αγνοώντας το μέγεθός του. Ο ίδιος ο McCabe έδειξε ότι η μετρική εξαρτάται από τη δομή ελέγχου του προγράμματος και όχι απλώς από τον αριθμό των συνολικών εντολών που περιέχει.

Η μέτρησή της βασίζεται στην αναπαράσταση του συστήματος μέσω ενός γράφου που ονομάζεται Control Flow Graph (CFG). Σε έναν τέτοιο γράφο, οι κόμβοι αναπαριστούν τμήματα ή διαδοχές εντολών και οι ακμές του τις πιθανές μεταβάσεις της ροής εκτέλεσης μεταξύ τους. Η Cyclomatic Complexity, η οποία συμβολίζεται συνήθως ως $V(G)$, ισούται με τον αριθμό των γραμμικά ανεξάρτητων διαδρομών (linearly independent paths) του γράφου ελέγχου. Μία διαδρομή θεωρείται ανεξάρτητη όταν εισάγει τουλάχιστον μία ακμή που δεν περιλαμβάνεται σε κάποια προηγούμενη ανεξάρτητη διαδρομή. Η μετρική, άρα, δεν εκφράζει το συνολικό πλήθος όλων των θεωρητικά δυνατών εκτελέσιμων μονοπατιών (execution paths) - το οποίο πιθανώς να είναι πολύ μεγαλύτερο ή απεριόριστο λόγω ύπαρξης δομών επανάληψης στον πηγαίο κώδικα- αλλά τον αριθμό των ανεξάρτητων διαδρομών που χαρακτηρίζουν τη δομή ελέγχου. Η συγκεκριμένη μετρική έχει οριστεί από το NIST ως ο αριθμός των linearly independent paths ενός control-flow graph.

Μαθηματικά, η μετρική ορίζεται με τον παρακάτω γενικό τύπο:

$$V(G) = E - N + 2P$$

Όπου E είναι ο συνολικός αριθμός των ακμών (edges) του γράφου, N ο αριθμός των κόμβων (nodes) και P ο αριθμός των συνδεδεμένων components του γράφου. Για έναν μεμονωμένο συνδεδεμένο control-flow graph, όπου $P = 1$, ο τύπος μπορεί να απλοποιηθεί σε:

$$V(G) = E - N + 2$$

Για τις συνηθισμένες δομημένες συναρτήσεις, η ίδια ποσότητα μπορεί πρακτικά να υπολογιστεί ως:

$$V(G) = D + 1$$

όπου το D αντιστοιχεί στον αριθμό των συνολικών decision ή predicate points. Η δεύτερη μορφή εξηγεί τη διαισθητική λειτουργία της συγκεκριμένης μετρικής. Μία συνάρτηση χωρίς καμία διακλάδωση έχει εξ ορισμού Cyclomatic Complexity ίση με 1, ενώ κάθε νέα απόφαση που δημιουργεί μία πρόσθετη ανεξάρτητη διαδρομή αυξάνει την πολυπλοκότητα.

Έστω, μία απλή συνάρτηση με σειριακή εκτέλεση και χωρίς καμία συνθήκη να περιλαμβάνεται σε αυτή. Η συγκεκριμένη συνάρτηση, βάσει των παραπάνω θα έχει τιμή $V(G) = 1$. Η προσθήκη μιας δομής επιλογής που δημιουργεί μία νέα διακλάδωση, αυξάνει την τιμή σε 2, ενώ η εισαγωγή πρόσθετων ανεξάρτητων αποφάσεων την αυξάνει περαιτέρω. Όσο περισσότερες εναλλακτικές διαδρομές δημιουργούνται από τη λογική ελέγχου, τόσο μεγαλύτερη θα είναι η τελική τιμή της Cyclomatic Complexity. Η συγκεκριμένη ερμηνεία έχει ιδιαίτερη σημασία για τις διαδικασίες του Software Testing, καθώς η τιμή της μετρικής μπορεί να χρησιμοποιηθεί για τον καθορισμό ενός συνόλου ανεξάρτητων basis paths που πρέπει να εξεταστούν στο πλαίσιο του structured testing.

Η Cyclomatic Complexity έχει χρησιμοποιηθεί ως ένδειξη για τη δυσκολία συντήρησης, επαλήθευσης και ελέγχου του λογισμικού. Όσο αυξάνεται ο αριθμός των ανεξάρτητων διαδρομών, τόσο περισσότερες διαφορετικές συμπεριφορές πρέπει να κατανοήσει ένας προγραμματιστής κατά την ανάλυση και τροποποίηση του κώδικα. Αντίστοιχα, αυξάνεται το σύνολο των λογικά διαφορετικών εκτελέσιμων paths τα οποία πρέπει να ληφθούν υπόψη κατά τον σχεδιασμό των δοκιμών. Η βιβλιογραφία, όμως, δεν υποστηρίζει ότι μία υψηλή τιμή της μετρικής αυτής αποτελεί από μόνη της απόδειξη χαμηλής ποιότητας του πηγαίου κώδικα ή ότι θα προκαλέσει σίγουρα περισσότερα defects. Ενώ αρκετές μελέτες έχουν εντοπίσει σχέσεις μεταξύ complexity και defect rates, μέρος αυτών των συσχετίσεων δύναται να μειωθεί όταν ληφθεί υπόψη το συνολικό μέγεθος του λογισμικού.

Η Cyclomatic Complexity αποτυπώνει μία συγκεκριμένη διάσταση της πολυπλοκότητας, τη δομή της ροής ελέγχου του συνολικού πηγαίου κώδικα, ή μέρος αυτού. Εξ' ορισμού δεν λαμβάνει υπόψη τη σημασιολογία του κώδικα, την ποιότητα της ονοματοδοσίας (σε συναρτήσεις, μεταβλητές, κ.α.), τις σχέσεις μεταξύ διαφορετικών modules, το επιχειρησιακό πλαίσιο ή όλες τις πλευρές της ανθρώπινης δυσκολίας κατανόησης. Επιπροσθέτως, δύο (ή περισσότερες συναρτήσεις) είναι δυνατόν να έχουν ίδια τιμή στην συγκεκριμένη μετρική αλλά να διαφέρουν αισθητά ως προς τη δομή και τη δυσκολία με την οποία ένας προγραμματιστής μπορεί να αντιληφθεί την λειτουργία και τον τρόπο εκτέλεσής τους. Επομένως, κρίνεται περισσότερο κατάλληλη η χρησιμοποίηση της μετρικής σε συνδυασμό με άλλες μετρήσεις και όχι ως απόλυτος δείκτης της συνολικής πολυπλοκότητας ή ποιότητας ενός συστήματος λογισμικού. Σημειώνεται ότι, η Cyclomatic Complexity έχει διακριθεί ρητά, από τον NIST οργανισμό, ως καταμέτρηση ανεξάρτητων paths από την ευρύτερη έννοια της πολυπλοκότητας την οποία και προσπαθεί να προσεγγίσει.

Σε σύγχρονα εργαλεία στατικής ανάλυσης, όπως για παράδειγμα το SonarQube το οποίο χρησιμοποιήθηκε και στην παρούσα έρευνα για την συγκέντρωση δεδομένων κώδικα, ο αρχικός ορισμός προσαρμόζεται στις συντακτικές δομές κάθε γλώσσας προγραμματισμού. Παραδείγματος χάριν, στο SonarQube η μετρική complexity υπολογίζεται αρχικά σε επίπεδο συνάρτησης (function) ή αντίστοιχης δομής, με ελάχιστη τιμή 1 και αυξάνεται στις περιπτώσεις όπου η ροή ελέγχου διαχωρίζεται σε νέα conditional branches. Οι ακριβείς γλωσσικές κατασκευές που αυξάνουν τη μετρική διαφοροποιούνται ανά γλώσσα, αλλά συνήθως είναι δομές όπως *if*, *for*, *while*, *case* και συγκεκριμένοι λογικοί τελεστές. Η συνολική Cyclomatic Complexity που αντιστοιχεί στον πηγαίο κώδικα προκύπτει κατά βάση από το άθροισμα των επιμέρους τιμών που υπολογίζονται σε χαμηλότερο επίπεδο ανάλυσης.

Όπως ορίζεται, στην παρούσα εργασία χρησιμοποιείται ως ποσοτικός δείκτης της δομικής πολυπλοκότητας της ροής ελέγχου. Η σύγκριση της τιμής της μεταξύ των διαφορετικών καταστάσεων ενός Pull Request επιτρέπει την εξέταση της περίπτωσης ή τροποποίηση του πηγαίου κώδικα να συνοδεύεται από αύξηση ή μείωση της συγκεκριμένης μορφής πολυπλοκότητας. Η μεταβολή αυτή δεν ερμηνεύεται αυτόματα ως αντίστοιχη μεταβολή της συνολικής ποιότητας, αλλά εξετάζεται σε συνδυασμό με τις υπόλοιπες μετρικές ποιότητας και ασφάλειας που χρησιμοποιούνται στην παρούσα εμπειρική ανάλυση.

3.5.2 Γνωστική Πολυπλοκότητα (Cognitive Complexity)

Η Γνωστική Πολυπλοκότητα (Cognitive Complexity) αποτελεί μετρική πηγαίου κώδικα η οποία σχεδιάστηκε με στόχο την προσέγγιση της σχετικής δυσκολίας κατανόησης της ροής ελέγχου ενός τμήματος κώδικα, κυρίως, από έναν προγραμματιστή. Ιστορικά, προτάθηκε από τη G. Ann Campbell στο πλαίσιο της SonarSource και παρουσιάστηκε ακαδημαϊκά το 2018. Η δημιουργία της βασίστηκε στην παρατήρηση ότι η Cyclomatic Complexity, παρόλο που είναι ιδιαίτερα χρήσιμη για την αποτύπωση ανεξάρτητων διαδρομών εκτέλεσης και τη σχεδίαση αντίστοιχων δοκιμών, δεν διαφοροποιεί καθολικά τον κώδικα που παρουσιάζει διαφορετικό βαθμό δυσκολίας ως προς την ανθρώπινη κατανόηση. Η Cognitive Complexity επιχειρεί να προσεγγίσει διαφορετική διάσταση της πολυπλοκότητας, έναντι του πλήθους των πιθανών μονοπατιών εκτέλεσης ακολουθίας, υπολογίζει και αποτυπώνει το νοητικό φορτίο που δημιουργεί η δομή της ροής ελέγχου κατά την ανάγνωση και κατανόηση του κώδικα.

Η Cognitive Complexity σχεδιάστηκε ώστε η βαθμολογία να προσεγγίζει καλύτερα τη δυσκολία κατανόησης της ροής ελέγχου. Η μέθοδος αγνοεί ορισμένες ευανάγνωστες συντομεύσεις, επιβαρύνει τις διακοπές της γραμμικής ροής και αυξάνει περαιτέρω τη βαθμολογία όταν οι σχετικές δομές είναι εμφωλευμένες (Campbell, 2018).

Η εμφώλευση (nesting) αποτελεί ένα από τα σημαντικότερα στοιχεία που διαφοροποιούν τη Cognitive Complexity. Μία διαδοχή ανεξάρτητων δομών ελέγχου θεωρείται ευκολότερη στην κατανόηση από ότι αν η μία δομή βρίσκεται μέσα στην άλλη. Εάν, για παράδειγμα, μία δομή επανάληψης βρίσκεται μέσα σε μία συνθήκη και μία δεύτερη συνθήκη βρίσκεται μέσα στην επανάληψη, ο προγραμματιστής πρέπει να διατηρεί ταυτόχρονα περισσότερα επίπεδα νοητικού context (περιεχομένου) για να μπορέσει να αντιληφθεί πλήρως τη συμπεριφορά του κώδικα. Για τον λόγο αυτό, η Cognitive Complexity δεν προσθέτει απλώς μία μονάδα για κάθε σχετική δομή, αλλά αυξάνει τη σχετική συνολική βαθμολογία σύμφωνα με το βάθος εμφώλευσης. Έτσι, δύο τμήματα κώδικα μπορούν να παρουσιάζουν παρόμοιο αριθμό αποφάσεων αλλά να βαθμολογούνται με αισθητά διαφορετική Cognitive Complexity λόγω της διαφορετικής οργάνωσής τους.

Συνακόλουθα, η συγκεκριμένη μετρική αντιμετωπίζει διαφορετικά ορισμένες γλωσσικές κατασκευές ανάλογα με το θεωρούμενο κόστος κατανόησής τους. Δομές οι οποίες είναι ικανές να διακόψουν τη γραμμική ροή εκτέλεσης, όπως είναι για παράδειγμα οι συνθήκες, οι επαναλήψεις και δομές διαχείρισης σφαλμάτων (catch clauses), αυξάνουν τη βαθμολογία, όπως επίσης και η αναδρομή και ορισμένες μορφές μη γραμμικών jumps είναι ικανές να επιβαρύνουν το τελικό αποτέλεσμα. Αντίθετα, κατασκευές οι οποίες μπορούν να θεωρηθούν ότι κάνουν τον κώδικα περισσότερο συνοπτικό και ευανάγνωστο μπορούν να αγνοούνται ή να επιβαρύνονται λιγότερο. Χαρακτηριστικό παράδειγμα αποτελεί η διαφορετική αντιμετώπιση μίας δομής switch σε σχέση με μία ακολουθία πολλαπλώς if/else if/else, καθώς στη δεύτερη εξ' αυτών περίπτωση ο προγραμματιστής μπορεί να

χρειάζεται να εξετάσει διαφορετικές μεταβλητές και λογικές εκφράσεις για κάθε συνθήκη. Η προσέγγιση αυτή δείχνει ότι η Cognitive Complexity περιλαμβάνει σχεδιαστικές παραδοχές οι οποίες σχετίζονται με το ανθρώπινο κόστος κατανόησης και δεν αποτελεί απλώς μαθηματική καταμέτρηση δομών.

Η διαφοροποίησή της από την Cyclomatic Complexity κρίνεται ουσιαστική. Η Cyclomatic Complexity αποτυπώνει κυρίως τη δομική πολυπλοκότητα των ανεξάρτητων διαδρομών εκτέλεσης, ενώ η Cognitive Complexity επιχειρεί να αποτυπώσει τη σχετική δυσκολία κατανόησης της ροής ελέγχου. Αυτό σημαίνει ότι δύο μέθοδοι με την ίδια Cyclomatic Complexity μπορούν να παρουσιάζουν μεταβλητότητα στην μετρική της Cognitive Complexity σε περιπτώσεις όπου, για παράδειγμα, η μία χρησιμοποιεί μεγάλο βάθος εμφώλευσης ενώ η άλλη οργανώνει τις αποφάσεις της με απλούστερη και περισσότερο γραμμική δομή.

Η σύνδεση της Cognitive Complexity με την Συντηρησιμότητα ενός συστήματος προκύπτει από το γεγονός ότι σημαντικό μέρος της συντήρησης λογισμικού απαιτεί από τους προγραμματιστές να κατανοήσουν υπάρχοντα κώδικα προτού τον τροποποιήσουν με ασφάλεια. Κώδικας με σύνθετη και βαθιά εμφωλευμένη ροή μπορεί να απαιτεί μεγαλύτερη νοητική προσπάθεια κατά την ανάλυση, την αλλαγή και τον έλεγχό του. Η Cognitive Complexity πρέπει να αντιμετωπίζεται ως δείκτης συγκεκριμένης όψης της κατανοησιμότητας και όχι ως άμεση ή πλήρης μέτρηση της ανθρώπινης γνωστικής προσπάθειας. Η πραγματική δυσκολία κατανόησης επηρεάζεται επίσης άμεσα από παράγοντες όπως η εμπειρία του εκάστοτε προγραμματιστή, η εξοικείωσή του με το σύστημα που πρόκειται να τροποποιήσει, η ονοματοδοσία, η αρχιτεκτονική και το ευρύτερο context του κώδικα.

Η Cognitive Complexity και η εννοιολογική της ερμηνεία, συμβάλλει στην παρούσα εμπειρική έρευνα και χρησιμοποιείται ως ένας ποσοτικός δείκτης της πολυπλοκότητας ο οποίος σχετίζεται με την κατανόηση της ροής ελέγχου του πηγαίου κώδικα. Η σύγκριση της τιμής της μεταξύ των διαφορετικών καταστάσεων ενός Pull Request ή ενός Repository επιτρέπει να εξεταστεί η πιθανότητα μία αλλαγή να συνοδεύεται από αύξηση ή μείωση αυτής της διάστασης της πολυπλοκότητας. Όπως αντίστοιχα στην περίπτωση της Cyclomatic Complexity, η μεταβολή της δεν ερμηνεύεται αυτοτελώς ως βελτίωση ή υποβάθμιση της συνολικής ποιότητας, αλλά εξετάζεται σε συνδυασμό με τις υπόλοιπες μετρικές του λογισμικού.

3.6 Διπλότυπος Κώδικα (Code Duplication)

Ως code clones περιγράφονται τμήματα κώδικα που εμφανίζουν ουσιαστική ομοιότητα σε κειμενικό, συντακτικό ή λειτουργικό επίπεδο. Η αντιγραφή και επικόλληση αποτελεί συνηθισμένη πηγή, αλλά η ομοιότητα μπορεί επίσης να προκύψει ανεξάρτητα (Roy, Cordy & Koschke, 2009). Η ανίχνευση και η ταξινόμησή τους αποτελούν ξεχωριστό και εκτεταμένο πεδίο έρευνας της Μηχανικής Λογισμικού.

Στη βιβλιογραφία έχει επικρατήσει μία γενική ταξινόμηση των clones σε διαφορετικούς τύπους. Τα Type-1 clones είναι τα πανομοιότυπα τμήματα κώδικα, με πιθανές διαφοροποιήσεις σε whitespace ή σχόλια. Τα Type-2 clones διατηρούν την ίδια δομή αλλά μπορεί να διαφοροποιούνται σε identifiers, literals ή άλλες επιφανειακές λεπτομέρειες. Τα Type-3 clones, τα επονομαζόμενα και ως near-miss clones, περιλαμβάνουν επιπλέον προσθήκες, διαγραφές ή τροποποιήσεις εντολών, διατηρώντας όμως σημαντικό βαθμό ομοιότητας. Ο τέταρτος και τελευταίος τύπος, ο Type-4 ή semantic clones, περιγράφει τμήματα κώδικα τα οποία υλοποιούν παρόμοια λειτουργικότητα χωρίς να παρουσιάζουν απαραίτητα υψηλή συντακτική ομοιότητα. Η συγκεκριμένη διάκριση είναι σημαντική επειδή διαφορετικές τεχνικές clone detection μπορούν να εντοπίζουν διαφορετικές κατηγορίες. Κατά συνέπεια, δύο εργαλεία δεν είναι απαραίτητο να παράγουν τα ίδια αποτελέσματα για την ίδια βάση κώδικα.

Η κύρια ανησυχία σχετικά με το code duplication αφορά στη συντήρηση και στην εξέλιξη του λογισμικού. Όταν η ίδια ή παρόμοια λογική βρίσκεται σε πολλά διαφορετικά σημεία, μία απαραίτητη αλλαγή ενδέχεται να πρέπει να εφαρμοστεί με συνέπεια σε όλα τα αντίστοιχα αντίγραφα. Στην περίπτωση όπου ένας προγραμματιστής εντοπίζει και τροποποιήσει μόνο μία από τις εμφανίσεις, οι υπόλοιπες μπορούν να παραμείνουν σε παλαιότερη κατάσταση, δημιουργώντας έτσι ασυνέπειες στη συμπεριφορά του συστήματος. Η ανάγκη εντοπισμού και συγχρονισμένης τροποποίησης πολλών παρόμοιων τμημάτων μπορεί με τον τρόπο αυτό να αυξήσει την προσπάθεια συντήρησης και να δυσχεράνει την ασφαλή εξέλιξη του πηγαίου κώδικα. Εμπειρικές μελέτες έχουν δείξει ότι ο διπλότυπος κώδικας και οι ασυνέπειες μεταξύ των clones εμφανίζονται συχνά και έχουν εντοπιστεί περιπτώσεις όπου συνδέονται με πραγματικά faults.

Επιπροσθέτως, μία συνέπεια είναι η πιθανή διάδοση υπαρχόντων σφαλμάτων μέσω της αντιγραφής κώδικα. Περιπτώσεις όπου ένα τμήμα κώδικα μπορεί να περιλαμβάνει κάποιο λανθάνον defect αντιγραφεί πριν το πρόβλημα εντοπιστεί και αναγνωριστεί, το ίδιο defect μπορεί να μεταφερθεί σε περισσότερες τοποθεσίες του συστήματος. Η μεταγενέστερη διόρθωσή του απαιτεί την αναγνώριση όλων των πολλαπλών σχετικών αντιγράφων.

Το code duplication, παρά τις διάφορες αρνητικές επιπτώσεις που μπορεί να δημιουργήσει στον πηγαίο κώδικα, δεν θα πρέπει να θεωρείται αυτομάτως ένδειξη κακής σχεδίασης ή χαμηλής ποιότητας. Σε ορισμένα περιβάλλοντα ανάπτυξης συστημάτων, η ελεγχόμενη αντιγραφή κώδικα μπορεί να αποτελεί συνειδητή τεχνική επιλογή, όπως για παράδειγμα σε περιπτώσεις πειραματισμού με μία υπάρχουσα λειτουργικότητα, απομόνωση αλλαγών ή όταν δημιουργία μιας νέας αφαίρεσης θα αύξανε δυσανάλογα την πολυπλοκότητα και τις εξαρτήσεις. Οι Kasper και Godfer, μέσω της εξέτασης πραγματικών συστημάτων λογισμικού, διαπίστωσαν ότι ορισμένα patterns cloning μπορούν να έχουν θετική επίδραση στη συντηρησιμότητα και υποστήριξαν ότι η γενική υπόθεση ότι κάθε clone είναι επιβλαβές είναι υπερβολικά απλουστευτική. Συνοψίζοντας εδώ, η ύπαρξη διπλότυπου κώδικα αποτελεί περισσότερο ένδειξη που απαιτεί αξιολόγηση σε συγκεκριμένο περιβάλλον και context και όχι απόδειξη ύπαρξης προβλήματος.

Η μαθηματική ποσοτικοποίηση του διπλότυπου κώδικα πραγματοποιείται μέσω τεχνικών clone detection, οι οποίες συγκρίνουν τον πηγαίο κώδικα με την χρήση, πάντα σύμφωνα με την δομή του εργαλείου, γραμμών, tokens, συντακτικές δομές ή πιο σύνθετες αναπαραστάσεις. Κάθε προσέγγιση χρησιμοποιεί διαφορετικά κριτήρια ομοιότητας και thresholds (κατώτατα όρια αποδεκτής αξιολόγησης), με αποτέλεσμα το ποσοστό του διπλότυπου κώδικα να μην αποτελεί απόλυτη ιδιότητα του συστήματος ανεξάρτητη από το εργαλείο μέτρησης. Το παραπάνω αποτελεί σημαντικό περιορισμό: μία τιμή duplication πρέπει πάντοτε να ερμηνεύεται σύμφωνα με τον τρόπο με τον οποίο το συγκεκριμένο εργαλείο ορίζει και ανιχνεύει ένα διπλότυπο μπλοκ κώδικα.

Στο SonarQube η ανάλυση αποτυπώνεται μέσω μετρικών όπως είναι οι: duplicated_lines, duplicated_blocks, duplicated_files και duplicated_lines_density. Η τελευταία εκφράζει ποσοστιαία αναλογία των γραμμών που συμμετέχουν σε duplications προς το σύνολο των γραμμών που χρησιμοποιεί η αντίστοιχη μέτρηση του εργαλείου. Η επίσημη τεκμηρίωση ορίζει τον υπολογισμό της μαθηματικά ως:

$$\text{Duplicated Lines Density} = \frac{\text{Duplicated Lines}}{\text{Lines}} \times 100$$

Η ίδια τεκμηρίωση επίσης δείχνει ότι ο ακριβής προσδιορισμός ενός duplicated bloc εξαρτάται κατά κύριο λόγο από τη γλώσσα -οι ιδιαιτερότητες και διαφοροποιήσεις που εμφανίζουν οι γλώσσες προγραμματισμού δεν είναι μέρος της παρούσας έρευνας και δεν θα αναλυθούν περαιτέρω-. Κρίνεται σημαντική η σημείωση ότι η συγκεκριμένη μετρική πρέπει να θεωρείται αποτέλεσμα του ορισμού του detection mechanism που ακολουθεί το κάθε εργαλείο -στο πλαίσιο της συγκεκριμένης εμπειρικής μελέτης το SonarQube-, και όχι καθολική ανεξάρτητη μέτρηση κάθε δυνατής μορφής duplication.

Στην παρούσα εργασία, η έννοια της υπολογιζόμενης Duplicated Lines Density τιμής χρησιμοποιείται ως δείκτης ποσοτικοποίησης της έκτασης του εντοπισμένου διπλότυπου κώδικα. Η σύγκρισή της μεταξύ των διαφορετικών καταστάσεων ενός Pull Request επιτρέπει να εξεταστεί εάν μία αλλαγή συνοδεύεται από αύξηση ή μείωση ή μη μεταβολή της αναλογίας του επαναλαμβανόμενου κώδικα. Η αύξηση της μετρικής δεν χαρακτηρίζεται αυτομάτως ως αποδεδειγμένη υποβάθμιση της ποιότητας ούτε η μείωσή της ως βελτιστοποίηση, καθώς ερμηνεύεται σε συνδυασμό με τις υπόλοιπες μετρικές και το ευρύτερο πλαίσιο της αλλαγής. Η προσέγγιση αυτή είναι συνεπής με την εμπειρική υπάρχουσα βιβλιογραφία, η οποία αναγνωρίζει τόσο τους κινδύνους του uncontrolled duplicated όσο και περιπτώσεις στις οποίες το cloning αποτελεί συνειδητή και πρακτικά δικαιολογημένη τεχνική επιλογή.

3.7 Κάλυψη Δοκιμών (Test Coverage)

Το code coverage αποτυπώνει ποιο μέρος προκαθορισμένων στοιχείων του κώδικα εκτελέστηκε από ένα test suite, όπως γραμμές, εντολές ή κλάδοι. Αποτελεί κριτήριο επάρκειας των δοκιμών και όχι αυτοτελή απόδειξη ότι το λογισμικό δεν περιέχει defects (Kochhar et al., 2017). Η βιβλιογραφία αντιμετωπίζει τέτοιες μετρήσεις ως test adequacy criteria, επεξηγηματικά, ως τρόπους ποσοτικής εκτίμησης του πόσο εκτεταμένα ένα test suite εξετάζει τη δομή ή άλλες ιδιότητες του λογισμικού που δοκιμάζεται.

Η σημασία του Test Coverage για την ποιότητα έγκειται κυρίως στη δυνατότητα εντοπισμού περιοχών του κώδικα που δεν έχουν εξεταστεί από τις υπάρχουσες δοκιμές. Χαμηλή κάλυψη δοκιμών σημαίνει ότι ένα σημαντικό μέρος (μεγάλο) της υλοποίησης ενδέχεται να μην εκτελείται καθόλου κατά το testing, περιορίζοντας τη δυνατότητα των tests να αποκαλύψουν προβλήματα στις συγκεκριμένες μη καλυπτόμενες περιοχές. Αντίθετα, η αύξηση της κάλυψης είναι ικανή να ενισχύσει την έκταση της επαλήθευσης και να υποστηρίξει την έγκαιρη ανίχνευση regressions και άλλων ανεπιθύμητων συμπεριφορών. Για τον λόγο αυτό, η Test Coverage χρησιμοποιείται ευρέως ως δείκτης πληρότητας ή επάρκειας της διαδικασίας δοκιμών, ιδιαίτερα σε αυτοματοποιημένα testing environments.

Ένα υψηλό Test Coverage δεν συνεπάγεται αυτομάτως υψηλή ποιότητα λογισμικού ή καθολική απουσία defects. Μία γραμμή μπορεί να έχει εκτελεστεί από ένα test χωρίς το test να επαληθεύει σωστά τη συμπεριφορά της, ενώ σημαντικές εισόδους, ακραίες περιπτώσεις ή προβλεπόμενες λανθασμένες συμπεριφορές μπορεί να μην έχουν υλοποιηθεί προς εξέταση παρά την υψηλή αριθμητική κάλυψη. Η ποιότητα των περιπτώσεων δοκιμής και των assertions που αυτές υλοποιούν παραμένει εξίσου σημαντική με το ποσοστό κάλυψης. Η διάκριση αυτή υποστηρίζεται και εμπειρικά μέσω διάφορων μελετών. Σε μελέτη 100 μεγάλων open-source Java projects, οι Kochhar et al. (2017) δεν εντόπισαν ισχυρή σχέση κατά την οποία η υψηλότερη code coverage συνεπάγεται απαραίτητα λιγότερα post-release defects. Σε επίπεδο έργου, η συσχέτιση αυτή κρίθηκε ασήμαντη και δεν παρατηρήθηκε αντίστοιχη σχέση σε επίπεδο αρχείου.

Το Test Coverage είναι περισσότερο χρήσιμο ως ένας συμπληρωματικός δείκτης της έκτασης του testing στον πηγαίο κώδικα και όχι ως αυτόνομη μέτρηση της ποιότητας του προϊόντος. Υψηλότερη κάλυψη μπορεί να αυξήσει την εμπιστοσύνη περί υποβολής κώδικα σε δοκιμές, αλλά η ουσιαστική ποιότητα της επαλήθευσης εξακολουθεί να εξαρτάται από την εξέταση που οι δοκιμές επιδιώκουν και την δυνατότητα να αποκαλυφθούν defects του κώδικα μέσα από την διαδικασία του testing.

Η Test Coverage αναγνωρίζεται ως σημαντικό χαρακτηριστικό της διαδικασίας επαλήθευσης και ως παράγοντας που μπορεί να συμβάλει στην αύξηση της συνολικής ποιότητας του λογισμικού. Ωστόσο, στα πλαίσια της εμπειρικής ανάλυσης που η παρούσα βιβλιογραφία περιγράφει, δεν χρησιμοποιείται ως μεταβλητή στον υπολογισμό των αποτελεσμάτων της κύριας εμπειρικής ανάλυσης. Για τον συγκεκριμένο λόγο δεν εξετάζεται και δεν υπολογίζεται περαιτέρω ως προς τις επιμέρους μορφές ή τεχνικές υπολογισμού της.

3.8 Τεχνικό Χρέος (Technical Debt)

Η μεταφορά του Technical Debt περιγράφει το μελλοντικό κόστος που μπορεί να δημιουργήσει μία βραχυπρόθεσμα ευνοϊκή τεχνική επιλογή. Η αρχική διατύπωση αποδίδεται στον Cunningham και η μεταγενέστερη βιβλιογραφία έχει επεκτείνει την έννοια σε πολλαπλές κατηγορίες τεχνικών συμβιβασμών (Cunningham, 1992; Li, Avgeriou & Liang, 2015).

Η οικονομική μεταφορά (χρέος) βασίζεται κυρίως στις έννοιες του κεφαλαίου (principal) και του τόκου (interest). Το principal μπορεί να θεωρηθεί ως το κόστος το οποίο απαιτείται για την αντιμετώπιση ή αντικατάσταση της μη βέλτιστης τεχνικής λύσης, ενώ το interest αντιπροσωπεύει την πρόσθετη προσπάθεια που χρειάζεται να καταβληθεί και επιβάλλει η ύπαρξη του χρέους σε μελλοντικές εργασίες ανάπτυξης και συντήρησης. Έστω ένα παράδειγμα όπου μία προσωρινή σχεδιαστική επιλογή μπορεί αρχικά να επιτρέψει την ταχύτερη υλοποίηση μιας λειτουργίας. Η συγκεκριμένη απόφαση ενέχει τον κίνδυνο να συνεπάγεται πως κάθε μεταγενέστερη αλλαγή στο συγκεκριμένο τμήμα του συστήματος μπορεί να απαιτεί περισσότερο χρόνο εξαιτίας της δυσκολίας κατανόησης ή τροποποίησής του. Σε περιπτώσεις όπου το χρέος παραμένει ανεξέλεγκτο, το συσσωρευμένο «interest» σταδιακά μπορεί να περιορίσει την ταχύτητα με την οποία ένα σύστημα αναπτύσσεται και εξελίσσεται.

Η δημιουργία Τεχνικού Χρέους δεν αποτελεί πάντοτε αποτέλεσμα αμέλειας ή ανεπαρκούς τεχνικής ικανότητας. Η βιβλιογραφία διακρίνει μεταξύ σκόπιμα αναλαμβανόμενου (intentional/deliberate) και μη σκόπιμου (unintentional/inadvertent) τεχνικού χρέους. Μία ομάδα ανάπτυξης μπορεί συνειδητά να αποδεχθεί έναν περιορισμένο τεχνικό συμβιβασμό προκειμένου να ικανοποιήσει μία κρίσιμη προθεσμία, να παραδώσει ταχύτερα μία λειτουργία ή να αξιοποιήσει μία επιχειρηματική ευκαιρία. Το μη σκόπιμο τεχνικό χρέος, αντίθετα, μπορεί να δημιουργηθεί λόγω ελλειπών γνώσης, μεταβολής των απαιτήσεων, αρχιτεκτονικών αποφάσεων των οποίων οι συνέπειες εμφανίζονται αργότερα ή σταδιακής υποβάθμισης του συστήματος. Η ύπαρξη Τεχνικού χρέους, συνεπώς, δεν καθίσταται αυτομάτως αρνητική. Καθοριστικό ζήτημα έγκειται στην ορατότητα, στην δυνατότητα ελέγχου και διαχείρισής του καθώς και η σχέση βραχυπρόθεσμου οφέλους και μελλοντικού κόστους που θα προκύψει από την ύπαρξή του.

Η ορολογία του Τεχνικού Χρέους, σε αντίθεση με τον αρχικό της ορισμό όπου επικεντρωνόταν κυρίως στην υλοποίηση, έχει πλέον επεκταθεί σε διαφορετικά artifacts και επίπεδα της διαδικασίας ανάπτυξης λογισμικού. Η σχετική βιβλιογραφία κάνει αναφορά, επίσης, σε όρους όπως code debt, design debt, architectural debt, testing debt, documentation debt και requirements debt. Ο ισχυρισμός αυτός έχει επιβεβαιωθεί από την systematic mapping study των Li, Avgeriou και Liang η οποία ταξινομήσε το Τεχνικό Χρέος σε δέκα διαφορετικούς τύπους, καταδεικνύοντας έτσι επίσης ότι το φαινόμενο δεν περιορίζεται μόνο στον πηγαίο κώδικα, αν και το code-related debt έχει προσελκύσει ιδιαίτερα μεγάλο μέρος της ερευνητικής προσοχής. Η διαφοροποίηση αυτή είναι σημαντική, καθώς ένα σύστημα δύναται να παρουσιάζει σχετικά καθαρό κώδικα αλλά να περιλαμβάνει σημαντικό αρχιτεκτονικό ή testing

debt το οποίο να εξακολουθεί να δυσχεραίνει τη μελλοντική ανάπτυξη και εξέλιξη του λογισμικού.

Η συσσώρευση τεχνικού χρέους μπορεί να σχετίζεται άμεσα με χαρακτηριστικά όπως είναι η αυξημένη πολυπλοκότητα, διπλότυπος κώδικας, δυσκολία νοητικής κατανόησης του κώδικα, ανεπαρκώς οργανωμένη δομή ή άλλες επιλογές οι οποίες και δυσχεραίνουν τη συνάρτηση και την εξέλιξη. Πρέπει να καταστεί σαφές εδώ ότι οι έννοιες αυτές δεν ταυτίζονται άμεσα με το Τεχνικό Χρέος. Ένα Code smell ή μία υψηλή τιμή πολυπλοκότητας μπορεί να αποτελεί ένδειξη πιθανής τεχνικής οφειλής, αλλά δεν αποδεικνύει από μόνο του ότι υπάρχει κάποιο είδος debt με την πλήρη οικονομική, όπως ορίστηκε παραπάνω, έννοια του συγκεκριμένου όρου. Για να υφίσταται το Technical Debt πρέπει να υπάρχει μία μελλοντική συνέπεια η οποία αναβλύζει από την ύπαρξή του, δηλαδή επιπλέον κόστος (συνήθως χρηματικό), προσπάθεια, κίνδυνος ή περιορισμός που σχετίζεται με τη διατήρηση της συγκεκριμένης τεχνικής κατάστασης. Η υπερβολικά γενική χρήση του όρου για κάθε πρόβλημα ποιότητας έχει αναγνωριστεί στη βιβλιογραφία ως πηγή ασάφειας.

Το Τεχνικό Χρέος και η Συντηρησιμότητα ενός λογισμικού σχετίζονται άμεσα και στενά. Ένα βασικό μέρος του «τόκου», που προκύπτει από την ύπαρξη του debt, εκδηλώνεται ως αυξημένη προσπάθεια κατά την κατανόηση, τροποποίηση, επιδιόρθωση και επέκταση του υπάρχοντος συστήματος. Όσο περισσότερες μελλοντικές αλλαγές πραγματοποιούνται πάνω σε μία μη βέλτιστη τεχνική βάση, τόσο μεγαλύτερο μπορεί να καταστεί το συνολικό κόστος διατήρησης της συγκεκριμένης απόφασης. Η ανεξέλεγκτη συσσώρευση χρέους μπορεί να περιορίσει καταλυτικά τη δυνατότητα εξέλιξης του λογισμικού. Αντίθετα, η πλήρης εξάλειψη κάθε μορφής χρέους δεν αποτελεί πάντα οικονομική ή τεχνικά βέλτιστη επιλογή. Η διαχείρισή του είναι ουσιαστικά μία διαδικασία λήψης αποφάσεων και εξισορρόπησης κόστους, αξίας και κινδύνου, η οποία εξαρτάται από το συγκεκριμένο έργο και τους μελλοντικούς στόχους που έχουν τεθεί.

Ο τρόπος αντιμετώπισης του Τεχνικού Χρέους αναφέρεται συνήθως ως Technical Debt Management (TDM) και περιλαμβάνει δραστηριότητες όπως είναι η αναγνώριση, η καταγραφή, η μέτρηση ή εκτίμηση, η παρακολούθηση, η ιεράρχηση και εν τέλει η αποπληρωμή ή η συνειδητή διατήρηση συγκεκριμένων αντικειμένων που προκαλούν χρέος. Η αποπληρωμή -βάσει οικονομικών ορολογιών- μπορεί να περιλαμβάνει ενέργειες όπως είναι το refactoring (αναδιάρθρωση κώδικα), ανασχεδιασμό, αντικατάσταση παλαιών τεχνικών λύσεων ή γενικότερα την πραγματοποίηση της τεχνικής εργασίας που είχε λάβει αναβολή σε προηγούμενο κύκλο εργασιών. Τονίζεται ιδιαίτερα ότι οι αποφάσεις σχετικά με το εάν και πότε πρέπει να αντιμετωπιστεί ένα debt item είναι έντονα και στενά συνδεδεμένες με το περιεχόμενο και χαρακτηρίζονται ως context-dependent.

Το Technical Debt λαμβάνει σημαντικό ρόλο στο πλαίσιο της παρούσας εργασίας και είναι σημαντικό κυρίως ως ευρύτερη συνέπεια της συσσώρευσης τεχνικών προβλημάτων ποιότητας κατά την εξέλιξη του πηγαίου κώδικα. Μετρικές όπως η πολυπλοκότητα, ο διπλότυπος κώδικας και τα maintainability-related issues μπορούν να παρέχουν ενδείξεις για τεχνικές καταστάσεις που ενδέχεται να συμβάλλουν στη δημιουργία ή αύξηση του χρέους, χωρίς όμως να αποτελούν από

μόνες τους πλήρη μέτρησή του. Η διάκριση αυτή είναι σημαντική, καθώς η εμπειρική ανάλυση εξετάζει άμεσα συγκεκριμένες μετρικές του πηγαίου κώδικα αγνοώντας το συνολικό οικονομικό ή οργανωτικό κόστος, χαρακτηριστικά ενός Technical Debt item.

3.9 Αναδιάρθρωση Κώδικα (Code Refactoring)

Το refactoring μεταβάλλει την εσωτερική δομή του κώδικα χωρίς να αλλάζει την εξωτερικά παρατηρήσιμη συμπεριφορά του. Η διαδικασία πραγματοποιείται συνήθως μέσω μικρών, ελεγχόμενων μετασχηματισμών και αποσκοπεί στη βελτίωση της σχεδίασης και της δυνατότητας συντήρησης (Opdyke, 1992; Fowler, 2018).

Βασικό σκοπό του refactoring αποτελεί η βελτίωση της υπάρχουσας τεχνικής δομής, αποφεύγοντας την προσθήκη νέας λειτουργικότητας, ώστε οι μελλοντικές αλλαγές να μπορούν να πραγματοποιούνται ευκολότερα και με μικρότερο κίνδυνο. Τυπικές επεμβάσεις μπορούν να περιλαμβάνουν τη διάσπαση μεγάλων τμημάτων κώδικα σε μικρότερες μονάδες, τη μετακίνηση λειτουργικότητας σε καταλληλότερα components, τη βελτίωση της ονοματοδοσίας ή την εξάλειψη περιττής επανάληψης. Μία τέτοια μεμονωμένη αλλαγή μπορεί να είναι μικρή, όμως η διαδοχική εφαρμογή πολλαπλών refactoring μπορεί να οδηγήσει σε σημαντική αναδιοργάνωση της εσωτερικής δομής του συστήματος.

Η περιγραφόμενη πρακτική συνδέεται στενά με την έννοια της Συντηρησιμότητας (Maintainability), καθώς επιδιώκει να καταστήσει τον υπάρχοντα κώδικα ευκολότερο στην κατανόηση και τροποποίηση. Code smells, duplication, υπερβολική πολυπλοκότητα και άλλες τεχνικές αδυναμίες μπορούν να λειτουργήσουν ως ενδείξεις ότι ένα τμήμα του συστήματος ενδέχεται να ωφεληθεί από αναδιάρθρωση. Παράλληλα, το refactoring αποτελεί έναν από τους βασικότερους μηχανισμούς μέσω του οποίου μπορεί να αντιμετωπιστεί ενεργητικά το Τεχνικό Χρέος, όταν η αποπληρωμή του απαιτεί βελτίωση της εσωτερικής σχεδίασης ή αντικατάσταση προσωρινών λύσεων. Διευκρινίζεται εδώ, ότι η ύπαρξη ενός Code Smell δεν συνεπάγεται αυτομάτως υποχρεωτική απαίτηση για refactoring καθώς επίσης ούτε κάθε refactoring οδηγεί αυτομάτως σε βελτίωση όλων των διαστάσεων ποιότητας, ή σε καθολική εξαφάνιση των quality issues.

Ο κλασικός ορισμός απαιτεί την διατήρηση της προϋπάρχουσας συμπεριφοράς του συστήματος λογισμικού. Για τον λόγο αυτό, η διαδικασία πραγματοποιείται συνήθως με μικρές και ελεγχόμενες αλλαγές, ενώ automated tests και κατάλληλα εργαλεία μπορούν να χρησιμοποιηθούν για να επιβεβαιώνεται ότι η υπάρχουσα λειτουργικότητα εξακολουθεί να συμπεριφέρεται όπως πριν. Στην πράξη, η εξασφάλιση της συγκεκριμένης προϋπόθεσης δεν είναι πάντα δυνατή. Όπως έχει δείξει η εργασία του Opdyke, η ασφαλής εφαρμογή ορισμένων transformation εξαρτάται από συγκεκριμένες προϋποθέσεις, ενώ μεταγενέστερη έρευνα έχει δείξει ότι ακόμη και αυτοματοποιημένα refactoring engines μπορούν να

πραγματοποιήσουν μετασχηματισμούς με αποτέλεσμα να προκληθούν compilation errors ή μεταβολές συμπεριφοράς. Αν και η αυτοματοποίηση μειώνει δραματικά το κόστος εφαρμογής ενός refactoring, δεν είναι σε θέση να εξαλειφθεί πλήρως η ανάγκη ανθρώπινης επαλήθευσης.

Από την υπάρχουσα εμπειρική βιβλιογραφία συμπεραίνεται ότι η επίδραση του refactoring στην ποιότητα είναι περισσότερο σύνθετη από μία απλή σχέση εξίσωσης *refactoring=βελτίωση*. Επιπλέον, παρόλο που το refactoring ορίζεται θεωρητικά ως behavior-preserving, οι πραγματικές δραστηριότητες αναδιάρθρωσης μπορούν να εισάγουν νέα defects. Σε μελέτη τους, οι Bavota et al. (2012) εξέτασαν refactoring δραστηριότητες σε τρία πραγματικά συστήματα Java και έδειξαν ότι ορισμένα refactorings είναι δυνατόν να συσχετιστούν με fault-inducing changes. Το εύρημα αυτό δεν αναιρεί τον ορισμό του refactoring, αλλά υπογραμμίζει τη διαφορά μεταξύ της ιδανικής behavior-preserving transformation και της πραγματικής εφαρμογής της από προγραμματιστές ή εργαλεία. Τα παραπάνω εξάγουν το συμπέρασμα ότι η εφαρμογή ενός refactoring στον πηγαίο κώδικα απαιτεί κατάλληλη επαλήθευση, ιδιαίτερα όταν επηρεάζονται μεγάλα ή σύνθετα τμήματα του συστήματος.

Στο πλαίσιο της παρούσας εργασίας, το refactoring έχει ιδιαίτερη σημασία αφού αποτελεί χαρακτηριστικό παράδειγμα μεταβολής πηγαίου κώδικα που μπορεί να βελτιώσει ορισμένες διαστάσεις ποιότητας (ή και ασφάλειας) χωρίς να εγγυάται ότι όλες οι υπόλοιπες ιδιότητες του συστήματος θα παραμείνουν ανεπηρέαστες. Για παράδειγμα, μία αναδιάρθρωση μπορεί να μειώσει duplication ή πολυπλοκότητα και να βελτιώσει τη maintainability, αλλά ταυτόχρονα μία λανθασμένη εφαρμογή του μπορεί να εισαγάγει regression ή άλλο defect. Επομένως, το refactoring αποτελεί χαρακτηριστική περίπτωση όπου η κατεύθυνση μιας αλλαγής δεν πρέπει να θεωρείται εκ των προτέρων θετική μόνο από την πρόθεσή της. Η πραγματική επίδρασή της πρέπει να αξιολογείται με βάση παρατηρούμενες μεταβολές στις αντίστοιχες μετρικές.

3.10 Αυτοματοποιημένες μετρικές ποιότητας και Quality Gates

Η αξιολόγηση της ποιότητας λογισμικού σε σύγχρονα έργα ανάπτυξης πραγματοποιείται σε σημαντικό βαθμό μέσω αυτοματοποιημένων μετρικών ποιότητας (automated quality metrics). Πρόκειται για ποσοτικές μετρήσεις που παράγονται από εργαλεία ανάλυσης με συστηματικό και επαναλήψιμο τρόπο και χρησιμοποιούνται για την αποτύπωση συγκεκριμένων χαρακτηριστικών του πηγαίου κώδικα ή του προϊόντος λογισμικού. Οι μετρήσεις μπορούν να αφορούν διαφορετικές πτυχές, όπως είναι η συντηρησιμότητα, η αξιοπιστία, η πολυπλοκότητα, ο διπλότυπος κώδικας ή διάφορα άλλα χαρακτηριστικά που εξετάζονται από το χρησιμοποιούμενο εργαλείο. Φυσικά, για τον προσδιορισμό τους έχουν αναπτυχθεί τα ISO/IEC 25020:2019 το οποίο παρέχει το γενικό πλαίσιο για την ανάπτυξη, επιλογή, εφαρμογή και ερμηνεία των μετρήσεων ποιότητας καθώς επίσης

και το ISO/IEC 25023:2016 το οποίο καθορίζει συγκεκριμένες μετρήσεις για την ποσοτική αξιολόγηση χαρακτηριστικών ποιότητας του προϊόντος λογισμικού.

Η συλλογή τους με αυτοματοποιημένα μέσα παρέχει σημαντικά πλεονεκτήματα, ιδιαίτερα όταν εξετάζεται μεγάλος αριθμός αλλαγών ή διαφορετικών εκδόσεων του ίδιου συστήματος. Η εφαρμογή των ίδιων κανόνων επιτρέπει τη δημιουργία συγκρίσιμων και επαναλαμβανόμενων αποτελεσμάτων, περιορίζοντας παράλληλα την ανάγκη χειροκίνητης αξιολόγησης κάθε επιμέρους στοιχείου του κώδικα. Στο SonarQube -το συγκεκριμένο εργαλείο χρησιμοποιήθηκε ενεργά για την συγκέντρωση μετρήσεων από έργα λογισμικού προς ανάλυση για συμπλήρωση του dataset που εξετάστηκε στην εμπειρική ανάλυση-, για παράδειγμα, παράγονται μετρικές που αφορούν την αξιοπιστία, την συντηρησιμότητα και security issues, test coverage, πολυπλοκότητα, διπλότυπο κώδικα και διάφορα άλλα χαρακτηριστικά. Οι ίδιες μετρικές μπορούν να αφορούν είτε συνολικό υπάρχον codebase είτε αποκλειστικά τον νέο ή τροποποιημένο κώδικα.

Στο SonarQube, ένα Quality Gate αποτελεί σύνολο συνθηκών που εφαρμόζονται στα αποτελέσματα της ανάλυσης. Η επιτυχία ή αποτυχία του gate δείχνει αν ο κώδικας ικανοποιεί την καθορισμένη πολιτική και μπορεί να χρησιμοποιηθεί ως κριτήριο συνέχισης της ροής παράδοσης (SonarSource, χ.χ.-c). Κάθε συνθήκη που ορίζεται σε ένα SonarQube Quality Gate αποτελείται από μία μετρική, έναν τελεστή σύγκρισης και μία οριακή τιμή (error value). Το συνολικό αποτέλεσμα εκφράζεται ως pass ή fail, ανάλογα με την ύπαρξη παραβιάσεων ή μη κάποιας από τις καθορισμένες συνθήκες.

Επί παραδείγματι, μία ομάδα δύναται να καθορίσει μη επιτρεπόμενα νέα issues συγκεκριμένης βαρύτητας και σοβαρότητας, κατώτατο ποσοστιαίο όριο διπλότυπου κώδικα ή αποδεκτό εύρος τιμών αξιολόγησης της συντηρησιμότητας. Τα thresholds δεν αποτελούν προκαθορισμένους και καθολικούς νόμους ποιότητας και πρέπει να επιλέγονται σύμφωνα με το είδος του συστήματος, τους στόχους και τις απαιτήσεις του έργου. Χαρακτηριστικό αποτελεί η μη αντιστοίχιση που ορίζεται στο πρότυπο ISO/IEC 25023:2016 των μετρήσεων ποιότητας σε καθολικές βαθμίδες συμμόρφωσης, καθώς τα κατάλληλα όρια εξαρτώνται από παράγοντες όπως η κατηγορία του προϊόντος, οι ανάγκες των χρηστών και το απαιτούμενο επίπεδο ακεραιότητας. Κατά συνέπεια, ένα Quality Gate προσεγγίζει περισσότερο τον ορισμό μίας κωδικοποίησης συγκεκριμένης και προσυμφωνημένης πολιτικής ποιότητας η οποία ακολουθείται παρά αντικειμενικό και καθολικό ορισμό του «ποιοτικού λογισμικού».

Τα Quality Gates αποκτούν ιδιαίτερη χρησιμότητα όταν ενσωματώνονται σε CI/CD pipelines και Pull Request workflows. Μετά την ανάλυση μιας αλλαγής, το αποτέλεσμα του Quality Gate μπορεί να επιστρέφεται στο pipeline ή στην πλατφόρμα διαχείρισης του repository, όπως είναι για παράδειγμα το GitHub. Ανάλογα με τη διαμόρφωση, μία αποτυχία του Quality Gate μπορεί να αποτρέψει την ενσωμάτωση ενός Pull Request ή να προκαλέσει αποτυχία ενός συνολικού CI workflow. Στη σύγχρονη προσέγγιση του εργαλείου SonarQube δίνεται ιδιαίτερη έμφαση στον new code, δηλαδή στον κώδικα ο οποίος προστίθεται ή τροποποιείται σε σχέση με μία κατάσταση αναφοράς, η οποία συνήθως είναι η κατάσταση του

κώδικα πριν το Pull Request γίνει αποδεκτό και συγχωνευθεί με την κύρια ροή ανάπτυξης του λογισμικού. Στην ανάλυση Pull Requests εφαρμόζονται οι συνθήκες του Quality Gates, ονομαζόμενες ως Quality Gates Conditions, που αφορούν νέο κώδικα, επιτρέποντας στις ομάδες να εμποδίζουν την εισαγωγή νέων προβλημάτων χωρίς να απαιτείται η άμεση επίλυση ολόκληρου του ιστορικού χρέους του συστήματος.

Στο περιβάλλον του SonarQube διακρίνονται το Quality Profile από το Quality Gate ως έννοιες. Το Quality Profile καθορίζει ποιοι κανόνες εφαρμόζονται κατά την ανάλυση του κώδικα και επομένως για ποια είδη issues υπάρχει η δυνατότητα αντιμετώπισής τους. Το Quality Gate χρησιμοποιεί στη συνέχεια τα αποτελέσματα και τις παραγόμενες μετρικές για να αποφασίσει εάν ικανοποιούνται ή όχι οι προκαθορισμένες απαιτήσεις -οι απαιτήσεις αυτές καθορίζονται συνήθως σε συνεννόηση από τα μέλη/προγραμματιστές της ομάδας ανάπτυξης του λογισμικού-. Απλουστευμένα, το Quality Profile καθορίζει σε μεγάλο βαθμό τι ελέγχεται, ενώ το δεύτερο καθορίζει ποια αποτελέσματα θεωρούνται έγκυρα και αποδεκτά.

Αν και η πρακτική τους αξία ανάγεται σε μεγάλο μέγεθος, τόσο οι αυτοματοποιημένοι υπολογισμοί των μετρικών (ή αυτοματοποιημένες μετρικές) όσο και τα Quality Gates παρουσιάζουν περιορισμούς. Η ποιότητα λογισμικού είναι πολυδιάστατη και δεν μπορεί να συμπυκνωθεί πλήρως σε ένα σύνολο αριθμητικών κατώτατων ορίων. Υπάρχει η πιθανότητα ένα έργο, συγκεκριμένα ένα καινούριο ανεπτυγμένο μέρος πηγαίου κώδικα, να περάσει επιτυχώς τους ελέγχους ενός Quality Gate και παρ' όλα αυτά να διαθέτει προβλήματα τα οποία δεν ανιχνεύονται από τους ενεργούς κανόνες ή δεν αποτυπώνονται στις επιλεγμένες μετρικές. Αντίστροφα, μία αποτυχία ενός Quality Gate δεν συνεπάγεται απαραίτητα λογισμικό χαμηλής ποιότητας, αλλά κυρίως καταδεικνύει την παραβίαση μίας συγκεκριμένης πολιτικής που είχε προηγουμένως οριστεί. Για τον λόγο αυτό, τα automated metrics και Quality Gates πρέπει να λειτουργούν ως μηχανισμοί συστηματικής υποστήριξης και αξιολόγησης και όχι ως υποκατάστατο της τεχνικής κρίσης, του code review και της συνολικότερης αξιολόγησης του προϊόντος.

Οι αυτοματοποιημένες μετρικές κατέχουν στην παρούσα εργασία σημαντικό ρόλο, καθώς επιτρέπουν την ποσοτική και συνεπή σύγκριση διαφορετικών καταστάσεων του μεταβαλλόμενου πηγαίου κώδικα. Τα Quality Gates παρουσιάζονται ως μία σημαντική πρακτική εφαρμογή αυτών των μετρήσεων στη σύγχρονη ανάπτυξη λογισμικού, αλλά δεν χρησιμοποιούνται ως αυτόνομος δείκτης στατιστικής αξιολόγησης της ποιότητας. Το ερευνητικό ενδιαφέρον επικεντρώνεται στις υποκείμενες μετρικές και στις μεταβολές τους, καθώς ένα συνολικό αποτέλεσμα με δυαδική τιμή pass/fail θα συμπύκνωνε σημαντικά πολλαπλές διαστάσεις και θα εξαρτιόταν σημαντικά από τα συγκεκριμένα thresholds και τους κανόνες που έχουν επιλεγεί.

3.11 Περιορισμοί αυτοματοποιημένων μετρικών ποιότητας

Οι αυτοματοποιημένες μετρικές ποιότητας παρέχουν έναν συστηματικό, επαναλήψιμο, και τις περισσότερες φορές, ολοκληρωμένο τρόπο εξέτασης χαρακτηριστικών του λογισμικού, αν και τα αποτελέσματά τους δεν θα πρέπει να αντιμετωπίζονται ως απόλυτη αναπαράσταση της πραγματικής ποιότητας ενός συστήματος. Η ποιότητα λογισμικού αποτελεί σύνθετη και πολυδιάστατη έννοια, ενώ μία μετρική αποτυπώνει συνήθως μόνο μία συγκεκριμένη παρατηρήσιμη και αριθμητικά προσεγγίσιμη ιδιότητά της. Λαμβάνοντας ως παράδειγμα την μέτρηση της πολυπλοκότητας, του διπλότυπου κώδικα ή του αριθμού των maintainability-related issues, αν και μπορεί να παρέχει χρήσιμες ενδείξεις για την τεχνική κατάσταση του πηγαίου κώδικα, δεν αποτυπώνει πλήρως την πραγματική ευκολία (ή αντίστοιχα δυσκολία) συντήρησης ή τη συνολική ποιότητα του τελικού προϊόντος. Στο πρότυπο ISO/IEC 25020:2019 αναγνωρίζεται η ανάγκη αυτή, καθώς περιλαμβάνει διαδικασίες για την αξιολόγηση τόσο της αξιοπιστίας (reliability) της μέτρησης όσο και της εγκυρότητας (validity) των quality measures.

Ένας βασικός περιορισμός αφορά την εγκυρότητα της αντιστοίχισης μεταξύ μιας μετρικής και της ιδιότητας που αυτή επιχειρεί να αναπαραστήσει. Πολλές μετρικές λογισμικού ουσιαστικά είναι αριθμητικές καταμετρήσεις ή παράγωγοι δείκτες που χρησιμοποιούνται ως proxies άλλων περισσότερο αφηρημένων εννοιών. Οι Flater et al. (2016) έχουν επισημάνει ότι, αντίθετα των φυσικών μετρήσεων, πολλές μετρικές λογισμικού αποτελούνται από απλές καταμετρήσεις οι οποίες χρησιμοποιούνται ως indicators σύνθετων και αφηρημένων χαρακτηριστικών. Βάσει αυτών, η αύξηση ή η μείωση μιας μετρικής δεν πρέπει να εξισώνεται μηχανιστικά με αντίστοιχη αύξηση ή μείωση της συνολικής ποιότητας. Για την ερμηνεία, θα πρέπει πάντοτε να λαμβάνεται υπόψη τι ακριβώς μετρά η συγκεκριμένη μεταβλητή και τι δεν είναι σε θέση να μετρήσει.

Ένας δεύτερος εξίσου σημαντικός περιορισμός είναι η εξάρτηση των αποτελεσμάτων από το εκάστοτε χρησιμοποιούμενο εργαλείο, τους καθορισμένους κανόνες και τη χρησιμοποιούμενη μεθοδολογία ανάλυσης. Διαφορετικά εργαλεία μπορούν να εφαρμόζουν διαφορετικούς κανόνες, ορισμούς και τεχνικές εντοπισμού με αποτέλεσμα να παράγουν διαφορετικά αποτελέσματα μεταξύ τους. Το συγκεκριμένο γεγονός έχει εμπεριστατωθεί επιστημονικά μέσω της ερευνητικής σύγκρισης των Lenarduzzi et al. (2023) σε σύγκριση έξι δημοφιλών static analysis tools -οι ονομασίες τους είναι άσχετες με την παράθεση των συγκεκριμένων πληροφοριών και παραλείπονται- σε 47 Java έργα, εντόπισαν μικρή έως και μηδενική συμφωνία μεταξύ των εργαλείων ως προς ορισμένες κατηγορίες ευρημάτων και περιορισμένη precision. Το εύρημα αυτό δείχνει ότι ένα αυτοματοποιημένο αποτέλεσμα δεν μπορεί να είναι ανεξάρτητο από το measurement instrument που χρησιμοποιείται. Για μία εμπειρική έρευνα είναι σημαντικό επομένως να διατηρούνται το δυνατόν σταθερά το εργαλείο, οι κανόνες και οι συνθήκες ανάλυσης μεταξύ των διαφορετικών συγκεντρωμένων παρατηρήσεων.

Στην περίπτωση των μετρικών που βασίζονται στον αυτόματο εντοπισμό issues, πρόσθετος περιορισμός είναι η παρουσία των false positives και των false negatives. Ένα false positive προκύπτει όταν το εργαλείο αναφέρει ένα πρόβλημα το

οποίο, μετά από ενδελεχή εξέταση, δεν αποτελεί πραγματικό πρόβλημα, ενώ αντίστοιχα ένα false negative αφορά ένα πραγματικό πρόβλημα το οποίο δεν είναι σε θέση να εντοπίσει το εργαλείο. Η εμπειρική έρευνα πάνω στο συγκεκριμένο πρόβλημα έχει δείξει ότι το φαινόμενο αποτελεί πραγματικό περιορισμό των automated static analysis tools και μπορεί να επηρεάζει τόσο την πρακτική χρήση όσο και την εμπιστοσύνη των προγραμματιστών στα αποτελέσματα που εξάγονται. Μία από τις πιο πρόσφατες μελέτες επικεντρωμένες σε PMD και SonarQube επιβεβαίωσε ότι συγκεκριμένοι κανόνες μπορούν να παρουσιάζουν false-positive αποτελέσματα και ότι η συμπεριφορά αυτή τείνει να διαφοροποιηθεί μεταξύ κανόνων αλλά και datasets.

Ωστόσο, η συγκεκριμένη παρατήρηση χρήζει μίας σημαντικής διάκρισης. Οι έννοιες false positive και false negative εφαρμόζονται κυρίως σε ευρήματα που προκύπτουν από μηχανισμούς detection, όπως bugs, code smells ή άλλα rule violations. Η έννοιά τους δεν αποδίδεται ίδια για μία ντετερμινιστικά υπολογιζόμενη μετρική, όπως ένας αριθμός γραμμών κώδικα ή μία τιμή complexity. Στις τελευταίες εξ αυτών περιπτώσεις, το βασικό ζήτημα δεν έγκειται συνήθως στο εάν η αριθμητική τιμή αποτελεί false positive αποτέλεσμα, αλλά εάν ο ορισμός και ο τρόπος υπολογισμού της μετρικής αποτυπώνουν κατάλληλα το χαρακτηριστικό που ενδιαφέρει τον ερευνητή. Η συγκεκριμένη διάκριση κρίνεται ουσιώδους προκειμένου να αποφευχθεί η σύγχυση της κοινής αντιμετώπισης όλων των αυτοματοποιημένων μετρικών ως μετρήσεις του ίδιου είδους.

Οι απόλυτες τιμές, επιπλέον, πολλών μετρικών επηρεάζονται άμεσα και σημαντικά από το μέγεθος καθώς και το πλαίσιο του εξεταζόμενου λογισμικού. Φυσιολογικά, ένα μεγαλύτερο σε μέγεθος λογισμικό τείνει να παρουσιάσει περισσότερα issues στο σύνολό τους ή μεγαλύτερες συνολικές τελικές τιμές σε ορισμένες μετρήσεις απλώς επειδή περιλαμβάνει περισσότερο κώδικα. Για τον λόγο αυτό χρησιμοποιούνται συχνά normalized metrics -όπως και η παρούσα έρευνα αναγάγει τις υπολογισμένες μετρικές σε normalized metrics, η διαδικασία περιγράφεται αναλυτικά στο ΜΕΡΟΣ Β του παρόντος- ή δείκτες πυκνότητας, με σκοπό να επιτραπεί περισσότερη συγκρίσιμη αξιολόγηση διαφορετικών μονάδων λογισμικού. Οι κανονικοποιημένες τιμές με την σειρά τους χρήζουν προσεκτικής ερμηνείας, ιδιαίτερα σε περιπτώσεις όπου ο παρονομαστής είναι πολύ μικρός ή παρουσιάζει σημαντικές μεταβολές. Στο πρότυπο ISO/IEC 25020 περιλαμβάνονται ειδικά normalized measurement functions, γεγονός που υπογραμμίζει την σημαντικότητα της επιλογής κατάλληλης μορφής μέτρησης καθώς και ότι η επιλογή αυτή αποτελεί μέρος της ίδιας της διαδικασίας μέτρησης ποιότητας και όχι απλώς μία τεχνική λεπτομέρεια (technicality).

Αντίστοιχα, δεν έχουν οριστεί στην δημοσιευμένη βιβλιογραφία καθώς επίσης και σε πρότυπα καθολικά thresholds που να διαχωρίζουν με απόλυτο τρόπο το λογισμικό με «υψηλή» ποιότητα από το λογισμικό με «χαμηλή» ποιότητα. Το πρότυπο ISO/IEC 25023:2016 αποφεύγει σκόπιμα να αντιστοιχίσει γενικά εύρη τιμών σε επίπεδα συμμόρφωσης, διότι τα κατάλληλα όρια εξαρτώνται από τη φύση του προϊόντος, την κατηγορία του, το απαιτούμενο επίπεδο ακεραιότητας και τις ανάγκες των χρηστών. Ένα threshold που είναι κατάλληλο για ένα project δεν είναι απαραίτητο να είναι εξίσου κατάλληλο για ένα άλλο. Ο ίδιος περιορισμός βρίσκεται

εφαρμογή και στα Quality Gates όπου η επιτυχία ενός gate σημαίνει ότι ικανοποιούνται συγκεκριμένες προκαθορισμένες συνθήκες χωρίς όμως να έχει αποδειχθεί με κάποιον συγκεκριμένο τρόπο η συνολική ποιότητα του προϊόντος.

Ιδιαίτερα όταν οι μετρικές παράγονται από στατική ανάλυση του πηγαίου κώδικα, υπάρχουν επίσης διαστάσεις της ποιότητας που δεν μπορούν να παρατηρηθούν, και κατά συνέπεια να αναλυθούν, πλήρως. Η πραγματική κατά την εκτέλεση συμπεριφορά του λογισμικού, η εμπειρία του χρήστη, ορισμένες μορφές performance (απόδοσης), οι αλληλεπιδράσεις με εξωτερικά συστήματα και γενικότερα η External Quality συνήθως απαιτούν διαφορετικές μορφές αξιολόγησης και υπολογισμού τους. Συνεπώς, ένα σύστημα το οποίο μπορεί να παρουσιάζει ευνοϊκές μετρικές πηγαίου κώδικα δεν είναι ορθό να χαρακτηριστεί αυτομάτως και συνολικά ως υψηλής ποιότητας λογισμικό. Η σύγχρονη βιβλιογραφία γύρω από τη software metrology επισημαίνει ακριβώς τη δυσκολία μετατροπής σύνθετων χαρακτηριστικών του λογισμικού σε πλήρως αντικειμενικές και καθολικά συγκρίσιμες ποσοτικές τιμές.

Συμπληρωματικά, οι διαφορετικές τιμές δεν είναι απαραίτητα ανεξάρτητες μεταξύ τους. Πολλές από αυτές μπορούν να επηρεάζονται από κοινά χαρακτηριστικά, όπως είναι το μέγεθος ή η δομή του κώδικα, ή να αποτυπώνουν επικαλυπτόμενες πλευρές της ποιότητας. Η ταυτόχρονη χρήση μεγάλου αριθμού μετρικών χωρίς εξέταση των μεταξύ τους σχέσεων μπορεί κατά συνέπεια να οδηγήσει σε πιθανή πλεονάζουσα πληροφορία ή σε λανθασμένη απόδοση ιδιαίτερης σημασίας σε μεταβλητές που ουσιαστικά μεταβάλλονται μαζί. Επιπλέον, η ύπαρξη στατιστικής σχέσης μεταξύ δύο metrics δεν αποδεικνύει ότι η μεταβολή της μίας προκαλεί την άμεση ή έμμεση μεταβολή της άλλης. Οι αυτοματοποιημένες μετρικές θα πρέπει επομένως να αποτελούν αντικείμενο κατάλληλης στατιστικής και επιστημονικής ερμηνείας και όχι απλής σύγκρισης συνόλου αριθμητικών τιμών.

Οι προαναφερθέντες περιορισμοί λαμβάνονται υπόψη κατά την ερμηνεία των αποτελεσμάτων της παρούσας εμπειρικής ανάλυσης. Οι μετρικές που παράγονται από την αυτοματοποιημένη ανάλυση αντιμετωπίζονται ως ποσοτικοί δείκτες συγκεκριμένων χαρακτηριστικών του πηγαίου κώδικα καθώς αποφεύγεται η αντιμετώπισή τους ως άμεσες, απόλυτες μετρήσεις της συνολικής ποιότητας. Για τον λόγο αυτό εξετάζονται πολλαπλές μετρικές, λαμβάνεται υπόψη το συνολικό μέγεθος του κώδικα -το οποίο όπου χρειάζεται κανονικοποιείται- και διερευνώνται τόσο οι μεταξύ τους σχέσεις όσο και οι διαφοροποιήσεις που εμφανίζονται μεταξύ περιβαλλόντων ανάπτυξης διαφορετικών projects. Η προσέγγιση αυτή επιτρέπει να αξιοποιηθούν τα πλεονεκτήματα της αυτοματοποιημένης μέτρησης χωρίς να αγνοούνται οι θεωρητικοί και εμπειρικοί περιορισμοί που έχουν τεθεί.

Ασφάλεια Λογισμικού – Software Security

Η Ασφάλεια Λογισμικού (Software Security) αφορά την ικανότητα ενός συστήματος να προστατεύει τα δεδομένα, τις λειτουργίες και τους διαθέσιμους πόρους του από μη εξουσιοδοτημένη πρόσβαση, τροποποίηση, αποκάλυψη ή διακοπή της λειτουργίας τους. Αποτελεί θεμελιώδη διάσταση της σύγχρονης ανάπτυξης λογισμικού, καθώς αδυναμίες στον σχεδιασμό ή στην υλοποίηση είναι ικανές να δημιουργήσουν ευπάθειες που ενδέχεται να αξιοποιηθούν από κακόβουλους χρήστες και να επηρεάσουν την εμπιστευτικότητα, την ακεραιότητα ή τη διαθεσιμότητα ενός συστήματος.

Η ασφάλεια δεν περιορίζεται αποκλειστικά στην αντιμετώπιση προβλημάτων έπειτα από την ολοκλήρωση της ανάπτυξης. Είναι σημαντικό να λαμβάνεται υπόψη σε ολόκληρο τον κύκλο ζωής του λογισμικού. Στο παρόν κεφάλαιο παρουσιάζονται οι βασικές έννοιες και αρχές της Software Security, οι συνηθέστερες μορφές αδυναμιών και ευπαθειών, καθώς και οι μηχανισμοί και πρακτικές που χρησιμοποιούνται για την αναγνώριση, αξιολόγηση και περιορισμό τους.

4.1 Θεμελιώδεις έννοιες της Ασφάλειας Λογισμικού

Η Ασφάλεια Λογισμικού (Software Security) βασίζεται σε ένα σύνολο θεμελιωδών εννοιών που επιτρέπουν τη συστηματική κατανόηση του τρόπου κατά τον οποίο ένα σύστημα μπορεί να προστατευθεί απέναντι σε σφάλματα, κακόβουλες ενέργειες και ανεπιθύμητες καταστάσεις. Οι έννοιες αυτές δεν αφορούν μόνο την ύπαρξη τεχνικών μηχανισμών προστασίας, αλλά διαμορφώνουν ένα ευρύτερο πλαίσιο μέσα στο οποίο μπορούν να περιγραφούν ο στόχοι της ασφάλειας, οι πιθανές πηγές κινδύνου, οι αδυναμίες ενός συστήματος και οι μηχανισμοί ελέγχου της πρόσβασης σε αυτό. Η κατανόησή τους αποτελεί απαραίτητη προϋπόθεση για την ανάλυση και αξιολόγηση της ασφάλειας ενός σύγχρονου συστήματος λογισμικού.

Στο πρώτο επίπεδο συναντάμε τους βασικούς στόχους προστασίας των πληροφοριών και των υπηρεσιών ενός συστήματος. Οι στόχοι αυτοί προσδιορίζουν τι ακριβώς επιχειρεί μία στρατηγική ασφαλείας να διαφυλάξει και αποτελούν σημείο αναφοράς για τον σχεδιασμό κατάλληλων μηχανισμών προστασίας. Η υλοποίηση ενός ολοκληρωμένου συστήματος με αρχές ασφάλειας προϋποθέτει την ύπαρξη της δυνατότητας αναγνώρισης χρηστών ή άλλων οντοτήτων που αλληλοεπιδρούν με το σύστημα και τον καθορισμό των ενεργειών που επιτρέπεται να πραγματοποιούν. Με τον τρόπο αυτό, η προστασία δεν περιορίζεται αποκλειστικά στην προσπάθεια αποτροπής εξωτερικών επιθέσεων, αλλά επεκτείνεται επίσης στον ορθό έλεγχο της πρόσβασης και των κατεχόμενων δικαιωμάτων στο εσωτερικό του συστήματος.

Η κατανόηση των εννοιών που περιγράφουν πιθανές πηγές και μηχανισμούς αστοχίας ή παραβίασης της ασφάλειας κρίνεται εξίσου σημαντική. Ένα σύστημα δύναται να περιλαμβάνει αδυναμίες στον σχεδιασμό, στην υλοποίηση ή στη

διαμόρφωσή του, οι οποίες υπό συγκεκριμένες συνθήκες μπορούν να χρησιμοποιηθούν κακόβουλα από μία αξιοποιούμενη απειλή απέναντι στο σύστημα. Η διάκριση μεταξύ της ύπαρξης της αδυναμίας, της δυνατότητας εκμετάλλευσης της και της πραγματικής αξιοποίησής της είναι ιδιαίτερως σημαντική, καθώς οι έννοιες αυτές χρησιμοποιούνται για την περιγραφή διαφορετικών σταδίων και πτυχών ενός πιθανού περιστατικού ασφαλείας.

Η εξέταση της ασφάλειας λογισμικού κρίνεται αναγκαίο να πραγματοποιείται μέσα από μία προσέγγιση βασισμένη σε κάποιον πιθανό κίνδυνο. Η ύπαρξη ενός προβλήματος δεν συνεπάγεται ότι όλες οι περιπτώσεις έχουν την ίδια σημασία ή προτεραιότητα στην αντιμετώπισή τους. Απαιτείται εκτίμηση της πιθανότητας εμφάνισης και των πιθανών επιπτώσεων, καθώς και κατανόηση των σημείων μέσω των οποίων ένα σύστημα μπορεί να δεχθεί αλληλεπίδραση ή επίθεση. Για τον λόγο αυτό, έννοιες που σχετίζονται με τον κίνδυνο και την επιφάνεια επίθεσης κρίνονται απαραίτητες για την ιεράρχηση των μέτρων προστασίας και την αποτελεσματική κατανομή των διαθέσιμων πόρων ασφαλείας.

Οι παραπάνω έννοιες αποτελούν ένα ενιαίο θεωρητικό πλαίσιο το οποίο ανάγεται στους στόχους προστασίας που καθορίζουν τι πρέπει να διαφυλαχθεί, οι μηχανισμοί ταυτοποίησης και ελέγχου πρόσβασης που καθορίζουν τα άτομα που τους επιτρέπεται η επικοινωνία καθώς και ο τρόπος με τον οποίο δύναται να πραγματοποιηθεί η επικοινωνία αυτή, οι έννοιες που σχετίζονται με απειλές και αδυναμίες περιγράφουν τους τρόπους με τους οποίους μία παραβίαση μπορεί να προκύψει, ενώ η αξιολόγηση κινδύνου επιτρέπει την συνολική εκτίμηση της σημασίας της προτεραιότητάς της.

4.1.1 Εμπιστευτικότητα (Confidentiality) – Ακεραιότητα (Integrity) – Διαθεσιμότητα (Availability)

Η Εμπιστευτικότητα (Confidentiality), η Ακεραιότητα (Integrity) και η Διαθεσιμότητα (Availability) αποτελούν τρεις από τους πιο θεμελιώδεις στόχους της ασφάλειας πληροφοριών και συστημάτων και βιβλιογραφικά είναι ευρέως ορισμένες και γνωστές ως CIA Triad (Το Τρίγωνο CIA). Το μοντέλο λειτουργεί ως ένα βασικό εννοιολογικό πλαίσιο για την κατανόηση του τι επιχειρεί να προστατεύσει ένα σύστημα. Η Εμπιστευτικότητα σχετίζεται με την αποτροπή μη εξουσιοδοτημένης αποκάλυψης πληροφοριών σε τρίτους -κακόβουλους συνήθως χρήστες ή πλευρές-, η Ακεραιότητα με την προστασία από μη επιτρεπτή ή ανεπιθύμητη αλλοίωση των δεδομένων στα οποία ένα σύστημα βασίζει την καλή του λειτουργία και η Διαθεσιμότητα με τη δυνατότητα έγκαιρης και αξιόπιστης πρόσβασης στις πληροφορίες και στις υπηρεσίες. Οι παραπάνω τρεις ιδιότητες, έχουν επίσης αντιμετωπιστεί από το NIST ως Security Objectives, ενώ το FIPS 199 τις χρησιμοποιεί ως θεμελιώδη βάση για την κατηγοριοποίηση των πιθανών επιπτώσεων μιας παραβίασης ασφαλείας. Αναλυτικότερα:

Η εμπιστευτικότητα αφορά την προστασία των περιορισμών πρόσβασης και αποκάλυψης πληροφοριών. Μαζί με την ακεραιότητα και τη διαθεσιμότητα αποτελεί

έναν από τους τρεις βασικούς στόχους κατηγοριοποίησης ασφάλειας του FIPS 199 (NIST, 2004). Η έννοια δεν αφορά μόνο την αποθήκευση δεδομένων, αλλά μπορεί να επεκτείνεται σε ολόκληρο τον κύκλο ζωής τους, συμπεριλαμβανομένης της επεξεργασίας και της μεταφοράς τους. Παράδειγμα παραβίασης της εμπιστευτικότητας μπορεί να ορισθεί η περίπτωση όπου ευαίσθητες πληροφορίες εκτεθούν σε μη εξουσιοδοτημένο χρήστη, δημοσιοποιηθούν ακούσια ή αποκτηθούν από έναν επιτιθέμενο. Η σημασία της εξαρτάται από το είδος των δεδομένων και τις συνέπειες της πιθανής τους αποκάλυψης.

Η Ακεραιότητα (Integrity) σχετίζεται με την προστασία των πληροφοριών του συστήματος απέναντι σε μη εξουσιοδοτημένη ή ακατάλληλη τροποποίηση και καταστροφή. Σκοπός της είναι να διασφαλίζεται ότι τα δεδομένα και οι κρίσιμες καταστάσεις του συστήματος διατηρούνται ορθά και δεν μεταβάλλονται με τρόπο που παραβιάζει τις απαιτήσεις του. Έχει επίσης συνδεθεί με έννοιες όπως η αυθεντικότητα και η μη αποποίηση. Σε επίπεδο δεδομένων ενός συστήματος, η ακεραιότητα αφορά ιδιαίτερα τη διατήρηση της πληροφορίας χωρίς μη εξουσιοδοτημένη μεταβολή κατά τη δημιουργία, αποθήκευση ή μεταφορά της. Παραβιάσεις ακεραιότητας μπορούν να αποτελέσουν περιπτώσεις μεταβολής δεδομένων από μία βάση, μη εξουσιοδοτημένης τροποποίησης διαμορφώσεων και ρυθμίσεων ή αλλοίωση του πηγαίου ή/και εκτελέσιμου κώδικα.

Η τρίτη στην σειρά, Διαθεσιμότητα (Availability), αφορά τη δυνατότητα των εξουσιοδοτημένων χρηστών και συστημάτων να έχουν έγκαιρη και αξιόπιστη πρόσβαση στις απαιτούμενες πληροφορίες και υπηρεσίες όταν αυτές είναι απαραίτητες. Ένα σύστημα θεωρείται ανεπαρκώς ασφαλές εάν οι νόμιμοι εξουσιοδοτημένοι χρήστες του δεν μπορούν να χρησιμοποιήσουν τις υπηρεσίες του όταν τις χρειάζονται, ακόμα και με αποτελεσματικά υλοποιημένες τις δύο παραπάνω αρχές. Προβλήματα διαθεσιμότητας συνήθως προκύπτουν από κακόβουλες ενέργειες, όπως εξάντληση πόρων, αλλά και από τεχνικές αστοχίες, λανθασμένες ρυθμίσεις ή άλλα γεγονότα που καθιστούν μία υπηρεσία μη προσβάσιμη και, ως επί τω πλείστων, μη εκτελέσιμη.

Οι τρεις παραπάνω στόχοι δεν πρέπει να αντιμετωπίζονται ανεξάρτητα, αλλά συνδυαστικά. Ένα ασφαλές σύστημα δεν αρκεί να ικανοποιεί μόνο τους αυτούς, καθώς η παραβίαση οποιασδήποτε διάστασης είναι ικανή να προκαλέσει σημαντικές συνέπειες στο περιβάλλον του λογισμικού. Παραδείγματος χάριν, η προστασία μιας πληροφορίας από μη εξουσιοδοτημένη ανάγνωση δεν κρίνεται επαρκής εάν ένας επιτιθέμενος είναι σε θέση να την τροποποιήσει, ενώ η προστασία τόσο της εμπιστευτικότητας όσο και της ακεραιότητάς της έχει περιορισμένη πρακτική αξία εάν η πληροφορία δεν είναι διαθέσιμη όταν αυτό απαιτείται. Πρακτικά, οι απαιτήσεις των τριών ιδιοτήτων μπορεί επίσης να χρειάζονται εξισορρόπηση. Πολύ αυστηρά μέτρα προστασίας, θα περιορίζαν τη παρεχόμενη χρηστικότητα ή την άμεση πρόσβαση σε μία υπηρεσία, ενώ υπερβολική έμφαση στη διαθεσιμότητα χωρίς κατάλληλες δικλίδες μπορεί να αυξάνει την έκθεση πληροφοριών. Η επιλογή των αντίστοιχων security controls θα πρέπει κάθε φορά να προσαρμόζεται στις απαιτήσεις και στους κινδύνους που ενέχει η χρήση του εκάστοτε συστήματος.

Παρότι το CIA Triad αποτελεί θεμελιώδες και καθολικά αποδεκτό μοντέλο, δεν εξαντλεί πλήρως όλες τις διαστάσεις της ασφάλειας. Ιδιότητες όπως authenticity,

accountability, non-repudiation και reliability μπορούν επίσης να είναι σημαντικές ανάλογα με το σύστημα και το περιβάλλον λειτουργίας του. Η αξία του CIA Triad εντοπίζεται κυρίως στην παροχή ενός απλού και συνάμα σταθερού τρόπου οργάνωσης των βασικών στόχων προστασίας, πάνω στον οποίο μπορούν να οικοδομηθούν πιο εξειδικευμένες απαιτήσεις και μηχανισμοί ασφάλειας.

Στο πλαίσιο της Ασφάλειας Λογισμικού, οι τρεις αυτές αρχές είναι ιδιαίτερα σημαντικές επειδή οι αδυναμίες του πηγαίου κώδικα μπορούν να οδηγήσουν σε διαφορετικά είδη επιπτώσεων. Μία αδυναμία μπορεί να επιτρέψει την αποκάλυψη ευαίσθητων και προστατευμένων πληροφοριών και δεδομένων, μία άλλη μπορεί να προκαλέσει την ανεπιθύμητη μεταβολή τους και μία Τρίτη τη διακοπή της λειτουργίας μιας υπηρεσίας. Συνεπώς, η διάκριση μεταξύ Εμπιστευτικότητας, Ακεραιότητας και Διαθεσιμότητας περιέχει ένα χρήσιμο θεωρητικό πλαίσιο για την κατανόηση του είδους της επίπτωσης που μπορεί να προκαλέσει μία αδυναμία ή ευπάθεια, χωρίς όμως να αποτελεί από μόνη της μέθοδο εκτίμησης και υπολογισμού της συνολικής σοβαρότητας ή του κινδύνου της.

4.1.2 Αυθεντικοποίηση (Authentication) και Εξουσιοδότηση (Authorization)

Δύο θεμελιώδεις αλλά διακριτές και όχι πλήρως ανεξάρτητες διαδικασίες ελέγχου της πρόσβασης σε ένα σύστημα λογισμικού αποτελούν οι διαδικασίες της Αυθεντικοποίησης (Authentication) και της Εξουσιοδότησης (Authorization). Η πρώτη εξ' αυτών σχετίζεται με την επαλήθευση της ταυτότητας μιας οντότητας που επιχειρεί να αλληλοεπιδράσει με το σύστημα, ενώ η δεύτερη καθορίζει ποιες ενέργειες και ποιοι πόροι επιτρέπεται να είναι διαθέσιμη προς χρήση από την συγκεκριμένη οντότητα. Με απλούστερους όρους, η αυθεντικοποίηση κυρίως απαντά στο ερώτημα «ποιος είσαι;», ενώ η εξουσιοδότηση στο «τι επιτρέπεται να κάνεις;». Η διάκριση αυτή κρίνεται θεμελιώδης, καθώς η επιτυχής επιβεβαίωση μιας ταυτότητας δεν συνεπάγεται την καθολική και απαραίτητη απόκτηση απεριόριστης πρόσβασης στους πόρους του συστήματος από τη συγκεκριμένη οντότητα.

Η authentication παρέχει διαβεβαίωση ότι ο claimant ελέγχει authenticator που συνδέεται με συγκεκριμένη ψηφιακή ταυτότητα. Η authorization είναι μεταγενέστερη και διακριτή απόφαση για το ποιες ενέργειες επιτρέπονται στην ήδη αυθεντικοποιημένη οντότητα (NIST, 2025). Οι authenticator μπορούν να βασίζονται σε διαφορετικούς παράγοντες. Κάτι που γνωρίζει η οντότητα, όπως για παράδειγμα ένας κωδικός ή PIN, κάτι που κατέχει, όπως μία κρυπτογραφική συσκευή ή token, και κάτι που αποτελεί χαρακτηριστικό της, όπως ένα βιομετρικό γνώρισμα (δαχτυλικό αποτύπωμα, αναγνώριση αναλογιών προσώπου, κ.α.).

Η αυθεντικοποίηση πλέον έχει επεκταθεί στην χρήση περισσότερων του ενός παραγόντων με αποτέλεσμα την δημιουργία της διαδικασίας Multi-Factor Authentication (MFA). Σκοπός της είναι να περιορίσει την εξάρτηση της διαδικασίας από αποκλειστικά έναν μηχανισμό απόδειξης ταυτότητας. Σε περίπτωση όπου, για παράδειγμα, ένας κωδικός πρόσβασης διαρρεύσει ή παραβιαστεί, η ύπαρξη ενός

ανεξάρτητου, από τον κωδικό, δεύτερου παράγοντα μπορεί να εμποδίσει την επιτυχή αυθεντικοποίηση κάποιου επιτιθέμενου. Η διαδικασία της MFA δεν εξαλείφει πλήρως όλους τους κινδύνους που σχετίζονται με την ψηφιακή ταυτότητα, αλλά μπορεί να αυξήσει σημαντικά την ανθεκτικότητα της διαδικασίας απέναντι σε περιπτώσεις όπου ένας μεμονωμένος authenticator έχει αποκαλυφθεί ή κλαπεί, συνήθως κακόβουλα. Το σύγχρονο πλαίσιο αναφοράς για τα authentication processes και τα αντίστοιχα assurance requirements αποτελεί το πρότυπο NIST SP 800-63-4.

Αντίθετα με την διαδικασία της Authentication, η Εξουσιοδότηση (Authorization) πραγματοποιεί τον έλεγχο και λαμβάνει την απόφαση σχετικά με τα δικαιώματα προσπέλασης πόρων ή εκτέλεσης συγκεκριμένων ενεργειών από μία οντότητα η οποία έχει ή όχι συνδεθεί στο λογισμικό σύστημα. Η απόφαση μπορεί να βασίζεται σε δικαιώματα, ρόλους, ιδιότητες της οντότητας ή του πόρου, σχέσεις μεταξύ οντοτήτων, καθώς επίσης και σε συνθήκες περιβάλλοντος. Δύο χρήστες, για παράδειγμα, μπορούν να έχουν αυθεντικοποιηθεί επιτυχώς στο ίδιο σύστημα αλλά να διαθέτουν διαφορετικούς ρόλους και δικαιώματα με αποτέλεσμα να έχουν διαφορετικές με την σειρά τους δυνατότητες. Ένας απλός χρήστης μπορεί να επιτρέπεται να διαβάσει αποκλειστικά τα δικά του δεδομένα, ενώ ένας διαχειριστής να έχει πρόσβαση σε πρόσθετες λειτουργίες διαχείρισης. Το NIST έχει αντιμετωπίσει την διαδικασία της εξουσιοδότησης ως απόφαση αποδοχής ή απόρριψης πρόσβασης ενός subject σε συγκεκριμένα system objects, ενώ σε μοντέλα όπως είναι το Attribute-Based Access Control η απόφαση μπορεί να λαμβάνει υπόψη attributes του subject, του object, της ζητούμενης λειτουργίας και του περιβάλλοντος.

Η εξουσιοδότηση συνδέεται στενά με την αρχή του Least Privilege, κατά την οποία κάθε χρήστης, διεργασία ή άλλο συστατικό στοιχείο του συστήματος πρέπει να διαθέτει μόνο τα δικαιώματα που είναι αναγκαία για την εκτέλεση της προβλεπόμενης λειτουργίας του. Η περιορισμένη παροχή δικαιωμάτων μειώνει δραματικά τις πιθανές συνέπειες ενός παραβιασμένου λογαριασμού, ενός σφάλματος ή μιας κακόβουλης ενέργειας να επέμβει στο σύστημα και να καταστρατηγήσει δεδομένα και διαδικασίες του, συνεπώς την ορθή του λειτουργία. Μία ασφαλής σχεδίαση authorization επίσης εφαρμόζει συχνά τη λογική deny by default, σύμφωνα με την οποία η πρόσβαση απορρίπτεται όταν δεν υπάρχει σαφής κανόνας που να την επιτρέπει, και επανεξετάζει την εξουσιοδότηση στις σχετικές αιτήσεις αντί να θεωρεί ότι μία προηγούμενη επιτυχής απόφαση παραμένει πάντοτε έγκυρη.

Αν και στην τυπική ροή ενός προστατευμένου συστήματος προηγείται η υλοποίηση και εκτέλεση της διαδικασίας Αυθεντικοποίησης και στην συνέχεια ακολουθεί η Εξουσιοδότηση, οι έννοιες δεν πρέπει να θεωρούνται απολύτως εξαρτημένες. Ένα σύστημα μπορεί, για παράδειγμα, να επιτρέπει την πρόσβαση σε δημόσιο περιεχόμενο χωρίς προηγούμενη αυθεντικοποίηση. Αντιθέτως, ένας πλήρως αυθεντικοποιημένος χρήστης εξακολουθεί να πρέπει να υπόκειται σε κανόνες authorization για κάθε προστατευμένη λειτουργία του συστήματος που αιτείται να εκτελέσει. Η διάκριση αυτή έχει επίσης τονιστεί έντονα από την OWASP, όπου αναφέρεται ότι η γνώση της ταυτότητας μιας οντότητας δεν συνεπάγεται

αυτόματα δικαίωμα πρόσβασης σε κάθε διαθέσιμο πόρο του συστήματος (υλικό ή άυλο πόρο).

Οι δύο αυτές διαδικασίες, επομένως, αν και ανεξάρτητες, λειτουργούν συμπληρωματικά η μία απέναντι στην άλλη. Η Authentication παρέχει τη βάση πάνω στην οποία ένα σύστημα μπορεί να συσχετίσει μία αίτηση με μία συγκεκριμένη ταυτότητα, ενώ η Authorization χρησιμοποιεί αυτή και άλλα διαθέσιμα στοιχεία για να αποφασίσει ποια πρόσβαση πρέπει να επιτραπεί. Αδυναμίες σε οποιοδήποτε από τα δύο αυτά επίπεδα, αναμένεται να οδηγήσουν σε σοβαρές επιπτώσεις και παραβιάσεις ασφαλείας. Ένας ανεπαρκής μηχανισμός αυθεντικοποίησης πιθανολογείται να επιτρέψει σε έναν επιτιθέμενο να παρουσιαστεί ως ένας νόμιμος χρήστης, ενώ εσφαλμένη υλοποίηση του Authorization μπορεί να επιτρέψει σε έναν νόμιμο αλλά μη επαρκώς προνομιούχο χρήστη να αποκτήσει πρόσβαση σε δεδομένα ή λειτουργίες τις οποίες δεν δικαιούται. Εν κατακλείδι, η σωστή εφαρμογή και των δύο εξ αυτών, αποτελεί βασική προϋπόθεση για αποτελεσματικό έλεγχο πρόσβασης σε ένα σύστημα λογισμικού.

4.1.3 Threat, Vulnerability, Weakness και Exploit

Οι έννοιες Threat, Vulnerability, Weakness και Exploit χρησιμοποιούνται για την περιγραφή διαφορετικών στοιχείων μέσω των οποίων είναι δυνατόν να προκύψει μία παραβίαση ασφαλείας σε κάποιο σύστημα λογισμικού. Αν και οι προαναφερθέντες όροι συνδέονται στενά, δεν είναι ταυτόσημοι. Μία αδυναμία μπορεί να υπάρξει στον σχεδιασμό ή στην υλοποίηση ενός συστήματος λογισμικού, υπό συγκεκριμένες συνθήκες να καταστήσει το προϊόν ευάλωτο (ή μη αξιόπιστο), και στην συνέχεια να αξιοποιηθεί μέσω μιας κατάλληλης τεχνικής ή ενέργειας. Παράλληλα, μία απειλή εκφράζει την πιθανότητα ύπαρξης ενός γεγονότος ή μιας περίπτωσης που μπορεί να προκαλέσει αρνητικές συνέπειες. Η σαφής διάκριση των εννοιών καθίσταται απαραίτητη για την κατανόηση του τρόπου με τον οποίο δύναται να δημιουργηθούν και να αξιοποιηθούν προβλήματα ασφαλείας.

Ως threat νοείται μία περίπτωση ή πηγή που έχει τη δυνατότητα να προκαλέσει δυσμενή συνέπεια αξιοποιώντας ή ενεργοποιώντας μία αδυναμία. Δεν ταυτίζεται κατ' ανάγκη με κακόβουλο πρόσωπο ούτε σημαίνει ότι έχει ήδη εκδηλωθεί περιστατικό (NIST, 2012). Μία απειλή μπορεί να πηγάζει από κακόβουλη ανθρώπινη δραστηριότητα, ανθρώπινο λάθος, τεχνική αστοχία ή άλλη κατάσταση ικανή να οδηγήσει σε ανεπιθύμητη λειτουργία. Το NIST διακρίνει επίσης το threat source, δηλαδή την πηγή ή την κατάσταση που μπορεί σκόπιμα να επιχειρήσει την εκμετάλλευση μιας ευπάθειας ή να την ενεργοποιήσει κατά λάθος.

Η Weakness (Αδυναμία) αφορά μία προβληματική συνθήκη στον σχεδιασμό, στην αρχιτεκτονική, στην υλοποίηση ή σε άλλο στοιχείο ενός προϊόντος, η οποία υπό κατάλληλες συνθήκες μπορεί να συμβάλει δραστικά στη δημιουργία μιας ευπάθειας (Vulnerability). Στο πλαίσιο του Common Weakness Enumeration (CWE) που διεξάγεται από την MITRE, έχει τονιστεί ότι μία αδυναμία συνήθως περιγράφει έναν γενικό τύπο προβλήματος χωρίς απαραίτητα να προσδιορίζει ένα συγκεκριμένο εκμεταλλεύσιμο περιστατικό σε κάποιο συγκεκριμένο προϊόν. Για παράδειγμα, μία

κατηγορία ανεπαρκούς ελέγχου εισόδου μπορεί να αποτελεί weakness που εμφανίζεται σε πολλά διαφορετικά συστήματα και υλοποιήσεις. Η ύπαρξή της αποτελεί ένδειξη ότι υπάρχει μία προβληματική τεχνική κατάσταση, χωρίς όμως να σημαίνει ότι μπορεί να αξιοποιηθεί επιτυχώς σε κάθε περιβάλλον με πιθανότητες εμφάνισης.

Μία weakness είναι γενικός τύπος προβληματικής συνθήκης, ενώ μία vulnerability είναι συγκεκριμένη εκμεταλλεύσιμη εμφάνισή της σε προϊόν ή σύστημα. Η διάκριση αυτή αποτρέπει την αυτόματη εξίσωση κάθε προβληματικής κατασκευής με επιβεβαιωμένη ευπάθεια (MITRE, χ.χ.; NIST, 2012). Η εκμετάλλευση μιας vulnerability μπορεί να επηρεάσει την εμπιστευτικότητα, την ακεραιότητα ή τη διαθεσιμότητα του συστήματος, δηλαδή μία ή περισσότερες από τις θεμελιώδεις αρχές του CIA Triad.

Ο όρος Exploit περιγράφει τον μηχανισμό, την είσοδο, την τεχνική ή την ενέργεια μέσω της οποίας αξιοποιείται μία αδυναμία ή ευπάθεια ώστε να προκληθεί ένα ανεπιθύμητο τεχνικό αποτέλεσμα. Ένα exploit δεν είναι απαραίτητα πάντοτε αυτόνομο πρόγραμμα ή ένα κακόβουλο αρχείο ή μέσο. Μπορεί να είναι ειδικά διαμορφωμένη είσοδος, αλληλουχία ενεργειών, τεχνική επίθεσης ή κώδικας σχεδιασμένος να εκμεταλλευτεί μία ευάλωτη κατάσταση. Το CAPEC ορίζει αυτή την έννοια με χρήση αυτής της ευρύτερης λογικής και αντιμετωπίζοντας το exploit ως input ή action που έχει σχεδιαστεί με στόχο να αξιοποιήσει μία ή περισσότερες αδυναμίες και να προκαλέσει αρνητική τεχνική επίπτωση. Η ύπαρξη λειτουργικού exploit αυξάνει, όπως είναι λογικό, την πρακτική δυνατότητα αξιοποίησης ενός vulnerability, χωρίς αυτό να σημαίνει ότι κάθε γνωστή ευπάθεια διαθέτει πάντοτε δημόσια διαθέσιμο exploit.

Η σχέση μεταξύ των παραπάνω περιγραφόμενων εννοιών μπορεί να αποδοθεί επίσης σχηματικά:



Εικόνα 4.1.1 – Σχέση μεταξύ Weakness, SEC, Vulnerability, Exploit και επιπτώσεων στο σύστημα *

ενώ το Threat περιγράφει την πηγή, περίσταση ή πιθανό γεγονός που μπορεί να οδηγήσει στην αξιοποίηση μίας ευπάθειας.

Η συγκεκριμένη ακολουθία είναι χρήσιμη για την πλήρη κατανόηση των εννοιών, αλλά πρέπει να αποφεύγεται η αντιμετώπισή της ως απόλυτος μηχανισμός. Δεν μετατρέπεται κάθε αδυναμία σε ευπάθεια, δεν αξιοποιείται κάθε ευπάθεια πρακτικά και δεν απαιτείται κάθε exploit να έχει τη μορφή εκτελέσιμου κώδικα. Μία

απειλή μπορεί επίσης να ενεργοποιήσει ένα vulnerability σκόπιμα ή ακόμα και τυχαία σε ορισμένες περιπτώσεις.

Η διάκριση μεταξύ Weakness και Vulnerability έχει ιδιαίτερη σημασία και στην αυτοματοποιημένη ανάλυση του πηγαίου κώδικα. Ένα εργαλείο μπορεί να αναγνωρίσει ένα pattern ή μία κατάσταση που αντιστοιχεί σε γνωστή κατηγορία αδυναμιών ασφάλειας, χωρίς αυτομάτως να αποδεικνύεται ότι υπάρχει μία πλήρως εκμεταλλεύσιμη ευπάθεια στο συγκεκριμένο deployment του λογισμικού. Για να χαρακτηριστεί μία κατάσταση ως πραγματικά εκμεταλλεύσιμη απαιτείται συχνά πρόσθετος ορισμός context και περιεχόμενο σχετικά με το περιβάλλον, τις εισόδους, τα δικαιώματα, τη διαμόρφωση και τη δυνατότητα ενός επιτιθέμενου να καταφέρει να φτάσει στο ευάλωτο σημείο. Τα ευρήματα της στατικής ανάλυσης επομένως, πρέπει να αντιμετωπίζονται ως ενδείξεις πιθανών προβλημάτων ασφαλείας που απαιτούν κατάλληλη αξιολόγηση και όχι ως απόλυτη απόδειξη επιτυχούς exploitation.

Η συγκεκριμένη διάκριση κρίνεται σημαντική και στο πλαίσιο της παρούσας εργασίας, αφού οι μετρικές ασφάλειας που παράγονται από αυτοματοποιημένη ανάλυση κυρίως σχετίζονται με εντοπισμένα security issues και πιθανές αδυναμίες ή/και ευπάθειες στον πηγαίο κώδικα, και όχι με πραγματικά παρατηρημένες επιθέσεις ή επιτυχημένα exploits στο εκτελέσιμο λογισμικό. Η αύξηση ή η μείωση τέτοιων ευρημάτων ερμηνεύεται ως μεταβολή των στατικά παρατηρούμενων ενδείξεων ασφάλειας και όχι ως άμεση μέτρηση της πραγματικής εκμετάλλευσης ενός συστήματος.

4.1.4 Ρίσκο (Risk) και Επιφάνεια Επίθεσης (Attack Surface)

Το Ρίσκο (Risk) αποτελεί κεντρική έννοια του τομέα της Ασφάλειας Λογισμικού, καθώς επιτρέπει την αξιολόγηση της σημασίας που έχει μία πιθανή απειλή για ένα συγκεκριμένο σύστημα. Η ύπαρξη μιας weakness ή vulnerability δεν συνεπάγεται αυτομάτως ότι το σύστημα αντιμετωπίζει τον ίδιο ακριβώς βαθμό κινδύνου σε κάθε πιθανή περίπτωση χρήσης ή επίθεσης. Σύμφωνα με επίσημη εξακρίβωση του NIST, το ρίσκο εκφράζει τον βαθμό στον οποίο μία οντότητα απειλείται από ένα πιθανό γεγονός ή περίσταση και συνδέεται κυρίως με δύο διαστάσεις. Η πρώτη εξ' αυτών είναι η πιθανότητα (likelihood) να συμβεί το γεγονός, ενώ η δεύτερη είναι η σοβαρότητα των επιθέσεων (impact severity) που θα προκύψουν εάν αυτό πραγματοποιηθεί. Μία ευπάθεια, επομένως, με πολύ σοβαρές δυνητικές συνέπειες μπορεί να παρουσιάζει διαφορετικό επίπεδο ρίσκου ανάλογα με το κατά πόσο είναι πρακτικά δυνατό να αξιοποιηθεί στο συγκεκριμένο περιβάλλον.

Η εκτίμηση κινδύνου συνδυάζει πληροφορίες για threat sources και events, vulnerabilities, πιθανότητα και δυνητικές επιπτώσεις, λαμβάνοντας παράλληλα υπόψη τα ήδη εφαρμοσμένα safeguards. Επομένως, δεν εξαντλείται μηχανιστικά σε ένα γινόμενο πιθανότητας και επίπτωσης (NIST, 2012). Στο NIST SP 800-30 Revision 1, η διαδικασία risk assessment οργανώνεται γύρω από τις πηγές απειλής και τα αντίστοιχα threat events, τις vulnerabilities και τις προϋπάρχουσες συνθήκες, την πιθανότητα εμφάνισης ή επιτυχούς εκμετάλλευσης και την έκταση των πιθανών

αρνητικών επιπτώσεων. Συνεπώς, η πιθανότητα επηρεάζεται όχι μόνο από την ύπαρξη μιας ευπάθειας, αλλά και από τις δυνατότητες της threat source, την προσβασιμότητα του ευάλωτου στοιχείου και τα safeguards ή countermeasures που έχουν ήδη εφαρμοστεί.

Η έννοια του risk είναι σημαντική επειδή επιτρέπει την ιεράρχηση των προβλημάτων ασφάλειας. Δεν είναι πρακτικά εφικτό να αντιμετωπίζονται όλες οι αδυναμίες με την ίδια προτεραιότητα. Σε μεγάλα συστήματα ιδιαίτερα μπορεί να υπάρχουν μερικές εκατοντάδες ή χιλιάδες security findings τα οποία είναι αδύνατο να αντιμετωπιστούν με όμοια προτεραιότητα και σοβαρότητα. Η αξιολόγηση του συνδυασμού πιθανότητας επίπτωσης επιτρέπει στις ομάδες να επικεντρώνονται πρώτα στα προβλήματα που μπορούν να προκαλέσουν τη μεγαλύτερη πραγματική ζημιά. Το πρότυπο ISO/IEC 27005:2022 τοποθετεί την αναγνώριση, ανάλυση, αξιολόγηση και αντιμετώπιση των information security risks μέσα σε μία συνεχή και επαναληπτική διαδικασία διαχείρισης, η οποία περιλαμβάνει επίσης και την επικοινωνία, παρακολούθηση και επανεξέταση των εντοπισμένων ή μη κινδύνων.

Διαφοροποιημένη αλλά στενά συνδεδεμένη με τον ορισμό του Ρίσκου είναι η έννοια της Επιφάνειας Επίθεσης (Attack Surface). Η έννοια αυτή ορίζεται από το NIST ως ένα σύνολο των σημείων στα όρια του συστήματος, component ή περιβάλλοντος στα οποία ένας επιτιθέμενος είναι πιθανό να επιχειρήσει να εισέλθει, να προκαλέσει κάποια επίδραση ή να εξαγάγει δεδομένα. Η Επιφάνεια Επίθεσης περιλαμβάνει τα σημεία εκείνα στα οποία το σύστημα αλληλοεπιδρά με το εξωτερικό του περιβάλλον και μέσω των οποίων μπορεί να δημιουργηθούν ευκαιρίες επίθεσης. Τέτοια σημεία μπορεί να περιλαμβάνουν δικτυακές υπηρεσίες, APIs (Application Programming Interface), ανοιχτές θύρες, interfaces εισόδου, μηχανισμούς επικοινωνίας, αρχεία ή διάφορες άλλες πηγές δεδομένων οι οποίες μπορούν να επηρεαστούν από εξωτερικές οντότητες.

Οι Manadhata και Wing (2011) έχουν αντιμετωπίσει την συγκεκριμένη έννοια μέσω των πόρων ενός συστήματος που μπορούν να χρησιμοποιηθούν κατά την πραγματοποίηση μίας επίθεσης, παρέχοντας έτσι μία περισσότερο συστηματική θεμελίωσή της. Ιδιαίτερη σημασία λαμβάνουν οι μέθοδοι, τα κανάλια επικοινωνίας και τα data items μέσω των οποίων εισέρχονται ή εξέρχονται από το σύστημα. Δεν συνεισφέρουν αντίστοιχα και στον ίδιο βαθμό στην επιφάνεια επίθεσης. Παράγοντες όπως δικαιώματα ενός component και η πιθανή ζημιά που μπορεί να προκύψει από την κατάχρησή του επηρεάζουν τη σχετική τους σημασία. Με τον τρόπο αυτό, η Attack Surface δεν αντιμετωπίζεται απλώς ως καταμέτρηση των διαθέσιμων interfaces, αλλά ως χαρακτηριστικό που σχετίζεται τόσο με τις διαθέσιμες δυνατότητες αλληλεπίδρασης όσο και με τη δυνητική αξιοποίησή τους από κάποιον τρίτο κακόβουλο.

Η έννοια της μείωσης της επιφάνειας επίθεσης (Attack Surface Reduction) βασίζεται στην αρχή της συνεπαγωγής των λιγότερων εκτιθέμενων μη αναγκαίων δυνατοτήτων αλληλεπίδρασης σε λιγότερες ευκαιρίες για πιθανό exploitation. Η απενεργοποίηση μη απαραίτητων υπηρεσιών, ο περιορισμός εκτεθειμένων interfaces και η μείωση των privileges μπορούν να περιορίσουν τις διαθέσιμες διαδρομές μέσω των οποίων ένας attacker μπορεί να αλληλοεπιδράσει με το σύστημα.

Οι έννοιες της Attack Surface και ο ποσοτικός αριθμός των vulnerabilities ή το ίδιο το ρίσκο δεν θα πρέπει να ταυτίζονται. Ένα σύστημα με μεγάλη επιφάνεια επίθεσης δεν διαθέτει αναγκαστικά περισσότερες vulnerabilities, όπως αντίστοιχα και ένα σύστημα με λίγες, σχετικά, vulnerabilities δεν διαθέτει απαραίτητα μικρή Attack Surface. Οι δύο αυτές έννοιες συμπεραίνουμε ότι είναι σχετικές αλλά διακριτές χωρίς να αλληλεξαρτώνται καθολικά. Η Attack Surface περιγράφει κυρίως την έκθεση και τις διαθέσιμες δυνατότητες αλληλεπίδρασης ενός συστήματος με δυνητικούς attackers, ενώ το Risk αξιολογεί τη σημασία που κατέχουν συγκεκριμένα ανεπιθύμητα γεγονότα για το σύστημα. Η μείωση του διαθέσιμου Attack Surface δύναται να αποτελέσει έναν τρόπο περιορισμού του security risk, δεν εξαλείφει όμως πλήρως την ανάγκη εντοπισμού και διόρθωσης vulnerabilities, εφαρμογής κατάλληλων security controls και συνεχούς αξιολόγησης των απειλών.

Στο πλαίσιο που η παρούσα εργασία ορίζει, η διάκριση αυτή είναι σημαντική. Τα ευρήματα της στατικής ανάλυσης παρέχουν πληροφορίες κυρίως για πιθανές αδυναμίες ή προβλήματα ασφαλείας στον πηγαίο κώδικα και όχι πλήρη μέτρηση του πραγματικού risk ενός συστήματος. Για την εκτίμηση του τελευταίου θα απαιτούνταν επιπλέον πληροφορίες σχετικά με την πραγματική δυνατότητα exploitation, το περιβάλλον του deployment, την Attack Surface, τις πιθανές επιπτώσεις και τις threat sources. Οι μετρικές ασφάλειας της εμπειρικής ανάλυσης πρέπει να ερμηνεύονται, συνεπώς, ως ενδείξεις συγκεκριμένων τεχνικών χαρακτηριστικών ασφάλειας και όχι ως άμεσες αριθμητικές μετρήσεις του συνολικού cybersecurity risk.

4.2 Αδυναμίες ασφαλείας στον πηγαίο κώδικα

Οι αδυναμίες ασφαλείας στον πηγαίο κώδικα (source code security weaknesses) αποτελούν προβληματικές καταστάσεις ή πρακτικές στην υλοποίηση ενός συστήματος, που υπό συγκεκριμένες συνθήκες μπορούν να συμβάλλουν στη δημιουργία εκμεταλλεύσιμων ευπαθειών. Όπως αναφέρθηκε και προηγουμένως, μία weakness δεν ταυτίζεται απαραίτητα με μία ευπάθεια. Η διάκριση μεταξύ τους, όπως περιεγράφηκε προηγουμένως είναι ιδιαίτερα σημαντική στην ανάλυση πηγαίου κώδικα, καθώς ο εντοπισμός μιας προβληματικής κατασκευής δεν αποδεικνύει από μόνος του ότι υπάρχει άμεσα αξιοποιήσιμη ευπάθεια.

Οι αδυναμίες αυτές μπορούν να εισέλθουν σε διαφορετικά στάδια της ανάπτυξης και να έχουν διαφορετικές αιτίες. Ορισμένες προκύπτουν από λανθασμένες σχεδιαστικές αποφάσεις που τελικά αποτυπώνονται στην υλοποίηση, ενώ άλλες μπορεί να εισαχθούν άμεσα κατά τη συγγραφή ή τροποποίηση του κώδικα. Ελλιπής έλεγχος εισόδων, λανθασμένοι μηχανισμοί ελέγχου πρόσβασης, μη ασφαλής διαχείριση και διακίνηση δεδομένων, ακατάλληλη χρήση κρυπτογραφικών μηχανισμών, ανεπαρκής διαχείριση σφαλμάτων, χρήση μη ασφαλών APIs ή αποκάλυψη ευαίσθητων πληροφοριών μέσα στον κώδικα αποτελούν ενδεικτικές κατηγορίες προβληματικών καταστάσεων. Το secure coding πρέπει να καλύπτει συστηματικά ζητήματα όπως είναι τα input validation, output encoding, access

control, cryptographic practices, error handling, logging και προστασία ευαίσθητων δεδομένων.

Ιδιαίτερη, επίσης, σημασία έχει το γεγονός όπου μία αδυναμία βρίσκεται λανθάνουσα στον πηγαίο κώδικα χωρίς να προκαλεί κάποιο εμφανές λειτουργικό πρόβλημα κατά τη συνηθισμένη χρήση του συστήματος. Ένα λογισμικό παράγει αναμενόμενα αποτελέσματα για τους νόμιμους χρήστες. Ταυτόχρονα όμως, μπορεί να περιλαμβάνει μία συνθήκη η οποία επιτρέπει σε έναν επιτιθέμενο να προκαλέσει διαφορετική, μη προβλεπόμενη συμπεριφορά. Αυτό διαφοροποιεί σε σημαντικό βαθμό πολλές αδυναμίες ασφαλείας από τα συμβατικές λειτουργικές απειλές/αδυναμίες/σφάλματα. Η ύπαρξη μιας αδυναμίας μπορεί να γίνει εμφανείς μόνο σε περιπτώσεις όπου το σύστημα δεχθεί μη αναμενόμενες ή ειδικά κατασκευασμένες εισόδους ή όταν ένας χρήστης επιχειρήσει να παρακάμψει τις προβλεπόμενες ροές λειτουργίας.

Επιπλέον, οι ορισμένες ως αδυναμίες ασφαλείας δεν περιορίζονται σε απλά συντακτικά λάθη ή σε μεμονωμένες γραμμές πηγαίου κώδικα. Ενδέχεται να προκύπτουν από τη ροή δεδομένων, τη ροή ελέγχου, και τη συνεργασία πολλαπλών components μεταξύ τους. Λαμβάνοντας ως παράδειγμα ένα δεδομένα το οποίο προέρχεται από μη έμπιστη πηγή, μπορεί να θεωρείται ασφαλές σε ένα σημείο του συστήματος και αργότερα να χρησιμοποιείται σε μία security-sensitive λειτουργία χωρίς επαρκή επαλήθευση. Ένας έλεγχος εξουσιοδότησης αντίστοιχα, μπορεί να υπάρχει υλοποιημένος στον κώδικα του λογισμικού αλλά να εφαρμόζεται μόνο σε ορισμένες από τις διαθέσιμες διαδρομές πρόσβασης. Η ασφαλής εξέταση του πηγαίου κώδικα, έχει τονιστεί από διάφορους οργανισμούς ότι συχνά κατανόηση της συνολικής λογικής, της ροής ελέγχου και του data flow και όχι απλή αναζήτηση συγκεκριμένων προβληματικών patterns.

Τέτοιες αδυναμίες οργανώνονται από την MITRE μέσω του Common Weakness Enumeration (CWE), το οποίο λειτουργεί ως κοινό λεξιλόγιο για την περιγραφή επαναλαμβανόμενων τύπων software και hardware αδυναμιών. Η σχετική οριζόμενη ταξινόμηση περιλαμβάνει προβλήματα που σχετίζονται και με μη ασφαλή κώδικα, πρακτικές, διαχείριση πόρων, APIs, ενθυλάκωση, χειρισμό δεδομένων και security decisions. Είναι σημαντικό ότι ορισμένες κακές προγραμματιστικές πρακτικές δεν δημιουργούν από μόνες τους μία άμεσα εκμεταλλεύσιμη ευπάθεια, αλλά μπορούν να αυξήσουν την πιθανότητα ύπαρξης ή εισαγωγής αδυναμιών στο σύστημα. Το CWE επισημαίνει χαρακτηριστικά ότι κώδικας που είναι υπερβολικά σύνθετος, δύσκολος στη συντήρηση ή παρουσιάζει ενδείξεις ανεπαρκούς φροντίδας μπορεί να αυξήσει την πιθανότητα ύπαρξης κρυμμένων αδυναμιών.

Η συγκεκριμένη αυτή σχέση παρουσιάζει ιδιαίτερο ενδιαφέρον για τη σύνδεση ποιότητας και ασφάλειας λογισμικού. Χαρακτηριστικά όπως αυξημένη πολυπλοκότητα, δυσκολία συντήρησης ή ακατάλληλες πρακτικές κωδικοποίησης δεν αποτελούν χωρίς αξιολόγηση ευπάθειες, μπορούν όμως να δημιουργήσουν ένα περιβάλλον μέσα στο οποίο είναι δυσκολότερο να εντοπιστούν, να κατανοηθούν ή να διορθωθούν αδυναμίες ασφάλειας. Η διάκριση μεταξύ προβλημάτων ποιότητας και αδυναμιών ασφάλειας πρέπει να παραμένει σαφής, χωρίς αυτό να σημαίνει ότι οι δύο κατηγορίες είναι πλήρως ανεξάρτητες μεταξύ τους. Η ταξινόμηση CWE περιλαμβάνει

κατηγορίες κακών coding practices που αυξάνουν την πιθανότητα ύπαρξης εκμεταλλεύσιμων αδυναμιών, χωρίς να χαρακτηρίζει κάθε τέτοια πρακτική ως ευπάθεια.

Από τα παραπάνω, εξάγεται συμπεραίνεται ότι η αναγνώριση των source-code weakness όσο το δυνατόν νωρίτερα στον κύκλο ανάπτυξης είναι ιδιαίτερα σημαντική. Το πρότυπο του NIST Secure Software Development Framework (SSDF) υποστηρίζει την ενσωμάτωση secure development practices σε ολόκληρο τον Κύκλο Ζωής της Ανάπτυξης Λογισμικού, με στόχο τόσο τη μείωση του αριθμού των συνολικών ευπαθειών που φτάνουν στις τελικές εκδόσεις, σε εκδόσεις production, όσο και την αντιμετώπιση των root causes που οδηγούν επανειλημμένα στη δημιουργία τους. Το NISTIR 8397 προτείνει συνδυασμό διαφορετικών τεχνικών developer verification, όπως threat modeling, static code scanning, έλεγχο για hard-coded secrets, automated testing, fuzzing και άλλες μορφές επιβεβαίωσης (verification). Η ύπαρξη πολλαπλών τεχνικών, αν και αυξάνει την πολυπλοκότητα, είναι σημαντική επειδή καμία μεμονωμένη μέθοδος δεν μπορεί να εντοπίσει αποτελεσματικά και συνολικά όλες τις κατηγορίες αδυναμιών ασφάλειας.

Ιδιαίτερη αξία επίσης παρουσιάζει η ανάλυση του πηγαίου κώδικα για τον εντοπισμό τέτοιων προβλημάτων, καθώς επιτρέπει την εξέταση της πραγματικής υλοποίησης και της λογικής του συστήματος. Η ανάλυση πηγαίου κώδικα μπορεί να χρησιμοποιηθεί για τον εντοπισμό αδυναμιών και προβλημάτων. Στην κατηγορία αυτή ανήκουν μη προστατευμένες κρίσιμες πληροφορίες, προβλήματα μνήμης, injection-related conditions ή άλλες μη ασφαλείς καταστάσεις. Έχει επισημανθεί ότι ορισμένες κατηγορίες αδυναμιών, μεταξύ αυτών προβλήματα επιχειρησιακής λογικής, ελέγχου πρόσβασης, υλοποίησης κρυπτογραφίας ή ταυτοχρονισμού, μπορούν να απαιτούν ανθρώπινη κατανόηση του πλαισίου και να μην εντοπίζονται αξιόπιστα μόνο μέσω αυτοματοποιημένων scanners. Η αυτοματοποιημένη ανάλυση και η χειροκίνητη αξιολόγηση ασφαλούς κώδικα (manual secure code review) πρέπει να θεωρούνται συμπληρωματικές προσεγγίσεις και να αποφεύγεται ο ορισμός τους ως αλληλοκαλυπτόμενες ή υποκατάστατες.

Παράλληλα, ο εντοπισμός μιας αδυναμίας στον πηγαίο κώδικα δεν παρέχει από μόνος του πλήρη εικόνα του πραγματικού security risk. Για να αξιολογηθεί η πρακτική σημασία ενός ευρήματος απαιτούνται συχνά πληροφορίες σχετικά με την προσβασιμότητα ενός συγκεκριμένου σημείου από έναν επιτιθέμενο, ποιες προϋποθέσεις απαιτούνται για exploitation, ποια δικαιώματα διαθέτει ο επιτιθέμενος, ποια security controls υπάρχουν και ποιες θα ήταν οι πιθανές επιπτώσεις μιας επιτυχούς εκμετάλλευσης. Τα αυτοματοποιημένα source-code findings είναι προτιμότερο να αντιμετωπίζονται, στην βιβλιογραφία αλλά και πρακτικά κατά την ανάπτυξη ενός λογισμικού, ως ενδείξεις τεχνικών προβλημάτων που χρειάζονται αξιολόγηση και να αποφεύγεται η αντιμετώπισή τους ως αυτόματες αποδείξεις πραγματικών exploits ή συγκεκριμένου επιπέδου κινδύνου.

Οι αδυναμίες ασφαλείας στον πηγαίο κώδικα, έχουν επίσης ιδιαίτερη σημασία στην διεκπεραίωση της παρούσας έρευνας, αφού αποτελούν το επίπεδο στο οποίο μπορεί να εξεταστεί ποσοτικά η μεταβολή της ασφάλειας μετά από μία αλλαγή λογισμικού. Ένα Pull Request είναι ικανό ενώ διορθώνει ήδη υπάρχουσες, να εισάγει νέες αδυναμίες, να εξαλείψει ήδη υπάρχουσες ή να μεταβάλλει τεχνικά

χαρακτηριστικά που σχετίζονται με την πιθανότητα εμφάνισής τους. Οι αντίστοιχες μετρικές της αυτοματοποιημένης ανάλυσης αντιμετωπίζονται ως παρατηρήσιμοι δείκτες της τεχνικής κατάστασης ασφάλειας του πηγαίου κώδικα, χωρίς να εξισώνονται με πραγματικές επιθέσεις, exploits ή με το συνολικό ρίσκο (risk) του λειτουργούντος συστήματος.

4.3 CWE, CVE, CVSS

Τα CWE, CVE και CVSS εξυπηρετούν διαφορετικές λειτουργίες: το CWE οργανώνει τύπους αδυναμιών, το CVE καταγράφει δημοσιοποιημένες συγκεκριμένες ευπάθειες και το CVSS περιγράφει με τυποποιημένο τρόπο τη σοβαρότητά τους (MITRE, χ.χ.; CVE Program, χ.χ.; FIRST, χ.χ.).

Η ταξινόμηση Common Weakness Enumeration (CWE) αναφέρεται και ταξινομεί οργανωμένα τις αδυναμίες λογισμικού και υλικού οι οποίες υπό συγκεκριμένες συνθήκες, μπορούν να συμβάλουν στη δημιουργία ευπαθειών. Κάθε ένα CWE identifier περιγράφει συνήθως έναν γενικό τύπο αδυναμίας και όχι μία συγκεκριμένη ευπάθεια ενός συγκεκριμένου προϊόντος. Με τον τρόπο αυτό, το CWE παρέχει ένα κοινό λεξιλόγιο για προγραμματιστές, ερευνητές, ομάδων ασφάλειας και εργαλεία ανάλυσης, επιτρέποντας την ομοιόμορφη περιγραφή επαναλαμβανόμενων προβλημάτων ασφάλειας.

Το CVE Program αποδίδει μοναδικά αναγνωριστικά σε δημοσίως γνωστοποιημένες ευπάθειες, ώστε προϊόντα, οργανισμοί και εργαλεία να μπορούν να αναφέρονται με συνέπεια στο ίδιο περιστατικό. Η λειτουργία αυτή αφορά αναγνώριση και καταλογογράφηση, όχι από μόνη της αξιολόγηση κινδύνου (CVE Program, χ.χ.). Τα CVE Records περιλαμβάνουν βασικές πληροφορίες, όπως περιγραφή της vulnerability, επηρεαζόμενα προϊόντα ή εκδόσεις και σχετικές αναφορές, ενώ η απόδοση και δημοσίευσή τους πραγματοποιείται μέσω του CVE Program και των εξουσιοδοτημένων CVE Numbering Authorities (CNAs).

Η σχέση που αναπτύσσεται ανάμεσα στο CWE και CVE μπορεί να γίνει αντιληπτή ως σχέση μεταξύ κατηγορίας αδυναμίας και συγκεκριμένης εκδήλωσής της. Ένα CWE μπορεί να βρίσκεται πίσω από πολλές διαφορετικές CVE καταχωρίσεις, εφόσον οι αντίστοιχες ευπάθειες προκύπτουν από την ίδια υποκείμενη κατηγορία αδυναμίας. Πρέπει να ξεκαθαριστεί ότι το National Vulnerability Database (NVD) δεν ταυτίζεται με το CVE. Η NVD χρησιμοποιεί τα δημοσιευμένα CVE Records και τα εμπλουτίζει με πρόσθετα δεδομένα, όπως αντιστοιχίσεις σε CWE, πληροφορίες για επηρεαζόμενα προϊόντα και αξιολογήσεις CVSS.

Το CVSS v4.0 είναι πλαίσιο επικοινωνίας της σοβαρότητας μιας ευπάθειας και οργανώνει τις μετρικές σε Base, Threat, Environmental και Supplemental. Η βαθμολογία κυμαίνεται από 0 έως 10, αλλά δεν αποτελεί πλήρη αποτίμηση του επιχειρησιακού risk (FIRST, χ.χ.).

Στο CVSS v4.0 οι βασικές κατηγορίες είναι οι Base, Threat, Environmental και Supplemental Metrics. Οι Base Metrics αποτυπώνουν κυρίως σχετικά σταθερά τεχνικά χαρακτηριστικά, όπως τον τρόπο με τον οποίο μπορεί να πραγματοποιηθεί μία επίθεση, τις απαιτούμενες προϋποθέσεις και privileges, την πιθανή ανάγκη αλληλεπίδρασης χρήστη και τις συνέπειες στο CIA Triad. Οι απειλές και οι μετρικές περιβάλλοντος προσθέτουν πληροφορίες για τη μεταβαλλόμενη δυνατότητα exploitation και το συγκεκριμένο περιβάλλον στο οποίο βρίσκεται το ευάλωτο σύστημα, ενώ οι Supplemental Metrics παρέχουν πρόσθετο πλαίσιο και περιεχόμενο χωρίς να μεταβάλλουν άμεσα τη βασική βαθμολογία.

Απαραίτητο κρίνεται να σημειωθεί ότι το CVSS δεν αποτελεί άμεσο μέτρο του συνολικού Risk. Περιγράφει κυρίως την τεχνική σοβαρότητα μιας ευπάθειας και μπορεί να αποτελέσει μία από τις πληροφορίες που χρησιμοποιούνται κατά τη διαδικασία ενός risk assessment. Το πραγματικό Risk εξαρτάται επιπλέον από παράγοντες όπως η έκθεση του συστήματος, η επιχειρησιακή σημασία του επηρεαζόμενου asset, η πραγματική δυνατότητα exploitation και τα υπάρχοντα security controls. Συμπεραίνουμε, ότι δύο vulnerabilities με παρόμοιες βαθμολογίες CVSS μπορούν να παρουσιάζουν πολύ διαφορετική σημασία για διαφορετικά συστήματα ή οργανισμούς.



Εικόνα 4.3.1 – Σχέση μεταξύ των εννοιών CWE, CVE, CVSS και Risk *

Οι τέσσερις (4) αυτές έννοιες δεν θα πρέπει να συγχέονται. Ένα security finding μπορεί να αντιστοιχεί σε μία κατηγορία CWE χωρίς να αποτελεί αυτόματα και απαραίτητα καταγεγραμμένο CVE, ενώ η ύπαρξη ενός CVE δεν καθορίζει από μόνη της ούτε τη συνολική επιχειρησιακή σημασία του προβλήματος ούτε το συνολικό πραγματικό Risk ενός συγκεκριμένου συστήματος. Στο πλαίσιο της παρούσας εργασίας, η διάκριση αυτή είναι σημαντική προκειμένου η ερμηνεία των automated

security findings να πραγματοποιείται σύμφωνα με το επίπεδο πληροφορίας που πραγματικά παρέχουν και να μην εξισώνονται αυθαίρετα με καταγεγραμμένες ευπάθειες ή με σύνολο security risk.

4.4 Κατηγορίες Ευπαθειών (Vulnerabilities)

Οι ευπάθειες λογισμικού μπορούν να εμφανιστούν σε διαφορετικά επίπεδα ενός συστήματος και να προκύψουν από διαφορετικές αιτίες, όπως λανθασμένο σχεδιασμό, ανεπαρκή έλεγχο δεδομένων, εσφαλμένη διαχείριση δικαιωμάτων ή μη ασφαλή χρήση εξωτερικών συστατικών. Δεν υπάρχει μία μοναδική και καθολικά αποδεκτή ταξινόμηση που να διαχωρίζει όλες τις ευπάθειες σε πλήρως ανεξάρτητες κατηγορίες. Πλαίσια/Ταξινομήσεις όπως το Common Weakness Enumeration (CWE) και το OWASP Top 10 οργανώνουν τα προβλήματα ασφάλειας με διαφορετικό επίπεδο αφαίρεσης και διαφορετικό σκοπό. Συγκεκριμένα, το CWE εστιάζει κυρίως στις υποκείμενες αδυναμίες σε αντίθεση με το OWASP πρότυπο το οποίο συγκεντρώνει σχετικές αδυναμίες και vulnerabilities σε ευρύτερες κατηγορίες application security risk. Οι βασικότερες κατηγορίες μπορούν να οργανωθούν ενδεικτικά ως εξής:

1. **Authentication και Access Control Vulnerabilities** – Αφορούν προβλήματα στην διαδικασία επαλήθευσης ταυτότητας μιας οντότητας ή στην επιβολή των κατάλληλων περιορισμών πρόσβασης. Παράδειγμα αποτελεί να επιτραπεί σε έναν επιτιθέμενο να παρακάμψει έναν μηχανισμό αυθεντικοποίησης, να αποκτήσει αυξημένα δικαιώματα ή να προσπελάσει δεδομένα και λειτουργίες που ανήκουν σε διαφορετικό χρήστη.
2. **Input Validation και Injection Vulnerabilities** – Εμφανίζονται όταν δεδομένα τα οποία προέρχονται από μη έμπιστες πηγές δεν ελέγχονται ή δεν διαχωρίζονται σωστά από εντολές και άλλες εκτελέσιμες δομές. Εδώ, περιλαμβάνονται προβλήματα όπως SQL Injection, Command Injection και Cross-Site Scripting (XSS) τεχνικές. Συχνά, βασική αιτία αποτελεί η εσφαλμένη διαχείριση δεδομένων που εισέρχονται στην εφαρμογή και χρησιμοποιούνται αργότερα σε security-sensitive λειτουργίες.
3. **Memory και Resource Management Vulnerabilities** – Αναφέρονται σε λανθασμένη διαχείριση μνήμης ή άλλων πόρων, συνήθως φυσικών, του συστήματος. Εδώ περιλαμβάνονται προβλήματα όπως out-of-bounds access, use-after-free, buffer-related weakness, resource exhaustion ή λανθασμένος συγχρονισμός. Αποτέλεσμα αυτών μπορεί να είναι διάφορα crashes, αλλοίωση δεδομένων ή ακόμη και εκτέλεση μη επιτρεπτού κώδικα. Ο συγκεκριμένος τύπος ευπαθειών βρίσκει εφαρμογή σε περιβάλλοντα όπου η διαχείριση της μνήμης πραγματοποιείται σε χαμηλότερο επίπεδο.
4. **Cryptographic και Data Protection Vulnerabilities** – Περιλαμβάνουν προβλήματα τα οποία σχετίζονται με την ακατάλληλη επιλογή, υλοποίηση ή

χρήση κρυπτογραφικών μηχανισμών. Η απλή ύπαρξη κρυπτογραφίας δεν μπορεί να εγγυηθεί από μόνη της την ασφάλεια, εάν ο μηχανισμός χρησιμοποιείται με εσφαλμένο τρόπο. Ενδεικτικές περιπτώσεις αποτελούν η χρήση αδύναμων αλγορίθμων, η εσφαλμένη διαχείριση κλειδίων κρυπτογράφησης, η ύπαρξη hard-coded secrets μέσα στον πηγαίο κώδικα, η ανεπαρκής προστασία ευαίσθητων δεδομένων και η λανθασμένη χρήση κάποιου integrated cryptographic API.

5. **Security Misconfiguration Vulnerabilities** – Προκύπτουν όταν το λογισμικό ή το περιβάλλον στο οποίο λειτουργεί έχει ρυθμιστεί και διαμορφωθεί με μη ασφαλή τρόπο. Παραδείγματα είναι περιττές ενεργοποιημένες υπηρεσίες, default credentials, υπερβολικά δοσμένα δικαιώματα, μη ασφαλή HTTP headers, λανθασμένες ρυθμίσεις cloud ή error messages που μπορεί να αποκαλύψουν ευαίσθητες πληροφορίες. Στις περιπτώσεις αυτές, το πρόβλημα μπορεί να μην βρίσκεται στον ίδιο τον πηγαίο κώδικα αυτόν καθ' αυτόν αλλά στον τρόπο με τον οποίο έχει εγκατασταθεί στο λειτουργικό σύστημα στο οποίο και εκτελείται.
6. **Error Handling και Exceptional Condition Vulnerabilities** – Περιγράφουν την λανθασμένη αντιμετώπιση μη φυσιολογικών καταστάσεων, σφαλμάτων ή απροσδόκητων εισόδων. Ένα σύστημα που αποτυγχάνει με μη ασφαλή τρόπο μπορεί να αποκαλύψει πληροφορίες, να παρακάμψει ελέγχους, να καταλήξει σε μη ασφαλή κατάσταση ή να επιτρέψει denial-of-service συμπεριφορά, συνήθως έπειτα από μία DDoS επίθεση. Η συγκεκριμένη κατηγορία βρίσκεται εφαρμογή τόσο στην ποιότητα όσο και στην ασφάλεια λογισμικού, τονίζοντας την μεταξύ τους σχέση, καθώς ζητήματα όπως robustness και error handling μπορούν να έχουν τόσο λειτουργικές όσο και επιπτώσεις/συνέπειες ασφάλειας.
7. **Software Supply Chain και Dependency Vulnerabilities** – Αφορούν προβλήματα που εισάγονται μέσω libraries, frameworks, packages, build tools ή άλλων εξωτερικών components που χρησιμοποιούνται για την ανάπτυξη και λειτουργία του λογισμικού. Μία εφαρμογή μπορεί να είναι ευάλωτη ακόμη και όταν ο κώδικας που έχει αναπτυχθεί εσωτερικά δεν περιλαμβάνει άμεσα την αντίστοιχη αδυναμία.
8. **Software και Data Integrity Vulnerabilities** – Προκύπτουν όταν το σύστημα εμπιστεύεται το λογισμικό, δεδομένα, updates ή άλλα artifacts χωρίς να επαληθευθεί πλήρως και επαρκώς η ακεραιότητά τους. Σε αυτή την κατηγορία συνήθως εντάσσονται μη ασφαλείς update mechanisms, insecure deserialization, μη επαληθευμένα artifacts και προβλήματα CI/CD pipelines που επιτρέπουν την εισαγωγή ή τροποποίηση κώδικα χωρίς κατάλληλο έλεγχο.
9. **Insecure Design και Business Logic Vulnerabilities** – Οι ευπάθειες αυτής της κατηγορίας δεν οφείλονται απαραίτητα σε ένα μεμονωμένο λάθος υλοποίησης αλλά στον ίδιο τον σχεδιασμό του συστήματος. Μπορεί, για παράδειγμα, να απουσιάζει ένας απαραίτητος security control ή να υπάρχει μία ακολουθία κατά τα άλλα νόμιμων ενεργειών που επιτρέπει σε έναν χρήστη να παρακάμψει έναν επιχειρησιακό κανόνα. Σε αυτές τις περιπτώσεις, ακόμα

και ένας τεχνικά ορθός κώδικας μπορεί να υλοποιεί μία εγγενώς μη ασφαλή σχεδιαστική απόφαση.

10. **Logging, Monitoring και Detection Vulnerabilities** – Αναφέρονται σε ανεπαρκή καταγραφή και παρακολούθηση γεγονότων ασφαλείας. Ξεκαθαρίζεται ότι η απουσία logging δεν δημιουργεί πάντα άμεσα μία exploitable vulnerability, μπορεί να επιτρέψει σε μία επίθεση να παραμείνει απαρατήρητη για μεγάλο χρονικό διάστημα και να δυσκολέψει σημαντικά την ανίχνευση και διερεύνηση περιστατικών.

Οι παραπάνω ορισμένες κατηγορίες δεν αποτελούν αυστηρά και αμοιβαία αποκλειόμενα σύνολα. Μία πραγματική ευπάθεια μπορεί να σχετίζεται ταυτόχρονα με περισσότερες από μία κατηγορίες. Ένα injection vulnerability, για παράδειγμα, μπορεί να γίνει περισσότερο επικίνδυνη λόγω ανεπαρκούς access control, ενώ μία αδυναμία στο error handling μπορεί να προκαλεί παράλληλα και information disclosure. Αντίστοιχα, διαφορετικά classification frameworks μπορούν να περιγράψουν το ίδιο πρόβλημα με διαφορετικό βαθμό λεπτομέρειας.

Η κατηγοριοποίηση είναι ιδιαίτερα χρήσιμη για την αυτοματοποιημένη ανάλυση και τη διαχείριση security findings, επειδή επιτρέπει την οργάνωση μεγάλου αριθμού ευρημάτων σε πιο γενικές κατηγορίες προβλημάτων. Ωστόσο, η δυνατότητα εντοπισμού μιας ευπάθειας εξαρτάται από τη φύση της. Για στατικές αναλύσεις, είναι συχνά περισσότερο κατάλληλα σύνολα αδυναμιών που εμφανίζουν αναγνωρίσιμα patterns στον πηγαίο κώδικα. Αντίθετα, προβλήματα που σχετίζονται με επιχειρησιακή λογική, runtime state, configuration ή system design μπορεί να απαιτούν διαφορετικές τεχνικές εντοπισμού, ανάλυσης και τελικά, αντιμετώπισης. Αποτελεσματικά, καμία μεμονωμένη μέθοδος ή εργαλείο δεν μπορεί να θεωρηθεί ότι παρέχει πλήρη κάλυψη όλων των κατηγοριών των ευπαθειών, όπως ορίστηκαν και αναλύθηκαν παραπάνω.

4.5 Static Application Security Testing

Το SAST εξετάζει πηγαίο κώδικα ή ενδιάμεσες αναπαραστάσεις χωρίς να εκτελεί την εφαρμογή, με στόχο τον εντοπισμό security-relevant patterns και ροών. Η κάλυψη και η ακρίβεια διαφέρουν ανά εργαλείο και κατηγορία αδυναμίας (Scarfone et al., 2008; Lenarduzzi et al., 2023). Στη βιβλιογραφία μπορεί επίσης να αναφερθεί και ως Static Code Analysis και Source Code Analysis, ενώ ο όρος SAST χρησιμοποιείται ειδικότερα όταν η στατική ανάλυση εφαρμόζεται με προσανατολισμό στην ασφάλεια εφαρμογών.

Η βασική αξία του SAST βρίσκεται στη δυνατότητα ενσωμάτωσής του κατά τη διάρκεια της ανάπτυξης και όχι αποκλειστικά μετά την ολοκλήρωση του λογισμικού. Η ανάλυση μπορεί να συμβαίνει επαναλαμβανόμενα καθώς ο κώδικας μεταβάλλεται και να εντάσσεται σε διαδικασίες αξιολόγησης κώδικα, Continuous Integration και γενικότερα σε διαδικασίες Secure SDLC και DevSecOps. Το Static Code Scanning

περιλαμβάνεται, σύμφωνα με το NIST, στις προτεινόμενες τεχνικές developer verification, ενώ το Secure Software Development Framework υποστηρίζει την ενσωμάτωση πρακτικών ασφάλειας σε ολόκληρο τον κύκλο ζωής ανάπτυξης, με στόχο τη μείωση των ευπαθειών και την αντιμετώπιση των αιτιών που οδηγούν στην εμφάνισή τους.

Στο πλαίσιο της παρούσας έρευνας, η SAST τεχνική κατέχει σημαντικό ρόλο. Μέσω της τεχνικής αυτής επιτρέπεται η συστηματική και επαναλήψιμη παρατήρηση χαρακτηριστικών ασφάλειας του πηγαίου κώδικα καθώς αυτός εξελίσσεται. Μέσω της αυτοματοποιημένης παραγωγής security-related findings καθίσταται δυνατή η σύγκριση διαφορετικών καταστάσεων του κώδικα και η εξέταση των μεταβολών που εμφανίζονται μετά από αλλαγές λογισμικού.

4.5.1 Τρόπος λειτουργίας

Η διαδικασία του Static Application Security Testing (SAST) πραγματοποιεί την λειτουργία και ανάλυση του λογισμικού (του πηγαίου του κώδικα) χωρίς να είναι απαραίτητη και να απαιτείται η εκτέλεση του λογισμικού. Η εφαρμογή της ανάλυσης εξαρτάται από την γλώσσα προγραμματισμού και το εργαλείο που θα χρησιμοποιηθεί, καθώς μπορεί να εφαρμοστεί απευθείας στον πηγαίο κώδικα ή σε ενδιάμεσες και μεταγλωττισμένες εκδόσεις/αναπαραστάσεις του. Βασικός στόχος της συγκεκριμένης ανάλυσης είναι η εξαγωγή πληροφοριών σχετικά με τη δομή και τη συμπεριφορά που ένα σύστημα μπορεί να παρουσιάσει και η σύγκρισή τους με κανόνες ή μοντέλα που περιγράφουν γνωστές μη ασφαλείς καταστάσεις. Το NIST περιγράφει έναν source-code security analyzer ως εργαλείο που εξετάζει τον κώδικα, εντοπίζει αδυναμίες ασφάλειας οι οποίες με την σειρά τους είναι ικανές να οδηγήσουν σε security vulnerabilities και αναφέρει τόσο το είδος όσο και τη θέση των αντίστοιχων ευρημάτων.

Συνήθως, η διαδικασία ξεκινά με την επεξεργασία και κατανόηση της δομής του κώδικα. Μία από τις πιο απλές τεχνικές είναι η lexical analysis. Στο πλαίσιο της συγκεκριμένης τεχνικής ο πηγαίος κώδικας διαχωρίζεται σε επιμέρους tokens, identifiers, operators, literals και keywords. Σε άλλες, περισσότερο εξελιγμένες αναλύσεις, ο κώδικας αναπαρίσταται σε δομές που αποτυπώνουν τις συντακτικές και σημασιολογικές σχέσεις μεταξύ των στοιχείων του, όπως ένα Abstract Syntax Tree (AST). Οι αναπαραστάσεις αυτές επιτρέπουν στο εκάστοτε εργαλείο να κατανοήσει ότι για παράδειγμα, διαφορετικές γραμμές κώδικα αποτελούν μία function call, μία assignment ή μία conditional structure, λογική που έρχεται σε αντίθεση με μία απλή, γραμμική συνήθως, αναζήτηση κειμένου. Η AST-based pattern matching έχει χρησιμοποιηθεί στη βιβλιογραφία ως βάση για την αναγνώριση security vulnerability patterns.

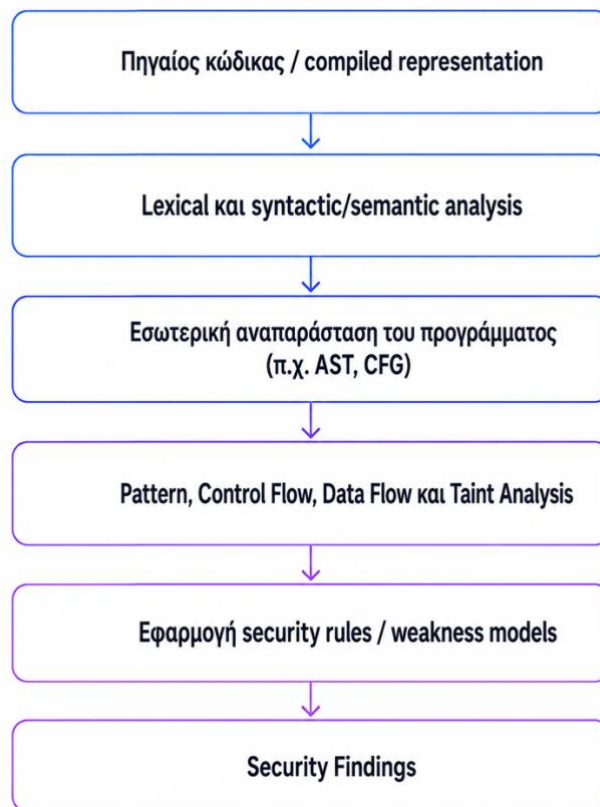
Για την εξέταση της πιθανής συμπεριφοράς του προγράμματος μπορεί να δημιουργηθεί στη συνέχεια ένα Control Flow Graph (CFG). Σε μία τέτοια γραφική αναπαράσταση, οι κόμβοι αντιστοιχούν σε τμήματα ή εντολές του προγράμματος και οι ακμές στις πιθανές μεταβάσεις ελέγχου μεταξύ τους. Έτσι, μέσω της χρήσης αυτής της μεθοδολογίας μπορούν να αναπαρασταθούν διαφορετικές διαδρομές που

προκύπτουν από conditional statements, loops, function calls και άλλες δομές ελέγχου. Η ανάλυση του control flow είναι ιδιαίτερα σημαντική σε περιπτώσεις όπου μία προβληματική κατάσταση δεν βρίσκεται αποκλειστικά σε μία μόνο γραμμή κώδικα αλλά εξαρτάται από την ακολουθία των ενεργειών που προηγούνται αυτής.

Ταυτόχρονα, χρησιμοποιείται η μέθοδος της Data Flow Analysis. Κατά την ανάλυση αυτή εξετάζει τον τρόπο με τον οποίο οι τιμές δημιουργούνται, μεταβάλλονται και μεταφέρονται στο εσωτερικό περιβάλλον στο πρόγραμμα. Το εργαλείο στην περίπτωση αυτή, δεν εξετάζει κάθε statement απομονωμένα, μπορεί να συσχετίζει την τιμή μιας μεταβλητής με το σημείο από το οποίο προήλθε και τα σημεία όπου χρησιμοποιείται στη συνέχεια. Η ανάλυση μπορεί να επεκτείνεται πέρα από τα όρια μιας μεμονωμένης συνάρτησης, μέσω interprocedural analysis, με σκοπό να ακολουθούνται οι σχέσεις μεταξύ διαφορετικών συναρτήσεων ή μεθόδων. Σύγχρονα εργαλεία static analyzers μπορούν να συνδυάζουν control-flow, data-flow, semantic και interprocedural πληροφορίες με στόχο την ανάλυση καταστάσεων που εκτείνονται σε μεγαλύτερα τμήματα του προγράμματος.

Η Taint Analysis αποτελεί μία εξειδικευμένη μορφή data-flow analysis με ιδιαίτερη σημασία για την ασφάλεια. Στην συγκεκριμένη προσέγγιση, δεδομένα τα οποία προέρχονται από μη έμπιστες ή εξωτερικά ελεγχόμενες πηγές χαρακτηρίζονται ως tainted. Στη συνέχεια η διάδοσή τους μέσα στο σύστημα, παρακολουθείται από το εργαλείο, από ένα source προς ένα δυνητικά ευαίσθητο σημείο χρήσης, χαρακτηριζόμενο ως sink. Παράλληλα εξετάζεται εάν κατά τη συγκεκριμένη διαδρομή εφαρμόζεται κάποιος κατάλληλος μηχανισμός validation ή sanitization. Λαμβάνοντας ως παράδειγμα δεδομένα προερχόμενα από έναν χρήστη, μπορούν να φτάσουν σε μία security-sensitive function χωρίς κατάλληλη επεξεργασία, η διαδρομή αυτή μπορεί να αναγνωριστεί από το εργαλείο ως security-relevant finding.

Μολαταύτα, δεν βασίζονται όλες οι αναλύσεις SAST αποκλειστικά στο data flow. Τα εργαλεία μπορούν να χρησιμοποιούν διαφορετικές και συχνά συνδυαστικές τεχνικές, όπως pattern matching, semantic analysis, rule-based analysis, abstract interpretation ή άλλες μορφές στατικής συλλογιστικής πάνω στο πρόγραμμα. Οι κανόνες που εφαρμόζονται κατά την ανάλυση μπορούν να αντιστοιχούν σε γνωστές κατηγορίες αδυναμιών, όπως αυτές που περιγράφονται μέσω του CWE. Σε περίπτωση πλήρωσης των συνθηκών ενός κανόνα, το εργαλείο δημιουργεί ένα finding και μπορεί να αναφέρει την κατηγορία του προβλήματος, το αρχείο και τη γραμμή στην οποία εντοπίστηκε, καθώς και πρόσθετες πληροφορίες που βοηθούν στην κατανόηση της διαδρομής που οδήγησε σε αυτό.



Εικόνα 4.5.1 – Αναπαράσταση βημάτων SAST ανάλυσης *

Η συγκεκριμένη ακολουθία αποτελεί μία εννοιολογική απεικόνιση και όχι υποχρεωτική αρχιτεκτονική κάθε SAST εργαλείου. Διαφορετικά εργαλεία χρησιμοποιούν διαφορετικούς συνδυασμούς τεχνικών, αναπαραστάσεων και αλγορίθμων ανάλογα με τις γλώσσες προγραμματισμού και τις κατηγορίες weaknesses που επιδιώκουν να εντοπίσουν. Το κοινό χαρακτηριστικό όλων είναι ότι επιχειρούν να συναγάγουν πληροφορίες για πιθανές προβληματικές καταστάσεις εξετάζοντας στατικά το πρόγραμμα, χωρίς να χρειάζεται να προκαλέσουν πρώτα την αντίστοιχη συμπεριφορά κατά την εκτέλεσή του.

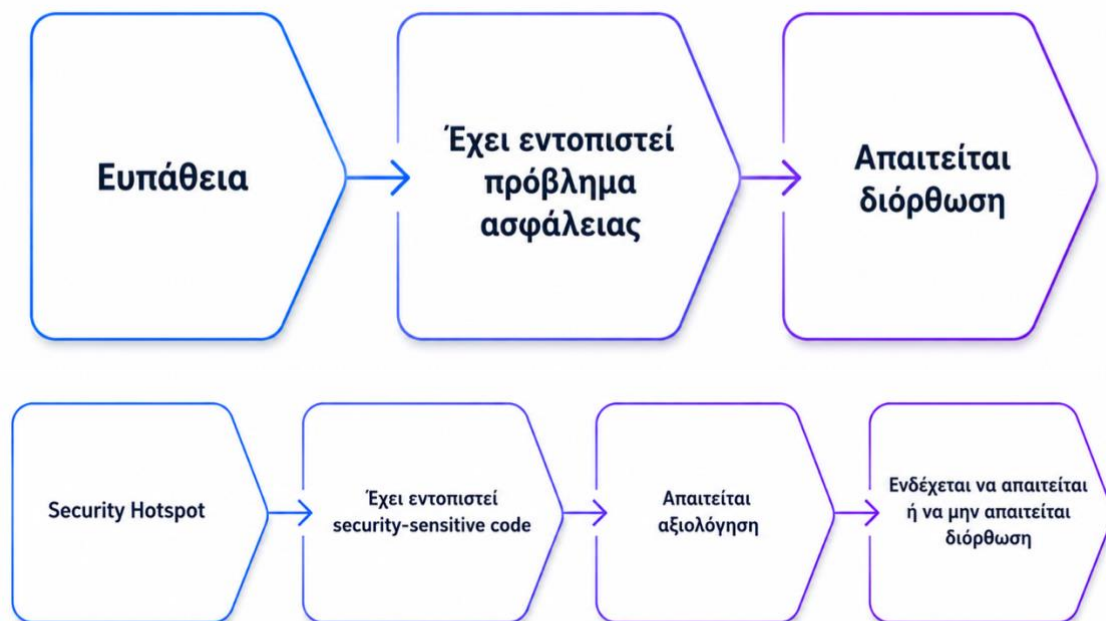
4.5.2 Vulnerabilities και Security Hotspots

Στο SonarQube, ένα Security Hotspot επισημαίνει security-sensitive κώδικα που χρειάζεται ανθρώπινο review. Δεν αποτελεί αυτομάτως επιβεβαιωμένη vulnerability: μετά την εξέταση μπορεί να κριθεί ασφαλές ή να απαιτήσει πρόσθετη προστασία (SonarSource, χ.χ.-α). Αντίθετα, ένα vulnerability δηλώνει πρόβλημα για το οποίο το εργαλείο αποδίδει υψηλότερο επίπεδο βεβαιότητας ότι απαιτείται διορθωτική παρέμβαση.

Στο συγκεκριμένο πλαίσιο, μία ευπάθεια αποτελεί εύρημα για το οποίο ο κανόνας στατιστικής ανάλυσης έχει αναγνωρίσει μία κατάσταση που θεωρείται ότι επηρεάζει την ασφάλεια της εφαρμογής και επομένως απαιτεί διόρθωση. Τέτοιες περιπτώσεις μπορούν να προκύπτουν όταν η ανάλυση αναγνωρίζει μία μη ασφαλή

ροή δεδομένων ή έναν συνδυασμό συνθηκών που αντιστοιχεί σε γνωστό security weakness. Η δυνατότητα μη έμπιστων δεδομένων να φτάσουν σε μία ευαίσθητη λειτουργία χωρίς επαρκή προστασία μπορεί να οδηγήσει στην αναγνώριση injection-related vulnerability. Η έννοια παραμένει συνεπής με τον γενικό ορισμό της vulnerability ως weakness ή flaw, όπως έχει ήδη οριστεί, που μπορεί να αξιοποιηθεί ώστε να επηρεαστούν οι στόχοι ασφάλειας ενός συστήματος.

Ένα Security Hotspot από την αντίθετη πλευρά δεν δηλώνει αυτομάτως ότι έχει εντοπιστεί πραγματική ευπάθεια. Ο όρος χρησιμοποιείται κυρίως για να επισημαίνει ένα τμήμα πηγαίου κώδικα που χρησιμοποιεί μία security-sensitive κατασκευή ή λειτουργία για το οποίο απαιτείται ανθρώπινη αξιολόγηση μέσα στο πραγματικό context της εφαρμογής. Μετά την εξέταση μπορεί να διαπιστωθεί ότι ο υπάρχων κώδικας είναι ασφαλής και δεν απαιτεί αλλαγή ή, αντίθετα, ότι χρειάζεται πρόσθετη προστασία ή/και τροποποίηση. Εν συνεχεία, τα Security Hotspots αντιμετωπίζονται περισσότερο ως σημεία που απαιτούν αξιολόγηση του security και αποφεύγεται να χαρακτηριστεί ως επιβεβαιωμένη ευπάθεια.



Εικόνα 4.5.2 – Διαφορά μεταξύ Vulnerability (Ευπάθειας) και Security Hotspot *

Χαρακτηριστικό παράδειγμα αποτελεί η χρήση συγκεκριμένων security-sensitive μηχανισμών προστασίας. Η απουσία ενός πρόσθετου defensive measure μπορεί να επισημανθεί ως ένα security hotspot, λόγω του ότι η αναγκαιότητά του εξαρτάται από το περιβάλλον και τον τρόπο χρήσης του κώδικα. Όταν η ανάλυση αναγνωρίζει μία κατάσταση που αποτελεί άμεσα security flaw, όπως μία επιβεβαιωμένη μη ασφαλή ροή που μπορεί να οδηγήσει σε injection, ονομάζεται vulnerability και είναι η αντίθετη περίπτωση του προηγουμένως περιγραφόμενου όρου. Η δυσκολία exploitation, όπως αυτό τονίζεται και από τη SonarSource, δεν αποτελεί στοιχείο διαφοροποίησης των δύο κατηγοριών. Η βασική διαφορά είναι αν

απαιτείται πρώτα ανθρώπινο review για να αποφασιστεί η ύπαρξη ενός πραγματικού προβλήματος.

Η παραπάνω διάκριση επηρεάζει άμεσα τον τρόπο με τον οποίο παρουσιάζονται οι αντίστοιχες μετρικές. Στο SonarQube Server 2026.1, η μετρική vulnerabilities καταγράφει τον συνολικό αριθμό των vulnerabilities, σε αντίθεση με την security_hotspots η οποία και καταγράφει ξεχωριστά τον αριθμό των Security Hotspots. Για τα hotspots χρησιμοποιούνται επιπλέον μετρικές, όπως είναι το ποσοστό των ευρημάτων που έχουν ήδη εξεταστεί και το Security Review Rating, ακριβώς επειδή η διαδικασία review αποτελεί αναπόσπαστο μέρος της αξιολόγησής τους. Η διαφορετική μεταχείριση, όπως αυτή περιγράφεται, επιβεβαιώνει ότι οι δύο μετρικές δεν αποτυπώνουν το ίδιο είδος πληροφορίας.

Ιδιαίτερη σημασία αποκτά και για την ερμηνεία των αποτελεσμάτων μιας εμπειρικής μελέτης. Ένας μεγαλύτερος αριθμός Security Hotspots υποδηλώνει περισσότερα σημεία του κώδικα, πιθανά να εμφανίσουν κάποιο vulnerability και απαιτούν με την σειρά τους security-oriented εξέταση. Αντίθετα περισσότερα vulnerabilities συνεπάγεται περισσότερα security findings τα οποία το εκάστοτε εργαλείο (εδώ, SonarQube) έχει ταξινομήσει ως προβλήματα που απαιτούν remediation. Ένα hotspot δεν πρέπει να αντιμετωπίζεται ως ισοδύναμο με μία ευπάθεια, ούτε ο αριθμός των hotspots να ερμηνεύεται αυτομάτως ως αριθμός πραγματικών ευπαθειών. Τέλος τονίζεται ότι οι δύο κατηγορίες παρέχουν διαφορετικές αλλά μεταξύ τους συμπληρωματικές πληροφορίες σχετικά με την κατάσταση ασφάλειας του πηγαίου κώδικα.

4.5.3 False positives και False negatives

Η αποτελεσματικότητα ενός SAST εργαλείου δεν έχει αποκλειστική εξάρτηση τον αριθμό των ευρημάτων που μπορεί να παραγάγει, αλλά επίσης εξαρτάται και από το κατά πόσο αυτά αντιστοιχούν σε πραγματικά προβλήματα ασφάλειας. Η στατική ανάλυση προσπαθεί να συμπεράνει πιθανές συμπεριφορές του λογισμικού χωρίς όμως να βρίσκεται σε εκτελέσιμη κατάσταση. Έτσι, τα αποτελέσματα που πρόκειται να εξαχθούν από μία τέτοια ανάλυση μπορούν να περιλαμβάνουν τόσο False Positives (FP) όσο και False Negatives (FN). Οι δύο αυτές κατηγορίες σφάλματος είναι διαφορετικές μεταξύ τους και επηρεάζουν με διαφορετικό τρόπο και βαρύτητα την αξιοπιστία και την πρακτική χρησιμότητα της ανάλυσης. Τα FP και FN έχουν χρησιμοποιηθεί ως βασικές ποσοτικές μεταβλητές για την εξέταση της αποτελεσματικότητας των εργαλείων SAST.

False positive είναι εύρημα που αναφέρεται από το εργαλείο αλλά δεν αντιστοιχεί σε πραγματικό πρόβλημα στο συγκεκριμένο context. Αντίστροφα, false negative είναι υπαρκτό πρόβλημα που δεν ανιχνεύεται. Οι δύο περιπτώσεις πρέπει να συνδέονται με precision και recall και να τεκμηριώνονται εμπειρικά (Lenarduzzi et al., 2023; Delaitre et al., 2023). Το αναμενόμενο αποτέλεσμα το οποίο παράγεται είναι ένα warning (ειδοποίηση) που απαιτεί ανθρώπινο έλεγχο αλλά τελικά δεν χρειάζεται remediation.

Συνεπαγωγικά, η ύπαρξη μεγάλου αριθμού false positives μπορεί να αυξήσει σημαντικά τον απαιτούμενο χρόνο για την εξέταση των αποτελεσμάτων. Οι προγραμματιστές πρέπει να διακρίνουν τα πραγματικά προβλήματα από τα μη σχετιζόμενα warnings, με αποτέλεσμα να αυξάνεται αυτομάτως το κόστος της διαδικασίας αξιολόγησης. Εμπειρικές μελέτες έχουν δείξει ότι τα False Positives αποτελούν έναν από τους παράγοντες που δυσκολεύουν την υιοθέτηση των static-analysis tools από τις ομάδες ανάπτυξης. Η σύγκριση έξι static-analysis tools ως ερευνητική διαδικασία από τους Lenarduzzi et al. έδειξε περιορισμένη συμφωνία μεταξύ των εργαλείων και χαμηλή precision σε αρκετές περιπτώσεις, δείχνοντας έντονα την ανάγκη ορθής και ανθρώπινης αξιολόγησης των warnings για να καταταχθούν ως πραγματικά defects.

Αντίθετα, ένα False Negative (FN) εμφανίζεται όταν υπάρχει πραγματική αδυναμία ή ευπάθεια στον κώδικα αλλά το εργαλείο δεν την αναγνωρίζει. Η συγκεκριμένη περίπτωση αναφέρεται ως κρίσιμη από πλευράς ασφαλείας, αφού η απουσία ενός finding μπορεί να δημιουργήσει την εσφαλμένη εντύπωση ότι το συγκεκριμένο τμήμα του λογισμικού που εξετάζεται δεν παρουσιάζει κάποιο πρόβλημα. Τα αποτελέσματα του NIST SATE VI δείχνουν ότι η ικανότητα των static analyzers να εντοπίζουν σφάλματα μεταβάλλεται συνεχώς και σημαντικά ανάλογα με την κατηγορία και την πολυπλοκότητά τους. Στην αξιολόγηση, περισσότερα σύνθετα σφάλματα (bugs) παρουσίασαν χαμηλότερα ποσοστά εντοπισμού από απλούστερα προβλήματα, γεγονός που επιβεβαιώνει ότι η απουσία warning δεν αποτελεί απόδειξη απουσίας αδυναμίας ασφαλείας. Συνοπτικά, βάσει μαθηματικών πράξεων:

Υπάρχει πραγματικό πρόβλημα + το SAST το εντοπίζει = True Positive (TP)

Δεν υπάρχει πραγματικό πρόβλημα + το SAST δεν εντοπίζει πρόβλημα = True Negative (TN)

Δεν υπάρχει πραγματικό πρόβλημα + το SAST εντοπίζει πρόβλημα = False Positive (FP)

Υπάρχει πραγματικό πρόβλημα + το SAST δεν εντοπίζει πρόβλημα = False Negative (FN)

Βάσει των παραπάνω κατηγοριών, όπως αυτές περιεγράφηκαν μπορούν να υπολογιστούν διαφορετικά μέτρα απόδοσης. Η μετρική Precision εκφράζει ποιο ποσοστό των περιπτώσεων χαρακτηρίστηκαν θετικές από το εργαλείο σε αντίθεση με τις πραγματικά θετικές περιπτώσεις:

$$Precision = \frac{TP}{TP+FP}$$

ενώ το Recall εκφράζει ποιο ποσοστό των πραγματικών προβλημάτων κατάφερε να εντοπίσει:

$$Recall = \frac{TP}{TP+FN}$$

Ένα εργαλείο με υψηλή Precision παράγει σχετικά λίγα false-positive findings, σε αντίθεση με ένα εργαλείο με υψηλό Recall που αφήνει σχετικά λίγα πραγματικά προβλήματα μη εντοπισμένα. Συμπεραίνουμε επομένως ότι η αξιολόγηση ενός SAST εργαλείου δεν θα πρέπει να στηρίζεται αποκλειστικά στον συνολικό αριθμό warnings, αλλά θα πρέπει επίσης να συνδυάζει και την ικανότητά του να διακρίνει

πραγματικές αδυναμίες και να καλύπτει τις κατηγορίες προβλημάτων που ενδιαφέρουν.

Η εμφάνιση False Positives και False Negatives συνδέεται με το γεγονός ότι κάθε analyzer διαθέτει συγκεκριμένο σύνολο κανόνων, μοντέλων και δυνατότητες κατανόησης του προγράμματος. Πολύπλοκες διαδρομές ελέγχου, interprocedural συμπεριφορά, framework-specific λειτουργίες ή μηχανισμοί validation που δεν αναγνωρίζονται από το εκάστοτε εργαλείο μπορούν να καταλήγουν σε διαφορετικά αποτελέσματα πάνω στον ίδιο πηγαίο κώδικα.

Η παρουσία ενός SAST finding δεν θα πρέπει να θεωρείται χωρίς αξιολόγηση απόλυτη απόδειξη πραγματικής vulnerability όπως επίσης και η απουσία findings δεν αποτελεί απόδειξη ότι ο κώδικας είναι ασφαλής. Στο πλαίσιο που ορίζεται από μια εμπειρική ανάλυση, τα automated findings είναι ορθότερο να αντιμετωπίζονται ως παρατηρήσιμες ενδείξεις που παράγονται από το συγκεκριμένο εργαλείο και τη συγκεκριμένη configuration, λαμβάνοντας υπόψη ότι ένα μέρος των πραγματικών προβλημάτων μπορεί να παρατηρηθεί και ένα μέρος των αναφερόμενων προβλημάτων ενδέχεται να απαιτεί περαιτέρω επιβεβαίωση.

4.5.4 Περιορισμοί ανάλυσης SAST

Παρόλο που το Static Application Security Testing (SAST) αποτελεί ιδιαίτερα χρήσιμη τεχνική για την έγκαιρη αναγνώριση security-related προβλημάτων στον πηγαίο κώδικα, παρουσιάζει σημαντικούς περιορισμούς οι οποίοι πρέπει να λαμβάνονται υπόψη κατά την ερμηνεία των αποτελεσμάτων του. Ο σημαντικότερος εξ αυτών προκύπτει από την φύση της ίδιας της μορφής της ανάλυσης, όπου το SAST εξετάζει το πρόγραμμα χωρίς να παρατηρεί την πραγματική του εκτέλεση. Μπορεί επομένως να συλλογίζεται πάνω στη δομή, στις πιθανές υπαρκτές ροές ελέγχου και δεδομένων και σε γνωστά security patterns, αλλά δεν διαθέτει πάντοτε πλήρη γνώση της συμπεριφοράς που θα εμφανίσει το σύστημα στο πραγματικό περιβάλλον εκτέλεσης και λειτουργίας του.

Επειδή το SAST δεν παρατηρεί πλήρως την εκτέλεση και το παραγωγικό περιβάλλον, δυσκολεύεται να αξιολογήσει προβλήματα που εξαρτώνται από runtime state, configuration ή αλληλεπιδράσεις με εξωτερικά συστήματα. Για τον λόγο αυτό τα ευρήματά του συμπληρώνονται από άλλες τεχνικές και ανθρώπινη εξέταση (Scarfione et al., 2008). Προβλήματα όπως ορισμένες μορφές αυθεντικοποίησης, ελέγχου πρόσβασης και μη ασφαλούς χρήσης κρυπτογραφίας είναι δύσκολο να εντοπιστούν πλήρως μέσω αυτοματοποιημένης στατικής ανάλυσης. Επίσης, configuration issues δεν συμπεριλαμβάνονται συχνά στο πεδίο ανάλυσης καθώς αποφεύγεται η αποτύπωσή τους στον αναλυόμενο πηγαίο κώδικα.

Αντίστοιχος περιορισμός εμφανίζεται σε design και business-logic vulnerabilities. Ενώ ένα σύστημα μπορεί να είναι συντακτικά και τεχνικά συνεπές με τους κανόνες τους οποίους και εφαρμόζει το SAST, ο ίδιος ο σχεδιασμός του μπορεί να επιτρέπει μία μη ασφαλή επιχειρησιακή ροή. Λόγου χάρη, ένας απαραίτητος έλεγχος ασφάλειας μπορεί να μην έχει προβλεφθεί καθόλου ή ένας χρήστης να

μπορεί να εκμεταλλευτεί μία ακολουθία νόμιμων ενεργειών για να επιτύχει αποτέλεσμα που δεν έχει προβλεφθεί από τους σχεδιαστές του λογισμικού.

Η αποτελεσματικότητα της εκτελούμενης ανάλυσης εξαρτάται επίσης από το εκάστοτε χρησιμοποιούμενο εργαλείο, τη γλώσσα προγραμματισμού, τα frameworks και τους διαθέσιμους κανόνες. Ένα SAST εργαλείο μπορεί να υποστηρίζει αποτελεσματικά ένα σύνολο συγκεκριμένων γλωσσών ή κλάσεων ευπαθειών χωρίς να διαθέτει πλήρης κάλυψη για άλλες. Η σωστή κατανόηση των libraries και frameworks είναι επίσης σημαντική, καθώς μεγάλο μέρος της πραγματικής ροής δεδομένων και των security controls μπορεί να υλοποιείται, εκτελείται και επεξεργάζεται μέσω άλλων εξωτερικών APIs. Συνεπώς, βασικά κριτήρια επιλογής ενός SAST εργαλείου είναι η υποστήριξη της χρησιμοποιούμενης στο λογισμικό γλώσσας προγραμματισμού, frameworks, build requirements και vulnerability classes.

Επιπροσθέτως, η στατική ανάλυση αντιμετωπίζει αναπόφευκτο συμβιβασμό μεταξύ ακρίβειας, κάλυψης και υπολογιστικού κόστους. Η πλήρης εξέταση όλων των δυνατών διαδρομών ενός υπολογιστικά σύνθετου συστήματος, ειδικά σε περιπτώσεις όπου υπάρχουν διακλαδώσεις, αλληλεπιδράσεις μεταξύ συναρτήσεων ή σύνθετες ροές δεδομένων, μπορεί να γίνει αρκετά απαιτητική. Όταν σκοπός είναι η διατήρηση του συνολικού χρόνου ανάλυσης σε πρακτικά επίπεδα, τα εργαλεία χρησιμοποιούν πολλαπλά διαφορετικές προσεγγίσεις, approximations και configurations. Οι επιλογές configuration είναι ικανές να επηρεάζουν άμεσα την precision, το recall και τον χρόνο εκτέλεσης, μεταξύ των οποίων πρακτικά υπάρχει trade-off.

Η πολυπλοκότητα που υπολογίζεται ως υπαρκτή στον ίδιο τον κώδικα επηρεάζει επίσης άμεσα την αποτελεσματικότητα των analyzers εργαλείων. Επίσης, μεταβάλλεται ανάλογα με το είδος του εντοπισμένου bug, τη γλώσσα προγραμματισμού και το εξεταζόμενο code base. Η δυνατότητα, επομένως, ενός SAST εργαλείου να αναγνωρίσει μία αδυναμία δεν είναι σταθερή για κάθε μία από τις κατηγορίες προβλημάτων.

Ένας ακόμα σημαντικός περιορισμός είναι η εξάρτηση των αποτελεσμάτων από το εκάστοτε χρησιμοποιούμενο analyzer εργαλείο και τη διαμόρφωση στην οποία έχει υποβληθεί. Διαφορετικά εργαλεία είθισται να εφαρμόζουν διαφορετικούς ή τροποποιημένους κανόνες και να μην χρησιμοποιούν τις ίδιες τεχνικές ανάλυσης. Ένα security finding, επομένως, θα πρέπει να θεωρείται αποτέλεσμα του συνδυασμού των κώδικα, analyzer, σύνολο διαμορφωμένων κανόνων (ruleset), configuration και έκδοσης του εργαλείου και να αποφεύγεται η απολύτως ανεξάρτητη μέτρηση της ασφάλειας του λογισμικού. Στον περιορισμό αυτό προστίθενται τα false positives και false negatives, όπως προηγουμένως αναλύθηκαν. Η παρουσία τους συνεπάγεται την αδυναμία της πλήρους αντιστοίχισης των πραγματικών vulnerabilities ενός συστήματος με το αποτέλεσμα που εξάγει η ανάλυση SAST. Έτσι, συνιστάται οι οργανισμοί να αξιολογούν την καταλληλότητα ενός εργαλείου εφαρμοσμένο στον πηγαίο κώδικα τον οποίο παράγουν και στις ανάγκες του ίδιου του οργανισμού πριν στηριχθούν σε αυτό επιχειρησιακά.

Οι παραπάνω περιορισμοί σημαίνουν ότι η ανάλυση SAST δεν πρέπει να χρησιμοποιείται ως μοναδικός και απόλυτος μηχανισμός απόδειξης της ασφάλειας

ενός συστήματος. Η απουσία findings δεν ταυτίζεται με την μη ύπαρξη vulnerabilities σε ένα λογισμικό, ενώ η ύπαρξη πολλών findings δεν αρκεί αντίστοιχα για να προσδιορίσει το πραγματικό security Risk. Είναι θεμιτό να χρησιμοποιείται συνδυασμός τεχνικών, όπως threat modeling, static analysis, dynamic testing, fuzzing, web application scanning και έλεγχο τρίτων components. Προτείνεται για τον λόγο ότι διαφορετικές τεχνικές παρατηρούν διαφορετικές πλευρές του ίδιου συστήματος λογισμικού. Το SAST πρέπει επομένως να αντιμετωπίζεται ως ένα επίπεδο ενός ευρύτερου Software Security Verification strategy και όχι ως πλήρης υποκατάσταση των υπόλοιπων ελέγχων.

Στο πλαίσιο της εργασίας, οι περιορισμοί και η αναφορά και περιγραφή τους είναι κρίσιμοι για την ερμηνεία των security metrics. Οι μετρικές που προκύπτουν από το SAST αποτυπώνουν το σύνολο των πραγματικών ή μη προβλημάτων που μπορεί να παρατηρήσει το συγκεκριμένο εργαλείο, εδώ το SonarQube, υπό τη συγκεκριμένη διαμόρφωση και όχι το σύνολο της πραγματικής ασφάλειας που είναι υλοποιημένη στο πλαίσιο του λογισμικού. Οι μεταβολές των Ευπαθειών και των Security Hotspots αντιμετωπίζονται ως μεταβολές στα αυτοματοποιημένα παρατηρούμενα security findings και όχι ως άμεση απόδειξη αντίστοιχης μεταβολής του συνολικού security Risk.

4.6 Συμπληρωματικές Τεχνικές Ασφάλειας

Αν και η μεθοδολογία της Static Application Security Testing (SAST) μπορεί να εντοπίσει σημαντικές κατηγορίες αδυναμιών μέσα στον πηγαίο κώδικα, δεν είναι ικανό να καλύψει εξ ολοκλήρου το φάσμα των προβλημάτων ασφάλειας ενός σύγχρονου συστήματος λογισμικού. Η πραγματικά υλοποιημένη ασφάλεια ενός συστήματος επηρεάζεται τόσο από τον κώδικα που αναπτύσσεται εσωτερικά σε αυτό, όσο και από τη συμπεριφορά της κατά την εκτέλεση, τις εξωτερικές βιβλιοθήκες και dependencies που χρησιμοποιεί, τις ρυθμίσεις του περιβάλλοντος, την υποδομή πάνω στην οποία εκτελείται και τον τρόπο διαχείρισης ευαίσθητων πληροφοριών. Τα σύγχρονα Secure SDLC και DevSecOps περιβάλλοντα, επομένως, βασίζονται συνήθως σε συνδυασμό διαφορετικών και συμπληρωματικών τεχνικών ασφάλειας.

Μία από τις σημαντικότερες συμπληρωματικές προσεγγίσεις ανάλυσης αποτελεί το Dynamic Application Security Testing (DAST). Το DAST στοχεύει στην εξέταση ενός συστήματος ενώ αυτό βρίσκεται σε κατάσταση εκτέλεσης και αλληλοεπιδρά με αυτή από εξωτερική οπτική. Μέσω αυτής της τεχνικής, μπορεί να παρατηρήσει προβλήματα που εμφανίζονται κατά την πραγματική εκτέλεση και τα οποία πιθανότατα να μην είναι εμφανή αποκλειστικά από την εξέταση του πηγαίου κώδικα, γραμμή προς γραμμή. Το Software Composition Analysis (SCA) αντίστοιχα και το Dependency Scanning επικεντρώνονται στα τρίτα components, libraries και packages από τα οποία εξαρτάται η ορθή και ολοκληρωμένη λειτουργία ενός

συστήματος. Με τον τρόπο αυτό καθιστά δυνατή την αναγνώριση γνωστών ευπαθειών και άλλων κινδύνων που προέρχονται από το Software Supply Chain.

Ένα σημαντικό μέρος της ασφάλειας αφορά artifacts και πληροφορίες οι οποίες βρίσκονται έξω από τον παραδοσιακό πηγαίο κώδικα της εφαρμογής. Σε αυτά περιλαμβάνονται πληροφορίες όπως τα passwords, API keys και άλλα credentials που μπορεί να έχουν αποθηκευτεί από λάθος σε repositories ή configuration files. Ο λάθος χειρισμός των προηγούμενων καλύπτεται από την τεχνική Secret Scanning. Αντίστοιχα, το Infrastructure as Code (IaC) και Configuration Scanning επιτρέπει την αυτοματοποιημένη εξέταση αρχείων που περιγράφουν υποδομές και ρυθμίσεις. Έτσι, εντοπίζονται πιθανές misconfigurations πριν αυτές εφαρμοστούν σε πραγματικά περιβάλλοντα. Η δυνατότητα εκτέλεσης τέτοιων ελέγχων μέσα σε repositories και CI/CD pipelines επιτρέπει την αναγνώριση προβλημάτων σε όσο το δυνατόν νωρίτερο στάδιο της διαδικασίας ανάπτυξης και deployment.

Είναι σημαντικό να διευκρινιστεί η σχέση την οποία αναπτύσσουν μεταξύ τους καθώς και με την μεθοδολογία SAST, η οποία περιεγράφηκε προηγουμένως. Οι τεχνικές αυτές δεν λειτουργούν ως υποκατάστατες, καθώς εξετάζουν διαφορετικές πλευρές της επιφάνειας ασφάλειας του υλοποιημένου λογισμικού. Το SAST εστιάζει κυρίως στην εσωτερική υλοποίηση, το DAST στη συμπεριφορά του εκτελούμενου συστήματος, το SCA στα εξωτερικά components και dependencies, ενώ τα Secret, IaC και Configuration Scanning στοχεύουν στην κάλυψη ευαίσθητων πληροφοριών και στοιχείων της υποδομής και της διαμόρφωσης. Η συνδυασμένη εφαρμογή τους παρέχει πληρέστερη εικόνα από την χρήση μεμονωμένης τεχνικής και αποτελεί βασικό χαρακτηριστικό μιας σύγχρονης προσέγγισης DevSecOps και Continuous Security Verification.

4.6.1 Dynamic Application Security Testing (DAST)

Το Dynamic Application Security Testing (DAST) αποτελεί τεχνική ελέγχου ασφάλειας κατά την οποία το λογισμικό εξετάζεται με γνώμονα την ασφάλεια ενώ εκτελείται και βρίσκεται σε ενεργή λειτουργία. Αντίθετα με την λογική του SAST, το οποίο προβαίνει στην στατική ανάλυση του πηγαίου κώδικα ή άλλες εσωτερικές αναπαραστάσεις του προγράμματος, το DAST αλληλοεπιδρά με την εκτελούμενη εφαρμογή και αξιολογεί απευθείας τη συμπεριφορά της μέσω των διαθέσιμων interfaces. Λόγω των προηγούμενων, είθισται να χαρακτηρίζεται ως black-box προσέγγιση, καθώς η ανάλυση μπορεί να πραγματοποιηθεί χωρίς άμεση πρόσβαση και γνώση του πηγαίου κώδικα ή της εσωτερικής υλοποίησης του συστήματος.

Το DAST αλληλεπιδρά με εκτελούμενη εφαρμογή, αποστέλλει διαμορφωμένα inputs σε προσβάσιμες διεπαφές και αξιολογεί τις αποκρίσεις για ενδείξεις ευπαθειών. Η τεχνική παρατηρεί πραγματική συμπεριφορά, αλλά η κάλυψή της εξαρτάται από τα σημεία και τις διαδρομές που εξετάστηκαν (Scarfone et al., 2008). Στην κατηγορία αυτή, το NIST κατατάσσει αντίστοιχα τους web application scanners ως εργαλεία που εξερευνούν μία εφαρμογή μέσω διαδικασίας crawling, παράγουν δυνητικά κακόβουλες εισόδους και αξιολογούν τις αποκρίσεις της προκειμένου να αναγνωρίσουν προβλήματα ασφάλειας.

Με την χρήση της συγκεκριμένης διαδικασίας, το DAST μπορεί να εντοπίζει προβλήματα που γίνονται εμφανή κατά την πραγματική αλληλεπίδραση με το σύστημα. Ενδεικτικά, μπορούν να εξετάζουν ζητήματα όπως input και output validation, injection, authentication, access control και server configuration. Ένα εργαλείο, για παράδειγμα, μπορεί να αποστείλει ειδικά διαμορφωμένα δεδομένα σε ένα input field ή API parameter και να αναλύσει εάν η απόκριση του server υποδηλώνει ότι τα δεδομένα χρησιμοποιήθηκαν με μη ασφαλή τρόπο. Ταυτόχρονα, η εξωτερική αυτή οπτική της συγκεκριμένης τεχνικής αποτελεί έναν από τους πιο σημαντικούς παράγοντες που διαφοροποιούν μία SAST με μία DAST ανάλυση. Το SAST επιχειρεί να εντοπίζει πιθανές αδυναμίες μέσω της ανάλυσης της εσωτερικής δομής και τις ροές ελέγχου και δεδομένων του πηγαίου κώδικα, σε αντίθεση με το DAST εξετάζει τι μπορεί να παρατηρηθεί και να αξιοποιηθεί μέσω του συστήματος που εκτελείται. Ένα DAST finding, συνεπώς, παρέχει συχνά ένδειξη ότι μία προβληματική συμπεριφορά είναι πραγματικά προσβάσιμη από κάποιο εκτεθειμένο interface, χωρίς όμως το χρησιμοποιούμενο εργαλείο να γνωρίζει απαραίτητα ποια συγκεκριμένη γραμμή ή εσωτερική διαδρομή κώδικα αποτελεί τη βασική αιτία του προβλήματος που προκαλείται. Η συμπληρωματικότητα αυτή είναι ένας από τους λόγους για τους οποίους οι δύο τεχνικές χρησιμοποιούνται συχνά μαζί.

Η αποτελεσματικότητα της μεθοδολογίας DAST εξαρτάται σημαντικά από το εάν και πόσο καλά μπορεί να εξερευνηθεί και να ενεργοποιηθεί η λειτουργικότητα της εφαρμογής. Το εργαλείο αποτελεί το σύστημα από εξωτερική οπτική, με αποτέλεσμα να μπορεί να εξετάσει μόνο λειτουργίες και code paths που είναι δυνατό να προσεγγίσει μέσω των διαθέσιμων interfaces και των δοκιμών τις οποίες και πραγματοποιεί. Παράμετροι όπως είναι οι πολύπλοκες διαδικασίες αυθεντικοποίησης, δυναμικές εφαρμογές, μη εμφανή endpoints ή επιχειρησιακές ροές πολλών σταδίων μπορούν να δυσκολέψουν την πλήρη κάλυψη. Η κάλυψη όλων των code paths σε ένα τεστ black-box τύπου πρακτικά είναι δύσκολη, καθώς ο tester δεν διαθέτει άμεση ορατότητα στις εσωτερικές διαδρομές εκτέλεσης και ο αριθμός των πιθανών paths πρόκειται να αυξηθεί σημαντικά.

Προϋποθέτει επίσης την ύπαρξη ενός εκτελούμενου και προσβάσιμου περιβάλλοντος. Για τον λόγο αυτό η εφαρμογή του συμβαίνει αργότερα στον κύκλο ανάπτυξης από ό,τι η στατική ανάλυση, χωρίς όμως να συνεπάγεται περιορισμός σε τελικό pre-release testing. Μπορεί να αυτοματοποιηθεί αντίστοιχα και να ενσωματωθεί σε κατάλληλα test ή staging περιβάλλοντα μέσα σε CI/CD pipelines, προκειμένου οι δυναμικοί έλεγχοι να επαναλαμβάνονται καθώς η εφαρμογή εξελίσσεται. Η SAST ανάλυση, όπως τοποθετείται από το OWASP DevSecOps Verification Standard, στη Code/Build φάση, σε αντίθεση με το DAST όπου τοποθετείται στη Test Phase.

Παρά τα πλεονεκτήματα τα οποία ανάγονται από την χρήση του DAST, δεν αποτελεί πλήρη μέθοδο αξιολόγησης της ασφάλειας. Η απουσία πρόσβασης στον πηγαίο κώδικα περιορίζει σημαντικά την ορατότητα σχετικά με την εσωτερική αιτία ενός προβλήματος, ενώ ευπάθειες που βρίσκονται σε μη προσβάσιμες διαδρομές ή απαιτούν ιδιαίτερα σύνθετο business context μπορεί να παραμείνουν μη εντοπισμένες. Έτσι, όπως αντίστοιχα και στην χρήση του SAST, προτείνεται η χρήση

web application scanners παράλληλα με static scanning, threat modeling, fuzzing και άλλες τεχνικές software verification και όχι ως μοναδικό μηχανισμό ελέγχου.

Το SAST και το DAST, επομένως, εξετάζουν διαφορετικές οπτικές του ίδιου συστήματος λογισμικού. Το πρώτο παρέχει εσωτερική οπτική της υλοποίησης χωρίς να απαιτεί εκτέλεση, ενώ το δεύτερο παρατηρεί τη συμπεριφορά της πραγματικά εκτελούμενης εφαρμογής μέσω των interfaces που αυτή εκθέτει. Η χρήση συνδυασμού τους επιτρέπει πληρέστερη αξιολόγηση από τη χρήση οποιασδήποτε από τις δύο τεχνικές μεμονωμένα και αποτελεί χαρακτηριστικό παράδειγμα της πολυεπίπεδης προσέγγισης που ακολουθείται στη σύγχρονη ανάπτυξη ασφαλούς λογισμικού.

4.6.2 Software Composition Analysis και Dependency Scanning

Τα σύγχρονα δημιουργημένα λογισμικά, σπάνια αποτελούνται αποκλειστικά από κώδικα που έχει αναπτυχθεί από την ίδια την ομάδα ανάπτυξης. Συνηθίζεται να αξιοποιούν μεγάλο αριθμό από open-source ή third-party libraries, frameworks, packages και άλλα components ή διασυνδεδεμένα συστήματα, τα οποία με τη σειρά τους μπορούν να εξαρτώνται από επιπλέον components. Η επαναχρησιμοποίηση τέτοιου λογισμικού επιταχύνει σημαντικά την ανάπτυξη, δημιουργώντας παράλληλα και μία πρόσθετη διάσταση ασφάλειας. Μία γνωστή vulnerability σε κάποιο εξωτερικό component μπορεί να επηρεάσει την εφαρμογή που το χρησιμοποιεί, ακόμη και όταν η αδυναμία δεν βρίσκεται στον κώδικα που έχει αναπτύξει η ίδια η ομάδα. Για τον λόγο αυτό, η διαχείριση των dependencies αποτελεί σημαντικό μέρος της ασφάλειας του Software Supply Chain. Για τον ορθό έλεγχο, έχουν δημιουργηθεί οι τεχνικές Software Composition Analysis (SCA) και Dependency Scanning.

Το Software Composition Analysis δημιουργεί απογραφή των εξωτερικών components και των εκδόσεών τους, ώστε να είναι δυνατή η συσχέτισή τους με γνωστές ευπάθειες, παρωχημένες εκδόσεις και κινδύνους της εφοδιαστικής αλυσίδας λογισμικού (Chandramouli, Kautz & Torres-Arias, 2024). Πάνω σε αυτή την πληροφορία μπορούν να πραγματοποιηθούν επιπλέον έλεγχοι για γνωστές ευπάθειες, παρωχημένα ή μη υποστηριζόμενα components, άδειες χρήσης και άλλους παράγοντες που σχετίζονται με το software supply chain. Η ακριβής γνώση του component inventory αποτελεί θεμελιώδη προϋπόθεση για την αποτελεσματική αξιολόγηση των σχετικών κινδύνων.

Το Dependency Scanning μπορεί από την άλλη να θεωρηθεί μία περισσότερο εστιασμένη δραστηριότητα μέσα σε αυτό το ευρύτερο πλαίσιο. Ο scanner αναγνωρίζει τις dependencies και τις συγκεκριμένες εκδόσεις που χρησιμοποιούνται από ένα project -μέσω της αξιοποίησης διάφορων package manifests, lockfiles, build configuration ή άλλα σχετικά artifacts συγκρίνοντάς τα με βάσεις γνωστών ευπαθειών-. Μέσω αυτής της τεχνικής μία ομάδα ανάπτυξης μπορεί να ενημερωθεί ότι μία συγκεκριμένη έκδοση βιβλιοθήκης που χρησιμοποιείται από την εφαρμογή σχετίζεται με ήδη γνωστή ευπάθεια και να εξετάσει την αναβάθμιση, αντικατάσταση, τροποποίηση ή διαφορετική αντιμετώπισή της.

Η διάκριση μεταξύ direct και transitive dependencies έχει ιδιαίτερη σημασία εδώ. Direct είναι τα components που δηλώνονται και χρησιμοποιούνται άμεσα από το αναπτυσσόμενο περιβάλλον του project, ενώ transitive είναι εκείνα που εισάγονται έμμεσα επειδή αποτελούν dependencies άλλων components και υπάρχουν στο λεγόμενο dependency tree. Μία εφαρμογή επομένως, μπορεί να επηρεάζεται από vulnerable component χωρίς η ομάδα ανάπτυξης να το έχει προσθέσει και δηλώσει άμεσα. Η παρακολούθηση ολόκληρου του dependency tree κρίνεται σημαντική για την πραγματική κατανόηση της software composition.

Στενά συνδεδεμένη με το SCA είναι η έννοια του Software Bill of Materials (SBOM), το οποίο αποτελεί δομημένη απογραφή των components που περιλαμβάνονται σε ένα λογισμικό και των σχέσεων μεταξύ τους. Μπορεί να περιλαμβάνει στοιχεία όπως όνομα και έκδοση component, software identifiers, dependency relationships, licenses, hashed και πληροφορίες σχετικά με τη δημιουργία ενός SBOM. Η ύπαρξη μιας τέτοιας απογραφής επιτρέπει σε έναν οργανισμό να απαντά ταχύτερα σε ερωτήματα που αφορούν σε συστήματα τα οποία χρησιμοποιούν αυτό το component ή τα components αυτής της εφαρμογής επηρεάζονται από μία νέα CVE.

Το SCA και το Dependency Scanning έχουν ιδιαίτερη αξία όταν εφαρμόζονται συνεχώς και όχι μόνο μία φορά κατά την εισαγωγή ενός dependency. Ένα component το οποίο θεωρείται ασφαλές όταν προστεθεί σε ένα project μπορεί να συσχετιστεί με την ύπαρξη ενός vulnerability στην συνέχεια. Η απογραφή των dependencies, άρα, πρέπει να συνδυάζεται με συνεχή παρακολούθηση ενημερωμένων vulnerability sources και, σε περιπτώσεις όπου καθίσταται δυνατό, να αυτοματοποιείται μέσα σε CI/CD pipelines. Η αναγνώριση, ωστόσο, ότι μία χρησιμοποιούμενη έκδοση ενός dependency συνδέεται με γνωστή vulnerability δεν σημαίνει πάντα ότι η vulnerability είναι πραγματικά exploitable μέσα στη συγκεκριμένη υλοποίηση του συστήματος. Η πρακτική σημασία ενός finding δύναται να εξαρτάται από το εάν χρησιμοποιείται το ευάλωτο τμήμα του component, από το runtime environment και από τα υπάρχοντα security controls.

Οι τεχνικές SAST, SCA, Dependency Scanning και SBOM δεν είναι εξαρτημένες ούτε ταυτιζόμενες, αλλά συμπληρωματικές και ανεξάρτητες μεταξύ τους. Το SAST είναι σε θέση να εντοπίζει security weaknesses στον κώδικα της εφαρμογής, ενώ το SCA και το Dependency Scanning επεκτείνουν την ανάλυση προς το λογισμικό που ενσωματώνεται από τρίτες πηγές. Η προσέγγιση αυτή είναι ιδιαίτερα σημαντική στα σύγχρονα συστήματα, όπου η τελική ασφάλεια εξαρτάται συνδυαστικά από τον κώδικα που δημιουργείται από την ομάδα ανάπτυξης, καθώς και από ολόκληρη τη software supply chain που συμμετέχει στην κατασκευή και λειτουργία του προϊόντος.

4.6.3 Secret, IaC (Infrastructure as Code) και Configuration Scanning

Η ασφάλεια ενός σύγχρονου λογισμικού δεν εξαρτάται αποκλειστικά από τον πηγαίο κώδικά του, αλλά και από τα credentials, τα αρχεία υποδομής και τις ρυθμίσεις του περιβάλλοντος. Τεχνικές όπως το Secret Scanning, Infrastructure as

Code (IaC) Scanning και Configuration Scanning χρησιμοποιούνται συμπληρωματικά προς το SAST, DAST και SCA. Με τον τρόπο αυτό, επεκτείνεται η αυτοματοποιημένη ανάλυση σε διάφορα artifacts και επίπεδα του συστήματος.

Το secret scanning αναζητά credentials ή άλλα ευαίσθητα tokens σε κώδικα, ρυθμίσεις και ιστορικό repository. Εάν ένα πραγματικό μυστικό έχει ήδη δημοσιοποιηθεί, η αφαίρεσή του από την τρέχουσα έκδοση δεν αρκεί· απαιτείται ανάκληση ή αντικατάσταση και έλεγχος του ιστορικού (Chandramouli, Kautz & Torres-Arias, 2024).

Η μέθοδος του Infrastructure as Code (IaC) Scanning εξετάζει αρχεία μέσω των οποίων περιγράφεται προγραμματιστικά ή δηλωτικά η υποδομής, όπως για παράδειγμα Terraform configurations, Kubernetes manifests, Helm charts ή CloudFormation templates. Στόχος καθορίζεται η αναγνώριση security misconfigurations πριν αυτές εφαρμοστούν στο πραγματικό περιβάλλον. Ενδεικτικά ως παράδειγμα, αναφέρονται τα υπερβολικά permissive access rules, μη ασφαλείς network configurations ή storage resources που πρόκειται να δημιουργηθούν χωρίς επαρκή προστασία. Με τον συγκεκριμένο τρόπο, ο έλεγχος μπορεί να πραγματοποιείται ήδη κατά το PR ή το CI/CD pipeline, προτού της φάσης του deployment.

Τα Configuration Scanning είναι ευρύτερης εφαρμογής και αφορούν την αξιολόγηση των ρυθμίσεων εφαρμογών, servers, containers, cloud services και άλλων components σε σχέση με καθορισμένες απαιτήσεις ή secure baselines. Αντίθετα με το IaC Scanning, που εστιάζει κυρίως στα artifacts που περιγράφουν την επιθυμητή υποδομή, το configuration scanning μπορεί να εφαρμοστεί και στην πραγματική κατάσταση ενός ήδη εγκατεστημένου περιβάλλοντος. Με την χρήση του τρόπου αυτού, καθίσταται δυνατός ο εντοπισμός προβλημάτων που ανήκουν στις κατηγορίες των λανθασμένων δοσμένων δικαιωμάτων, περιττές υπηρεσίες, μη ασφαλείς network settings ή αποκλίσεις από την αναμενόμενη ασφαλή ρύθμιση.

4.7 Secure Software Development Lifecycle

Το Secure SDLC ενσωματώνει πρακτικές ασφάλειας σε όλον τον κύκλο ζωής και όχι μόνο πριν από τη διάθεση του προϊόντος. Το NIST SSDF οργανώνει τις πρακτικές σε Prepare the Organization, Protect the Software, Produce Well-Secured Software και Respond to Vulnerabilities (Souppaya, Scarfone & Dodson, 2022). Η ανάγκη δημιουργίας της συγκεκριμένης προσέγγισης έρχεται από την μη λεπτομερή ενσωμάτωση της ασφάλειας και των secure software development practices σε παραδοσιακά SDLC models.

Για ένα πλήρες και αξιόπιστο λογισμικό, κρίνεται επιτακτική ανάγκη η ασφάλεια να εξετάζεται ήδη από τα αρχικά στάδια της ανάπτυξης. Στην φάση του σχεδιασμού και καθορισμού απαιτήσεων, προσδιορίζονται και οι αντίστοιχες απαιτήσεις ασφάλειας, εξετάζονται πιθανές απειλές και λαμβάνονται αρχιτεκτονικές

αποφάσεις με γνώμονα την προστασία του συστήματος. Κατά τη φάση της υλοποίησης, εξετάζονται *secure coding practices*, προστατεύεται ο πηγαίος κώδικας και αξιολογούνται τα *components* και *dependencies* τα οποία έχουν ενσωματωθεί ή συνεχίζουν να ενσωματώνονται στο προϊόν. Κατά το *verification*, δύναται να χρησιμοποιηθούν διαφορετικές τεχνικές, όπως *code review*, *SAST*, *DAST*, *SCA* και άλλοι μηχανισμοί *security testing*, με στόχο τον εντοπισμό αδυναμιών πριν από την εμπορική διάθεση του λογισμικού. Η επίτευξη της διασφάλισης της ποιότητας συνεχίζεται και μετά το *deployment* μέσω παρακολούθησης, *vulnerability management*, ενημερώσεων και αντιμετώπισης νέων προβλημάτων που ανακαλύπτονται στη διάρκεια ζωής του προϊόντος.

Για τον ορισμό των *secure development practices* έχουν δημιουργηθεί αρκετά πλαίσια. Ένα από τα σημαντικότερα σύγχρονα πλαίσια για την εφαρμογή της φιλοσοφίας αυτής είναι το *NIST Secure Software Development Framework (SSDF)*, το οποίο στην τελική του έκδοση *SSDF 1.1* τις οργανώνει σε τέσσερις (4) βασικές ομάδες:

1. **Prepare the Organization (PO)** – προετοιμασία ανθρώπων, διαδικασιών και τεχνολογίας με γνώμονα την συστηματική πραγματοποίηση της ανάπτυξης ασφαλούς λογισμικού.
2. **Protect the Software (PS)** – προστασία του κώδικα και των υπόλοιπων *software components* από μη εξουσιοδοτημένη πρόσβαση και αλλοίωση.
3. **Protect Well-Secured Software (PW)** – σχεδιασμός, ανάπτυξη και επαλήθευση λογισμικού με τελικό στόχο την μέγιστη ελαχιστοποίηση των *vulnerabilities* στις τελικές δημοσιευμένες εκδόσεις.
4. **Respond to Vulnerabilities (RV)** – αναγνώριση και αντιμετώπιση των *vulnerabilities* που παραμένουν ή ανακαλύπτονται μετά τη διάθεση του λογισμικού, καθώς και διερεύνηση των αιτιών τους ώστε να αποφεύγεται η επανεμφάνιση παρόμοιων προβλημάτων.

Η παραπάνω αυτή δομή αναδεικνύει ότι το *Secure SDLC* δεν περιορίζεται στην ανίχνευση των *vulnerabilities*. Επίσης, περιλαμβάνονται και οργανωτικές διαδικασίες, καθορισμός των *security requirements*, προστασία των *development environments* και των *artifacts*, ασφαλής σχεδιασμός και υλοποίηση, επιβεβαίωση και διαχείριση των ευπαθειών μετά την κυκλοφορία. Ιδιαίτερη σημασία έχει η αντιμετώπιση των *root causes* των ευπαθειών, με σκοπό την επίτευξη της διόρθωσης ενός προβλήματος χωρίς να περιορίζεται μόνο στο συγκεκριμένο περιστατικό, αλλά οδηγώντας και σε βελτίωση της διαδικασίας ανάπτυξης που επέτρεψε εκ πρώτης να ενσωματωθεί στον πηγαίο κώδικα. Οι βασικοί στόχοι που θέτονται για το *SSDF* μοντέλο αντίστοιχα από το *NIST* αφορούν τη μείωση των ευπαθειών στα *releases* των συστημάτων, τον περιορισμό των επιπτώσεων από ευπάθειες που δεν έχουν εντοπιστεί και, τέλος, την αποτροπή της επανεμφάνισης προβλημάτων με κοινές αιτίες.

Λογική παρόμοια με το *SSDF*, ακολουθεί και το *OWASP Software Assurance Maturity Model (SAMM)*, κατά το οποίο το *software assurance* οργανώνεται σε πέντε ευρύτερες λειτουργίες, τις *Governance*, *Design*, *Implementation*, *Verification* και *Operations*. Το μοντέλο αυτό υπογραμμίζει ότι η ανάπτυξη ασφαλούς λογισμικού δεν

αποτελεί αποκλειστική ευθύνη ενός security testing stage, αλλά προϋποθέτει δραστηριότητα ασφάλειας από την οργανωτική στρατηγική και τον σχεδιασμό μέχρι και το στάδιο της υλοποίησης, την επαλήθευση και τη λειτουργία του συστήματος. Παράλληλα, το SAMM αντιμετωπίζει την ωριμότητα της secure-development διαδικασίας ως κάτι που μπορεί να αξιολογείται και να βελτιώνεται σταδιακά και συνεχώς.



Εικόνα 4.7.1 – Φιλοσοφία ενός Secure SDLC *

Η ακολουθία της εικόνας 4.7.1 είναι εννοιολογική και όχι ένα υποχρεωτικό γραμμικό μοντέλο, το οποίο απαραίτητα ακολουθείται. Σε iterative και agile περιβάλλοντα ανάπτυξης, τα συγκεκριμένα στάδια επαναλαμβάνονται συνεχώς καθώς το λογισμικό εξελίσσεται. Το ουσιαστικό χαρακτηριστικό είναι ότι η ασφάλεια αντιμετωπίζεται ως διαρκής ιδιότητα της διαδικασίας ανάπτυξης και συντήρησης και όχι ως μία μεμονωμένη δραστηριότητα στο τέλος της διαδικασίας.

Η συγκεκριμένη προσέγγιση είναι ιδιαίτερα σημαντική και για τις μεταβολές του πηγαίου κώδικα. Κάθε καινούρια προτεινόμενη αλλαγή/τροποποίηση είναι ικανή να εισάγει, διορθώσει ή επαναφέρει μία οποιασδήποτε φύσης αδυναμία, γεγονός που καθιστά απαραίτητη την επαναλαμβανόμενη αξιολόγηση της ασφάλειας καθώς το λογισμικό αναπτύσσεται και εξελίσσεται. Οι αυτοματοποιημένες τεχνικές, όπως παρουσιάστηκαν σε προηγούμενες ενότητες της παρούσας βιβλιογραφίας, αποτελούν επιμέρους μηχανισμούς ενός ευρύτερου Secure SDLC και όχι ανεξάρτητες διαδικασίες που μπορούν από μόνες τους να εγγυηθούν την ασφάλεια του τελικού συστήματος.

4.8 DevSecOps και CI/CD pipelines

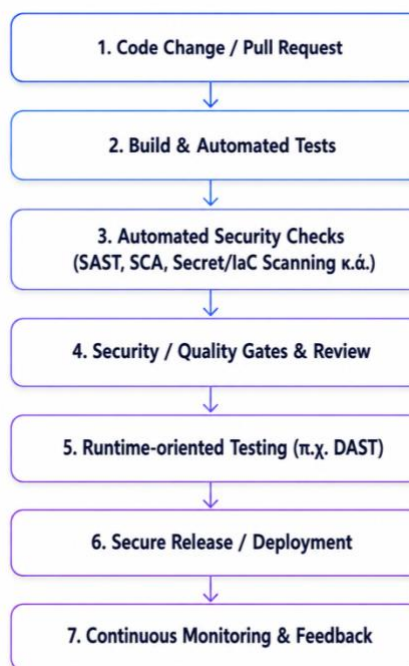
Το DevSecOps ενσωματώνει ελέγχους και ευθύνες ασφάλειας στις επαναληπτικές ροές ανάπτυξης και λειτουργίας. Η ασφάλεια αντιμετωπίζεται ως συνεχής δραστηριότητα και επιδιώκεται η όσο το δυνατόν έγκαιρη ανίχνευση προβλημάτων μέσα στο CI/CD pipeline (Chandramouli, Kautz & Torres-Arias, 2024; OWASP Foundation, χ.χ.). Στόχος του είναι τα προβλήματα ασφάλειας του συστήματος να αναγνωρίζονται, να αξιολογούνται και να αντιμετωπίζονται το δυνατόν πληρέστερα και πλησιέστερα στο σημείο στο οποίο δημιουργούνται, χωρίς

όμως παράλληλα να αναιρείται η ταχύτητα και η επαναληπτικότητα που χαρακτηρίζουν τις σύγχρονες DevOps διαδικασίες.

Κεντρικό ρόλο στην πρακτική εφαρμογή του DevSecOps κατέχουν τα CI/CD pipelines. Όπως ήδη έχει περιγραφεί, πρόκειται για αυτοματοποιημένες διαδικασίες και ροές εργασίας μέσω των οποίων οι αλλαγές στον πηγαίο κώδικα περνούν διαδοχικά από διαδικασίες όπως build, testing, packaging και deployment. Η φιλοσοφία του DevSecOps εμπλουτίζει την ίδια ροή με κατάλληλους security μηχανισμούς συνεχούς αξιολόγησης (review) της ασφάλειας του παραγόμενου λογισμικού, παράλληλα με τους ελέγχους λειτουργικότητας και ποιότητας. Πρακτικά, διαφορετικοί έλεγχοι μπορούν να τοποθετούνται σε διαφορετικά σημεία του pipeline. Κατά τις φάσεις Code και Build, μπορούν, για παράδειγμα, να πραγματοποιούνται Secret Scanning, SAST και SCA, ενώ κατά το στάδιο του Test μπορούν να εφαρμόζονται DAST και άλλες runtime-oriented τεχνικές.

Η προσέγγιση αυτή συνδέεται με την έννοια του shift-left, δηλαδή τη μεταφορά ενός μέρους των security ενεργειών νωρίτερα κατά τη διαδικασία ανάπτυξης. Ένα Security Weakness μπορεί, για παράδειγμα, να εντοπιστεί κατά την ανάλυση ενός Pull Request πριν ενσωματωθεί στην κύρια ροή ανάπτυξης του λογισμικού, ένα vulnerable dependency κατά το στάδιο του build ή μία μη ασφαλής IaC configuration πριν γίνει provisioning. Σημειώνεται ότι, το DevSecOps δεν περιορίζεται αποκλειστικά στην έννοια του shift-left. Επιτακτική ανάγκη κρίνεται ο έλεγχος και διαχείριση της ασφάλειας να μην περιορίζεται και να συνεχίζεται και μετά το deployment μέσω ενεργειών όπως το monitoring, vulnerability management, logging, incident response και νέων ελέγχων καθώς το threat landscape και το ίδιο το λογισμικό μεταβάλλονται συνεχώς και δυναμικά. Ουσιαστικό μέρος της αποτελεί η Continuous Security σε ολόκληρο το lifecycle παρά η απλή μεταφορά όλων των ελέγχων στα πρώτα στάδια της ανάπτυξης.

Τα αποτελέσματα τα οποία εξάγονται μέσω των αυτοματοποιημένων ελέγχων ασφαλείας μπορούν επίσης να χρησιμοποιηθούν ως quality ή security gates στη συνολική δραστηριότητα του pipeline. Σε περιπτώσεις όπου μία αλλαγή παραβιάσει προκαθορισμένα κριτήρια, η διαδικασία μπορεί να αποτύχει, να εμποδιστεί προσωρινά το merge/deployment ή να απαιτηθεί ανθρώπινη παρέμβαση και αξιολόγηση. Η αυτοματοποίηση αυτή παρέχει άμεσο feedback στους προγραμματιστές και συμβάλλει άμεσα στην επαναληψιμότητα των security controls. Δεν πρέπει όμως να θεωρείται ότι η επιτυχής ολοκλήρωση όλων των automated checks αποδεικνύει άμεσα και απόλυτα την ασφάλεια του συστήματος, καθώς κάθε τεχνική έχει διαφορετικό πεδίο κάλυψης και ορισμένα προβλήματα απαιτούν contextual ή χειροκίνητο assessment.



Σημαντική είναι η μη παραμέληση της προστασίας και του ίδιου του CI/CD pipeline. Τα repositories, dependencies, credentials, generated artifacts και deployment mechanisms αποτελούν αναπόσπαστα τμήματα της Software Supply Chain και μπορούν τα ίδια να αποτελέσουν στόχους επίθεσης. Για τον λόγο αυτό απαιτούνται έλεγχοι πρόσβασης, προστασία secrets, επαλήθευση της ακεραιότητας των artifacts, διατήρηση provenance και περιορισμός της δυνατότητας μη εξουσιοδοτημένης τροποποίησης των διαδικασιών build και deployment. Το NIST SP 800-204D εστιάζει στην διασφάλιση της ακεραιότητας των CI/CD δραστηριοτήτων, αναδεικνύοντας έτσι ότι η ασφάλεια ενός pipeline αφορά τόσο το προϊόν που ελέγχεται όσο και τον ίδιο τον μηχανισμό που το κατασκευάζει και το διανέμει στους χρήστες του προϊόντος αυτού.

Εικόνα 4.8.1 – Εννοιολογική ακολουθία διασφάλισης ασφάλειας σε DevSecOps και CI/CD pipelines *

Το DevSecOps παρουσιάζει ιδιαίτερη σημασία και ενδιαφέρον και στο πλαίσιο της παρούσας εργασίας, καθώς τα Pull Requests αποτελούν φυσικό σημείο εφαρμογής αυτοματοποιημένων ελέγχων ποιότητας και ασφάλειας πριν από την ενσωμάτωση μίας αλλαγής στην κύρια ροή ανάπτυξης του συστήματος. Η επαναλαμβανόμενη ανάλυση διαφορετικών καταστάσεων του κώδικα επιτρέπει την παρατήρηση του τρόπου με τον οποίο οι αλλαγές μπορούν να μεταβάλλουν τόσο quality όσο και security indicators. Η φιλοσοφία του DevSecOps και των διαδικασιών CI/CD παρέχει το πρακτικό συνολικό πλαίσιο μέσα στο οποίο τέτοιες μετρικές μπορούν να αξιοποιηθούν ως συνεχής ανατροφοδότηση κατά την εξέλιξη του λογισμικού.

Αλληλεπίδραση συνιστωσών Ποιότητας και Ασφάλειας πηγαίου κώδικα και λογισμικού

Η ποιότητα και η ασφάλεια του λογισμικού, αν και αποτελούν διακριτές διαστάσεις και μπορούν να εξεταστούν ανεξάρτητα, συνδέονται με διαφορετικές πτυχές της σχεδίασης, της υλοποίησης και της εξέλιξης ενός συστήματος. Οι μεταβολές που πραγματοποιούνται στον πηγαίο κώδικα κατά τη διάρκεια της ανάπτυξης μπορούν να επηρεάζουν ταυτόχρονα περισσότερα από ένα χαρακτηριστικά του λογισμικού, καθιστώντας σημαντική τη διερεύνηση των πιθανών σχέσεων μεταξύ των συνιστωσών ποιότητας και ασφάλειας.

Στο παρόν κεφάλαιο εξετάζεται ενδελεχώς η θεωρητική και εμπειρική βιβλιογραφία σχετικά με την αλληλεπίδραση, όπως αυτή περιεγράφηκε. Παρουσιάζονται οι κυριότερες συνδέσεις που έχουν μελετηθεί μεταξύ επιμέρους χαρακτηριστικών ποιότητας και ασφάλειας, οι περιπτώσεις στις οποίες οι μεταβολές μπορούν να εμφανίζουν συνέργεια ή αντικρουόμενη μεταξύ τους συμπεριφορά, καθώς και οι παράγοντες που πρέπει να λαμβάνονται υπόψη κατά την ερμηνεία αυτών των σχέσεων. Τέλος, συντίθενται τα ευρήματα της υπάρχουσας βιβλιογραφίας και διαμορφώνεται το θεωρητικό πλαίσιο πάνω στο οποίο βασίζεται η εμπειρική διερεύνηση της παρούσας εργασίας.

5.1 Θεωρητική σχέση ποιότητας και ασφάλειας

Η security περιλαμβάνεται στο ISO/IEC 25010:2023 ως ένα από τα χαρακτηριστικά της ποιότητας προϊόντος, χωρίς αυτό να σημαίνει ότι ταυτίζεται με τη συνολική ποιότητα. Η διάκριση είναι κρίσιμη, επειδή οι δύο διαστάσεις μπορούν να σχετίζονται αλλά μετρώνται μέσω διαφορετικών constructs (ISO/IEC, 2023).

Η μεταξύ τους σύνδεση μπορεί να γίνει περισσότερο εμφανής σε περίπτωση όπου συμβαίνει η εξέταση της κοινής προέλευσης ορισμένων προβλημάτων λογισμικού. Ένα defect στον σχεδιασμό ή στην υλοποίηση είναι ικανό να επηρεάζει μόνο τη λειτουργικότητα ή την αξιοπιστία του λογισμικού, αλλά υπό συγκεκριμένες συνθήκες μπορεί παράλληλα να δημιουργεί δυνατότητα exploitation με συνέπειες στην ασφάλειά του. Λόγω αυτών, πρακτικές οι οποίες μειώνουν γενικότερα την εισαγωγή σφαλμάτων και ενισχύουν τη συστηματική επιβεβαίωση πληρότητας, σε ορισμένες περιπτώσεις ωφελούν τόσο την ποιότητα όσο και την ασφάλεια. Σημειώνεται ότι κάθε υπαρκτό quality defect, security weakness όπως επίσης κάθε ευπάθεια ασφάλειας δεν είναι δυνατόν να αντιμετωπιστεί αποκλειστικά μέσω γενικών πρακτικών βελτίωσης της ποιότητας ενός λογισμικού.

Οι δύο αυτές διαστάσεις, ταυτόχρονα, επηρεάζονται από κοινές τεχνικές και αρχιτεκτονικές αποφάσεις. Η βιβλιογραφία των software quality attributes αναφέρει

και αναγνωρίζει ότι χαρακτηριστικά όπως είναι τα security, maintainability, reliability, performance και availability μπορούν να αλληλοεπιδρούν, να αλληλοενισχύονται ή να δημιουργούν trade-offs. Επομένως, βάσει των προηγούμενων, μία τροποποίηση στον πηγαίο κώδικα η οποία συμβαίνει με στόχο την βελτίωση μία τεχνικής ιδιότητας μπορεί να έχει θετική, αρνητική ή ακόμα και ουδέτερη επίδραση σε κάποια άλλη. Η προσθήκη security mechanisms μπορεί να επηρεάσει την πολυπλοκότητα, τη συντηρησιμότητα ή άλλες ιδιότητες του πηγαίου κώδικα ή/και του λογισμικού αντίστοιχα.

Εν κατακλείδι, η σχέση μεταξύ ποιότητας και ασφάλειας πρέπει να αντιμετωπίζεται ως πολυδιάστατη και context-dependent αλληλεπίδραση και όχι ως απλή αιτιακή σχέση όπου η βελτίωση της μίας πρόκειται απαραίτητα να οδηγήσει στη βελτίωση και της άλλης. Οι δύο διαστάσεις μπορούν να επηρεάζονται από τις ίδιες μεταβολές του πηγαίου κώδικα, χωρίς όμως να μεταβάλλονται πάντοτε προς την ίδια κατεύθυνση (θετική → θετική, αρνητική → αρνητική). Η θεωρητική θέση όπως αυτή παρουσιάζεται αποτελεί τη βάση για την εξέταση των επιμέρους μηχανισμών και χαρακτηριστικών μέσω των οποίων η ποιότητα και η ασφάλεια μπορούν να αλληλοεπιδρούν στις επόμενες ενότητες.

5.2 Κοινές πηγές Quality και Security Defects

Κοινές αποφάσεις, λάθη και συνθήκες κατά τη διάρκεια του SDLC, μπορούν να οδηγήσουν σε προβλήματα ποιότητας ή ασφάλειας ή συνδυασμού τους. Ανεπαρκείς απαιτήσεις, προβληματικές σχεδιαστικές αποφάσεις, σφάλματα κατά την υλοποίηση, σύνθετες μεταβολές του κώδικα ή ελλιπής διαδικασία επιβεβαίωσης ανά τα στάδια ανάπτυξης, μπορούν να επηρεάσουν διαφορετικές ιδιότητες του ίδιου του συστήματος. Η τελική συνέπεια ενός defect εξαρτάται από τη φύση και το περιβάλλον του. Ένα πρόβλημα μπορεί να επηρεάζει μόνο τη λειτουργικότητα ή τη maintainability, ενώ σε άλλες περιπτώσεις η ίδια ή παρόμοια αστοχία μπορεί να δημιουργεί συνθήκες που είναι δυνατό να αξιοποιηθούν από έναν επιτιθέμενο. Ιδιαίτερη έμφαση δίνεται παράλληλα και στην αναγνώριση των root causes που επιτρέπουν την εμφάνιση ευπαθειών στο περιβάλλον του συστήματος.

Μία πρώτη πηγή εντοπίζεται στις απαιτήσεις και στον σχεδιασμό. Ελλιπείς ή λανθασμένες απαιτήσεις οδηγούν σε μη ικανοποίηση των πραγματικών αναγκών του συστήματος, αλλά και στην απουσία απαραίτητων για το σύστημα, ελέγχων ασφάλειας. Μία ακατάλληλη αρχιτεκτονική απόφαση, επίσης, μπορεί να επηρεάσει ταυτόχρονα χαρακτηριστικά όπως maintainability, reliability και security.

Μία δεύτερη σημαντική πηγή βρίσκεται στην ίδια την υλοποίηση του λογισμικού. Προγραμματιστικά λάθη, ανεπαρκής έλεγχος δεδομένων, λανθασμένη διαχείριση πόρων ή μη ορθός χειρισμός των exceptional conditions μπορούν αρχικά να εμφανιστούν ως γενικά software defects, αλλά ορισμένα από αυτά αποκτούν security significance όταν μπορούν να αξιοποιηθούν κακόβουλα. Διακρίνονται δύο

σημαντικές πηγές προβλημάτων software security, σφάλματα κατά την υλοποίηση και ευπάθειες ασφάλειας.

Οι συχνές και πολύπλοκες μεταβολές του κώδικα αποτελούν, επίσης, παράγοντα που μπορεί να συνδέεται με την εμφάνιση και των δύο τύπων προβλημάτων. Όσο ο κώδικας μεταβάλλεται, οι προγραμματιστές χρειάζεται να κατανοούν υπάρχουσες αλληλεξαρτήσεις και να διατηρούν προηγούμενες λειτουργικές και security assumptions. Η εισαγωγή μίας ή πολλών αλλαγών ή η πολυπλοκότητα, δεν προκαλεί αυτομάτως ευπάθειες, αλλά υποδεικνύουν ότι τα χαρακτηριστικά που παραδοσιακά ενδιαφέρουν την ποιότητα του λογισμικού μπορούν να είναι άμεσα σχετικά και για την ανάλυση της ασφάλειας του λογισμικού.

Κοινή πηγή προβλημάτων και των δύο συνιστωσών μπορεί επίσης να αποτελέσει και η ανεπαρκής διαδικασία verification και review. Η αποτελεσματικότητα του secure code review εξαρτάται από την εμπειρία των reviewers, τη γνώση του codebase και το πλαίσιο της αλλαγής. Επομένως, τα review artifacts μπορούν να παρέχουν χρήσιμα security signals, αλλά δεν πρέπει να αντιμετωπίζονται ως πλήρης εγγύηση ασφάλειας (Thompson & Wagner, 2017; Alfadel et al., 2023).

Η διαδικασία της σύγχρονης ανάπτυξης λογισμικού, όπως έχει περιγραφεί σε προηγούμενες ενότητες, εισάγει πρόσθετες κοινές πηγές προβλημάτων μέσω third-party components, configurations και της ευρύτερης software supply chain. Μία χρησιμοποιούμενη dependency η οποία είναι παρωχημένη ή εσφαλμένη είναι ικανή να προκαλέσει λειτουργικές αστοχίες και ταυτόχρονα να εισάγει γνωστές ευπάθειες, ενώ μία λανθασμένη configuration μπορεί να επηρεάζει τόσο την ορθή λειτουργία όσο και την δημόσια έκθεση του συστήματος. Λόγω των παραπάνω, τα σύγχρονα secure development frameworks δεν περιορίζονται στην εξέταση και ανάλυση του πηγαίου κώδικα, αλλά επιπλέον περιλαμβάνουν διαδικασίες για software components, development environments, configurations και released artifacts.

Η ύπαρξη κοινών πηγών δεν σημαίνει ότι Quality Defects και Security Defects είναι μεταξύ τους ισοδύναμα ή ότι απαραίτητα εμφανίζονται ως συνδυασμός. Αντίθετα, υποδηλώνει ότι οι δύο αυτές διαστάσεις μπορούν να επηρεάζονται από κοινές συνθήκες ανάπτυξης και από τις ίδιες τις μεταβολές του πηγαίου κώδικα. Η διάκριση αυτή αποτελεί ουσιώδες χαρακτηριστικό. Ένας κοινός παράγοντας μπορεί να σχετίζεται με quality και security outcomes παράλληλα, χωρίς να αποδεικνύεται ότι η μία διάσταση προκαλεί την άλλη.

5.3 Πολυπλοκότητα, Συντηρησιμότητα και Security Defects

Στενά συνδεδεμένα χαρακτηριστικά της εσωτερικής κατάστασης του λογισμικού, αποτελούν η Πολυπλοκότητα (Complexity) και η Συντηρησιμότητα (Maintainability). Η αύξηση των διακλαδώσεων, των εξαρτήσεων και των διαφορετικών πιθανών ροών ενός λογισμικού, μπορεί να απαιτεί περισσότερη γνωστική προσπάθεια για την κατανόηση, τον έλεγχο και την ασφαλή τροποποίησή

του. Η ύπαρξη πολυπλοκότητας δεν συνεπάγεται και δεν αποτελεί αυτομάτως vulnerability ούτε αυτόνομο χαρακτηριστικό ασφάλειας, μπορεί όμως να δημιουργεί τις προϋποθέσεις για την ευκολότερη εμφάνιση σφαλμάτων καθιστώντας παράλληλα δυσκολότερη την πρόβλεψη συνεπειών. Αντίστοιχα, η Maintainability αποτελεί διακριτό quality characteristic στο πλαίσιο του προτύπου ISO/IEC 25010:2023 και σχετίζεται με ιδιότητες όπως η αναλυσιμότητα, η modifiability και η testability, οι οποίες είναι σημαντικές και για την αποτελεσματική εξέταση security-sensitive κώδικα.

Η σύνδεση της πολυπλοκότητας με την ασφάλεια έχει εξεταστεί εμπειρικά μέσω vulnerability prediction models. Αποτελέσματα έδειξαν ότι μετρικές πολυπλοκότητας σε συνδυασμό με code churn και developer-activity metrics, περιέχουν χρήσιμη πληροφορία για την αναγνώριση vulnerability-prone code. Η προαναφερθείσα σχέση ανήκει στο στατιστικό και προβλεπτικό φάσμα, χωρίς να επιβεβαιώνει ότι η αύξηση μιας μετρικής πολυπλοκότητας συνεπάγεται άμεσα την αυτόματη πρόκληση μίας vulnerability. Εργασίες έχουν εξετάσει τη δυνατότητα χρήσης πολυπλοκότητας και άλλως μετρικών πηγαίου κώδικα για την πρόβλεψη διαφόρων vulnerabilities, με αποτελέσματα που υποστηρίζουν τη χρησιμότητά τους αλλά ταυτόχρονα αναδεικνύουν τη σημαντική εξάρτηση από το project, το dataset και το χρησιμοποιούμενο μοντέλο. Καθολικά, δεν έχει ακόμα διαμορφωθεί ένα ενιαίο σύνολο μετρικών προς εφαρμογή για την διασφάλιση ενός ορθού vulnerability prediction. Συμπερασματικά, η πολυπλοκότητα είναι προτιμότερο να αντιμετωπίζεται ως ένας πιθανός δείκτης και όχι ως γενικός και αυτόνομος δείκτης ασφάλειας.

Η vulnerability proneness είναι πολυπαραγοντικό φαινόμενο. Εκτός από τη στατική πολυπλοκότητα, μετρικές όπως code churn, developer activity και συνεργατικές σχέσεις μπορεί να προσφέρουν πρόσθετη πληροφορία, χωρίς να αποτελούν από μόνες τους αιτιώδεις δείκτες ευπάθειας (Shin et al., 2011; Meneely & Williams, 2009). Οι Shin et al. (2011), για παράδειγμα, εξέτασαν από κοινού complexity, code churn και developer-activity metrics ως δείκτες vulnerability-prone κώδικα, ενώ οι Meneely και Williams (2009) έδειξαν ότι χαρακτηριστικά της συνεργασίας μεταξύ προγραμματιστών μπορούν επίσης να παρουσιάζουν σχέση με την παρουσία γνωστών ευπαθειών. Η δομή των dependencies, αντίστοιχα, έχει αποδειχθεί εξίσου χρήσιμη στη συγγενή περιοχή του defect prediction, όπου network-based measures μπορούν να προσφέρουν πληροφορία που δεν αποτυπώνεται πλήρως από τις παραδοσιακές complexity metrics (Zimmermann & Nagappan, 2008). Τα παραπάνω δεν σημαίνουν ότι κάθε τέτοια μεταβλητή αποτελεί ανεξάρτητο ή αιτιώδη predictor ασφάλειας. Εν αντιθέσει, δείχνουν ότι η vulnerability prediction είναι πολυπαραγοντικό πρόβλημα καθώς επίσης και ότι η πολυπλοκότητα πρέπει να ερμηνεύεται ως μία μόνο διάσταση ενός ευρύτερου τεχνικού και κοινωνικοτεχνικού πλαισίου.

Η θεωρητική σχέση της ασφάλειας με την συντηρησιμότητα προκύπτει μέσω της δυνατότητας ενός συστήματος να κατανοείται, να αναλύεται και να μεταβάλλεται με ελεγχόμενο τρόπο. Κώδικας ο οποίος έχει μεγαλύτερο επίπεδο δυσκολίας κατανόησης και τροποποίησης μπορεί να αυξάνει τη δυσκολία της διαδικασίας του code review, της διερεύνησης μιας security-sensitive ροής και της ασφαλούς

εφαρμογής μιας διόρθωσης. Υψηλή maintainability δεν εγγυάται την απουσία vulnerabilities, όπως και χαμηλή maintainability δεν αποτελεί από μόνη της security defect. Πρόκειται για διαφορετικές ιδιότητες που μπορούν να επηρεάζονται από κοινά χαρακτηριστικά της υλοποίησης και να αλληλοεπιδρούν κατά την εξέλιξη του λογισμικού.

Η σχέση δύναται να λειτουργεί και προς την αντίθετη κατεύθυνση. Η διόρθωση μίας vulnerability ενδέχεται να εισάγει νέο πηγαίο κώδικα και να αυξήσει την πολυπλοκότητα ή να επιβαρύνει άλλα χαρακτηριστικά συντηρησιμότητας όταν η απαιτούμενη προστασία εισάγει επιπλέον ελέγχους, conditions ή structural changes.

Η σχέση μεταξύ Complexity, Maintainability και Security Defects πρέπει, επομένως, να θεωρείται πολυπαραγοντική και άμεσα εξαρτώμενη από το context στο οποίο ορίζεται. Η αυξημένη πολυπλοκότητα μπορεί να συνδέεται με μεγαλύτερη δυσκολία κατανόησης και ελέγχου του κώδικα και να παρουσιάζει στατιστική σχέση με vulnerability-prone περιοχές, χωρίς να αποτελεί από μόνη της ευπάθεια ασφάλειας. Αντίστοιχα, η βελτίωση της maintainability μπορεί να διευκολύνει την ασφαλέστερη εξέλιξη και verification του κώδικα, αλλά δεν αποτελεί εγγύηση ασφάλειας. Στην παρούσα εργασία, για τον λόγο αυτό, οι μεταβολές των complexity και maintainability indicators και οι μεταβολές των security findings εξετάζονται ως διακριτές αλλά δυνητικά σχετιζόμενες μεταβλητές, χωρίς να προϋποτίθεται αιτιακή σχέση μεταξύ τους.

5.4 Code Smells, Duplication και Vulnerabilities

Τα code smells δεν είναι από μόνα τους vulnerabilities, αλλά μπορούν να χαρακτηρίζουν περιοχές όπου η κατανόηση και η ασφαλής τροποποίηση είναι δυσκολότερες. Κάθε σύνδεση με fault- ή vulnerability-proneness πρέπει να παρουσιάζεται ως εμπειρική συσχέτιση και να συνοδεύεται από την αντίστοιχη πρωτογενή μελέτη (Yamashita & Moonen, 2013). Μεγάλης κλίμακας εμπειρική μελέτη των Palomba et al., κατά την οποία εξετάστηκε ένα σύνολο δειγμάτων 17.350 επιβεβαιωμένων code smells σε 395 διαφορετικά releases ανάμεσα σε 30 ανοιχτού κώδικα συστήματα, έδειξε ότι classes που περιείχαν smells παρουσίαζαν υψηλότερη change/fault-proneness από classes χωρίς smells. Η συγκεκριμένη σχέση αφορά πρωτίστως τη software quality και δεν συνεπάγεται ότι ένα smelly component είναι αναγκαστικά ευάλωτο σε επιθέσεις.

Η πιθανή σύνδεση με την ασφάλεια προκύπτει όταν τα χαρακτηριστικά που εκφράζει ένα smell δυσκολεύουν την κατανόηση ή τον σωστό χειρισμό security-sensitive συμπεριφοράς. Πολύπλοκες αρμοδιότητες, ακατάλληλες εξαρτήσεις μεταξύ components ή προβληματική κατανομή λειτουργικότητας μπορούν να καταστήσουν δυσκολότερο το reasoning σχετικά με data flows, access control και άλλες security-relevant λειτουργίες. Η σχέση μεταξύ code smells και vulnerabilities έχει αποδειχθεί μέσω διάφορων εμπειρικών ερευνών. Τα ευρήματα τέτοιων

ερευνών υποστηρίζουν την ύπαρξη ανθρώπινης σύνδεσης, όχι όμως μία γενική σχέση σύμφωνα με την οποία περισσότερα code smells συνεπάγονται αυτομάτως περισσότερες vulnerabilities.

Το Code Duplication, δηλαδή η ύπαρξη πανομοιότυπων ή παρόμοιων τμημάτων κώδικα σε περισσότερα από ένα σημεία ενός συστήματος, αποτελεί μία ιδιαίτερη περίπτωση. Η πλευρά της ποιότητας, υποστηρίζει ότι η ύπαρξη duplication στον κώδικα είναι ικανή να αυξήσει το maintenance burden επειδή μία αλλαγή που αφορά μία κοινή υλοποιημένη λογική ενδέχεται να χρειάζεται να εφαρμοστεί σε περισσότερα από ένα αντίγραφα. Σε περίπτωση όπου τα αντίγραφα του κώδικα μεταβληθούν ασυνεπώς, είναι δυνατό να δημιουργηθούν ή να παραμείνουν defects. Η βιβλιογραφία, παρ' όλα αυτά, δείχνει ότι η επίδραση της duplication δεν είναι πάντα αρνητική και εξαρτάται σημαντικά από τον τρόπο με τον οποίο τα αντίγραφα δημιουργούνται και συντηρούνται. Η άποψη αυτή, έχει επιστημονικά εμπεριστατωθεί από μελέτες σε release level οι οποίες έχουν δείξει ότι μόνο ένα σχετικά μικρό τμήμα των clone genealogies οδήγησε σε defects, γεγονός το οποίο αποτρέπει μία απλουστευμένη αντιμετώπιση κάθε duplication ως ένα ακόμα πρόβλημα ποιότητας.

Η σύνδεση του διπλότυπου κώδικα με την συνιστώσα της ασφάλειας αποκτά ιδιαίτερη σημασία σε περιπτώσεις όπου ο κώδικας που αντιγράφεται περιέχει ήδη μία vulnerability. Το cloning, εδώ, μπορεί να μεταφέρει την ίδια ευπάθεια υλοποίησης σε επιπλέον θέσεις μέσα στο ίδιο ή σε διαφορετικά συστήματα. Οι Kim και Lee μέσω της μελέτης πραγματικών συστημάτων ανοιχτού κώδικα με την χρήση της τεχνικής VUDDY, εντόπισαν περιπτώσεις όπου γνωστές ευπάθειες είχαν διαδοθεί μέσω code clones και παρέμεναν χωρίς patch για σημαντικά χρονικά διαστήματα. Παράλληλα, η πλήρης remediation μπορεί να δυσκολευτεί από την ύπαρξη περισσότερων vulnerable copies, καθώς η διόρθωση ενός μόνο clone δεν συνεπάγεται αυτόματα την διόρθωση όλων των αντιγράφων του.

Τα Code Smells και το Duplication, επομένως, πρέπει να αντιμετωπίζονται ως πιθανοί παράγοντες ή indicators που σχετίζονται με το περιβάλλον δημιουργίας, διατήρησης και διάδοσης defects και όχι ως άμεσες μετρήσεις της ασφάλειας. Τα smells, όπως ήδη έχει αναφερθεί, μπορούν να χαρακτηρίζουν, δυσκολότερο στην κατανόηση και συντήρηση, τον πηγαίο κώδικα, σε αντίθεση με το διπλότυπο κώδικα ο οποίος είναι ικανός να πολλαπλασιάσει ένα ήδη υπάρχον πρόβλημα ή να δυσκολέψει τη συνεπή διόρθωσή του. Βέβαια, η ύπαρξη ενός smell ή duplicated fragment δεν αποδεικνύει αυτομάτως την ύπαρξη ενός επιβεβαιωμένου vulnerability. Οι αντίστοιχες quality και security μετρικές, επομένως, είναι καταλληλότερο να εξετάζονται ως διακριτές αλλά δυνητικά συσχετιζόμενες μεταξύ τους διαστάσεις.

5.5 Reliability, Error handling και Security

Οι έννοιες Reliability και Security αποτελούν διακριτές και ανεξάρτητες μεταξύ τους ιδιότητες ενός λογισμικού, παρουσιάζοντας παράλληλα σημαντικά σημεία αλληλεπίδρασης. Η reliability σχετίζεται με την ικανότητα ενός συστήματος να εκτελεί τις προβλεπόμενες λειτουργίες του χωρίς αποτυχία υπό καθορισμένες συνθήκες, ενώ η security εξετάζει επιπλέον τη συμπεριφορά του απέναντι σε κακόβουλες ενέργειες και μη εξουσιοδοτημένη χρήση. Η σχέση τους αντικατοπτρίζεται εμφανώς στην Availability (Διαθεσιμότητα), η οποία αποτελεί ταυτόχρονα βασική απαίτηση dependability και θεμελιώδη στόχο ασφάλειας. Αντίστοιχα, η reliability, availability και maintainability έχουν τοποθετηθεί από τους Avizienis et al. (2004) στο γενικότερο πλαίσιο της dependability, ενώ η security μοιράζεται με αυτήν ιδιότητες όπως availability και integrity.

Μία αποτυχία λογισμικού δεν αποτελεί απαραίτητα ή και εξ ορισμού security vulnerability. Λαμβάνοντας ως παράδειγμα μία μη αναμενόμενη διακοπή λειτουργίας, μπορούμε να διαπιστώσουμε ότι αποτελεί αποκλειστικά πρόβλημα reliability χωρίς επιπτώσεις στην ασφάλεια του λογισμικού. Σε περίπτωση, όμως, όπου ένας χρήστης μπορεί να προκαλέσει επαναλαμβανόμενα αυτή την αποτυχία μέσω ειδικά διαμορφωμένης εισόδου, η ίδια συμπεριφορά μπορεί να αποκτήσει security significance, όπως μέσω Denial of Service. Errors τα οποία οδηγούν το εκτελέσιμο σύστημα σε μη προβλέψιμη ή ασυνεπή κατάσταση μπορούν να επηρεάσουν όχι μόνο τη σωστή λειτουργία αλλά και τις αρχές του CIA Triad αντίστοιχα. Η MITRE καταγράφει για τον λόγο αυτό την ανεπαρκή διαχείριση exceptional conditions ως κατηγορία αδυναμίας με πιθανές συνέπειες τις crashes, unexpected states και έκθεση δεδομένων σε οντότητες χωρίς εξουσιοδότηση.

Η φιλοσοφία του Error Handling αποτελεί έναν ιδιαίτερα σημαντικό μηχανισμό σύνδεσης. Ένα σύστημα, για να είναι ολοκληρωμένο, είναι αναγκαίο να είναι σε θέση να αναγνωρίζει μη αναμενόμενες καταστάσεις, να αντιδρά με προβλέψιμο τρόπο και να επιστρέφει σε ασφαλή κατάσταση όταν η κανονική λειτουργία δεν μπορεί να συνεχιστεί. Η ανεπαρκής διαχείριση των διάφορων κατηγοριών exceptions που μπορούν να εμφανιστούν, δύναται να οδηγήσει σε crashes και λειτουργικές αστοχίες καθώς και να δημιουργήσει vulnerabilities. Χαρακτηριστική περίπτωση αποτελεί το failing open. Όταν μία λειτουργία ασφαλείας αποτυγχάνει, η εφαρμογή δεν πρέπει να συνεχίζει τη λειτουργία της παρέχοντας πρόσβαση περισσότερη από την προβλεπόμενη και από όση επιτρέπεται. Για παράδειγμα, μία αποτυχία κατά την εκτέλεση του μηχανισμού του ελέγχου authorization δεν θα πρέπει να έχει ως αποτέλεσμα την παράκαμψη οποιουδήποτε ελέγχου. Παρομοίως, η μη ορθή επαναφορά μιας συναλλαγής μετά από failure μπορεί να αφήσει το σύστημα σε ασυνεπή κατάσταση, δημιουργώντας τόσο reliability όσο και integrity προβλήματα.

Μία δεύτερη, εξίσου σημαντική, διάσταση αφορά το περιεχόμενο των παρεχόμενων από το σύστημα error messages. Αναλυτικές πληροφορίες του εμφανιζόμενου σφάλματος, όπως είναι stack traces, database queries, internal paths ή στοιχεία configuration κρίνονται χρήσιμες για εσωτερική αποσφαλμάτωση από την ομάδα ανάπτυξης και monitoring του συστήματος, αλλά μπορούν να αποτελέσουν πηγή ευπάθειας σε περιπτώσεις όπου επιστραφούν σε μη εξουσιοδοτημένους χρήστες, καθώς μπορούν να αποκαλύψουν ευαίσθητες

πληροφορίες. Προτείνεται οι εξωτερικές αποκρίσεις να είναι γενικές και οι απαραίτητες τεχνικές λεπτομέρειες να καταγράφονται λεπτομερώς εσωτερικά για ενδεχόμενο troubleshooting και monitoring.

Βάσει των παραπάνω, μπορεί να γίνει αντιληπτό ότι η σχέση Reliability-Security δεν περιορίζεται στην κοινή επιδίωξη της διαθεσιμότητας. Αφορά επίσης, και το κατά πόσο το σύστημα μπορεί να παραμένει σε προβλέψιμη και ασφαλή κατάσταση όταν κάτι μπορεί να πάει στραβά. Η μείωση γενικότερων software defects μπορεί να περιορίζει ένα μέρος των vulnerabilities, χωρίς όμως reliability και security defects να ταυτίζονται μεταξύ τους. Η reliability και το error handling μπορούν να επηρεάζουν σημαντικά την ασφάλεια, η σχέση όμως πρέπει να εξετάζεται ως πιθανή αλληλεπίδραση και όχι ως υπόθεση ότι κάθε πρόβλημα reliability αποτελεί αυτομάτως και security problem.

5.6 Refactoring και Security

Η διαδικασία του Refactoring (Αναδιάρθρωση) αποτελεί διαδικασία τροποποίησης της εσωτερικής δομής του πηγαίου κώδικα με στόχο τη βελτίωση διαφόρων χαρακτηριστικών του πηγαίου κώδικα, όπως είναι η αναγνωσιμότητα, η συντηρησιμότητα και η οργάνωσή του, αποφεύγοντας οποιαδήποτε μεταβολή της εξωτερικά παρατηρήσιμης λειτουργικότητας του συστήματος. Αν και ο βασικός του στόχος σχετίζεται κυρίως με την διάσταση της ποιότητας του πηγαίου κώδικα, οι μεταβολές που πραγματοποιούνται κατά τη διαδικασία αυτή μπορούν να επηρεάσουν και χαρακτηριστικά τα οποία πιθανώς να σχετίζονται με την ασφάλεια.

Η συγκεκριμένη επίδραση δεν είναι μονοσήμαντη αναγκαστικά. Ένα refactoring μπορεί να συμβάλει στη βελτίωση της ασφάλειας, για παράδειγμα διευκολύνοντας την κατανόηση και τον έλεγχο security-sensitive κώδικα, αλλά μπορεί επίσης να εισαγάγει νέα προβλήματα ή να μεταβάλλει υπάρχουσες security assumptions. Για τον λόγο αυτό, η σχέση μεταξύ των refactoring και security εννοιών εξετάζεται στη συνέχεια από δύο συμπληρωματικές πλευρές. Η πρώτη εξ αυτών είναι οι πιθανές βελτιώσεις που μπορεί να προκύψουν και η δεύτερη, και τελευταία, οι πιθανές security regressions που ενδέχεται να εισαχθούν κατά την τροποποίηση του κώδικα.

5.6.1 Πιθανές βελτιώσεις

Το Refactoring πραγματοποιείται από μία ομάδα ανάπτυξης κατά κύριο λόγο για τη βελτίωση της εσωτερικής ποιότητας του κώδικα. Ορισμένες από τις μεταβολές που προκαλεί όμως, μπορούν να έχουν παράλληλα θετικές συνέπειες για την ασφάλεια. Οι ποικίλες προσφορές του, η απλοποίηση της δομής, η καλύτερη κατανομή των ευθυνών και ο περιορισμός προβληματικών εξαρτήσεων μπορούν να

καταστήσουν τον κώδικα ευκολότερο στην κατανόηση και στον έλεγχο. Ιδιαίτερη σημασία αποκτάει σε security-sensitive τμήματα, όπου η δυνατότητα κατανόησης των data flows, των access restrictions και των σχέσεων μεταξύ των components μπορεί να διευκολύνει τόσο την αναγνώριση όσο και τη διόρθωση αδυναμιών.

Ορισμένες εφαρμογές refactoring έχουν συσχετιστεί με βελτιώσεις σε security-related μετρικές ή με περιορισμό προβληματικών κατασκευών. Οι σχετικές διαπιστώσεις αφορούν τα συστήματα και τις συνθήκες που εξετάστηκαν στις αντίστοιχες εμπειρικές μελέτες και δεν γενικεύονται σε κάθε περίπτωση refactoring (Mumtaz et al., 2018; Almogahed, Omar & Zakaria, 2022). Υπό συγκεκριμένες προϋποθέσεις, μία δομική βελτίωση του πηγαίου κώδικα μπορεί επομένως να περιορίζει χαρακτηριστικά που σχετίζονται με την έκθεση security-sensitive δεδομένων ή λειτουργιών.

Μία ακόμη μορφή θετικής επίδρασης εμφανίζεται όταν η διόρθωση ενός quality issue αφαιρεί ταυτόχρονα και ένα security issue. Στο συγκεκριμένο, εφαρμόζεται η μελέτη που διενέργησαν οι Gigante et al. (2023) εξέτασαν ακριβώς αυτή τη σχέση συγκρίνοντας quality και security oriented ευρήματα στατικών αναλύσεων. Τα αποτελέσματα που εξήχθησαν, έδειξαν ότι η επίλυση προβλημάτων ποιότητας μέσω refactoring μπορούσε σε ορισμένες περιπτώσεις να επιλύσει παράλληλα security findings. Η επικάλυψη, ωστόσο, ήταν μόνο μερική, καθώς σημαντικός αριθμός security issues παρέμενε μετά το quality-oriented refactoring που εφαρμόστηκε. Συνεπώς, η βελτίωση της ποιότητας μπορεί να δημιουργεί ένα security benefit ως παράπλευρο αποτέλεσμα, αλλά δεν μπορεί να αντικαταστήσει την ειδικά προσανατολισμένη ανάλυση ασφάλειας.

Η αξιοποίηση τεχνικών refactoring με επίκεντρο την ασφάλεια του λογισμικού μπορεί επίσης να γίνει περισσότερο συστηματικά. Μέσω της εξέτασης των επιδράσεων διαφορετικών μεταξύ τους refactorings τόσο σε quality attributes όσο και σε security metrics, προτάθηκε από τους Abid et al. (2022) μία security-aware refactoring approach, στην οποία οι δύο κατηγορίες στόχων εξετάζονται ταυτόχρονα. Η μελέτη σε 30 projects ανοιχτού κώδικα έδειξε ότι μπορούν να επιλεγούν refactoring solutions που βελτιώνουν security-related metrics με σχετικά περιορισμένο ενδεχόμενο κόστος ως προς τους υπόλοιπους quality objectives. Το αποτέλεσμα αυτό αναδεικνύει τη σημασία της επιλογής refactoring με βάση περισσότερα από ένα χαρακτηριστικά και όχι αποκλειστικά με βάση τη συντηρησιμότητα ή την επαναχρησιμότητα.

Συμπεραίνουμε επομένως, ότι το refactoring μπορεί να συμβάλει τόσο στην βελτίωση της ποιότητας όσο και της ασφάλειας ενός συστήματος. Μπορεί να μειώνει τις προβληματικές δομές, ενισχύει το encapsulation, διευκολύνει την κατανόηση security-critical κώδικα ή απομακρύνει quality issues που επικαλύπτονται με security weaknesses. Σημειώνεται όμως, ότι οι θετικές αυτές επιδράσεις είναι context-dependent και διαφέρουν ανάλογα με τον τύπο του refactoring, τη δομή του συστήματος και το security characteristic που εξετάζεται. Για τον λόγο αυτό, το refactoring θα πρέπει να αντιμετωπίζεται ως πιθανός μηχανισμός υποστήριξης της ασφαλέστερης εξέλιξης του λογισμικού και όχι ως αυτόνομη τεχνική αντιμετώπισης διάφορων vulnerabilities.

5.6.2 Πιθανές regressions

Το refactoring σαν διαδικασία αποσκοπεί άμεσα στη βελτίωση της εσωτερικής δομής του κώδικα χωρίς πρόθεση μεταβολής της εξωτερικά παρατηρήσιμης συμπεριφοράς. Πολλές φορές όμως, δεν είναι πλήρως απαλλαγμένο από τον κίνδυνο εισαγωγής νέων προβλημάτων. Μεταβολές στη δομή των classes, στις εξαρτήσεις, στο encapsulation ή στη θέση και προσβασιμότητα μεθόδων και δεδομένων μπορούν να επηρεάσουν υπάρχουσες security assumptions. Ένα security regression επομένως, μπορεί να εμφανιστεί όταν μία quality-oriented αλλαγή αποδυναμώνει μία προηγουμένως ισχύουσα ιδιότητα ασφάλειας ή συμβάλλει στην εισαγωγή νέας αδυναμίας.

Ορισμένες refactoring operations έχουν συσχετιστεί με fault-inducing changes, ιδίως όταν μεταβάλλουν σύνθετες δομικές σχέσεις. Η συσχέτιση αυτή δεν σημαίνει ότι κάθε refactoring προκαλεί defect ή ότι κάθε defect είναι security-relevant (Bavota et al., 2012; Iannone et al., 2023). Ένα τέτοιο defect δεν αποτελεί αυτομάτως πρόβλημα ασφαλείας. Όμως, μπορεί να αποκτήσει security significance όταν επηρεάζει λειτουργίες όπως authentication, authorization, input validation ή τη διαχείριση ευαίσθητων δεδομένων.

Η βελτίωση της εσωτερικής ποιότητας μέσω της διαδικασίας του refactoring δεν εγγυάται ότι η ασφάλεια θα παραμείνει αμετάβλητη ή θα βελτιωθεί παράλληλα. Οι security συνέπειες εξαρτώνται από το είδος και το context της μεταβολής και για τον λόγο αυτό σημαντικά refactorings, ιδιαίτερα σε security-sensitive τμήματα του συστήματος, πρέπει να συνοδεύονται από κατάλληλο review και security verification. Η δυνατότητα εμφάνισης τέτοιων regressions αποτελεί χαρακτηριστικό παράδειγμα του τρόπου με τον οποίο μία βελτίωση σε μία διάσταση ποιότητας μπορεί υπό συγκεκριμένες συνθήκες να έχει ανεπιθύμητη επίδραση στην ασφάλεια.

5.7 Technical και Security Debt

Όπως έχει ήδη αναλυθεί σε προηγούμενη ενότητα του παρόντος, περιγράφει τεχνικές αποφάσεις ή κατασκευές που μπορεί να προσφέρουν ένα βραχυπρόθεσμο όφελος, αλλά δημιουργούν ένα τεχνικό περιβάλλον στο οποίο οι μελλοντικές αλλαγές δυσκολότερες ή περισσότερο δαπανηρές. Η συσσώρευσή του συνδέεται κυρίως με την υποβάθμιση εσωτερικών χαρακτηριστικών του λογισμικού και του πηγαίου κώδικα όπως η maintainability και evolvability και μπορεί να εκφράζεται μέσα από προβληματικές σχεδιαστικές επιλογές, αυξημένη πολυπλοκότητα, duplication, ανεπαρκές testing ή άλλες εκκρεμότητες οι οποίες μεταφέρονται συνεχώς σε μελλοντικές φάσεις ανάπτυξης.

Η παραπάνω αυτή λογική, επεκτείνεται και στον τομέα της ασφάλειας μέσω του ορισμού της έννοιας του Security Debt. Ο όρος χρησιμοποιείται για να

περιγράψει security-related υποχρεώσεις ή κινδύνους που δεν αντιμετωπίζονται άμεσα και μεταφέρονται στο μέλλον. Για παράδειγμα, τέτοιες περιπτώσεις μπορούν να προκύψουν όταν αναβάλλεται η αντιμετώπιση μιας γνωστής αδυναμίας, χρησιμοποιούνται προσωρινά υπερβολικά permissions ή μία security requirement παραμένει ανεκπλήρωτη με στόχο την επιτάχυνση της παράδοσης της προβλεπόμενης λειτουργίας. Αντίστοιχα, η συγκεκριμένη αναβολή μπορεί να παρέχει βραχυπρόθεσμο όφελος, αλλά αφήνει μία εκκρεμότητα η οποία πρέπει αργότερα να αντιμετωπιστεί και η οποία ενδέχεται στο μεταξύ να αυξήσει την έκθεση του συστήματος σε εξωτερικούς κινδύνους και επιτιθέμενους.

Οι έννοιες του Technical και Security Debt δεν είναι ανεξάρτητες. Μέρος του συσσωρευμένου Technical Debt μπορεί να έχει security implications, επειδή χαρακτηριστικά όπως προβληματική δομή, υψηλή πολυπλοκότητα, code smells και δυσκολία συντήρησης είναι ικανές να δυσχεράνουν την κατανόηση, τον έλεγχο και την ασφαλή τροποποίηση του λογισμικού. Αποτελέσματα εμπειρικών μελετών, υποστηρίζουν την ύπαρξη στατιστικής σχέσης μεταξύ ορισμένων debt indicators και security-related outcomes, καθώς η εικόνα αυτή ενισχύεται και από περισσότερο σύγχρονες μελέτες, χωρίς να αποδεικνύεται αιτιότητα μεταξύ των δύο, Technical και Security Debt.

Τα Technical και Security Debt μπορούν να θεωρηθούν δύο συνδεδεμένες μεταξύ τους μορφές μεταφοράς κόστους και κινδύνου στο μέλλον του κύκλου ανάπτυξης του λογισμικού. Η μη αντιμετώπιση προβλημάτων εσωτερικής ποιότητας μπορεί να καταστήσει τη μελλοντική security analysis και remediation δυσκολότερη, ενώ η αναβολή security-related εργασιών μπορεί να δημιουργήσει ένα διαφορετικό είδος συσσωρευμένης υποχρέωσης με δυνητικά σοβαρές συνέπειες. Ξεκαθαρίζεται, ότι κάθε Technical Debt item δεν αποτελεί Security Debt καθώς και κάθε vulnerability δεν είναι αποτέλεσμα Technical Debt. Η αξία των δύο εννοιών βρίσκεται κυρίως στο ότι επιτρέπουν να εξεταστεί το αποτέλεσμα τωρινών τεχνικών αποφάσεων στην δημιουργία μελλοντικού κόστους συντήρησης, remediation και security risk.

5.8 Synergies και trade-offs

Οι επιδράσεις μεταξύ ποιότητας και ασφάλειας δεν έχουν ενιαία κατεύθυνση. Η βιβλιογραφία καταγράφει τόσο περιπτώσεις παράλληλης βελτίωσης όσο και trade-offs, γι' αυτό κάθε συμπέρασμα πρέπει να περιορίζεται στο είδος της αλλαγής, στα χρησιμοποιούμενα metrics και στο εξεταζόμενο δείγμα (Abid et al., 2022; Reis, Abreu & Cruz, 2021; Iannone et al., 2023). Οι δύο γενικές αυτές περιπτώσεις περιγράφονται ως συνέργειες (synergies) και αντισταθμίσεις (trade-offs) αντίστοιχα.

Οι έννοιες αυτές χρησιμοποιούνται στην παρούσα ενότητα ως θεωρητικό και βιβλιογραφικά τεκμηριωμένο πλαίσιο πιθανών σχέσεων μεταξύ ποιότητας και ασφάλειας. Οι τέσσερις περιπτώσεις που ακολουθούν οργανώνουν διαφορετικούς πιθανούς συνδυασμούς μεταβολής των δύο διαστάσεων και δεν πρέπει να

ερμηνεύονται ως εμπειρικές κατηγορίες που επιβεβαιώθηκαν από το dataset της παρούσας εργασίας.

Η προαναφερθείσα διάκριση είναι σημαντική για την κατανόηση της σχέσης μεταξύ Quality και Security, καθώς αποφεύγει την υπόθεση ότι οι δύο διαστάσεις μεταβάλλονται πάντα προς την ίδια κατεύθυνση, βελτίωση ή υποβάθμιση. Παρακάτω, εξετάζονται οι διαφορετικοί δυνατοί συνδυασμοί μεταβολών μεταξύ τους, με στόχο να αποσαφηνιστούν τόσο οι περιπτώσεις αμοιβαίας ενίσχυσης όσο και εκείνες στις οποίες μπορεί να εμφανιστεί σύγκρουση μεταξύ των δύο διαστάσεων.

5.8.1 Quality improvement – Security improvement

Μία από τις πιθανές μορφές συνέργειας μεταξύ των συνιστωσών ποιότητας και ασφάλειας εμφανίζεται όταν μία μεταβολή που αποσκοπεί στη βελτίωση της ποιότητας του λογισμικού συνοδεύεται από βελτίωση κάποιου/κάποιων security-related χαρακτηριστικών. Αυτό μπορεί να συμβεί όταν ένα quality defect και ένα security weakness έχουν κοινή τεχνική προέλευση. Στην περίπτωση αυτή, η διόρθωση του υποκείμενου προβλήματος μπορεί να ωφελήσει ταυτόχρονα και τις δύο διαστάσεις.

Ένας βασικός μηχανισμός αφορά τη μείωση των development errors και τη βελτίωση της εσωτερικής δομής του κώδικα. Η απομάκρυνση προβληματικών κατασκευών, η απλοποίηση δυσνόητου κώδικα, η βελτίωση της maintainability και η περισσότερο συστηματική verification μπορούν να περιορίσουν τις συνθήκες κατά τις οποίες δημιουργούνται ή παραμένουν ορισμένες από τις αδυναμίες. Η μείωση των γενικότερων software errors μπορεί να μειώσει παράλληλα μέρος των security vulnerabilities, καθώς ορισμένες από αυτές αποτελούν ταυτόχρονα quality defects. Αντίστοιχη σχέση έχει παρατηρηθεί και σε quality-oriented μεταβολές του πηγαίου κώδικα.

Η διαθέσιμη βιβλιογραφία περί της συγκεκριμένης σχέσης δεν υποστηρίζει ότι κάθε quality improvement δημιουργεί αντίστοιχη security improvement. Πρόσφατη διεξαγωγή μελέτης από τους Fontana et al. (2026) για την αφαίρεση architectural smells βρήκε σαφέστερη βελτίωση στους quality και performance indicators, αλλά σημαντικά μικρότερη επίδραση σε security vulnerabilities. Συμπερασματικά, η σχέση Quality improvement → Security improvement πρέπει να θεωρείται πιθανή συνέργεια η οποία εμφανίζεται κυρίως όταν οι δύο διαστάσεις επηρεάζονται από κοινό υποκείμενο πρόβλημα και όχι ως καθολικός κανόνας. Μία ταυτόχρονη βελτίωση ποιότητας και ασφάλειας αποτελεί ένδειξη συμβατής μεταβολής των δύο διαστάσεων, χωρίς από μόνη της να αποδεικνύει ότι η βελτίωση της ποιότητας προκάλεσε τη βελτίωση της ασφάλειας.

5.8.2 Quality improvement – Security degradation

Μία δεύτερη πιθανή μορφή αλληλεπίδρασης εμφανίζεται στην περίπτωση όπου μία μεταβολή η οποία στοχεύει στη βελτίωση κάποιου χαρακτηριστικού ποιότητας συνοδεύεται από επιδείνωση/υποβάθμιση της ασφάλειας. Η συγκεκριμένη αυτή περίπτωση αποτελεί επομένως χαρακτηριστικό trade-off, καθώς μία τεχνική απόφαση μπορεί να είναι ωφέλιμη ως προς τη συντηρησιμότητα του συστήματος, τη modularity, τη reusability ή άλλα quality attributes, αλλά ταυτόχρονα να μεταβάλλει αρνητικά security-relevant χαρακτηριστικά του συστήματος.

Ένα refactoring μπορεί να βελτιώσει παραδοσιακές quality metrics αλλά ταυτόχρονα να αυξήσει την προσβασιμότητα στοιχείων ή την attack surface. Η συγκεκριμένη πιθανότητα έχει μελετηθεί σε object-oriented refactorings και δεν πρέπει να γενικεύεται πέρα από τις συνθήκες της αντίστοιχης μελέτης (Ruland et al., 2018; Manadhata & Wing, 2011). Οι Ruland et al. (2018), για παράδειγμα, έδειξαν ότι η βελτιστοποίηση παραδοσιακών quality metrics μέσω object-oriented refactorings μπορεί να έρχεται σε σύγκρουση με την προσπάθεια περιορισμού της attack surface, καθώς ορισμένες μεταβολές απαιτούν χαλάρωση των accessibility restrictions.

Σαν αποτέλεσμα των παραπάνω προαναφερθέντων, η σχέση Quality improvement → Security degradation δεν θα πρέπει να λαμβάνεται ως αναπόφευκτη, αλλά ως ένα πιθανό σενάριο αποτελέσματος όταν μία αλλαγή βελτιστοποιεί ένα quality objective χωρίς να λαμβάνει επαρκώς υπόψη τις security implications της. Η βελτίωση της εσωτερικής ποιότητας μπορεί να είναι, τελικά πραγματική, ενώ ταυτόχρονα να δημιουργείται μεγαλύτερη έκθεση, να αποδυναμώνεται το encapsulation ή να εισάγονται νέες security-sensitive συνθήκες. Η περίπτωση αυτή αναδεικνύει την ανάγκη της αξιολόγησης των μεταβολών σε περισσότερες από μία διαστάσεις και όχι αποκλειστικά με βάση τον αρχικό στόχο της αλλαγής.

5.8.3 Security improvement – Quality improvement

Η θετική αλληλεπίδραση μεταξύ των δύο διαστάσεων, ασφάλειας και ποιότητας, μπορεί να εμφανιστεί και προς την αντίθετη, από την προηγούμενως περιγραφόμενη (Ενότητα 5.8.1) σχέση τους, όταν μία μεταβολή η οποία πραγματοποιείται πρωτίστως για τη βελτίωση της ασφάλειας συνοδεύεται από παράλληλη βελτίωση χαρακτηριστικών ποιότητας. Αυτό μπορεί να συμβεί σε περιπτώσεις όπου η αντιμετώπιση μιας ευπάθειας απαιτεί την απομάκρυνση τμήματος προβληματικού κώδικα, την απλοποίηση μιας υλοποίησης, την καλύτερη απομόνωση security-sensitive λειτουργιών ή γενικότερα μία περισσότερο οργανωμένη μεταβολή της δομής του συστήματος. Η αντιμετώπιση vulnerabilities δεν αποτελεί μόνο πιθανή πηγή quality trade-offs. Σε σημαντικό αριθμό περιπτώσεων μπορεί να συνοδεύεται και από πραγματική βελτίωση χαρακτηριστικών συντηρησιμότητας.

Security-oriented αλλαγές μπορεί να συνοδεύονται από refactorings που βελτιώνουν την οργάνωση ή την κατανοησιμότητα του κώδικα. Η ταυτόχρονη εμφάνιση των δύο δραστηριοτήτων δεν αποδεικνύει αυτομάτως βελτίωση ποιότητας και πρέπει να τεκμηριώνεται με τα αποτελέσματα της συγκεκριμένης μελέτης (Gigante et al., 2023).

Ορισμένες security-oriented διορθώσεις, παράλληλα, μπορούν να ενισχύουν χαρακτηριστικά όπως η reliability και η robustness. Για παράδειγμα, η καλύτερη διαχείριση exceptional conditions, η αυστηρότερη input validation ή η ορθή αποδέσμευση πόρων μπορούν να περιορίζουν vulnerabilities και ταυτόχρονα να μειώνουν crashes, inconsistent states και μη προβλέψιμη συμπεριφορά.

Εν κατακλείδι, η σχέση Security improvement → Quality improvement αποτελεί μία πραγματική αλλά όχι αναγκαία μορφή συνέργειας. Μία security fix μπορεί να βελτιώσει παράλληλα τη maintainability, τη δομή ή τη reliability του λογισμικού όταν αντιμετωπίζει ένα κοινό υποκείμενο πρόβλημα ή όταν η ίδια η remediation πραγματοποιείται με τρόπο που βελτιώνει τον πηγαίο κώδικα, στο σύνολό του. Το γεγονός όμως, ότι μία αλλαγή επιλύει μία vulnerability δεν αρκεί από μόνο του για να θεωρηθεί ότι βελτιώνει και την ποιότητα. Υπενθυμίζεται ότι οι δύο αυτές διαστάσεις είναι αναγκαίο να αξιολογούνται ανεξάρτητα μεταξύ τους.

5.8.4 Security improvement – Quality degradation

Αντίστοιχα με την παραπάνω περίπτωση, μία ακόμη πιθανή μορφή trade-off εμφανίζεται όταν μία αλλαγή που πραγματοποιείται για τη βελτίωση της ασφάλειας επιτυγχάνει τον security στόχο της, αλλά ταυτόχρονα επιβαρύνει κάποιο χαρακτηριστικό ποιότητας του λογισμικού. Η αντιμετώπιση μιας vulnerability μπορεί, παραδείγματος χάριν, να απαιτεί την προσθήκη νέων ελέγχων, validation logic, conditions ή επιπλέον μηχανισμών προστασίας, αυξάνοντας με τον τρόπο αυτό το μέγεθος ή την πολυπλοκότητα του κώδικα και δυσχεραίνοντας τη μελλοντική συντήρησή του.

Η διόρθωση μιας vulnerability μπορεί να εισαγάγει πρόσθετους ελέγχους ή πολυπλοκότητα και, σε ορισμένες περιπτώσεις, να επιβαρύνει τη maintainability. Το εύρημα αποτελεί πιθανό trade-off και όχι καθολική συνέπεια κάθε security patch (Reis, Abreu & Cruz, 2021).

Άρα, η σχέση Security improvement → Quality degradation δεν σημαίνει ότι η security fix είναι λανθασμένη ή ανεπιθύμητη. Περιγράφει, αντίθετα, μία περίπτωση κατά την οποία το security benefit συνοδεύεται από πραγματικό κόστος σε άλλη διάσταση του λογισμικού, για παράδειγμα στην ποιότητα. Η ύπαρξη τέτοιων trade-offs αναδεικνύει την ανάγκη οι security-oriented μεταβολές να αξιολογούνται όχι μόνο ως προς την επιτυχή αντιμετώπιση ενός vulnerability, αλλά και ως προς τις επιπτώσεις στην maintainability, την complexity και γενικότερα στην εξέλιξη του κώδικα.

5.9 Confounding variables και περιορισμοί αιτιώδους ερμηνείας

Σε observational repository data, μία στατιστική σχέση δεν αρκεί για αιτιώδες συμπέρασμα. Μη παρατηρούμενες μεταβλητές, selection effects και χαρακτηριστικά του project μπορούν να επηρεάζουν ταυτόχρονα predictors και outcomes, γι' αυτό οι ερμηνείες πρέπει να παραμένουν συσχετιστικές (Kalliamvakou et al., 2016; Furia & Torkar, 2026). Η βιβλιογραφία της causal inference στο Software Engineering επισημαίνει ότι η ανάλυση observational data απαιτεί πρόσθετες υποθέσεις και μεθοδολογικούς μηχανισμούς προκειμένου να υποστηριχθούν πραγματικά causal claims.

Κεντρικό πρόβλημα εδώ αποτελεί το confounding. Μία μεταβλητή μπορεί να λειτουργεί ως confounder όταν με κατάλληλο τρόπο επηρεάζει αιτιώδη τρόπο τόσο τη μεταβλητή που εξετάζεται ως πιθανή αιτία όσο και το αποτέλεσμα, δημιουργώντας με τον τρόπο αυτό μία σχέση η οποία μπορεί να αποδοθεί λανθασμένα στην άμεση επίδραση μεταξύ των δύο. Η αναγνώριση ενός confounder δεν βασίζεται απλώς στο ότι μία Τρίτη μεταβλητή παρουσιάζει στατιστική συσχέτιση με τις υπόλοιπες, αλλά απαιτεί θεωρητικές υποθέσεις σχετικά με την αιτιώδη δομή του φαινομένου. Για τον λόγο αυτό χρησιμοποιούνται συνήθως τα causal models και Directed Acyclic Graphs (DAGs), τα οποία επιτρέπουν τη ρητή αποτύπωση πιθανών causal paths.

Στην ανάλυση των software repositories μπορούν να υπάρχουν πολλοί πιθανοί confounding ή contextual factors. Το μέγεθος μιας αλλαγής, το μέγεθος και η ηλικία του κώδικα, η εμπειρία των προγραμματιστών, οι διαδικασίες review και testing, οι τεχνολογίες που χρησιμοποιούνται και ιδιαίτερα το project στο οποίο πραγματοποιείται μία αλλαγή μπορούν να επηρεάζουν τόσο quality όσο και security-related μετρήσεις. Έστω ένα Pull Request το οποίο μπορεί να παρουσιάζει περισσότερη πολυπλοκότητα. Το ίδιο PR μπορεί να παρουσιάζει επίσης και περισσότερα security findings απλώς επειδή περιλαμβάνει μεγαλύτερη ποσότητα κώδικα προς ανάλυση. Διαφορετικά projects μπορούν να έχουν διαφορετικές coding conventions, architectures, development practices και security requirements, αντίστοιχα. Τέτοιοι παράγοντες πρέπει να εξετάζονται ως υποψήφιοι confounders ή covariates, χωρίς να θεωρούνται αυτομάτως confounders πριν καθοριστεί η υποκείμενη causal structure. Σύγχρονες μελέτες vulnerability prediction αναγνωρίζουν αντίστοιχα παράγοντες όπως developer experience, code age και review practices ως πιθανές εναλλακτικές εξηγήσεις παρατηρούμενων σχέσεων μεταξύ μετρικών quality και security defects.

Ιδιαίτερη σημασία επίσης έχει η ύπαρξη πιθανότητας αντίστροφης αιτιότητας (reverse causality). Για παράδειγμα, μία θετική σχέση μεταξύ των complexity και security findings δεν συνεπάγεται απαραίτητα αιτιότητα της αυξημένης complexity στην δημιουργία περισσότερων προβλημάτων ασφάλειας. Μία security-oriented αλλαγή είναι δυνατό να απαιτεί πρόσθετους ελέγχους και λογική και να αυξάνει η ίδια την πολυπλοκότητα. Συμπεραίνουμε άρα ότι χωρίς σαφή χρονική και αιτιώδη θεμελίωση, η κατεύθυνση μιας παρατηρούμενης σχέσης δεν μπορεί να συναχθεί μόνο από έναν regression coefficient ή μία στατιστική συσχέτιση.

Μέρος των περιγραφόμενων προβλημάτων μπορεί να περιοριστεί με την χρήση πολυμεταβλητών regression models και mixed-effects models, υπό την προϋπόθεση ότι κατάλληλες παρατηρούμενες μεταβλητές θα συμπεριληφθούν στο μοντέλο και θα ληφθεί υπόψη η ιεραρχική δομή των δεδομένων. Η διαφοροποίηση μεταξύ projects, για παράδειγμα, μπορεί να μοντελοποιηθεί έτσι ώστε να μην αντιμετωπίζονται όλες οι παρατηρήσεις ως ανεξάρτητες και απολύτως ομοιογενείς. Εντούτοις, η στατιστική προσαρμογή δεν εξαλείφει αυτομάτως το confounding. Μη παρατηρούμενες ή μη μετρημένες μεταβλητές μπορούν να εξακολουθούν να προκαλούν omitted-variable bias (συστημικό σφάλμα παραλειπόμενης μεταβλητής - στατιστικό σφάλμα που συμβαίνει στη στατιστική ανάλυση παλινδρόμησης όταν ένα μοντέλο παραλείπει μία ή περισσότερες σημαντικές, σχετικές μεταβλητές-), ενώ η απλή προσθήκη όσο το δυνατόν περισσότερων μεταβλητών σε ένα μοντέλο δεν αποτελεί εγγύηση ορθής causal adjustment. Το omitted-variable bias, όπως αυτό επισημαίνεται από τους Furia και Torkar (2026), αποτελεί σημαντικό πρόβλημα σε μελέτες παρατήρησης στον τομέα της Μηχανικής Λογισμικού και μπορεί να οδηγήσει τόσο σε υπερεκτίμηση όσο και σε υποεκτίμηση των πραγματικών effects.

Οι σχέσεις επομένως μεταξύ quality και security indicators πρέπει να ερμηνεύονται πρωτίστως ως στατιστικές συσχετίσεις, εκτός εάν το ερευνητικό design και οι σχετικές causal assumptions επιτρέπουν ισχυρότερη ερμηνεία. Ένας statistically significant coefficient μπορεί να υποδεικνύει ότι δύο μεταβλητές μεταβάλλονται συστηματικά μεταξύ τους αφού ληφθούν υπόψη οι παράγοντες που περιλαμβάνει το μοντέλο, αλλά δεν αποδεικνύει άμεσα ότι η μία προκαλεί απαραίτητα την άλλη. Όταν τα εξεταζόμενα μεγέθη προέρχονται από automated software-analysis tools (εδώ από το εργαλείο SonarQube), τα αποτελέσματα αφορούν μετρημένες quality και security indicators και όχι το σύνολο της πραγματικής ποιότητας ή του πραγματικού security risk ενός συστήματος, στοιχεία που θα μπορούσαν να αντληθούν από τον συνδυασμό των τεχνικών SAST και DAST analysis -εδώ τα ευρήματα της εργασίας έχουν προέλθει από την χρησιμοποίηση SAST ανάλυσης-. Η προσεκτική αυτή διάκριση κρίνεται απαραίτητη προκειμένου τα εμπειρικά αποτελέσματα να μην οδηγούν σε ισχυρότερα συμπεράσματα από εκείνα τα οποία μπορεί να υποστηρίξει ο ερευνητικός σχεδιασμός.

5.10 Προηγούμενες εμπειρικές μελέτες

Η θεωρητική σχέση μεταξύ της ποιότητας και της ασφάλειας του πηγαίου κώδικα και του λογισμικού έχει ήδη διερευνηθεί και μέσα από πλήθος εμπειρικών μελετών, οι οποίες αξιοποιούν πραγματικά δεδομένα λογισμικού με στόχο την αναγνώριση πιθανών συσχετίσεων, κοινών παραγόντων και μοτίβων αλληλεπίδρασης μεταξύ των δύο διαστάσεων. Ανάλογα με το ερευνητικό ερώτημα που τίθεται κάθε φορά, οι μελέτες αυτές βασίζονται σε διαφορετικές μονάδες ανάλυσης, πηγές δεδομένων και τεχνικές μέτρησης, όπως repositories, commits, Pull Requests, ιστορικά αλλαγών, static-analysis findings και source-code metrics.

Παρακάτω, παρουσιάζονται οι βασικές κατηγορίες προηγούμενων ερευνών που σχετίζονται περισσότερο με το αντικείμενο της παρούσας εργασίας. Εξετάζονται μελέτες που βασίζονται σε Pull Request και repository mining, έρευνες που διερευνούν άμεσα την αλληλεπίδραση quality και security, μελέτες που αξιοποιούν static analysis για την εξαγωγή σχετικών δεικτών, καθώς και εργασίες που εξετάζουν τη σχέση μεταξύ complexity, software defects και vulnerabilities. Η επισκόπηση αυτή αποσκοπεί στη συνοπτική αποτύπωση της υπάρχουσας εμπειρικής γνώσης και στη δημιουργία του απαραίτητου υπόβαθρου για την τοποθέτηση της παρούσας έρευνας σε σχέση με τη σχετική βιβλιογραφία.

5.10.1 Pull Request και repository mining

Η ευρεία χρήση συστημάτων ελέγχου εκδόσεων και πλατφορμών συνεργατικής ανάπτυξης έχει δημιουργήσει σημαντικούς όγκους δεδομένων σχετικά με την πραγματική εξέλιξη ενός λογισμικού. Το Mining Software Repositories (MSR) αξιοποιεί πληροφορίες από repositories, commits, issues, Pull Requests και άλλα development artifacts για την εμπειρική μελέτη φαινομένων όπως η εξέλιξη του κώδικα, η εμφάνιση defects, οι πρακτικές συνεργασίας και η ίδια η ποιότητα του λογισμικού. Η δυνατότητα ανάλυσης πραγματικών ιστορικών δεδομένων έχει καταστήσει το repository mining βασική προσέγγιση της empirical software engineering.

Τα Pull Requests παρουσιάζουν ιδιαίτερο ενδιαφέρον, καθώς οριοθετούν συγκεκριμένες μεταβολές του λογισμικού και συγκεντρώνουν πληροφορίες για τον τροποποιημένο κώδικα, τα commits, το review και την τελική έκβαση της προτεινόμενης προς συγχώνευση αλλαγής. Η μελέτη που διεξήχθη από τους Gousios et al. (2014) αποτέλεσε μία από τις πρώτες μεγάλης κλίμακας διερευνήσεις του pull-based development και έδειξε τη χρησιμότητα των PRs για τη μελέτη των contributions και των merge decisions. Αντίστοιχα, η αξιοποίηση των δεδομένων για εμπειρική μελέτη της ποιότητας επιβεβαιώθηκε από τους McIntosh et al. (2016), οι οποίοι συνέδεσαν χαρακτηριστικά της διαδικασίας code review με μεταγενέστερα software defects.

Σε επίπεδο Pull Request η ανάλυση έχει χρησιμοποιηθεί με στόχο την εξέταση χαρακτηριστικών του ίδιου του πηγαίου κώδικα. Οι Lenarduzzi et al. (2021) μέσω της ανάλυσης περίπου 36.000 PRs και εκατομμύρια static-analysis findings διερεύνησαν τη σχέση code-quality issues και Pull Request acceptance. Παράλληλα, repository και code-review δεδομένα έχουν αξιοποιηθεί και σε security-oriented έρευνες, όπως οι μελέτες των Thompson και Wagner (2017) και Alfadel et al. (2023), οι οποίες εξέτασαν τη σχέση μεταξύ code review και security-related defects ή discussions.

Τα δεδομένα δεν αποτελούν χωρίς περιορισμούς αναπαράσταση της πραγματικής διαδικασίας ανάπτυξης. Μελέτες έχουν αναδείξει προβλήματα που μπορούν να προκύψουν από την επιλογή repositories, την ερμηνεία του Pull Request status και τη χρήση μη φιλτραρισμένων GitHub δεδομένων. Η εγκυρότητα, συνεπώς, μιας repository-mining μελέτης εξαρτάται σε σημαντικό βαθμό από τον τρόπο επιλογής των projects, τον ορισμό της μονάδας ανάλυσης, τα filtering criteria

και την ποιότητα της διαδικασίας συλλογής δεδομένων. Παρά τους προαναφερθέντες περιορισμούς, τα Pull Requests αποτελούν ιδιαίτερα χρήσιμη μονάδα εμπειρικής ανάλυσης, καθώς επιτρέπουν τη σύνδεση συγκεκριμένων μεταβολών του πηγαίου κώδικα με μετρήσιμα χαρακτηριστικά της ποιότητας και της ασφάλειας.

5.10.2 Μελέτες *Quality-Security interactions*

Η εμπειρική διερεύνηση της σχέσης μεταξύ Software Quality και Software Security έχει βασιστεί σε διαφορετικές προσεγγίσεις, όπως quality και reliability measures, Technical Debt, refactoring και security patches. Κοινός στόχος των σχετικών μελετών είναι να εξεταστεί κατά πόσο χαρακτηριστικά που συνδέονται με την ποιότητα μπορούν να παρέχουν πληροφορία για την ασφάλεια και κατά πόσο μεταβολές στη μία διάσταση συνοδεύεται από αντίστοιχες μεταβολές στην άλλη.

Ένα μέρος των security vulnerabilities συνδέεται με γενικότερα software defects, όπως οι Woody, Ellison και Nichols (2014) έδειξαν, υποστηρίζοντας ότι η μείωση πλήθους των development errors μπορεί να περιορίσει και ορισμένα security problems. Μελέτες των Siavvas et al. (2019, 2022) εντόπισαν σχέσεις μεταξύ Technical Debt και security-related indicators, υποδεικνύοντας ότι χαρακτηριστικά εσωτερικής ποιότητας μπορούν να περιέχουν οποιαδήποτε πληροφορία σχετικά με την ασφάλεια. Οι σχέσεις, όμως, αυτές δεν συνεπάγονται ότι τα quality metrics αποτελούν άμεσες μετρήσεις ή αιτίες της ασφάλειας.

Επίσης, ιδιαίτερο ενδιαφέρον αναφοράς παρουσιάζουν οι μελέτες οι οποίες εξετάζουν μεταβολές του πηγαίου κώδικα. Οι Abid et al. (2022) έδειξαν ότι quality-oriented refactorings μπορούν άλλοτε να βελτιώνουν security characteristics και άλλοτε να δημιουργούν trade-offs. Οι Gigante et al. (2023) διαπίστωσαν, αντίστοιχα, ότι η επίλυση quality issues μπορεί να αντιμετωπίζει παράλληλα ορισμένα security findings, αλλά η επικάλυψη είναι μόνο μερική. Οι Iannone et al. (2023) κατέληξαν, με την σειρά τους, στην διαπίστωση ότι η επίδραση του refactoring στην ασφάλεια διαφοροποιείται ανάλογα με τον τύπο της αλλαγής και το security characteristic που εξετάζεται.

Η μελέτη της επίδρασης καλύπτεται και προς την αντίστροφη κατεύθυνση. Οι Reis, Abreu και Cruz με έρευνά τους το 2021 έδειξαν ότι security patches μπορούν σε ορισμένες περιπτώσεις να βελτιώνουν τη maintainability, ενώ σε άλλες να την επιβαρύνουν, παρέχοντας άμεση εμπειρική ένδειξη τόσο για συνέργειες όσο και για trade-offs μεταξύ των δύο διαστάσεων.

Η υπάρχουσα βιβλιογραφία στο σύνολό της, δεν υποστηρίζει μία ενιαία ή μονοσήμαντη σχέση μεταξύ των διαστάσεων Quality και Security. Αντιθέτως, καταγράφονται επικαλύψεις, θετικές συνέργειες, ουδέτερες επιδράσεις και trade-offs, τα οποία εξαρτώνται από τις εξεταζόμενες μετρικές, τον τύπο της μεταβολής, το project και τη μεθοδολογία της εκάστοτε μελέτης. Η συγκεκριμένη εικόνα ενισχύει την αντιμετώπιση της αλληλεπίδρασης μεταξύ Quality και Security ως ενός πολυδιάστατου και context-dependent φαινομένου.

5.10.3 Static-analysis-based studies

Η τεχνική της Στατικής Ανάλυσης Πηγαίου Κώδικα βρίσκει εφαρμογή ευρέως στην εμπειρική έρευνα για την αυτοματοποιημένη συλλογή quality και security-related δεδομένων σε μεγάλο αριθμό συστημάτων. Η εφαρμογή κοινών κανόνων και μετρικών σε διαφορετικά projects και εκδόσεις των ίδιων επιτρέπει τη συστηματική παρατήρηση και καταγραφή χαρακτηριστικών του κώδικα και την αξιοποίησή τους ως empirical indicators.

Από την σκοπιά της πλευράς της ποιότητας, οι Nagappan και Ball (2005) έδειξαν ότι static-analysis findings μπορούν να παρουσιάζουν σχέση με πραγματικά software defects και να χρησιμοποιούνται για τον εντοπισμό περισσότερων fault-prone components. Στην άλλη διάσταση, στην ασφάλεια, οι Aloraini et al. (2019) ανέλυσαν security warnings από διαφορετικά SAST εργαλεία και έδειξαν ότι τα findings μπορούν να χρησιμοποιηθούν για τη μελέτη security-related χαρακτηριστικών, αλλά η αποτελεσματικότητα και τα αποτελέσματα διαφοροποιούνται σημαντικά μεταξύ εργαλείων και κατηγοριών προβλημάτων.

Οι μελέτες οι οποίες συνδυάζουν διαφορετικές μορφές static analysis, όπως αυτή των Gigante et al. (2023), καθίστανται ιδιαίτερα σχετικές με την αλληλεπίδραση Quality-Security. Οι Gigante et al. συνέκριναν quality findings του SonarQube με security findings του Fortify και εντόπισαν μερική επικάλυψη μεταξύ των δύο συνιστωσών. Η επίλυση ορισμένων quality issues μπορούσε να αντιμετωπίζει παράλληλα security findings, όμως σημαντικό μέρος των security issues παρέμενε ανεξάρτητο, υποδεικνύοντας ότι quality-oriented και security-oriented analyses παρέχουν διαφορετική αλλά συμπληρωματική μεταξύ τους πληροφορία.

Τα αποτελέσματα τέτοιων μελετών πρέπει να ερμηνεύονται με αρκετή προσοχή. Η απόδοση και η ορθότητα των static analyzers εξαρτάται από το εργαλείο και την υλοποίησή του, τους ενεργούς κανόνες, τη γλώσσα με την χρήση της οποίας έχει υλοποιηθεί το υπό εξέταση λογισμικό, την πολυπλοκότητα του πηγαίου κώδικα και το είδος του εξεταζόμενου προβλήματος. Αξιολογήσεις του NIST SATE έχουν καταγράψει σημαντικές διαφορές στην ικανότητα των εργαλείων να εντοπίζουν διαφορετικές κατηγορίες αδυναμιών. Τα static-analysis findings αποτελούν χρήσιμους και επαναλήψιμους δείκτες της παρατηρούμενης κατάστασης του κώδικα, αλλά δεν πρέπει να θεωρούνται πλήρης και ολοκληρωμένη καταγραφή πραγματικών defects, vulnerabilities ή συνολικού security risk.

5.10.4 Complexity, defects και vulnerabilities

Η σχέση μεταξύ της πολυπλοκότητας του πηγαίου κώδικα, των software defects και των vulnerabilities έχει αποτελέσει αντικείμενο σημαντικού αριθμού εμπειρικών μελετών. Μετρικές οι οποίες περιγράφουν την complexity, το coupling, το cohesion και γενικότερα τη δομή του κώδικα έχουν χρησιμοποιηθεί αρχικά για την αναγνώριση defect-prone components και στη συνέχεια ως πιθανοί predictors

περιοχών με αυξημένη την πιθανότητα εμφάνισης vulnerabilities. Η βιβλιογραφία, βέβαια, δεν αντιμετωπίζει τις μετρικές αυτές ως άμεσες μετρήσεις της ασφάλειας, αλλά κυρίως ως indicators που μπορούν να υποστηρίξουν την προτεραιοποίηση testing και inspection activities. Η βασική ιδέα η οποία υποστηρίζεται μέσω των αποτελεσμάτων ερευνών της συγκεκριμένης θεματολογίας είναι ότι structural χαρακτηριστικά τα οποία παραδοσιακά χρησιμοποιούνται στην αξιολόγηση της software quality μπορούν να περιέχουν πληροφορία σχετική και με την ασφάλεια του λογισμικού ή/και του πηγαίου κώδικα.

Η χρησιμότητα των complexity metrics δεν είναι καθολική. Η αρχική εργασία των Shin και Williams (2008) έδειξε ότι μοντέλα βασισμένα σε complexity μπορούσαν να προβλέψουν vulnerabilities με σχετικά χαμηλό false-positive rate αλλά με υψηλό false-negative. Στην ευρύτερη βιβλιογραφία defect-prediction, οι Zimmermann και Nagappan (2008) έδειξαν ότι network measures των dependencies μπορούσαν να ξεπεράσουν μοντέλα που βασίζονταν αποκλειστικά σε μετρικές πολυπλοκότητας ως προς την πρόβλεψη defects στο λογισμικό Windows Server 2003. Τα ευρήματα υποδεικνύουν ότι η πολυπλοκότητα αποτελεί μόνο μία από τις πολλές διαστάσεις που σχετίζονται με defect-vulnerability proneness.

Η παραπάνω εικόνα επιβεβαιώνεται και από νεότερες βιβλιογραφικές συνθέσεις, οι οποίες -στην πλειοψηφία τους- έχουν καταλήξει ότι δεν υπάρχει ενιαίο ή καθολικά εφαρμόσιμο σύνολο μετρικών για την πρόβλεψη των vulnerabilities. Η complexity, επομένως και άλλες structural μετρικές μπορούν να παρέχουν χρήσιμη πληροφορία για την αναγνώριση περισσότερο vulnerability-prone περιοχών, αλλά η ερμηνεία τους εξαρτάται από το project, dataset, μονάδα ανάλυσης και το χρησιμοποιούμενο prediction model. Η ύπαρξη στατιστικής σχέσης μεταξύ complexity, defects και vulnerabilities δεν συνεπάγεται αυτομάτως ότι η πολυπλοκότητα αποτελεί από μόνη της αιτία ή πλήρη μέτρηση της ασφάλειας.

5.11 Στατιστικές Προσεγγίσεις Υπαρχουσών Εργασιών

Η εμπειρική διερεύνηση της ποιότητας και της ασφάλειας λογισμικού βασίζεται συχνά σε μεγάλο αριθμό ετερογενών μετρικών και παρατηρήσεων που προέρχονται από διαφορετικά έργα λογισμικού, εκδόσεις, commits ή άλλα artifacts της διαδικασίας ανάπτυξης. Για την ανάλυση τέτοιων δεδομένων έχουν χρησιμοποιηθεί διαφορετικές στατιστικές προσεγγίσεις, ακολουθούμενες σε κάθε περίπτωση το ερευνητικό ερώτημα και τη δομή του dataset. Ορισμένες μέθοδοι επικεντρώνονται στην εκτίμηση των σχέσεων μεταξύ μεταβλητών και στην εξέταση του κατά πόσο συγκεκριμένα χαρακτηριστικά του λογισμικού συνδέονται με quality ή security outcomes (αποτελέσματα εξαγόμενα από την έρευνα), ενώ άλλες εξ αυτών αποσκοπούν στην αναγνώριση υποκείμενων δομών μέσα σε σύνολα πολλών και συχνά αλληλοσχετιζόμενων software metrics.

Παρακάτω, παρουσιάζονται δύο βασικές κατηγορίες στατικών προσεγγίσεων ανάλυσης που έχουν ιδιαίτερη σημασία για την εμπειρική Μηχανική Λογισμικού.

Αρχικά, εξετάζεται η χρήση Regression και Mixed Effects Models, τα οποία επιτρέπουν τη μελέτη σχέσεων μεταξύ μεταβλητών και, στην περίπτωση των mixed models, τη μοντελοποίηση δεδομένων με ιεραρχική ή ομαδοποιημένη δομή. Στη συνέχεια παρουσιάζεται η χρήση των Principal Component Analysis (PCA) και Factor Analysis, οι οποίες αξιοποιούνται για τη μείωση της διαστατικότητας και τη διερεύνηση κοινών ή λανθανουσών δομών μεταξύ πολλών μετρικών λογισμικού. Η επισκόπηση αυτών των προσεγγίσεων παρέχει το απαραίτητο μεθοδολογικό υπόβαθρο για την κατανόηση των αναλυτικών επιλογών που χρησιμοποιούνται σε προηγούμενες μελέτες και αξιοποιούνται και για την παρούσα.

5.11.1 Regression και Mixed Effects Models

Η χρησιμοποίηση των regression models στην empirical software engineering καθίσταται ευρύτερη για τη διερεύνηση σχέσεων μεταξύ χαρακτηριστικών του λογισμικού και διαφορετικών outcomes. Ανάλογα με τη φύση της εξαρτημένης μεταβλητής, έχουν χρησιμοποιηθεί γραμμικά, logistic και άλλοι τύποι regression για ερωτήματα τα οποία αφορούν σε software defects, vulnerabilities, Technical Debt ή την έκβαση των Pull Requests. Μέσω των ορισμένων συντελεστών του μοντέλου είναι δυνατό να εξεταστεί η κατεύθυνση και το μέγεθος της στατιστικής σχέσης μιας μεταβλητής με το εξεταζόμενο outcome, λαμβάνοντας ταυτόχρονα υπόψη και άλλες μεταβλητές που περιλαμβάνονται στο στατιστικό μοντέλο.

Χαρακτηριστικό παράδειγμα αποτελεί η έρευνα vulnerability prediction, όπου μετρικές λογισμικού έχουν χρησιμοποιηθεί ως predictors σε regression-based models. Προηγούμενες έρευνες vulnerability prediction έχουν αξιοποιήσει την πολυπλοκότητα, την ανακύκλωση ή ρυθμό αλλαγής κώδικα (code churn) και άλλες πηγαίου κώδικα ή προγραμματιστικές μετρικές για την εκτίμηση της πιθανότητας εμφάνισης vulnerable components. Τα μοντέλα αυτά επιτρέπουν την ταυτόχρονη εξέταση περισσότερων predictors, χωρίς όμως οι παρατηρούμενες σχέσεις να αποκτούν αυτομάτως αιτιώδη ερμηνεία.

Στις repository-mining μελέτες εμφανίζεται συχνά μία επιπλέον σχετική δυσκολία, η οποία αναφέρεται στην οργάνωση των παρατηρήσεων σε ομαδοποίηση ή ιεράρχηση. Πολλά Pull Requests μπορεί να προέρχονται από το ίδιο project, πολλές μετρήσεις από τον ίδιο προγραμματιστή ή διαφορετικές παρατηρήσεις από το ίδιο αποθετήριο κώδικα. Στις περιπτώσεις αυτές, οι παρατηρήσεις της ίδια ομάδας μπορούν να μοιάζουν περισσότερο μεταξύ τους από ότι με παρατηρήσεις άλλων ομάδων. Τα Mixed Effects Models επιτρέπουν να συνδυαστούν οι γενικές επιδράσεις των predictors (fixed effects) με διαφοροποιήσεις που σχετίζονται με συγκεκριμένες ομάδες (random effects), λαμβάνοντας έτσι υπόψη μέρος της συγκεκριμένης ετερογένειας που προκύπτει.

Η περιγραφόμενη προσέγγιση χρησιμοποιείται ήδη σε μεγάλης κλίμακας Pull Request studies. Οι Zhang et al. (2023) ανέλυσαν περισσότερα από 3,3 εκατομμύρια Pull Request σε πάνω από 11.230 projects με την χρήση του mixed-effects logistic regression, με το project identifier ως random effect. Η χρησιμότητα των mixed models επεκτείνεται και πέρα από το επίπεδο του project, καθώς οι Kuuttila et al.

(2021) τα αξιοποίησαν σε longitudinal repository data, ενσωματώνοντας random effects για τις επαναλαμβανόμενες παρατηρήσεις των ίδιων προγραμματιστών. Τα προαναφερθέντα παραδείγματα δείχνουν ότι τα mixed models είναι κατάλληλα όταν οι παρατηρήσεις είναι ομαδοποιημένες ή εξαρτώμενες και όταν απαιτείται να ληφθεί υπόψη η ετερογένεια μεταξύ έργων, γλωσσών προγραμματισμού ή προγραμματιστών γενικότερα.

Συνοψίζοντας, τόσο τα συμβατικά regression models όσο και τα Mixed Effects Models αποτελούν σημαντικές προσεγγίσεις στην εμπειρική Μηχανική Λογισμικού, αλλά εξυπηρετούν διαφορετικές ανάγκες. Τα πρώτα παρέχουν ένα σχετικά απλό πλαίσιο για την εκτίμηση σχέσεων μεταξύ predictors και outcomes, ενώ τα δεύτερα επεκτείνουν αυτή τη λογική προκειμένου να λαμβάνεται υπόψη η ομαδοποίηση και η μεταξύ-ομάδων εκ φύσεως ετερογένεια. Σε δεδομένα τα οποία προέρχονται από μεγάλο αριθμό Pull Requests κατανεμημένων σε διαφορετικά μεταξύ τους έργα λογισμικού, η διάκριση αυτή είναι ιδιαίτερα σημαντική, καθώς η αγνόηση των project-specific διαφορών μπορεί να οδηγήσει σε υπεραπλουστευμένη εικόνα των σχέσεων μεταξύ των εξεταζόμενων μετρικών λογισμικού.

5.11.2 PCA και Factor Analysis σε μετρικές λογισμικού

Οι εμπειρικές μελέτες λογισμικού χρησιμοποιούν συχνά μεγάλο αριθμό από μετρικές οι οποίες περιγράφουν διαφορετικά μεταξύ τους χαρακτηριστικά του πηγαίου κώδικα, μεταξύ αυτών το μέγεθος, η πολυπλοκότητα, το coupling, cohesion και maintainability. Οι μετρικές αυτές δεν αποτελούν απαραίτητα ανεξάρτητα μεταξύ τους μεγέθη και αρκετές μπορούν να αποτυπώνουν παρόμοιες πτυχές του ίδιου χαρακτηριστικού. Η υψηλή μεταξύ τους συσχέτιση δημιουργεί δυσκολίες τόσο στην ερμηνεία όσο και στη χρήση τους σε μεταγενέστερα στατιστικά μοντέλα. Για τον λόγο αυτό τεχνικές όπως η Principal Component Analysis (PCA) και η Factor Analysis (FA) έχουν χρησιμοποιηθεί στην empirical software engineering για τη διερεύνηση και τη συμπίκνωση της πληροφορίας η οποία περιέχεται σε σύνολα software metrics. Η ύπαρξη σημαντικών correlations και πιθανής redundancy μεταξύ των μετρικών πηγαίου κώδικα έχει επιβεβαιωθεί σε μεγάλης κλίμακας εμπειρικές αναλύσεις.

Η Principal Component Analysis χρησιμοποιείται κυρίως ως τεχνική μείωσης της διαστατικότητας. Αντί να εξετάζονται πολλές συσχετισμένες μετρικές ανεξάρτητα, η PCA τις μετασχηματίζει σε μικρότερο αριθμό principal components που συγκεντρώνουν όσο το δυνατόν μεγαλύτερο μέρος της μεταβλητότητας των αρχικών δεδομένων. Η προσέγγιση αυτή έχει αξιοποιηθεί ιδιαίτερα σε μελέτες defect-prediction. Η στατιστική ανάλυση PCA χρησιμοποιήθηκε σε μελέτη από τους Nagappan, Ball και Zeller (2006) εφαρμοσμένη σε μετρικές πολυπλοκότητας πέντε διαφορετικών συστημάτων της Microsoft με στόχο την αξιοποίηση των παραγόμενων components σε regression models για την πρόβλεψη post-release failures. Η συγκεκριμένη μελέτη έδειξε επίσης ότι δεν υπήρχε ένα μοναδικό σύνολο μετρικών πολυπλοκότητας που να αποτελεί τον καλύτερο predictor για όλα τα διαφορετικά μεταξύ τους έργα λογισμικού, αναδεικνύοντας τη σημασία του project-specific context.

Επίσης, η PCA έχει χρησιμοποιηθεί για να εξεταστεί εάν διαφορετικές οικογένειες μετρικών περιέχουν πλεονάζουσα ή συμπληρωματική πληροφορία. Οι Ouellet και Badri (2024) οι οποίοι εφάρμοσαν PCA σε object-oriented metrics και centrality measures με σκοπό να προσδιορίσουν τις βασικές διαστάσεις που αποτυπώνουν από τα δύο σύνολα. Η ανάλυση ανέδειξε διαστάσεις σχετικές με size/complexity, coupling και network structure και έδειξε ότι οι δύο οικογένειες μετρικών περιείχαν τόσο κοινή όσο και συμπληρωματική πληροφορία. Σύγχρονες defect-prediction προσεγγίσεις, αντίστοιχα, συνεχίζουν να χρησιμοποιούν τη μέθοδο PCA για feature reduction και περιορισμό προβλημάτων που προκύπτουν από μεγάλο αριθμό συσχετισμένων μετρικών.

Από την άλλη, η Factor Analysis έχει διαφορετικό αλλά συγγενικό στόχο. Δεν επικεντρώνεται πρωτίστως στη δημιουργία μικρότερου αριθμού συνθετικών μεταβλητών που διατηρούν τη μεταβλητότητα των δεδομένων. Αντίθετα, χρησιμοποιείται για τη διερεύνηση του κατά πόσο οι παρατηρούμενες συσχετίσεις μεταξύ μετρικών μπορούν να εξηγηθούν από μικρότερο αριθμό υποκείμενων ή λανθανουσών διαστάσεων. Η δυνατότητα του Factor Analysis να αποκαλύπτει διαφορετικές υποκείμενες διαστάσεις μπορεί να γίνει αντιληπτή στη μελέτη των Cook και Roesch (1994). Σε 18 complexity και information-flow metrics από δέκα releases ενός βιομηχανικού real-time συστήματος, η ανάλυση ανέδειξε τέσσερις διαστάσεις, οι οποίες σχετίζονταν με τις διαστάσεις μέγεθος, control flow και διαφορετικές μορφές information flow. Το αποτέλεσμα αυτό αποτελεί χαρακτηριστικό παράδειγμα του τρόπου με τον οποίο πολλές επιμέρους μετρικές λογισμικού μπορούν να οργανωθούν σε μικρότερο αριθμό ευρύτερων constructs.

Οι τεχνικές PCA και Factor Analysis δεν αποτελούν ισοδύναμες μεταξύ τους τεχνικές, αν και μπορούν να οδηγήσουν σε μείωση και κατά συνέπεια μικρότερο αριθμό διαστάσεων. Η PCA χρησιμοποιείται κυρίως για dimensionality reduction και αντιμετωπίζει τα components ως συνδυασμούς των παρατηρούμενων μεταβλητών, σε αντίθεση με την Factor Analysis, η οποία ενδιαφέρεται περισσότερο για την ερμηνεία της κοινής τους μεταβλητότητας μέσω υποκείμενων factors. Επιπλέον, τα components ή factors που προκύπτουν δεν πρέπει να θεωρούνται καθολικές ιδιότητες του λογισμικού, καθώς εξαρτώνται από το σύνολο των μετρικών, το δείγμα, την κλίμακα και την πρότερη επεξεργασία των δεδομένων. Στην Εμπειρική Μηχανική Λογισμικού οι δύο τεχνικές μπορούν να λειτουργήσουν συμπληρωματικά. Η PCA για τη συμπύκνωση και κατάργηση συσχέτισης της πληροφορίας και η Factor Analysis για τη διερεύνηση πιθανών λανθανουσών δομών μεταξύ των software metrics.

5.12 Σύνθεση της βιβλιογραφίας και τοποθέτηση της παρούσας εργασίας

Η βιβλιογραφία η οποία εξετάστηκε στο παρόν κεφάλαιο χαρακτηρίζει τη σχέση μεταξύ των διαστάσεων της ποιότητας και ασφάλειας λογισμικού είναι

πολυδιάστατη και μη μονοσήμαντη. Χαρακτηριστικά όπως είναι η complexity, η maintainability, τα code smells, το duplication, η reliability, το Technical Debt και οι μεταβολές μέσω refactoring έχουν συνδεθεί με security-related outcomes, χωρίς όμως οι σχέσεις αυτές να είναι καθολικές. Αντίθετα, η εμπειρική έρευνα καταγραφεί τόσο θετικές συνέργειες όσο και ουδέτερες μεταβολές ή trade-offs, γεγονός που υποδεικνύει ότι η βελτίωση της μίας διάστασης δεν συνεπάγεται απαραίτητα αντίστοιχη βελτίωση της άλλης.

Οι προηγούμενες υπάρχουσες μελέτες έχουν αξιοποιήσει τεχνικές όπως repository mining, Pull Requests, code reviews, static-analysis findings, software metrics, refactoring commits και security patches για τη διερεύνηση διαφορετικών πτυχών της σχέσης μεταξύ τους. Η υπάρχουσα βιβλιογραφικά αποτυπωμένη γνώση παραμένει διαχωρισμένη μεταξύ διαφορετικών μονάδων ανάλυσης, μετρικών, εργαλείων και ερευνητικών προσεγγίσεων. Παράλληλα, η ύπαρξη πολλών αλληλοσχετιζόμενων μετρικών και η διαφοροποίηση μεταξύ έργων λογισμικού καθιστούν απαραίτητη την προσεκτική μοντελοποίηση τόσο της δομής των μεταβλητών όσο και της ετερογένειας των δεδομένων.

Η παρούσα εργασία τοποθετείται στο παραπάνω ερευνητικό πλαίσιο χρησιμοποιώντας τα Pull Requests ως μονάδα οργάνωσης της εμπειρικής παρατήρησης και εξετάζοντας παράλληλα πολλαπλές μετρικές ποιότητας, ασφάλειας και δομικών χαρακτηριστικών του αναλυόμενου πηγαίου κώδικα. Ο αρχικός ερευνητικός σχεδιασμός προέβλεπε τη συγκριτική αξιοποίηση των καταστάσεων `pr_base` και `pr_closed`, ώστε να διερευνηθεί η πιθανότητα οι μεταβολές των quality-related χαρακτηριστικών να συνοδεύονται από αντίστοιχες μεταβολές των security-related δεικτών. Η αξιολόγηση των πραγματικών δεδομένων έδειξε, αντίθετα, ότι η μεταβλητότητα μεταξύ των δύο snapshots ήταν εξαιρετικά περιορισμένη και δεν παρείχε επαρκή πληροφορία για αξιόπιστη κύρια μοντελοποίηση των within-PR διαφορών.

Η τελική εμπειρική προσέγγιση διαμορφώθηκε ως cross-sectional ανάλυση της τελικής κατάστασης των Pull Requests, με μία `pr_closed` παρατήρηση ανά PR. Η ερευνητική εστίαση μετατοπίστηκε με τον τρόπο αυτό από την άμεση εκτίμηση των μεταβολών `pr_base` $\Rightarrow$ `pr_closed` στη μελέτη των σχέσεων μεταξύ των παρατηρούμενων quality και security indicators, των χαρακτηριστικών του τελικού code snapshot, της κατηγορίας του Pull Request και του project στο οποίο κάθε παρατήρηση ανήκει. Η συγκεκριμένη προσέγγιση επιτρέπει να εξεταστεί η περίπτωση κατά την οποία δύο διαστάσεις παρουσιάζουν συστηματικές σχέσεις μεταξύ τους, χωρίς εκ των προτέρων να θεωρηθεί ότι αυτές εκφράζονται απαραίτητα μέσω παράλληλων χρονικών μεταβολών μέσα στο ίδιο Pull Request.

Μεθοδολογικά, η εργασία συνδυάζει regression models και Linear Mixed-Effects Models, ώστε να εξετάζονται οι σχέσεις μεταξύ μεταβλητών λαμβάνοντας ταυτόχρονα υπόψη την ιεραρχική οργάνωση των Pull Requests μέσα σε διαφορετικά projects. Η Principal Components Analysis και Factor Analysis παράλληλα χρησιμοποιούνται για τη διερεύνηση της πολυμεταβλητής δομής των software metrics και για την αξιολόγηση του κατά πόσο οι επιμέρους δείκτες μπορούν να συνοψιστούν σε ευρύτερες υποκείμενες διαστάσεις. Η εργασία μέσω αυτού, επιδιώκει να συμβάλει στη βιβλιογραφία μέσω της συνδυαστικής εξέτασης των

μεταξύ τους σχέσεων, της πολυδιάστατης δομής των σχετικών μετρικών και της ετερογένειας που χαρακτηρίζει τα διαφορετικά έργα λογισμικού.

5.13 Conceptual model της παρούσας έρευνας

Με βάση τη θεωρητική και εμπειρική βιβλιογραφία όπως αυτή παρουσιάστηκε στις προηγούμενες ενότητες και υποενότητές τους, η παρούσα έρευνα οργανώνεται γύρω από ένα conceptual model το οποίο συνδέει τις μετρικές των διαστάσεων ποιότητας και ασφάλειας του πηγαίου κώδικα και του λογισμικού με χαρακτηριστικά του αναλυόμενου κώδικα, την κατηγορία του Pull Request και το project στο οποίο αυτό ανήκει. Βασική μονάδα ανάλυσης παραμένει το Pull Request, το οποίο αντιμετωπίζεται ως διακριτή μονάδα της διαδικασίας εξέλιξης του λογισμικού.

Ο αρχικός ερευνητικός σχεδιασμός προέβλεπε τη συγκριτική εξέταση μεταξύ των καταστάσεων `pr_base` και `pr_closed`, με στόχο να διερευνηθούν πιθανές ταυτόχρονες μεταβολές της ποιότητας και της ασφάλειας. Ο έλεγχος, όμως, της μεταβλητότητας των διαθέσιμων μετρικών έδειξε ότι οι δύο καταστάσεις ήταν πανομοιότυπες για τη μεγάλη πλειονότητα των Pull Requests. Εν συνεχεία, οι δείκτες μεταβολής δεν παρείχαν επαρκή πληροφορία για αξιόπιστη κύρια στατιστική μοντελοποίηση και η τελική εμπειρική στρατηγική διαμορφώθηκε σε cross-sectional ανάλυση, με μία τελική παρατήρηση `pr_closed` ανά έγκυρο Pull Request με `ncloc > 0`.

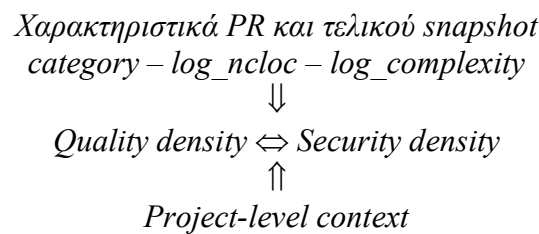
Οι δύο κύριες μεταβλητές αποτελέσματος αποτυπώνουν την πυκνότητα προβλημάτων συντηρησιμότητας και ασφάλειας ανά μονάδα KLOC, οι οποίες στην κύρια παλινδρομική ανάλυση χρησιμοποιούνται στη λογαριθμικά μετασχηματισμένη μορφή τους. Ως explanatory variables εξετάζονται το μέγεθος του αναλυόμενου πηγαίου κώδικα μέσω του `log_ncloc`, η πολυπλοκότητά του μέσω του `log_complexity` και η κατηγορία του Pull Request (quality, security, both). Οι μεταβλητές αυτές επιτρέπουν τη διερεύνηση του κατά πόσο διαφορετικά χαρακτηριστικά του τελικού snapshot συνδέονται με διαφοροποιήσεις στην παρουσία προβλημάτων ποιότητας και ασφάλειας.

Λαμβάνεται, παράλληλα, υπόψη η ιεραρχική δομή των δεδομένων, καθώς πολλαπλά Pull Requests προέρχονται από το ίδιο software project. Το project αντιμετωπίζεται επομένως ως επίπεδο ομαδοποίησης και ως πιθανή πηγή μη παρατηρούμενης ετερογένειας. Η συγκεκριμένη προσέγγιση επιτρέπει τη διάκριση μεταξύ των γενικών σχέσεων που παρατηρούνται στο σύνολο των Pull Requests και των διαφορών που συνδέονται με το ιδιαίτερο τεχνικό και οργανωτικό context κάθε project.

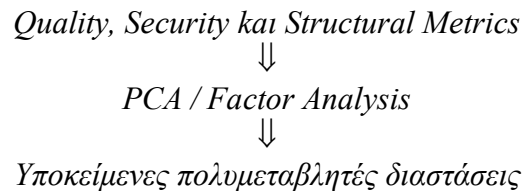
Η σχέση μεταξύ των συνιστωσών ποιότητας και ασφάλειας δεν περιορίζεται στα δύο κύρια outcomes. Λόγω του ότι οι έννοιες αποτυπώνονται μέσω πολλαπλών και συσχετισμένων μετρικών, το conceptual model περιλαμβάνει επιπλέον μία πολυμεταβλητή διάσταση, μέσω της οποίας εξετάζεται η πιθανότητα οι δείκτες

αξιοπιστίας, συντηρησιμότητας, ασφάλειας, πολυπλοκότητας, γνωστικής πολυπλοκότητας, security hotspots και duplication να οργανώνονται σε μικρότερο αριθμό κοινών ή λανθανουσών διαστάσεων. Για τον σκοπό αυτό συμπληρωματικά των κύριων μοντέλων χρησιμοποιούνται η Principal Component Analysis και η Factor Analysis.

Συνοπτικά, η τελική εννοιολογική δομή της έρευνας δύναται να αποδοθεί ως εξής:



και συμπληρωματικά:



Η εικονιζόμενη αμφίδρομη σχέση μεταξύ της ποιότητας και της ασφάλειας στο παραπάνω σχήμα εκφράζει στατιστική σχέση προς διερεύνηση και όχι θεμελιωμένη αιτιώδη επίδραση. Η επίδραση, αντίστοιχα, του project στο μοντέλο δεν συνεπάγεται ότι το project αποτελεί αιτία των παρατηρούμενων προβλημάτων, αλλά ότι οι παρατηρήσεις που προέρχονται από το ίδιο έργο ενδέχεται να μοιράζονται κοινά χαρακτηριστικά και να παρουσιάζουν συστηματική ομοιότητα. Συνεπώς, το conceptual model εκφράζει την τελική cross-sectional, ιεραρχική και πολυμεταβλητή προσέγγιση της παρούσας έρευνας.

ΜΕΡΟΣ Β

ΕΜΠΕΙΡΙΚΗ ΕΡΕΥΝΑ

Ερευνητικός σχεδιασμός, συλλογή και προετοιμασία δεδομένων

6.1 Ερευνητική προσέγγιση

Η παρούσα ερευνητική εργασία βασίζεται σε ποσοτική εμπειρική προσέγγιση, με αξιοποίηση δεδομένων που προέρχονται από πραγματικές διαδικασίες ανάπτυξης και εξέλιξης λογισμικού. Η διερεύνηση πραγματοποιείται με χρήση συστηματικής συλλογής, επεξεργασίας και στατιστικής ανάλυσης μετρήσιμων χαρακτηριστικών του πηγαίου κώδικα και των μεταβολών που παρατηρούνται κατά την ανάπτυξη και εξέλιξή του, με βασικό στόχο την εξέταση πιθανών σχέσεων μεταξύ των διαστάσεων ποιότητας και ασφάλειας.

Ο ερευνητικός σχεδιασμός έχει observational χαρακτήρα, καθώς τα υπό εξέταση δεδομένα προκύπτουν από ήδη πραγματοποιημένες αλλαγές λογισμικού και δεν εφαρμόζεται πειραματική παρέμβαση ή ελεγχόμενη μεταβολή των εξεταζόμενων παραγόντων από τον ερευνητή. Στη συνέχεια παρουσιάζονται αναλυτικότερα ο ποσοτικός και εμπειρικός χαρακτήρας της μελέτης, καθώς και οι

ιδιαιτερότητες και οι ερμηνευτικοί περιορισμοί που απορρέουν από τον observational σχεδιασμό της.

6.1.1 Ποσοτική εμπειρική έρευνα

Η παρούσα μελέτη εντάσσεται στο πεδίο της ποσοτικής εμπειρικής έρευνας στη Μηχανική Λογισμικού, καθώς βασίζεται στη συλλογή και ανάλυση μετρήσιμων δεδομένων που προέρχονται από πραγματικά συστήματα λογισμικού και από τη διαδικασία εξέλιξης του πηγαίου κώδικά τους. Κατά την *empirical software engineering* επιδιώκεται η μελέτη φαινομένων της ανάπτυξης λογισμικού μέσω συστηματικής παρατήρησης και αξιολόγησης πραγματικών δεδομένων, ενώ η επιλογή ποσοτικής προσέγγισης είναι κατάλληλη όταν τα εξεταζόμενα χαρακτηριστικά μπορούν να λειτουργήσουν ως αριθμητικές μεταβλητές και να αναλυθούν με στατιστικές μεθόδους. Η βιβλιογραφία της *empirical software engineering* αναγνωρίζει τη μέτρηση και τη στατιστική ανάλυση ως βασικά εργαλεία για την εξέταση σχέσεων και μοτίβων που εμφανίζονται στα δεδομένα λογισμικού.

Η ποσοτική εμπειρική έρευνα εξετάζει υποθέσεις μέσω συστηματικά συλλεγμένων και αναλύσιμων δεδομένων. Στη μηχανική λογισμικού, η επιλογή σχεδιασμού πρέπει να συνδέεται με το ερευνητικό ερώτημα, τη μονάδα ανάλυσης και τις απειλές εγκυρότητας (Wohlin et al., 2024). Χαρακτηριστικά των Pull Requests, παράλληλα, καθώς και η ένταξή τους σε διαφορετικά έργα λογισμικού παρέχουν πρόσθετες μεταβλητές για τη διερεύνηση των παρατηρούμενων διαφοροποιήσεων.

Η επεξεργασία των δεδομένων πραγματοποιείται με στατιστικές και πολυμεταβλητές τεχνικές ανάλυσης. Η εκτελεσμένη μελέτη συνδυάζει περιγραφική ανάλυση με regression και Mixed Effects Models για τη διερεύνηση σχέσεων μεταξύ των εξεταζόμενων μεταβλητών, ενώ η Principal Component Analysis και Factor Analysis χρησιμοποιούνται για τη διερεύνηση της δομής ενός συνόλου σχετιζόμενων μεταξύ τους μετρικών λογισμικού. Η χρήση διαφορετικών πολλαπλών στατιστικών προσεγγίσεων επιτρέπει την εξέταση του ερευνητικού προβλήματος τόσο σε επίπεδο επιμέρους μετρικών όσο και σε επίπεδο ευρύτερων δομών που μπορεί να εμφανίζονται μεταξύ τους. Η αξιοποίηση στατιστικών μεθόδων για την ανάλυση δεδομένων πραγματικών artifacts λογισμικού αποτελεί μία καθιερωμένη πλέον πρακτική στην *empirical software engineering*, με την επιλογή κάθε τεχνικής να εξαρτάται από το ερευνητικό ερώτημα και τη δομή των δεδομένων.

Ο χαρακτηρισμός της έρευνας ως ποσοτικής αφορά τόσο τη φύση των, εξαγόμενων από την διαδικασία συλλογής, δεδομένων όσο και τον τρόπο με τον οποίο διερευνώνται τα ερευνητικά ερωτήματα. Στόχος δεν είναι η ποιοτική ερμηνεία μεμονωμένων αλλαγών κώδικα, αλλά η αναγνώριση συστηματικών μοτίβων και στατιστικών σχέσεων σε ένα ευρύτερο σύνολο παρατηρήσεων. Σημειώνεται ότι, η ποσοτική φύση της μελέτης δεν συνεπάγεται αυτομάτως ότι πρόκειται για ελεγχόμενο πείραμα ούτε ότι οι παρατηρούμενες στατιστικές σχέσεις μπορούν να ερμηνευθούν αυτομάτως ως αιτιώδεις. Η διάσταση αυτή απορρέει από τον observational χαρακτήρα του παρόντος ερευνητικού σχεδιασμού.

6.1.2 *Observational study*

Ο χαρακτηρισμός ο οποίος λαμβάνει η παρούσα έρευνα είναι *observational study*, καθώς βασίζεται στην παρατήρηση και ανάλυση δεδομένων που προέκυψαν από πραγματικές διαδικασίες ανάπτυξης λογισμικού, χωρίς πειραματική παρέμβαση στις εξεταζόμενες μεταβλητές. Τα PRs, οι μεταβολές του πηγαίου κώδικα και τα αντίστοιχα *quality* και *security-related* χαρακτηριστικά είχαν ήδη δημιουργηθεί στο πλαίσιο της φυσιολογικής εξέλιξης των εξεταζόμενων, στο πλαίσιο της εργασίας, έργων λογισμικού. Δεν έχουν καθοριστεί, εκ του ερευνητή, ποιες αλλαγές επρόκειτο να πραγματοποιηθούν ούτε ανέθεσε διαφορετικές συνθήκες σε *projects* ή *Pull Requests*, αλλά αξιοποίησε τις ήδη υπάρχουσες παρατηρήσεις για τη διερεύνηση των ερευνητικών ερωτημάτων. Η μελέτη πραγματικών φαινομένων στο φυσικό τους περιβάλλον αποτελεί καθιερωμένη κατεύθυνση της *empirical software engineering* και επιτρέπει την εξέταση της ανάπτυξης λογισμικού όπως αυτή πραγματοποιείται στην πράξη.

Η παρούσα μελέτη είναι *observational*, επειδή αναλύει αλλαγές που προέκυψαν σε πραγματικές διαδικασίες ανάπτυξης χωρίς πειραματική ανάθεση των PRs ή χειρισμό των *predictors*. Συνεπώς, τα αποτελέσματα τεκμηριώνουν συσχετίσεις και όχι αιτιώδεις επιδράσεις (Wohlin et al., 2024). Χαρακτηριστικά του *Pull Request*, ιδιαιτερότητες του *Software Project*, *development practices* και άλλοι μετρημένοι ή μη μετρημένοι παράγοντες είναι δυνατό να συνδέονται τόσο με τους *quality* όσο και με τους *security indicators*.

Τα αποτελέσματα της παρούσας μελέτης, ερμηνεύονται πρωτίστως ως στατιστικές σχέσεις και όχι ως αποδείξεις αιτιότητας. Η βιβλιογραφία επί της θεματολογίας της *causal inference* στη Μηχανική Λογισμικού επισημαίνει ότι η εκτίμηση πραγματικών *causal effects* από *observational data* απαιτεί πρόσθετες υποθέσεις σχετικά με την αιτιώδη δομή των δεδομένων και την αντιμετώπιση πιθανών *confounding factors*. Ακόμη και η χρήση *regression* ή άλλων πολυμεταβλητών μοντέλων δεν μετατρέπει από μόνη της μία *observational* ανάλυση σε *causal*, ιδιαίτερα σε περιπτώσεις όπου μπορεί να υπάρχουν μη παρατηρούμενες μεταβλητές ή *omitted-variable bias*.

Στην παρούσα εργασία, οι στατιστικές τεχνικές χρησιμοποιούνται για να διερευνηθεί εάν και με ποιον τρόπο οι παρατηρούμενες μεταβολές των χαρακτηριστικών ποιότητας και ασφάλειας συνδέονται μεταξύ τους, λαμβάνοντας παράλληλα υπόψη τη δομή και την ετερογένεια των δεδομένων. Ο *observational* σχεδιασμός της έρευνας επιτρέπει έτσι την εμπειρική μελέτη της πραγματικής εξέλιξης του λογισμικού, ενώ τα συμπεράσματα διατυπώνονται με την απαραίτητη επιφύλαξη ως προς την αιτιώδη ερμηνεία των παρατηρούμενων σχέσεων.

6.2 Μονάδα ανάλυσης: Pull Request

Η μονάδα ανάλυσης (unit of analysis) που ορίζεται στην παρούσα εργασία είναι το Pull Request (PR). Κάθε παρατήρηση του αναλυτικού συνόλου δεδομένων αντιστοιχεί σε μία συγκεκριμένη και οριοθετημένη διαδικασία μεταβολής του πηγαίου κώδικα μέσα σε ένα software project. Η επιλογή αυτή επιτρέπει τη μελέτη ποιότητας και της ασφάλειας στο επίπεδο πραγματικών αλλαγών του λογισμικού και όχι σε επίπεδο ολόκληρων αποθετηρίων κώδικα ή μεμονωμένων χρονικών στιγμών αποκλειστικά. Η χρήση Pull Requests ως βασικής μονάδας εμπειρικής ανάλυσης έχει αξιοποιηθεί ευρέως στη σχετική βιβλιογραφία για τη μελέτη της εξέλιξης του κώδικα, του code review και χαρακτηριστικών ποιότητας.

Εδώ, για κάθε PR, εξετάζονται δύο καταστάσεις του πηγαίου κώδικα ενός λογισμικού, οι οποίες αναπαρίστανται από τα snapshots περιγραφόμενα ως *pr_base* και *pr_closed*. Στις δύο αυτές καταστάσεις εξάγονται οι διαθέσιμες software metrics και τα automated findings, ώστε μέσω της σύγκρισής τους να αποτυπωθεί η μεταβολή που σχετίζεται με το συγκεκριμένο PR. Τα *pr_base* και *pr_closed* δεν αποτελούν επομένως ανεξάρτητες μονάδες ανάλυσης, αλλά δύο μετρήσεις του ίδιου Pull Request, πάνω στις οποίες βασίζεται ο υπολογισμός των μεταβολών των quality και security-related χαρακτηριστικών.

Η συγκεκριμένη επιλογή ως μονάδα μέτρησης προσφέρει ένα ενδιάμεσο επίπεδο ανάλυσης μεταξύ του μεμονωμένου commit και του συνολικού repository. Ένα PR μπορεί να περιλαμβάνει περισσότερα από ένα commits (1 ή περισσότερα) και να τροποποιεί αντίστοιχα ένα ή περισσότερα αρχεία, αλλά εξακολουθεί να αποτελεί μία ενιαία αλλαγή στο πλαίσιο της διαδικασίας ανάπτυξης. Με αυτόν τον τρόπο, οι μεταβολές των software metrics μπορούν να συσχετιστούν με συγκεκριμένες αλλαγές του κώδικα, χωρίς να χάνεται το ευρύτερο project context στο οποίο και πραγματοποιούνται.

Τα Pull Requests παράλληλα, δεν θεωρούνται πλήρως ανεξάρτητα μεταξύ τους, καθώς πολλαπλά PRs ανήκουν στο ίδιο έργο, γεγονός το οποίο συνεπάγεται διαμοιρασμό κοινών χαρακτηριστικών, όπως είναι η αρχιτεκτονική, οι χρησιμοποιούμενες τεχνολογίες και οι προγραμματιστικές πρακτικές/τεχνικές. Αυτή η ιεραρχική δομή που εμφανίζεται στα συλλεγμένα δεδομένα, με τα Pull Requests να είναι ενσωματωμένα σε διαφορετικά projects, λαμβάνεται υπόψη στη μεταγενέστερη στατιστική ανάλυση. Στο πλαίσιο της παρούσας έρευνας, το PR αντιμετωπίζεται ως βασική εμπειρικά παρατηρήσιμη μονάδα μεταβολής, πάνω στην οποία διερευνώνται οι σχέσεις μεταξύ των χαρακτηριστικών της αλλαγής και των μεταβολών των δεικτών που αντιπροσωπεύουν την ποιότητα και την ασφάλεια του πηγαίου κώδικα και του λογισμικού.

6.3 Ιεραρχική δομή των δεδομένων

Τα δεδομένα τα οποία χρησιμοποιήθηκαν για τη διεξαγωγή της παρούσας έρευνας δεν αποτελούν ένα σύνολο πλήρως ανεξάρτητων μεταξύ τους παρατηρήσεων, αλλά παρουσιάζουν ιεραρχική δομή. Η βασική μονάδα ανάλυσης είναι το Pull Request, όμως κάθε PR ανήκει σε ένα συγκεκριμένο software project και επομένως επηρεάζεται από το ευρύτερο περιβάλλον μέσα στο οποίο πραγματοποιείται η αντίστοιχη αλλαγή του πηγαίου κώδικα.



Εικόνα 6.3.1 – Ιεραρχική δομή δεδομένων έρευνας *

Η συγκεκριμένη οργάνωση, όπως ορίζεται, των δεδομένων δημιουργεί δύο επίπεδα πληροφορίας. Στο πρώτο επίπεδο βρίσκονται τα Pull Requests, με τα ιδιαίτερα χαρακτηριστικά και τις μεταβολές που παρατηρούνται σε κάθε αλλαγή. Στο δεύτερο επίπεδο των δεδομένων βρίσκεται το project, το οποίο μπορεί να παρουσιάζει δικά του σταθερά ή μεταβαλλόμενα χαρακτηριστικά που διαφοροποιούν συστηματικά τις παρατηρήσεις που προέρχονται από αυτό. Ως αποτέλεσμα αυτού, Pull Requests που ανήκουν στο ίδιο project είναι πιθανό να εμφανίζουν μεγαλύτερη μεταξύ τους ομοιότητα σε σχέση με Pull Requests διαφορετικών έργων.

Η αναγνώριση αυτής της ιεραρχικής δομής είναι σημαντική τόσο για την περιγραφή του dataset όσο και για τη μεταγενέστερη στατιστική μοντελοποίηση, καθώς η αγνόηση της εξάρτησης μεταξύ παρατηρήσεων του ίδιου project μπορεί να οδηγήσει σε υπεραπλουστευμένη εικόνα των εξεταζόμενων σχέσεων. Παρακάτω παρουσιάζεται ειδικότερα η οργάνωση των Pull Requests εντός των projects, καθώς και η ύπαρξη project-specific events και διαφοροποιήσεων που αποτελούν μέρος του ευρύτερου context κάθε project.

6.3.1 Pull Requests εντός projects

Στην παρούσα έρευνα, κάθε Pull Request ανήκει σε ένα συγκεκριμένο software project, ενώ κάθε project μπορεί να περιλαμβάνει περισσότερα από ένα Pull Requests. Η δομή αυτή δημιουργεί μία φυσική ιεραρχία, κατά την οποία τα Pull Requests αποτελούν το χαμηλότερο επίπεδο παρατήρησης και τα projects το ανώτερο επίπεδο ομαδοποίησης.

Επειδή πολλά Pull Requests ανήκουν στο ίδιο project, οι παρατηρήσεις σχηματίζουν ιεραρχική δομή και ενδέχεται να εμφανίζουν ενδοομαδική συσχέτιση. Τα mixed-effects models επιτρέπουν τη ρητή μοντελοποίηση αυτής της εξάρτησης μέσω project-level random effects (Bates et al., 2015). Για τον λόγο αυτό, δύο PRs του ίδιου project είναι αρκετά πιθανό να παρουσιάζουν μεγαλύτερη ομοιότητα από ότι PRs που προέρχονται από διαφορετικά.

Η ύπαρξη αυτής της συσχέτισης εσωτερικά του project σημαίνει ότι η παραδοχή της πλήρους ανεξαρτησίας των παρατηρήσεων δεν είναι αναγκαστικά κατάλληλη για όλα τα στάδια της στατιστικής ανάλυσης. Αν αγνοηθεί η κοινή προέλευσή τους, μέρος της μεταβλητότητας που οφείλεται στο περιβάλλον του ίδιου project μπορεί να αποδοθεί λανθασμένα στα χαρακτηριστικά των επιμέρους PRs ή στις εξεταζόμενες quality και security-related μεταβολές.

Η ιεραρχική ένταξη των PRs στα projects λαμβάνεται ρητά υπόψη στη μεθοδολογία της έρευνας. Η συγκεκριμένη δομή αποτελεί τη βάση για τη μεταγενέστερη χρήση Mixed Effects Models, μέσω των οποίων μπορούν να διαχωριστούν οι γενικές σχέσεις μεταξύ των εξεταζόμενων μεταβλητών από μέρους της ετερογένειας που σχετίζονται με το project όπου και ανήκει το κάθε Pull Request.

6.3.2 Project-specific effects

Η ένταξη πολλών Pull Requests στο ίδιο έργο λογισμικού σημαίνει ότι μέρος της μεταβλητότητας των συνολικά παρατηρούμενων χαρακτηριστικών μπορεί να σχετίζεται όχι με το κάθε Pull Request μεμονωμένα, αλλά με το project στο οποίο αυτό ανήκει. Στην παρούσα έρευνα, η οποία αναλύεται εδώ, οι διαφοροποιήσεις αυτές αναφέρονται ως project-specific effects και εκφράζουν την δυνατότητα διαφορετικά επίπεδα στα εξεταζόμενα quality και security outcomes, ακόμη και όταν συγκρίνονται παρατηρήσεις με παρόμοια μετρήσιμα χαρακτηριστικά. Η κύρια random-intercept προδιαγραφή αποτυπώνει διαφορές στο βασικό επίπεδο των outcomes μεταξύ projects και δεν συνεπάγεται ότι οι κλίσεις των σχέσεων μεταξύ των predictors και των outcomes μεταβάλλονται επίσης ανά project. Η διερεύνηση τέτοιας slope heterogeneity απαιτεί κατάλληλα random-slope ή interaction models.

Οι υπαρκτές διαφοροποιήσεις μεταξύ των projects μπορούν να προέρχονται από ένα ευρύ σύνολο χαρακτηριστικών, όπως η αρχιτεκτονική και το μέγεθος του συστήματος, οι χρησιμοποιούμενες τεχνολογίες και γλώσσες προγραμματισμού, οι πρακτικές ανάπτυξης και code review, η ωριμότητα του project ή άλλες τεχνικές και οργανωτικές ιδιαιτερότητες. Όλα τα παραπάνω χαρακτηριστικά, δεν κρίνεται απαραίτητο να παρατηρούνται ή να περιλαμβάνονται ως ξεχωριστές μεταβλητές στο dataset. Η κοινή, όμως, προέλευση μπορεί να δημιουργεί μία project-level συστηματική διαφοροποίηση, η οποία πρέπει να διαχωρίζεται από τη μεταβλητότητα που εμφανίζεται μεταξύ των PRs του ίδιου project.

Στην παρούσα έρευνα, η συγκεκριμένη λογική λαμβάνεται υπόψη κατά τη στατιστική μοντελοποίηση με χρήση των Mixed Effects Models, όπου το project λειτουργεί ως παράγοντας ομαδοποίησης και επιτρέπει την αποτύπωση της μεταξύ-project ετερογένειας. Με αυτόν τον τρόπο, οι γενικότερες σχέσεις μεταξύ των

εξεταζόμενων μεταβλητών μπορούν να εκτιμηθούν χωρίς να θεωρείται ότι όλα τα projects έχουν αναγκαστικά το ίδιο βασικό επίπεδο συμπεριφοράς. Ο όρος *project-specific effects* χρησιμοποιείται, άρα, για να περιγράψει αυτές τις στατιστικές διαφοροποιήσεις μεταξύ έργων λογισμικού και όχι αποδεδειγμένες αιτιώδεις επιδράσεις του project στις μεταβολές ποιότητας ή ασφάλειας.

6.4 Population, sample και διαδικασία επιλογής

Η εμπειρική ανάλυση της παρούσας έρευνας βασίζεται σε ένα δείγμα *software projects* και *Pull Requests*, το οποίο προέκυψε μέσα από διαδοχικά στάδια επιλογής και φιλτραρίσματος. Η διάκριση μεταξύ του ευρύτερου πληθυσμού ενδιαφέροντος και του τελικού αναλυτικού δείγματος είναι σημαντική, αφού τα αποτελέσματα της μελέτης προκύπτουν από συγκεκριμένο σύνολο *repositories* και αλλαγών πηγαίου κώδικα. Επομένως, δεν θα πρέπει να θεωρούνται αυτομάτως αντιπροσωπευτικά για κάθε πιθανό *software project* ή διαδικασία ανάπτυξης.

Η διαδικασία διαμόρφωσης του συνολικού δείγματος πραγματοποιήθηκε σε δύο βασικά επίπεδα, όπως έχει ήδη εισαχθεί παραπάνω. Αρχικά εφαρμόστηκε διαδικασία *repository selection*, μέσω της οποίας προσδιορίστηκαν τα *software projects* που μπορούσαν να ενταχθούν στην έρευνα βάσει των καθορισμένων προϋποθέσεων. Στη συνέχεια, στο εσωτερικό των επιλεγμένων έργων λογισμικού εφαρμόστηκαν συγκεκριμένα κριτήρια *inclusion* και *exclusion* για κάθε δεδομένο *Pull Request*. Στόχος του ήταν το τελικό *dataset* να διατηρηθούν μόνο οι παρατηρήσεις που ήταν κατάλληλες για τη συλλογή των απαιτούμενων δεδομένων και τη μεταγενέστερη στατιστική ανάλυση.

Η διαδοχική αυτή διαδικασία είναι ουσιώδης για την ερμηνεία του τελικού δείγματος. Αυτό συμβαίνει αφού οι αποφάσεις επιλογής των *repositories* και των *Pull Requests* καθορίζουν το πεδίο στο οποίο μπορούν να γενικευθούν τα αποτελέσματα της έρευνας. Παρακάτω, επομένως, παρουσιάζονται η διαδικασία επιλογής των *repositories* και τα κριτήρια ένταξης και αποκλεισμού των *Pull Requests*. Στόχος του είναι να καθοριστεί σαφώς ο τρόπος με τον οποίο προέκυψε το τελικό σύνολο δεδομένων που χρησιμοποιήθηκε στις αναλύσεις.

6.4.1 Repository selection

Η διαδικασία συγκρότησης του δείγματος ξεκίνησε από μία αρχική δεξαμενή με πλήθος 2.839 *repositories*, τα οποία εντοπίστηκαν μέσω των συστημάτων GitHub και GitHub API. Η επιλογή των *repositories* πραγματοποιήθηκε σκόπιμα, με στόχο το ερευνητικό δείγμα να αποτελείται από πραγματικά έργα λογισμικού που αντανakλούν συνθήκες πραγματικής ανάπτυξης και χρήσης και όχι από πειραματικά, εκπαιδευτικά ή ανενεργά *repositories*.

Για την ένταξη ενός repository εξετάστηκαν τρεις βασικές προϋποθέσεις οι οποίες είναι να είναι δημόσια διαθέσιμο, να αποτελεί πραγματικό και ενεργό software project και να χρησιμοποιείται σε production περιβάλλον. Τα κριτήρια αυτά επιλέχθηκαν με στόχο τα δεδομένα που θα συλλέγονταν να προέρχονται από συστήματα τα οποία αναπτύσσονται και χρησιμοποιούνται στην πράξη, ενισχύοντας τη σύνδεση της έρευνας με πραγματικές διαδικασίες εξέλιξης λογισμικού.

Η διαδικασία επιλογής είχε επομένως purposive χαρακτήρα και δεν αποτέλεσε τυχαία δειγματοληψία από το σύνολο των διαθέσιμων GitHub repositories. Η επιλογή βασίστηκε στη συνάφεια των projects με τους στόχους της έρευνας που έχουν τεθεί και στην καταλληλότητά τους για την ανάλυση πραγματικών αλλαγών πηγαίου κώδικα μέσω Pull Requests. Δεν έχει εφαρμοστεί ξεχωριστό στάδιο αποκλεισμού repositories μετά την αρχική επιλογή τους βάσει των συγκεκριμένων κριτηρίων.

Τελικά, το σύνολο δεδομένων το οποίο χρησιμοποιήθηκε στην εμπειρική ανάλυση περιλαμβάνει 242 software projects. Η χρήση πολλαπλών projects περιορίζει την εξάρτηση των αποτελεσμάτων από τις ιδιαιτερότητες ενός μεμονωμένου repository και επιτρέπει παράλληλα τη διερεύνηση της ετερογένειας μεταξύ διαφορετικών περιβαλλόντων ανάπτυξης. Εντούτοις λόγω της σκόπιμης διαδικασίας επιλογής, το δείγμα δεν πρέπει να θεωρείται τυχαία ή πλήρως αντιπροσωπευτική αναπαράσταση του συνόλου των software repositories του GitHub, γεγονός που λαμβάνεται υπόψη κατά την ερμηνεία και τη γενίκευση των αποτελεσμάτων.

6.4.2 PR inclusion/exclusion criteria

Αφού επιλέχθηκε το πλήθος των repositories προς εξέταση εφαρμόστηκε στη συνέχεια διαδικασία φιλτραρίσματος σε επίπεδο Pull Request, προκειμένου το τελικό αναλυτικό δείγμα να περιλαμβάνει παρατηρήσεις με συνεπή δομή και διαθέσιμη πληροφορία για τις μεταβλητές που απαιτούνταν στις στατιστικές αναλύσεις. Τα inclusion και exclusion κριτήρια εφαρμόστηκαν όχι ως αυθαίρετος περιορισμός του δείγματος, αλλά ως μέρος της διαδικασίας διασφάλισης της δυνατότητας σύγκρισης και της εγκυρότητας των δεδομένων.

Αρχικά, για κάθε Pull Request απαιτήθηκε η ύπαρξη πλήρους ζεύγους παρατηρήσεων *pr_base* και *pr_closed*. Τα δύο snapshots χρησιμοποιήθηκαν κατά τον έλεγχο και την προετοιμασία των δεδομένων, ενώ PRs που δεν παρουσίαζαν την αναμενόμενη δομή των δύο εγγραφών αποκλείστηκαν. Μετά τον έλεγχο των ζευγών, για το κύριο αναλυτικό dataset διατηρήθηκε μία παρατήρηση ανά Pull Request, η κατάσταση *pr_closed*. Αυτό απέτρεψε τον τεχνητό διπλασιασμό των ίδιων PRs ως ανεξάρτητων παρατηρήσεων και βασίστηκε στον προηγούμενο έλεγχο των δεδομένων, ο οποίος έδειξε ιδιαίτερα μεγάλη ομοιότητα μεταξύ των metrics των δύο snapshots για το μεγαλύτερο μέρος των Pull Requests.

Επιπλέον, διατηρήθηκαν αποκλειστικά το πλήθος των Pull Requests για τα οποία η τιμή NCLOC ήταν μεγαλύτερη του μηδενός ($ncloc > 0$). Παρατηρήσεις με μηδενικό NCLOC αντιστοιχούσαν πρακτικά σε περιπτώσεις όπου η στατική ανάλυση

δεν είχε καταγράψει αναλύσιμο πηγαίο κώδικα, όπως μπορεί να συμβεί σε αλλαγές οι οποίες αφορούν αποκλειστικά documentation ή άλλα μη αναλύσιμα artifacts. Η εξαίρεσή τους από το συνολικό δείγμα κρίθηκε, επίσης, απαραίτητη επειδή στην ανάλυση χρησιμοποιούνται δείκτες πυκνότητας issues ανά KLOC, οι οποίοι δεν μπορούν να οριστούν όταν ο αριθμός των γραμμών κώδικα είναι μηδενικός.

Τα Pull Requests επίσης ήταν απαραίτητο να αντιστοιχιστούν σε μία από τις τρεις κατηγορίες που χρησιμοποιήθηκαν στην ανάλυση, quality-security-both. Η αντιστοίχιση πραγματοποιήθηκε βάσει των διαθέσιμων κατηγοριοποιήσεων των δεδομένων, ενώ παρατηρήσεις που δεν αντιστοιχούσαν σε quality ή security-related αλλαγή, όπως περιπτώσεις αποκλειστικά σχετικές με documentation, δεν συμπεριλήφθηκαν στο τελικό αναλυτικό σύνολο.

Ένα Pull Request μπορούσε να ενταχθεί στο κύριο analytical dataset όταν διέθετε την απαιτούμενη δομή των snapshots, έγκυρη τελική παρατήρηση pr_closed, θετικό αριθμό NCLOC και κατηγορία σχετική με quality ή/και security. Αποκλείστηκαν, αντίστοιχα PRs με μη τυπική προκαθορισμένη ή ελλιπή δομή εγγραφών, μηδενικό αναλύσιμο κώδικα ή κατηγορία εκτός του ερευνητικού πεδίου. Ως αποτέλεσμα, το τελικό δείγμα περιορίστηκε στις παρατηρήσεις για τις οποίες μπορούσαν να υπολογιστούν και να συγκριθούν με συνεπή τρόπο οι μεταβλητές που χρησιμοποιήθηκαν στις επόμενες αναλύσεις.

6.5 Ερευνητικό pipeline

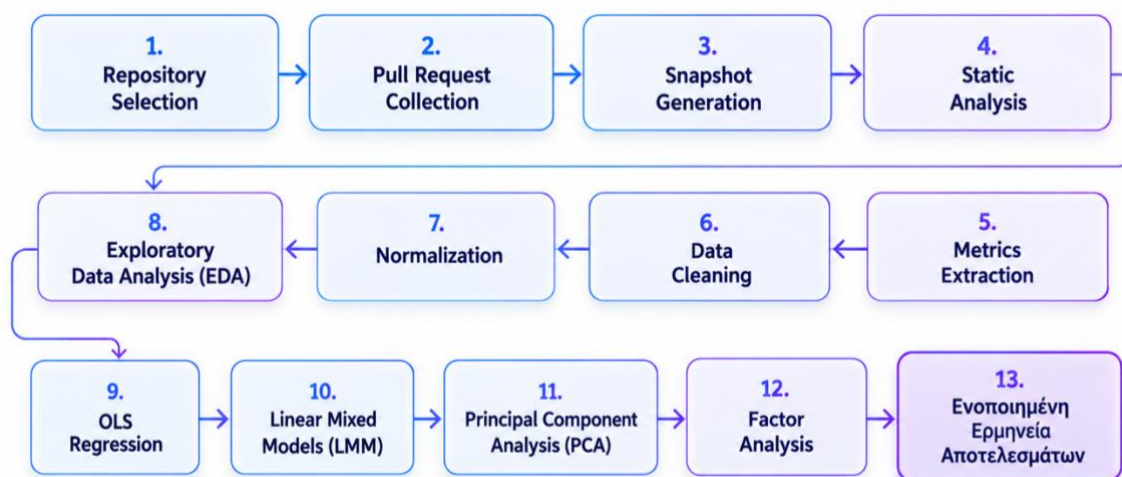
Η εμπειρική έρευνα υλοποιήθηκε με την χρήση διαδοχικής διαδικασίας συλλογής, επεξεργασίας και ανάλυσης δεδομένων, η οποία συνδέει τα αρχικά software repositories με τα τελικά στατιστικά αποτελέσματα. Η χρήση ενός οργανωμένου ερευνητικού pipeline ήταν απαραίτητη προκειμένου τα δεδομένα να μετασχηματιστούν με συστηματικό τρόπο από ακατέργαστες πληροφορίες σχετικά με repositories και Pull Requests σε ένα αναλυτικό σύνολο δεδομένων κατάλληλο για τη διερεύνηση των ερευνητικών ερωτημάτων.

Η διαδικασία ξεκίνησε με την επιλογή των repositories και τη συλλογή των αντίστοιχων Pull Requests και των διαθέσιμων μεταδεδομένων τους. Για κάθε επιλεγμένο Pull Request δημιουργήθηκαν οι απαιτούμενες καταστάσεις του πηγαίου κώδικα μέσω της διαδικασίας snapshot generation. Στη συνέχεια, τα παραγόμενα snapshots υποβλήθηκαν σε στατική ανάλυση, από την οποία εξήχθησαν οι μετρικές λογισμικού και τα quality/security-related findings που αποτέλεσαν τη βάση του ερευνητικού dataset. Η σχεδιασμένη διαδικασία συλλογής περιλαμβάνει επίσης την αντιστοίχιση των snapshots και ελέγχους ακεραιότητας των δεδομένων, όπως ελέγχους για αποτυχημένα scans, duplicated records και παρατηρήσεις με μηδενικό NCLOC.

Μετά τη συλλογή τους, ακολούθησαν τα στάδια data cleaning και normalization, ώστε να διαμορφωθεί ένα συνεπές σύνολο παρατηρήσεων και να περιοριστεί η επίδραση διαφορών που σχετίζονται, μεταξύ άλλων, με το μέγεθος του

αναλυόμενου κώδικα. Στο ερευνητικό πλαίσιο προβλέπεται ειδικότερα η χρήση κανονικοποιημένων μετρικών, όπως *issue densities* ως προς το NCLOC, όπου αυτό κρίνεται απαραίτητο για την επίτευξη της σύγκρισης των παρατηρήσεων.

Το αναλυτικότερο στάδιο ξεκίνησε με Exploratory Data Analysis (EDA) για την αρχική διερεύνηση των κατανομών, των συσχετίσεων και των βασικών χαρακτηριστικών του dataset. Στη συνέχεια εφαρμόστηκαν τεχνικές Ordinary Least Squares regression models (OLS) για τη διερεύνηση σχέσεων μεταξύ των μεταβλητών και Linear Mixed Models (LMM) με σκοπό να ληφθεί υπόψη η ιεραρχική οργάνωση των Pull Requests μέσα στα διαφορετικά projects. Η ανάλυση, στην συνέχεια, συμπληρώθηκε με Principal Component Analysis (PCA) για τη διερεύνηση της δυνατότητας μείωσης της διαστατικότητας των συσχετισμένων μετρικών και Factor Analysis για την αναζήτηση πιθανών λανθανουσών δομών πίσω από τις παρατηρούμενες μεταβλητές. Οι διαφορετικές αυτές μέθοδοι δεν αντιμετωπίζονται ως ανεξάρτητες μεταξύ τους αναλύσεις, αλλά ως συμπληρωματικά στάδια μίας ενιαίας διαδικασίας διερεύνησης της σχέσης μεταξύ ποιότητας και ασφάλειας.



Εικόνα 6.5.1 – Ερευνητικό Pipeline *

Η παραπάνω ακολουθία λειτουργεί ως ο συνολικός χάρτης της εμπειρικής διαδικασίας ανάλυσης. Τα επιμέρους στάδια της συλλογής και προετοιμασίας των δεδομένων, η κατασκευή των μεταβλητών και οι στατιστικές τεχνικές αναλύονται αναλυτικότερα παρακάτω στη βιβλιογραφία, ώστε να είναι σαφής τόσο η προέλευση των δεδομένων όσο και ο τρόπος με τον οποίο προέκυψαν τα τελικά αποτελέσματα.

6.6 Συλλογή δεδομένων Pull Requests

Αφού επιλέχθηκαν τα repositories, στη συνέχεια πραγματοποιήθηκε η συλλογή των Pull Requests που ήταν διαθέσιμα για τα αντίστοιχα software projects.

Η ανάκτηση των δεδομένων πραγματοποιήθηκε με χρήση του GitHub και το GitHub API, ώστε η συλλογή να βασίζεται σε δομημένες πληροφορίες που παρέχονται από την ίδια την πλατφόρμα φιλοξενίας των repositories των έργων λογισμικού. Τα Pull Requests αποτέλεσαν το βασικό σημείο σύνδεσης μεταξύ της ιστορικής εξέλιξης κάθε project και των μεταβολών του πηγαίου κώδικά τους οι οποίες και εξετάστηκαν στη συνέχεια.

Για κάθε ένα από τα Pull Requests συλλέχθηκαν οι πληροφορίες που ήταν απαραίτητες τόσο για τη μοναδική αναγνώρισή του όσο και για τη μεταγενέστερη επεξεργασία του. Σε αυτές περιλαμβάνονταν στοιχεία τα οποία αφορούσαν το repository αλλά και το ίδιο το PR, καθώς και διαθέσιμα μεταδεδομένα (ημερομηνία δημιουργίας, τίτλος, αναγνωριστικό, κ.α.) σχετικά με την αλλαγή του κώδικα. Οι πληροφορίες αυτές στη συνέχεια χρησιμοποιήθηκαν αργότερα για την αντιστοίχιση κάθε παρατήρησής με το project στο οποίο ανήκε, την ανακατασκευή των απαιτούμενων καταστάσεων του πηγαίου κώδικα και τη σύνδεση των αποτελεσμάτων της στατικής ανάλυσης με το αντίστοιχο Pull Request.

Κατά το συγκεκριμένο στάδιο φιλτραρίσματος και συλλογής συλλέχθηκαν κυρίως repository και Pull Request data, ενώ οι μετρικές ποιότητας και ασφάλειας που χρησιμοποιήθηκαν στην εμπειρική ανάλυση δεν προέρχονταν απευθείας από το GitHub API, πράγμα που αποτελεί σημαντική διάκριση. Οι συγκεκριμένες μετρικές προέκυψαν σε επόμενο στάδιο, μετά την ανακατασκευή και τη στατική ανάλυση του πηγαίου κώδικα. Με τον τρόπο αυτό διατηρείται ορισμένος διαχωρισμός μεταξύ των δεδομένων που περιγράφουν το PR ως development artifact και των μετρήσεων που παράγοντα από την ανάλυση του αντίστοιχου κώδικα.

Η συλλογή των PRs δημιούργησε επομένως την αρχική βάση πάνω στην οποία στηρίχθηκαν τα επόμενα στάδια του ερευνητικού pipeline. Τα δεδομένα τα οποία αφού ανακτήθηκαν δεν χρησιμοποιήθηκαν αυτούσια ως τελικό analytical dataset, αλλά υποβλήθηκαν στη συνέχεια σε διαδικασίες snapshot generation, static analysis, αντιστοίχισης, ελέγχου ακεραιότητας και φιλτραρίσματος, πριν διαμορφωθεί το σύνολο παρατηρήσεων που χρησιμοποιήθηκε στις στατιστικές αναλύσεις.

6.6.1 Κατηγοριοποίηση των Pull Requests

Η κατηγοριοποίηση των Pull Requests πραγματοποιήθηκε ως διακριτό στάδιο της διαδικασίας συλλογής, προετοιμασίας και φιλτραρίσματος των δεδομένων και είχε ως στόχο την αποτύπωση του κύριου τεχνικού προσανατολισμού κάθε εξεταζόμενης αλλαγής ως προς την ποιότητα και την ασφάλεια. Η μεταβλητή αυτή (Category) του Pull Request δεν προέκυψε από τις μετρικές του εργαλείου SonarQube που χρησιμοποιήθηκαν αργότερα ως outcomes της στατιστικής ανάλυσης, αλλά από πληροφορίες που σχετίζονται με το ίδιο το Pull Request και τις αλλαγές κώδικα που περιείχε. Με τον τρόπο αυτό αποφεύχθηκε η άμεση κατασκευή της κατηγορίας από τις ίδιες quality και security μετρικές που χρησιμοποιήθηκαν στη συνέχεια ως εξαρτημένες μεταβλητές καθώς και η άμεση

κυκλικότητα μεταξύ κατηγορικής explanatory variable και των εξαρτημένων μεταβλητών.

Για κάθε Pull Request αξιοποιήθηκαν οι διαθέσιμες πληροφορίες της αντίστοιχης σελίδας του GitHub (συγκεκριμένα στο GitHub API), μεταξύ των οποίων ο τίτλος και το περιεχόμενο του PR, οι υπάρχουσες ετικέτες, τα commits, τα τροποποιημένα αρχεία και, όπου ήταν διαθέσιμες, πληροφορίες από comments και reviews. Η ανάλυση συμπληρώθηκε από το περιεχόμενο των αρχείων .diff ή .patch, ώστε η ταξινόμηση να μην βασίζεται αποκλειστικά στη λεκτική περιγραφή η οποία είχε δοθεί από τον δημιουργό του εκάστοτε Pull Request, αλλά να λαμβάνει υπόψη παράλληλα και τις πραγματικές μεταβολές του πηγαίου κώδικα. Το .diff δεν περικόπτονταν ή δειγματοληπτούνταν πριν από την ταξινόμηση, αλλά παρέχόταν ολόκληρο, στόχος του οποίου ήταν η εκτίμηση να μπορεί να λαμβάνει υπόψη το πλήρες σύνολο των καταγεγραμμένων αλλαγών του συγκεκριμένου Pull Request.

Η επεξεργασία των παραπάνω πληροφοριών πραγματοποιήθηκε μέσω LLM-based classification pipeline -με ειδικότερη χρήση του gpt-oss-120b μοντέλου-. Η αρχική ταξινόμια περιλάμβανε τέσσερις διακριτές κατηγορίες: code_quality, code_security, software_quality και software_security. Η διάκριση μεταξύ code-level και software-level χαρακτηρισμών χρησιμοποιήθηκε στο στάδιο της αρχικής ταξινόμησης, ώστε να μπορεί να αποτυπωθεί εάν η αλλαγή αφορούσε περισσότερο εσωτερικές ιδιότητες του πηγαίου κώδικα ή ευρύτερα χαρακτηριστικά της συμπεριφοράς του λογισμικού.

Η ταξινόμηση δεν ήταν υποχρεωτικά αποκλειστική. Ένα Pull Request μπορούσε να συνδέεται με περισσότερες από μία κατηγορίες όταν το περιεχόμενό του περιλάμβανε ουσιαστικές αλλαγές διαφορετικού χαρακτήρα. Για κάθε κατηγορία εκτιμούνταν επίσης το σχετικό ποσοστό συμμετοχής της στην εξεταζόμενη αλλαγή και αντίστοιχος βαθμός εμπιστοσύνης της ταξινόμησης. Οι επιμέρους εκτιμήσεις μπορούσαν να αθροίζονται μέχρι το 100%, ενώ τυχόν υπόλοιπο αντιπροσώπευε αλλαγές οι οποίες δεν εντάσσονταν στις τέσσερις κύριες κατηγορίες.

Για τη διατήρηση μιας κατηγορίας χρησιμοποιήθηκε κατώφλι συνολικής συμμετοχής 15%. Όταν καμία κατηγορία δεν υπερέβαινε το συγκεκριμένο όριο, διατηρούνταν εκείνη με την υψηλότερη εκτιμώμενη συμμετοχή. Κατηγορίες, επίσης, οι οποίες δεν ικανοποιούσαν το συγκεκριμένο threshold δεν διατηρούνταν ως ενεργοί χαρακτηρισμοί της αντίστοιχης αλλαγής. Το στάδιο αυτό προηγήθηκε της στατιστικής ανάλυσης και τα αποτελέσματα της ταξινόμησης αποθηκεύτηκαν στη μεταβλητή Category. Στόχος ήταν το Pull Request να μην παραμένει αταξινομητο αποκλειστικά λόγω της κατανομής μικρών ποσοστών μεταξύ διαφορετικών τύπων αλλαγών. Σε περιπτώσεις όπου υπήρχαν ανταγωνιστικές ενδείξεις, προτεραιότητα δινόταν σε σαφείς ενδείξεις vulnerability ή code-security μεταβολής, αρχικά, και εν συνεχεία σε αναφορές όπως CVE, GSHA ή security tooling, έπειτα σε αλλαγές οι οποίες σχετίζονταν με bugs, performance ή software quality και, τέλος, σε refactoring, style και λοιπές code-quality μεταβολές. Ο συγκεκριμένος κανόνας χρησιμοποιήθηκε για να περιοριστεί η ασάφεια στις περιπτώσεις όπου μία αλλαγή μπορούσε εύλογα να ενταχθεί σε περισσότερες από μία τεχνικές κατηγορίες.

Για τις τελικές στατιστικές αναλύσεις η αρχική τετραμερής ταξινόμηση απλοποιήθηκε σε τρεις ενοποιημένες κατηγορίες. Οι χαρακτηρισμοί code_quality και

software_quality συνενώθηκαν στην κατηγορία quality, ενώ οι code_security και software_security στην κατηγορία security. Όταν σε ένα Pull Request συνυπήρχε τουλάχιστον ένας quality και ένας security χαρακτηρισμός, το Pull Request εντασσόταν στην κατηγορία both. Περιπτώσεις στις οποίες δεν υπήρξε κανένας από τους αναγνωρισμένους quality ή security χαρακτηρισμούς το Pull Request δεν εντάχθηκε σε κάποια ενοποιημένη κατηγορία και απορρίφθηκε από το τελικό σύνολο δεδομένων. η συγκεκριμένη επανακωδικοποίηση εφαρμόστηκε μεταγενέστερα από τα analytical scripts από τα analytical scripts και δεν αποτελεί μέρος της αρχικής LLM-based ταξινόμησης.

Η διαδικασία ταξινόμησης ήταν πλήρως αυτοματοποιημένη και δεν πραγματοποιήθηκε ανεξάρτητη χειροκίνητη επικύρωση ή επανάληψη ταξινόμησης υποδείγματος των αποτελεσμάτων. Οι ακριβείς, επιπλέον inference parameters που χρησιμοποιήθηκαν κατά την εκτέλεση του gpt-oss-120b δεν είναι διαθέσιμες προς τεκμηρίωση. Οι δύο αυτές παράμετροι λαμβάνονται υπόψη ως περιορισμοί της construct validity και της πλήρους end-to-end αναπαραγωγιμότητας της διαδικασίας ταξινόμησης των δεδομένων.

6.7 Ανακατασκευή των δύο καταστάσεων των εξεταζόμενων PR commits

Αφού η συλλογή των Pull Requests ολοκληρώθηκε, ακολούθησε ανακατασκευή των δύο καταστάσεων του πηγαίου κώδικα που χρησιμοποιήθηκαν για την εξέταση κάθε PR. Η πραγματοποίηση της συγκεκριμένης διαδικασίας κρίθηκε απαραίτητη ώστε η ανάλυση να μην περιορίζεται αποκλειστικά στα μεταδεδομένα που παρέχονται από το εκάστοτε Pull Request, αλλά να βασίζεται στον πραγματικό πηγαίο κώδικα ο οποίος αντιστοιχεί στις διαφορετικές καταστάσεις της εξεταζόμενης αλλαγής.

Για κάθε Pull Request που εξετάστηκε, δημιουργήθηκαν δύο διακριτά snapshots, τα οποία στο dataset της έρευνας χαρακτηρίζονται ως *pr_base* και *pr_closed*. Τα snapshots αυτά αντιπροσωπεύουν δύο διαφορετικές καταστάσεις του κώδικα που σχετίζονται με το ίδιο Pull Request και χρησιμοποιήθηκαν ως βάση για τη μεταγενέστερη διαδικασία static analysis και εξαγωγής των μετρικών λογισμικού. Με τον τρόπο αυτό κατέστη δυνατή η συστηματική αντιστοίχιση των αποτελεσμάτων της ανάλυσης με συγκεκριμένα PRs και συγκεκριμένες καταστάσεις του κώδικα.

Η δημιουργία των δύο αυτών καταστάσεων αποτέλεσε ένα ενδιάμεσο στάδιο μεταξύ της συλλογής των Pull Requests και της στατιστικής ανάλυσης του πηγαίου κώδικα. Ο τρόπος με τον οποίο ορίζεται και ανακατασκευάζεται η *pr_base* καθώς και η *pr_closed* κατάσταση περιγράφεται παρακάτω.

6.7.1 *pr_base*

Η *pr_base* κατάσταση αντιπροσωπεύει το ανακατασκευασμένο snapshot του πηγαίου κώδικα πριν από την ενσωμάτωση των αλλαγών που σχετίζονται με το εξεταζόμενο Pull Request και λειτουργεί ως η αρχική κατάσταση αναφοράς για το συγκεκριμένο PR. Για κάθε τέτοια κατάσταση διατηρείται η αντιστοίχιση με το parent repository και το αντίστοιχο Pull Request, ώστε να μπορεί να συσχετιστεί με την τελική κατάσταση *pr_closed* και να υποβληθεί στην ίδια διαδικασία στατικής ανάλυσης και εξαγωγής μετρικών λογισμικού. Τα δύο snapshots αντιμετωπίζονται ως συζευγμένες καταστάσεις του ίδιου Pull Request και όχι ως ανεξάρτητες παρατηρήσεις, γεγονός που επιτρέπει τόσο τον έλεγχο της μεταξύ τους αντιστοίχισης όσο και τη διερεύνηση των διαφορών που προκύπτουν κατά την εξέλιξη της συγκεκριμένης αλλαγής.

6.7.2 *pr_closed*

Οι μετρικές και τα findings που εξάγονται από το *pr_closed* αποτυπώνουν την κατάσταση του αναλυόμενου πηγαίου κώδικα στο συγκεκριμένο τελικό snapshot. Η αντιστοίχισή τους με ένα Pull Request δεν συνεπάγεται ότι τα αντίστοιχα quality ή security issues δημιουργήθηκαν, εισήχθησαν ή προκλήθηκαν από τις αλλαγές του συγκεκριμένου Pull Request. Ιδιαίτερα μετά τη διαπίστωση της εξαιρετικά περιορισμένης μεταβλητότητας μεταξύ *pr_base* και *pr_closed*, η τελική cross-sectional ανάλυση επιτρέπει τη μελέτη της παρουσίας και της πυκνότητας των findings στην τελική κατάσταση του κώδικα, αλλά όχι την ασφαλή απόδοση της προέλευσής τους στην εξεταζόμενη αλλαγή.

6.8 Static analysis και εξαγωγή μετρικών

Μετά την ολοκλήρωση της ανακατασκευής των προαναφερθέντων καταστάσεων (*pr_base*, *pr_closed*), ο πηγαίος κώδικας των αντίστοιχων snapshots υποβλήθηκε σε στατική ανάλυση με χρήση του εργαλείου SonarQube. Η διαδικασία πραγματοποιήθηκε χωρίς την εκτέλεση του λογισμικού και είχε ως στόχο την παραγωγή ενός κοινού συνόλου ποσοτικών μετρήσεων για τα διαφορετικά projects και Pull Requests. Με τον τρόπο αυτό, οι δύο καταστάσεις κάθε Pull Request αναλύθηκαν μέσω της ίδιας βασικής διαδικασίας και τα αποτελέσματα μπορούσαν στη συνέχεια να αντιστοιχιστούν με το repository, το Pull Request και τον αντίστοιχο ρόλο ανάλυσης (*pr_base* ή *pr_closed*).

Από τις αναλύσεις εξήχθησαν μετρικές που αποτυπώνουν διαφορετικές πτυχές του πηγαίου κώδικα και αποτέλεσαν τη βάση του analytical dataset το οποίο χρησιμοποιήθηκε στη συνέχεια. Στις μετρικές αυτές περιλαμβάνονται τα NCLOC, bugs, vulnerabilities, code smells, security hotspots, coverage, duplicated lines

density, complexity, cognitive complexity, καθώς και τα Software Quality Reliability Issues, Software Quality Maintainability Issues και Software Quality Security Issues. Το σύνολο αυτό περιλαμβάνει τόσο χαρακτηριστικά μεγέθους και δομικής πολυπλοκότητας του πηγαίου κώδικα όσο και automated findings που σχετίζονται με την αξιοπιστία, τη συντηρησιμότητα και την ασφάλειά του.

Τα αποτελέσματα κάθε scan αποθηκεύτηκαν μαζί με πληροφορίες που καθιστούσαν δυνατή τη σύνδεσή τους με το αντίστοιχο software project και Pull Request. Έτσι, ήταν δυνατή η αναγνώριση των δύο scans που αντιστοιχούσαν στο ίδιο PR και η μεταγενέστερη εφαρμογή ελέγχων ακεραιότητας και φιλτραρίσματος. Αν και πραγματοποιήθηκε στατική ανάλυση και στις δύο καταστάσεις (pr_based, pr_closed), τα δύο snapshots δεν αντιμετωπίστηκαν ως ανεξάρτητες μεταξύ τους παρατηρήσεις. Η αντιστοίχισή τους χρησιμοποιήθηκε αρχικά για τον έλεγχο της δομής και της συνέπειας των δεδομένων, ενώ μετά το preprocessing η κύρια στατιστική ανάλυση βασίστηκε σε μία τελική pr_closed παρατήρηση ανά PR.

Οι μετρικές οι οποίες εξήχθησαν από το SonarQube αντιμετωπίζονται στην παρούσα έρευνα ως αυτοματοποιημένοι και επαναλήψιμοι δείκτες των χαρακτηριστικών του εξεταζόμενου κώδικα και όχι ως απόλυτη μέτρηση της συνολικής ποιότητας ή ασφάλειας ενός συστήματος. Ειδικότερα, η ύπαρξη ενός vulnerability, security hotspot ή άλλου finding δεν ισοδυναμεί με επιβεβαιωμένη εκμετάλλευση ή πραγματικό security risk, αλλά εκφράζει αποτέλεσμα της εφαρμοζόμενης static-analysis διαδικασίας και των κανόνων όπως αυτοί έχουν τεθεί στο εργαλείο. Η συγκεκριμένη διάκριση λαμβάνεται υπόψη στην κατασκευή των ερευνητικών μεταβλητών και στην ερμηνεία των αποτελεσμάτων.

6.9 Μετρικές της εμπειρικής ανάλυσης

Η εμπειρική ανάλυση της παρούσας έρευνας βασίζεται σε ένα σύνολο μετρικών πηγαίου κώδικα και αυτοματοποιημένων ευρημάτων, οι οποίες εξήχθησαν μέσω static analysis και χρησιμοποιήθηκαν για την ποσοτική αποτύπωση διαφορετικών χαρακτηριστικών του λογισμικού. Οι μετρικές καλύπτουν διαστάσεις που σχετίζονται με το μέγεθος και την πολυπλοκότητα του κώδικα, τη συντηρησιμότητα, την αξιοπιστία, την επανάληψη κώδικα και την ασφάλεια.

Πιο συγκεκριμένα, εξετάζονται οι μετρικές NCLOC, Bugs, Vulnerabilities, Code Smells, Security Hotspots, Duplicated Lines Density, Complexity, Cognitive Complexity, Reliability Issues, Maintainability Issues και Security Issues. Οι μετρικές αυτές δεν αποτελούν πλήρεις ή ισοδύναμες μετρήσεις ποιότητας και ασφάλειας, αλλά αποτυπώνουν επιμέρους χαρακτηριστικά του κώδικα και findings που προκύπτουν από τη διαδικασία στατικής ανάλυσης.

6.9.1 NCLOC

Η μετρική NCLOC (Non-Comment Lines of Code) εκφράζει το μέγεθος του αναλυόμενου πηγαίου κώδικα σε φυσικές γραμμές. Συγκεκριμένα στο SonarQube, η μετρική *ncloc* υπολογίζεται ως ο αριθμός των φυσικών γραμμών που περιέχουν τουλάχιστον έναν χαρακτήρα που δεν αποτελεί κενό, `tab` ή μέρος σχολίου. Σε αντίθεση με μία απλή καταμέτρηση όλων των γραμμών ενός αρχείου, εδώ εξαιρούνται κενές γραμμές και γραμμές που αποτελούνται αποκλειστικά από σχόλια. Μεγαλύτερη τιμή NCLOC υποδηλώνει μεγαλύτερη έκταση αναλύσιμου κώδικα, χωρίς να συνεπάγεται καλύτερη ή χειρότερη τελική ποιότητα, πολυπλοκότητα ή ασφάλεια.

Στην παρούσα εργασία, το NCLOC χρησιμοποιήθηκε τόσο ως μετρική μεγέθους όσο και ως βάση για την κανονικοποίηση ορισμένων *issue counts*, επιτρέποντας τη σύγκριση Pull Requests που διαφέρουν ως προς την ποσότητα του αναλυόμενου κώδικα. Ειδικότερα, όπου χρησιμοποιήθηκαν *densities* ανά KLOC, ο αριθμός των findings σχετίστηκε με χίλιες γραμμές κώδικα μέσω του όρου $NCLOC / 1000$. Επιπλέον, παρατηρήσεις με *ncloc* = 0 αποκλείστηκαν από το τελικό analytical dataset, λόγω του ότι δεν αντιπροσώπευαν αναλύσιμο πηγαίο κώδικα και δεν επέτρεπαν τον υπολογισμό αντίστοιχων *densities*. Ως περιορισμός, το NCLOC αποτελεί πρωτίστως δείκτη μεγέθους και δεν αποτυπώνει τη σημασιολογική πολυπλοκότητα, τη λειτουργικότητα ή την ποιότητα του κώδικα. Επομένως, πρέπει να ερμηνεύεται σε συνδυασμό με τις υπόλοιπες μετρικές της ανάλυσης.

Σημειώνεται ότι το *ncloc* δεν χρησιμοποιείται στην παρούσα εργασία ως μέτρο του μεγέθους της ίδιας της αλλαγής του Pull Request. Αποτυπώνει την έκταση του πηγαίου κώδικα που περιλαμβάνεται στο αναλυόμενο snapshot. Το μέγεθος της τροποποίησης αποτελεί διαφορετική έννοια και μπορεί να περιγραφεί μέσω του αριθμού των προστιθέμενων, διαγραμμένων ή τροποποιημένων γραμμών του αντίστοιχου diff.

6.9.2 Bugs

Η μετρική Bugs εκφράζει τον συνολικό αριθμό των issues που το SonarQube ταξινομεί ως προβλήματα τα οποία σχετίζονται με την αξιοπιστία (reliability) του λογισμικού. Τα findings αυτά προκύπτουν αυτοματοποιημένα από την εφαρμογή των κανόνων static analysis στον εξεταζόμενο πηγαίο κώδικα και υποδεικνύουν περιπτώσεις στις οποίες εντοπίζεται κώδικας ο οποίος ενδέχεται να οδηγήσει σε λανθασμένη ή μη αναμενόμενη συμπεριφορά. Στην παρούσα έρευνα, ο αριθμός των Bugs εξήχθη για τα αναλυόμενα snapshots και χρησιμοποιήθηκε ως ποσοτικός δείκτης προβλημάτων που σχετίζονται με την αξιοπιστία του κώδικα. Υψηλή τιμή αυτού υποδηλώνει μεγαλύτερο αριθμό σχετικών ευρημάτων, χωρίς όμως να σημαίνει αυτομάτως ότι το λογισμικό έχει παρουσιάσει αντίστοιχο αριθμό πραγματικών failures κατά την εκτέλεσή του. Η μετρική εξαρτάται από τους κανόνες που έχουν τεθεί, τη διαμόρφωση και την έκδοση του εργαλείου και, ως αποτέλεσμα static analysis, μπορεί να περιλαμβάνει findings που δεν εκδηλώνονται τελικά ως

πραγματικά defects. Αποτέλεσμα των παραπάνω, είναι τα Bugs να αντιμετωπίζονται ως αυτοματοποιημένος δείκτης reliability-related προβλημάτων και όχι ως άμεση μέτρηση πραγματικών αποτυχιών του λογισμικού.

6.9.3 Vulnerabilities

Η μετρική Vulnerabilities εκφράζει τον συνολικό αριθμό όλων των security-related issues που το SonarQube ταξινομεί ως ευπάθειες στον αναλύόμενο πηγαίο κώδικα. Τα findings προκύπτουν από την εφαρμογή των ενεργών security rules κατά τη στατική ανάλυση και αντιστοιχούν σε σημεία του κώδικα που το εργαλείο θεωρεί ότι παρουσιάζουν πρόβλημα ασφάλειας και απαιτούν διόρθωση. Στην παρούσα έρευνα, η μετρική εξήχθη για τα εξεταζόμενα snapshots και χρησιμοποιήθηκε ως ένας από τους ποσοτικούς δείκτες για την περιγραφή της διάστασης της ασφάλειας, με υψηλότερη τιμή να υποδηλώνει μεγαλύτερο αριθμό vulnerabilities που εντοπίστηκαν από τη διαδικασία στατικής ανάλυσης. Σημειώνεται ότι, η τιμή της δεν αποτελεί άμεση μέτρηση πραγματικά εκμεταλλεύσιμων ευπαθειών ή του συνολικού security risk, καθώς εξαρτάται από τους κανόνες και το Quality Profile που εφαρμόζονται, την υποστήριξη της αντίστοιχης γλώσσας και τις δυνατότητες του εργαλείου, ενώ η στατική ανάλυση μπορεί να παράγει false positives ή να μην εντοπίσει υπαρκτές αδυναμίες. Για τον λόγο αυτό, η μετρική ερμηνεύεται ως automated security indicator και όχι ως οριστική επιβεβαίωση της πραγματικής κατάστασης ασφάλειας του λογισμικού.

6.9.4 Code smells

Η μετρική Code Smells (*code_smells*) αντιπροσωπεύει τον συνολικό αριθμό των maintainability-related issues τα οποία εντοπίζει το SonarQube στον αναλύόμενο πηγαίο κώδικα. Τα συγκεκριμένα findings προκύπτουν από την εφαρμογή κανόνων static analysis και υποδεικνύουν δομές ή πρακτικές προγραμματισμού που, χωρίς να αποτελούν αναγκαστικά άμεσα λειτουργικά σφάλματα, μπορούν να δυσχεράνουν την κατανόηση, τη συντήρηση ή τη μελλοντική τροποποίηση του κώδικα. Στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης προβλημάτων που σχετίζονται άμεσα με την συντηρησιμότητα, με υψηλότερη τιμή να υποδηλώνει μεγαλύτερο αριθμό αντίστοιχων automated findings. Αναγκαίος είναι ο καθορισμός της συγκεκριμένης μετρικής ως automated maintainability indicator και όχι ως απόλυτη εκτίμηση της ποιότητας του πηγαίου κώδικα. Αυτό συμβαίνει διότι δεν αποτελεί άμεση ή πλήρη μέτρηση της πραγματικής maintainability ενός συστήματος, καθώς εξαρτάται από τους ενεργούς κανόνες που έχουν τεθεί, το Quality Profile και τις δυνατότητες του εργαλείου, ενώ διαφορετικά code smells μπορούν να διαφέρουν σημαντικά ως προς τη σοβαρότητα και την πρακτική του επίδραση.

6.9.5 Security hotspots

Η μετρική Security Hotspots (*security_hotspots*) εκφράζει τον αριθμό των σημείων του πηγαίου κώδικα που το SonarQube εντόπισε και χαρακτήρισε ως security-sensitive και τα οποία απαιτούν ανθρώπινη αξιολόγηση. Αντίθετα με τις Vulnerabilities, ένα Security Hotspot δεν αποτελεί εξαρχής επιβεβαιωμένο πρόβλημα ασφαλείας. Υποδεικνύει τμήμα(τα) του πηγαίου κώδικα που σχετίζεται με λειτουργίες ή πρακτικές αυξημένου security ενδιαφέροντος και για τον οποίο πρέπει να εξεταστεί η ύπαρξη πραγματικού κινδύνου. Αν και στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης security-sensitive περιοχών του κώδικα, η τιμή της δεν πρέπει να ερμηνεύεται ως αριθμός πραγματικών vulnerabilities ή ως άμεση μέτρηση του συνολικού security risk, καθώς ένα hotspot μπορεί μετά από review να χαρακτηριστεί ασφαλές. Η ανίχνευσή του εξαρτάται από τους ενεργούς κανόνες και τις δυνατότητες της στατικής ανάλυσης. Υψηλότερη τιμή υποδηλώνει μεγαλύτερο αριθμό σημείων που απαιτούν αξιολόγηση. Λόγω των παραπάνω, η μετρική αντιμετωπίζεται ως automated indicator περιοχών αυξημένου security ενδιαφέροντος και όχι ως επιβεβαιωμένη ένδειξη ευπάθειας.

6.9.6 Duplicated lines density

Η μετρική Duplicated Lines Density (*duplicated_lines_density*) υποδηλώνει το ποσοστό των γραμμών του αναλυόμενου κώδικα που συμμετέχουν σε τμήματα τα οποία το SonarQube αναγνωρίζει ως duplicated code. Υπολογίζεται ως:

$$\frac{\text{duplicated_lines}}{\text{lines}} \times 100$$

Υψηλότερη τιμή υποδηλώνει μεγαλύτερη αναλογία επαναλαμβανόμενου κώδικα στο εξεταζόμενο snapshot κώδικα. Στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως δείκτης της έκτασης του code duplication, το οποίο μπορεί να σχετίζεται με αυξημένο κόστος συντήρησης και δυσκολία συνεπούς τροποποίησης επαναλαμβανόμενων τμημάτων πηγαίου κώδικα. Σημειώνεται ότι, η τιμή της δεν αποτελεί από μόνη της απόδειξη χαμηλής ποιότητας ή ύπαρξης security problem, καθώς η επανάληψη μπορεί να είναι σκόπιμη ή να μην έχει την ίδια πρακτική σημασία σε όλα τα projects. Η ανίχνευση duplication εξαρτάται από τους αλγορίθμους, τα thresholds και τη γλώσσα προγραμματισμού που υποστηρίζει το εργαλείο, οπότε η μετρική ερμηνεύεται ως ποσοτικός δείκτης επανάληψης κώδικα και όχι ως απόλυτη μέτρηση maintainability ή security risk.

6.9.7 Complexity

Η μετρική Complexity αποτυπώνει τη συνολική κυκλωματική πολυπλοκότητα (Cyclomatic Complexity) του αναλυόμενου πηγαίου κώδικα, δηλαδή τον αριθμό των ανεξάρτητων διαδρομών ελέγχου που προκύπτουν από τη δομή του προγράμματος. Στο εργαλείο SonarQube, η τιμή υπολογίζεται με βάση τα σημεία του κώδικα που αυξάνουν τη ροή ελέγχου, όπως συνθήκες και διακλαδώσεις, και συγκεντρώνεται στο αντίστοιχο επίπεδο ανάλυσης. Υψηλότερη τιμή υποδεικνύει μία γενικότερα πιο σύνθετη δομή ελέγχου και πιθανώς μεγαλύτερη δυσκολία κατανόησης, ελέγχου και συντήρησης του κώδικα, χωρίς όμως να συνεπάγεται αυτομάτως χαμηλή ποιότητα ή επιβεβαιωμένη ύπαρξη προβλήματος ασφάλειας. Στη παρούσα έρευνα, η Complexity χρησιμοποιήθηκε ως ποσοτικός δείκτης της δομικής πολυπλοκότητας του κώδικα και εξετάστηκε σε συνδυασμό με τις υπόλοιπες quality και security-related μεταβλητές. Ως περιορισμός, η μετρική αποτυπώνει κυρίως την πολυπλοκότητα η οποία εμφανίζεται στη ροή ελέγχου και όχι το σύνολο των παραγόντων που επηρεάζουν τη δυσκολία κατανόησης ή συντήρησης ενός συστήματος. Για τον λόγο αυτό, η ερμηνεία της δεν συμβαίνει ανεξάρτητα αλλά συμπληρωματικά με άλλες μετρικές, όπως η Cognitive Complexity.

6.9.8 Cognitive complexity

Η Cognitive Complexity (*cognitive_complexity*) αποτυπώνει τον βαθμό στον οποίο η δομή ελέγχου του πηγαίου κώδικα είναι δύσκολη να κατανοηθεί από έναν προγραμματιστή. Σε αντίθεση με την Cyclomatic Complexity, η οποία επικεντρώνεται κυρίως στον αριθμό των ανεξάρτητων διαδρομών εκτέλεσης, η Cognitive Complexity αυξάνεται όταν ο κώδικας διακόπτει τη γραμμική ροή ανάγνωσης, για παράδειγμα μέσω conditionals, loops, jumps και recursion, ενώ λαμβάνει επιπλέον υπόψη το βάθος του nesting. Στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης της γνωστικής δυσκολίας κατανόησης και συντήρησης του κώδικα, με υψηλότερες τιμές να υποδηλώνουν πιο απαιτητική λογική δομή. Δεν αποτελεί μέτρηση και αποτύπωση της συνολικής maintainability, της ποιότητας ή της ασφάλειας ενός συστήματος, καθώς δεν αποτυπώνει όλους τους παράγοντες που επηρεάζουν την κατανόηση του λογισμικού και η τιμή της εξαρτάται από τους κανόνες υπολογισμού που το εκάστοτε εργαλείο (εδώ το SonarQube) εφαρμόζει. Έτσι, ερμηνεύεται συμπληρωματικά με άλλες μετρικές, όπως η Complexity, το NCLOC και τα maintainability-related findings.

6.9.9 Reliability issues

Η μετρική Reliability Issues (*software_quality_reliability_issues*) εκφράζει τον συνολικό αριθμό των issues που το SonarQube ταξινομεί ως προβλήματα που επηρεάζουν άμεσα την αξιοπιστία του λογισμικού. Η τιμή της προκύπτει από την εφαρμογή των ενεργών κανόνων static analysis και αυξάνεται για κάθε finding που

αποδίδεται στη διάσταση Reliability. Η υψηλότερη τιμή υποδηλώνει μεγαλύτερο αριθμό σχετικών προβλημάτων στον αναλυόμενο κώδικα. Στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης της reliability-related διάστασης της ποιότητας και εξετάστηκε σε συνδυασμό με τις υπόλοιπες μετρικές του dataset. Η ερμηνεία ως αριθμός πραγματικών failures που έχουν εκδηλωθεί κατά την εκτέλεση του λογισμικού θα πρέπει να αποφεύγεται. Αντίθετα, θα πρέπει να ερμηνεύεται ως automated findings με πιθανή επίδραση στην αξιοπιστία. Η σχέση της, επιπλέον, με τη μετρική bugs εξαρτάται από την έκδοση, το configuration και το mode ταξινόμησης των rules του SonarQube. Οι δύο αυτές μετρικές, άρα, δεν πρέπει να θεωρούνται εξαρχής ανεξάρτητες μεταξύ τους.

6.9.10 Maintainability issues

Η μετρική Maintainability Issues (*software_quality_maintainability_issues*) εκφράζει τον συνολικό αριθμό των issues που το SonarQube ταξινομεί ως προβλήματα τα οποία επηρεάζουν άμεσα τη συντηρησιμότητα του λογισμικού. Σύμφωνα με την επίσημη τεκμηρίωση, η τιμή της αντιστοιχεί στο πλήθος των findings που αποδίδονται στη διάσταση Maintainability και προκύπτουν από την εφαρμογή των ενεργών κανόνων static analysis. Στο πλαίσιο της παρούσας έρευνας, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης maintainability-related προβλημάτων. Η υψηλότερη τιμή να υποδηλώνει μεγαλύτερο αριθμό σχετικών findings στον αναλυόμενο κώδικα. Μολαταύτα, δεν αποτελεί άμεση ή πλήρη και ολοκληρωμένη μέτρηση της πραγματικής συντηρησιμότητας ενός συστήματος, καθώς διαφορετικά issues μπορεί να διαφέρουν σημαντικά ως προς τη σοβαρότητα και την απαιτούμενη προσπάθεια αποκατάστασης, ενώ η τιμή εξαρτάται από τους ενεργούς κανόνες, το Quality Profile και την έκδοση του εργαλείου. Επιπλέον, η μετρική παρουσιάζει εννοιολογική επικάλυψη με τα Code Smells, τα οποία με την σειρά του συνδέονται με maintainability-related findings στο SonarQube. Τέλος, οι δύο αυτές μεταβλητές δεν θα πρέπει να θεωρούνται εξαρχής πλήρως ανεξάρτητες μεταξύ τους.

6.9.11 Security issues

Με την χρήση της μετρικής Security Issues (*software_quality_security_issues*) εκφράζεται ο συνολικός αριθμός των issues που το SonarQube ταξινομεί ως προβλήματα τα οποία επηρεάζουν τη διάσταση της ασφάλειας του λογισμικού. Στο Multi-Quality Rule Mode που υλοποιεί το SonarQube, η τιμή της προκύπτει από τα findings των ενεργών κανόνων static analysis που αποδίδονται στο software quality Security, με υψηλότερη τιμή να υποδηλώνει μεγαλύτερο αριθμό security-related issues στον αναλυόμενο κώδικα. Στην παρούσα έρευνα, η μετρική χρησιμοποιήθηκε ως ποσοτικός δείκτης της security-related διάστασης των automated findings και εξετάστηκε σε συνδυασμό με τις υπόλοιπες μετρικές ποιότητας και ασφάλειας. Δεν αποτελεί άμεση μέτρηση πραγματικά εκμεταλλεύσιμων ευπαθειών ή συνολικού

security risk, καθώς εξαρτάται από τους ενεργούς κανόνες, το configuration και την έκδοση του SonarQube. Επιπροσθέτως, παρουσιάζει πιθανή εννοιολογική και μετρητική επικάλυψη με άλλες security metrics, ιδιαίτερα τις Vulnerabilities, και επομένως οι δύο μεταβλητές δεν πρέπει να θεωρούνται εξαρχής πλήρως ανεξάρτητες μεταξύ τους.

6.10 Καθορισμός και έλεγχος ποιότητας δεδομένων

Προτού λάβει χώρα η στατιστική ανάλυση πραγματοποιήθηκε έλεγχος ποιότητας των δεδομένων. Στόχος του, το τελικό analytical dataset να περιλαμβάνει έγκυρες, συνεπείς και συγκρίσιμες παρατηρήσεις. Επειδή τα δεδομένα προήλθαν από αυτοματοποιημένη συλλογή και static analysis πραγματικών software projects, ήταν απαραίτητο να εντοπιστούν περιπτώσεις που θα μπορούσαν να επηρεάσουν την αξιοπιστία των αναλύσεων.

Οι έλεγχοι αφορούσαν σε missing και invalid observations, duplicates, περιπτώσεις με NCLOC = 0, failed scans, την ομοιότητα μεταξύ των pr_base και pr_closed, καθώς και outliers και ακραίες παρατηρήσεις. Παρακάτω, παρουσιάζεται ο τρόπος εντοπισμού και αντιμετώπισης κάθε κατηγορίας προβλημάτων κατά τη διαμόρφωση του τελικού συνόλου δεδομένων.

6.10.1 *Missing και invalid observations*

Κατά την προ-επεξεργασία πραγματοποιήθηκε έλεγχος για missing και μη έγκυρες παρατηρήσεις, με στόχο στο analytical dataset να διατηρηθούν αποκλειστικά εγγραφές που μπορούσαν να χρησιμοποιηθούν με συνέπεια στις επόμενες αναλύσεις. Ειδικότερα, κατά την απλοποίηση της μεταβλητής Category, ελλιπείς τιμές καθώς και κατηγορίες που δεν μπορούσαν να αντιστοιχιστούν στις ερευνητικά σχετικές ομάδες quality-security-both χαρακτηρίστηκαν μη έγκυρες για τη συγκεκριμένη ανάλυση και αποκλείστηκαν. Αποτέλεσμα της συγκεκριμένης διαδικασίας ήταν η αποτροπή αυθαίρετης ταξινόμησης Pull Requests για τα οποία δεν υπήρχε επαρκής πληροφορία ή τα οποία αφορούσαν διαφορετικό τύπο αλλαγής, όπως αποκλειστικά documentation. Οι συγκεκριμένοι αποκλεισμοί αντιμετωπίζονται ως μέρος του data cleaning και όχι ως αντικατάσταση των ελλিপών τιμών με εκτιμώμενες τιμές, αποφεύγοντας με τον τρόπο αυτό την εισαγωγή τεχνητής πληροφορίας στο dataset.

6.10.2 Duplicates

Κατά τον έλεγχο των δεδομένων εξετάστηκε η ύπαρξη μη αναμενόμενων πολλαπλών διπλότυπων εγγραφών για το ίδιο Pull Request, ώστε να καταστεί δυνατή η αποφυγή τεχνητής υπερ-εκπροσώπησης συγκεκριμένων PRs στο analytical dataset. Για κάθε παρατήρηση δημιουργήθηκε μοναδικό αναγνωριστικό από τον συνδυασμό του *repositoryName* και του *pullRequestNumber* και διατηρήθηκαν αποκλειστικά τα PRs που εμφάνιζαν ακριβώς δύο εγγραφές, μία για *pr_base* και μία για *pr_closed* κατάσταση. PRs με διαφορετικό αριθμό εγγραφών αποκλείστηκαν ως μη τυπικές περιπτώσεις. Στη συνέχεια, για την κύρια στατιστική ανάλυση διατηρήθηκε μόνο η εγγραφή *pr_closed*, ώστε κάθε Pull Request να αντιπροσωπεύεται από μία και μοναδική παρατήρηση και να αποφεύγεται ο ψευδο-διπλασιασμός του δείγματος από τις δύο ιδιαίτερα όμοιες καταστάσεις του ίδιου PR.

6.10.3 NCLOC = 0

Στη διάρκεια ελέγχου ποιότητας των δεδομένων εντοπίστηκαν παρατηρήσεις με $NCLOC = 0$, δηλαδή περιπτώσεις στις οποίες η στατική ανάλυση δεν κατέγραψε αναλύσιμο πηγαίο κώδικα για το αντίστοιχο snapshot. Τέτοιες παρατηρήσεις μπορούν να προκύψουν, για παράδειγμα, όταν ένα Pull Request αφορά αποκλειστικά documentation ή άλλα αρχεία που δεν συνεισφέρουν σε μετρήσιμο πηγαίο κώδικα. Οι συγκεκριμένες εγγραφές αποκλείστηκαν από το analytical dataset, καθώς δεν ήταν συγκρίσιμες με τις υπόλοιπες παρατηρήσεις και δεν επέτρεπαν τον υπολογισμό κανονικοποιημένων δεικτών που βασίζονται στο μέγεθος του κώδικα, όπως densities ανά KLOC. Η εξαίρεσή τους αποτέλεσε αναγκαίο βήμα για τη διατήρηση της συνέπειας και της αριθμητικής εγκυρότητας των επόμενων αναλύσεων.

6.10.4 Failed scans

Κατά την εκτέλεση της διαδικασίας static analysis ήταν δυνατό ορισμένα snapshots να μην παράγουν πλήρες και αξιοποιήσιμο αποτέλεσμα ανάλυσης. Οι περιπτώσεις αυτές αντιμετωπίστηκαν στο πλαίσιο των ελέγχων ακεραιότητας του dataset, καθώς για την ένταξη ενός Pull Request απαιτούνταν η ύπαρξη της αναμενόμενης δομής των δύο αντίστοιχων εγγραφών *pr_base* και *pr_closed*. Pull Requests για τα οποία δεν υπήρχε πλήρες ζεύγος δεν διατηρήθηκαν στο σύνολο δεδομένων μετά το clearing. Έτσι, αποφεύχθηκε η χρήση παρατηρήσεων για τις οποίες η διαδικασία ανάλυσης ή συλλογής δεν είχε αποδώσει ολοκληρωμένα δεδομένα και διασφαλίστηκε ότι τα PRs που προχώρησαν στα επόμενα στάδια διέθεταν την απαιτούμενη πληροφορία για συνεπή επεξεργασία.

6.10.5 Base/closed similarity

Η σύγκριση των δεδομένων ανέδειξε την ομοιότητα μεταξύ των καταστάσεων *pr_base* και *pr_closed* για κάθε Pull Request, με στόχο να διαπιστωθεί κατά πόσο οι δύο αναλύσεις αποτύπωναν ουσιαστικά διαφορετικές καταστάσεις των εξεταζόμενων μετρικών. Η σύγκριση ανέδειξε ιδιαίτερα υψηλή ομοιότητα, καθώς για τη πλειονότητα των PRs οι βασικές *quality* και *security-related* μετρικές παρουσίαζαν μηδενικές ή πολύ περιορισμένες μεταβολές μεταξύ των δύο snapshots. Το εύρημα αυτό κατέστησε περιορισμένη την πληροφοριακή αξία μιας κύριας ανάλυσης βασισμένης αποκλειστικά στις διαφορές που προκύπτουν από

$$pr_closed - pr_base$$

και αποτέλεσε βασικό λόγο για τη διατήρηση της *pr_closed* κατάστασης ως μοναδικής PR-level παρατήρησης στο κύριο *analytical dataset*. Παράλληλα, η σύγκριση των δύο snapshots διατηρήθηκε ως σημαντικός έλεγχος συνέπειας και ακεραιότητας της διαδικασίας συλλογής και ανακατασκευής των δεδομένων.

6.10.6 Outliers και ακραίες περιπτώσεις

Κατά τον έλεγχο του dataset εξετάστηκε η παρουσία outliers και ακραίων τιμών στις αριθμητικές μεταβλητές, καθώς οι *software metrics* μπορούν να παρουσιάζουν έντονα ασύμμετρες κατανομές και πολύ υψηλές τιμές σε Pull Requests ή projects με ιδιαίτερα χαρακτηριστικά. Οι ακραίες παρατηρήσεις δεν θεωρήθηκαν χωρίς ενδελεχή εξέταση λανθασμένες, δεδομένου ότι μπορούν να αντιπροσωπεύουν πραγματικές και έγκυρες περιπτώσεις, όπως μεγάλης έκτασης αλλαγές κώδικα ή projects με αυξημένο αριθμό *automated findings*. Η αξιολόγησή τους, επομένως, πραγματοποιήθηκε μέσω συνδυασμού της κατανομής και του περιεχομένου των αντίστοιχων μεταβλητών, ώστε να διαχωρίζονται οι πραγματικές ακραίες περιπτώσεις από πιθανές ανωμαλίες της διαδικασίας συλλογής ή ανάλυσης. Η επίδραση των έντονα ασύμμετρων κατανομών και των ακραίων τιμών λαμβάνεται περαιτέρω υπόψη στη συνέχεια της προ-επεξεργασίας των δεδομένων και στατιστικής ανάλυσης.

6.11 Κατασκευή αναλυτικών μεταβλητών

Μετά τον καθορισμό και τον έλεγχο ποιότητας των δεδομένων, ακολούθησε η κατασκευή των αναλυτικών μεταβλητών που χρησιμοποιήθηκαν στις επόμενες στατιστικές διαδικασίες. Οι αρχικές μετρικές δεν χρησιμοποιήθηκαν αποκλειστικά ως ακατέργαστοι αριθμοί, καθώς οι διαφορές στο μέγεθος του κώδικα και στη

βασική κατάσταση κάθε Pull Request μπορούσαν να επηρεάσουν τη δυνατότητα σύγκρισης των παρατηρήσεων.

Έτσι, εξετάστηκαν τόσο raw counts όσο και κανονικοποιημένες μορφές των μετρικών, όπως issue densities και δείκτες complexity ως προς το NCLOC, καθώς και μεταβλητές που αποτυπώνουν απόλυτες σχετικές μεταβολές μεταξύ των εξεταζόμενων καταστάσεων. Ιδιαίτερη αντιμετώπιση απαιτήθηκε στις περιπτώσεις όπου η baseline τιμή ήταν μηδενική, καθώς τότε ορισμένοι relative ή percentage δείκτες δεν μπορούν να υπολογιστούν με τον συνήθη τρόπο.

Παρακάτω, παρουσιάζονται ο τρόπος με τον οποίο κατασκευάστηκαν οι raw και normalized μεταβλητές, τα issue densities, οι complexity-based δείκτες, οι μεταβολές Δx και τα relative/percentage changes, καθώς και ο χειρισμός των περιπτώσεων με $baseline = 0$.

6.11.1 Raw counts και normalization

Οι αρχικές μετρικές οι οποίες συλλέχθηκαν στο πλαίσιο της παρούσας έρευνας περιλαμβάνουν τους απόλυτους αριθμούς findings ή χαρακτηριστικών που καταγράφηκαν για κάθε Pull Request, όπως ο αριθμός των bugs, vulnerabilities, code smells και security-related issues, χαρακτηριζόμενα ως raw counts. Οι τιμές αυτές διατηρούν άμεσα την πληροφορία που παράγει άμεσα την πληροφορία που παράγει το εργαλείο ανάλυσης, αλλά η μεταξύ τους σύγκριση μπορεί να επηρεάζεται σημαντικά από το συνολικό μέγεθος του εξεταζόμενου κώδικα -συνήθως το μη κανονικοποιημένο NCLOC-. Αυτό συμβαίνει διότι ένα μεγαλύτερο codebase έχει περισσότερες ευκαιρίες να περιλαμβάνει findings σε σχέση με ένα μικρότερο. Για τον λόγο αυτό, σε περιπτώσεις όπου ήταν κατάλληλο, εφαρμόστηκε διαδικασία normalization ως προς το NCLOC, ώστε οι αντίστοιχες μεταβλητές να εκφράζουν την ποσότητα των findings σε σχέση με την έκταση του αναλυόμενου κώδικα και να διευκολύνεται η σύγκριση Pull Requests ποικίλου μεγέθους. Τα raw counts δεν θεωρήθηκαν λανθασμένα ή άχρηστα, καθώς οι δύο μορφές αποτυπώνουν διαφορετικές όψεις της ίδιας πληροφορίας, το απόλυτο πλήθος και τη σχετική συγκέντρωση των findings.

6.11.2 Issue densities

Για τη βελτίωση της συγκρισιμότητας μεταξύ παρατηρήσεων που αντιστοιχούν σε code snapshots διαφορετικού μεγέθους κατασκευάστηκαν δείκτες πυκνότητας προβλημάτων (issue densities), στους οποίους το πλήθος των findings εκφράζεται σε σχέση με το NCLOC. Η χρήση αποκλειστικά απόλυτων counts μπορεί να δυσχεράνει τη σύγκριση παρατηρήσεων διαφορετικής έκτασης, καθώς ένα μεγαλύτερο αναλυόμενο codebase παρέχει μεγαλύτερη έκθεση για την καταγραφή περισσότερων findings. Για τον λόγο αυτό, το NCLOC κανονικοποιήθηκε και μετατράπηκε σε KLOC (thousands of lines of code) μέσω της σχέσης

$$KLOC = NCLOC / 1000$$

και οι πυκνότητες εκφράστηκαν ως αριθμός issues ανά 1.000 γραμμές αναλύσιμου κώδικα.

Αυτή η κανονικοποίηση δεν θεωρείται ότι εξαλείφει πλήρως την επίδραση του μεγέθους του κώδικα. Οι ratio-based μεταβλητές εξακολουθούν να εξαρτώνται μαθηματικά από τον χρησιμοποιούμενο παρονομαστή και, όταν περισσότερες μεταβλητές μοιράζονται τον ίδιο παρονομαστή, μέρος της παρατηρούμενης μεταξύ τους συσχέτισης μπορεί να επηρεάζεται από αυτή την κοινή μαθηματική δομή. Για τον λόγο αυτό, το *log_ncloc* διατηρήθηκε ως ξεχωριστή μεταβλητή στα regression και mixed-effects models, ενώ για τις PCA και Factor Analysis πραγματοποιήθηκαν επιπλέον size-adjusted sensitivity analyses.

Στην κύρια εμπειρική ανάλυση κατασκευάστηκαν δύο βασικές μεταβλητές πυκνότητας:

$$\begin{aligned} \text{Quality Density} &= \text{Maintainability Issues} / KLOC \\ \text{Security Density} &= \text{Security Issues} / KLOC \end{aligned}$$

Η πρώτη μεταβλητή (Quality Density) εκφράζει τον αριθμό των *softwareQualityMaintainabilityIssues* ανά KLOC και η δεύτερη τον αντίστοιχο αριθμό των *softwareQualitySecurityIssues* ανά KLOC. Υψηλότερη τιμή υποδηλώνει άρα μεγαλύτερη συγκέντρωση findings σε σχέση με την έκταση του αναλυόμενου κώδικα και όχι απλώς μεγαλύτερο απόλυτο αριθμό προβλημάτων. Στη συνέχεια χρησιμοποιήθηκαν επίσης λογαριθμικά μετασχηματισμένες μορφές

$$\begin{aligned} \log(1 + \text{quality_density}), \\ \log(1 + \text{security_density}) \end{aligned}$$

με στόχο την αντιμετώπιση της έντονης ασυμμετρίας που παρουσιάστηκε στις αντίστοιχες κατανομές. Η κανονικοποίηση δεν εξαλείφει πλήρως κάθε επίδραση του μεγέθους, ενώ πολύ μικρές τιμές NCLOC μπορούν να δημιουργήσουν ιδιαίτερα υψηλές densities. Εδώ έγκειται η απόρριψη των παρατηρήσεων με $NCLOC = 0$ και το μέγεθος του κώδικα εξακολούθησε να λαμβάνεται υπόψη στις στατιστικές αναλύσεις.

6.11.3 Complexity και Έλεγχος ως προς NCLOC

Αντίθετα με τα issue densities, η Complexity δεν κανονικοποιήθηκε μέσω άμεσης διαίρεσης με το NCLOC. Στην τελική εμπειρική ανάλυση οι δύο μεταβλητές διατηρήθηκαν χωριστά και μετασχηματίστηκαν λογαριθμικά ως

$$\begin{aligned} \log_complexity &= \log(1 + \text{Complexity}), \\ \log_ncloc &= \log(1 + NCLOC) \end{aligned}$$

Η επιλογή αυτή επιτρέπει να εξετάζεται η σχέση της πολυπλοκότητας με τα quality και security-related outcomes, λαμβάνοντας παράλληλα υπόψη το μέγεθος του αναλυόμενου κώδικα ως ξεχωριστό χαρακτηριστικό. Ειδικότερα, τόσο στα OLS όσο και στα Linear Mixed Models, τα log_complexity και log_ncloc περιλήφθηκαν ταυτόχρονα ως predictors, ώστε η επίδραση της πολυπλοκότητας να εκτιμάται υπό έλεγχο για τις διαφορές στο μέγεθος του κώδικα. Ο λογαριθμικός μετασχηματισμός χρησιμοποιήθηκε επίσης για τον περιορισμό της έντονης ασυμμετρίας που εμφανίζουν οι συγκεκριμένες μετρικές, χωρίς όμως να εξαλείφει πιθανές μεταξύ τους συσχετίσεις. Η Complexity επομένως αντιμετωπίστηκε ως ξεχωριστή δομική μεταβλητή και όχι ως απλός δείκτης πολυπλοκότητας ανά γραμμή κώδικα.

6.11.4 Αρχικός ορισμός μεταβολών και αξιολόγηση της delta προσέγγισης

Για την αποτύπωση της μεταβολής μιας μετρικής μεταξύ των δύο καταστάσεων ενός Pull Request εξετάστηκαν δείκτες απόλυτης μεταβολής οι οποίοι ορίζονται γενικά ως

$$\Delta x = x_{closed} - x_{base}$$

όπου x_{base} είναι η τιμή της εξεταζόμενης μετρικής στην κατάσταση pr_base και x_{closed} η αντίστοιχη τιμή στην κατάσταση pr_closed. Το τελικό πρόσημο της μεταβολής ερμηνεύεται ανάλογα με τη φύση της εκάστοτε μετρικής. Για παράδειγμα, αύξηση του αριθμού προβλημάτων συντηρησιμότητας ή ασφάλειας υποδηλώνει πιο δυσμενή εξέλιξη, ενώ η αντίστοιχη λογική δεν ισχύει κατ' ανάγκη για μετρικές στις οποίες υψηλότερες τιμές εκφράζουν βελτίωση.

Κατά τον αρχικό σχεδιασμό εξετάστηκε επίσης η δυνατότητα χρήσης σχετικών και ποσοστιαίων μεταβολών, ώστε η απόλυτη διαφορά να ερμηνεύεται σε σχέση με την αρχική τιμή της μετρικής. Αυτή η προσέγγιση παρουσίαζε επιπροσθέτως περιορισμούς όταν η τιμή pr_base ήταν μηδενική, καθώς η σχετική ή ποσοστιαία μεταβολή δεν μπορεί να οριστεί με κανονικό τρόπο χωρίς την εισαγωγή αυθαίρετων τιμών. Για τον λόγο αυτό, οι συγκεκριμένοι δείκτες δεν χρησιμοποιήθηκαν ως κύριες αναλυτικές μεταβλητές.

Παράλληλα, είχε εξεταστεί ένα περιγραφικό πλαίσιο ταξινόμησης των μεταβολών σε Improvement, Deterioration και Unchanged, καθώς και της συνδυαστικής συμπεριφοράς ποιότητας και ασφάλειας σε περιπτώσεις Synergy, Trade-off και Joint Deterioration. Οι κατηγορίες αυτές είχαν, αρχικά, σχεδιαστεί για να περιγράφουν την κατεύθυνση των μεταβολών και όχι για να τεκμηριώνουν αιτιώδεις σχέσεις μεταξύ ποιότητας και ασφάλειας. Παραδείγματος χάριν, μία ταυτόχρονη βελτίωση των δύο διαστάσεων θα μπορούσε να χαρακτηριστεί ως Synergy, ενώ μεταβολές προς αντίθετες κατευθύνσεις ως Trade-off.

Ο έλεγχος των πραγματικών δεδομένων έδειξε, από την άλλη, ότι η μεταβλητότητα μεταξύ pr_base και pr_closed ήταν εξαιρετικά περιορισμένη. Σε μεγάλη πλειονότητα των Pull Requests οι εξεταζόμενες μετρικές παρέμειναν

αμετάβλητες. Αποτέλεσμα αυτού ήταν οι περισσότερες τιμές Δx να είναι μηδενικές. Οι μεταβλητές, επομένως, μεταβολής δεν παρείχαν επαρκή πληροφορία για αξιόπιστη κύρια στατιστική μοντελοποίηση των within-PR διαφορών.

Για τον λόγο αυτό, η delta προσέγγιση διατηρήθηκε μόνο ως διαγνωστικό και περιγραφικό στάδιο του ερευνητικού σχεδιασμού. Η τελική εμπειρική ανάλυση βασίστηκε σε μία παρατήρηση ανά Pull Request, συγκεκριμένα στην κατάσταση `pr_closed`, μετά τον αποκλεισμό πρατηρήσεων με `ncloc = 0`. Οι βασικές μεταβλητές αποτελέσματος διαμορφώθηκαν ως πυκνότητες προβλημάτων ποιότητας και ασφάλειας, ενώ οι αρχικά προβλεπόμενες κατηγορίες Synergy, Trade-off και Joint Deterioration δεν χρησιμοποιήθηκαν για την εξαγωγή κύριων συμπερασμάτων.

6.11.5 Relative και percentage changes

Πέρα από την απόλυτη μεταβολή Δx , εξετάστηκε και η δυνατότητα έκφρασης της αλλαγής μιας μετρικής σε σχέση με την αρχική τιμή της. Η σχετική μεταβολή ορίστηκε ως:

$$RelativeChange_x = \frac{x_{closed} - x_{base}}{x_{base}}$$

ενώ η αντίστοιχη ποσοστιαία μεταβολή (percentage change) προκύπτει ως εξής:

$$PercentageChange_x = \frac{x_{closed} - x_{base}}{x_{base}} \times 100$$

Οι παραπάνω ορισμένοι δείκτες επιτρέπουν η ίδια απόλυτη διαφορά να ερμηνεύεται σε σχέση με το αρχικό επίπεδο της μετρικής. Έστω το παράδειγμα όπου τα issues αυξάνονται κατά δύο. Αυτό έχει διαφορετική σχετική σημασία όταν η baseline τιμή είναι δύο (2) και διαφορετική όταν είναι εκατό (100). Ορίζονται επομένως με στόχο τη διευκόλυνση της σύγκρισης των PRs με διαφορετικές αρχικές τιμές. Η ερμηνεία της κατεύθυνσης της μεταβολής εξακολουθεί να εξαρτάται από τη φύση της εκάστοτε μετρικής και δεν συνεπάγεται αυτομάτως improvement ή deterioration.

Στην παρούσα έρευνα οι δείκτες αυτοί εξετάστηκαν ως πιθανές μορφές περιγραφής των διαφορών μεταξύ `pr_base` και `pr_closed`, αλλά δεν χρησιμοποιήθηκαν ως κύρια outcomes στην τελική στατιστική μοντελοποίηση. Η ιδιαίτερα υψηλή ομοιότητα των δύο snapshots και η μεγάλη συγκέντρωση μηδενικών μεταβολών περιόρισαν την πληροφοριακή αξία `changed-base` μεταβλητών, αποτέλεσμα της οποίας ήταν η κύρια ανάλυση να βασιστεί στην κατάσταση `pr_closed` και στις αντίστοιχες κανονικοποιημένες μεταβλητές. Οι relative και percentage changes, επιπροσθέτως, παρουσιάζουν ιδιαίτερο πρόβλημα όταν $x_{base} = 0$, καθώς ο παρονομαστής μηδενίζεται και ο δείκτης δεν ορίζεται.

6.12 Τελικό analytical dataset

Αφού οι διαδικασίες συλλογής, αντιστοίχισης, ελέγχου ποιότητας, φιλτραρίσματος και κατασκευής των αναλυτικών μεταβλητών ολοκληρώθηκαν και διαμορφώθηκε το τελικό analytical dataset της έρευνας. Η βασική μονάδα παρατήρησης παρέμεινε το PR, με κάθε ένα από αυτά να συνδέεται με το software project στο οποίο και ανήκει. Παρόλο που κατά τη φάση της συλλογής των δεδομένων υπήρχαν δύο snapshots για κάθε Pull Request (pr_base, pr_closed), η ιδιαίτερα υψηλή ομοιότητα τους οδήγησε στην επιλογή της κατάστασης pr_closed ως μοναδικής παρατήρησης ανά PR για την κύρια στατιστική ανάλυση, αποφεύγοντας τον ψευδο-διπλασιασμό των παρατηρήσεων.

Στο τελικό dataset διατηρήθηκαν Pull Requests που διέθεταν την αναμενόμενη αντιστοίχιση pr_base-pr_closed, παρουσίαζαν NCLOC > 0 και μπορούσαν να ταξινομηθούν σε μία από τις ερευνητικά σχετικές κατηγορίες quality, security ή both. Μετά την εφαρμογή των σχετικών κριτηρίων, το σύνολο δεδομένων που χρησιμοποιήθηκε στην έρευνα περιλαμβάνει περίπου 3.700 Pull Requests από ένα σύνολο 242 διαφορετικών μεταξύ τους software projects. Η παρουσία πολλαπλών PRs ανά project διατηρεί την ιεραρχική δομή των δεδομένων και αποτελεί βασικό χαρακτηριστικό το οποίο και λαμβάνεται υπόψη στη μεταγενέστερη στατιστική μοντελοποίηση.

Το analytical dataset περιλαμβάνει τόσο τις αρχικές μετρικές που εξήχθησαν κατά τη διαδικασία του static analysis, όπως NCLOC, Complexity, Cognitive Complexity και quality/security-related issue counts, όσο και νέες μεταβλητές που δημιουργήθηκαν κατά το pre-processing. Σε αυτές περιλαμβάνονται οι Quality και Security Issue Densities ανά KLOC, οι λογαριθμικά μετασχηματισμένες μορφές των βασικών συνεχών μεταβλητών, η απλοποιημένη κατηγορία του Pull Request και το project identifier. Για τις ανάγκες των Mixed Effects Models διατηρήθηκε επίσης ρητά η πληροφορία του project ως grouping variable, ενώ υπολογίστηκε και ο αριθμός των Pull Requests που αντιστοιχούν σε κάθε project ξεχωριστά.

Το τελικό analytical dataset αποτελεί συνεπώς το κοινό σημείο εκκίνησης για τις αναλύσεις οι οποίες επρόκειτο να ακολουθήσουν. Η διαμόρφωσή του διασφαλίζει ότι κάθε παρατήρηση αντιστοιχεί σε ένα διακριτό και αναλύσιμο Pull Request, ότι οι βασικές μεταβλητές έχουν συνεπή σημασιολογική και αριθμητική μορφή και ότι διατηρείται η πληροφορία που απαιτείται για τη διερεύνηση τόσο των σχέσεων μεταξύ των μετρικών λογισμικού όσο και της διαφοροποίησης που εμφανίζεται μεταξύ διαφορετικών έργων.

6.13 Δυνατότητα αναπαραγωγής και ερευνητική διαφάνεια

Η δυνατότητα αναπαραγωγής (reproducibility) και η ερευνητική διαφάνεια αποτέλεσαν βασικές αρχές κατά την οργάνωση της εμπειρικής διαδικασίας. Ως

computational reproducibility νοείται η δυνατότητα επανεκτέλεσης μιας ανάλυσης με τα ίδια δεδομένα, τις ίδιες μεθόδους και τον ίδιο υπολογιστικό κώδικα, ώστε να μπορούν να επαληθευτούν τα παραγόμενα αποτελέσματα. Στην παρούσα έρευνα διατηρήθηκαν τα δεδομένα όπως αυτά προέκυψαν από τη διαδικασία συλλογής, η οποία περιεγράφηκε σε προηγούμενα κεφάλαια της παρούσας βιβλιογραφίας, τα αρχεία που χρησιμοποιήθηκαν στα επιμέρους στάδια επεξεργασίας, τα scripts των στατιστικών αναλύσεων και τα παραγόμενα αποτελέσματα και γραφήματα. Με αυτόν τον τρόπο είναι δυνατή η αναδρομή από τα τελικά αποτελέσματα προς τα δεδομένα και τις υπολογιστικές διαδικασίες από τις οποίες προέκυψαν.

Η μεθοδολογία, παράλληλα, περιγράφει διαδοχικά τα βασικά στάδια του pipeline -από την επιλογή repositories και τη συλλογή των PRs έως το snapshot reconstruction, το static analysis, το data cleaning, την κατασκευή των αναλυτικών μεταβλητών και τις στατιστικές αναλύσεις- με στόχο οι αποφάσεις που επηρεάζουν το τελικό dataset να είναι εμφανείς και ελέγξιμες. Η διατήρηση δεδομένων, κώδικα ανάλυσης, παραμέτρων και παραγόμενων artifacts αποτελεί βασική πρακτική για την υποστήριξη της αναπαραγωγιμότητας στην υπολογιστική έρευνα. Η δυνατότητα αναπαραγωγής των υπολογιστικών αποτελεσμάτων δεν συνεπάγεται ότι μία νέα συλλογή δεδομένων από το GitHub σε διαφορετική χρονική στιγμή θα παράγει απαραίτητα το ίδιο dataset, καθώς τα repositories και τα διαθέσιμα δεδομένα μεταβάλλονται δυναμικά και συνεχώς με την πάροδο του χρόνου. Η διατήρηση δεδομένων, επομένως, που χρησιμοποιήθηκαν στην παρούσα έρευνα αποτελεί ουσιώδες μέρος της τεκμηρίωσης και της ερευνητικής της διαφάνειας.

Στατιστική μεθοδολογία

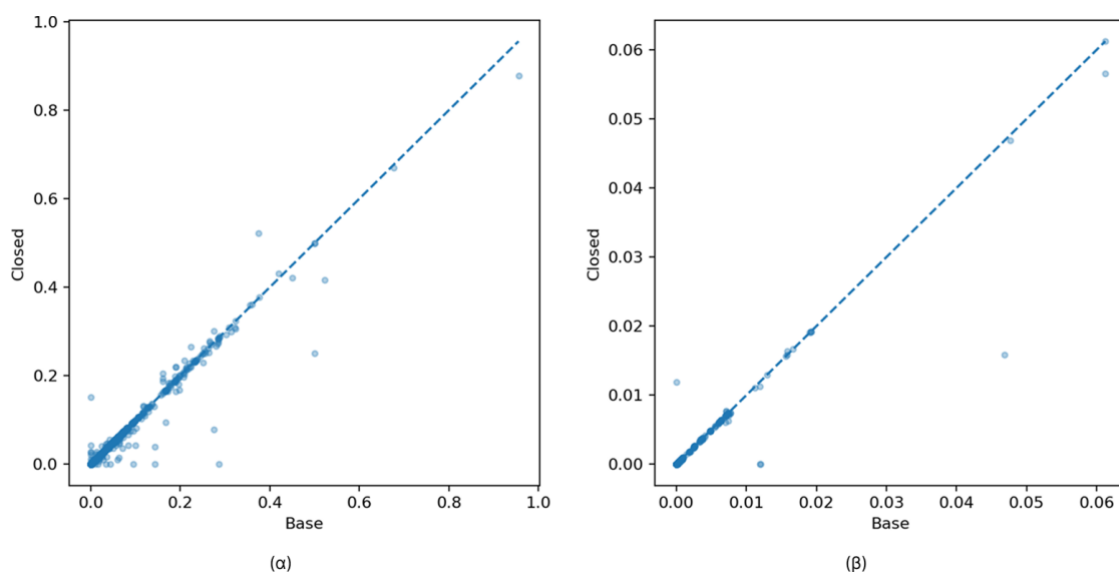
Έπειτα της συλλογής, του καθορισμού και της διαμόρφωσης του τελικού analytical dataset, το συνακόλουθο στάδιο της έρευνας αφορά τη στατιστική διερεύνηση των δεδομένων. Στόχος του παρόντος κεφαλαίου είναι να παρουσιάσει τις μεθόδους που χρησιμοποιήθηκαν για την περιγραφή της δομής του dataset, την εξέταση των σχέσεων μεταξύ επιλεγμένων software metrics και τη διερεύνηση της ετερογένειας που εμφανίζεται μεταξύ διαφορετικών software projects.

Η στατιστική προσέγγιση της έρευνας συνδυάζει περιγραφικές, παλινδρομικές και πολυμεταβλητές τεχνικές, καθώς καθεμία εξετάζει διαφορετική πτυχή του ερευνητικού προβλήματος. Η Exploratory Data Analysis χρησιμοποιείται για την αρχική κατανόηση των κατανομών και των σχέσεων μεταξύ μεταβλητών, ενώ τα Ordinary Least Squares (OLS) Regression Models παρέχουν ένα αρχικό πλαίσιο εκτίμησης των στατιστικών συσχετίσεων. Η ιεραρχική δομή των δεδομένων αντιμετωπίζεται μέσω της χρήσης Linear Mixed Models (LMM), τα οποία επιτρέπουν να ληφθεί υπόψη η ομαδοποίηση των Pull Requests μέσα στα αντίστοιχα projects. Η Principal Component Analysis (PCA) και Factor Analysis (FA) χρησιμοποιούνται για τη διερεύνηση της εσωτερικής δομής και της πιθανής αλληλοεπικάλυψης μεταξύ των διαθέσιμων μετρικών λογισμικού.

Οι επιμέρους μέθοδοι δεν χρησιμοποιούνται ως ανεξάρτητες και αποσπασματικές αναλύσεις, αλλά ως συμπληρωματικές μεταξύ τους προσεγγίσεις πάνω στο ίδιο ερευνητικό πρόβλημα. Η ερμηνεία των αποτελεσμάτων παραμένει συνεπής με τον γενικότερο observational χαρακτήρα της μελέτης, επομένως οι στατιστικές σχέσεις που εντοπίζονται αντιμετωπίζονται στη συνέχεια ως συσχετίσεις και όχι ως τεκμήρια αιτιώδους επίδρασης.

7.1 Στρατηγική στατιστικής ανάλυσης

Η στρατηγική της στατικής ανάλυσης διαμορφώθηκε σταδιακά, λαμβάνοντας κυρίως ως βάση τόσο το αρχικό ερευνητικό πλαίσιο όσο και τα χαρακτηριστικά που προέκυψαν κατά τη διερεύνηση των δεδομένων. Ο αρχικός σχεδιασμός προέβλεπε τη σύγκριση των καταστάσεων pr_base και pr_closed και τη μοντελοποίηση των αντίστοιχων μεταβολών Δx. Οι προκαταρκτικοί έλεγχοι όμως, έδειξαν ότι, σύμφωνα με την εκάστοτε μετρική, περίπου 94% έως και 99,9% των Pull Requests παρουσίαζαν ίδιες τιμές στις δύο καταστάσεις, με αποτέλεσμα οι μεταβλητές μεταβολής να διαθέτουν εξαιρετικά περιορισμένη διακύμανση.



Εικόνα 7.1.1 - Σύγκριση των καταστάσεων `pr_base` και `pr_closed` ως προς (α) την πυκνότητα Maintainability Issues και (β) την πυκνότητα Security Issues.

Παράλληλα, περίπου 60% των αρχικών παρατηρήσεων εμφάνιζαν $NCLOC = 0$, γεγονός που περιόριζε σημαντικά τη δυνατότητα υπολογισμού κανονικοποιημένων δεικτών. Για τον λόγο αυτό, η κύρια ανάλυση οργανώθηκε τελικά ως cross-sectional ανάλυση σε επίπεδο Pull Request, με μία παρατήρηση `pr_closed` ανά PR και με διατήρηση μόνο των περιπτώσεων με $NCLOC > 0$. Μετά την εφαρμογή των σχετικών κριτηρίων, τα οποία περιεγράφηκαν παραπάνω, το analytical dataset περιλάμβανε περίπου 3.700 Pull Requests καταναμημένα σε 242 διαφορετικά έργα λογισμικού.

Οι κύριες εξαρτημένες μεταβλητές κατασκευάστηκαν ως πυκνότητες issues ανά KLOC, με τα *softwareQualityMaintainabilityIssues* και *softwareQualitySecurityIssues* να χρησιμοποιούνται αντίστοιχα για την αποτύπωση των quality και security-related outcomes. Η μεταβλητή Category ενοποιήθηκε στις κατηγορίες quality, security και both, ώστε να αντιμετωπιστούν με συνεπή τρόπο οι διαφορετικές ταξινομήσεις που υπήρχαν στα αρχικά δεδομένα. Το μέγεθος του κώδικα και η πολυπλοκότητα συμπεριλήφθηκαν ως explanatory variables, με το NCLOC να χρησιμοποιείται σε λογαριθμικά μετασχηματισμένη μορφή. Ιδιαίτερη σημασία κατέχει επίσης η διατήρηση του project identifier, καθώς ο αριθμός των Pull Request ανά project παρουσίαζε έντονη ανισορροπία και οι παρατηρήσεις του ίδιου project δεν μπορούσαν να θεωρηθούν πλήρως ανεξάρτητες.

Η οργάνωση της inferential στρατηγικής συνέβη γύρω από διαδοχικά μοντέλα με διαφορετικό ρόλο. Ως αρχικό σημείο αναφοράς χρησιμοποιήθηκε Ordinary Least Squares regression με cluster-robust standard errors σε επίπεδο project, προκειμένου να υπάρχει ένα απλούστερο pooled baseline model. Το επιλεγμένο ως κύριο στατιστικό μοντέλο της έρευνας είναι το Linear Mixed Model με random intercept για κάθε project, το οποίο επιτρέπει τη ρητή μοντελοποίηση της ετερογένειας μεταξύ των project και εφαρμόζει partial pooling στις παρατηρήσεις. Η επιλογή αυτή κρίνεται σημαντική λόγω της μεγάλης διαφοροποίησης στον αριθμό από PRs ανά project, ο οποίος κυμαίνεται από πού μικρό αριθμό παρατηρήσεων έως

δεκάδες PRs στο ίδιο project. Ακολουθώντας τον τρόπο αυτό περιορίζεται σημαντικά η πιθανότητα τα μεγαλύτερα projects να καθορίζουν δυσανάλογα την εκτίμηση των γενικών στοιχείων.

Δύο επιπλέον μοντέλα χρησιμοποιήθηκαν συμπληρωματικά. Ένα Negative Binomial GLM με offset το $\log(NCLOC)$ εφαρμόστηκε ως robustness analysis, επειδή οι αρχικές issue μεταβλητές αποτελούν counts και παρουσιάζουν χαρακτηριστικά που δεν αποτυπώνονται πλήρως από ένα Gaussian linear model. Το μοντέλο αυτό δεν αντικαθιστά το LMM ως κύρια προσέγγιση, καθώς δεν μοντελοποιεί την project-level ετερογένεια μέσω random effects, αλλά παρέχει έναν εναλλακτικό έλεγχο των αποτελεσμάτων με κατανομή καταλληλότερη για count data. Παράλληλα, εφαρμόστηκε GEE binomial model στην αναλογία των Security Issues προς το συνολικό πλήθος Security και Maintainability Issues, με στόχο να εξεταστεί ένα διαφορετικό ερευνητικό ερώτημα, ποια χαρακτηριστικά σχετίζονται με μεγαλύτερη σχετική παρουσία security-related προβλημάτων έναντι maintainability-related προβλημάτων.

Η σταθερότητα των βασικών αποτελεσμάτων εξετάστηκε επιπλέον μέσω sensitive analysis, περιορίζοντας το δείγμα σε έργα λογισμικού με τουλάχιστον πέντε (5) Pull Requests μέσα. Ο έλεγχος αυτός χρησιμοποιήθηκε για να αξιολογηθεί κατά πόσο οι εκτιμήσεις του κύριου μοντέλου επηρεάζονται από projects με πολύ μικρό αριθμό παρατηρήσεων. Η regression-based ανάλυση, τέλος, συμπληρώθηκε από Principal Component Analysis (PCA) και Factor Analysis (FA), οι οποίες εξετάζουν το ίδιο dataset από διαφορετική οπτική, διερευνώντας τη συσχέτιση, την πιθανή redundancy και τις ευρύτερες δομές που εμφανίζονται μεταξύ των software metrics. Η συνολική στρατηγική συνδυάζει ένα κύριο ιεραρχικό μοντέλο, απλούστερα και εναλλακτικά μοντέλα ελέγχου, sensitive analyses και πολυμεταβλητές τεχνικές, ώστε τα συμπεράσματα να μην εξαρτώνται αποκλειστικά από μία μοναδική στατιστική προσέγγιση.

7.2 Περιγραφική στατιστική και Exploratory Data Analysis (EDA)

Η Exploratory Data Analysis προηγείται της συμπερασματικής μοντελοποίησης και εξετάζει κατανομές, ακραίες τιμές, ελλείψεις και αρχικές σχέσεις μεταξύ μεταβλητών. Η φάση αυτή καθοδηγεί τις μεταγενέστερες επιλογές, χωρίς να υποκαθιστά τους προκαθορισμένους στατιστικούς ελέγχους (Field, 2018).

Η EDA οργανώθηκε γύρω από τρεις βασικούς άξονες: τη διερεύνηση των κατανομών και των ακραίων παρατηρήσεων, τη σύγκριση μεταξύ των καταστάσεων pr_base και pr_closed, καθώς και την εξέταση των project-level distributions. Οι αναλύσεις αυτές χρησιμοποιήθηκαν τόσο για την περιγραφή των δεδομένων όσο και για τη διαμόρφωση κρίσιμων μεθοδολογικών αποφάσεων, όπως η επιλογή της pr_closed κατάστασης ως κύριας PR-level παρατήρησης και η ανάγκη ρητής αντιμετώπισης της ετερογένειας μεταξύ των project στα μοντέλα τα οποία επρόκειτο να ακολουθήσουν.

7.2.1 Κατανομές και ακραίες τιμές

Η διερεύνηση των κατανομών των αριθμητικών μεταβολών αποτέλεσε βασικό στάδιο της Exploratory Data Analysis, καθώς οι μετρικές λογισμικού δεν αναμένεται να κατανέμονται ομοιόμορφα μεταξύ των PRs. Ιδιαίτερη προσοχή δόθηκε στις μεταβλητές οι οποίες χρησιμοποιήθηκαν άμεσα στη μοντελοποίηση, όπως το NCLOC, η Complexity και οι πυκνότητες Maintainability και Security issues ανά KLOC. Η περιγραφική εξέταση πραγματοποιήθηκε τόσο συνολικά όσο και ανά κατηγορία Pull Request (quality, security, both). Στόχος ήταν να εντοπιστούν διαφορές στη κεντρική τάση, στη διασπορά και στην παρουσία ακραίων παρατηρήσεων. Στο αντίστοιχο analysis script υπολογίζονται και οι διάμεσες τιμές των βασικών μεταβλητών ανά PR category και δημιουργούνται boxplots για τις δύο κύριες issue densities.

Ιδιαίτερη προσοχή δόθηκε επίσης στην παρουσία μεγάλου αριθμού μηδενικών παρατηρήσεων για ορισμένες μεταβλητές, κυρίως στις security-related metrics. Η υψηλή συγκέντρωση τιμών στο μηδέν αντιμετωπίζεται εδώ ως χαρακτηριστικό της εμπειρικής κατανομής και δεν χαρακτηρίζεται αυτομάτως zero inflation. Ο τελευταίος όρος έχει ειδικότερη στατιστική σημασία και προϋποθέτει ότι τα παρατηρούμενα μηδενικά υπερβαίνουν εκείνα που μπορεί να αποδώσει το επιλεγμένο count model ή/και ότι υπάρχει διακριτός μηχανισμός παραγωγής ενός μέρους των μηδενικών παρατηρήσεων.

Οι βασικές συνεχείς μεταβλητές που παρουσίαζαν έντονη δεξιά ασυμμετρία μετασχηματίστηκαν μέσω της συνάρτησης

$$\log(1 + x)$$

όπου με χρήση της οποίας δημιουργήθηκαν οι μεταβλητές *log_ncloc*, *log_complexity*, *log_cognitive*, *log_quality_density* και *log_security_density*. Ο μετασχηματισμός $\log(1+x)$ περιορίζει την επίδραση εξαιρετικά υψηλών θετικών τιμών και, σε αντίθεση με τον απλό λογάριθμο, παραμένει ορισμένος όταν η αρχική τιμή είναι μηδενική. Η ιδιότητα αυτή είναι σημαντική για τις issue densities, καθώς η απουσία καταγεγραμμένων findings σε ένα Pull Request αποτελεί έγκυρη παρατήρηση και όχι missing value. Δεν εξαλείφει τη μεγάλη συγκέντρωση παρατηρήσεων σε μηδενικές τιμές, η οποία ήταν ιδιαίτερα έντονη σε security-related μεταβλητές. Η σημείωση αυτή κρίνεται ιδιαίτερα σημαντική για τις issues densities, καθώς η απουσία findings σε ένα Pull Request αποτελεί έγκυρη παρατήρηση και δεν πρέπει να αντιμετωπίζεται ως κάποιο missing value. Οι μετασχηματισμένες μεταβλητές χρησιμοποιήθηκαν στη συνέχεια στα OLS και LMM, με σκοπό η μοντελοποίηση να είναι λιγότερο ευαίσθητη στις μεγάλες διαφορές κλίμακας μεταξύ των παρατηρήσεων.

Τα OLS και LMM πάνω στις λογαριθμικά μετασχηματισμένες πυκνότητες δεν αποτέλεσαν τη μοναδική στατιστική προσέγγιση. Συμπληρωματικά εφαρμόστηκε Negative Binomial GLM στα αρχικά count outcomes με *offset = log(ncloc)*, ώστε να εξεταστεί η σταθερότητα των συμπερασμάτων μέσω μοντέλου ειδικά

διαμορφωμένου για count δεδομένα. Η χρήση του συγκεκριμένου robustness model δεν υποκαθιστά το κύριο LMM, αλλά επιτρέπει να αξιολογηθεί η εξάρτηση των βασικών αποτελεσμάτων από την αντιμετώπιση των issues ως λογαριθμικά μετασχηματισμένων densities ή ως counts προσαρμοσμένα ως προς το μέγεθος του αναλυόμενου κώδικα.

Οι ακραίες τιμές δεν αφαιρέθηκαν αυτομάτως αποκλειστικά και μόνο επειδή βρίσκονταν σε μεγάλη απόσταση απόλυτης τιμής από το μεγαλύτερο μέρος της κατανομής. Στα δεδομένα πραγματικών software projects μία υψηλή τιμή πιθανώς να αντιστοιχεί σε πραγματικό Pull Request του οποίου το αντίστοιχο αναλυόμενο snapshot χαρακτηρίζεται από μεγάλο codebase, αυξημένη πολυπλοκότητα ή ιδιαίτερα υψηλή συγκέντρωση findings και επομένως να αποτελεί έγκυρη παρατήρηση. Η EDA, επομένως, χρησιμοποιήθηκε για τον εντοπισμό και την αξιολόγηση τέτοιων περιπτώσεων, ενώ η επίδρασή τους περιορίστηκε κυρίως μέσω των μετασχηματισμών και της μεταγενέστερης χρήσης μοντέλων που λαμβάνουν υπόψη τη δομή των δεδομένων. Συνοψίζοντας, η παρουσία ακραίων παρατηρήσεων αντιμετωπίζεται ως χαρακτηριστικό της πραγματικής ετερογένειας του dataset και όχι εξ ορισμού ως λόγος αποκλεισμού.

7.2.2 Base-closed comparisons

Στο πλαίσιο που ορίζεται από την EDA πραγματοποιήθηκε σύγκριση των τιμών των βασικών μετρικών μεταξύ των καταστάσεων pr_base και pr_closed για κάθε Pull Request. Στόχος του συγκεκριμένου ελέγχου ήταν να διαπιστωθεί η περίπτωση συνοδείας από επαρκή μεταβολή στις διαστάσεις quality και security-related μετρικές ώστε να υποστηριχθεί μία before/after προσέγγιση της μετάβασης από την αρχική προς την τελική κατάσταση του PR. Τα υπολογιζόμενα αποτελέσματα έδειξαν ιδιαίτερα υψηλή ομοιότητα μεταξύ των δύο snapshots. Εξαρτώμενα από την μετρική, περίπου το 94 έως και 99.9% των PRs παρουσίαζαν ταυτόσημες τιμές και στις δύο καταστάσεις. Ειδικότερα, περισσότερο από 94% των PRs εμφάνιζαν μηδενική μεταβολή στα Maintainability Issues, ενώ για τα Security Issues το αντίστοιχο ποσοστό ξεπερνούσε το 99%.

Η εξαιρετικά υψηλή συγκέντρωση μηδενικών τελικών μεταβολών οδήγησε στο συμπέρασμα ότι μεταβλητές της μορφής $\Delta x = x_{closed} - x_{base}$ διέθεταν αρκετά περιορισμένη διακύμανση, με αποτέλεσμα να μην καθίσταται δυνατή η παροχή επαρκούς πληροφορίας προκειμένου να αποτελέσουν τη βάση της κύριας στατιστικής μοντελοποίησης. Έτσι, η αρχική before/after καθορισμένη στρατηγική εγκαταλείφθηκε ως κύρια αναλυτική προσέγγιση και η έρευνα οργανώθηκε τελικά ως cross-sectional ανάλυση σε επίπεδο Pull Request, με μία παρατήρηση pr_closed ανά PR με προϋπόθεση NCLOC > 0. Η σύγκριση pr_base-pr_closed διατηρήθηκε σε σημαντικό διαγνωστικό στάδιο, καθώς τεκμηρίωσε εμπειρικά τη συγκεκριμένη μεθοδολογική επιλογή και απέτρεψε την εφαρμογή μοντέλων πάνω σε μεταβλητές με σχεδόν μηδενική μεταβλητότητα.

7.2.3 Project-level distributions

Η διερευνητική ανάλυση εξετάστηκε και σε επίπεδο project, καθώς τα Pull Requests του τελικού dataset δεν κατανέμονται ομοιόμορφα μεταξύ των διαφορετικών repositories. Για κάθε project υπολογίστηκε ο αριθμός Pull Requests που περιλαμβάνονταν στο analytical dataset, με στόχο την αξιολόγηση της ισορροπίας της ιεραρχικής δομής των δεδομένων. Η κατανομή παρουσίασε έντονη ανισορροπία με τον αριθμό των Pull Requests να κυμαίνεται μεταξύ του συνόλου [1, 94] ανά project. Ορισμένα projects, αναπόφευκτα, συνεισφέρουν πολύ μεγαλύτερο αριθμό παρατηρήσεων από άλλα και δεν θα ήταν κατάλληλο να αντιμετωπιστεί το dataset ως ένα ομοιογενές σύνολο ανεξάρτητων PRs.

Η ομαδοποίηση των παρατηρήσεων ανά project επιτρέπει τη διερεύνηση πιθανών between-project διαφοροποιήσεων στις quality, security-related μεταβλητές. PRs τα οποία προέρχονται από το ίδιο repository μοιράζονται κοινό τεχνικό και αναπτυξιακό περιβάλλον, ενώ διαφορετικά projects μπορούν να παρουσιάζουν συστηματικά διαφορετικά επίπεδα των εξεταζόμενων metrics. Η συγκεκριμένη δομή αποτέλεσε σημαντική ένδειξη ότι η μεταξύ-project ετερογένεια θα πρέπει να λαμβάνεται ρητά υπόψη στη μοντελοποίηση και υποστήριξη τη μεταγενέστερη χρήση LMM με το project ως ορισμένο grouping factor. Λόγω της παράλληλης ύπαρξης projects με πολύ μικρό αριθμό PRs, πραγματοποιήθηκε αργότερα sensitive analysis περιορισμένη σε projects με τουλάχιστον πέντε παρατηρήσεις, προκειμένου να εξεταστεί κατά πόσο τα βασικά αποτελέσματα εξαρτώνται από τις μικρότερες ομάδες.

7.3 Συσχετίσεις

Συμπληρωματικό στάδιο της Exploratory Data Analysis αποτέλεσε η διερεύνηση των συσχετίσεων μεταξύ αριθμητικών μεταβλητών. Στόχος της ορίστηκε η αποτύπωση της κατεύθυνσης και της έντασης των σχέσεων μεταξύ των software metrics προτού εφαρμοστούν τα πολυμεταβλητά μοντέλα. Η ανάλυση συσχετίσεων επιτρέπει να εντοπιστούν μεταβλητές που μεταβάλλονται από κοινού, πιθανές επικαλύψεις μεταξύ διαφορετικών μετρικών και σχέσεις οι οποίες ενδέχεται να απαιτούν περαιτέρω διερεύνηση στα regression και mixed-effects models.

Ιδιαίτερη σημασία δόθηκε στις σχέσεις που προκύπτουν μεταξύ μεταβλητών που αποτυπώνουν διαφορετικές διαστάσεις του λογισμικού, όπως το μέγεθος και η πολυπλοκότητα του κώδικα, τα maintainability-related findings και τα security-related findings. Η διερευνητική εξέταση των σχέσεων αυτών συμβάλλει στην κατανόηση της εσωτερικής δομής του dataset και στην αξιολόγηση πιθανής αλληλοεπικάλυψης ή συνύπαρξης μεταξύ διαφορετικών μετρικών.

Κεντρικό ενδιαφέρον της παρούσας έρευνας αποτελεί ειδικότερα η διερεύνηση των Quality-Security relationships, δηλαδή του κατά πόσο οι δείκτες που σχετίζονται με την ποιότητα και την ασφάλεια παρουσιάζουν συστηματικές μεταξύ

τους σχέσεις. Οι συσχετίσεις οι οποίες παρατηρήθηκαν αντιμετωπίζονται ως περιγραφικά και διερευνητικά ευρήματα και όχι ως τεκμήρια αιτιώδους επίδρασης. Η πιο ολοκληρωμένη αξιολόγηση των σχέσεων πραγματοποιείται μέσω πολυμεταβλητών μοντέλων, τα οποία επιτρέπουν να ληφθούν ταυτόχρονα υπόψη πρόσθετοι παράγοντες, όπως το μέγεθος, η πολυπλοκότητα και η ομαδοποίηση των Pull Requests στα διαφορετικά έργα λογισμικού που εξετάστηκαν.

7.3.1 Quality-Security Relationships

Η διερεύνηση της σχέσης μεταξύ των quality-related και security-related χαρακτηριστικών του πηγαίου κώδικα αποτελεί κεντρικό στόχο της παρούσας έρευνας. Στο τελικό analytical dataset οι δύο διαστάσεις αποτυπώνονται κυρίως μέσω των πυκνοτήτων που λαμβάνουν τα Maintainability Issues και Security Issues ανά KLOC, επιτρέποντας τη σύγκρισή τους με τρόπο που λαμβάνει υπόψη το διαφορετικό μέγεθος του αναλυόμενου κώδικα. Η σχέση που αναπτύσσεται Quality-Security δεν αντιμετωπίζεται ως θετική εκ των προτέρων, αρνητική ή αιτιώδης αντίστοιχα, αλλά ως εμπειρικό ερώτημα που εξετάζεται μέσα από τα μοτίβα όπως εμφανίζονται στις δύο κατηγορίες findings.

Ο Pearson correlation αποτυπώνει γραμμική συνδιακύμανση συνεχών μεταβλητών, ενώ ο Spearman βασίζεται στις τάξεις και είναι λιγότερο ευαίσθητος σε μη κανονικότητα και ακραίες τιμές. Και οι δύο δείκτες περιγράφουν συσχέτιση, όχι αιτιότητα (Field, 2018). Η ένταξη του project στη μοντελοποίηση επιτρέπει τον διαχωρισμό της ετερογένειας που συνδέεται με το repository από τις γενικότερες σχέσεις που εμφανίζονται στο σύνολο των δεδομένων.

Συμπληρωματικά των παραπάνω, η σχετική ισορροπία μεταξύ των δύο διαστάσεων εξετάζεται μέσω της αναλογίας των Security Issues προς το συνολικό πλήθος των Security και Maintainability Issues η οποία μοντελοποιείται με ξεχωριστή προσέγγιση GEE. Η συγκεκριμένη ανάλυση επιτρέπει να διερευνηθεί εάν χαρακτηριστικά του πηγαίου κώδικα συνδέονται παράλληλα με το απόλυτο ή κανονικοποιημένο πλήθος των findings, η σχετική επικράτηση security-related έναντι maintainability-related προβλημάτων. Με τον τρόπο αυτό, η σχέση Quality-Security εξετάζεται από περισσότερες από μία οπτικές, χωρίς οι παρατηρούμενες στατιστικές σχέσεις να ερμηνεύονται ως αποδείξεις αιτιώδους επίδρασης της μίας διάστασης στην άλλη.

7.4 Ordinary Least Squares

Η στατιστική μεθοδολογία Ordinary Least Squares (OLS) regression χρησιμοποιήθηκε ως αρχικό μοντέλο αναφοράς για τη διερεύνηση των σχέσεων μεταξύ των εξεταζόμενων χαρακτηριστικών των Pull Requests και των quality και

security-related outcomes. Η συγκεκριμένη μέθοδος εκτιμά τη σχέση μεταξύ μίας εξαρτημένης μεταβλητής και ενός ή περισσότερων predictors μέσω της ελαχιστοποίησης του αθροίσματος των τετραγώνων των residuals. Στην παρούσα έρευνα, το OLS παρέχει ένα σχετικά απλό και άμεσα ερμηνεύσιμο πλαίσιο για την αρχική αξιολόγηση της κατεύθυνσης και του μεγέθους των συσχετίσεων μεταξύ των μεταβλητών.

Στο πλαίσιο το οποίο ορίζεται από τα μοντέλα έγινε χρήση των λογαριθμικά μετασχηματισμένων πυκνοτήτων Maintainability και Security Issues ως outcomes, ενώ χαρακτηριστικά όπως είναι το μέγεθος του κώδικα, η πολυπλοκότητα και η κατηγορία που το Pull Request ανήκει (διαθέσιμες τιμές κατηγορίας: quality, security, both) συμπεριλήφθηκαν ως explanatory variables. Πολλαπλά Pull Requests μπορούν να προέρχονται από το ίδιο project, με αποτέλεσμα τα standard errors του baseline OLS να υπολογίζονται με cluster-robust προσέγγιση σε επίπεδο project. Στόχος του ήταν ο περιορισμός των συνεπειών της εξάρτησης στο εσωτερικό περιβάλλον του software project.

Παρά την προσαρμογή, το OLS δεν μοντελοποιεί άμεσα τη μεταξύ-project ετερογένεια ούτε την ιεραρχική δομή των δεδομένων. Για τον λόγο αυτό χρησιμοποιείται κυρίως ως σημείο σύγκρισης για το μεταγενέστερο Linear Mixed Model, το οποίο αποτελεί το κύριο μοντέλο της έρευνας. Παρακάτω παρουσιάζεται αναλυτικά η μορφή και ερμηνεία του στατιστικού μοντέλου OLS, οι βασικές παραδοχές του, ο έλεγχος multicollinearity μέσω VIF, τα residual diagnostics και τέλος, ο περιορισμός που προκύπτει από τη μη πλήρη ανεξαρτησία των Pull Requests.

7.4.1 Μοντέλο και συντελεστές

Η Ordinary Least Squares (OLS) regression χρησιμοποιήθηκε ως το αρχικό μοντέλο αναφοράς για τη διερεύνηση των σχέσεων μεταξύ των χαρακτηριστικών των διαφορετικών Pull Requests και των quality και security-related outcomes. Ξεχωριστά μοντέλα εφαρμόστηκαν για την πυκνότητα Maintainability Issues και την πυκνότητα των Security Issues, χρησιμοποιώντας τις λογαριθμικά μετασχηματισμένες μορφές τους ως εξαρτημένες μεταβλητές. Ως predictors συμπεριλήφθηκαν το μέγεθος του αναλυόμενου κώδικα μέσω του $\log(\text{NCLOC})$, η Complexity σε λογαριθμικά μετασχηματισμένη μορφή και η κατηγορία του Pull Request (quality, security, both). Η κατηγορία quality χρησιμοποιήθηκε ως κατηγορία αναφοράς, προκειμένου μέσω των συντελεστών των security και both να εκφραστεί η διαφορά τους σε σχέση με τα quality-only Pull Requests.

Το OLS εκτιμά τους συντελεστές που ελαχιστοποιούν το άθροισμα των τετραγώνων των residuals. Οι συντελεστές ερμηνεύουν τη μέση αναμενόμενη μεταβολή του outcome για δεδομένη μεταβολή ενός predictor, υπό σταθερούς τους υπόλοιπους όρους του μοντέλου (Field, 2018).

$$Y_i = \beta_0 + \beta_1 \log(1 + \text{NCLOC}_i) + \beta_2 \log(1 + \text{Complexity}_i) + \beta_3 \text{Security}_i + \beta_4 \text{Both}_i + \varepsilon_i$$

καθώς αντίστοιχα για το security-related outcome ορίζεται ως:

$$Y_i = \beta_0 + \beta_1 \log(1 + NCLOC_i) + \beta_2 \log(1 + Complexity_i) + \beta_3 Security_i + \beta_4 Both_i + \varepsilon_i$$

Όπου το (i) αντιστοιχεί σε μία οντότητα Pull Request, ο όρος β_0 είναι το intercept (σταθερός όρος του μοντέλου), οι β_1 και β_2 εκφράζουν τις σχέσεις των συνεχών predictors με το αντίστοιχο outcome και οι β_3 και β_4 αποτυπώνουν τις διαφορές των κατηγοριών security και both σε σχέση με την κατηγορία αναφοράς quality. Ο όρος ε_i αντιπροσωπεύει το μέρος της μεταβλητότητας της εξαρτημένης μεταβλητής που δεν εξηγείται από τους predictors του μοντέλου.

Η μέθοδος των ελαχίστων τετραγώνων δίνει την δυνατότητα να εκτιμηθούν οι regression coefficients, δηλαδή η επιλογή των τιμών των β που ελαχιστοποιούν το άθροισμα των τετραγώνων των residuals. Ένας θετικός συντελεστής υποδηλώνει θετική σχέση του αντίστοιχου predictor με το outcome, ενώ ένας αρνητικός συντελεστής υποδηλώνει αντίθετη σχέση, υπό την προϋπόθεση ότι οι υπόλοιπες μεταβλητές του μοντέλου παραμένουν σταθερές. Λόγω του ότι τόσο τα outcomes όσο και οι συνεχείς predictors έχουν υποστεί λογαριθμικό μετασχηματισμό, η ακριβής ποσοτική ερμηνεία των coefficients διαφοροποιείται από εκείνη ενός μοντέλου στις αρχικές κλίμακες και πραγματοποιείται με βάση τις μετασχηματισμένες μεταβλητές.

Στο πλαίσιο της παρούσας έρευνας τα OLS coefficients συνοδεύτηκαν από cluster-robust standard errors σε επίπεδο project. Η συγκεκριμένη επιλογή δεν είναι ικανή να μεταβάλλει τις ίδιες τις εκτιμήσεις των συντελεστών, αλλά προσαρμόζει την εκτίμηση της αβεβαιότητάς τους με σκοπό να λαμβάνεται υπόψη ότι Pull Requests του ίδιου project ενδέχεται να παρουσιάζουν μεταξύ τους συσχέτιση. Παρότι η προσαρμογή αυτή καθιστά την inferential αξιολόγηση του OLS πιο κατάλληλη για τη δομή των δεδομένων, το μοντέλο εξακολουθεί να μην αποτυπώνει ρητά τη μεταξύ-project μεταβλητότητα. Έτσι, χρησιμοποιείται κυρίως ως baseline model, βάσει του οποίου αξιολογείται παρακάτω η καταλληλότερη ιεραρχική προσέγγιση των Linear Mixed Models (LMM).

7.4.2 Παραδοχές

Η εφαρμογή της μεθόδου Ordinary Least Squares βασίζεται σε ένα σύνολο παραδοχών που αφορούν τη μορφή του μοντέλου, τη συμπεριφορά των υπολοίπων και τις σχέσεις μεταξύ των μεταβλητών που συμμετέχουν σε αυτό. Η εξέταση αυτών των παραδοχών κρίνεται απαραίτητη, καθώς πιθανές αποκλίσεις δεν επηρεάζουν πάντα με τον ίδιο ακριβώς τρόπο τις εκτιμήσεις των συντελεστών, μπορούν όμως να περιορίσουν την αξιοπιστία της στατιστικής συμπερασματολογίας που προκύπτει από το μοντέλο. Οι κυριότερες παραδοχές του OLS που σχετίζονται με την παρούσα ανάλυση είναι οι ακόλουθες:

- **Η εγκυρότητα της OLS συμπερασματολογίας εξετάζεται μέσω παραδοχών για τη μορφή της σχέσης, τη διασπορά και την κατανομή των σφαλμάτων, την ανεξαρτησία και την πολυσυγγραμμικότητα. Οι έλεγχοι**

περιγράφονται μαζί με τα **diagnostics** που εφαρμόστηκαν στα συγκεκριμένα δεδομένα (Field, 2018).

- **Ανεξαρτησία των σφαλμάτων:** Στο κλασικό μοντέλο OLS θεωρείται ότι τα σφάλματα των διαφορετικών παρατηρήσεων δεν παρουσιάζουν συστηματική συσχέτιση μεταξύ τους. Η συγκεκριμένη παραδοχή έχει ιδιαίτερη σημασία στην παρούσα έρευνα, καθώς διαφορετικά PRs μπορεί να ανήκουν στο ίδιο repository και να μοιράζονται χαρακτηριστικά που σχετίζονται με την αρχιτεκτονική, τις τεχνολογίες, τις πρακτικές ανάπτυξης ή τους κανόνες ποιότητας του εκάστοτε έργου λογισμικού. Η πλήρης ανεξαρτησία, άρα, των παρατηρήσεων δεν μπορεί να θεωρηθεί δεδομένη. Για τον περιορισμό της επίδρασης αυτής της εξάρτησης, τα OLS μοντέλα χρησιμοποιούν cluster-robust standard errors σε επίπεδο project, με το repositoryName ως μεταβλητή ομαδοποίησης των Pull Requests. Η προσέγγιση αυτή καθιστά τη στατιστική συμπερασματολογία πιο ανθεκτική στην ενδοομαδική συσχέτιση, χωρίς όμως να μοντελοποιεί άμεσα την ιεραρχική δομή των δεδομένων.
- **Ομοσκεδαστικότητα:** -ορίζεται ως η ιδιότητα σύμφωνα με την οποία ένα σύνολο τυχαίων μεταβλητών ή τα σφάλματα ενός μοντέλου παλινδρόμησης έχουν την ίδια και παράλληλα σταθερή διασπορά (διακύμανση)- Η παραδοχή της ομοσκεδαστικότητας απαιτεί η διακύμανση των σφαλμάτων να παραμένει περίπου σταθερή για διαφορετικά επίπεδα επεξηγηματικών μεταβλητών και των προβλεπόμενων τιμών του μοντέλου. Στην περίπτωση των μετρικών λογισμικού, αυτή η συνθήκη δεν λαμβάνεται ως αυτονόητη, καθώς Pull Request διαφορετικού μεγέθους ή πολυπλοκότητας μπορεί να εμφανίζουν διαφορετικά επίπεδα μεταβλητότητας. Η χρήση cluster-robust standard errors μειώνει αρκετά την εξάρτηση της στατιστικής συμπερασματολογίας από την αυστηρή υπόθεση της ομοσκεδαστικότητας, χωρίς αυτό να συνεπάγεται ότι μία πιθανή ετεροσκεδαστικότητα παύει να αποτελεί χαρακτηριστικό των δεδομένων.
- **Κανονικότητα των υπολοίπων:** Η κανονικότητα αφορά την κατανομή των σφαλμάτων του μοντέλου και όχι απαραίτητα την κατανομή των αρχικών μεταβλητών. Η παραδοχή είναι περισσότερο σημαντική για την ακριβή στατιστική συμπερασματολογία, ιδιαίτερα σε μικρά δείγματα. Στην παρούσα ανάλυση, όπως περιγράφεται, οι λογαριθμικοί μετασχηματισμοί περιορίζουν την έντονη ασυμμετρία των μετρικών, χωρίς όμως από μόνοι τους να αποδεικνύουν ότι τα υπόλοιπα ακολουθούν κανονική κατανομή.
- **Απουσία έντονης πολυσυγγραμμικότητας:** Το OLS μοντέλο ανάλυσης προϋποθέτει εξ ορισμού την συνθήκη που θέλει τις ανεξάρτητες μεταβλητές να μην παρουσιάζουν τέλεια ή εξαιρετικά υψηλή μεταξύ τους γραμμική συσχέτιση. Αντίθετα, δυσχεραίνεται ο διαχωρισμός της επιμέρους επίδρασης κάθε μεταβλητής και αυξάνεται η αβεβαιότητα των εκτιμώμενων συντελεστών. Συνεπώς, πιθανή συσχέτιση μεταξύ μεγέθους και πολυπλοκότητας πρέπει να λαμβάνεται υπόψη κατά την ερμηνεία των αντίστοιχων συντελεστών.

Το OLS, συνολικά, χρησιμοποιείται στην παρούσα εργασία ως ένα βασικό και εύκολα ερμηνεύσιμο μοντέλο αναφοράς για την αποτύπωση των γενικών σχέσεων μεταξύ των χαρακτηριστικών των Pull Requests και των μετρικών ποιότητας και ασφάλειας. Η ύπαρξη, όμως, πολλών PRs ανά μερίδα project δημιουργεί μία εγγενή ιεραρχική δομή στα δεδομένα, η οποία δεν μπορεί να αποδοθεί πλήρως από ένα συμβατικό γραμμικό μοντέλο. Η χρήση cluster-robust standard errors, εδώ, λειτουργεί ως πρώτο επίπεδο αντιμετώπισης της ενδοομαδικής εξάρτησης, ενώ η δομή αυτή εξετάζεται πληρέστερα μέσω της εφαρμογής των Linear Mixed-Effects Models που ακολουθούν στη συνέχεια της παρούσας βιβλιογραφίας.

7.4.3 Multicollinearity και VIF

Σημαντικό ζήτημα στην πολλαπλή γραμμική παλινδρόμηση αποτελεί η ύπαρξη ισχυρής γραμμικής σχέσης μεταξύ δύο ή περισσότερων ανεξάρτητων μεταβλητών, επονομαζόμενη ως πολυσυγγραμμικότητα (multicollinearity). Η παρουσία της δεν συνεπάγεται απαραίτητως κακή συνολική προσαρμογή του μοντέλου, μπορεί όμως να αυξήσει τα standard errors των εκτιμώμενων συντελεστών και να δυσχεράνει τον διαχωρισμό της επιμέρους επίδρασης κάθε predictor.

Ο VIF ποσοτικοποιεί πόσο αυξάνεται η διακύμανση ενός εκτιμώμενου συντελεστή λόγω γραμμικής συσχέτισης του αντίστοιχου predictor με τους υπόλοιπους predictors. Δεν υπάρχει απολύτως καθολικό cut-off, επομένως οι τιμές του πρέπει να αξιολογούνται μαζί με το μοντέλο και το ερευνητικό πλαίσιο (Field, 2018).

$$VIF_j = \frac{1}{1 - R_j^2}$$

Όπου R_j^2 είναι ο συντελεστής προσδιορισμού που προκύπτει όταν η x_j παλινδρομείται ως προς τις υπόλοιπες ανεξάρτητες μεταβλητές του μοντέλου. Τιμή ίση με 1 υποδηλώνει απουσία γραμμικής εξάρτησης από τους υπόλοιπους predictors, ενώ υψηλότερες τιμές αποτελούν ένδειξη αυξανόμενης πολυσυγγραμμικότητας. Στη βιβλιογραφία συνηθίζεται η χρήση τιμών όπως είναι το 5 ή 10 ως ενδεικτικά όρια περαιτέρω διερεύνησης, χωρίς όμως να αποτελούν απόλυτους κανόνες.

Το ζήτημα στα OLS μοντέλα της παρούσας εργασίας αφορά κυρίως τις συνεχείς μεταβλητές *log_ncloc* και *log_complexity*. Η μεταξύ τους συσχέτιση είναι θεωρητικά πιθανή, καθώς μεγαλύτερα τμήματα κώδικα μπορούν παράλληλα να εμφανίζουν μεγαλύτερη συνολική πολυπλοκότητα. Παρ' όλα αυτά, οι δύο μεταβλητές εκφράζουν διαφορετικές διαστάσεις του κώδικα και έχουν ξεχωριστή ερμηνευτική σημασία, γεγονός που δεν δικαιολογεί την αυτόματη αφαίρεση κάποιας από τις δύο ακόμη και σε περίπτωση αυξημένης συσχέτισης.

7.4.4 Residual diagnostics

Τα residual diagnostics χρησιμοποιούνται προκειμένου να αξιολογηθεί η συμπεριφορά των υπολοίπων ενός μοντέλου παλινδρόμησης και αποτελούν βασικό εργαλείο για τον εντοπισμό πιθανών αποκλίσεων από τις παραδοχές του OLS. Για κάθε παρατήρηση, το residual εκφράζει τη διαφορά μεταξύ της πραγματικής και της προβλεπόμενης τιμής της εξαρτημένης μεταβλητής, η οποία ορίζεται αλγεβρικά με τον παρακάτω τύπο:

$$e_i = y_i - \hat{y}_i$$

όπου y_i είναι η πραγματική τιμή και $\hat{y}_i$ η τιμή που το μοντέλο προβλέπει.

Η διερεύνηση των residuals πραγματοποιείται συνήθως με γραφικές διαγνωστικές μεθόδους. Ένα Residuals vs Fitted plot επιτρέπει την εξέταση της περίπτωσης της κατανομής των υπολοίπων χωρίς εμφανές συστηματικό μοτίβο γύρω από το μηδέν. Η εμφάνιση καμπυλότητας μπορεί να υποδηλώνει ότι η σχέση δεν αποδίδεται επαρκώς από τη γραμμική μορφή του μοντέλου, ενώ μεταβολές στη διασπορά των residuals κατά μήκος των fitted values μπορούν να αποτελούν έδειξε ετεροσκεδαστικότητας. Ένα Q-Q plot, αντίστοιχα, χρησιμοποιείται με σκοπό την αξιολόγηση της προσεγγιστικής κανονικότητας των υπολοίπων, συγκρίνοντας την εμπειρική τους κατανομή με εκείνη που θα αναμενόταν υπό κανονική κατανομή.

Επίσης, μπορούν να αναδείξουν παρατηρήσεις οι οποίες αποκλίνουν έντονα από τις προβλέψεις του μοντέλου. Τέτοιες περιπτώσεις δεν πρέπει να απομακρύνονται αυτόματα, καθώς μπορεί να αντιπροσωπεύουν πραγματικά χαρακτηριστικά των δεδομένων. Σημειώνεται η απαίτηση περαιτέρω διερεύνησης, ιδιαίτερα σε περιπτώσεις όπου μία παρατήρηση ασκεί δυσανάλογη επίδραση στην εκτίμηση των συντελεστών.

7.4.5 Περιορισμός ανεξαρτησίας των PRs

Ένας σημαντικός περιορισμός της εφαρμογής του OLS στην παρούσα έρευνα αφορά την ανεξαρτησία των παρατηρήσεων. Η βασική μονάδα ανάλυσης, όπως αυτή ορίστηκε, είναι το Pull Request, ωστόσο τα PRs δεν αποτελούν πλήρως ανεξάρτητες παρατηρήσεις, καθώς πολλά από αυτά προέρχονται από το ίδιο repository. PRs του ίδιου project είναι πιθανό να επηρεάζονται από κοινά χαρακτηριστικά, όπως η αρχιτεκτονική του συστήματος, οι χρησιμοποιούμενες τεχνολογίες, οι πρακτικές ανάπτυξης, οι κανόνες ποιότητας και οι συνήθειες της ομάδας ανάπτυξης.

Η συγκεκριμένη ομαδοποίηση δημιουργεί πιθανή ενδοομαδική συσχέτιση μεταξύ των σφαλμάτων των PRs που ανήκουν στο ίδιο project. Σε περίπτωση αγνόησης αυτής και χρήσης συμβατικών standard errors του OLS, είναι δυνατό να υποεκτιμηθεί η πραγματική αβεβαιότητα των εκτιμήσεων και, κατά συνέπεια, να προκύψουν υπερβολικά αισιόδοξα confidence intervals ή έλεγχοι στατιστικής σημαντικότητας.

Για τον περιορισμό του συγκεκριμένου προβλήματος, τα OLS μοντέλα της παρούσας ανάλυσης χρησιμοποιούν cluster-robust standard errors σε επίπεδο repository, με το repositoryName να καθορίζει τις ομάδες δεδομένων Pull Request. Με τον τρόπο αυτό επιτρέπεται στα σφάλματα των PRs του ίδιου project να παρουσιάζουν μεταξύ τους συσχέτιση, ενώ οι διαφορετικές ομάδες αντιμετωπίζονται ως ανεξάρτητες. Η προσαρμογή αυτή αφορά κυρίως την εκτίμηση της αβεβαιότητας των συντελεστών και δεν μεταβάλλει από μόνη της τη βασική δομή του OLS μοντέλου.

Αν και τα cluster-robust standard errors προσφέρουν μία πιο κατάλληλη βάση για τη στατική συμπεριματολογία, δεν μοντελοποιούν άμεσα τις διαφορές μεταξύ των projects. Έτσι, το μοντέλο OLS χρησιμοποιείται ως baseline, ενώ η ιεραρχική δομή των δεδομένων αντιμετωπίζεται πληρέστερα μέσω των Linear Mixed-Effects Models, στα οποία εισάγεται τυχαίο intercept ανά project. Με αυτόν τον τρόπο μπορεί να αποτυπωθεί ρητά η μεταβλητότητα που οφείλεται στο repository και να διαχωριστεί από τη μεταβλητότητα που παρατηρείται μεταξύ των επιμέρους Pull Requests.

7.5 Linear Mixed-Effects Models (LMM)

Τα Linear Mixed-Effects Models συνδυάζουν fixed effects, που περιγράφουν μέσες σχέσεις στον πληθυσμό, και random effects, που αποτυπώνουν μεταβλητότητα μεταξύ ομάδων. Στην παρούσα έρευνα η ομαδοποίηση αντιστοιχεί στα projects (Bates et al., 2015). Η δομή αυτή δημιουργεί μία φυσική ιεραρχία στα δεδομένα, με τα Pull Requests να βρίσκονται σε πρώτο επίπεδο και τα projects να αποτελούν το ανώτερο επίπεδο ομαδοποίησης.

Προκειμένου να αντιμετωπιστεί πληρέστερα αυτή η εξάρτηση χρησιμοποιήθηκαν Linear Mixed-Effects Models (LMM). Αντίθετα με ένα συμβατικό OLS μοντέλο, τα μικτά γραμμικά μοντέλα επιτρέπουν την ταυτόχρονη εκτίμηση επιδράσεων που θεωρούνται κοινές για το σύνολο των παρατηρήσεων και επιδράσεων που σχετίζονται με τις επιμέρους ομάδες του δείγματος. Με τον τρόπο αυτό μπορούν να ληφθούν υπόψη συστηματικές διαφορές μεταξύ του επιπέδου των repositories, χωρίς να προβλέπεται η επίδραση των χαρακτηριστικών κάθε Pull Request.

Στην παρούσα ανάλυση, η εφαρμογή των LMM επιτρέπει τη διάκριση της μεταβλητότητας η οποία εμφανίζεται μεταξύ των διαφορετικών projects από εκείνη που παρατηρείται μεταξύ Pull Requests του ίδιου project. Παρακάτω παρουσιάζονται τα fixed και random effects, η δομή των random-intercept και, σε περιπτώσεις που αυτό δικαιολογείται, random-slope μοντέλων. Τα variance components και ο Intraclass Correlation Coefficient (ICC). Εξετάζονται, παράλληλα, μέθοδοι εκτίμησης ML και REML, οι δείκτες marginal και conditional (R^2), τα κριτήρια σύγκρισης μοντέλων AIC και BIC, καθώς και οι απαραίτητοι διαγνωστικοί έλεγχοι και ζητήματα σύγκλισης των μοντέλων.

7.5.1 Fixed και Random effects

Τα Linear Mixed-Effects Models συνδυάζουν δύο διαφορετικά είδη επιδράσεων, τα fixed effects και τα random effects. Η διάκριση αυτή επιτρέπει στο μοντέλο να εξετάζει τόσο τις σχέσεις που αφορούν συνολικά το σύνολο των Pull Requests όσο και τη μεταβλητότητα που προκύπτει από την ομαδοποίησή τους σε διαφορετικά projects.

Τα Fixed Effects αντιπροσωπεύουν τις επιδράσεις των επεξηγηματικών μεταβλητών οι οποίες εκτιμώνται για το σύνολο του πληθυσμού το οποίο εξετάζεται. Οι αντίστοιχοι συντελεστές περιγράφουν τη μέση αναμενόμενη μεταβολή της εξαρτημένης μεταβλητής όταν μεταβάλλεται ένας predictor, διατηρώντας τους υπόλοιπους σταθερούς. Στην παρούσα έρευνα, στα fixed effects περιλαμβάνονται μεταβλητές που χαρακτηρίζουν τα Pull Requests, όπως το μέγεθος του κώδικα μέσω του `log_ncloc`, η πολυπλοκότητα μέσω του `log_complexity` και η κατηγορία του PR. Οι συντελεστές τους, επομένως, εκφράζουν τις μέσες σχέσεις των συγκεκριμένων χαρακτηριστικών με τις εξεταζόμενες μετρικές ποιότητας ή ασφάλειας, λαμβάνοντας παράλληλα υπόψη την ομαδοποίηση των παρατηρήσεων ανά project.

Από την άλλη πλευρά, η χρήση των Random Effects έγκειται στην αποτύπωση της μεταβλητότητας που συνδέεται με τις ομάδες στις οποίες ανήκουν οι παρατηρήσεις. Στη συγκεκριμένη περίπτωση, η ομάδα αντιστοιχεί στο γενικότερο project/repository. Τα PRs του ίδιου project, πιθανώς να εμφανίζουν μεγαλύτερη ομοιότητα μεταξύ τους σε σχέση με PRs διαφορετικών projects, λόγω κοινών χαρακτηριστικών όπως η αρχιτεκτονική, οι τεχνολογίες, οι πρακτικές ανάπτυξης ή οι διαδικασίες ελέγχου του κώδικα. Τα random effects επιτρέπουν στο μοντέλο να ενσωματώσει τη μεταξύ-project μεταβλητότητα αντί να θεωρεί ότι όλα τα repositories έχουν ακριβώς το ίδιο βασικό επίπεδο της εξαρτημένης μεταβλητής.

Σε μία απλή μορφή Mixed Model, η σχέση μπορεί να αποδοθεί αλγεβρικά ως εξής:

$$\beta_0 + \beta_1 X_{1ij} + \dots + \beta_p X_{pij} + u_{0j} + \varepsilon_{ij}$$

όπου (*i*) συμβολίζει την οντότητα του Pull Request και (*j*) το project στο οποίο αυτό ανήκει. Οι παράμετροι $\beta_0, \beta_1, \dots, \beta_p$ αποτελούν τα fixed effects, ενώ το u_{0j} εκφράζει την απόκλιση του project (*j*) από το γενικό intercept του μοντέλου. Το ε_{ij} αντιπροσωπεύει την υπολειμματική μεταβλητότητα η οποία παραμένει στην ανάλυση σε επίπεδο Pull Request.

Η χρήση των random effects δεν σημαίνει ότι τα projects θεωρούνται τυχαία, όπως ο συγκεκριμένος όρος χρησιμοποιείται. Η μεταξύ τους διαφοροποίηση, αντίθετα, αντιμετωπίζεται ως μία πηγή μεταβλητότητας που εκτιμάται από τα δεδομένα. Έτσι, αντί να υπολογίζεται ένας ανεξάρτητος σταθερός συντελεστής για κάθε repository, εκτιμάται η συνολική διακύμανση που χαρακτηρίζει τις αποκλίσεις των projects από τη μέση συμπεριφορά.

Ο συνδυασμός αυτών των δύο τύπων, fixed και random effects, είναι και το βασικό πλεονέκτημα των LMM για την παρούσα έρευνα. Τα fixed effects επιτρέπουν

την εξέταση των γενικών σχέσεων μεταξύ των χαρακτηριστικών των PRs και των μετρικών ποιότητας ή ασφάλειας, ενώ τα random effects λαμβάνουν υπόψη το γεγονός ότι οι παρατηρήσεις είναι οργανωμένες μέσα σε διαφορετικά projects, όπως προβλέπεται εκ φύσεως εδώ. Η απλούστερη εφαρμογή της λογικής είναι το random-intercept model, το οποίο παρουσιάζεται στην συνέχεια.

7.5.2 Random-intercept model

Το Random-intercept model είναι η βασική μορφή που χρησιμοποιείται στην παρούσα έρευνα. Με την χρήση αυτού επιτρέπεται σε κάθε project να διαθέτει διαφορετικό αρχικό επίπεδο της εξαρτημένης μεταβλητής. Η επιλογή αυτή ανταποκρίνεται στην ιεραρχική δομή των δεδομένων, καθώς τα PRs είναι ομαδοποιημένα μέσα σε repositories και επομένως PRs του ίδιου project ενδέχεται να παρουσιάζουν μεγαλύτερη μεταξύ τους ομοιότητα από PRs διαφορετικών projects.

Μαθηματικά, η γενική μορφή του μοντέλου εκφράζεται ως εξής:

$$\beta_0 + \beta_1 X_{1ij} + \beta_2 X_{2ij} + \dots + \beta_p X_{pij} + u_{0j} + \varepsilon_{ij}$$

όπου:

- y_i : τιμή της εξαρτημένης μεταβλητής για το Pull Request (i) του project (j),
- $\beta_0, \beta_1, \beta_2, \dots, \beta_p$: fixed effects,
- u_{0j} : random intercept του project (j),
- ε_{ij} : υπολειμματικό σφάλμα σε επίπεδο Pull Request.

Στην παρούσα ανάλυση, τα fixed effects περιλαμβάνουν δύο μετρήσιμα χαρακτηριστικά της τελικής κατάστασης του αναλυόμενου κώδικα, το log_ncloc και το log_complexity, καθώς και την κατηγορία του αντίστοιχου Pull Request. Το γενικό intercept (β_0) αντιπροσωπεύει το αναμενόμενο επίπεδο της εξαρτημένης μεταβλητής για το σύνολο των projects, ενώ το (u_{0j}) επιτρέπει σε κάθε repository να αποκλίνει από το επίπεδο αυτό. Έτσι, το πραγματικό intercept για ένα συγκεκριμένο project ανάγεται μαθηματικά σε:

$$\beta_0 + u_{0j}$$

Με τον τρόπο αυτό, δύο παρατηρήσεις με παρόμοιο μέγεθος και πολυπλοκότητα του αναλυόμενου code snapshot και ίδια κατηγορία Pull Request δεν είναι υποχρεωτικό να έχουν την ίδια προβλεπόμενη τιμή όταν προέρχονται από διαφορετικά projects. Το μοντέλο αναγνωρίζει ότι μέρος της διαφοροποίησης μπορεί να οφείλεται σε χαρακτηριστικά του repository τα οποία δεν συμπεριλαμβάνονται στην ορισμό του ρητά ως predictors, όπως διαφορετικές πρακτικές ανάπτυξης, αρχιτεκτονικές επιλογές ή διαδικασίες ελέγχου του πηγαίου κώδικα.

Συνήθως θεωρείται ότι τα random intercepts ακολουθούν κανονική κατανομή γύρω από το μηδέν:

$$u_{0j} \sim N(0, \sigma_u^2)$$

Τα υπολειμματικά σφάλματα περιγράφονται ως:

$$\varepsilon_{0j} \sim N(0, \sigma_\varepsilon^2)$$

Ο ποσοτικός όρος σ_u^2 εκφράζει τη μεταβλητότητα μεταξύ των projects, ενώ η σ_ε^2 τη μεταβλητότητα που παραμένει μεταξύ των PRs του ίδιου project. Η διάκριση αυτών των δύο πηγών διακύμανσης αποτελεί ένα από τα βασικά πλεονεκτήματα του mixed model.

Το random-intercept model εφαρμόζει partial pooling: οι εκτιμήσεις ομάδων με περιορισμένη πληροφορία συρρικνώνονται περισσότερο προς τη συνολική μέση τιμή, ενώ ομάδες με περισσότερες παρατηρήσεις υποστηρίζουν περισσότερο διακριτές εκτιμήσεις (Bates et al., 2015). Με τον τρόπο αυτό αποφεύγονται τόσο η πλήρης αγνόηση των διαφορών μεταξύ repositories όσο και η αντιμετώπιση κάθε project ως εντελώς ανεξάρτητης περίπτωσης. Η ιδιότητα αυτή είναι ιδιαίτερα σημαντική εδώ, λόγω της μεγάλης ανισορροπίας που παρουσιάζει ο αριθμός των PRs που αντιστοιχεί σε κάθε έργο λογισμικού.

7.5.3 Variance components

Ένα από τα βασικά πλεονεκτήματα των Linear Mixed-Effects Models είναι η δυνατότητα διαχωρισμού της συνολικής μεταβλητότητας της εξαρτημένης μεταβλητής σε διαφορετικές πηγές. Οι επιμέρους αυτές πηγές ονομάζονται variance components και, στην περίπτωση ενός random-intercept μοντέλου, επιτρέπουν να προσδιοριστεί ποιο μέρος της διακύμανσης συνδέεται με τις διαφορές μεταξύ των projects και ποιο παραμένει σε επίπεδο Pull Request.

Η μεταβλητότητα για το random-intercept της παρούσας ανάλυσης, διακρίνεται κυρίως σε δύο συνιστώσες:

- **Between-project variance (σ_u^2):** εκφράζει τον βαθμό στον οποίο τα διαφορετικά projects αποκλίνουν μεταξύ τους ως προς το βασικό επίπεδο της εξαρτημένης μεταβλητής. Μεγάλη τιμή της συγκεκριμένης διακύμανσης υποδηλώνει ότι το repository στο οποίο ανήκει ένα PR αποτελεί σημαντική πηγή διαφοροποίησης των παρατηρούμενων τιμών.
- **Residual ή within-project variance (σ_ε^2):** εκφράζει τη μεταβλητότητα που εξακολουθεί να υπάρχει μεταξύ των Pull Requests του ίδιου project, αφού έχουν ληφθεί υπόψη τόσο τα fixed effects όσο και το random intercept του project. Περιλαμβάνει επομένως διαφοροποιήσεις μεταξύ PRs που δεν εξηγούνται από τις μεταβλητές του μοντέλου.

Στην απλή περίπτωση του random-intercept μοντέλου, η συνολική μη εξηγούμενη διακύμανση μπορεί να θεωρηθεί ότι αποτελείται από τις δύο αυτές συνιστώσες:

$$\sigma_u^2 + \sigma_\varepsilon^2$$

Ο διαχωρισμός αυτός είναι ιδιαίτερος σημαντικός για την παρούσα έρευνα, καθώς επιτρέπει την εξέταση της περίπτωσης διαφοροποιήσεων πυκνοτήτων να οφείλονται κυρίως σε χαρακτηριστικά των επιμέρους Pull Requests ή σε συστηματικές διαφορές μεταξύ των repositories. Ένα υψηλό between-project variance θα σήμαινε ότι PRs που ανήκουν στο ίδιο project τείνουν να εμφανίζουν μεγαλύτερη ομοιότητα μεταξύ τους, ακόμη και μετά τον έλεγχο μεταβλητών όπως το μέγεθος, η πολυπλοκότητα και η κατηγορία του PR.

Η εκτίμηση των variance components παρέχει εν τέλει πληροφορία που δεν μπορεί να αποδώσει ένα συμβατικό pooled OLS μοντέλο. Στην παρούσα ανάλυση, η έντονη διαφοροποίηση σε επίπεδο project αποτελεί βασικό στοιχείο για την επιλογή του mixed-effects μοντέλου ως κύριας στατιστικής προσέγγισης. Η σχετική συμμετοχή της between-project variance στη συνολική διακύμανση αποτυπώνεται ποσοτικά μέσω του Intraclass Correlation Coefficient (ICC).

7.5.4 Intraclass Correlation Coefficient (ICC)

Ο Interclass Correlation Coefficient (ICC) λαμβάνεται ως βασικός δείκτης για την αξιολόγηση της ομαδοποίησης των παρατηρήσεων σε δεδομένα με ιεραρχική δομή. Στην περίπτωση της παρούσας έρευνας, χρησιμοποιείται για να εκφράσει το ποσοστό της συνολικής διακύμανσης της εξαρτημένης μεταβλητής που αποδίδεται στις διαφορές μεταξύ των projects και, παράλληλα, σε ποιο βαθμό τείνουν να παρουσιάζουν μεγαλύτερη ομοιότητα μεταξύ τους τα Pull Requests που ανήκουν στο ίδιο repository/project.

Στο random-intercept LMM, το ICC εκφράζει το ποσοστό της συνολικής διακύμανσης που αποδίδεται στις διαφορές μεταξύ projects. Υψηλή τιμή υποδηλώνει ότι Pull Requests του ίδιου project τείνουν να μοιάζουν περισσότερο μεταξύ τους ως προς το outcome (Nakagawa, Johnson & Schielzeth, 2017).

$$ICC = \frac{\sigma_u^2}{\sigma_u^2 + \sigma_\varepsilon^2}$$

όπου σ_u^2 είναι η between-project variance και σ_ε^2 η residual ή within-project variance. Ο δείκτης μπορεί να λάβει τιμές από 0 έως και 1. Τιμή κοντά στο 0 υποδηλώνει ότι μικρό μέρος της συνολικής μεταβλητότητας σχετίζεται με το επίπεδο του project, ενώ υψηλότερες τιμές φανερώνουν ισχυρότερη ομοιότητα μεταξύ παρατηρήσεων της ίδιας ομάδας και μεγαλύτερη σημασία της ιεραρχικής δομής των δεδομένων.

Στην παρούσα ανάλυση, το επίπεδο του project βρέθηκε να εξηγεί το ~86.6%, της διακύμανσης στην πυκνότητα των quality issues, γεγονός που αντιστοιχεί σε ιδιαίτερα υψηλό βαθμό ενδοομαδικής συσχέτισης. Πρακτικά, τα αποτελέσματα υποδηλώνει ότι σημαντικό μέρος των διαφορών που παρατηρούνται στην

πυκνότητα των προβλημάτων ποιότητας σχετίζεται με το repository στο οποίο και ανήκει κάθε PR και όχι αποκλειστικά τα επιμέρους χαρακτηριστικά του PR.

Αυτό το εύρημα έχει άμεση μεθοδολογική σημασία. Με τόσο έντονη εξάρτηση σε επίπεδο project, η αντιμετώπιση όλων των Pull Requests ως πλήρως ανεξάρτητων μεταξύ τους παρατηρήσεων θα παράβλεπε ένα ουσιαστικό μέρος της δομής και της φύσης των δεδομένων. Ένα pooled OLS μοντέλο, ακόμη και όταν χρησιμοποιείται ως χρήσιμο baseline, δεν αποτυπώνει ρητά τη μεταξύ-project μεταβλητότητα. Αντίθετα, το random-intercept LMM επιτρέπει σε κάθε project να διαθέτει διαφορετικό βασικό επίπεδο της εξαρτημένης μεταβλητής και ενσωματώνει τη συγκεκριμένη πηγή διαφοροποίησης απευθείας στο μοντέλο.

Ο υψηλός ICC δεν πρέπει να ερμηνευθεί ως ένδειξη ότι το ίδιο το project αποτελεί αιτία ύπαρξης των quality issues. Ο δείκτης αποτυπώνει το ποσοστό της διακύμανσης που σχετίζεται με την ομαδοποίηση των παρατηρήσεων και όχι κάποια αιτιώδη σχέση. Οι διαφορές μεταξύ projects μπορούν να αντανakλούν ένα σύνολο μη παρατηρούμενων χαρακτηριστικών, μεταξύ των οποίων διαφορετικές αρχιτεκτονικές, γλώσσες και τεχνολογίες, πρακτικές ανάπτυξης, βαθμό ωριμότητας του κώδικα ή διαδικασίες ελέγχου ποιότητας.

Συμπεραίνουμε επομένως ότι η ιδιαίτερα υψηλή συμμετοχή του project στη συνολική μεταβλητότητα παρέχει ισχυρή εμπειρική τεκμηρίωση για τη χρήση των Linear Mixed-Effects Models ως κύριας μεθόδου της έρευνας. Παράλληλα, αναδεικνύει ότι η ποιότητα του κώδικα δεν δύναται να εξεταστεί αποκλειστικά ως χαρακτηριστικό μεμονωμένων Pull Requests, αλλά πρέπει να λαμβάνεται υπόψη και το ευρύτερο project context μέσα στο οποίο αυτά δημιουργούνται.

7.5.5 ML και REML

Τα LMM μπορούν να εκτιμηθούν με Maximum Likelihood ή Restricted Maximum Likelihood. Το REML διορθώνει την εκτίμηση των variance components για την αβεβαιότητα των fixed effects, ενώ το ML είναι συνήθως η κατάλληλη βάση όταν συγκρίνονται μοντέλα με διαφορετική fixed-effects δομή (Bates et al., 2015). Για την διάκριση της συνεισφοράς μεταξύ των fixed και random effects συχνά χρησιμοποιούνται τα Marginal και Conditional R^2 , χωρίς να αποτελούν μέρος μελέτης και παρουσίασης της συγκεκριμένης βιβλιογραφικής προσέγγισης.

Η Maximum Likelihood εκτιμά ταυτόχρονα τόσο τα fixed effects όσο και τις παραμέτρους διακύμανσης του μοντέλου επιλέγοντας τις τιμές που καθιστούν τα παρατηρούμενα δεδομένα όσο το δυνατόν πιθανότερα. Η προσέγγιση αυτή είναι ιδιαίτερα χρήσιμη όταν συγκρίνονται μοντέλα που περιλαμβάνουν διαφορετικά fixed effects, καθώς οι αντίστοιχες likelihood-based ποσότητες μπορούν να χρησιμοποιηθούν για τη σύγκριση της προσαρμογής τους.

Αντίθετα με την ML, η Restricted Maximum Likelihood προσαρμόζει τη διαδικασία εκτίμησης ώστε να λαμβάνει υπόψη την απώλεια βαθμών ελευθερίας που προκαλείται από την εκτίμηση των fixed effects. Η REML, επομένως, παρέχει συνήθως λιγότερο μεροληπτικές εκτιμήσεις των variance components, ιδιαίτερα όταν ο αριθμός των ομάδων ή των διαθέσιμων παρατηρήσεων δεν είναι πολύ

μεγάλος. Αποτελεί συνηθισμένη επιλογή όταν το ενδιαφέρον επικεντρώνεται στην τελική εκτίμηση ενός ήδη καθορισμένου mixed model.

Στην παρούσα εργασία, τα κύρια random-intercept LMM για την πυκνότητα των quality και security issues εκτιμήθηκαν με χρήση της REML. Η επιλογή αυτή είναι συνεπής με τον βασικό στόχο της ανάλυσης, ο οποίος δεν περιορίζεται στην εκτίμηση των fixed-effect coefficients, αλλά περιλαμβάνει και την αξιόπιστη αποτύπωση της μεταβλητότητας μεταξύ των projects μέσω του random effect. Η ίδια μέθοδος χρησιμοποιήθηκε και στο sensitivity analysis που περιορίζει το δείγμα των εξεταζόμενων έργων λογισμικού με κριτήριο το «τουλάχιστον πέντε (5) Pull Requests».

Η επιλογή μεταξύ ML και REML αποκτά ιδιαίτερη σημασία κατά τη σύγκριση διαφορετικών μεταξύ τους mixed models. Σε περιπτώσεις όπου δύο μοντέλα διαφέρουν ως προς τα fixed effects, η σύγκρισή τους μέσω likelihood-based κριτηρίων πρέπει να πραγματοποιείται αφού έχουν εκτιμηθεί με ML. Όταν η δομή, αντίθετα, των fixed effects παραμένει ίδια και το ενδιαφέρον βρίσκεται στην τελική εκτίμηση των variance components, η REML αποτελεί πιο κατάλληλη επιλογή τεχνικής. Βάσει των παραπάνω, στην παρούσα εργασία η REML χρησιμοποιείται για την τελική προσαρμογή και ερμηνεία του κύριου LMM, ενώ ενδεχόμενες συγκρίσεις εναλλακτικών προδιαγραφών με διαφορετικά fixed effects θα απαιτούσαν επανεκτίμησή τους με ML.

7.5.6 AIC, BIC και model comparison

Τα AIC και BIC εξισορροπούν την προσαρμογή ενός μοντέλου με ποινή για την πολυπλοκότητά του. Χρησιμοποιούνται συγκριτικά μεταξύ μοντέλων που έχουν εκτιμηθεί σε συμβατή βάση και στο ίδιο δείγμα· μικρότερη τιμή υποδηλώνει καλύτερη σχετική ισορροπία, όχι απόλυτη ορθότητα (Burnham & Anderson, 2002).

Το κριτήριο AIC ορίζεται μαθηματικά ως:

$$AIC = -2\log(L) + 2k$$

όπου:

- (L) είναι η μέγιστη τιμή της likelihood του μοντέλου
- (k) είναι ο αριθμός των εκτιμώμενων παραμέτρων

Το κριτήριο BIC αντίστοιχα υπολογίζεται ως:

$$BIC = -2\log(L) + k\log(n)$$

όπου:

- (n) είναι ο αριθμός παρατηρήσεων
- (L) και (k) ορίζονται αντίστοιχα όπως και στο κριτήριο AIC

Και στα δύο παραπάνω κριτήρια, η μικρότερη τιμή υποδηλώνει προτιμότερο μοντέλο μεταξύ των μοντέλων που συμμετέχουν στη σύγκριση. Το BIC επιβάλλει, συνήθως, ισχυρότερη ποινή στην προσθήκη παραμέτρων, με αποτέλεσμα να τείνει να ευνοεί

περισσότερο απλές προδιαγραφές όταν η βελτίωση της προσαρμογής είναι περιορισμένη.

Οι απόλυτες τιμές των AIC και BIC δεν έχουν αυτοτελή ερμηνεία. Η χρησιμότητά τους προκύπτει από τη διαφορά των τιμών μεταξύ εναλλακτικών μοντέλων που έχουν εφαρμοστεί στην ίδια εξαρτημένη μεταβλητή και στο ίδιο σύνολο παρατηρήσεων. Με τον τρόπο αυτό μπορεί να εξεταστεί αν η πρόσθετη πολυπλοκότητα ενός μοντέλου δικαιολογείται από αντίστοιχη βελτίωση της προσαρμογής του.

Στα Linear Mixed-Effects Models επιπλέον πρέπει να λαμβάνεται υπόψη η μέθοδος εκτίμησης. Όταν συγκρίνονται μοντέλα με διαφορετική δομή fixed effects, η σύγκριση likelihood-based κριτηρίων πρέπει να πραγματοποιείται με μοντέλα εκτιμημένα μέσω Maximum Likelihood (ML) και όχι μέσω REML. Μετά την επιλογή της κατάλληλης προδιαγραφής, το τελικό μοντέλο μπορεί να επανεκτιμηθεί με REML για την εκτίμηση των variance components.

Στην παρούσα ανάλυση, τα κύρια LMM έχουν προσαρμοστεί με την χρήση της REML, καθώς το ενδιαφέρον επικεντρώνεται στην τελική εκτίμηση των fixed effects και της μεταξύ-project μεταβλητότητας. Παρότι έχει υλοποιηθεί και μία προαιρετική προδιαγραφή με random slope για το log_ncloc, το υπάρχον analysis pipeline δεν πραγματοποιεί ξεχωριστή σύγκριση των μοντέλων μέσω AIC ή BIC. Συνεπώς, δεν γίνεται χρήση αριθμητικών τιμών των συγκεκριμένων κριτηρίων ως τεκμήριο υπεροχής του κύριου random-intercept μοντέλου.

Η επιλογή του τελικού μοντέλου δεν θα πρέπει να βασίζεται αποκλειστικά σε ένα κριτήριο προσαρμογής. Τα AIC και BIC λειτουργούν συμπληρωματικά προς τη θεωρητική αιτιολόγηση της δομής του μοντέλου, τις εκτιμήσεις των variance components, τη σταθερότητα των συντελεστών, τη σύγκλιση και τους διαγνωστικούς ελέγχους. Εδώ, η επιλογή του random-intercept LMM βασίζεται κυρίως στην έντονη ομαδοποίηση ανά project των PRs που εμφανίζει καθώς και στην ανάγκη να ληφθεί υπόψη η μεταξύ-project μεταβλητότητα μέσω του partial pooling.

Οι τιμές log-likelihood, AIC και BIC των τελικών REML LMM της κύριας ανάλυσης δεν αντιπαραβάλλονται απευθείας με τις αντίστοιχες τιμές των OLS μοντέλων με σκοπό την τεκμηρίωση υπεροχής του ενός μοντέλου έναντι του άλλου. Στην παρούσα έρευνα η επιλογή του random-intercept LMM ως κύριου inferential model βασίζεται πρωτίστως στην ιεραρχική δομή των δεδομένων, στην έντονη project-level clustering που αποτυπώνεται μέσω των ICC και στην ανάγκη ρητής μοντελοποίησης της μεταξύ-project μεταβλητότητας.

7.5.7 Diagnostics και model convergence

Ο βαθμός αξιοπιστίας ενός Linear Mixed-Effects Model δεν αξιολογείται αποκλειστικά από τους εκτιμώμενους συντελεστές και τη στατιστική τους σημαντικότητα. Απαραίτητος είναι και ο έλεγχος της συμπεριφοράς του μοντέλου μετά την προσαρμογή του στα δεδομένα, προκειμένου να εντοπιστούν πιθανές αποκλίσεις από τις παραδοχές του, ακραίες παρατηρήσεις ή προβλήματα κατά την αριθμητική εκτίμηση των παραμέτρων.

Βασικό μέρος των διαγνωστικών ελέγχων αφορά τα residuals, τις διαφορές, δηλαδή, μεταξύ των πραγματικών και των προβλεπόμενων τιμών. Η εξέτασή τους μπορεί να αναδείξει συστηματικά μοτίβα που σχετίζονται με μη γραμμικότητα ετεροσκεδαστικότητα ή ακραίες παρατηρήσεις. Στην παρούσα ανάλυση υπολογίζονται residuals για το κύριο LMM των quality issues, χρησιμοποιώντας προβλέψεις που περιλαμβάνουν τόσο τα fixed όσο και τα random effects. Μεγάλες θετικές αποκλίσεις αντιστοιχούν σε Pull Requests που παρουσιάζουν περισσότερα quality issues από όσο θα αναμένονταν με βάση τα χαρακτηριστικά τους και το project στο οποίο και ανήκουν.

Σημαντικός έλεγχος, επίσης, είναι ο έλεγχος των random effects, καθώς ιδιαίτερα μεγάλες θετικές ή αρνητικές αποκλίσεις μπορεί να υποδηλώνουν projects που διαφοροποιούνται συστηματικά από τη συνολική συμπεριφορά του δείγματος. Στην παρούσα υλοποίηση οι εκτιμημένες project-specific αποκλίσεις εξάγονται και χρησιμοποιούνται για την κατάταξη των repositories βάσει του random intercept τους.

Παράλληλα, ένα mixed model θα πρέπει να παρουσιάζει αριθμητική σύγκλιση (convergence). Κατά τη διαδικασία προσαρμογής του μοντέλου χρησιμοποιείται επαναληπτική διαδικασία βελτιστοποίησης για την εκτίμηση των παραμέτρων. Η σύγκλιση σημαίνει ότι η διαδικασία κατέληξε σε σταθερή λύση σύμφωνα με τα κριτήρια του αλγορίθμου. Σε περιπτώσεις όπου ένα μοντέλο δεν συγκλίνει, οι εκτιμήσεις των συντελεστών και των variance components δεν μπορούν να θεωρηθούν εξίσου αξιόπιστες και απαιτείται διερεύνηση της προδιαγραφής του μοντέλου, της κλίμακας των μεταβλητών ή της πολυπλοκότητας της random-effects δομής.

Σημειώνεται επίσης, ότι ιδιαίτερη προσοχή απαιτείται επίσης σε περιπτώσεις singular fit, όπου μία ή περισσότερες συνιστώσες διακύμανσης προσεγγίζουν το μηδέν ή η random-effects δομή αποδεικνύεται υπερβολικά σύνθετη σε σχέση με την παρεχόμενη από τα δεδομένα πληροφορία. Το ζήτημα αυτό είναι ιδιαίτερα σχετικό με πιο σύνθετες προδιαγραφές, όπως τα random-slope models, και αποτελεί έναν επιπλέον λόγο για την προτίμηση της απλούστερης random-intercept δομής όταν δεν υπάρχει επαρκής εμπειρική τεκμηρίωση για μεγαλύτερη πολυπλοκότητα.

Στην τρέχουσα υλοποίηση του analysis pipeline πραγματοποιείται πρακτικός έλεγχος των residuals και των random effects του κύριου quality LMM, ενώ το προαιρετικό random-slope μοντέλο προστατεύεται από πλήρη αποτυχία της διαδικασίας προσαρμογής μέσω χειρισμού εξαιρέσεων. Ωστόσο, δεν έχουν υλοποιηθεί ξεχωριστά γραφικά diagnostics για την κατανομή των residuals ή των random effects, ούτε αυτοματοποιημένοι έλεγχοι singularity και influential observations. Τα συγκεκριμένα διαγνωστικά δεν παρουσιάζονται ως ήδη μέσω της έρευνας επιβεβαιωμένα αποτελέσματα.

Εν κατακλείδι, η τελική αξιολόγηση του κύριου random-intercept LMM βασίζεται στη σταθερότητα των εκτιμήσεών του, στη δυνατότητα προσαρμογής του μοντέλου και στη συνέπεια των αποτελεσμάτων με το sensitivity analysis. Η σχεδόν αμετάβλητη συμπεριφορά των βασικών συντελεστών όταν η ανάλυση περιορίζεται σε projects με τουλάχιστον πέντε (5) PRs διαθέσιμα παρέχει επιπλέον ένδειξη ότι τα

κύρια συμπεράσματα δεν εξαρτώνται αποκλειστικά από projects με πολύ μικρό αριθμό παρατηρήσεων.

7.6 Σύγκριση OLS και LMM

Η παράλληλη εφαρμογή των Ordinary Least Squares (OLS) και Linear Mixed Models (LMM) μοντέλων επιτρέπει την αξιολόγηση της σταθερότητας των παρατηρούμενων σχέσεων, αλλά και της επίδρασης που έχει η ιεραρχική δομή των δεδομένων στη στατιστική συμπερασματολογία. Για τον σκοπό αυτό, τα δύο μοντέλα εφαρμόστηκαν στο ίδιο τελικό δείγμα και με κοινή βασική προδιαγραφή, με στόχο οι μεταξύ τους διαφορές να μπορούν να αποδοθούν κυρίως στον διαφορετικό τρόπο με τον οποίο αντιμετωπίζουν την εξάρτηση των παρατηρήσεων.

Στις δύο αυτές περιπτώσεις εξετάστηκαν ως εξαρτημένες μεταβλητές η λογαριθμικά μετασχηματισμένη πυκνότητα των προβλημάτων ποιότητας και η αντίστοιχη πυκνότητα των προβλημάτων ασφάλειας. Ως ερμηνευτικές μεταβλητές χρησιμοποιήθηκαν η κατηγορία στην οποία το Pull Request ανήκει, το λογαριθμικά μετασχηματισμένο μέγεθος του κώδικα (`log_ncloc`) και η λογαριθμικά μετασχηματισμένη πολυπλοκότητα (`log_complexity`). Η ουσιαστική διαφοροποίηση μεταξύ των δύο αυτών προσεγγίσεων αφορά τον τρόπο κατά τον οποίο λαμβάνεται υπόψη η ομαδοποίηση των PRs στα αντίστοιχα projects.

Στο OLS κάθε Pull Request συμμετέχει στην εκτίμηση της ίδιας συνολικής εξίσωσης παλινδρόμησης. Στην παρούσα έρευνα χρησιμοποιήθηκαν cluster-robust standard errors ανά μονάδα project, με στόχο η εκτίμηση της αβεβαιότητας των συντελεστών να είναι ανθεκτικότερη στην ύπαρξη συσχέτισης μεταξύ Pull Requests του ίδιου project. Η συγκεκριμένη προσέγγιση, όμως, δεν εισάγει ρητά στο χρησιμοποιούμενο στατιστικό μοντέλο τις διαφορές μεταξύ των projects. Το LMM, από την άλλη πλευρά περιλαμβάνει τυχαίο intercept ανά project, επιτρέποντας σε κάθε project να αποκλίνει από το συνολικό intercept και διαχωρίζοντας με τον τρόπο αυτό τη διακύμανση που εμφανίζεται μεταξύ διαφορετικών projects από την υπολειμματική διακύμανση μεταξύ των PRs του ίδιου project.

Μεταξύ των δύο προαναφερθέντων στατιστικών προσεγγίσεων (OLS, LMM), η σύγκριση των αποτελεσμάτων για την πυκνότητα των προβλημάτων ποιότητας παρουσιάζει σημαντική συνέπεια. Στο OLS, το `log_ncloc` εμφανίζει αρνητική σχέση με την πυκνότητα των quality issues ($\beta = -0,169$, $p = 0,021$), αντίθετα με τη `log_complexity` η οποία εμφανίζει θετική και συνάμα ισχυρότερη σχέση ($\beta = 0,368$, $p < 0,001$). Μετά την ενσωμάτωση του project-level random intercept στο LMM, οι κατευθύνσεις των αποτελεσμάτων παραμένουν ίδιες, με το `log_ncloc` να παρουσιάζει συντελεστή περίπου $-0,152$ και την `log_complexity` περίπου $0,368$, ενώ και οι δύο σχέσεις είναι στατιστικά σημαντικές σε επίπεδο $p < 0,001$. Παράλληλα οι κατηγορίες security και both δεν εμφανίζουν στατιστικά σημαντική διαφοροποίηση από την κατηγορία αναφοράς quality στο LMM (έχει τεθεί ως κατηγορία σύγκρισης

και αναφοράς η κατηγορία quality με βάση την οποία εξάγονται τα αποτελέσματα της στατιστικής ανάλυσης για τις δύο κατηγορίες security και both).

Η γενική σταθερότητα κατεύθυνσης και μεγέθους των βασικών συντελεστών μεταξύ OLS και LMM αποτελεί ένδειξη -και όχι απόδειξη- ότι οι κυριότερες σχέσεις που παρατηρούνται για τα προβλήματα ποιότητας δεν αποτελούν αποκλειστικά αποτέλεσμα της επιλογής του συγκεκριμένου στατιστικού μοντέλου. Η μικρότερη αβεβαιότητα, παράλληλα, των εκτιμήσεων στο LMM και η ισχυρότερη στατιστική ένδειξη για το μέγεθος του κώδικα προκύπτουν όταν η διαφοροποίηση μεταξύ των projects μοντελοποιείται άμεσα.

Εικόνα διαφορετική των παραπάνω παρουσιάζεται για την πυκνότητα των προβλημάτων ασφαλείας, χωρίς να αποτελεί καθολικό γεγονός. Στο OLS ούτε το `log_ncloc` ούτε η `log_complexity` παρουσιάζουν στατιστικά σημαντική σχέση με την εξαρτημένη μεταβλητή. Στο LMM, το μέγεθος του κώδικα παραμένει μη σημαντικό στατιστικά, όπως η πολυπλοκότητα εμφανίζει μικρή αρνητική σχέση ($\beta \sim -0,011$, $p = 0,017$). Αυτές οι διαφορές καταδεικνύουν ότι η ρητή μοντελοποίηση της ομαδοποίησης ανά project μπορεί να επηρεάσει όχι μόνο την εκτιμώμενη αβεβαιότητα αλλά, σε ορισμένες περιπτώσεις, και τα συμπεράσματα σχετικά με τη στατιστική σημαντικότητα επιμέρους μεταβλητών.

Η καταλληλότητα, που εμφανίζει το LMM μοντέλο για την διεξαγωγή αποτελεσμάτων στην συγκεκριμένη έρευνα με βάση το εξαχθέν dataset, ενισχύεται περαιτέρω από την ύπαρξη σημαντικής διακύμανσης μεταξύ των projects. Στο μοντέλο των quality issues η εκτιμώμενη διακύμανση του random intercept είναι σημαντικά μεγαλύτερη από την αντίστοιχη υπολειμματική διακύμανση, ενώ ανάλογη εικόνα τείνει να παρατηρείται και στο μοντέλο των security issues. Το αποτέλεσμα αυτό υποδεικνύει ότι τα Pull Requests του ίδιου project δεν μπορούν να θεωρηθούν πλήρως ανεξάρτητες παρατηρήσεις και ότι μέρος της συμπεριφοράς των μετρικών συνδέεται με χαρακτηριστικά του project στο οποίο και ανήκουν.

Η σύγκριση μεταξύ των δύο προσεγγίσεων δεν βασίζεται σε άμεση αντιπαραβολή των τιμών AIC, BIC ή log-likelihood των συγκεκριμένων τελικών μοντέλων, καθώς το κύριο LMM έχει εκτιμηθεί κάνοντας χρήση της μεθόδου REML, ενώ το OLS με διαφορετική διαδικασία εκτίμησης. Η αξιολόγηση βασίζεται στη συνέπεια και τη μεταβολή των εκτιμώμενων fixed effects, στη στατιστική συμπερασματολογία, στις variance components και στους ICC, καθώς και στην καταλληλότητα κάθε μοντέλου ως προς τη δομή δεδομένων. Το OLS χρησιμοποιείται ως baseline μοντέλο, ενώ το random-intercept LMM αποτελεί την κύρια inferential προσέγγιση επειδή μοντελοποιεί ρητά την ομαδοποίηση των PRs ανά μονάδα project.

Το OLS, συμπεραίνουμε ότι λειτουργεί ως ένα χρήσιμο baseline μοντέλο. Προσφέρει μία απλούστερη αναφορά για την κατεύθυνση και το μέγεθος των εξεταζόμενων σχέσεων που προκύπτουν. Το LMM αποτελεί την κύρια προσέγγιση της παρούσας έρευνας, όμως. Αυτό συμβαίνει λόγω της ιδιότητάς του να ενσωματώνει άμεσα την ιεραρχική διόρθωση των δεδομένων, επιτρέπει τη μοντελοποίηση της ετερογένειας μεταξύ των projects και παρέχει καταλληλότερο πλαίσιο για την ορθή εξαγωγή συμπερασμάτων σε περιπτώσεις όπου πολλαπλά PRs

προέρχονται από το ίδιο project και κατά συνέπεια έχουν λάβει την ίδια ή παρόμοια παραμετροποίηση.

7.7 Principal Component Analysis

Η PCA μετασχηματίζει συσχετισμένες μεταβλητές σε ασυσχέτιστες γραμμικές συνιστώσες που διατάσσονται κατά τη διακύμανση που εξηγούν. Στόχος είναι η μείωση της διαστατικότητας με όσο το δυνατόν μικρότερη απώλεια πληροφορίας, όχι η ανακάλυψη λανθανόντων αιτιωδών παραγόντων (Jolliffe & Cadima, 2016).

Η κύρια PCA εφαρμόστηκε σε επτά μεταβλητές. Οι μεταβλητές αυτές ήταν reliability issues ανά KLOC, maintainability issues ανά KLOC, security issues ανά KLOC, security hotspots ανά KLOC, complexity ανά KLOC, cognitive complexity ανά KLOC και duplicated lines density (όπου KLOC είναι η κανονικοποιημένη τιμή της εκάστοτε NCLOC). Με τον τρόπο αυτό συνδυάζονται στο ίδιο πολυμεταβλητό πλαίσιο δείκτες οι οποίοι αφορούν την αξιοπιστία, τη συντηρησιμότητα, την ασφάλεια, την πολυπλοκότητα και την επανάληψη κώδικα.

Ο ρόλος της PCA καθίσταται και περιορίζεται στην εξέταση της πιθανότητας οι παραπάνω μεταβλητές να παρουσιάζουν κοινά πρότυπα μεταβολής και αν μπορούν να συνοψιστούν σε ευρύτερες συνθετικές διαστάσεις. Σε αντίθεση με τα OLS και LMM, τα οποία και χρησιμοποιούνται για την εξέταση συγκεκριμένων σχέσεων μεταξύ προκαθορισμένων μεταβλητών, η PCA λειτουργεί συμπληρωματικά, αποκαλύπτοντας τη δομή των συσχετίσεων μεταξύ των μετρικών αυτών. Οι κύριες συνιστώσες που προκύπτουν αποτελούν γραμμικούς συνδυασμούς των αρχικών μεταβλητών, ενώ για κάθε Pull Request υπολογίζονται αντίστοιχα component scores.

7.7.1 Στόχος και standardization

Βασικός στόχος της Principal Component Analysis (PCA) στην παρούσα έρευνα είναι η συμπύκνωση της πληροφορίας που περιέχεται σε ένα σύνολο συσχετισμένων μετρικών σε μικρότερο αριθμό κύριων συνιστωσών. Εξετάζεται, έτσι, εάν διαφορετικές μετρικές ποιότητας, ασφάλειας και πολυπλοκότητας παρουσιάζουν κοινά πρότυπα μεταβολής και μπορούν να περιγραφούν μέσω ευρύτερων πολυμεταβλητών διαστάσεων, χωρίς να απαιτείται η ανεξάρτητη εξέταση κάθε δείκτη.

Η κύρια PCA εφαρμόστηκε σε επτά μεταβλητές, όπως αναφέρθηκε και παραπάνω. Ανακεφαλαιώνοντας, αναφέρονται ξανά: reliability issues ανά KLOC, maintainability issues ανά KLOC, security issues ανά KLOC, security hotspots ανά KLOC, complexity ανά KLOC, cognitive complexity ανά KLOC και duplicated lines density. Επιλέχθηκε το ίδιο σύνολο μεταβλητών με εκείνο της κύριας Factor Analysis,

με στόχο τη συνέχεια ουσιαστικής σύγκρισης των δομών που προκύπτουν από τις δύο αυτές πολυμεταβλητές προσεγγίσεις. Η PCA, σημειώνεται, δεν επιχειρεί να προσδιορίσει εκ των προτέρων υποκείμενες λανθάνουσες έννοιες, αλλά αναζητάει τις γραμμικές κατευθύνσεις κατά μήκος των οποίων συγκεντρώνεται το μεγαλύτερο μέρος της διακύμανσης των παρατηρούμενων δεδομένων.

Πριν την εφαρμογή της PCA επί των δεδομένων, πραγματοποιήθηκε τυποποίηση (standardization) των μεταβλητών. Το στάδιο αυτό είναι απαραίτητο, καθώς ακόμη και όταν αρκετές μετρικές εκφράζονται ως πυκνότητες ανά KLOC, εξακολουθούν να διαφέρουν ως προς το εύρος τιμών και τη διακύμανσή τους. Χωρίς τυποποίηση, μία μεταβλητή με μεγαλύτερη αριθμητική διακύμανση θα μπορούσε να ασκήσει δυσανάλογη επίδραση στον σχηματισμό των κύριων συνιστωσών.

Για κάθε μεταβλητή X_j , η τυποποιημένη τιμή μιας παρατήρησης υπολογίζεται ως:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

όπου

- x_{ij} , είναι η αρχική τιμή της παρατήρησης (i) για τη μεταβλητή (j)
- $\bar{x}_j$, ορίζεται ως ο μέσος όρος της μεταβλητής
- s_j , είναι η τυπική απόκλιση

Μετά την τυποποίηση, οι μεταβλητές εκφράζονται σε κοινή κλίμακα, με μέσο όρο κοντά στο μηδέν και τυπική απόκλιση κοντά στη μονάδα. Η συνεισφορά τους, άρα, στην PCA καθορίζεται από τη δομή των μεταξύ τους σχέσεων και όχι από τις διαφορετικές μονάδες ή κλίμακες μέτρησης.

Επειδή αρκετές από τις βασικές μεταβλητές της κύριας ανάλυσης χρησιμοποιούν ως κοινό παρονομαστή το KLOC, πραγματοποιήθηκε επιπλέον size-adjusted PCA ως έλεγχος ευαισθησίας. Η εναλλακτική αυτή ανάλυση χρησιμοποιείται για να εξεταστεί κατά πόσο η πολυμεταβλητή δομή παραμένει σταθερή όταν η επίδραση του μεγέθους του κώδικα ελέγχεται με διαφορετικό τρόπο. Η κύρια ερμηνεία, επιμένει να βασίζεται στην PCA των αρχικών επτά τυποποιημένων δεικτών.

7.7.2 Eigenvalues και explained variance

Συνακόλουθα της τυποποίησης των μεταβλητών, η PCA προσδιορίζει διαδοχικά τις κύριες συνιστώσες με βάση το μέρος της συνολικής διακύμανσης των δεδομένων που μπορεί η καθεμία να αποτυπώσει. Η ποσότητα αυτή εκφράζεται μέσω της ιδιοτιμής (eigenvalue) κάθε συνιστώσας. Όσο μεγαλύτερη είναι η ιδιοτιμή, τόσο μεγαλύτερο μέρος της πληροφορίας των αρχικών μεταβλητών συγκεντρώνεται στη συγκεκριμένη συνιστώσα.

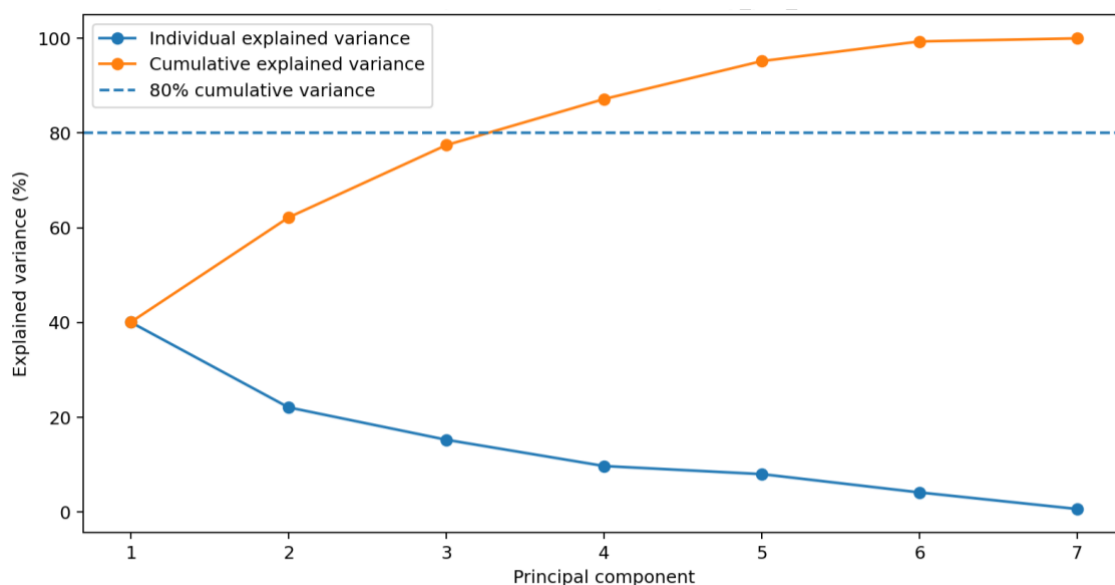
Οι eigenvalues εκφράζουν τη διακύμανση που αντιστοιχεί σε κάθε principal component. Η explained variance ratio προκύπτει από τη διαίρεση της eigenvalue

της συνιστώσας με το άθροισμα όλων των eigenvalues και χρησιμοποιείται για την αξιολόγηση της πληροφορίας που διατηρείται (Jolliffe & Cadima, 2016).

$$EVR_k = \frac{\lambda_k}{\sum_{j=1}^p \lambda_j}$$

όπου λ_k είναι η ιδιοτιμή της συνιστώσας k και p ο συνολικός αριθμός των αρχικών μεταβλητών. Η συγκεκριμένη ποσότητα αναφέρεται ως explained variance ratio (EVR) και εκφράζει το ποσοστό της συνολικής διακύμανσης που κάθε component διατηρεί. Η cumulative explained variance (CEV) αντίστοιχα, προκύπτει από το άθροισμα των ποσοστών των διαδοχικών συνιστωσών και δείχνει πόση από την αρχική πληροφορία διατηρείται συνολικά όταν χρησιμοποιούνται περισσότερα από ένα components.

Στην κύρια PCA της παρουσιαζόμενης έρευνας, η πρώτη συνιστώσα παρουσίασε ιδιοτιμή περίπου 2,806 και εξήγησε το 40,1% της συνολικής διακύμανσης. Η δεύτερη συνιστώσα είχε ιδιοτιμή περίπου 1,547 και πρόσθεσε 22,1%, με αποτέλεσμα το ανέβασμα της σωρευτικής explained variance στο 62,2%. Η τρίτη συνιστώσα, με ιδιοτιμή περίπου 1,069, εξήγησε επιπλέον 15,3%. Οι τρεις πρώτες κύριες συνιστώσες, επομένως, διατήρησαν συνολικά περίπου το 77,5% της διακύμανσης που περιεχόταν στις επτά αρχικές μεταβλητές.



Εικόνα 7.7.1 – Ποσοστό και σωρευτικό ποσοστό της συνολικής διακύμανσης που εξηγείται μέσω των κύριων συνιστωσών της PCA

Έπειτα από την Τρίτη συνιστώσα, παρατηρείται αισθητή μείωση της πρόσθετης πληροφορίας που αποτυπώνεται από κάθε νέο component. Ενδεικτικά λαμβανόμενα, για την υπέρβαση του 80% της συνολικής explained variance θα απαιτούνταν τέσσερις συνιστώσες, ενώ για επίπεδα 90% και 95% θα απαιτούνταν πέντε. Η επιλογή του αριθμού των components εδώ, δεν βασίστηκε αποκλειστικά σε

ένα προκαθορισμένο ποσοστό explained variance, αλλά εξετάστηκε σε συνδυασμό με τις ιδιοτιμές, το Kaiser criterion και την Parallel Analysis.

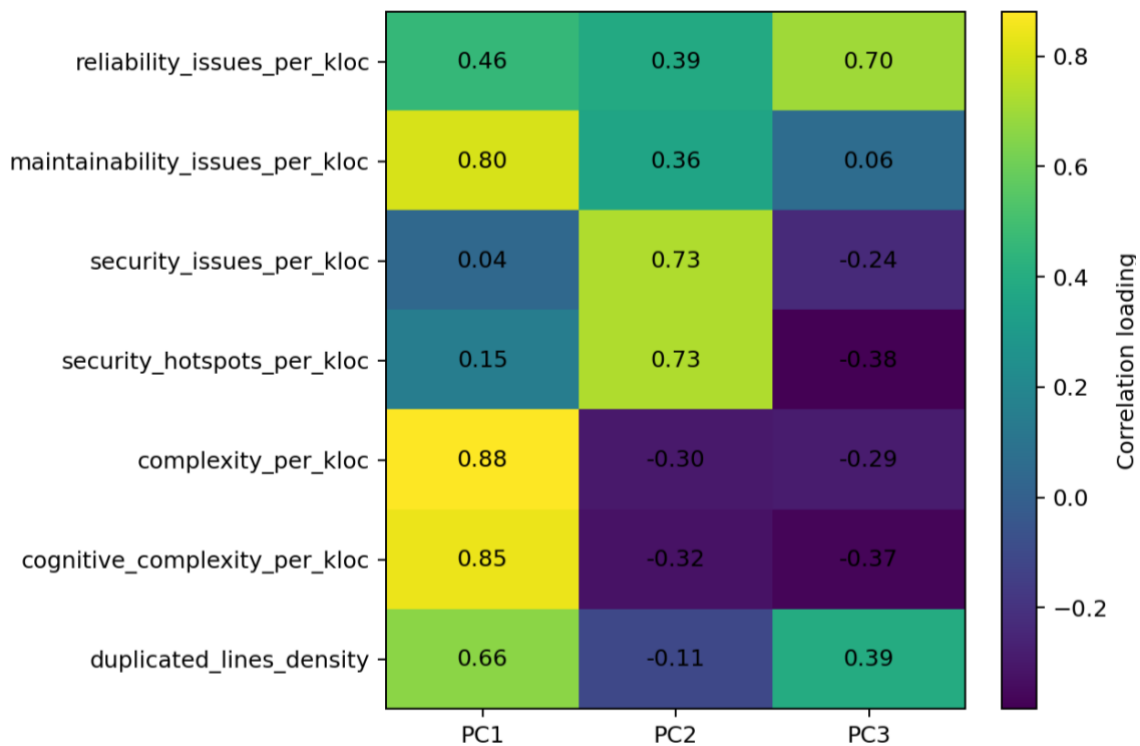
Συνοψίζοντας, η explained variance παρέχει μία πρώτη ένδειξη του βαθμού στον οποίο η αρχική πληροφορία δύναται να συμπυκνωθεί χωρίς σημαντική απώλειά της. Στην περίπτωση της παρούσας έρευνας, το γεγονός ότι οι τρεις πρώτες συνιστώσες συγκεντρώνουν περισσότερο από τα τρία τέταρτα (3/4) της συνολικής διακύμανσης υποδεικνύει ότι οι επτά αρχικές μετρικές παρουσιάζουν σημαντική κοινή πολυμεταβλητή δομή. Η σημασία και το περιεχόμενο αυτής της δομής διερευνώνται στη συνέχεια με χρήση loadings και component scores.

7.7.3 Loadings και component scores

Με την ολοκλήρωση των κύριων συνιστωσών, η κατανόηση της δομής τους βασίζεται κυρίως στα loadings και στα component scores. Οι δύο έννοιες περιγράφουν διαφορετικές πλευρές της τεχνικής PCA. Τα loadings από τη μία, δείχνουν τη σχέση των αρχικών μεταβλητών με κάθε κύρια συνιστώσα, ενώ τα component scores από την άλλη, προσδιορίζουν τη θέση κάθε παρατήρησης στον νέο πολυδιάστατο χώρο που δημιουργείται από τις συνιστώσες.

Στην παρούσα ανάλυση χρησιμοποιούνται correlation loadings, τα οποία εκφράζουν τη συσχέτιση κάθε αρχικής μεταβλητής με μία κύρια συνιστώσα. Όσο μεγαλύτερη είναι η απόλυτη τιμή ενός loading, τόσο ισχυρότερη σύνδεση δημιουργείται μεταξύ της μεταβλητής αυτής με το αντίστοιχο component. Το πρόσημο υποδεικνύει την κατεύθυνση της σχέσης, ενώ μεταβλητές με υψηλά loadings στην ίδια συνιστώσα τείνουν να συμμετέχουν σε ένα κοινό πρότυπο μεταβολής.

Στην κύρια PCA, το PC1 παρουσιάζει υψηλότερες θετικές συσχετίσεις με την complexity_per_kloc (0,880), την cognitive_complexity_per_kloc (0,845) και την maintainability_issues_per_kloc (0,803), ενώ σημαντική συμμετοχή εμφανίζει και η duplicated_lines_density (0,661). Το PC2 συνδέεται περισσότερο ισχυρά με την security_issues_per_kloc (0,732) και την security_hotspots_per_kloc (0,730). Στο PC3, αντίστοιχα, η υψηλότερη θετική συσχέτιση εμφανίζεται για την reliability_issues_per_kloc (0,700), ενώ οι υπόλοιπες μεταβλητές παρουσιάζουν μικρότερες θετικές ή αρνητικές συνεισφορές.



Εικόνα 7.7.2 – Correlation loadings των επτά (7) αρχικών μεταβλητών στις τρεις κύριες συνιστώσες της PCA

Τα component scores αφορούν τις ίδιες τις παρατηρήσεις, αντίθετα με τα loadings, όπως ήδη αναφέρθηκε. Για κάθε μονάδα Pull Request υπολογίζεται μία τιμή για κάθε διατηρούμενη κύρια συνιστώσα, μέσω της προβολής των τυποποιημένων τιμών των αρχικών μεταβλητών πάνω στους αντίστοιχους άξονες της PCA. Κάθε Pull Request, έτσι, δεν περιγράφεται πλέον μόνο από τις επτά αρχικές μετρικές, αλλά και από ένα μικρότερο σύνολο τιμών, όπως τα PC1, PC2 και PC3 scores.

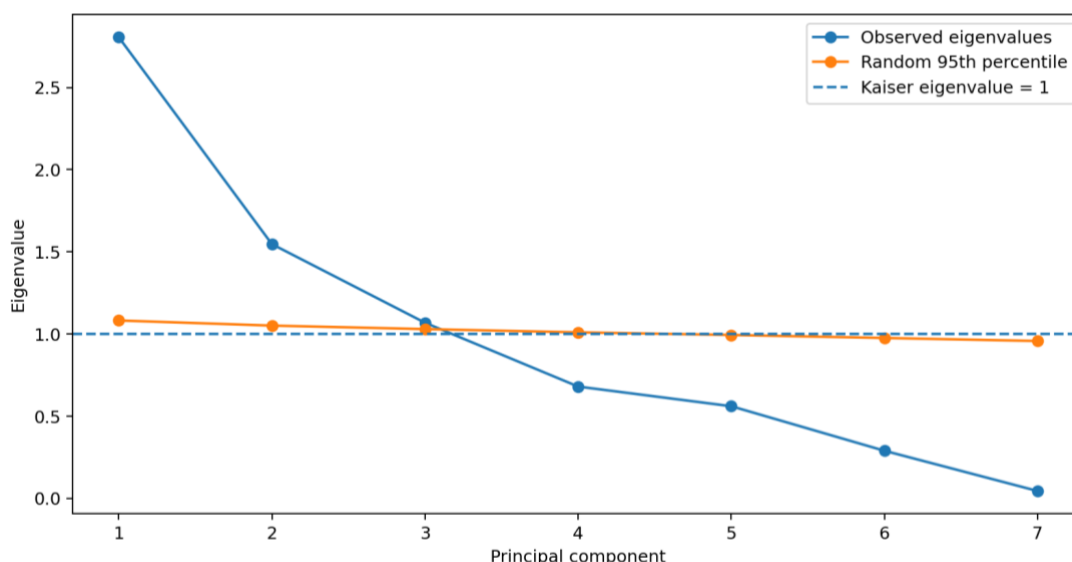
Ένα υψηλό ή χαμηλό component score δεν έχει από μόνο του θετική ή αρνητική έννοια ως προς την ποιότητα ή την ασφάλεια. Η σημασία του εξαρτάται από τα loadings της συγκεκριμένης συνιστώσας. Για παράδειγμα, εάν ένα Pull Request παρουσιάζει υψηλό score σε ένα component στο οποίο φορτίζουν έντονα οι μετρικές πολυπλοκότητας και συντηρησιμότητας, αυτό υποδηλώνει ότι εμφανίζει εντονότερα το συνδυασμένο πρότυπο που περιγράφεται από τη συγκεκριμένη συνιστώσα.

Στη συνέχεια, τα component scores αξιοποιούνται και ως συνθετικές μεταβλητές σε πρόσθετες αναλύσεις OLS και LMM. Με αυτόν τον τρόπο είναι δυνατό να εξεταστεί εάν οι ευρύτερες πολυμεταβλητές διαστάσεις που προκύπτουν από την PCA παρουσιάζουν συστηματικές σχέσεις με την κατηγορία του Pull Request και το μέγεθος του κώδικα, καθώς και κατά πόσο διαφοροποιούνται μεταξύ διαφορετικών projects. Τελικά, η χρήση τους επιτρέπει τη σύνδεση της μείωσης της διαστατικότητας με την ιεραρχική στατιστική ανάλυση που εφαρμόζεται στο παρόν ερευνητικό κείμενο.

7.7.4 Επιλογή και ερμηνεία components

Καθοριστικό στάδιο στην εφαρμογή της PCA τεχνικής αποτελεί η επιλογή του αριθμού των κύριων συνιστωσών, καθώς επιδιώκεται η διατήρηση επαρκούς πληροφορίας με ταυτόχρονο περιορισμό της αρχικής διαστατικότητας. Στην παρούσα έρευνα, η κύρια απόφαση βασίστηκε στην Parallel Analysis, ενώ συμπληρωματικά εξετάστηκε και το Kaiser criterion, σύμφωνα με το οποίο διατηρούνται οι συνιστώσες με eigenvalue μεγαλύτερο της μονάδας.

Η Parallel Analysis συνέκρινε τις παρατηρούμενες eigenvalues με τις αντίστοιχες eigenvalues που προκύπτουν με τη χρήση τυχαίων δεδομένων της ίδιας κατάστασης. Με βάση το συγκεκριμένο αυτό κριτήριο διατηρήθηκαν οι πρώτες τρεις κύριες συνιστώσες, αποτέλεσμα που επιβεβαιώθηκε και από το Kaiser criterion. Οι τρεις αυτές συνιστώσες εξηγούν συνολικά το ~77,5% της συνολικής διακύμανσης που παρατηρείται, παρέχοντας σημαντική συμπύκνωση της πληροφορίας των επτά αρχικών μεταβλητών σε τρεις νέες διαστάσεις.



Εικόνα 7.7.3 – Parallel Analysis για την επιλογή του αριθμού των κύριων συνιστωσών

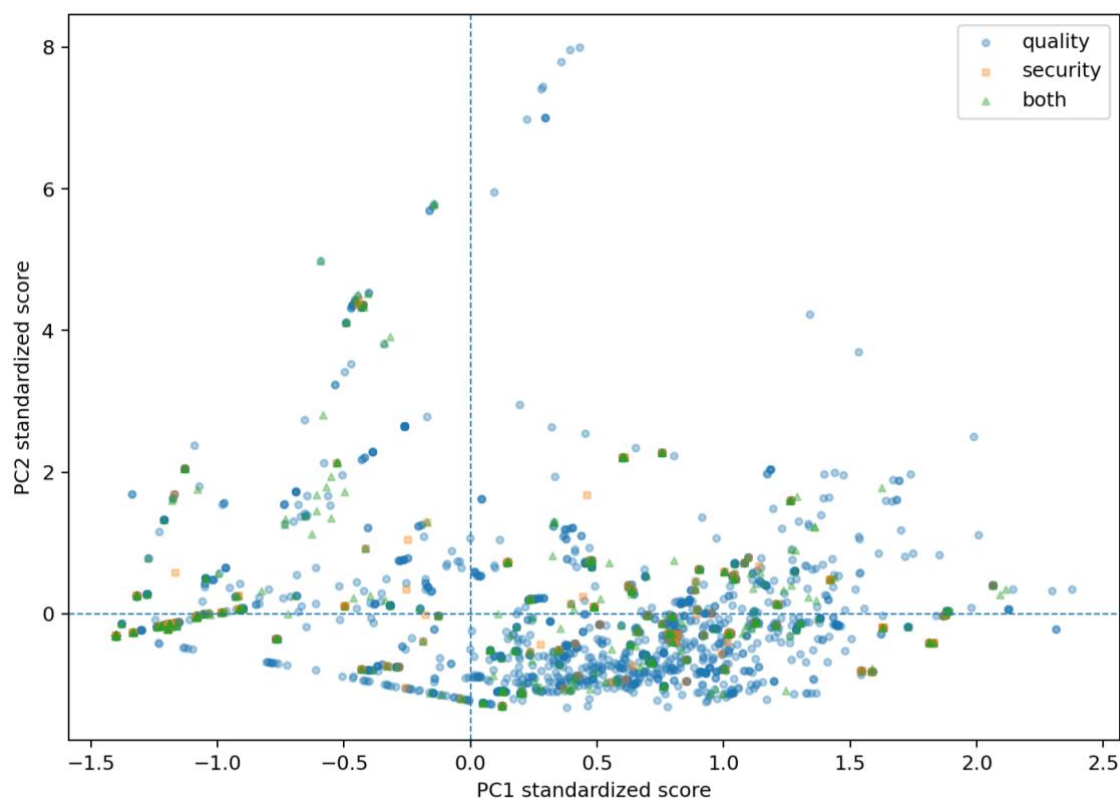
Η ερμηνεία των συνιστωσών πραγματοποιήθηκε με βάση κυρίως το μέγεθος και την κατεύθυνση των correlation loadings των αρχικών μεταβλητών. Η ονομασία τους δεν αποτελεί άμεσο αποτέλεσμα του αλγορίθμου που δημιουργήθηκε για την τεχνική PCA, αλλά ερμηνευτική περιγραφή του συνδυασμού των μεταβλητών που εμφανίζουν την ισχυρότερη σχέση με κάθε component. Παρακάτω αναλύονται οι τρεις (3) συνιστώσες, PC1, PC2 και PC3:

- Η πρώτη κύρια συνιστώσα (PC1) χαρακτηρίζεται κυρίως από υψηλά θετικά loadings στις μετρικές complexity, cognitive complexity και maintainability issues, ενώ σημαντική θετική συνεισφορά παρουσιάζει και η duplicated lines density. Η συγκεκριμένη συνιστώσα μπορεί επομένως να ερμηνευθεί ως μία ευρύτερη διάσταση η οποία συνδέεται με τη δομική πολυπλοκότητα και τη

συντηρησιμότητα του κώδικα. Υψηλότερα PC1 scores αντιστοιχούν σε PRs που εμφανίζουν εντονότερα το συγκεκριμένο κοινό πρότυπο.

- Η δεύτερη κύρια συνιστώσα (PC2) παρουσιάζει τις ισχυρότερες θετικές συσχετίσεις με τα security issues και τα security hotspots, ενώ οι μεταβλητές πολυπλοκότητας εμφανίζουν ασθενέστερες αρνητικές σχέσεις με αυτή. Το PC2, μπορεί επομένως, να ερμηνευθεί ως μία διάσταση ασφάλειας η οποία κατά συνέπεια αποτυπώνει ένα μοτίβο των δεδομένων διαφορετικό από εκείνο στο οποίο κυριαρχεί στη PC1. Η συγκεκριμένη αυτή διάκριση είναι ιδιαίτερα σημαντική για την παρούσα έρευνα, καθώς υποδεικνύει ότι οι μετρικές ασφάλειας δεν αποτελούν απλώς προέκταση της διάστασης της πολυπλοκότητας και της συντηρησιμότητας.
- Η τρίτη κύρια συνιστώσα (PC3) χαρακτηρίζεται με τη σειρά της κυρίως από υψηλή θετική συσχέτιση με τα reliability issues, καθώς και από μικρότερη θετική συμμετοχή της duplicated lines density, ενώ security hotspots και μεταβλητές πολυπλοκότητας παρουσιάζουν αρνητικά loadings. Η συνιστώσα αυτή μπορεί να ερμηνευθεί πιο επιφυλακτικά ως μία διάσταση η οποία διαφοροποιεί κυρίως την αξιοπιστία από ορισμένα χαρακτηριστικά ασφάλειας και πολυπλοκότητας. Η PC3 παρουσιάζει περισσότερο το μικτό πρότυπο loadings, σε αντίθεση με τις PC1 και PC2, με αποτέλεσμα η εννοιολογική της σημασία να απαιτεί μεγαλύτερη προσοχή και περισσότερο ορθό χειρισμό.

Η παραπάνω δομή δείχνει ότι οι αρχικές μετρικές δεν συγκεντρώνονται σε μία μοναδική γενική διάσταση ποιότητας ή ασφάλειας. Αναδεικνύονται διακριτά αλλά αλληλοσυμπληρούμενα πρότυπα, με το πρώτο component να σχετίζεται κυρίως με την πολυπλοκότητα και τη συντηρησιμότητα, το δεύτερο με την ασφάλεια και το τρίτο κυρίως με την αξιοπιστία. Η PCA προσφέρει μία περισσότερο συνοπτική εικόνα της αλληλεπίδρασης των μετρικών, χωρίς όμως να εξαλείφει τη διακριτή πληροφορία την οποία περιέχουν οι διαφορετικές διαστάσεις του λογισμικού.



Εικόνα 7.7.4 – Κατανομή των Pull Requests στον χώρο των δύο πρώτων κύριων συνιστωσών ανά κατηγορία

Επιπλέον πραγματοποιήθηκε size-adjusted PCA ως έλεγχος ευαισθησίας, στην οποία ο έλεγχος του μεγέθους του κώδικα έγινε με διαφορετικό τρόπο. Στην περίπτωση αυτή, τόσο η Parallel Analysis όσο και το Kaiser criterion οδήγησαν στη διατήρηση δύο (2) αντί τριών (3) components, τα οποία είναι σε θέση να εξηγήσουν στατιστικά το περίπου 65,7% της διακύμανσης. Το αποτέλεσμα αυτό δείχνει ότι η γενικότερη ύπαρξη μιας συμπυκνωμένης πολυμεταβλητής δομής παραμένει. Ο ακριβής αριθμός των διαστάσεων παρουσιάζει κάποια ευαισθησία στον τρόπο με τον οποίο λαμβάνεται υπόψη το συνολικό μέγεθος (ή μέρος του) κώδικα. Η κύρια ερμηνεία της έρευνας βασίζεται επομένως στις τρεις συνιστώσες της primary PCA, ενώ η size-adjusted ανάλυση χρησιμοποιείται ως συμπληρωματικός έλεγχος robustness.

7.8 Factor Analysis

Η Exploratory Factor Analysis μοντελοποιεί τη συνδιακύμανση των παρατηρούμενων μεταβλητών μέσω μικρότερου αριθμού κοινών λανθανόντων παραγόντων και ειδικής διακύμανσης. Σε αντίθεση με την PCA, δεν αντιμετωπίζει όλη τη διακύμανση ως κοινή πληροφορία προς αναπαραγωγή (Fabrigar et al., 1999).

Η ανάλυση εφαρμόστηκε στις ίδιες επτά μεταβλητές με την PCA. Οι μεταβλητές είναι reliability issues ανά KLOC, maintainability issues ανά KLOC, security issues ανά KLOC, cognitive complexity ανά KLOC και duplicated lines density. Εξετάζεται, έτσι, εάν οι δείκτες ποιότητας, ασφάλειας, πολυπλοκότητας και επανάληψης κώδικα μπορούν να οργανωθούν σε μικρότερο αριθμό κοινών παραγόντων.

Η διαδικασία περιλαμβάνει αρχικά την αξιολόγηση της καταλληλότητας των δεδομένων μέσω των Kaiser-Meyer-Olkin (KMO) και Bartlett's test of sphericity, την επιλογή του αριθμού των παραγόντων με χρήση της Parallel Analysis, καθώς και τα στάδια extraction και rotation. Η τελική ερμηνεία βασίζεται στα factor loadings, στις communalities, στις uniqueness values και στα factor scores, τα οποία αποτυπώνουν τη σχέση των αρχικών μεταβλητών με τους παράγοντες καθώς και τη θέση κάθε μονάδας Pull Request στις λανθάνουσες καταστάσεις που προκύπτουν.

7.8.1 Latent factors

Βασική έννοια της Factor Analysis είναι οι λανθάνοντες παράγοντες (latent factors), δηλαδή μη άμεσα παρατηρήσιμες διαστάσεις οι οποίες θεωρείται ότι βρίσκονται πίσω από τις συσχετίσεις μεταξύ των μετρούμενων μεταβλητών. Σε αντίθεση με τις αρχικές μετρικές, οι οποίες παρατηρούνται και καταγράφονται άμεσα, οι latent factors προκύπτουν στατιστικά από την κοινή διακύμανση των μεταβλητών.

Η βασική λογική του παραγοντικού μοντέλου μπορεί να αποδοθεί ως:

$$X_j = \lambda_{j1}F_1 + \lambda_{j2}F_2 + \dots + \lambda_{jm}F_m + \varepsilon_j$$

όπου:

- X_j είναι μία παρατηρούμενη μεταβλητή
- $F_1, \dots, F_m$ είναι οι λανθάνοντες παράγοντες
- λ_{jm} τα Factor Loadings που εκφράζουν τη σχέση της μεταβλητής με κάθε παράγοντα και
- ε_j το μέρος της μεταβλητής το οποίο δεν εξηγείται από την κοινή παραγοντική δομή.

Στην παρούσα έρευνα, οι παρατηρούμενες μεταβλητές είναι οι reliability issues ανά KLOC, maintainability issues ανά KLOC, security issues ανά KLOC, security hotspots ανά KLOC, complexity ανά KLOC, cognitive complexity ανά KLOC και duplicated lines density. Η Factor Analysis εξετάζει κατά πόσο οι συσχετίσεις μεταξύ αυτών των μετρικών μπορούν να αποδοθούν σε μικρότερο αριθμό κοινών λανθανουσών διαστάσεων, οι οποίες ενδέχεται να εκφράζουν ευρύτερα χαρακτηριστικά του πηγαίου κώδικα.

Οι παράγοντες αυτοί δεν αντιστοιχούν από την αρχή σε προκαθορισμένες έννοιες που θέτει η έρευνα, όπως «ποιότητα» ή «ασφάλεια». Η ερμηνεία και η ονομασία τους πραγματοποιούνται στη συνέχεια, με βάση κυρίως το πρότυπο των

factor loadings και το σύνολο των μεταβλητών που συνδέονται εντονότερα με κάθε παράγοντα. Με αυτόν τον τρόπο περιορίζεται ο κίνδυνος να επιβληθεί εκ των προτέρων μία θεωρητική δομή στα δεδομένα και επιτρέπεται η παραγοντική οργάνωση να προκύψει από τις ίδιες τις παρατηρούμενες συσχετίσεις.

7.8.2 KMO και Bartlett

Ο KMO αξιολογεί αν οι συσχετίσεις είναι αρκετά συμπαγείς για παραγοντική μοντελοποίηση, ενώ το Bartlett's test εξετάζει αν ο correlation matrix διαφέρει από identity matrix. Τα κριτήρια λειτουργούν ως διαγνωστικές ενδείξεις και δεν εγγυώνται από μόνα τους ερμηνεύσιμη λύση (Field, 2018; Fabrigar et al., 1999).

Ο KMO δείκτης εξετάζει αν οι προηγούμενες συσχετίσεις μεταξύ των μεταβλητών μπορούν να αποδοθούν σε κοινή υποκείμενη δομή, συγκρίνοντάς τις με τις αντίστοιχες μερικές συσχετίσεις. Τιμές KMO πλησιέστερες στη μονάδα (1) υποδεικνύουν μεγαλύτερη κοινή διακύμανση και καταλληλότερη συσχετιστική δομή για παραγοντική ανάλυση, ενώ χαμηλές τιμές υποδεικνύουν ανεπαρκή κοινή πληροφορία.

Στο πλαίσιο που ορίζεται από την παρούσα έρευνα, στην κύρια Factor Analysis, προέκυψε συνολικός δείκτης KMO = 0,631. Αυτή η τιμή υποδεικνύει μέτρια αλλά επαρκή συνάμα καταλληλότητα του συνόλου των επτά μεταβλητών, όπως ήδη έχουν αναφερθεί, για διερευνητική παραγοντική ανάλυση. Οι επιμέρους τιμές KMO δεν ήταν ομοιόμορφες μεταξύ όλων των μεταβλητών. Ιδιαίτερα υψηλή τιμή παρουσίασε η duplicated_lines_density με τιμή 0,877, ενώ ικανοποιητικά επίπεδα τιμών παρατηρήθηκαν για τα maintainability_issues_per_kloc (0,730) και complexity_per_kloc (0,806). Η security_hotspots_per_kloc, αντίθετα, παρουσίασε οριακά χαμηλή τιμή, 0,495, γεγονός το οποίο υποδεικνύει ότι η συγκεκριμένη μεταβλητή ενσωματώνεται λιγότερο καθαρά στην κοινή συσχετιστική δομή. Η παραγοντική λύση, επομένως, θεωρείται στατιστικά εφαρμόσιμη, αλλά τα αποτελέσματά της απαιτούν προσεκτική ερμηνεία.

Το Bartlett's test of sphericity εξετάζει τη μηδενική υπόθεση ότι ο πίνακας συσχετίσεων των μεταβλητών είναι πίνακας ταυτότητας. Αυτό σημαίνει ότι οι μεταξύ τους συσχετίσεις είναι συνολικά μηδενικές. Σε μία τέτοια περίπτωση η εφαρμογή Factor Analysis δεν θα είχε ουσιαστική βάση, καθώς δεν θα υπήρχε κοινή συσχετιστική δομή προς εξήγηση.

Στην κύρια ανάλυση το Bartlett's test ήταν στατιστικά σημαντικό, με $\chi^2 = 13.877,232$, $df = 21$ και $p < 0,001$. Συμπεραίνουμε ότι η μηδενική υπόθεση απορρίπτεται και προκύπτει ότι ο πίνακας συσχετίσεων διαφέρει σημαντικά από έναν πίνακα ταυτότητας. Το αποτέλεσμα αυτό επιβεβαιώνει ότι μεταξύ των επτά μεταβλητών υπάρχουν επαρκείς συσχετίσεις ώστε να δικαιολογείται η διερεύνηση κοινών λανθανόντων παραγόντων.

Τα συμπεράσματα υποστηρίζονται και από τη size-adjusted sensitivity analysis, στην οποία ο συνολικός KMO αυξήθηκε σε 0,684, ενώ το Bartlett's test παρέμεινε έντονα στατιστικά σημαντικό με $\chi^2 = 15.242,444$, $df = 21$ και $p < 0,001$. Συνολικά, οι δύο έλεγχοι υποστηρίζουν την εφαρμογή Factor Analysis στα δεδομένα,

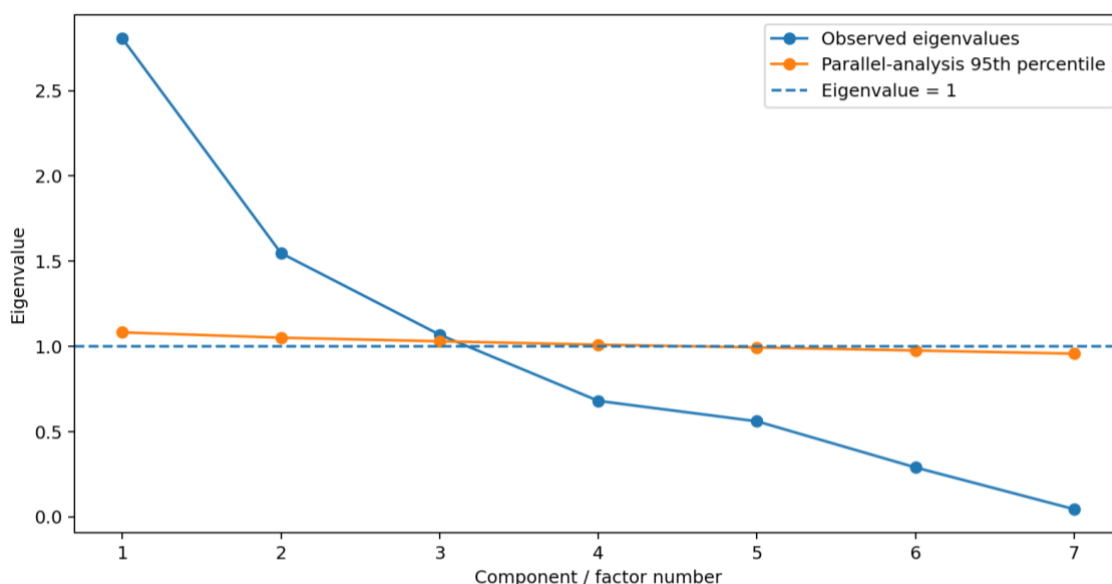
χωρίς ωστόσο να υποδεικνύουν τόσο ισχυρή παραγοντική δομή ώστε τα αποτελέσματα να θεωρηθούν χωρίς επιφυλάξεις. Για τον λόγο αυτό η παραγοντική λύση αντιμετωπίζεται ως διερευνητική και αξιολογείται περαιτέρω μέσω της Parallel Analysis, των factor loadings, των communalities και της σταθερότητας των αποτελεσμάτων.

7.8.3 Parallel analysis

Μετά την επιβεβαίωση της καταλληλότητας των δεδομένων, το επόμενο στάδιο αφορά τον προσδιορισμό του αριθμού των παραγόντων που πρέπει να διατηρηθούν. Η επιλογή αυτή είναι ιδιαιτέρως σημαντική, καθώς η διατήρηση υπερβολικά πολλών παραγόντων μπορεί να οδηγήσει σε τεχνητά πολύπλοκη δομή, ενώ η διατήρηση λιγότερων μπορεί να αποκρύψει ουσιαστικές διαστάσεις των δεδομένων. Για τον σκοπό αυτό έγινε χρήση της Parallel Analysis.

Η Parallel Analysis συγκρίνει τις empirical eigenvalues με eigenvalues που παράγονται από τυχαία δεδομένα αντίστοιχων διαστάσεων. Διατηρούνται οι διαστάσεις των οποίων οι eigenvalues υπερβαίνουν το επιλεγμένο τυχαίο σημείο αναφοράς, περιορίζοντας έτσι την πιθανότητα διατήρησης παραγόντων που αντανakλούν κυρίως τυχαία διακύμανση (Horn, 1965).

Στην κύρια Factor Analysis των επτά per-KLOC και density μεταβλητών, η διαδικασία οδήγησε στη διατήρηση τριών παραγόντων. Οι τρεις πρώτες παρατηρούμενες eigenvalues υπερέβησαν με την σειρά τους τα όρια που έχουν τεθεί από την Parallel Analysis, ενώ έπειτα από τον τρίτο παράγοντα (>3) η συνθήκη αυτή φαίνεται να μην ικανοποιείται. Συνεπώς, ο αριθμός των factors που χρησιμοποιήθηκε στο επόμενο στάδιο extraction ορίστηκε σε τρεις.



Εικόνα 7.8.1 – Parallel Analysis για τον προσδιορισμό του αριθμού των παραγόντων της κύριας Factor Analysis

Η ίδια διαδικασία εφαρμόστηκε και στη size-adjusted sensitivity analysis, όπου διατηρήθηκαν δύο παράγοντες. Η διαφοροποίηση αυτή δείχνει ότι η ακριβής διαστατικότητα της παραγοντικής δομής παρουσιάζει κάποια ευαισθησία στον τρόπο με τον οποίο ελέγχεται το μέγεθος του κώδικα. Για τον λόγο αυτό, η λύση των τριών παραγόντων χρησιμοποιείται ως κύρια παραγοντική αναπαράσταση, αλλά ερμηνεύεται ως διερευνητική και αξιολογείται παράλληλα με τα αποτελέσματα της ανάλυσης ευαισθησίας.

Εν κατακλείδι, η Parallel Analysis καθορίζει το πλήθος των παραγόντων που θα εξαχθούν, χωρίς να έχει τη δυνατότητα να διαχειρίζεται και να θέτει το εννοιολογικό περιεχόμενό τους. Η ερμηνεία των τριών παραγόντων προκύπτει στα επόμενα στάδια, μέσα από τη διαδικασία extraction και rotation και, κυρίως, από την εξέταση των factor loadings, των communalities και της uniqueness κάθε μεταβλητής.

7.8.4 Extraction

Εφόσον ο προσδιορισμός των παραγόντων που επρόκειτο να διατηρηθούν έχει ολοκληρωθεί ακολουθεί το στάδιο της extraction, δηλαδή της εκτίμησης της παραγοντικής δομής που μπορεί να εξηγήσει τις συσχετίσεις μεταξύ των παρατηρούμενων μεταβλητών. Σε αντίθεση με την PCA, όπου οι συνιστώσες επιδιώκουν να αποτυπώσουν τη συνολική διακύμανση των δεδομένων, η Factor Analysis επικεντρώνεται στην κοινή διακύμανση που μπορεί να αποδοθεί σε ένα μικρότερο αριθμό λανθανόντων παραγόντων.

Στην παρούσα έρευνα χρησιμοποιήθηκε Maximum Likelihood Factor Analysis. Η μέθοδος εκτιμά τα factor loadings και την ειδική διακύμανση κάθε μεταβλητής, αναζητώντας εκείνη την παραγοντική δομή που αναπαριστά όσο το δυνατόν καλύτερα την παρατηρούμενη συνδιακύμανση των δεδομένων. Το μοντέλο θεωρεί ότι οι παρατηρούμενες μεταβλητές προκύπτουν μέσω του γραμμικού συνδυασμού ενός μικρότερου αριθμού λανθανόντων παραγόντων και ενός ειδικού σφάλματος ή μέρους διακύμανσης που δεν εξηγείται από την κοινή παραγοντική δομή.

Η Parallel Analysis είχε υποδείξει τη διατήρηση τριών παραγόντων, για την κύρια ανάλυση. Το extraction πραγματοποιήθηκε με προκαθορισμένο αριθμό τριών latent factors πάνω στις τυποποιημένες μεταβλητές της κύριας EFA. Πριν από την διαδικασία τυποποίησης, στις per-KLOC και density μεταβλητές εφαρμόστηκε μετασχηματισμός $\log(1 + x)$, με στόχο τον περιορισμό της έντονα εμφανιζόμενης ασυμμετρίας των μη αρνητικών δεικτών. Στη συνέχεια, οι μεταβλητές μεταφέρθηκαν σε κοινή κλίμακα μέσω standardization και πάνω σε αυτόν τον χώρο εκτιμήθηκε το παραγοντικό μοντέλο.

Το αποτέλεσμα του extraction δεν αποτελεί ακόμη την τελική ερμηνεύσιμη παραγοντική λύση, όπως υλοποιήθηκε. Η αρχική δομή των loadings μπορεί να περιλαμβάνει μεταβλητές που συνδέονται ταυτόχρονα με περισσότερους από έναν παράγοντες και συνεπώς να μην παρουσιάζει σαφή διάκριση μεταξύ τους. Για τον

λόγο αυτό, μετά το extraction εφαρμόζεται rotation, με στόχο τη δημιουργία απλούστερης και περισσότερο ερμηνεύσιμης παραγοντικής δομής. Η διαδικασία αυτή παρουσιάζεται στην συνέχεια της παρούσας βιβλιογραφικής προσέγγισης.

7.8.5 Rotation

Μετά την εξαγωγή (extraction) των παραγόντων εφαρμόζεται rotation, ώστε να δημιουργηθεί μία απλούστερη και περισσότερο ερμηνεύσιμη παραγοντική δομή. Η περιστροφή δεν μεταβάλλει τον βασικό παραγοντικό χώρο που έχει εξαχθεί, αλλά αναπροσανατολίζει τους άξονες των παραγόντων προκειμένου οι σχέσεις τους με τις αρχικές μεταβλητές να γίνονται σαφέστερες. Κάθε μεταβλητή, σε ιδανικό σενάριο, παρουσιάζει υψηλό loading σε έναν περιορισμένο αριθμό παραγόντων και χαμηλότερα loadings στους υπόλοιπους.

Η rotation δεν αλλάζει τη συνολική κοινή διακύμανση που αναπαράγει η επιλεγμένη factor solution, αλλά ανακατανέμει τα loadings ώστε η δομή να είναι ευκολότερα ερμηνεύσιμη. Η Varimax επιβάλλει ορθογώνιους παράγοντες, ενώ oblique rotations επιτρέπουν μεταξύ τους συσχέτιση (Fabrigar et al., 1999). Λόγω του ότι πρόκειται για orthogonal rotation, διατηρείται η ορθογωνιότητα των αξόνων των παραγόντων και επομένως η λύση παραμένει προσανατολισμένη σε διακριτές παραγοντικές διαστάσεις.

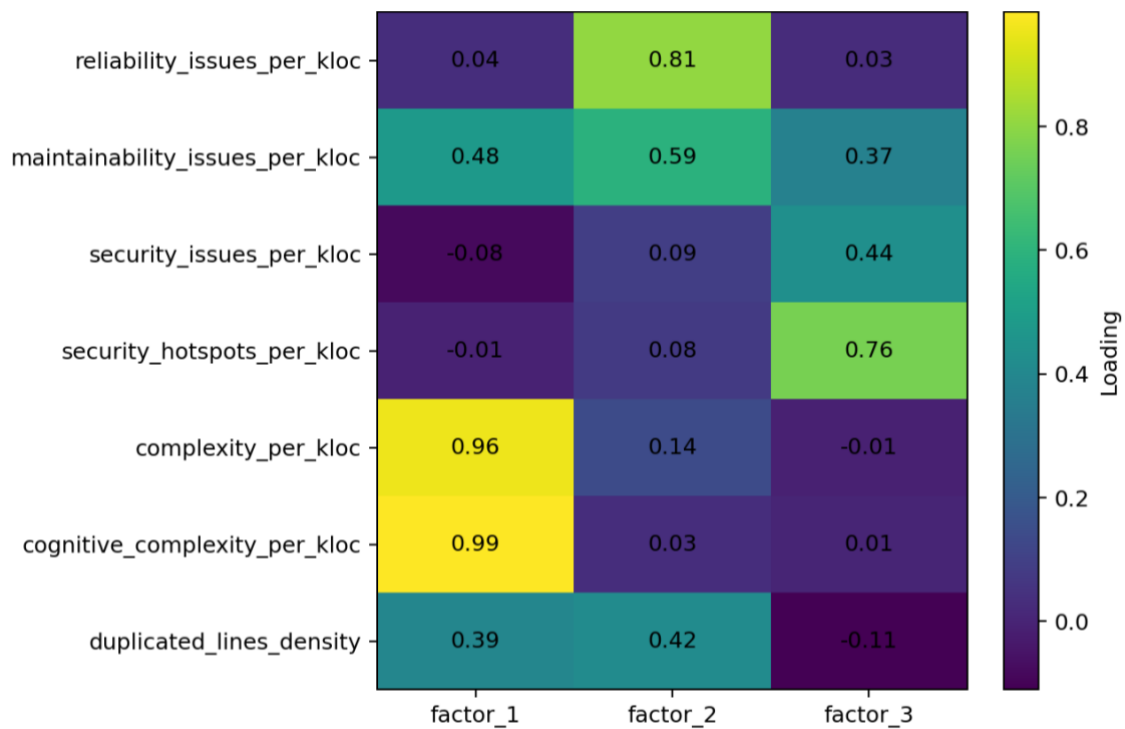
Η χρήση rotation δεν δημιουργεί νέους παράγοντες ούτε μεταβάλλει τον αριθμό τους, ο οποίος έχει ήδη προσδιοριστεί μέσω της Parallel Analysis. Αναδιανέμει, αντίθετα, τα loadings μεταξύ των διατηρούμενων παραγόντων με στόχο την επίτευξη μιας περισσότερο απλής και ερμηνεύσιμης δομής. Παράλληλα, σε μία ορθογώνια περιστροφή, η συνολική κοινή πληροφορία που αναπαρίσταται από το παραγοντικό μοντέλο διατηρείται, παρότι αλλάζει ο τρόπος με τον οποίο αυτή κατανέμεται μεταξύ των επιμέρους factors.

Στην κύρια Factor Analysis, η Varimax εφαρμόστηκε στους τρεις παράγοντες που είχαν επιλεγεί από την Parallel Analysis. Η ίδια διαδικασία εφαρμόστηκε και στη size-adjusted sensitivity analysis, στην οποία είχαν διατηρηθεί δύο παράγοντες. Η περιστραμμένη λύση χρησιμοποιείται στη συνέχεια για την εξέταση των factor loadings, των communalities και της uniqueness, καθώς για την ουσιαστική ερμηνεία του περιεχομένου κάθε παράγοντα.

7.8.6 Loadings, communalities και uniqueness

Η αξιολόγηση της παραγοντικής δομής βασίζεται κυρίως στα factor loadings, στις communalities και στις uniqueness values. Τα factor loadings εκφράζουν την ένταση και την κατεύθυνση της σχέσης κάθε παρατηρούμενης μεταβλητής με έναν λανθάνοντα παράγοντα. Όσο μεγαλύτερη είναι η απόλυτη τιμή ενός loadings, τόσο ισχυρότερη θεωρείται η σύνδεση της μεταβλητής με τον συγκεκριμένο factor, ενώ το πρόσημο υποδεικνύει την κατεύθυνση της σχέσης.

Τα factor loadings αποτυπώνουν τη σχέση κάθε δείκτη με τον παράγοντα. Η communality εκφράζει το μέρος της διακύμανσης που αναπαράγεται από τους κοινούς παράγοντες, ενώ η uniqueness το μέρος που παραμένει ειδικό για τη μεταβλητή ή μη εξηγημένο από τη λύση (Fabrigar et al., 1999). Με την εφαρμογή Varimax rotation, οι ισχυρότερες θετικές φορτίσεις του Factor 1 παρατηρούνται στις complexity_per_kloc (0,959) και cognitive_complexity_per_kloc (0,985), ενώ θετική είναι και η φόρτιση της maintainability_issues_per_kloc (0,484). Στον Factor 2, η υψηλότερη φόρτιση αφορά τη reliability_issues_per_kloc (0,807) και ακολουθούν οι maintainability_issues_per_kloc (0,589) και duplicated_lines_density (0,418). Ο Factor 3 χαρακτηρίζεται κυρίως από τη μετρική security_hotspots_per_kloc (0,759) και, σε μικρότερο βαθμό, από την security_issues_per_kloc (0,436). Το πρότυπο αυτό υποδεικνύει ότι οι επτά αρχικές μεταβλητές δεν συνδέονται ομοιόμορφα με όλους τους factors, αλλά οργανώνονται σε διακριτότερες ομάδες κοινής διακύμανσης.



Εικόνα 7.8.2 – Περιστραμμένα factor loadings των επτά αρχικών μεταβλητών στους τρεις λανθάνοντες παράγοντες της κύριας Factor Analysis έπειτα της εφαρμογής της Varimax rotation

Η communality μιας μεταβλητής εκφράζει το ποσοστό της διακύμανσής της που αναπαρίσταται από το σύνολο των διατηρούμενων παραγόντων. Για μία ορθογώνια παραγοντική λύση μπορεί να υπολογιστεί ως το άθροισμα των τετραγώνων των loadings της μεταβλητής (m) factors:

$$h_j^2 = \sum_{k=1}^m \lambda_{jk}^2$$

Η uniqueness, από την άλλη πλευρά, εκφράζει το μέρος της διακύμανσης που δεν εξηγείται από την κοινή παραγοντική δομή και, για τυποποιημένες μεταβλητές, συνδέεται με την communality ως:

$$u_j^2 = 1 - h_j^2$$

Υψηλή communality συνεπάγεται αποτελεσματική αναπαράσταση της μεταβλητής από τους διατηρούμενους factors. Στην κύρια ανάλυση, η complexity_per_kloc παρουσιάζει communality περίπου (0,940), ενώ η cognitive_complexity_per_kloc περίπου (0,972), γεγονός που δείχνει ότι σχεδόν ολόκληρη η διακύμανσή τους αποτυπώνεται από την κοινή παραγοντική δομή. Αντίθετα, η security_issues_per_kloc εμφανίζει communality ~0,206 και uniqueness ~0,794, υποδεικνύοντας έτσι ότι μεγάλο μέρος της διακύμανσης παραμένει ειδικό στη συγκεκριμένη μεταβλητή. Η duplicated_lined_density παρουσιάζει communality περίπου 0,342 και uniqueness (0,658), επομένως αναπαρίσταται μόνο μερικώς από τους τρεις factors.

Η εξέταση των loadings, επίσης, υποδηλώνει διαφορετικό βαθμό σταθερότητας μεταξύ των παραγόντων. Οι δύο πρώτοι factors (F1, F2) υποστηρίζονται από περισσότερες μεταβλητές με ουσιαστικές φορτίσεις, ενώ ο τρίτος παρουσιάζει πιο περιορισμένη δομή, καθώς στηρίζεται κυρίως στις δύο μεταβλητές ασφάλειας. Για τον λόγο αυτό, η ερμηνεία του τρίτου factor πρέπει να πραγματοποιείται με μεγαλύτερη επιφύλαξη. Η τελική ονομασία και η ουσιαστική ερμηνεία των παραγόντων, καθώς και η αξιοποίηση των αντίστοιχων factor scores, εξετάζονται στη συνέχεια.

7.8.7 Factor scores και ερμηνεία

Εφόσον έχουν προσδιοριστεί η περιστροφή της παραγοντικής δομής, για κάθε μονάδα Pull Request υπολογίζονται τα αντίστοιχα factor scores. Τα factor scores εκφράζουν τον βαθμό στον οποίο κάθε παρατήρηση χαρακτηρίζεται από τον αντίστοιχο λανθάνοντα παράγοντα, με βάση τον συνδυασμό των τιμών των αρχικών μεταβλητών και των εκτιμημένων factor loadings. Ενώ τα loadings περιγράφουν τη σχέση των μεταβλητών με τους factors, τα factor scores περιγράφουν τη θέση των PRs ως προς τις λανθάνουσες διαστάσεις που προέκυψαν.

Τα factor scores δεν αποτελούν άμεσα παρατηρούμενες τιμές των λανθανουσών μεταβλητών, αλλά εκτιμήσεις που προκύπτουν από το προσαρμοσμένο παραγοντικό μοντέλο και τις παρατηρούμενες τιμές των επιμέρους δεικτών. Η χρήση τους στις επόμενες στατιστικές αναλύσεις πρέπει να ερμηνεύεται

ως χρήση εκτιμημένων παραγοντικών βαθμολογιών, αποφεύγοντας την ερμηνεία ως άμεση παρατήρηση των ίδιων των latent factors.

Στην ανάλυση που διενεργείται στο πλαίσιο της παρούσας έρευνας υπολογίστηκαν αρχικά τα raw factor scores και στη συνέχεια τυποποιήθηκαν, ώστε να χρησιμοποιηθούν σε κοινή και περισσότερο ερμηνεύσιμη κλίμακα στις επόμενες στατιστικές αναλύσεις. Ένα υψηλότερο factor score υποδηλώνει εντονότερη παρουσία του προτύπου που χαρακτηρίζει τον αντίστοιχο factor, χωρίς να αποτελεί όμως αυτούσια ένδειξη «καλής» ή «κακής» ποιότητας. Η ερμηνεία του κάθε φορά εξαρτάται από τις μεταβλητές και τις κατευθύνσεις των loadings που συγκροτούν τον συγκεκριμένο παράγοντα.

Βάση της περιστραμμένης παραγοντικής λύσης, ο Factor 1 (F1) χαρακτηρίζεται κυρίως από την complexity και την cognitive complexity, με ουσιαστική συμμετοχή και των maintainability issues. Μπορεί επομένως να ερμηνευθεί ως μία διάσταση δομικής πολυπλοκότητας και επιβάρυνσης συντηρησιμότητας. Υψηλότερο score του Factor 1 αντιστοιχεί σε Pull Requests στα οποία εμφανίζεται εντονότερα ο συγκεκριμένος συνδυασμός χαρακτηριστικών.

Ο Factor 2 (F2) χαρακτηρίζεται κυρίως από reliability issues, ενώ θετική συμμετοχή εμφανίζουν επίσης τα maintainability issues και η duplicated lines density. Η συγκεκριμένη αυτή διάσταση μπορεί να ερμηνευθεί ως ένας ευρύτερος παράγοντας αξιοπιστίας και συντηρησιμότητας, ο οποίος αποτυπώνει διαφορετικό πρότυπο από την καθαρότερη διάσταση πολυπλοκότητας του F1.

Ο Factor 3 (F3) συνδέεται κυρίως με τα security hotspots και σε μικρότερο βαθμό με τα security issues. Μπορεί να ερμηνευθεί και ως μία διάσταση ασφάλειας. Η συγκεκριμένη ερμηνεία απαιτεί μεγαλύτερη επιφύλαξη, καθώς ο τρίτος factor υποστηρίζεται από μικρότερο αριθμό μεταβλητών με ισχυρά loadings και χαρακτηρίστηκε ως πιο εύθραυστος στην αξιολόγηση της παραγοντικής λύσης. Επιπλέον, η size-adjusted sensitivity analysis οδήγησε σε λύση δύο (2) έναντι τριών (3) factors, γεγονός που ενισχύει την ανάγκη προσεκτικής αντιμετώπισης του τρίτου παράγοντα.

Τα τυποποιημένα factor scores, στη συνέχεια, χρησιμοποιήθηκαν ως εξαρτημένες μεταβλητές σε OLS και Linear Mixed Models, προκειμένου να εξεταστεί εάν οι λανθάνουσες αυτές διαστάσεις σχετίζονται με την κατηγορία του Pull Request και το μέγεθος του πηγαίου κώδικα, λαμβάνοντας παράλληλα υπόψη την ομαδοποίηση των PRs στα projects, όπως αυτή συμβαίνει. Η κατηγορία quality χρησιμοποιήθηκε ως κατηγορία αναφοράς, ενώ στα LMM εφαρμόστηκε random intercept ανά project. Η διαδικασία αυτή συνδέει την παραγοντική ανάλυση με την ιεραρχική δομή του ερευνητικού δείγματος, χωρίς να αντιμετωπίζει την EFA και το μοντέλο LMM ως ανταγωνιστικές μεταξύ τους μεθόδους.

Τα αποτελέσματα εκτέλεσης των LMM εμφανίζουν ότι το log_ncloc παρουσιάζει θετική και στατιστικά σημαντική σχέση με τον F1 ($\beta \sim 0,154$, $p < 0,001$) και τον F2 ($\beta \sim 0,078$, $p < 0,001$), ενώ για τον F3 η σχέση είναι μικρή και αρνητική ($\beta \sim 0,034$, $p = 0,004$). Οι κατηγορίες, αντίθετα, both και security δεν παρουσιάζουν στατιστικά σημαντικές διαφοροποιήσεις από την κατηγορία quality σε κανέναν από τους τρεις factors. Παράλληλα, οι τιμές του ICC είναι ιδιαίτερα υψηλές και για τις τρεις λανθάνουσες διαστάσεις, με τιμές ~94,7% για τον Factor 1, ~87,1% για τον

Factor 2 και ~78,5% για τον Factor 3. Το αποτέλεσμα αυτό υποδεικνύει έντονη συγκέντρωση των factor scores σε επίπεδο project και ενισχύει τη σημασία της ιεραρχικής δομής των δεδομένων ακόμη και όταν οι αρχικές μετρικές έχουν αντικατασταθεί από ευρύτερες λανθάνουσες διαστάσεις. Και τα τρία κύρια LMM συγκλίνουν επιτυχώς.

Καταλήγοντας, η Factor Analysis αναδεικνύει μία περισσότερο σύνθετη εικόνα από έναν απλό διαχωρισμό μεταξύ «ποιότητας» και «ασφάλειας». Η πολυπλοκότητα και η συντηρησιμότητα, η αξιοπιστία και η ασφάλεια εμφανίζονται ως μεταξύ τους διακριτές και παράλληλα αλληλένδετες διαστάσεις, ενώ η ισχυρή επίδραση του project στα factor scores υποδεικνύει ότι τα χαρακτηριστικά του εκάστοτε λογισμικού και του repository διαδραματίζουν σημαντικό ρόλο στη διαμόρφωση αυτών των προτύπων. Η ερμηνεία αυτή παραμένει διερευνητική, ιδιαίτερα ως προς τον τρίτο περιγραφόμενο factor, και πρέπει να συνεκτιμηθεί με τα αποτελέσματα της PCA και των προηγούμενα χρησιμοποιημένων μοντέλων παλινδρόμησης.

7.9 PCA έναντι Factor Analysis

Οι δύο στατιστικές τεχνικές, Principal Component Analysis (PCA) και Factor Analysis (FA), εφαρμόστηκαν στο ίδιο σύνολο επτά αρχικών μεταβλητών, προκειμένου να είναι δυνατή η άμεση σύγκριση της πολυμεταβλητής δομής που αναδεικνύει καθεμία. Παρότι οι δύο μέθοδοι χρησιμοποιούν τη συσχετιστική πληροφορία των ίδιων δεδομένων, εξυπηρετούν διαφορετικούς σκοπούς. Η PCA επιδιώκει κυρίως τη συμπύκνωση της συνολικής διακύμανσης των παρατηρούμενων μεταβλητών σε μικρότερο αριθμό συνιστωσών, ενώ η Factor Analysis εστιάζει στην κοινή διακύμανση και στη διερεύνηση πιθανών λανθανουσών διαστάσεων που βρίσκονται πίσω από τις μεταξύ τους συσχετίσεις.

Παρά τη διαφορετική λογική την οποία η κάθε ανάλυση ακολουθεί, τα αποτελέσματα των δύο αυτών προσεγγίσεων παρουσιάζουν σημαντικά σημεία σύγκλισης. Στην κύρια ανάλυση, η Parallel Analysis οδήγησε στη διατήρηση τριών principal components στην PCA και τριών factors στη Factor Analysis. Επιπλέον, οι δομές που προέκυψαν εμφανίζουν ουσιαστικές αντιστοιχίες ως προς τις μεταβλητές που συμμετέχουν περισσότερο σε καθεμία από τις βασικές διαστάσεις.

Πιο ειδικά, η πρώτη διάσταση και στις δύο αναλύσεις συνδέεται έντονα με την complexity, την cognitive complexity και τη maintainability. Στην PCA, οι συγκεκριμένες μεταβλητές παρουσιάζουν από τα υψηλότερα loadings στο PC1, ενώ στην Factor Analysis η complexity και η cognitive complexity κυριαρχούν στον Factor 1 και συνοδεύονται από ουσιαστική συμμετοχή των maintainability issues. Η συμφωνία αυτή παρέχει ισχυρή ένδειξη για την ύπαρξη μιας ευρύτερης διάστασης που σχετίζεται με τη δομική πολυπλοκότητα και την επιβάρυνση συντηρησιμότητας του κώδικα.

Σύγκλιση αντίστοιχη με την παραπάνω παρατηρείται και στις μεταβλητές ασφάλειας. Στην PCA, το PC2 χαρακτηρίζεται κυρίως από υψηλά loadings στα security issues και στα security hotspots, ενώ στην Factor Analysis οι ίδιες μεταβλητές συγκεντρώνονται κυρίως στον Factor 3. Παρότι, άρα, η αρίθμηση των διαστάσεων μεταξύ των δύο αυτών μεθόδων διαφέρει, και οι δύο αναλύσεις αναδεικνύουν μια σχετικά διακριτή διάσταση ασφάλειας.

Η σημαντικότερη διαφοροποίηση έγκειται στη διάσταση που σχετίζεται με την αξιοπιστία. Στην PCA, η reliability_issues_per_kloc συνδέεται ισχυρότερα με το PC3, μαζί με μικρότερες θετικές και αρνητικές συνεισφορές άλλων μεταβλητών. Στην Factor Analysis, αντίθετα, η αξιοπιστία οργανώνεται περισσότερο καθαρά στον Factor 2, μαζί με τα maintainability issues και την duplicated lines density. Η Factor Analysis προσφέρει στο σημείο αυτό περισσότερο διακριτή ερμηνευτική δομή, καθώς επιχειρεί να απομονώσει την κοινή διακύμανση των μεταβλητών και όχι απλώς να μεγιστοποιήσει τη συνολική διακύμανση που αποτυπώνεται σε κάθε διάσταση.

Διάσταση	PCA	Factor Analysis	Συνολική ερμηνεία
Πολυπλοκότητα / Συντηρησιμότητα	PC1	Factor 1	Ισχυρή σύγκλιση
Ασφάλεια	PC2	Factor 3	Ισχυρή σύγκλιση, αλλά ο Factor 3 είναι πιο εύθραυστος
Αξιοπιστία	PC3	Factor 2	Μερική σύγκλιση, διαφορετική οργάνωση των συνοδευτικών μετρικών
Κύρια διαστατικότητα	3 PCs	3 factors	Συμφωνία
Size-adjusted sensitivity	2 PCs	2 factors	Συμφωνία και ένδειξη sensitivity

Πίνακας 7.9.1 – Συγκριτική αντιστοίχιση των κύριων πολυμεταβλητών διαστάσεων που προέκυψαν από την PCA και τη Factor Analysis

Η ερμηνεία της συμφωνίας που εμφανίζεται μεταξύ των προσεγγίσεων δεν θα πρέπει να ταυτίζεται με απόδειξη των τριών διαστάσεων ως οριστικών και ανεξάρτητων χαρακτηριστικών του λογισμικού. Στην Factor Analysis, ο τρίτος παράγοντας χαρακτηρίστηκε ως περισσότερο εύθραυστος, καθώς υποστηρίζεται ουσιαστικά από περιορισμένο αριθμό μεταβλητών, ενώ τα security issues παρουσιάζουν χαμηλή communality. Παράλληλα, και στις δύο μεθόδους η size-adjusted sensitivity analysis μετέβαλε τον αριθμό των διατηρούμενων διαστάσεων από τρεις σε δύο -η συγκεκριμένη sensitivity analysis πραγματοποιήθηκε επειδή αρκετοί από τους δείκτες της κύριας πολυμεταβλητής ανάλυσης χρησιμοποιούσαν το KLOC ως κοινό παρονομαστή. Κατά συνέπεια η μετάβαση από τρεις σε δύο διαστάσεις έπειτα του size adjustment αντιμετωπίζεται ως ένδειξη ότι μέρος της ακριβούς dimensionality επηρεάζεται από τον τρόπο κανονικοποίησης ως προς το μέγεθος του πηγαίου κώδικα και όχι ως αναίρεση της συνολικής πολυδιάστατης

δομής-. Η κοινή αυτή μεταβολή υποδεικνύει ότι η γενικότερη πολυμεταβλητή δομή είναι υπαρκτή, αλλά η ακριβής διαστατικότητα της επηρεάζεται από τον τρόπο με τον οποίο λαμβάνεται υπόψη το μέγεθος του κώδικα.

Οι δύο παραπάνω προσεγγίσεις παρουσιάζουν ταυτόχρονα και κοινά ευρήματα όταν οι αντίστοιχες component και factor scores εξετάζονται στο πλαίσιο των LMM. Παρατηρείται έντονη διακύμανση, αφορά και τις δύο προαναφερθείσες περιπτώσεις, σε επίπεδο project, με χαρακτηριστικά υψηλές τιμές ICC, γεγονός το οποίο δείχνει ότι οι ευρύτερες πολυμεταβλητές διαστάσεις των μετρικών δεν διαφοροποιούνται μόνο μεταξύ Pull Requests αλλά συνδέονται σε μεγάλο βαθμό και με το project στο οποίο αυτά ανήκουν. Οι κατηγορίες quality, security και both παράλληλα δεν εμφανίζουν σταθερά στατιστικά σημαντικές διαφοροποιήσεις στις αντίστοιχες συνθετικές διαστάσεις.

Καταλήγουμε ότι, συνολικά, η PCA και Factor Analysis δεν λειτουργούν ως ανταγωνιστικές τεχνικές, αλλά ως συμπληρωματικές προσεγγίσεις της ίδιας πολυμεταβλητής δομής. Η PCA δείχνει ότι σημαντικό μέρος της πληροφορίας των επτά αρχικών μεταβλητών μπορεί να συμπυκνωθεί σε μικρότερο αριθμό διαστάσεων, ενώ η Factor Analysis επιτρέπει την περισσότερο ερμηνευτική διερεύνηση της κοινής διακύμανσης πίσω από αυτές. Η σύγκλιση των δύο μεθόδων ως προς τη διάσταση πολυπλοκότητας και συντηρησιμότητας, τη διάσταση ασφάλειας και τη σημασία της αξιοπιστίας ενισχύει την αξιοπιστία της συνολικής πολυμεταβλητής εικόνας. Ταυτόχρονα, οι διαφοροποιήσεις τους και η ευαισθησία της τρίτης διάστασης υποδεικνύουν ότι η δομή των μετρικών πρέπει να ερμηνεύεται ως σύνθετη και διερευνητική και όχι ως ένας αυστηρός διαχωρισμός μεταξύ ανεξάρτητων διαστάσεων ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού που υλοποιείται από την χρήση του.

7.10 Robustness και sensitivity analyses

Η αξιολόγηση της αξιοπιστίας των αποτελεσμάτων της παρούσας έρευνας έγινε μέσω συμπληρωματικών robustness και sensitivity analyses, με στόχο να εξεταστεί κατά πόσο τα βασικά συμπεράσματα εξαρτώνται από συγκεκριμένες επιλογές μοντελοποίησης, από τον τρόπο προσαρμογής ως προς το μέγεθος του κώδικα ή από τη σύνθεση του ερευνητικού δείγματος. Οι έλεγχοι επιλέχθηκαν με βάση τα χαρακτηριστικά των δεδομένων και εφαρμόστηκαν μόνο σε σημεία όπου υπήρχε συγκεκριμένη μεθοδολογική αιτιολόγηση.

Στα baseline μοντέλα OLS, για αρχή, χρησιμοποιήθηκαν cluster-robust standard errors σε επίπεδο project. Με τον τρόπο αυτό, η εκτίμηση της αβεβαιότητας των συντελεστών δεν βασίζεται στην υπόθεση ότι όλα τα PRs είναι ανεξάρτητα μεταξύ τους, αλλά επιτρέπει την ύπαρξη συσχέτισης μεταξύ παρατηρήσεων που προέρχονται από το ίδιο project. Η ίδια λογική cluster-robust εκτίμησης χρησιμοποιήθηκε και στα συμπληρωματικά Negative Binomial models, καθώς

επίσης το GEE χρησιμοποιεί αντίστοιχα robust covariance estimation στο επίπεδο του project.

Ιδιαίτερος έλεγχος ευαισθησίας πραγματοποιήθηκε ως προς το μέγεθος των project-level clusters. Τα κύρια LMM επανεκτιμήθηκαν περιορίζοντας με τον τρόπο αυτό το δείγμα αποκλειστικά σε projects που περιείχαν τουλάχιστον πέντε (5) Pull Requests. Το sensitivity sample περιλάμβανε 3.544 Pull Requests από 149 projects, έναντι 3.701 Pull Requests και 242 projects στην κύρια ανάλυση. Οι βασικές σχέσεις για την πυκνότητα των προβλημάτων ποιότητας παρέμειναν σταθερές. Ο συντελεστής του *log_ncloc* μεταβλήθηκε από $\sim -0,152$ σε $\sim -0,141$, ενώ ο συντελεστής του *log_complexity* από $\sim 0,398$ σε $\sim 0,369$, με τις δύο σχέσεις να παραμένουν στατιστικά σημαντικές. Οι κατηγορίες *security* και *both* εξακολούθησαν να μην παρουσιάζουν στατιστικά σημαντικές διαφοροποιήσεις από την κατηγορία αναφοράς *quality*. Η γενική σταθερότητα των εκτιμήσεων δείχνει ότι τα κύρια αποτελέσματα δεν καθαρίζονται από projects με πολύ μικρό αριθμό πλήθους Pull Requests.

Η επιλογή της δομής των random effects εξετάστηκε επίσης μέσω ενός πιο σύνθετου random-slope model, στο οποίο επιτράπηκε η επίδραση του *log_ncloc* να διαφοροποιείται μεταξύ projects. Το συγκεκριμένο μοντέλο δεν πέτυχε κάποια σύγκλιση, σε αντίθεση με τα κύρια random-intercept LMM, τα οποία συγκλίνουν κανονικά. Το αποτέλεσμα αυτό δεν αποτελεί από μόνο του απόδειξη ότι οι πραγματικές slopes είναι ίδιες μεταξύ όλων των projects, αλλά υποδεικνύει ότι τα διαθέσιμα δεδομένα δεν υποστηρίζουν με επαρκή σταθερότητα την πιο σύνθετη random-effects structure. Έτσι, διατηρήθηκε ως κύρια προδιαγραφή το απλούστερο και σταθερότερο random-intercept model.

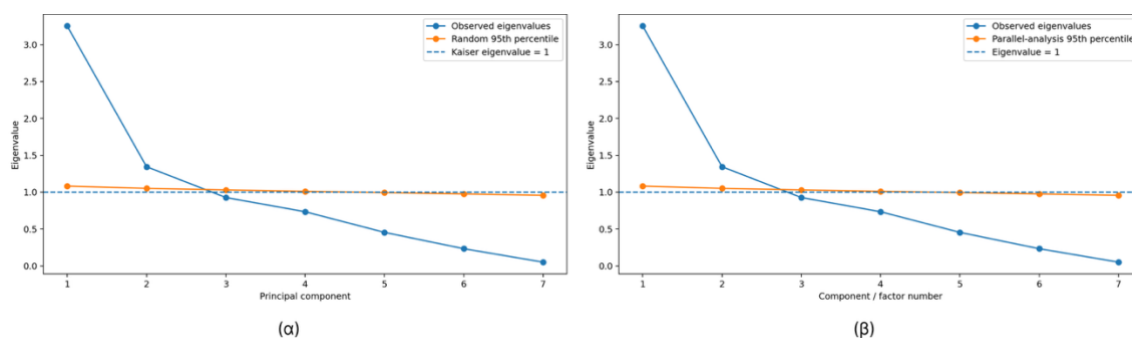
Εξαιτίας του γεγονότος ότι αρχικά τα *maintainability* και *security issues* αποτελούν count variables, η ανάλυση επαναλήφθηκε συμπληρωματικά με Negative Binomial Generalized Linear Models, χρησιμοποιώντας το *log(ncloc)* ως offset. Με τον τρόπο αυτό το μέγεθος του κώδικα αντιμετωπίζεται ως exposure χωρίς να απαιτείται προηγούμενος σχηματισμός πυκνότητας issues ανά KLOC μονάδα. Στο μοντέλο των *maintainability issues*, η πολυπλοκότητα παρέμεινε θετικά και στατιστικά σημαντικά συνδεδεμένη με τον αριθμό των issues, ενώ η κατηγορία *security* εμφάνισε χαμηλότερο αναμενόμενο πλήθος *maintainability issues* από την κατηγορία *quality* ($p = 0,003$). Στο αντίστοιχο μοντέλο των *security issues*, η κατηγορία *both*, σύμφωνα με τα αποτελέσματα της ανάλυσης, παρουσίασε υψηλότερο αναμενόμενο πλήθος *security issues* ($p = 0,002$). Οι διαφοροποιήσεις που αναλύθηκαν σε σχέση με τα κύρια LMM δείχνουν ότι ορισμένα αποτελέσματα που αφορούν την κατηγορία του Pull Request είναι ευαίσθητα στη μορφή του μοντέλου και επομένως δεν ερμηνεύονται ως πλήρως σταθερά ευρήματα.

Εφαρμόστηκε συμπληρωματικά GEE Binomial model στην εξέταση της αναλογίας των *security issues* προς το σύνολο των *security* και *maintainability issues*. Η συγκεκριμένη ανάλυση αφορούσε διαφορετικό αλλά συναφές ερευνητικό ερώτημα, το οποίο αναφέρεται στον βαθμό κατά τον οποίο το μέγεθος και η πολυπλοκότητα σχετίζονται με σχετική επικράτηση προβλημάτων ασφάλειας έναντι προβλημάτων συντηρησιμότητας. Στο υποσύνολο των Pull Requests με τουλάχιστον ένα issue, ούτε το *log_ncloc* ($p = 0,527$) καθώς και ούτε το *log_complexity* ($p = 0,135$)

παρουσίασαν στατιστικά σημαντική σχέση με το security share. Το αποτέλεσμα αυτό δεν υποδεικνύει ότι το μέγεθος ή η πολυπλοκότητα του πηγαίου κώδικα εμφανίζονται ως άσχετα με την ασφάλεια του λογισμικού. Αφορά ένα συγκεκριμένο estimand, δηλαδή τη σχετική αναλογία των security issues έναντι του συνολικού πλήθους security και maintainability issues, και επομένως δεν αποτελεί άμεσο έλεγχο της πιθανότητας ύπαρξης vulnerability ή του συνολικού security risk. Παράλληλα, η απουσία σταθερής σχέσης των δύο συγκεκριμένων predictors με το security_share είναι συμβατή με την ευρύτερη εικόνα της βιβλιογραφίας πρόβλεψης ευπαθειών, σύμφωνα με την οποία η security-related συμπεριφορά του κώδικα δεν αποτυπώνεται επαρκώς από μία μικρή ομάδα απλών structural metrics.

Υπό αυτό το πρίσμα, το μη στατιστικά σημαντικό αποτέλεσμα του GEE δεν θα πρέπει να ερμηνεύεται ως απόρριψη της πιθανής σχέσης μεταξύ χαρακτηριστικών του κώδικα και ασφάλειας. Αντίθετα, υποδηλώνει ότι, για το συγκεκριμένο outcome και το συγκεκριμένο δείγμα, το μέγεθος και η πολυπλοκότητα από μόνα τους δεν παρείχαν επαρκή στατιστική πληροφορία για τη διαφοροποίηση της σχετικής σύνθεσης των security και maintainability findings. Η διερεύνηση security-related outcomes ενδέχεται επομένως να απαιτεί πιο πλούσιο σύνολο από predictors, στο οποίο να συνδυάζονται παράλληλα structural metrics με χαρακτηριστικά της ιστορίας ανάπτυξης, της αλλαγής καθώς και του τεχνικού περιβάλλοντος κάθε project.

Ιδιαίτερη σημασία αποκτάει επίσης η εξέταση εναλλακτικού τρόπου προσαρμογής ως προς το μέγεθος του κώδικα στις πολυμεταβλητές αναλύσεις. Η κύρια PCA και Factor Analysis βασίστηκαν σε per-KLOC και density metrics, αρκετές από τις οποίες μοιράζονται έναν κοινό παρονομαστή (σε κατάσταση αλγεβρικού κλάσματος). Για τον λόγο αυτό, πραγματοποιήθηκαν κάθε φορά ξεχωριστές size-adjusted sensitivity analyses, χωρίς την ίδια ratio-based κανονικοποίηση. Στην PCA, η κύρια ανάλυση διατήρησε τρεις τελικές συνιστώσες οι οποίες ήταν ικανές να εξηγήσουν συνολικά το ~77,5% της διακύμανσης, ενώ η size-adjusted εκδοχή διατήρησε δύο συνιστώσες με σωρευτική explained variance ~65,7%. Αντίστοιχα, στη Factor Analyses η κύρια λύση περιλάμβανε τρεις factors, ενώ η size-adjusted ανάλυση οδήγησε σε δύο.



Εικόνα 7.10.1 – Αποτελέσματα της Parallel Analysis στις size-adjusted sensitivity αναλύσεις, (α) PCA και (β) Factor Analysis

Η συγκεκριμένη μεταβολή κρίνεται ως ουσιαστική για την ορθή ερμηνεία των πολυμεταβλητών αποτελεσμάτων. Οι βασικές διαστάσεις που σχετίζονται με την πολυπλοκότητα, τη συντηρησιμότητα και την αξιοπιστία εξακολουθούν να εμφανίζονται, όμως ο ακριβής αριθμός των διαστάσεων δεν είναι πλήρως invariant ως προς τον τρόπο ελέγχου του code size. Επομένως, η τρισδιάστατη λύση της κύριας PCA και της Factor Analysis αντιμετωπίζονται ως χρήσιμη και ερμηνεύσιμη, αλλά όχι ως μοναδική ή οριστική αναπαράσταση της υποκείμενης δομής.

Ανάλογοι sensitivity checks με περιορισμό σε projects με τουλάχιστον πέντε (5) Pull Requests πραγματοποιήθηκαν και στα LMM των component scores και factor scores. Οι κύριες κατευθύνσεις των εκτιμήσεων και η έντονη project-level διακύμανση διατηρήθηκαν, υποστηρίζοντας ότι η ισχυρή επίδραση του project δεν αποτελεί αποκλειστικά αποτέλεσμα των μικρότερων clusters.

Συνολικά, οι robustness και sensitivity analyses δείχνουν ότι τα βασικά συμπεράσματα σχετικά με τη σημασία της ιεραρχικής δομής των δεδομένων, τη σχέση της πολυπλοκότητας με τη διάσταση της συντηρησιμότητας και τη σημαντική διαφοροποίηση μεταξύ projects παραμένουν σταθερά κάτω από διαφορετικές αναλυτικές προδιαγραφές. Ορισμένα ευρήματα, παράλληλα, παρουσιάζουν μεγαλύτερη ευαισθησία στις μεθοδολογικές επιλογές -ιδιαίτερα εκείνα τα οποία αφορούν τις κατηγορίες των Pull Requests και τον ακριβή αριθμό των πολυμεταβλητών διαστάσεων-. Η διάκριση αυτή σημειώνεται ως σημαντική, καθώς επιτρέπει τα αποτελέσματα της έρευνας να ερμηνεύονται με βάση τον πραγματικό βαθμό σταθερότητάς τους και όχι αποκλειστικά βάσει ενός μοναδικού στατιστικού μοντέλου.

7.10.1 Negative Binomial GLM

Η Negative Binomial regression είναι κατάλληλη για count outcomes όταν η διακύμανση υπερβαίνει ουσιαστικά τον μέσο και η Poisson specification δεν αποτυπώνει την overdispersion. Με log link και offset $\log(ncloc)$, το μοντέλο εκτιμά ρυθμούς συμβάντων ως προς την έκθεση σε κώδικα (Hilbe, 2011).

Η επιλογή της Negative Binomial specification σχετίζεται με τη φύση των outcomes ως count variables και με τη σημαντική διασπορά τους. Παρότι ιδιαίτερα οι security-related counts χαρακτηρίζονται από μεγάλο ποσοστό μηδενικών τιμών, το μοντέλο που εφαρμόστηκε δεν αποτελεί zero-inflated Negative Binomial model. Η Negative Binomial ανάλυση, καταλήγουμε, δεν χρησιμοποιείται στην παρούσα εργασία ως ειδική μοντελοποίηση ενός υποτιθέμενου πρόσθετου μηχανισμού παραγωγής μηδενικών παρατηρήσεων. Η ύπαρξη υψηλής συγκέντρωσης στο μηδέν καταγράφεται περιγραφικά και λαμβάνεται υπόψη κατά την ερμηνεία των robustness results.

Το στατιστικό μοντέλο χρησιμοποιεί λογαριθμική συνάρτηση σύνδεσης και μπορεί γενικά να αποδοθεί ως:

$$\log(E[Y_i]) = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi} + \log(ncloc_i)$$

όπου Y_i χαρακτηρίζεται ο αριθμός των εξεταζόμενων issues και το $\log(ncloc_i)$ εισάγεται ως offset. Ο συντελεστής του offset δεν εκτιμάται αλλά θεωρείται ίσος με τη μονάδα (1), με αποτέλεσμα το μέγεθος του αναλυόμενου κώδικα να λειτουργεί ως exposure. Έτσι, το μοντέλο εξετάζει τον αναμενόμενο ρυθμό εμφάνισης issues λαμβάνοντας υπόψη τη διαφορετική ποσότητα κώδικα, χωρίς να απαιτείται προηγούμενος σχηματισμός density variable.

Στην παρούσα έρευνα χρησιμοποιήθηκαν επιπλέον cluster-robust errors σε επίπεδο project. Το Negative Binomial model αποτελεί συμπληρωματικό έλεγχο της σταθερότητας των αποτελεσμάτων και όχι αντικατάσταση του κύριου LMM, καθώς δεν εισάγει random project effect και συνεπώς δεν πραγματοποιεί το αντίστοιχο partial pooling μεταξύ projects.

Η συγκεκριμένη προδιαγραφή λειτουργεί παράλληλα ως συμπληρωματικός έλεγχος της ratio-based κύριας ανάλυσης. Ενώ τα OLS και LMM χρησιμοποιούν issue densities ανά KLOC ως outcomes, το Negative Binomial μοντελοποιεί άμεσα τα issue counts και ενσωματώνει το ncloc ως exposure μέσω του offset. Με τον τρόπο αυτό παρέχεται μία εναλλακτική εκτίμηση των σχέσεων η οποία δεν βασίζεται στην άμεση χρήση του λόγου *issues/KLOC* ως εξαρτημένης μεταβλητής.

7.10.2 Generalized Estimating Equations (GEE)

Ως δεύτερη συμπληρωματική προσέγγιση χρησιμοποιήθηκαν Generalized Estimating Equations, οι οποίες επιτρέπουν την εκτίμηση population-averaged σχέσεων όταν οι παρατηρήσεις οργανώνονται σε συσχετισμένες ομάδες (Liang & Zeger, 1986). Στην παρούσα ανάλυση η ομαδοποίηση ορίστηκε σε επίπεδο project. Στόχος ήταν η λήψη ως παραμέτρου πιθανής συσχέτισης μεταξύ Pull Requests που προέρχονται από το ίδιο έργο.

Η αναλογία η οποία χρησιμοποιήθηκε ως outcome είναι η εξής:

$$security_share_i = \frac{SecurityIssues_i}{SecurityIssues_i + MaintainabilityIssues_i}$$

για Pull Requests στα οποία το συνολικό πλήθος των δύο κατηγοριών issues ήταν μεγαλύτερο από μηδέν. Το μοντέλο εφαρμόστηκε με Binomial family, exchangeable working correlation structure και robust covariance estimation σε επίπεδο project.

Η συγκεκριμένη ανάλυση δεν εκτιμά το συνολικό επίπεδο κινδύνου ασφάλειας ενός Pull Request, την πιθανότητα ύπαρξης exploitable vulnerability ή την πιθανότητα πραγματοποίησης περιστατικού ασφάλειας. Το outcome εκφράζει αποκλειστικά τη σχετική σύνδεση των δύο κατηγοριών findings. Όσο μεγαλύτερη είναι η τιμή του security_share, τόσο μεγαλύτερο είναι το μερίδιο των καταγεγραμμένων security issues σε σχέση με το συνολικό πλήθος security και maintainability issues της συγκεκριμένης παρατήρησης. Συνεπώς, το GEE εξετάζει την περίπτωση τα log_ncloc και log_complexity να σχετίζονται με τη σχετική κατανομή των δύο ειδών findings και όχι με το απόλυτο επίπεδο ασφάλειας ή κινδύνου του λογισμικού.

7.10.3 Sensitivity analysis ως προς το μέγεθος των project clusters

Η κύρια ανάλυση περιλάμβανε projects με ιδιαίτερα άνισο αριθμό Pull Requests. Παρότι το random-intercept LMM αντιμετωπίζει αυτή τη δομή μέσω partial pooling, πραγματοποιήθηκε πρόσθετος έλεγχος ευαισθησίας ώστε να εξεταστεί η ύπαρξη των πιθανοτήτων τα κύρια αποτελέσματα να εξαρτώνται από projects με πολύ μικρό αριθμό παρατηρήσεων.

Για τον σκοπό αυτό, το κύριο quality LMM επανεκτιμήθηκε σε υποσύνολο που περιλάμβανε αποκλειστικά projects με τουλάχιστον πέντε (5) περιεχόμενα Pull Requests. Η επιλογή του συγκεκριμένου threshold χρησιμοποιήθηκε ως πρακτικός έλεγχος σταθερότητας και όχι ως θεωρητικό όριο επάρκειας ενός project. Η σύγκριση των εκτιμήσεων του πλήρους και του περιορισμένου δείγματος επιτρέπει να αξιολογηθεί εάν η κατεύθυνση, το μέγεθος και η στατιστική αβεβαιότητα των βασικών fixed effects μεταβάλλονται ουσιαστικά μετά τον αποκλεισμό των μικρότερων clusters.

7.10.4 Size-adjusted sensitivity analysis των PCA και Factor Analysis

Η κύρια Principal Component Analysis και η Factor Analysis εφαρμόστηκαν σε επτά δείκτες οι οποίες επέτρεπαν την κοινή εξέταση χαρακτηριστικών ποιότητας, ασφάλειας και δομής του κώδικα. Οι περισσότεροι από αυτούς εφαρμόστηκαν ως πυκνότητες ή αναλογίες ανά μονάδα KLOC, προκειμένου να καταστούν περισσότερο συγκρίσιμοι μεταξύ codebases διαφορετικού μεγέθους. Η συγκεκριμένη κανονικοποίηση είναι χρήσιμη για τον περιορισμό της άμεσης επίδρασης του μεγέθους, δημιουργεί όμως μία πρόσθετη μεθοδολογική προσοχή η οποία έγκειται στον διαμοιρασμό του ίδιου NCLOC παρονομαστή σε αρκετές μεταβλητές.

Η χρήση κοινού παρονομαστή σε περισσότερες ratio-based μεταβλητές απαιτεί ιδιαίτερη προσοχή, καθώς έχει καταδειχθεί ότι η κοινή μεταβλητότητα του παρονομαστή μπορεί να δημιουργήσει ή να ενισχύσει παρατηρούμενες συσχετίσεις ακόμα και σε περιπτώσεις όπου οι αντίστοιχοι αριθμητές δεν παρουσιάζουν την ίδια υποκείμενη σχέση (Kim, 1999). Για τον λόγο αυτό, η πολυμεταβλητή δομή που προέκυψε από τις per-KLOC μεταβλητές δεν θεωρήθηκε επαρκής να αξιολογηθεί χωρίς συμπληρωματικό size-adjusted sensitivity check.

Πραγματοποιήθηκε ξεχωριστή size-adjusted sensitivity analysis τόσο για την PCA όσο και για τη Factor Analysis. Αντί της άμεσης διαίρεσης κάθε μετρικής με το KLOC, για κάθε αρχική μεταβλητή εκτιμήθηκε η σχέση της λογαριθμικά μετασχηματισμένης τιμής της με το $\log(1 + ncloc)$. Τα residuals αυτής της σχέσης χρησιμοποιήθηκαν στη συνέχεια ως size-adjusted indicators:

$$\log(1 + x_i) = \alpha + \beta \log(1 + NCLOC_i) + \varepsilon_i$$

όπου το ϵ_i εκφράζει το τμήμα της μεταβλητής το οποίο δεν αποδίδεται στη γραμμική σχέση της με το μέγεθος του αναλυόμενου κώδικα. Η διαδικασία αυτή επιτρέπει να εξεταστεί αν η πολυμεταβλητή δομή παραμένει παρόμοια όταν η άμεση σχέση των επιμέρους μετρικών με το μέγεθος του πηγαίου κώδικα αφαιρείται πριν από την εφαρμογή της PCA ή της Factor Analysis. Η συγκεκριμένη διαδικασία αντιστοιχεί άμεσα στην υλοποίηση των *verified analyses*.

Ο έλεγχος αυτός είναι ιδιαιτέρως σημαντικός για την ερμηνεία των αποτελεσμάτων στην συνέχεια, καθώς η κύρια *per-KLOC* ανάλυση οδήγησε στη διατήρηση τριών διαστάσεων, ενώ οι *size-adjusted* αναλύσεις οδήγησαν σε δύο τελικές διαστάσεις τόσο στην PCA όσο και στη Factor Analysis. Για παράδειγμα, στην PCA, η *Parallel Analysis* διατήρησε τρία *components* στην κύρια ανάλυση και δύο τελικά έπειτα από το *size adjustment*. Η μεταβολή αυτή δείχνει ότι η γενική παρουσία πολυμεταβλητής δομής είναι σταθερή, αλλά ο ακριβής αριθμός των διαστάσεων είναι ευαίσθητος στον τρόπο με τον οποίο λαμβάνεται υπόψη το συνολικό μέγεθος του πηγαίου κώδικα. Η ερμηνεία των αποτελεσμάτων, λόγω των ανωτέρω, δίνει μεγαλύτερη έμφαση στα επαναλαμβανόμενα δομικά πρότυπα μεταξύ των δύο αναλύσεων και όχι στην αντιμετώπιση των τριών αρχικών *dimensions* ως οριστικών.

ΜΕΡΟΣ Γ

ΑΠΟΤΕΛΕΣΜΑΤΑ ΚΑΙ ΕΡΜΗΝΕΙΑ

Αποτελέσματα

Στο παρόν κεφάλαιο παρουσιάζονται τα κύρια εμπειρικά αποτελέσματα της έρευνας, όπως αυτά προέκυψαν από την εφαρμογή των στατιστικών και πολυμεταβλητών αναλύσεων που περιεγράφηκαν στο προηγούμενο μέρος της παρούσας βιβλιογραφίας. Η παρουσίαση επικεντρώνεται στα ευρήματα τα οποία αφορούν τις σχέσεις μεταξύ των μετρικών ποιότητας, ασφάλειας, μεγέθους και πολυπλοκότητας του κώδικα, καθώς και στις διαφοροποιήσεις που εμφανίζονται μεταξύ των επιμέρους Pull Requests και των projects όπου ανήκουν.

Ιδιαίτερη έμφαση δίνεται στη συνέπεια των αποτελεσμάτων μεταξύ των διαφορετικών αναλυτικών προσεγγίσεων, στη στατιστική και πρακτική σημασία των εκτιμήσεων, καθώς και στα ευρήματα που προκύπτουν από τις αναλύσεις PCA και Factor Analysis. Παράλληλα, παρουσιάζονται οι βασικοί έλεγχοι robustness και sensitivity, ώστε να αξιολογηθεί ο βαθμός σταθερότητας των συμπερασμάτων απέναντι σε εναλλακτικές προδιαγραφές και αναλυτικές επιλογές.

8.1 Τελικό δείγμα

Προτού λάβει μέρος την παρουσίαση των επιμέρους στατιστικών αποτελεσμάτων, κρίνεται απαραίτητη η αποτύπωση της τελικής μορφής του ερευνητικού δείγματος πάνω στο οποίο βασίστηκαν οι αναλύσεις. Η διαμόρφωσή

του προέκυψε μέσα από διαδοχικά στάδια φιλτραρίσματος και καθαρισμού των αρχικών δεδομένων, ώστε να διατηρηθούν μόνο οι παρατηρήσεις που πληρούσαν τα κριτήρια της τελικής αναλυτικής στρατηγικής.

Στην παρούσα ενότητα παρουσιάζεται αρχική η ροή φιλτραρίσματος (filtering flow) που οδήγησε από το αρχικό σύνολο δεδομένων στο τελικό δείγμα. Στη συνέχεια καταγράφονται ο τελικός αριθμός των PRs και των projects, η κατανομή των Pull Requests ανά project και η σύνθεση του δείγματος ως προς τις κατηγορίες quality, security και both. Η συγκεκριμένη περιγραφή παρέχει το απαραίτητο πλαίσιο για την ερμηνεία των αποτελεσμάτων που ακολουθούν.

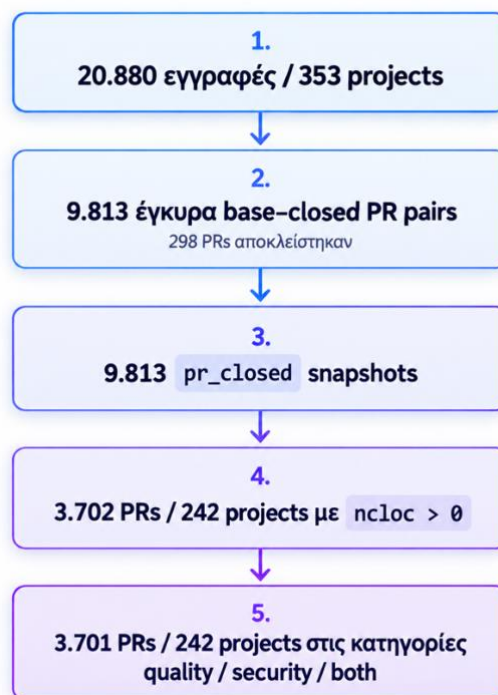
8.1.1 Filtering flow

Η διαμόρφωση του τελικού ερευνητικού δείγματος πραγματοποιήθηκε μέσα από διαδοχικά στάδια ελέγχου και φιλτραρίσματος των αρχικών δεδομένων. Το αρχικό σύνολο περιλάμβανε 20.880 εγγραφές που εξήχθησαν από 353 projects, με κάθε Pull Request να μπορεί να αντιπροσωπεύεται από τις δύο καταστάσεις ανάλυσης *pr_base* και *pr_closed*.

Διατηρήθηκαν, αρχικά, τα Pull Requests για τα οποία υπήρξε έγκυρο ζεύγος των δύο καταστάσεων ανάλυσης. Μετά τον συγκεκριμένο έλεγχο παρέμειναν στο δείγμα 9.813 Pull Requests, ενώ 298 PRs με μη τυπικό αριθμό εγγραφών αποκλείστηκαν. Καθώς η σύγκριση των δύο snapshots είχε δείξει ότι οι περισσότερες μετρικές παρέμεναν αμετάβλητες μεταξύ των καταστάσεων *pr_base* και *pr_closed*, ως τελική μονάδα ανάλυσης διατηρήθηκε μία μόνο εγγραφή ανά Pull Request, η κατάσταση *pr_closed*. Καταλήγουμε, άρα, σε 9.813 ανεξάρτητες παρατηρήσεις σε επίπεδο PR προς περαιτέρω φιλτράρισμα.

Στη συνέχεια εφαρμόστηκε το κριτήριο φιλτραρίσματος $ncloc > 0$, καθώς οι εγγραφές με μηδενικές γραμμές κώδικα δεν επιτρέπουν τον ουσιαστικό υπολογισμό μετρικών πυκνότητας ανά KLOC. Μετά την εφαρμογή του συγκεκριμένου φίλτρου παρέμειναν 3.702 Pull Requests από 242 projects, γεγονός που δείχνει ότι σημαντικό μέρος των αρχικών snapshots δεν περιείχε αναλύσιμο πηγαίο κώδικα.

Τέλος, οι αρχικές κατηγορίες των Pull Requests ενοποιήθηκαν στις τρεις κατηγορίες quality, security και both. Παρατηρήσεις που δεν μπορούσαν να αντιστοιχιστούν σε κάποια από αυτές τις κατηγορίες αποκλείστηκαν από την κύρια ανάλυση. Μετά και το τελευταίο αυτό στάδιο, το τελικό δείγμα αποτελούνταν από 3.701 Pull Requests που προέρχονταν από 242 projects.



Εικόνα 8.1.1 – Ροή φιλτραρίσματος δεδομένων και διαμόρφωσης τελικού ερευνητικού δείγματος *

Η συγκεκριμένη διαδικασία διαμόρφωσε ένα συνεκτικό cross-sectional σύνολο δεδομένων, με μία τελική παρατήρηση ανά Pull Request και διαθέσιμες μετρικές κώδικα κατάλληλες για τις στατιστικές και πολυμεταβλητές αναλύσεις που ακολουθούν.

8.1.2 Αριθμός Pull Requests και projects

Με την ολοκλήρωση των σταδίων φιλτραρίσματος, το τελικό σύνολο δεδομένων που χρησιμοποιήθηκε στις κύριες αναλύσεις περιλάμβανε 3.701 Pull Requests, τα οποία προέρχονταν από 242 διαφορετικά περιβάλλοντα projects. Κάθε Pull Request αντιπροσωπεύεται από μία τελική παρατήρηση (pr_closed), με αποτέλεσμα το δείγμα να περιλαμβάνει μία και μοναδική εγγραφή για κάθε μονάδα ανάλυσης -ως μονάδα ανάλυσης στην παρούσα ερευνητική εργασία έχει οριστεί η μονάδα Pull Request-.

Η παρουσία Pull Requests από μεγάλο αριθμό διαφορετικών projects παρέχει σημαντική ετερογένεια ως προς τα συστήματα λογισμικού που εκπροσωπούνται στο δείγμα. Παράλληλα, η ύπαρξη πολλαπλών Pull Requests μέσα στο ίδιο project δημιουργεί την ιεραρχική δομή των δεδομένων που εξετάζεται στις επόμενες αναλύσεις, καθώς οι παρατηρήσεις που προέρχονται από το ίδιο project ενδέχεται να μοιράζονται κοινά χαρακτηριστικά.

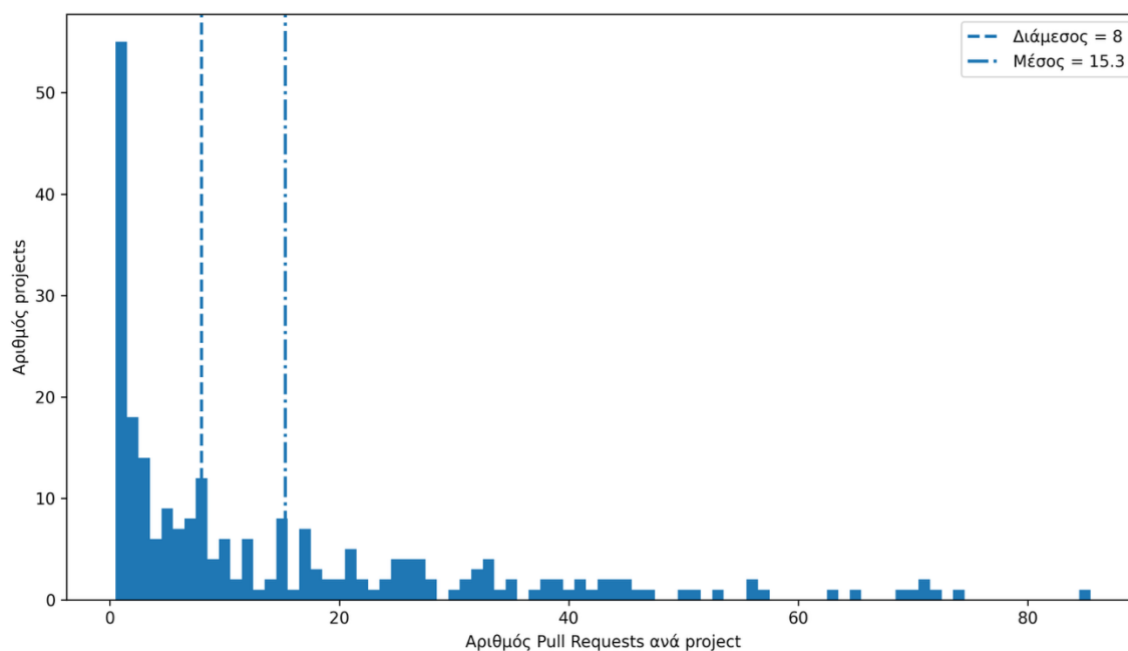
Σε σχέση με τα 353 projects του αρχικού συνόλου δεδομένων, μετά την εφαρμογή των κριτηρίων επιλογής παρέμειναν 242 projects, γεγονός που αντανάκλα την επίδραση του καθαρισμού και κυρίως του αποκλεισμού παρατηρήσεων χωρίς

αναλύσιμο πηγαίο κώδικα. Το τελικό δείγμα των 3.701 Pull Requests και 242 projects αποτέλεσε τη βάση για τις περιγραφικές, παλινδρομικές και πολυμεταβλητές αναλύσεις της παρούσας έρευνας.

8.1.3 PRs ανά project

Η κατανομή των 3.701 Pull Requests στα 242 projects δεν ήταν ομοιόμορφη. Ο μέσος αριθμός Pull Requests ανά project ήταν ~15,3 ενώ η διάμεσος ήταν μόλις 8 PRs ανά project. Ο αριθμός των παρατηρήσεων ανά project κυμαινόταν από 1 έως και 85 Pull Requests.

Η σημαντική διαφορά μεταξύ του μέσου και διάμεσου, σε συνδυασμό με το εύρος των παρατηρήσεων, υποδεικνύει μία δεξιά ασύμμετρη κατανομή ως προς το μέγεθος των project-level clusters. Πολλά projects εκπροσωπούνται από σχετικά μικρό αριθμό Pull Requests, ενώ ένας μικρότερος αριθμός projects συγκεντρώνει σημαντικά περισσότερες παρατηρήσεις.

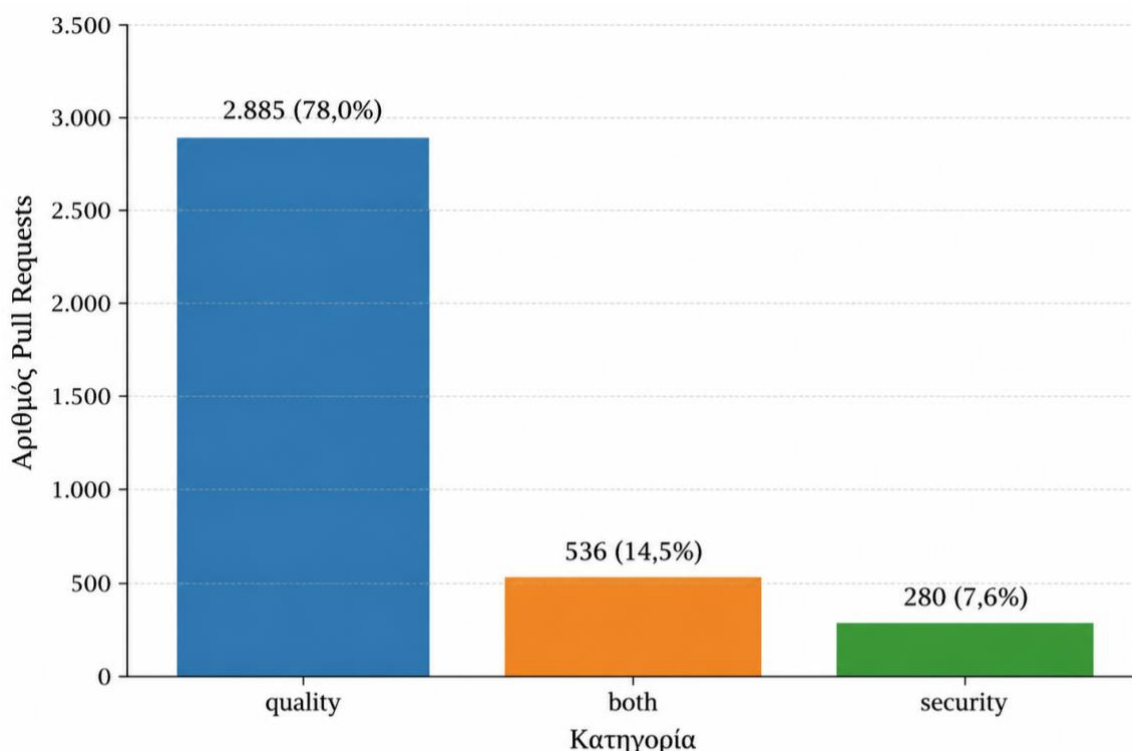


Εικόνα 8.1.2 – Κατανομή του αριθμού των Pull Requests ανά project στο τελικό ερευνητικό δείγμα

Η συγκεκριμένη ανομοιογένεια είναι σημαντική για την ερμηνεία των αποτελεσμάτων, καθώς τα Pull Requests δεν κατανέμονται ισόποσα μεταξύ των projects. Παράλληλα, επιβεβαιώνει την πολυεπίπεδη φύση του δείγματος δεδομένων, όπου οι επιμέρους παρατηρήσεις είναι οργανωμένες μέσα σε clusters διαφορετικού μεγέθους. Για τον λόγο αυτό, η σταθερότητα των κύριων αποτελεσμάτων εξετάστηκε επιπλέον σε sensitivity analysis που περιορίστηκε σε projects με τουλάχιστον πέντε (5) Pull Requests.

8.1.4 Κατηγορίες των Pull Requests

Το τελικό δείγμα που περιέχει 3.701 Pull Requests ταξινομήθηκε σε τρεις (3) ενοποιημένες κατηγορίες: quality, security και both. Οι τρεις αυτές κατηγορίες αποτελούν ενοποίηση της αρχικής PR-level ταξινόμησης code_quality, software_quality, code_security και software_security, όπως περιγράφεται και στην §6.6.1 και δεν κατασκευάστηκαν από την παρουσία ή την απουσία SonarQube findings στο τελικό snapshot. Η κατηγορία quality περιλάμβανε 2.885 Pull Requests, δηλαδή περίπου το 78,0% του συνολικού δείγματος. Στην κατηγορία both εντάχθηκαν 536 Pull Requests (14,5%), ενώ η κατηγορία security περιλάμβανε 280 Pull Requests (7,6%).



Εικόνα 8.1.3 – Κατανομή των Pull Requests του τελικού δείγματος στις κατηγορίες quality, security, both *

Η κατανομή αυτή δείχνει ότι το δείγμα είναι σαφώς μη ισορροπημένο ως προς τις κατηγορίες, με την πλειονότητα των Pull Requests να αφορά ζητήματα που είχαν ταξινομηθεί αποκλειστικά ως σχετιζόμενα με την ποιότητα του λογισμικού. Τα Pull Requests, από την άλλη πλευρά, τα οποία σχετίζονταν αποκλειστικά με την ασφάλεια αποτελούσαν το μικρότερο τμήμα του δείγματος, ενώ η ενδιάμεση κατηγορία both συγκέντρωνε περιπτώσεις στις οποίες συνυπήρχαν χαρακτηριστικά ποιότητας και ασφάλειας.

Η ανισορροπία αυτή πρέπει να λαμβάνεται υπόψη κατά την ερμηνεία των αποτελεσμάτων που συγκρίνουν τις τρεις κατηγορίες, καθώς οι εκτιμήσεις για τις μικρότερες ομάδες βασίζονται σε λιγότερες παρατηρήσεις. Για τον λόγο αυτό, οι διαφορές μεταξύ των κατηγοριών αξιολογούνται στο πλαίσιο των αντίστοιχων

στατιστικών μοντέλων και όχι αποκλειστικά μέσω των ακατέργαστων συχνοτήτων τους.

Είναι σημαντικό να διευκρινιστεί ότι η κατηγορία του Pull Request και οι μετρικές που καταγράφηκαν από το SonarQube αποτελούν δύο διαφορετικές αναλυτικές έννοιες. Η Category αποτυπώνει τον τεχνικό προσανατολισμό που αποδόθηκε στην εξεταζόμενη αλλαγή κατά το στάδιο της ταξινόμησης, ενώ τα `softwareQualityMaintainabilityIssues` και `softwareQualitySecurityIssues` αποτυπώνουν findings που παρατηρήθηκαν στο αναλυόμενο code snapshot. Επομένως, η ταξινόμηση ενός Pull Request ως security δεν προϋποθέτει την παρουσία security issues στο αντίστοιχο SonarQube snapshot και, αντίστοιχα, η ταξινόμηση ως quality δεν συνεπάγεται ότι τα παρατηρούμενα findings περιορίζονται στη διάσταση της συντηρησιμότητας.

Η διάκριση αυτή εμφανίζεται και στα περιγραφικά αποτελέσματα. Η διάμεσος του `security_density` εμφανίστηκε μηδενική και στις τρεις κατηγορίες, ενώ θετική διάμεσος `quality_density` παρατηρήθηκε ακόμη και στα Pull Requests που είχαν ταξινομηθεί ως security. Συγκεκριμένα, οι διάμεσες τιμές `quality_density` ήταν περίπου 9,45 issues/KLOC για την κατηγορία quality, 8,05 για την both και 4,69 για την security. Τα αποτελέσματα αυτά δείχνουν ότι η κατηγορία του Pull Request δεν αποτελεί άμεσο δείκτη παρουσίας ή απουσίας συγκεκριμένου τύπου SonarQube finding, αλλά διαφορετικό χαρακτηριστικό της εξεταζόμενης αλλαγής.

8.2 Περιγραφικά αποτελέσματα

Η περιγραφική ανάλυση αποτελεί το πρώτο στάδιο παρουσίασης των εμπειρικών ευρημάτων και ως στόχο έχει την αποτύπωση των βασικών χαρακτηριστικών των μετρικών που χρησιμοποιήθηκαν στην έρευνα. Μέσα από τη μελέτη των κατανομών, των κεντρικών τάσεων και της μεταβλητότητας των δεδομένων διαμορφώνεται μία αρχική εικόνα για τη συμπεριφορά των μετρικών ποιότητας και ασφάλειας πριν από την εφαρμογή των παλινδρομικών και πολυμεταβλητών μοντέλων.

Στην παρούσα ενότητα της βιβλιογραφίας παρουσιάζονται αρχικά τα περιγραφικά αποτελέσματα των μετρικών ποιότητας και ασφάλειας, με στόχο να εξεταστούν η συχνότητα, η διασπορά και η γενικότερη κατανομή τους στο τελικό δείγμα. Στη συνέχεια εξετάζεται η σχέση μεταξύ των μετρήσεων `pr_base` και `pr_closed`, η οποία αποτέλεσε σημαντικό στοιχείο για τη διαμόρφωση της τελικής αναλυτικής στρατηγικής. Τέλος, παρουσιάζεται η μεταβλητότητα σε επίπεδο project, με στόχο να διερευνηθεί κατά πόσο οι παρατηρήσεις διαφοροποιούνται συστηματικά μεταξύ διαφορετικών έργων λογισμικού.

Τα αποτελέσματα αναδεικνύουν τη δομή, την ετερογένεια και τα βασικά πρότυπα που χαρακτηρίζουν το ερευνητικό δείγμα, καθώς λειτουργούν ως βάση για την ερμηνεία επόμενων στατιστικών αναλύσεων.

8.2.1 Quality metrics

Οι μετρικές που σχετίζονται με την ποιότητα του λογισμικού παρουσίασαν έντονη ετερογένεια στο τελικό δείγμα, με σημαντικές διαφορές μεταξύ των Pull Requests τόσο ως προς τον αριθμό των καταγεγραμμένων issues όσο και ως προς την πολυπλοκότητα και την επανάληψη του κώδικα. Χαρακτηριστικό των περισσότερων μεταβλητών ήταν η δεξιά ασυμμετρία των κατανομών, καθώς η πλειονότητα των Pull Requests εμφάνιζε σχετικά χαμηλές τιμές, ενώ ένα μικρότερο σύνολο παρατηρήσεων συγκέντρωνε ιδιαίτερα υψηλές τιμές.

Τα software quality maintainability issues παρουσίασαν διάμεσο 12 issues, ενώ ο μέσος όρος ήταν σημαντικά υψηλότερος, περίπου 1.288,2 issues. Το 95^ο εκατοστημόριο ανερχόταν σε 3.256 issues, ενώ περίπου το 22,5% των PRs δεν εμφάνιζε πλήθος maintainability issues διαφορετικό του μηδενός. Παρόμοια εικόνα παρουσίασαν τα code Smells, με διάμεσο 13, μέσο όρο περίπου 1.292,5 και 95^ο εκατοστημόριο 3.319. Η μεγαλύτερη απόσταση μεταξύ μέσου και διαμέσου καταδεικνύει την επίδραση ενός σχετικά μικρού αριθμού Pull Requests με πολύ υψηλές τιμές.

Στις μετρικές αξιοπιστίας παρατηρήθηκε ακόμα μεγαλύτερη συγκέντρωση μηδενικών τιμών. Τα software quality reliability issues είχαν διάμεσο 0, ενώ περίπου το 56,6% των Pull Requests δεν παρουσίαζε κανένα σχετικό issue. Ο μέσος όρος εδώ ήταν περίπου 269,7 issues και το 95^ο εκατοστημόριο 535. Η μετρική bugs, αντίστοιχα, παρουσίασε επίσης διάμεσο 0, με περίπου 63,8% των παρατηρήσεων να έχουν μηδενική τιμή. Ο μέσος αριθμός bugs ήταν 43,0, ενώ το 95^ο εκατοστημόριο έφθανε τα 302. Τα αποτελέσματα αυτά αποτυπώνουν μία κατανομή στην οποία τα προβλήματα αξιοπιστίας δεν εμφανίζονται ομοιόμορφα στο σύνολο των Pull Requests.

Η duplicated lines density παρουσίασε έντονη συγκέντρωση στις χαμηλές τιμές. Η διάμεσος ήταν 0%, ενώ ο μέσος όρος ήταν ~4,57% και το 95^ο εκατοστημόριο 28,5%. Το ~61,8% των PRs είχε λάβει μηδενική τιμή στη συγκεκριμένη μετρική. Η επανάληψη, επομένως, πηγαίου κώδικα ήταν περιορισμένη ή μη ανιχνεύσιμη σε μεγάλο μέρος του δείγματος, αλλά σε ορισμένες παρατηρήσεις έφθανε σε αισθητά υψηλότερα επίπεδα.

Μεγάλη διακύμανση παρατηρήθηκε επίσης και στις μετρικές πολυπλοκότητας. Η complexity είχε διάμεσο 50, αλλά μέσο όρο περίπου 2.184,9 και 95^ο εκατοστημόριο 7.903. Αντίστοιχα, η cognitive Complexity παρουσίασε διάμεσο 19, μέσο όρο ~2.226,4 και 95^ο εκατοστημόριο 9.321. Οι μεγάλες αποκλίσεις μεταξύ μέσων και διαμέσων τιμών συνδέονται με την παρουσία Pull Requests που αφορούν σημαντικά μεγαλύτερα και πιο σύνθετα τμήματα πηγαίου κώδικα από την πλειονότητα του δείγματος.

Για τις κύριες στατιστικές αναλύσεις, ιδιαίτερη σημασία επίσης είχε η πυκνότητα των maintainability issues ανά μονάδα KLOC (quality_density). Η συγκεκριμένη μεταβλητή είχε διάμεσο περίπου 9,02 issues ανά KLOC, μέσο όρο 28,11 και 95^ο εκατοστημόριο περίπου 106,43. Το ~22,5% των Pull Requests είχε μηδενική πυκνότητα. Η ισχυρή δεξιά ασυμμετρία της αρχικής μεταβλητής αποτέλεσε

έναν από τους λόγους για τους οποίους στις παλινδρομικές αναλύσεις χρησιμοποιήθηκε η λογαριθμικά μετασχηματισμένη μορφή $\log(1 + \text{quality_density})$.

Η μεταβλητή coverage, τελικά, δεν παρείχε ουσιαστική διαφοροποίηση στο συγκεκριμένο σύνολο δεδομένων. Από τις 2.280 διαθέσιμες τιμές, όλες ήταν ίσες με μηδέν, ενώ για 1.421 Pull Requests η εξαγόμενη τιμή κρίθηκε ελλιπής. Κατά συνέπεια, η συγκεκριμένη μετρική δεν συνέβαλε ουσιαστικά στη διαφοροποίηση των μονάδων Pull Requests και δεν αποτέλεσε κεντρική μεταβλητή των επόμενων στατιστικών μοντέλων.

Μετρική	Μέσος	Διάμεσος	95° εκατοστημόριο	Μηδενικές τιμές
Software Quality Maintainability Issues	1.288,21	12	3.256	22,48%
Code Smells	1.292,53	13	3.319	22,45%
Software Quality Reliability Issues	269,65	0	535	56,55%
Bugs	43,04	0	302	63,77%
Duplicated Lines Density (%)	4,57	0	28,50	61,79%
Complexity	2.184,86	50	7.903	31,10%
Cognitive Complexity	2.226,38	19	9.321	39,04%
Quality Density (issues/KLOC)	28,11	9,02	106,43	22,48%
Coverage (%) – non relevant	0,00	0,00	0,00	100,00%

Πίνακας 8.2.1 – Βασικά περιγραφικά στατιστικά των μετρικών ποιότητας στο τελικό ερευνητικό δείγμα

Εν κατακλείδι, ως συνολική εικόνα τα περιγραφικά αποτελέσματα των quality metrics αναδεικνύουν κατανομές με έντονη ασυμμετρία, μεγάλο αριθμό μηδενικών τιμών σε ορισμένες μεταβλητές και περιορισμένο αριθμό παρατηρήσεων με εξαιρετικά υψηλές τιμές. Η εικόνα αυτή κρίνεται σημαντική για την ερμηνεία των αποτελεσμάτων που ακολουθούν, καθώς δείχνει την μη ομοιόμορφη κατανομή της ποιότητας του κώδικα μεταξύ των Pull Requests και την δυνατότητα απόκρυψης σημαντικών χαρακτηριστικών του δείγματος μέσω της χρήσης αποκλειστικά μέσω των τιμών.

8.2.2 Security metrics

Οι μετρικές οι οποίες σχετίζονται με την ασφάλεια παρουσίασαν ακόμη εντονότερη συγκέντρωση στις χαμηλές και μηδενικές τιμές σε σχέση με αρκετές από τις αντίστοιχες μετρικές ποιότητας. Στο μεγαλύτερο μέρος του δείγματος (dataset) των Pull Requests δεν καταγράφηκαν ζητήματα ασφάλειας, ενώ ένα μικρότερο υποσύνολο συγκέντρωνε αισθητά υψηλότερες τιμές. Ως αποτέλεσμα, οι κατανομές των security metrics χαρακτηρίζονται από έντονη δεξιά ασυμμετρία και υψηλό ποσοστό μηδενικών παρατηρήσεων.

Βασική μεταβλητή για την αποτύπωση των ζητημάτων αποτέλεσαν τα software quality security issues (softwareQualitySecurityIssues). Αυτά είχαν διάμεσο 0 και μέσο όρο περίπου 1,55 issues ανά Pull Request. Το 95° εκατοστημόριο ήταν 8 issues, ενώ 2.900 από τα 3.701 (78,4%) δεν εμφάνιζαν κανένα security issue. Αντίθετα, μόνο 801 Pull Requests παρουσίαζαν τουλάχιστον ένα σχετικό issue. Η μέγιστη τιμή στο δείγμα έφθανε τα 135 issues, γεγονός το οποίο και ενισχύει την εικόνα μίας κατανομής όπου σχετικά λίγες παρατηρήσεις συγκεντρώνουν μεγάλο μέρος των προβλημάτων ασφάλειας.

Ανάλογη συμπεριφορά παρατηρήθηκε και σε αυτή των vulnerabilities. Η διάμεσος ήταν επίσης 0, ο μέρος αριθμός ήταν ~1,23 ενώ το 95° εκατοστημόριο ήταν 6. Συνολικά, 80,98% των Pull Requests δεν περιείχαν καταγεγραμμένη vulnerability, με μόνο 704 παρατηρήσεις να εμφανίζουν θετική τιμή. Η μέγιστη τιμή ανερχόταν σε 135 vulnerabilities.

Τα Security Hotspots (securityHotspots) είχαν πιο συχνή εμφάνιση σε σχέση με τις δύο προηγούμενες μετρικές, χωρίς όμως να παύουν να παρουσιάζουν σημαντική συγκέντρωση στο μηδέν. Η διάμεσος παρέμεινε 0, αλλά ο μέσος όρος ήταν περίπου 17,48 hotspots και το 95° εκατοστημόριο 40. Περίπου το 54,4% των Pull Requests δεν εμφάνιζαν κανένα security hotspot, ενώ η μέγιστη παρατηρούμενη τιμή έφθανε τα 1.733 hotspots. Η μεγάλη απόσταση μεταξύ της διαμέσου και του μέσου όρου αποτυπώνει την επίδραση ενός περιορισμένου αριθμού Pull Requests με ιδιαίτερα υψηλές τιμές.

Για τις κύριες αναλύσεις ιδιαίτερη σημασία είχε η πυκνότητα των security issues ανά KLOC (security_density), ώστε τα καταγεγραμμένα issues να εξετάζονται σε σχέση με το μέγεθος του αναλυόμενου κώδικα. Η διάμεσος αυτής της μεταβλητής ήταν 0 issues ανά KLOC, ο μέρος όρος περίπου 0,337 issues ανά KLOC και το 95° εκατοστημόριο περίπου 0,749 issues ανά KLOC. Το 78,4% των παρατηρήσεων είχε μηδενική πυκνότητα, ενώ σε ορισμένες περιπτώσεις παρατηρήθηκαν σημαντικά υψηλότερες τιμές, με μέγιστη τιμή ~61,22 issues ανά KLOC.

Μετρική	Μέσος	Διάμεσος	95° εκατοστημόριο	Μηδενικές τιμές
Software Quality Security Issues	1,55	0	8	78,36%
Vulnerabilities	1,23	0	6	80,98%
Security Hotspots	17,48	0	40	54,36%
Security Density (issues/KLOC)	0,337	0	0,749	78,36%

Πίνακας 8.2.2 – Βασικά περιγραφικά στατιστικά των μετρικών ασφάλειας στο τελικό ερευνητικό δείγμα

Η έντονη συγκέντρωση των security metrics στο μηδέν αποτελεί σημαντικό χαρακτηριστικό του ερευνητικού δείγματος. Σε αντίθεση με ορισμένες μετρικές ποιότητας, όπου τα issues εμφανίζονται σε μεγαλύτερο μέρος των PRs, τα ζητήματα ασφάλειας παρουσιάζονται περισσότερο σποραδικά. Όταν, όμως, εμφανίζονται μπορούν να παρουσιάσουν σημαντική διακύμανση ως προς την έκτασή τους. Το χαρακτηριστικό αυτό πρέπει να λαμβάνεται υπόψη κατά την ερμηνεία των

μεταγενέστερων αποτελεσμάτων και αποτελεί έναν από τους λόγους για τους οποίους τα κύρια LMM συμπληρώθηκαν από count-based robustness analyses.

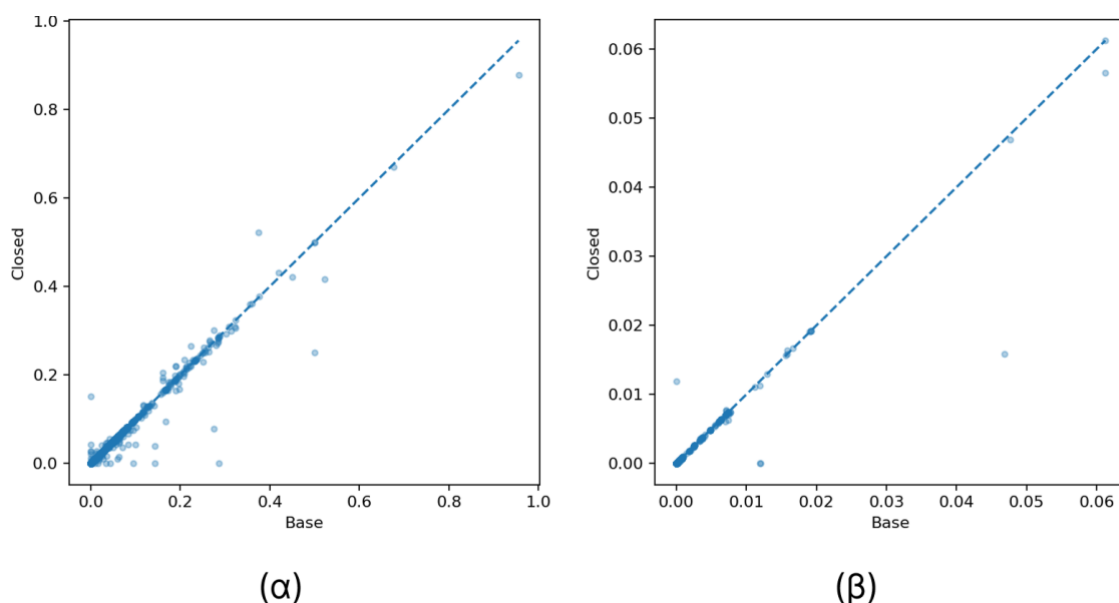
Η περιγραφική εικόνα δείχνει ότι τα security-related προβλήματα δεν κατανέμονται ομοιόμορφα στο σύνολο του δείγματος. Οι μηδενικές τιμές κυριαρχούν, ενώ παράλληλα υφίσταται μία μικρότερη ομάδα Pull Requests με αισθητά υψηλότερες τιμές vulnerabilities, security issues ή hotspots. Η συγκεκριμένη δομή των δεδομένων εξηγεί και την ανάγκη προσεκτικής ερμηνείας των μέσων τιμών, καθώς αυτές επηρεάζονται σημαντικά από τις υψηλές παρατηρήσεις της δεξιάς ουράς των κατανομών.

8.2.3 Base έναντι closed

Η αρχική σχεδίαση της έρευνας βασίστηκε στη σύγκριση δύο χρονικών καταστάσεων για κάθε Pull Request, του pr_base και του pr_closed. Το pr_base αντιπροσώπευε την κατάσταση του κώδικα πριν από την ενσωμάτωση των αλλαγών του Pull Request, ενώ το pr_closed αντιπροσώπευε αντίστοιχα την τελική κατάσταση μετά την ολοκλήρωσή του. Μετά τον έλεγχο εγκυρότητας των δεδομένων ήταν διαθέσιμα 9.813 έγκυρα ζεύγη base-closed Pull Requests, παρέχοντας αρχικά τη δυνατότητα εξέτασης της μεταβολής των μετρικών μεταξύ των δύο καταστάσεων.

Παρόλα αυτά, η περιγραφική σύγκριση έδειξε ότι οι τιμές των περισσότερων μετρικών παρουσίαζαν εξαιρετικά περιορισμένη μεταβολή μεταξύ pr_base και pr_closed. Ανάλογα με τη μετρική, περίπου 94% έως 99,9% των Pull Requests εμφάνιζαν μηδενική μεταβολή. Η εικόνα αυτή ήταν ιδιαίτερα έντονη στις μεταβλητές που σχετίζονταν άμεσα με τα software quality και security issues, όπου οι παρατηρήσεις στις δύο καταστάσεις ήταν στη μεγάλη πλειονότητά τους ταυτόσημες.

Η υψηλή ομοιότητα γίνεται εμφανής και από τη γραφική αντιπαραβολή των δύο snapshots. Στα διαγράμματα base έναντι closed, οι παρατηρήσεις συγκεντρώνονται σε μεγάλο βαθμό πάνω ή πολύ κοντά στη διαγώνιο ισότητας, γεγονός που σημαίνει ότι η τιμή της αντίστοιχης μετρικής στο pr_closed ήταν στις περισσότερες εξεταζόμενες περιπτώσεις -ζευγάρια pr_base και pr_closed- ίδια με την τιμή της στο pr_base. Αντίστοιχα, οι κατανομές των διαφορών μεταξύ των δύο καταστάσεων παρουσιάζουν έντονη συγκέντρωση γύρω από το μηδέν.



Εικόνα 8.2.1 – Σύγκριση των τιμών `pr_base` και `pr_closed` για τις κανονικοποιημένες μετρικές `maintainability` και `security issues`

Το αποτέλεσμα αυτό έχει ιδιαίτερη σημασία για την ερμηνεία της έρευνας. Μία before-after προσέγγιση προϋποθέτει ότι υπάρχει επαρκής μεταβλητότητα στις διαφορές μεταξύ των δύο χρονικών σημείων, προκειμένου να καθίσταται δυνατή η διερεύνηση συστηματικών αυξήσεων ή μειώσεων. Στο συγκεκριμένο σύνολο δεδομένων, η σχεδόν καθολική απουσία μεταβολής θα οδηγούσε σε *delta variables* με πολύ περιορισμένη πληροφορία και κατανομές που κυριαρχούνται από μηδενικές τιμές.

Λόγω των ανωτέρω, τα `pr_base` και `pr_closed` δεν χρησιμοποιήθηκαν τελικά ως δύο διαδοχικές παρατηρήσεις σε κύρια longitudinal ή before-after ανάλυση. Αντίθετα, ως κύρια μονάδα ανάλυσης διατηρήθηκε το τελικό `pr_closed` snapshot κάθε μονάδας Pull Request, με στόχο οι επόμενες αναλύσεις να επικεντρωθούν στις διαφορές που παρατηρούνται μεταξύ Pull Requests και projects και όχι σε μεταβολές οι οποίες, στο διαθέσιμο σύνολο δεδομένων, ήταν στις περισσότερες περιπτώσεις μηδενικές.

Η σύγκριση άρα `base` και `closed` δεν αποτελεί απλώς ένα προκαταρκτικό περιγραφικό εύρημα, αλλά εξηγεί μία από τις σημαντικότερες αποφάσεις της τελικής αναλυτικής στρατηγικής. Τα δεδομένα δεν παρείχαν επαρκή εμπειρική βάση για αξιόπιστη ανάλυση των μεταβολών πριν και μετά το PR, με αποτέλεσμα η έρευνα να προσανατολιστεί σε cross-sectional ανάλυση της τελικής κατάστασης των Pull Requests.

Συνεπώς, τα findings που παρατηρούνται στο τελικό `pr_closed` snapshot ερμηνεύονται ως χαρακτηριστικά της τελικής κατάστασης του αναλυόμενου πηγαίου κώδικα και όχι ως προβλήματα που η εισαγωγή τους από το αντίστοιχο Pull Request έχει αποδειχθεί. Η τελική ανάλυση εξετάζει, άρα, συσχετίσεις με τα επίπεδα και τις πυκνότητες των `quality` και `security issues` και δεν επιχειρεί να αποδώσει την εμφάνιση συγκεκριμένων findings στην ίδια την αλλαγή.

8.2.4 Project-level variability

Εκτός από τη μεταβλητότητα η οποία παρατηρήθηκε μεταξύ των επιμέρους Pull Requests, τα περιγραφικά αποτελέσματα ανέδειξαν σημαντικές διαφοροποιήσεις και μεταξύ των projects στα οποία αυτά ανήκαν. Τα Pull Requests του ίδιου project δεν αποτελούν πλήρως ανεξάρτητες παρατηρήσεις, καθώς αναπτύσσονται μέσα σε κοινό τεχνικό και οργανωτικό περιβάλλον και μπορούν να μοιράζονται χαρακτηριστικά που σχετίζονται με τη δομή, την πολυπλοκότητα και τις πρακτικές ανάπτυξης του εκάστοτε συστήματος.

Μία πρώτη ποσοτική ένδειξη της συγκεκριμένης διαφοροποίησης προέκυψε από τον προκαταρκτικό έλεγχο της `log_quality_density`. Ο λόγος του between-group sum of squares προς το συνολικό sum of squares (between-group SS / total SS) ήταν ~86,6%, υποδεικνύοντας ότι οι μισές τιμές της πυκνότητας προβλημάτων ποιότητας διαφοροποιούνται έντονα μεταξύ των projects. Η τιμή αυτή δεν ταυτίζεται με τον Intraclass Correlation Coefficient (ICC), ούτε αποτελεί εκτίμηση variance component από mixed-effects model.

Για τον λόγο αυτό, το 86,6% χρησιμοποιείται αποκλειστικά ως προκαταρκτικό exploratory diagnostic της ισχυρής ομαδοποίησης των δεδομένων. Η επίσημη ποσοτικοποίηση της project-level συσχέτισης πραγματοποιείται στη συνέχεια μέσω των ICC που προκύπτουν από τα null random-intercept LMM.

Το μέγεθος του προκαταρκτικού δείκτη αποτελεί ισχυρή ένδειξη ουσιαστικής μεταξύ-project ετερογένειας και υποστηρίζει την ανάγκη να εξεταστεί ρητά η ιεραρχική δομή των δεδομένων με κατάλληλο mixed-effects model. Αναλυτικότερα, Pull Requests τα οποία ανήκουν στο ίδιο project τείνουν να παρουσιάζουν περισσότερο παρόμοια χαρακτηριστικά μεταξύ τους σε σχέση με Pull Requests που προέρχονται από διαφορετικά projects. Κατά συνέπεια, ένα σημαντικό μέρος της ετερογένειας του δείγματος δεν εντοπίζεται αποκλειστικά στο επίπεδο του μεμονωμένου Pull Request, αλλά στο ευρύτερο περιβάλλον του project.

Η εικόνα αυτή έχει ιδιαίτερη σημασία, καθώς δείχνει ότι οι διαφορές στις μετρικές δεν μπορούν να ερμηνευθούν αποκλειστικά μέσα από χαρακτηριστικά όπως το μέγεθος ή η πολυπλοκότητα ενός Pull Request. Παράγοντες που λειτουργούν σε επίπεδο project, όπως η αρχιτεκτονική του συστήματος, οι πρακτικές ανάπτυξης, οι τεχνολογικές επιλογές και η συνολική κατάσταση της βάσης κώδικα, μπορούν επίσης να συνδέονται με το επίπεδο ποιότητας και ασφάλειας που παρατηρείται στις επιμέρους αλλαγές.

Η έντονη project-level μεταβλητότητα αποτέλεσε επομένως ένα από τα βασικά εμπειρικά ευρήματα της περιγραφικής διερεύνησης και παρείχε ισχυρή ένδειξη ότι η ιεραρχική δομή των δεδομένων έπρεπε να ληφθεί υπόψη σε επόμενες αναλύσεις. Η έκταση της συγκεκριμένης επίδρασης εξετάζεται αναλυτικότερα μέσω των Linear Mixed Models, όπου η μεταξύ-project καθώς και η εντός-project διακύμανση εκτιμώνται χωριστά και ποσοτικοποιούνται μέσω της χρήσης του formal ICC.

8.3 Exploratory analysis και συσχετίσεις

Στόχος της Exploratory Analysis ήταν η αρχική διερεύνηση των σχέσεων μεταξύ των βασικών μετρικών ποιότητας, ασφάλειας, μεγέθους και πολυπλοκότητας του κώδικα, πριν από την εφαρμογή των παλινδρομικών και πολυεπίπεδων μοντέλων. Η εξέταση των συσχετίσεων επιτρέπει να εντοπιστούν μεταβλητές οι οποίες τείνουν να μεταβάλλονται από κοινού, καθώς και ευρύτερα πρότυπα σύνδεσης μεταξύ των διαφορετικών διαστάσεων του λογισμικού.

Οι σχέσεις μεταξύ των quality και security μετρικών παρουσιάζουν ιδιαίτερο ενδιαφέρον. Αυτό συμβαίνει διότι αποτελούν άμεσο εμπειρικό σημείο αναφοράς για το βασικό ερευνητικό αντικείμενο της εργασίας. Παράλληλα, εξετάζονται οι συσχετίσεις των συγκεκριμένων μετρικών με το μέγεθος και την πολυπλοκότητα του πηγαίου κώδικα, δεδομένου ότι οι μεταβλητές όπως τα ncloc, complexity και cognitiveComplexity μπορούν να συνδέονται ταυτόχρονα με περισσότερους από έναν δείκτες ποιότητας ή ασφάλειας.

Η exploratory διερεύνηση είναι επίσης σημαντική για την αναγνώριση ισχυρών συσχετίσεων μεταξύ ανεξάρτητων μεταβλητών. Τέτοιες σχέσεις μπορούν να υποδεικνύουν ότι διαφορετικές μετρικές αποτυπώνουν εν μέρει κοινές ιδιότητες του κώδικα και παρέχουν χρήσιμη πληροφορία για την ερμηνεία των μεταγενέστερων αποτελεσμάτων της OLS, των LMM, της PCA και της Factor Analysis.

Οι συσχετίσεις οι οποίες δημιουργούνται και παρατηρούνται στο συγκεκριμένο στάδιο ερμηνεύονται αποκλειστικά ως στατιστικές σχέσεις και όχι ως ενδείξεις αιτιότητας. Επιπλέον, λόγω του ότι οι παρατηρήσεις προέρχονται από διαφορετικά projects και παρουσιάζουν έντονη project-level ομαδοποίηση, μία συνολική συσχέτιση σε επίπεδο PR δεν διαχωρίζει τις σχέσεις που εμφανίζονται εντός ενός project από εκείνες που προκύπτουν λόγω διαφορών μεταξύ projects. Τα αποτελέσματα, έτσι, της exploratory analysis χρησιμοποιούνται κυρίως για την αναγνώριση προτύπων και συμπληρώνονται στη συνέχεια από τα μοντέλα που λαμβάνουν ρητά υπόψη την ιεραρχική δομή των δεδομένων.

Συνοψίζοντας, η ανάλυση συσχετίσεων λειτουργεί ως συνδετικός κρίκος μεταξύ της περιγραφικής παρουσίασης των δεδομένων και της επαγωγικής στατιστικής ανάλυσης που ακολουθεί. Προσφέρεται έτσι μία πρώτη εικόνα για το κατά πόσο οι διαφορετικές διαστάσεις ποιότητας και ασφάλειας εμφανίζονται ανεξάρτητες ή παρουσιάζουν κοινά πρότυπα μεταβολής.

8.4 Αποτελέσματα OLS

Η στατιστική τεχνική OLS χρησιμοποιήθηκε ως αρχικό σημείο αναφοράς για τη διερεύνηση των σχέσεων μεταξύ του μεγέθους και της πολυπλοκότητας του κώδικα, της κατηγορίας του Pull Request και των πυκνοτήτων προβλημάτων ποιότητας και ασφάλειας. Τα μοντέλα εκτιμήθηκαν με βάση το τελικό δείγμα των 3.701 Pull Requests -έπειτα από την διαδικασία του φιλτραρίσματος-, με cluster-robust standard errors σε επίπεδο project. Ως κατηγορία αναφοράς χρησιμοποιήθηκε η quality, ενώ εξετάστηκαν ξεχωριστά τα outcomes log_quality_density και log_security_density.

Για την πυκνότητα προβλημάτων ποιότητας, το συνολικό OLS μοντέλο κρίθηκε στατιστικά σημαντικό ($F = 23,32$, $p < 0,001$) και παρουσίασε $R^2 = 0,339$ και adjusted $R^2 = 0,388$. Οι μεταβλητές του μοντέλου εξηγούσαν περίπου το 33,9% της συνολικής διακύμανσης της λογαριθμικά μετασχηματισμένης πυκνότητας quality issues.

Η λογαριθμική μεταβλητή log_complexity παρουσίασε τη σαφέστερη θετική σχέση με το outcome. Ο εκτιμημένος συντελεστής ήταν $\beta = 0,368$, με $p < 0,001$ και 95% διάστημα εμπιστοσύνης $[0,278, 0,458]$. Αποτελεσματικά, υψηλότερη πολυπλοκότητα συνδεόταν με υψηλότερη πυκνότητα προβλημάτων ποιότητας, όταν οι υπόλοιπες μεταβλητές του μοντέλου παρέμεναν σταθερές.

Αντίθετα, η log_ncloc παρουσίασε αρνητική και στατιστικά σημαντική σχέση με την πυκνότητα quality issues ($\beta = -0,169$, $p = 0,021$, 95% CI = $[-0,313, -0,025]$). Το αποτέλεσμα υποδεικνύει ότι, αφού παρέλθει ο έλεγχος της πολυπλοκότητας και της κατηγορίας του Pull Request, μεγαλύτερο έκταση του αναλυόμενου τμήματος κώδικα, όπως αυτή αποτυπώνεται από το ncloc, συνδεόταν με χαμηλότερη τιμή της πυκνότητας προβλημάτων ποιότητας. Η συγκεκριμένη αρνητική σχέση πρέπει να ερμηνεύεται με πρόσθετη επιφύλαξη, επειδή το ncloc συμμετέχει τόσο ως predictor μέσω του log_ncloc όσο και στον παρονομαστή της εξαρτημένης μεταβλητής quality_density. Μέρος της παρατηρούμενης αρνητικής συσχέτισης μπορεί να σχετίζεται με τη μαθηματική κατάσταση του ratio και δεν πρέπει να ερμηνεύεται αυτομάτως ως ένδειξη ότι μεγαλύτερα codebases διαθέτουν εγγενώς καλύτερη ποιότητα.

Οι κατηγορίες των Pull Requests δεν παρουσίασαν ουσιαστικές διαφοροποιήσεις στο συγκεκριμένο μοντέλο. Σε σύγκριση με την κατηγορία quality, η κατηγορία both είχε συντελεστή $\beta = -0,004$, $p = 0,968$, ενώ η κατηγορία security είχε $\beta = 0,008$, $p = 0,933$. Και στις δύο περιπτώσεις τα 95% διαστήματα εμπιστοσύνης περιλάμβαναν το μηδέν, επομένως δεν προέκυψε ένδειξη ότι η αρχική κατηγορία του Pull Request συνδεόταν ανεξάρτητα με την πυκνότητα προβλημάτων ποιότητας.

Διαφορετική εικόνα προέκυψε για την πυκνότητα προβλημάτων ασφάλειας. Το συνολικό OLS μοντέλο είχε $F = 2,887$ και $p = 0,023$, αλλά η ερμηνευτική του ικανότητα ήταν εξαιρετικά περιορισμένη, με $R^2 = 0,006$ και adjusted $R^2 = 0,005$. Συνεπώς, οι εξεταζόμενοι predictors εξηγούσαν μόλις το ~0,6% της διακύμανσης του log_security_density.

Στο security model, ούτε το μέγεθος ούτε η πολυπλοκότητα παρουσίασαν στατιστικά σημαντική σχέση με το outcome. Για τη log_ncloc εκτιμήθηκε $\beta = -0,003$ και $p = 0,595$, ενώ για τη log_complexity $\beta = -0,006$ και $p = 0,595$. Τα αντίστοιχα διαστήματα εμπιστοσύνης περιλαμβάνουν το μηδέν, παρέχοντας περιορισμένη

ένδειξη για συστηματική σχέση των συγκεκριμένων μεταβλητών με την πυκνότητα των security issues στο OLS μοντέλο.

Στατιστικά σημαντικές διαφορές δεν παρουσίασαν ούτε οι κατηγορίες των Pull Requests ως προς το log_security_density. Η κατηγορία security είχε συντελεστή $\beta = 0,015$ και $p = 0,474$, ενώ η κατηγορία both παρουσίασε θετικό συντελεστή $\beta = 0,049$, ο οποίος πλησίασε αλλά δεν κατάφερε να φθάσει το συμβατικό επίπεδο στατιστικής σημαντικότητας, ορισμένο στο $p = 0,082$. Κατά συνέπεια, τα αποτελέσματα δεν παρέχουν επαρκή βάση για να θεωρηθεί ότι η κατηγορία του Pull Request διαφοροποιεί συστηματικά την πυκνότητα των security issues.

Ούτε οι κατηγορίες των PRs παρουσίασαν στατιστικά σημαντικές διαφορές ως προς το log_security_density. Η κατηγορία security είχε συντελεστή $\beta = 0,015$ και $p = 0,474$, ενώ η κατηγορία both παρουσίασε θετικό συντελεστή $\beta = 0,049$, ο οποίος πλησίασε χωρίς να φθάσει το συμβατικό επίπεδο στατιστικής σημαντικότητας $p = 0,082$. Συνεπώς, τα αποτελέσματα δεν παρέχουν επαρκή βάση για να θεωρηθεί ότι η κατηγορία του Pull Request διαφοροποιεί συστηματικά την πυκνότητα των security issues.

Predictor	Quality: β	95% CI	p-value	Security: β	95% CI	p-value
both vs quality	-0,004	[-0,183, 0,176]	0,968	0,049	[-0,006, 0,104]	0,082
security vs quality	0,008	[-0,177, 0,193]	0,933	-0,015	[-0,056, 0,026]	0,474
log_ncloc	-0,169	[-0,313, -0,025]	0,021	-0,003	[-0,025, 0,018]	0,768
log_complexity	0,368	[0,278, 0,458]	<0,001	-0,006	[-0,027, 0,016]	0,595
R ²	0,339			0,006		
Adjusted R ²	0,338			0,005		

Πίνακας 8.4.1 – Εκτιμήσεις OLS μοντέλων για τη λογαριθμικά μετασχηματισμένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας, με cluster-robust standard errors σε επίπεδο project

Τα OLS αποτελέσματα αναδεικνύουν μία σαφώς ισχυρότερη σχέση των εξεταζόμενων χαρακτηριστικών με την ποιότητα από ό,τι με την ασφάλεια. Η πολυπλοκότητα συνδέεται θετικά και έντονα με την πυκνότητα προβλημάτων ποιότητας, ενώ το μέγεθος του κώδικα παρουσιάζει μικρότερη αλλά στατιστικά σημαντική αρνητική σχέση. Για την πυκνότητα προβλημάτων ασφάλειας, αντίθετα, το OLS μοντέλο εμφανίζει πολύ χαμηλή ερμηνευτική ικανότητα και κανένας μεμονωμένος predictor δεν παρουσιάζει στατιστικά σημαντική επίδραση στο επίπεδο $p < 0,05$.

8.5 Αποτελέσματα LMM

Η εφαρμογή των Linear Mixed-Effects Models (LMM) αποτέλεσε το κύριο στάδιο της πολυεπίπεδης στατιστικής ανάλυσης, καθώς επέτρεψε να ληφθεί υπόψη η ιεραρχική δομή του δείγματος, με τα επιμέρους PRs να βρίσκονται μέσα σε διαφορετικά projects ομαδοποιημένα. Αντίθετα με τα OLS μοντέλα, τα LMM διαχωρίζουν την επίδραση των χαρακτηριστικών του κάθε Pull Request από τη μεταβλητότητα που οφείλεται στις συστηματικές διαφορές μεταξύ των projects.

Η παρουσίαση των αποτελεσμάτων ξεκινά με τα null models και τον Intraclass Correlation Coefficient (ICC), με στόχο την πληρέστερη εκτίμηση του ποσοστού της συνολικής διακύμανσης που αποδίδεται στο επίπεδο του project. Στη συνέχεια εξετάζονται οι fixed effects, δηλαδή οι σχέσεις του μεγέθους, της πολυπλοκότητας και της κατηγορίας του Pull Request με τις πυκνότητες προβλημάτων ποιότητας και ασφάλειας. Ακολουθεί η παρουσίαση των random effects, μέσω των οποίων αποτυπώνεται η διαφοροποίηση των projects ως προς το βασικό επίπεδο των εξεταζόμενων outcomes.

Τα αποτελέσματα των LMM κρίνεται σκόπιμο να συγκριθούν με εκείνα των αντίστοιχων OLS μοντέλων, ώστε να αξιολογηθεί κατά πόσο η ρητή ενσωμάτωση της project-level μεταβλητότητας μεταβάλλει την κατεύθυνση, το μέγεθος ή τη στατιστική σημαντικότητα των εκτιμώμενων σχέσεων. Η σύγκριση αυτή είναι ιδιαίτερα σημαντική για τον προσδιορισμό του βαθμού στον οποίο τα συμπεράσματα εξαρτώνται από τη σωστή μοντελοποίηση της εξάρτησης μεταξύ Pull Requests που προέρχονται από το ίδιο project.

8.5.1 Null models και ICC

Πριν συμβεί η εισαγωγή των επιμέρους predictors στα Linear Mixed Models, εκτιμήθηκαν null models (unconditional models) για τις δύο κύριες εξαρτημένες μεταβλητές, `log_quality_density` και `log_security_density`. Τα μοντέλα αυτά περιλαμβάνουν μόνο έναν συνολικό intercept και έναν τυχαίο intercept για κάθε project και επιτρέπουν να εκτιμηθεί πόσο από τη συνολική διακύμανση των outcomes εντοπίζεται μεταξύ διαφορετικών projects και πόσο παραμένει μεταξύ Pull Requests του ίδιου project.

Για την `log_quality_density`, η εκτιμώμενη διακύμανση μεταξύ των projects ήταν 2,868, ενώ η υπολειμματική διακύμανση εντός των projects ήταν 0,397. Με βάση τις δύο αυτές συνιστώσες, ο Intraclass Correlation Coefficient (ICC) υπολογίστηκε σε:

$$ICC = \frac{2,868}{2,868+0,397} \sim 0,878$$

δηλαδή ~87,8%. Το παραπάνω αποτέλεσμα αποτυπώνει ότι το ~87,8% της συνολικής διακύμανσης της λογαριθμικά μετασχηματισμένης πυκνότητας των quality issues συνδέεται με διαφορές μεταξύ των projects, ενώ το ~12,2% αντιστοιχεί σε διαφοροποίηση μεταξύ Pull Requests μέσα στο ίδιο project.

Ακόμη υψηλότερη ομαδοποίηση παρατηρήθηκε για την log_security_density. Η project-level διακύμανση εκτιμήθηκε σε περίπου 0,155, ενώ η υπολειμματική διακύμανση υπολογίστηκε ως 0,013 περίπου. Ο αντίστοιχος ICC ήταν:

$$ICC = \frac{0,155}{0,155+0,013} \sim 0,922$$

ή περίπου 92,2%. Επομένως, περισσότερο από 9/10 (εννέα δέκατα) της συνολικής διακύμανσης της πυκνότητας των security issues εντοπίζονται στο επίπεδο του project, ενώ μόνο περίπου 7,8% αντιστοιχεί σε διαφοροποίηση μεταξύ Pull Requests του ίδιου project.

Outcome	Project variance	Residual variance	ICC	Project-level	Within-project
log_quality_density	2,868	0,397	0,878	87,8%	12,2%
log_security_density	0,155	0,013	0,922	92,2%	7,8%

Πίνακας 8.5.1 – Συνιστώσες διακύμανσης και ICC των null LMM για τις πυκνότητες προβλημάτων ποιότητας και ασφάλειας

Τα ιδιαίτερα υψηλά ICC και για τις δύο εξαρτημένες μεταβλητές αποτελούν ισχυρή ένδειξη ότι η ταυτότητα του project σχετίζεται ουσιαστικά με τα επίπεδα των quality και security issues. Pull Requests τα οποία προέρχονται από το ίδιο περιβάλλον project τείνουν να παρουσιάζουν περισσότερο όμοιες τιμές μεταξύ τους από ό, τι Pull Requests που προέρχονται από διαφορετικά projects. Η παρατήρηση αυτή επιβεβαιώνει ότι η υπόθεση πλήρους ανεξαρτησίας των παρατηρήσεων δεν ανταποκρίνεται επαρκώς στη δομή του συγκεκριμένου δείγματος.

Παράλληλα, η υψηλότερη τιμή του ICC για το security outcome υποδηλώνει ότι η πυκνότητα των προβλημάτων ασφάλειας εμφανίζει ακόμη εντονότερη project-level συγκέντρωση σε σχέση με την πυκνότητα των προβλημάτων ποιότητας. Το εύρημα δεν σημαίνει ότι το project αποτελεί αιτία των προβλημάτων ασφάλειας, αλλά ότι σημαντικό μέρος της παρατηρούμενης διαφοροποίησης σχετίζεται με χαρακτηριστικά τα οποία είναι κοινά στα Pull Requests του ίδιου project.

Συνοψίζοντας, τα αποτελέσματα που εξάγονται από τα null models παρέχουν άμεση εμπειρική αιτιολόγηση για τη χρήση των Linear Mixed-Effects Models με random intercept ανά μονάδα project (σε επίπεδο project). Η ένταξη του project ως τυχαίας επίδρασης επιτρέπει στα επόμενα μοντέλα να λαμβάνουν υπόψη αυτή την έντονη ομαδοποίηση και να εκτιμούν τις fixed effects των χαρακτηριστικών του κάθε Pull Request αφού πρώτα διαχωριστεί η μεταβλητότητα που βρίσκεται στο επίπεδο του project.

8.5.2 Fixed effects

Οι Fixed Effects των LMM αποτυπώνουν τις σχέσεις των μετρήσιμων χαρακτηριστικών της τελικής κατάστασης του αναλυόμενου κώδικα και της κατηγορίας των Pull Requests με τις δύο κύριες εξαρτημένες μεταβλητές, αφού ληφθούν υπόψη οι συστηματικές διαφορές μεταξύ των projects μέσω του random intercept. Στα δύο μοντέλα εξετάστηκαν το μέγεθος του κώδικα (`log_ncloc`), η πολυπλοκότητα (`log_complexity`) και η κατηγορία του Pull Request, με την κατηγορία `quality` να χρησιμοποιείται ως κατηγορία αναφοράς.

Για την πυκνότητα προβλημάτων ποιότητας, η πολυπλοκότητα παρουσίασε την ισχυρότερη και σαφέστερη σχέση. Ο συντελεστής της `log_complexity` ήταν θετικός και στατιστικά σημαντικός ($\beta = 0,398$ και $p < 0,001$), 95% διάστημα εμπιστοσύνης $[0,355, 0,442]$. Το αποτέλεσμα δείχνει ότι, για παρατηρήσεις με συγκρίσιμο μέγεθος αναλυόμενου κώδικα και ίδια κατηγορία Pull Request, και αφού ληφθεί υπόψη το project στο οποίο ανήκουν, υψηλότερα επίπεδα πολυπλοκότητας του τελικού snapshot συνδέονται με υψηλότερη πυκνότητα `maintainability issues`.

Κατεύθυνση αντίθετη με το παραπάνω παρουσίασε το `log_ncloc`. Ο εκτιμημένος συντελεστής ήταν $\beta = -0,152$ με $p < 0,001$ και 95% διάστημα εμπιστοσύνης $[-0,203, -0,101]$. Όταν η πολυπλοκότητα, η κατηγορία και οι διαφορές μεταξύ των projects διατηρούνται σταθερές, μεγαλύτερο μέγεθος κώδικα συνδέεται με χαμηλότερη λογισμικά μετασχηματισμένη πυκνότητα προβλημάτων ποιότητας. Η αρνητική αυτή σχέση αφορά την πυκνότητα `issues` ανά KLOC και δεν σημαίνει ότι μεγαλύτερα `code snapshots` περιέχουν κατ' ανάγκη μικρότερο απόλυτο αριθμό προβλημάτων. Επιπλέον, το `ncloc` δεν αποτελεί μέτρο του μεγέθους της ίδιας της αλλαγής του Pull Request, αλλά της έκτασης του πηγαίου κώδικα που αναλύθηκε στο αντίστοιχο snapshot.

Οι κατηγορίες των Pull Requests, αυτόνομα, δεν παρουσίασαν στατιστικά σημαντική ανεξάρτητη σχέση με την πυκνότητα των `quality issues`. Σε σύγκριση με την κατηγορία `quality`, η κατηγορία `both` είχε συντελεστή $\beta = -0,027$ και $p = 0,368$, ενώ η κατηγορία `security` είχε $\beta = -0,038$ και $p = 0,356$. Στις δύο αυτές περιπτώσεις τα διαστήματα εμπιστοσύνης περιλάμβαναν το μηδέν (0). Συνεπώς, μετά τον έλεγχο του μεγέθους της πολυπλοκότητας και της project-level ετερογένειας, δεν προέκυψε ένδειξη ότι η αρχική κατηγοριοποίηση του Pull Request διαφοροποιεί συστηματικά την πυκνότητα των προβλημάτων ποιότητας.

Στο μοντέλο για την πυκνότητα προβλημάτων ασφάλειας, τα αποτελέσματα ήταν διαφορετικά. Η `log_complexity` εμφάνισε μικρή αλλά στατιστικά σημαντική σχέση με το outcome ($\beta = 0,011$, $p = 0,017$), με 95% διάστημα εμπιστοσύνης $[-0,020, -0,002]$. Αφού ληφθούν υπόψη το μέγεθος, η κατηγορία και οι διαφορές μεταξύ projects, υψηλότερη πολυπλοκότητα συνδέθηκε με ελαφρώς χαμηλότερη πυκνότητα `security issues`. Το μέγεθος της εκτιμώμενης σχέσης είναι περιορισμένο. Ως εκ τούτου, το εύρημα πρέπει να ερμηνεύεται περισσότερο ως ένδειξη ασθενούς στατιστικής συσχέτισης παρά ως ισχυρή πρακτική επίδραση.

Το `log_ncloc` δεν παρουσίασε στατιστικά σημαντική σχέση με την πυκνότητα των προβλημάτων ασφαλείας ($\beta = -0,004$ και $p = 0,455$). Παρόμοια, δεν παρατηρήθηκαν σημαντικές μεταξύ των κατηγοριών διαφορές. Η κατηγορία `both`

είχε συντελεστή ίσο με μηδέν ($\beta \sim 0,000$ και $p = -0,991$), ενώ για την κατηγορία security εκτιμήθηκε $\beta = -0,008$ και $p = 0,320$.

Predictor	Quality β	95% CI	p-value	Security β	95% CI	p-value
both vs quality	-0,027	[-0,086, 0,032]	0,368	0,000	[-0,011, 0,011]	0,991
security vs quality	-0,038	[-0,118, 0,042]	0,356	-0,008	[-0,023, 0,007]	0,320
log_ncloc	-0,152	[-0,203, -0,101]	< 0,001	-0,004	[-0,014, 0,006]	0,455
log_complexity	0,398	[0,355, 0,442]	< 0,001	-0,011	[-0,020, -0,002]	0,017

Πίνακας 8.5.2 – Εκτιμήσεις των fixed effects των LMM για τη λογαριθμικά μετασχηματισμένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας

Τα fixed effects αναδεικνύουν διαφορετική συμπεριφορά των δύο εξεταζόμενων διαστάσεων. Για την ποιότητα, η πολυπλοκότητα αποτελεί τον ισχυρότερο θετικό predictor, ενώ το μέγεθος παρουσιάζει ανεξάρτητη αρνητική σχέση με την πυκνότητα των issues. Από την αντίθετη πλευρά, για την ασφάλεια οι fixed effects είναι σημαντικά ασθενέστερες. Το μέγεθος και οι κατηγορίες των Pull Requests δεν εμφανίζουν στατιστικά σημαντικές σχέσεις, ενώ μόνο η πολυπλοκότητα παρουσιάζει μία μικρή αρνητική συσχέτιση.

Ως ιδιαίτερα σημαντικό σημειώνεται το γεγονός ότι η κατηγορία του Pull Request δεν αποτέλεσε σημαντικό predictor σε κανένα από τα δύο κύρια LMM. Το αποτέλεσμα αυτό υποδηλώνει ότι η ταξινόμηση ενός Pull Request ως quality, security ή both δεν μεταφράζεται αυτούσια σε συστηματικά διαφορετικό επίπεδο τελικών quality ή security issues, όταν ελέγχονται τα υπόλοιπα χαρακτηριστικά του Pull Request και το project στο οποίο και ανήκει.

8.5.3 Random effects

Τα random effects των Linear Mixed Models αποτυπώνουν τις ευρεθείσες συστηματικές διαφοροποιήσεις μεταξύ των projects που παραμένουν ακόμη και μετά τον έλεγχο των fixed effects. Στα μοντέλα που υλοποιήθηκαν και χρησιμοποιήθηκαν στην παρούσα έρευνα χρησιμοποιήθηκε το random intercept ανά project, επιτρέποντας σε κάθε project να έχει διαφορετικό βασικό επίπεδο πυκνότητας προβλημάτων ποιότητας ή/και ασφάλειας. Δύο παρατηρήσεις με συγκρίσιμο μέγεθος και πολυπλοκότητα της τελικής κατάστασης του αναλυόμενου τμήματος κώδικα και ίδια κατηγορία Pull Request μπορούν να παρουσιάζουν διαφορετική αναμενόμενη τιμή του outcome εξαιτίας της διαφορετικής project-level συνιστώσας που αντιστοιχεί στο project στο οποίο και ανήκουν.

Στο πλήρες μοντέλο για την log_quality_density, η εκτιμώμενη project-level variance υπολογίστηκε σε 1,825, ενώ η υπολειμματική διακύμανση εντός των

projects ήταν 0,3616. Παρά την εισαγωγή των fixed effects για το μέγεθος, την πολυπλοκότητα του αναλυόμενου τμήματος κώδικα και την κατηγορία του Pull Request παρέμεινε σημαντική διαφοροποίηση μεταξύ των projects. Ο αντίστοιχος ICC του πλήρους μοντέλου ανέρχεται περίπου σε 83,5%, γεγονός που δείχνει ότι η project-level δομή εξακολουθεί να αποτελεί κυρίαρχη πηγή μεταβλητότητας στην πυκνότητα των quality issues.

Σε διενεργούμενη σύγκριση με το αντίστοιχο null model, όπου η between-project variance ήταν 2,868, η τιμή μειώθηκε σε 1,825 μετά την εισαγωγή των fixed effects. Η μεταβολή που αναφέρθηκε υποδεικνύει ότι χαρακτηριστικά όπως είναι το μέγεθος και η πολυπλοκότητα, κυρίως, εξηγούν μέρος της διαφοροποίησης που αρχικά παρατηρείται μεταξύ των projects. Η project-level variance όμως, παραμένει υψηλή. Το γεγονός αυτό συνεπάγεται εννοιολογικά ότι σημαντικό μέρος των διαφορών δεν αποτυπώνεται από τους predictors που περιλαμβάνονται στο μοντέλο.

Η κατανομή των random intercepts ενισχύει την ίδια εικόνα. Για το quality model, οι project-specific αποκλίσεις κυμαίνονταν μεταξύ των -3,318 και +2,887 (μαθηματικό διάστημα [-3,318, +2,887]) στην κλίμακα του $\log(1 + \text{quality_density})$. Θετικό random intercept σημαίνει ότι ένα project παρουσιάζει συστηματικά υψηλότερη πυκνότητα quality issues από εκείνη που θα αναμενόταν με βάση το μέγεθος, την πολυπλοκότητα και την κατηγορία των Pull Requests του, ενώ αρνητική τιμή υποδηλώνει συστηματικά χαμηλότερη πυκνότητα.

Ιδιαίτερη προσοχή απαιτείται στην ερμηνεία των διαφόρων ακραίων random effects για projects με πολύ μικρό αριθμό Pull Requests. Ορισμένα από τα υψηλότερα ή χαμηλότερα estimates προέρχονται από clusters με μία ή λίγες μόνο παρατηρήσεις με συνέπεια να περιέχουν μεγαλύτερη αβεβαιότητα. Η χρήση partial pooling στα LMM περιορίζει σε έναν βαθμό το πρόβλημα αυτό, ενώ η σταθερότητα των αποτελεσμάτων εξετάστηκε επιπλέον μέσω sensitivity analysis που περιορίστηκε σε projects με τουλάχιστον πέντε (5) Pull Requests.

Ακόμη εντονότερη project-level διαφοροποίηση παρέμεινε στο μοντέλο για την log_security_density. Η variance του random intercept εκτιμήθηκε σε 0,156, έναντι residual variance 0,0130, με τον ICC του πλήρους μοντέλου να ανέρχεται περίπου σε 92,3%. Η απόλυτη τιμή της variance δεν είναι άμεσα συγκρίσιμη με εκείνη του quality model, καθώς τα δύο outcomes βρίσκονται σε διαφορετικές κατανομές και κλίμακες. Το αντίστοιχο ICC δείχνει ότι η σχετική συγκέντρωση της μεταβλητότητας στο επίπεδο του project είναι ακόμη ισχυρότερη για τα security issues.

Outcome	Project variance	Residual variance	ICC πλήρους LMM
log_quality_density	1,825	0,3616	83,5%
log_security_density	0,156	0,0130	92,3%

Πίνακας 8.5.3 – Εκτιμώμενες συνιστώσες διακύμανσης και ICC των πλήρων random-intercept Linear Mixed Models

Επιπροσθέτως, σε αντίθεση με το quality outcome, η project-level variance του security model παρέμεινε ουσιαστικά αμετάβλητη σε σχέση με το null model, όπου είχε εκτιμηθεί σε 0,155. Περιγραφικά, το αποτέλεσμα αυτό υποδηλώνει ότι οι fixed effects του μεγέθους, της πολυπλοκότητας και της κατηγορίας του εκάστοτε Pull Request εξηγούν πολύ μικρό μέρος των συστηματικών διαφορών ασφάλειας μεταξύ των projects. Το εύρημα συμφωνεί και με τα αποτελέσματα των fixed effects, όπου μόνο η πολυπλοκότητα εμφάνισε μία μικρή στατιστικά σημαντική σχέση με το security outcome.

Τα random effects αναδεικνύουν το project ως έναν από τους σημαντικότερους παράγοντες διαφοροποίησης των αποτελεσμάτων. Ιδιαίτερα υψηλή project-level μεταβλητότητα παραμένει τόσο στην ποιότητα όσο και στην ασφάλεια ακόμη και μετά τον έλεγχο των παρατηρούμενων χαρακτηριστικών των Pull Requests. Το αποτέλεσμα υποδεικνύει ότι χαρακτηριστικά που λειτουργούν σε επίπεδο project και δεν περιλαμβάνονται άμεσα στα μοντέλα, μεταξύ αυτών η αρχιτεκτονική, η τεχνολογική βάση, οι πρακτικές ανάπτυξης ή άλλα χαρακτηριστικά της βάσης κώδικα, ενδέχεται να συνδέονται με σημαντικό μέρος της ετερογένειας που παρατηρείται στα δεδομένα. Η συγκεκριμένη ερμηνεία παραμένει συσχετιστική και δεν συνεπάγεται αιτιώδη επίδραση των παραπάνω παραγόντων.

8.5.4 Σύγκριση LMM με OLS

Η σύγκριση των αποτελεσμάτων των Linear Mixed Models με τα αντίστοιχα OLS μοντέλα επιτρέπει να αξιολογηθεί κατά πόσο η ρητή ενσωμάτωση της project-level ετερογένειας μεταβάλλει τα συμπεράσματα της ανάλυσης. Τα δύο είδη μοντέλων χρησιμοποιούν τις ίδιες βασικές εξαρτημένες και ανεξάρτητες μεταβλητές, αντιμετωπίζουν διαφορετικά τη δομή των δεδομένων. Στα OLS μοντέλα χρησιμοποιήθηκαν cluster-robust standard errors σε επίπεδο project, ενώ στα LMM η διαφοροποίηση μεταξύ projects μοντελοποιείται άμεσα μέσω του random intercept.

Για την πυκνότητα προβλημάτων ποιότητας, τα δύο μοντέλα κατέληξαν σε ιδιαίτερα συνεπή αποτελέσματα ως προς την κατεύθυνση των βασικών σχέσεων. Στο OLS μοντέλο, η log_ncloc είχε αρνητικό συντελεστή ($\beta = -0,169, p = 0,021$), ενώ στο LMM η αντίστοιχη εκτίμηση ήταν $\beta = -0,152$ με $p < 0,001$. Η σχέση επομένως παρέμεινε αρνητική και παρόμοια ως προς το μέγεθος ενώ μετά την ενσωμάτωση της project-level δομής η στατιστική ένδειξη ισχυροποιήθηκε.

Αντίστοιχα σταθερή ήταν η επίδραση της log_complexity. Στο OLS μοντέλο εκτιμήθηκε $\beta = 0,368$, ενώ στο LMM $\beta = 0,398$, με $p < 0,001$ και στις δύο περιπτώσεις. Η μικρή διαφοροποίηση που εντοπίζεται στο μέγεθος του συντελεστή δεν αλλάζει το βασικό συμπέρασμα. Η μεγαλύτερη πολυπλοκότητα συνδέεται σταθερά με υψηλότερη πυκνότητα προβλημάτων ποιότητας. Παράλληλα, οι κατηγορίες security και both δεν παρουσίασαν στατιστικά σημαντικές διαφορές από την κατηγορία αναφοράς quality ούτε στο OLS καθώς επίσης και ούτε στο LMM.

Η συμφωνία των δύο προσεγγίσεων στο quality outcome ενισχύει τη σταθερότητα του συγκεκριμένου ευρήματος. Παρότι τα μοντέλα αντιμετωπίζουν

διαφορετικά την ομαδοποίηση των παρατηρήσεων, τόσο η κατεύθυνση όσο και το μέγεθος των βασικών fixed effects παραμένουν σε μεγάλο βαθμό παρόμοια. Η θετική σχέση της πολυπλοκότητας και η αρνητική σχέση του μεγέθους με την πυκνότητα των quality issues δεν φαίνεται να αποτελούν αποτέλεσμα αποκλειστικά της επιλογής ενός συγκεκριμένου στατιστικού μοντέλου.

Διαφοροποίηση, περισσότερο εμφανής των προηγούμενων, παρατηρήθηκε για την πυκνότητα προβλημάτων ασφάλειας. Στο OLS μοντέλο, η log_complexity δεν παρουσίασε στατιστικά σημαντική σχέση με το outcome ($\beta = -0,006, p = 0,595$). Αφού ενσωματώθηκε το random intercept ανά project, ο αντίστοιχος συντελεστής στο LMM μειώθηκε σε $\beta = -0,011$ και στατιστικά σημαντικός ($p = 0,017$). Παρότι το μέγεθος της σχέσης παραμένει μικρό, η διαφορά αυτή δείχνει ότι η project-level ετερογένεια επηρέαζε την εκτίμηση της σχέσης μεταξύ πολυπλοκότητας και security density.

Αντίθετα, για το log_ncloc δεν προέκυψε στατιστικά σημαντική σχέση σε κανένα από τα δύο security models. Το ίδιο ισχύει και για τις κατηγορίες των Pull Requests, οι οποίες δεν παρουσίασαν σημαντικές διαφοροποιήσεις από την κατηγορία quality σε κανένα εκ των δύο μοντέλων που χρησιμοποιήθηκαν και εξετάστηκαν. Συνεπώς, η σημαντικότερη διαφοροποίηση μεταξύ των δύο προσεγγίσεων αφορά τη μικρή αρνητική σχέση της πολυπλοκότητας με την πυκνότητα των security issues, η οποία γίνεται εμφανής μόνο όταν ληφθεί ρητά υπόψη η μεταξύ-project μεταβλητότητα.

Η παραπάνω σύγκριση καταδεικνύει τον διαφορετικό ρόλο των δύο προσεγγίσεων στην παρούσα έρευνα. Τα cluster-robust standard errors του OLS διορθώνουν την εκτίμηση της αβεβαιότητας ώστε να επιτρέπεται συσχέτιση μεταξύ Pull Requests του ίδιου project, χωρίς όμως να εισάγουν ξεχωριστή παράμετρο που να περιγράφει τις συστηματικές διαφορές μεταξύ projects. Τα LMM μοντέλα, αντιθέτως, μοντελοποιούν άμεσα την διαφοροποίηση όπως αποτυπώθηκε μέσω του random intercept και διαχωρίζουν τη between-projects από την within-project μεταβλητότητα.

Το πλεονέκτημα που προσδίδει είναι ιδιαίτερα σημαντικό στο συγκεκριμένο σύνολο δεδομένων, καθώς τα προηγούμενα αποτελέσματα έδειξαν πολύ υψηλά επίπεδα project-level clustering. Στα πλήρη LMM, το ~83,5% της υπολειπόμενης διακύμανσης του quality outcome και 92,3% της αντίστοιχης διακύμανσης του security outcome αποδίδονται στο επίπεδο του project. Υπό αυτές τις συνθήκες, η μοντελοποίηση του project ως random effect προσφέρει πληρέστερα αναπαράσταση της πραγματικής δομής του δείγματος.

Καταλήγοντας, η σύγκριση OLS και LMM δείχνει ότι τα βασικά συμπεράσματα για την ποιότητα είναι σε μεγάλο βαθμό σταθερά μεταξύ των δύο προσεγγίσεων, ενώ στο security outcome η ενσωμάτωση της project-level μεταβλητότητας επηρεάζει περισσότερο την εκτίμηση της σχέσης με την πολυπλοκότητα. Για τον λόγο αυτό, τα OLS μοντέλα διατηρούνται ως χρήσιμα baseline models, ενώ τα LMM αποτελούν την κύρια βάση ερμηνείας των αποτελεσμάτων της έρευνας.

Predictor	OLS Quality β	LMM Quality β	OLS Security β	LMM Security β
both vs quality	-0,004	-0,027	0,049	0,000

security vs quality	0,008	-0,038	-0,015	-0,008
log_ncloc	-0,169 *	-0,152 *	-0,003	-0,004
log_complexity	0,368 *	0,398 *	-0,006	-0,011 *

* Υποδηλώνει στατιστικά σημαντική σχέση στο αντίστοιχο μοντέλο

Πίνακας 8.5.4 – Σύγκριση των εκτιμώμενων συντελεστών των OLS και LMM για τις πυκνότητες προβλημάτων ποιότητας και ασφάλειας

8.6 Αποτελέσματα PCA

Όπως έχει προαναφερθεί σε άλλη ενότητα της παρούσας βιβλιογραφίας, η Principal Component Analysis εφαρμόστηκε στις επτά βασικές μετρικές οι οποίες χρησιμοποιήθηκαν για την κοινή αποτύπωση διαστάσεων ποιότητας, ασφάλειας και πολυπλοκότητας. Υπενθυμίζονται επιγραμματικά, *reliability_issues_per_kloc*, *maintainability_issues_per_kloc*, *security_issues_per_kloc*, *security_hotspots_per_kloc*, *complexity_per_kloc*, *cognitive_complexity_per_kloc* και *duplicated_lines_density*. Στόχος της ανάλυσης ορίστηκε η εξέταση του κατά πόσο οι επιμέρους μετρικές οργανώνονται σε έναν μικρότερο αριθμό πολυμεταβλητών διαστάσεων.

Η επιλογή του αριθμού των principal components οδήγησε σε σαφή τρισδιάστατη λύση. Τόσο η Parallel Analysis όσο και το Kaiser criterion υπέδειξαν τη διατήρηση τριών components. Οι αντίστοιχες eigenvalues ήταν 2,806 για το PC1, 1,547 για το PC2 και 1,069 για το PC3 αντίστοιχα, ενώ τα επόμενα components εμφάνιζαν eigenvalues μικρότερες της μονάδας και δεν διατηρήθηκαν στην κύρια λύση.

Το PC1 εξηγούσε περίπου το 40,1% της συνολικής διακύμανσης, το PC2 επιπλέον 22,1% και το PC3 ακόμη 15,3%. Συνολικά, τα τρία (3) διατηρούμενα components εξηγούσαν περίπου το 77,5% της συνολικής διακύμανσης των επτά αρχικών μεταβλητών. Επομένως, περισσότερο από τα ¾ (τρία τέταρτα) της πληροφορίας του αρχικού πολυμεταβλητού χώρου μπορούσαν να αποδοθούν μέσω τριών σύνθετων διαστάσεων.

Η ερμηνεία των components βασίστηκε στα correlation loadings των αρχικών μεταβλητών. Το PC1 παρουσίασε υψηλές θετικές φορτίσεις για την *complexity_per_kloc* (0,880), την *cognitive_complexity_per_kloc* (0,845), τα *maintainability_issues_per_kloc* (0,803) και την *duplicated_lines_density* (0,661). Μικρότερη αλλά θετική συνεισφορά παρατηρήθηκε και για τα *reliability_issues_per_kloc* (0,458). Η συγκεκριμένη διάταξη, όπως αυτή αποδόθηκε, υποδεικνύει ότι το πρώτο components αποτυπώνει κυρίως μία ευρύτερη διάσταση δομικής πολυπλοκότητας και επιβάρυνσης συντηρησιμότητας, στην οποία μεγαλύτερη πολυπλοκότητα, *cognitive complexity*, *maintainability issues* και *duplication* τείνουν να εμφανίζονται από κοινού.

Το PC2 χαρακτηριζόταν κυρίως από υψηλές θετικές φορτίσεις των *security_issues_per_kloc* (0,732) και *security_hotspots_per_kloc* (0,730). Οι υπόλοιπες μεταβλητές είχαν σαφώς μικρότερη σχέση με το συγκεκριμένο component, ενώ η *complexity* και η *cognitive complexity* παρουσίαζαν μικρές έως μέτριες αρνητικές φορτίσεις. Το PC2 μπορεί επομένως να ερμηνευθεί ως μία σχετικά διακριτή διάσταση ασφάλειας, η οποία συγκεντρώνει κυρίως τη συνδιακύμανση μεταξύ των δύο βασικών security-related indicators.

Το PC3 παρουσίασε πιο ισχυρή θετική φόρτιση στα *reliability_issues_per_kloc* (0,700) και μικρότερη θετική φόρτιση στην *duplicated_lines_density* (0,395). Παράλληλα, εμφανίστηκαν αρνητικές φορτίσεις για τα *security_hotspots_per_kloc* (-0,384), *cognitive_complexity_per_kloc* (-0,365) και *complexity_per_kloc* (-0,289). Η Τρίτη συνιστώσα αποτυπώνει κυρίως μία διάσταση αξιοπιστίας, η οποία όμως παρουσιάζει περισσότερο σύνθετη δομή σε σχέση με τα δύο πρώτα components. Για τον λόγο αυτό η ερμηνεία του PC3 απαιτεί μεγαλύτερη προσοχή και δεν αντιμετωπίζεται ως μία απολύτως καθαρή και ανεξάρτητη έννοια.

Component	Eigenvalue	Explained variance	Cumulative variance	Κύρια ερμηνεία
PC1	2,806	40,1%	40,1%	Δομική πολυπλοκότητα / συντηρησιμότητα
PC2	1,547	22,1%	62,2%	Ασφάλεια
PC3	1,069	15,3%	77,5%	Αξιοπιστία / μικτή διάσταση

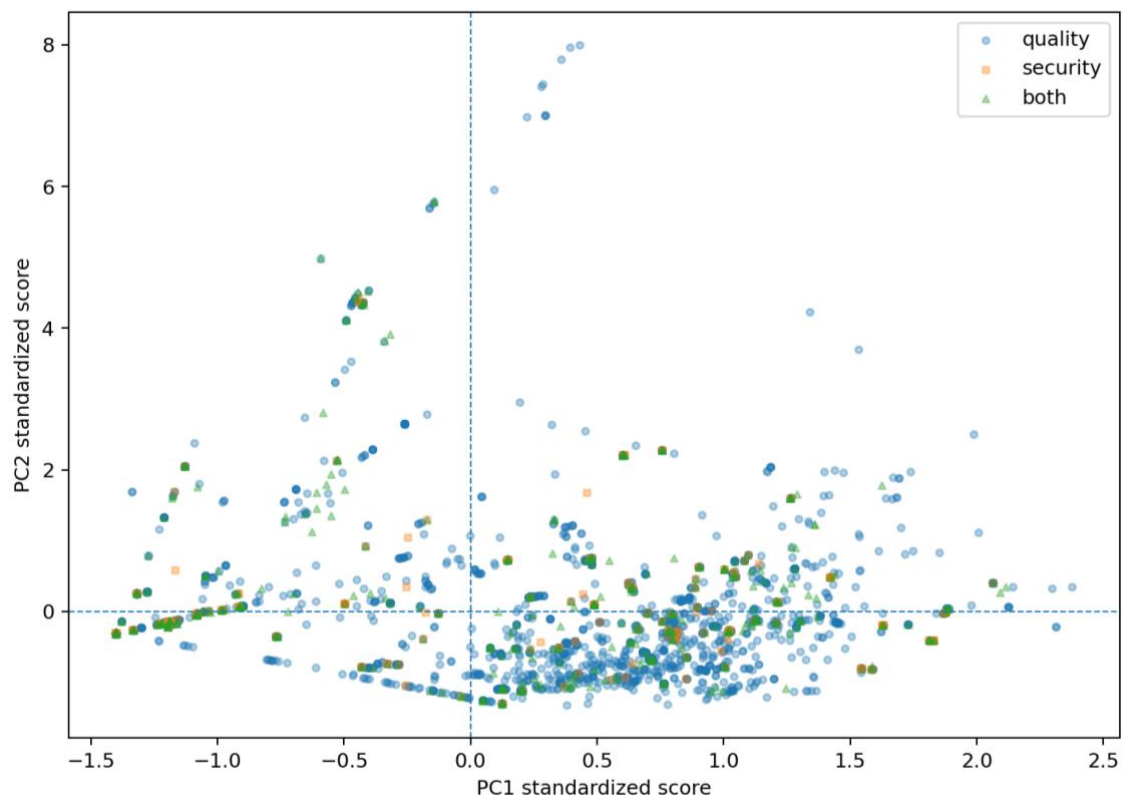
Πίνακας 8.6.1 – Eigenvalues, ποσοστά εξηγούμενης διακύμανσης και συνοπτική ερμηνεία των τριών principal components της κύριας διενεργηθείσας PCA

Η συνολική εικόνα των loadings παρουσιάζει ιδιαίτερο ενδιαφέρον ως προς τη σχέση ποιότητας και ασφάλειας. Οι μεταβλητές οι οποίες σχετίζονται με την πολυπλοκότητα, τη συντηρησιμότητα και την επανάληψη κώδικα συγκεντρώνονται κυρίως στο πρώτο component, ενώ οι βασικές μετρικές ασφάλειας συγκεντρώνονται στο δεύτερο. Τα αποτελέσματα δεν υποστηρίζουν, συνεπώς, μία μοναδική κοινή διάσταση στην οποία η ποιότητα και η ασφάλεια μεταβάλλονται με τον ίδιο τρόπο αυτόν καθ' αυτόν. Από την άλλη, αναδεικνύουν σχετιζόμενες αλλά διακριτές μεταξύ τους πολυμεταβλητές δομές, με την ασφάλεια να εμφανίζει σαφέστερη διαφοροποίηση από τη διάσταση της δομικής ποιότητας και πολυπλοκότητας.

Η προβολή των Pull Requests στον χώρο των δύο πρώτων principal components δεν έδειξε σαφή διαχωρισμό μεταξύ των διαχειριζόμενων κατηγοριών quality, security και both. Οι παρατηρήσεις των τριών δημιουργημένων κατηγοριών παρουσίαζαν σημαντική αλληλοεπικάλυψη, γεγονός που υποδηλώνει ότι η αρχική κατηγοριοποίηση των PRs δεν αντιστοιχεί σε καθαρά τρεις διαχωρισμένες περιοχές του πολυμεταβλητού χώρου. Το εύρημα αυτό είναι συνεπές με τα αποτελέσματα των πρότερα εκτελούντων LMM, κατά τα οποία η κατηγορία του Pull Request δεν αποτέλεσε σημαντικό predictor των κύριων quality και security outcomes.

Η περαιτέρω ανάλυση των component scores μέσω random-intercept LMM ανέδειξε επίσης ιδιαίτερα ισχυρή project-level δομή. Οι ICC εκτιμήθηκαν σε περίπου 94,3% για το PC1, 86,5% για το PC2 και 85,6% για το PC3, δείχνοντας ότι μεγάλο μέρος της διαφοροποίησης των πολυμεταβλητών διαστάσεων συγκεντρώνεται μεταξύ διαφορετικών projects. Το log_ncloc συνδέθηκε θετικά με το PC1 ($\beta \sim 0,176$, $p < 0,001$), αρνητικά με το PC2 ($\beta \sim -0,050$, $p < 0,001$) και θετικά με το PC3 ($\beta \sim 0,073$, $p < 0,001$). Οι κατηγορίες των PRs δεν παρουσίασαν, όμως, σημαντικές διαφοροποιήσεις σε κανένα εκ των τριών component-score models.

Εν κατακλείδι, η PCA ανέδειξε ότι η πληροφορία των επτά επιμέρους μετρικών μπορεί να συνοψιστεί αποτελεσματικά σε τρεις κύριες διαστάσεις. Οι διαστάσεις αυτές είναι μία κυρίαρχη διάσταση δομικής πολυπλοκότητας και συντηρησιμότητας, μία σαφέστερη διάσταση ασφάλειας και μία τρίτη, περισσότερο σύνθετη διάσταση που σχετίζεται κυρίως με την αξιοπιστία. Τα αποτελέσματα ενισχύουν την εικόνα ότι η ποιότητα και η ασφάλεια δεν αποτελούν πλήρως ανεξάρτητες ή πλήρως ταυτόσημες διαστάσεις, αλλά εντάσσονται σε μία πολυδιάστατη δομή στην οποία διαφορετικά χαρακτηριστικά του κώδικα συνδέονται μεταξύ τους με διαφορετική ένταση.



Εικόνα 8.6.1 – Προβολή των PRs στον χώρο των δύο πρώτων principal components ανά κατηγορία PR (quality, security, both)

8.7 Αποτελέσματα Factor Analysis

Αντίστοιχα με την PCA, η Factor Analysis εφαρμόστηκε στις ίδιες επτά βασικές μετρικές, με στόχο να διερευνηθεί εάν οι παρατηρούμενες μεταβλητές μπορούν να αποδοθούν σε έναν μικρότερο αριθμό υποκείμενων λανθανόντων παραγόντων. Η ανάλυση ανέδειξε μία πολυδιάστατη δομή στην οποία οι μετρικές πολυπλοκότητας, συντηρησιμότητας, αξιοπιστίας και ασφάλειας οργανώνονται σε διακριτές αλλά εν μέρει αλληλένδετες ομάδες.

Η καταλληλότητα των δεδομένων για Factor Analysis αξιολογήθηκε αρχικά μέσω των Kaiser-Meyer-Olkin (KMO) και Bartlett's test of sphericity (BTS). Ο συνολικός δείκτης KMO ήταν 0,631, αριθμός που υποδεικνύει μέτρια καταλληλότητα του συνόλου των μεταβλητών για παραγοντική ανάλυση. Παράλληλα, το BTS ήταν έντονα στατιστικά σημαντικό ($\chi^2 = 13.877,232$, $df = 21$, $p < 0,001$), απορρίπτοντας με τον τρόπο αυτό την υπόθεση ότι ο πίνακας συσχετίσεων είναι μοναδιαίος. Συνεπώς, υπήρχε επαρκής κοινή συσχέτιση μεταξύ των μεταβλητών για να δικαιολογηθεί η εφαρμογή της Factor Analysis, αν και η συνολική καταλληλότητα του δείγματος δεν ήταν ιδιαίτερως υψηλή.

Η Parallel Analysis υπέδειξε τη διατήρηση τριών factors στην κύρια ανάλυση. Οι τρεις πρώτες παρατηρούμενες eigenvalues ήταν 2,806, 1,547 και 1,069 και υπερέβαιναν τα αντίστοιχα thresholds που προέκυψαν από τα τυχαία δεδομένα. Έπειτα του τρίτου παράγοντα, οι παρατηρούμενες eigenvalues δεν παρείχαν επαρκή ένδειξη για τη διατήρηση πρόσθετων λανθανόντων διαστάσεων. Η κύρια λύση, τελικά, βασίστηκε σε τρεις περιστραμμένους factors (παράγοντες).

Αφού η διαδικασία της περιστροφής των παραγόντων ολοκληρώθηκε, ο Factor 1 χαρακτηριζόταν κυρίως από ιδιαίτερα υψηλά loadings της cognitive_complexity_per_kloc (0,985) και της complexity_per_kloc (0,959), καθώς και από θετικό loading των maintainability_issues_per_kloc (0,484). Η duplicated_lines_density παρουσίαζε επίσης θετική, αλλά χαμηλότερη φόρτιση (0,394). Η συγκεκριμένη δομή επιτρέπει την ερμηνεία του πρώτου παράγοντα ως μία διάσταση δομικής πολυπλοκότητας και επιβάρυνσης συντηρησιμότητας, όπου η αυξημένη πολυπλοκότητα και η δυσκολία κατανόησης της βάσης του πηγαίου κώδικα συνδέονται με την ύπαρξη περισσότερων προβλημάτων συντήρησης.

Ο Factor 2 παρουσίασε την υψηλότερη φόρτιση στα reliability_issues_per_kloc (0,807), ενώ σημαντικές θετικές επίσης φορτίσεις εμφάνισαν τα maintainability_issues_per_kloc (0,589) και η duplicated_lines_density (0,418). Ο συγκεκριμένος παράγοντας αποτυπώνει επομένως μία ευρύτερη διάσταση αξιοπιστίας και συντηρησιμότητας, στην οποία τα reliability issues συνδέονται με προβλήματα συντήρησης. Σημειώνεται ότι σε μικρότερο βαθμό συνδέονται με την επανάληψη του κώδικα.

Ο Factor 3 συγκέντρωσε με την σειρά του κυρίως τις μεταβλητές ασφάλειας. Τα security_hotspots_per_kloc εμφάνισαν loading 0,759, ενώ τα security_issues_per_kloc loading 0,436. Βάσει αυτού, η διάσταση μπορεί να ερμηνευθεί ως security factor, καθώς οι δύο κυριότερες μετρικές ασφάλειας φορτίζουν θετικά στον ίδιο λανθάνοντα παράγοντα. Η συγκεκριμένη λύση, όμως, είναι περισσότερο εύθραυστη σε σχέση με τους δύο προηγούμενους factors, καθώς

στηρίζεται ουσιαστικά σε δύο salient indicators, από τους οποίους μόνο τα security hotspots παρουσιάζουν ιδιαίτερα υψηλή φόρτιση.

Η εξέταση communalities και uniqueness values ενισχύει αυτή την επιφύλαξη. Η complexity_per_kloc και η cognitive_complexity_per_kloc παρουσίασαν πολύ υψηλές communalities, περίπου 0,940 και 0,972 αντίστοιχα, γεγονός το οποίο σημαίνει ότι η τριπαραγοντική λύση εξηγεί το μεγαλύτερο μέρος της διακύμανσής τους. Από την άλλη πλευρά, τα security_issues_per_kloc είχαν communality ~0,206 και uniqueness 0,794, ενώ η duplicated_lines_density communality ~0,342 και uniqueness 0,658. Σημειώνεται ότι σημαντικό μέρος της διακύμανσης των συγκεκριμένων μεταβλητών δεν αποδίδεται από τους τρεις κοινούς factors.

Τα αποτελέσματα δείχνουν ότι η παραγοντική δομή δεν αντιστοιχεί σε μία απλή διάκριση μεταξύ «ποιότητας» και «ασφάλειας». Τα χαρακτηριστικά, αντίθετα, που συνδέονται με την ποιότητα κατανέμονται τουλάχιστον σε δύο διαφορετικές λανθάνουσες διαστάσεις: μία που σχετίζεται κυρίως με την πολυπλοκότητα και τη συντηρησιμότητα και μία δεύτερη που σχετίζεται περισσότερο με την αξιοπιστία, τη συντηρησιμότητα και την επανάληψη τμημάτων πηγαίου κώδικα. Η ασφάλεια σχηματίζει μία τρίτη, περισσότερο διακριτή αλλά λιγότερο σταθερή διάσταση.

Η περαιτέρω ανάλυση των factor scores μέσω random-intercept Linear Mixed-Effects Models ανέδειξε έντονη project-level δομή. Για τον Factor 1, το log_ncloc παρουσίασε θετική σχέση με $\beta \sim 0,154$ και $p < 0,001$, ενώ ο ICC ανήλθε περίπου στο 94,7%. Για τον Factor 2, το log_ncloc παρουσίασε θετική σχέση, όπως αντίστοιχα και στο Factor 1, με $\beta \sim 0,078$, $p < 0,001$, με ICC στο 87,1% κατά προσέγγιση. Για τον Factor 3, αντίθετα με τους Factor 1 και Factor 2, παρατηρήθηκε εμφάνιση μικρού βεληνεκούς αρνητικής σχέσης του log_ncloc με τον ICC να βρίσκεται στο 78,5% (estimation) και $\beta \sim -0,034$ και $p = 0,004$. Και στα τρία μοντέλα, οι κατηγορίες quality, security και both δεν παρουσίασαν στατιστικά σημαντικές διαφοροποιήσεις.

Μετρική	Factor 1	Factor 2	Factor 3
Reliability issues / KLOC	0,035	0,807	0,028
Maintainability issues / KLOC	0,484	0,589	0,373
Security issues / KLOC	-0,081	0,092	0,436
Security hotspots / KLOC	-0,007	0,077	0,759
Complexity / KLOC	0,959	0,139	-0,008
Cognitive complexity / KLOC	0,985	0,033	0,005
Duplicated lines density	0,394	0,418	-0,110

Πίνακας 8.7.1 – Περιστραμμένα factor loadings των επτά βασικών μεταβλητών στους τρεις factors της κύριας Factor Analysis

Τα υψηλά ICC των factor-score models υποδεικνύουν ότι οι λανθάνουσες πολυμεταβλητές διαστάσεις διαφοροποιούνται έντονα μεταξύ της εκάστοτε μονάδας project. Ειδικότερα, η πολύ υψηλή project-level συγκέντρωση του Factor 1 υποδηλώνει ότι η διάσταση της πολυπλοκότητας και της επιβάρυνσης

συντηρησιμότητας αποτελεί σε σημαντικό βαθμό χαρακτηριστικό που διαφοροποιεί ολόκληρα projects και όχι αποκλειστικά μεμονωμένα Pull Requests.

Η size-adjusted sensitivity analysis παρείχε μία σημαντική συμπληρωματική ένδειξη σχετικά με τη σταθερότητα της παραγοντικής δομής. Όταν εφαρμόστηκε εναλλακτική προσαρμογή ως προς το μέγεθος του κώδικα, η Parallel Analysis υπέδειξε τη διατήρηση δύο αντί για τρεις factors. Οι βασικές διαστάσεις της πολυπλοκότητας/συντηρησιμότητας και της αξιοπιστίας παρέμειναν εμφανείς, ενώ η ανεξάρτητη security διάσταση δεν αναπαράχθηκε με την ίδια σαφήνεια. Το εξαγόμενο αποτέλεσμα ενισχύει την ανάγκη προσεκτικής ερμηνείας του τρίτου factor και δείχνει ότι η ακριβής παραγοντική δομή είναι σε κάποιο βαθμό ευαίσθητη στον τρόπο προσαρμογής των μετρικών ως προς το μέγεθος του πηγαίου κώδικα που εξετάζεται.

Συνοψίζοντας, η factor analysis υποστηρίζει την ύπαρξη μίας πολυδιάστατης υποκείμενης δομής στα δεδομένα. Οι μετρικές ποιότητας δεν σχηματίζουν έναν ενιαίο λανθάνοντα παράγοντα, αλλά διαχωρίζονται σε διαστάσεις οι οποίες σχετίζονται με την πολυπλοκότητα, τη συντηρησιμότητα και την αξιοπιστία. Οι μετρικές ασφάλειας συγκεντρώνονται σε ξεχωριστό factor στην κύρια ανάλυση. Η μικρότερη, ωστόσο, σταθερότητα και η υψηλή uniqueness των security_issues_per_kloc απαιτούν πιο συγκρατημένη ερμηνεία. Κατά συνέπεια, τα αποτελέσματα ενισχύουν την εικόνα ότι quality-security related indicators εντάσσονται σε μία σύνθετη πολυμεταβλητή δομή, στην οποία εμφανίζουν τόσο κοινά όσο και διακριτά πρότυπα, χωρίς να μπορούν να αναχθούν σε μία μοναδική κοινή διάσταση.

8.8 Robustness analyses

Προκειμένου να εξεταστεί κατά πόσο τα βασικά ευρήματα της έρευνας παραμένουν σταθερά όταν μεταβάλλονται επιμέρους αναλυτικές επιλογές ή η σύνθεση του δείγματος επιλέχθηκε και χρησιμοποιήθηκε η robustness analyses. Συνολικά, τα αποτελέσματα έδειξαν ότι τα βασικά συμπεράσματα των κύριων μοντέλων διατηρούνται σε σημαντικό βαθμό, αν και ορισμένα επιμέρους ευρήματα παρουσίασαν μεγαλύτερη ευαισθησία ως προς τη χρησιμοποιούμενη μοντελοποίηση.

Αρχικά, τα κύρια Linear Mixed Models επανεκτιμήθηκαν αποκλειστικά σε projects που περιείχαν τουλάχιστον πέντε (5) Pull Requests. Με αυτόν τον περιορισμό το δείγμα μειώθηκε από 3.701 Pull Requests και 242 projects σε 3.544 Pull Requests από 149 projects. Στο quality model, η αρνητική σχέση του log_ncloc παρέμεινε ουσιαστικά αμετάβλητη, με τον συντελεστή να μεταβάλλεται από περίπου -0,152 σε -0,141. Αντίστοιχα, η θετική επίδραση της log_complexity παρέμεινε ισχυρή, με τον συντελεστή να μεταβάλλεται από 0,398 σε 0,369, όπως κατά προσέγγιση υπολογίστηκε. Οι κατηγορίες των Pull Requests εξακολούθησαν να μην παρουσιάζουν στατιστικά σημαντικές διαφοροποιήσεις. Η σταθερότητα τόσο της

κατεύθυνσης όσο και του μεγέθους των συντελεστών δείχνει ότι τα κύρια αποτελέσματα δεν καθορίζονται από projects με πολύ μικρό αριθμό παρατηρήσεων.

Σταθερότητα παρόμοια με αυτή που περιεγράφηκε παρατηρήθηκε επίσης στα μοντέλα τα οποία χρησιμοποιούσαν ως outcomes τα PCA component scores και τα Factor Analysis factor scores. Μετά τον αποκλεισμό των μικρότερων project-level clusters, οι βασικές κατευθύνσεις των εκτιμήσεων διατηρήθηκαν, όπως και η έντονη μεταξύ-project διαφοροποίηση. Η υψηλή, επομένως, project-level συγκέντρωση που παρατηρήθηκε στις κύριες αναλύσεις δεν μπορεί να αποδοθεί αποκλειστικά στην παρουσία projects με μία ή λίγες μόνο παρατηρήσεις.

Επίσης, εξετάστηκε μία πιο σύνθετη random-effects δομή, στην οποία επιτρεπόταν η επίδραση του log_ncloc να διαφοροποιείται μεταξύ των projects. Το αντίστοιχο random-slope model δεν πέτυχε σύγκλιση, σε αντίθεση με τα random-intercept models που χρησιμοποιήθηκαν στις κύριες αναλύσεις. Το εξαχθέν αποτέλεσμα υποδεικνύει ότι τα διαθέσιμα δεδομένα δεν παρείχαν επαρκή υποστήριξη για σταθερή εκτίμηση της συγκεκριμένης πιο σύνθετης δομής. Η επιλογή του random-intercept model παρέμεινε, σύμφωνα με τα παραπάνω, η περισσότερη σταθερή και υποστηριζόμενη προδιαγραφή βάσει του διαθέσιμου συνόλου δεδομένων.

Μέσω Negative Binomial models πραγματοποιήθηκε πρόσθετος έλεγχος, στόχος του οποίου ήταν να εξεταστούν τα maintainability και security issues ως count outcomes με προσαρμογή ως προς το μέγεθος του κώδικα. Τα αποτελέσματα επιβεβαίωσαν σε γενικές γραμμές τη σημασία της πολυπλοκότητας για την ή ύπαρξη ή όχι maintainability issues, αλλά ανέδειξαν και ορισμένες διαφοροποιήσεις ως προς τις κατηγορίες των Pull Requests. Πιο ειδικά, τα PRs της κατηγορίας security εμφάνισαν χαμηλότερο αναμενόμενο αριθμό maintainability issues σε σχέση με την κατηγορία quality ($p = 0,003$), ενώ η κατηγορία both παρουσίασε υψηλότερο αναμενόμενο αριθμό security issues ($p = 0,002$).

Τα παραπάνω περιγραφόμενα category effects δεν εμφανίστηκαν με αντίστοιχη στατιστική σημαντικότητα στα κύρια Linear Mixed Models. Η διαφοροποίηση αυτή δείχνει ότι τα αποτελέσματα που αφορούν την κατηγορία του Pull Request είναι περισσότερο model-sensitive και επομένως πρέπει να ερμηνεύονται με μεγαλύτερη επιφύλαξη. Οι σχέσεις, αντίθετα, που αφορούν την πολυπλοκότητα και την έντονη project-level ετερογένεια εμφανίζουν σαφώς μεγαλύτερη σταθερότητα μεταξύ διαφορετικών προσεγγίσεων.

Με την χρήση του GEE Binomial model, συμπληρωματικά των προηγούμενων, εξετάστηκε η σχετική αναλογία των security issues ως προς το σύνολο των security και maintainability issues. Στη συγκεκριμένη ανάλυση ούτε το log_ncloc ($p = 0,527$) καθώς ούτε και το log_complexity ($p = 0,135$) παρουσίασαν στατιστικά σημαντική σχέση με το security share. Το εύρημα υποδηλώνει ότι η αύξηση του μεγέθους ή της πολυπλοκότητας του κώδικα δεν συνοδεύεται από συστηματική μετατόπιση της σύνθεσης των issues ως προς την ασφάλεια.

Ιδιαίτερη σημασία απέκτησε επίσης και η ευαισθησία που παρουσίαζαν τα πολυμεταβλητά αποτελέσματα στον τρόπο προσαρμογής ως προς το μέγεθος του εξεταζόμενου πηγαίου κώδικα. Στην κύρια PCA διατηρήθηκαν τρεις (3) principal components, οι οποίες εξηγούσαν συνολικά περίπου 77,5% της διακύμανσης. Στη

size-adjusted sensitivity analysis η Parallel Analysis υπέδειξε τη διατήρηση μόνο δύο components, με συνολική explained variance περίπου 65,7%.

Εικόνα αντίστοιχη με την παραπάνω παρατηρήθηκε και στην εκτέλεση της Factor Analysis. Η κύρια λύση περιλάμβανε τρεις (3) factors, ενώ η size-adjusted εκδοχή οδήγησε σε δύο factors. Οι κυριότερες διαστάσεις που σχετίζονταν με την πολυπλοκότητα, τη συντηρησιμότητα και την αξιοπιστία παρέμειναν αναγνωρίσιμες, όμως η τρίτη διάσταση που συνδεόταν κυρίως με την ασφάλεια δεν αναπαράχθηκε με την ίδια σταθερότητα. Το αποτέλεσμα αυτό δείχνει ότι η ακριβής πολυμεταβλητή διαστατικότητα είναι ευαίσθητη στον τρόπο με τον οποίο ελέγχεται το μέγεθος του κώδικα.

Οι robustness analyses ενισχύουν κυρίως τρία βασικά συμπεράσματα της παρούσας έρευνας. Τα συμπεράσματα συνοψίζονται στην παρακάτω αριθμημένη λίστα:

1. Η πολυπλοκότητα συνδέεται σταθερά με τη διάσταση της ποιότητας και της συντηρησιμότητας, πράγμα ανεξάρτητο από επιμέρους αλλαγές στη σύνθεση του δείγματος.
2. Η project-level ετερογένεια, που εμφανίζεται, παραμένει ιδιαίτερα ισχυρή σε διαφορετικές προδιαγραφές και outcomes.
3. Τα αποτελέσματα τα οποία αφορούν την κατηγορία των Pull Requests και τον ακριβή αριθμό των υποκείμενων πολυμεταβλητών διαστάσεων εμφανίζουν μεγαλύτερη ευαισθησία στις αναλυτικές επιλογές, με αποτέλεσμα να απαιτούν πιο συγκροτημένη ερμηνεία.

8.9 Ενοποίηση αποτελεσμάτων

Έπειτα της επιμέρους παρουσίασης των αποτελεσμάτων από τις διαφορετικές στατιστικές και πολυμεταβλητές αναλύσεις, κρίνεται απαραίτητη η συνολική τους ενοποίηση, με στόχο την διαμόρφωση ενιαίας εικόνας των εμπειρικών ευρημάτων της παρούσας έρευνας. Αυτή η ενότητα, θέτει ως στόχο τη σύνθεσή τους και την αξιολόγηση του βαθμού στον οποίο οι διαφορετικές αναλυτικές προσεγγίσεις οδηγούν σε συγκλίνουσες, αποκλίνουσες ή διαφοροποιημένες διαπιστώσεις, αγνοώντας την επανάληψη των προηγούμενων αποτελεσμάτων.

Εξετάζονται αρχικά οι παρατηρούμενες συμφωνίες μεταξύ των στατιστικών μεθόδων που ακολουθήθηκαν, με έμφαση στα ευρήματα που επαναλαμβάνονται με σταθερή κατεύθυνση και παρόμοια ερμηνεία σε περισσότερες από μία αναλύσεις. Στη συνέχεια παρουσιάζονται οι διαφορές μεταξύ των μεθόδων, ιδιαίτερα σε περιπτώσεις όπου η στατιστική σημαντικότητα, το μέγεθος των επιδράσεων ή η πολυμεταβλητή δομή διαφοροποιούνται ανάλογα με το μοντέλο ή την αναλυτική προδιαγραφή.

Προσδιορίζονται, ακολούθως, τα ισχυρότερα και πιο σταθερά εμπειρικά ευρήματα, δηλαδή εκείνα που υποστηρίζονται ταυτόχρονα από περισσότερες αναλυτικές προσεγγίσεις και παραμένουν ανθεκτικά στους ελέγχους robustness και

sensitivity. Τέλος, τα συνολικά αποτελέσματα οργανώνονται σε άμεση αντιστοίχιση με τα ερευνητικά ερωτήματα και τις υποθέσεις -όπως αυτά έχουν τεθεί κατά την έναρξη της παρούσας εργασίας-, προκειμένου να αποσαφηνιστεί ποια από αυτά υποστηρίζονται από το διαθέσιμο σύνολο δεδομένων, ποια υποστηρίζονται μόνο εν μέρη και ποια δεν επιβεβαιώνονται.

Με τον τρόπο αυτό, η πραγματευόμενη ενότητα λειτουργεί ως το βασικό σημείο σύνθεσης του παρόντος κεφαλαίου αποτελεσμάτων (βλ. Κεφάλαιο 8 – Αποτελέσματα), συνδέοντας τα αποτελέσματα των OLS και LMM μοντέλων, της PCA, της Factor Analysis και των robustness analysis σε ένα συνεκτικό πλαίσιο ερμηνείας της σχέσης μεταξύ ποιότητας και ασφάλειας του λογισμικού.

8.9.1 Συμφωνίες μεταξύ μεθόδων

Η συνδυαστική εξέταση των αποτελεσμάτων ανέδειξε αρκετά σημεία όπου οι διαφορετικές στατιστικές και πολυμεταβλητές προσεγγίσεις συγκλίνουν. Παρόλο που κάθε μέθοδος εξετάζει διαφορετική πτυχή του ίδιου συνόλου δεδομένων, ορισμένα βασικά πρότυπα επανεμφανίζονται με συνέπεια, γεγονός το οποίο με την σειρά του ενισχύει την αξιοπιστία της συνολικής ερμηνείας.

Η σαφέστερη συμφωνία αφορά τη σχέση της πολυπλοκότητας με την ποιότητα και τη συντηρησιμότητα του κώδικα. Στο OLS μοντέλο για την πυκνότητα προβλημάτων ποιότητας, η λογαριθμική μεταβλητή *log_complexity* παρουσίασε θετική και στατιστικά σημαντική σχέση (υπενθυμίζεται $\beta = 0,368$ και $p < 0,001$), ενώ στο αντίστοιχο LMM η σχέση παρέμεινε θετική και παρόμοιου μεγέθους ($\beta = 0,398$, $p < 0,001$). Η ίδια εικόνα εμφανίστηκε και στις πολυμεταβλητές αναλύσεις, αντίστοιχα. Στην PCA, το πρώτο principal component χαρακτηριζόταν κυρίως από υψηλές φορτίσεις της complexity, της cognitive complexity και των maintainability issues, ενώ στη Factor Analysis ο πρώτος factor παρουσίασε σχεδόν αντίστοιχη δομή. Η επανεμφάνιση του ίδιου προτύπου σε παλινδρομικές και πολυμεταβλητές αναλύσεις υποστηρίζει ισχυρά την ύπαρξη μίας ευρύτερης διάστασης δομικής πολυπλοκότητας και επιβάρυνσης της συντηρησιμότητας.

Ιδιαίτερα σταθερό εύρημα μεταξύ των αναλύσεων αποτέλεσε, επιπλέον, η έντονη διαφοροποίηση μεταξύ των εξεταζόμενων software projects. Τα null και πλήρη Linear Mixed Models παρουσίασαν πολύ υψηλούς ICC τόσο για το quality όσο και για το security outcome, ενώ αντίστοιχα υψηλή project-level συγκέντρωση παρατηρήθηκε και στα LMM που εφαρμόστηκαν στα PCA component scores και στα Factor Analysis factor scores. Η συγκεκριμένη εικόνα διατηρήθηκε και σε περιπτώσεις όπου η ανάλυση περιορίστηκε στα projects με τουλάχιστον πέντε (5) Pull Requests ανά μονάδα project. Η ισχυρή επίδραση της ομαδοποίησης ανά project δεν φαίνεται, βάσει των αποτελεσμάτων, να αποτελεί εύρημα ενός μοντέλου αποκλειστικά, αλλά επαναλαμβανόμενο χαρακτηριστικό διαφορετικών αναλυτικών προσεγγίσεων.

Σημαντική συμφωνία παρατηρήθηκε και ως προς την περιορισμένη ερμηνευτική σημασία της αρχικής κατηγορίας του Pull Request. Στα κύρια OLS και LMM, οι κατηγορίες security και both δεν διαφοροποιούνταν στατιστικά σημαντικά

από την κατηγορία, που ορίστηκε ως κατηγορία αναφοράς, quality ως προς τις κύριες πυκνότητες προβλημάτων. Παρόμοια με το προηγούμενο, στα μοντέλα PCA component scores και των factor scores οι κατηγορίες δεν παρουσίασαν στατιστικά σημαντικές μεταξύ τους διαφορές. Επιπροσθέτως, η προβολή των Pull Requests στον χώρο των δύο πρώτων principal components έδειξε έντονη αλληλοεπικάλυψη μεταξύ των κατηγοριών. Τα αποτελέσματα τα οποία εξήχθησαν συγκλίνουν στην εκτίμηση ότι η αρχική ταξινόμηση ενός Pull Request δεν αρκεί από μόνη της για να περιγράψει την τελική πολυμεταβλητή κατάσταση του κώδικα.

Σύγκλιση μεταξύ αποτελεσμάτων εμφανίστηκε, επίσης, μεταξύ της PCA και της Factor Analysis ως προς τη γενική πολυδιάστατη δομή των δεδομένων. Στην κύρια ανάλυση, και οι δύο μέθοδοι υποστήριξαν τη διατήρηση τριών διαστάσεων μέσω της Parallel Analysis. Η πρώτη διάσταση και στις δύο περιπτώσεις συνδεόταν κυρίως με την πολυπλοκότητα και τη συντηρησιμότητα, ενώ μία διαφορετική διάσταση συγκέντρωνε τις βασικές μετρικές ασφάλειας. Αν και η ακριβής αντιστοίχιση και η ερμηνεία του τρίτου άξονα διέφεραν σε κάποιο βαθμό, οι δύο μέθοδοι συμφωνούν ότι τα δεδομένα δεν μπορούν να περιγραφούν επαρκώς αποκλειστικά από έναν μοναδικό άξονα «ποιότητας-ασφάλειας».

Η συμφωνία των δύο πολυμεταβλητών μεθόδων στις size-adjusted sensitivity analyses κρίνεται ακόμα πιο χαρακτηριστική. Όταν άλλαξε ο τρόπος προσαρμογής ως προς το συνολικό μέγεθος του πηγαίου κώδικα, τόσο η στατιστική τεχνική PCA όσο και η Factor Analysis μετέβησαν από τρεις (3) σε δύο (2) διατηρούμενες διαστάσεις. Η κοινή αυτή μεταβολή δείχνει ότι, παρόλο που η ακριβής dimensionality είναι ευαίσθητη στη μεταχείριση του code size, η συγκεκριμένη ευαισθησία εμφανίζεται και στις δύο ανεξάρτητες πολυμεταβλητές προσεγγίσεις.

Τελικά, οι ισχυρότερες παρατηρημένες συμφωνίες μεταξύ των στατιστικών μεθόδων που χρησιμοποιήθηκαν για τη διεξαγωγή της παρούσας έρευνας αφορούν σε τρία βασικά στοιχεία. Πρώτο εξ' αυτών ορίζεται η στενή σύνδεση της πολυπλοκότητας με τη συντηρησιμότητα, δεύτερο την ιδιαίτερα ισχυρή project-level εμφανιζόμενη ετερογένεια και τελευταίο, την απουσία σταθερού παράγοντα διαφοροποίησης βάσει της αρχικής κατηγορίας των Pull Requests. Παράλληλα, PCA και Factor Analysis συγκλίνουν στην ύπαρξη μίας πολυδιάστατης δομής, στην οποία η δομική ποιότητα και η ασφάλεια εμφανίζονται ως σχετιζόμενες αλλά όχι μεταξύ τους ταυτόσημες έννοιες και διαστάσεις. Η επανάληψη αυτών των προτύπων μέσα από διαφορετικές μεθόδους αποτελεί ένα από τα βασικότερα στοιχεία που ενισχύουν τη συνοχή των εμπειρικών αποτελεσμάτων της παρούσας έρευνας.

Στον παρακάτω πίνακα συνοψίζονται όλες οι παραπάνω συσχετίσεις που παρατηρήθηκαν μεταξύ των υπολογισμένων αποτελεσμάτων της έρευνας.

Κύριο εύρημα	OLS	LMM	PCA	FA	Robustness
Πολυπλοκότητα $\leftrightarrow$ ποιότητα/συντηρησιμότητα	NAI	NAI	NAI	NAI	NAI
Ισχυρή διαφοροποίηση ανά project	-	NAI	NAI*	NAI*	NAI
Κατηγορία PR χωρίς σταθερή επίδραση	NAI	NAI	NAI	NAI	**
Πολυδιάστατη δομή	-	-	NAI	NAI	NAI
Ευαισθησία dimensionality στο size adjustment	-	-	NAI	NAI	NAI

* Μέσω των LMM στα component/factor scores

** Σε μεγάλο βαθμό

Πίνακας 8.9.1 – Συνοπτική αποτύπωση των βασικών σημείων συμφωνίας μεταξύ των στατιστικών και πολυμεταβλητών μεθόδων της έρευνας

8.9.2 Διαφορές μεταξύ μεθόδων

Αντίθετα με τα σημαντικά σημεία σύγκλισης, όπως αυτά αναδείχθηκαν και σημειώθηκαν στην προηγούμενη ενότητα (βλ. ενότητα 8.9.1), μεταξύ των διαφορετικών αναλυτικών στατιστικών προσεγγίσεων, ορισμένα αποτελέσματα διαφοροποιήθηκαν ανάλογα με τη στατιστική μέθοδο, τον τρόπο μοντελοποίησης του outcome και το επίπεδο στο οποίο εξετάστηκε η μεταβλητότητα. Οι διαφοροποιήσεις αυτές δεν αποτελούν κατ' ανάγκη αντιφάσεις, καθώς κάθε μέθοδος απαντά σε διαφορετική πτυχή του ερευνητικά εξεταζόμενου προβλήματος και βασίζεται σε διαφορετικές παραδοχές.

Η σημαντικότερη διαφοροποίηση μεταξύ των OLS και LMM εμφανίστηκε στο μοντέλο της πυκνότητας προβλημάτων ασφάλειας. Στο OLS, η *log_complexity* δεν παρουσίασε στατιστικά σημαντική σχέση με το *log_security_density* ($\beta = -0,006$, $p = 0,595$). Έπειτα από την παρέλευση της ρητής ενσωμάτωσης της διαφοροποίησης μεταξύ των projects μέσω του random intercept, η αντίστοιχη σχέση στο LMM έγινε αρνητική και στατιστικά σημαντική ($\beta = -0,011$, $p = 0,017$). Το αποτέλεσμα αυτό δείχνει ότι η ισχυρή project-level ετερογένεια μπορεί να αποκρύπτει ή να μεταβάλλει ορισμένες σχέσεις όταν τα δεδομένα εξετάζονται χωρίς άμεση μοντελοποίηση της ιεραρχικής τους δομής.

Από την άλλη πλευρά, για το quality outcome οι διαφορές μεταξύ OLS και LMM ήταν περιορισμένες. Οι εκτιμήσεις της *log_ncloc* και της *log_complexity* διατήρησαν την ίδια κατεύθυνση και παρόμοιο μέγεθος και στις δύο προσεγγίσεις. Η διαφοροποίηση μεταξύ τους, δεν αφορά το βασικό εξαχθέν συμπέρασμα για την ποιότητα, αλλά κυρίως τον τρόπο με τον οποίο εκτιμάται η αβεβαιότητα και διαχωρίζεται η μεταβλητότητα που βρίσκεται στο επίπεδο του project.

Πιο εμφανείς διαφοροποιήσεις προέκυψαν όταν τα κύρια LMM συγκρίθηκαν με τα Negative Binomial models. Στα LMM, οι κατηγορίες security και both δεν εμφάνισαν στατιστικά σημαντικές διαφορές από την κατηγορία αναφοράς quality. Στη Negative Binomial models ανάλυση, αντίθετα, η κατηγορία security συνδέθηκε με χαμηλότερο αναμενόμενο αριθμό maintainability issues ($p = 0,003$), ενώ η κατηγορία both με υψηλότερο αναμενόμενο αριθμό security issues ($p = 0,002$). Τα αποτελέσματα αυτά υποδεικνύουν ότι ορισμένες category effects εξαρτώνται από το αν το outcome αντιμετωπίζεται ως λογαριθμικά μετασχηματισμένη πυκνότητα ή ως count variable με exposure ως προς το μέγεθος του πηγαίου κώδικα.

Η συγκεκριμένη διαφοροποίηση είναι σημαντική για την ερμηνεία των ευρημάτων. Τα category effects που εμφανίζονται μόνο σε μία εναλλακτική προδιαγραφή δεν μπορούν να θεωρηθούν εξίσου σταθερά με σχέσεις που

επαναλαμβάνονται σε περισσότερες μεθόδους. Για τον λόγο αυτό, τα αποτελέσματα των Negative Binomial models αντιμετωπίζονται ως συμπληρωματικές ενδείξεις και όχι ως ανατροπή του βασικού συμπεράσματος των κύριων LMM ότι η κατηγορία του Pull Request δεν αποτελεί σταθερό predictor των τελικών quality και security outcomes.

Διαφορετική οπτική παρείχε και το GEE model το οποίο εξέτασε το security share αντί για τον απόλυτο αριθμό ή την πυκνότητα των issues. Στην περίπτωση αυτή, το μέγεθος και η πολυπλοκότητα δεν παρουσίασαν στατιστικά σημαντική σχέση με τη σχετική αναλογία των security issues. Το αποτέλεσμα αυτό διαφοροποιείται από τη θετική σχέση της πολυπλοκότητας με τα quality issues και τη μικρή αρνητική σχέση της με το security density στο LMM, επειδή το GEE εξετάζει διαφορετικό ερώτημα, το οποίο δεν έγκειται στο ποσοτικό πλήθος των εμφανιζόμενων προβλημάτων, αλλά στο μέγεθος της μεταβολής της σύνθεσης των συνολικών issues προς την ασφάλεια.

Μεταξύ των μεθόδων των PCA και Factor Analysis εμφανίστηκαν επίσης διαφορές ως προς την ακριβή οργάνωση των πολυμεταβλητών διαστάσεων. Και οι δύο μέθοδοι ανέδειξαν μία σαφή διάσταση της πολυπλοκότητας και συντηρησιμότητας και μία διάσταση οργανώθηκε διαφορετικά. Στην PCA, το PC3 χαρακτηριζόταν κυρίως από την προσφερόμενη αξιοπιστία, αλλά ταυτόχρονα περιλάμβανε θετικές και αρνητικές συνεισφορές άλλων διαθέσιμων μεταβλητών. Στη Factor Analysis, αντίθετα, η αξιοπιστία οργανώθηκε καθαρότερα στον Factor 2 μαζί με τη συντηρησιμότητα και την επανάληψη του κώδικα, ενώ οι security metrics συγκεντρώθηκαν -σχεδόν ολοκληρωτικά- στον Factor 3.

Η διαφοροποίηση αυτή συνδέεται με τον διαφορετικό στόχο των δύο τεχνικών. Η PCA επιδιώκει να συνοψίσει τη συνολική διακύμανση των αρχικών μεταβλητών σε έναν μικρότερο αριθμό συνιστωσών, ενώ η Factor Analysis εστιάζει στην κοινή διακύμανση και στην αναζήτηση υποκείμενων λανθανόντων παραγόντων. Δεν αναμένεται, επομένως, οι δύο μέθοδοι να παράγουν ταυτόσημες δομές, ακόμη και όταν εφαρμόζονται στις ίδιες μεταβλητές.

Ιδιαίτερη διαφοροποίηση αφορά και τη σταθερότητα της security διάστασης. Στην κύρια PCA, το PC2 σχημάτισε έναν σχετικά καθαρό άξονα ασφάλειας, με υψηλές φορτίσεις τόσο των security issues όσο και των security hotspots. Στη Factor Analysis, ο αντίστοιχος Factor 3 κρίθηκε ως περισσότερο εύθραυστος, καθώς βασιζόταν σε ουσιαστικά δύο (2) indicators και τα security_issues_per_kloc παρουσίαζαν υψηλή uniqueness. Επιπλέον, στη size-adjusted Factor Analysis η ξεχωριστή Τρίτη διάσταση δεν διατηρήθηκε καθόλου. Κατά συνέπεια, η ύπαρξη μίας διακριτής security dimension υποστηρίζεται από τα δεδομένα, αλλά η ακριβής μορφή και η ανεξαρτησία της είναι λιγότερο σταθερές από τη διάσταση της πολυπλοκότητας και συντηρησιμότητας.

Συνοψίζοντας, οι διαφορές μεταξύ των μεθόδων εντοπίζονται κυρίως σε δευτερεύοντα ή περισσότερο ευαίσθητα ευρήματα. Στη στατιστική σημαντικότητα της πολυπλοκότητας για το security outcome, στις επιδράσεις των κατηγοριών των Pull Requests και στην ακριβή οργάνωση της πολυμεταβλητής δομής. Αντίθετα, τα κεντρικά συμπεράσματα σχετικά με τη σύνδεση πολυπλοκότητας και συντηρησιμότητας και με την ισχυρή project-level ετερογένεια περιμένουν σε

μεγάλο βαθμό ανεπηρέαστα. Η διάκριση αυτή επιτρέπει τα αποτελέσματα να αξιολογούνται με βάση τον βαθμό σταθερότητάς τους και όχι αποκλειστικά βάσει της στατιστικής σημαντικότητας που προκύπτει από μία μεμονωμένη μέθοδο.

Εύρημα	Μέθοδος / αποτέλεσμα α	Μέθοδος / αποτέλεσμα β	Ερμηνεία
Complexity → Security density	OLS: μη σημαντική	LMM < 0, p = 0,017	Η project-level δομή επηρεάζει την εκτίμηση
Κατηγορία PR	LMM: μη σημαντική	Negative Binomial: ορισμένα significant effects	Model-sensitive αποτέλεσμα
Μέγεθος / Complexity → Security share	GEE: μη σημαντικά	LMM εξετάζει διαφορετικό outcome	Δεν μεταβάλλεται απαραίτητα η σύνθεση των issues
Reliability dimension	PCA: PC3, περισσότερο μικτή	FA: Factor 2, καθαρότερη latent δομή	Διαφορετικός στόχος των δύο τεχνικών
Security dimension	PCA: σχετικά καθαρό PC2	FA: πιο εύθραυστος Factor 3	Μερική αλλά όχι πλήρης σταθερότητα;
Dimensionality μετά το size adjustment	PCA: 3 → 2 PCs	FA: 3 → 2 factors	Κοινή ευαισθησία στη μεταχείριση του code size

Πίνακας 8.9.2 – Κύριες διαφοροποιήσεις μεταξύ των στατιστικών και πολυμεταβλητών μεθόδων και η ερμηνευτική τους σημασία

8.9.3 Ισχυρότερα ευρήματα

Η συνολική αξιολόγηση των αποτελεσμάτων επιτρέπει να διακριθούν ορισμένα ευρήματα τα οποία παρουσιάζουν μεγαλύτερη εμπειρική σταθερότητα αντί άλλων. Ως ισχυρότερα δεν θεωρούνται απλώς εκείνα που εμφάνισαν το μικρότερο p-value σε μία μεμονωμένη ανάλυση, αλλά κυρίως όσα επαναλήφθηκαν σε διαφορετικές μεθόδους, διατήρησαν την ίδια κατεύθυνση και παρέμειναν σταθερά στους robustness και sensitivity ελέγχους. Με βάση το κριτήριο αυτό, τέσσερα αποτελέσματα αναδεικνύονται ως τα σημαντικότερα της έρευνας.

Το πιο ισχυρό εύρημα αφορά την έντονη διαφοροποίηση που παρατηρείται μεταξύ των projects. Στα null Linear Mixed Models, το ~87,8% της συνολικής διακύμανσης της log_quality_density και 92,2% της διακύμανσης της log_security_density αποδόθηκαν σε διαφορές μεταξύ projects. Ακόμη και μετά την εισαγωγή του μεγέθους, της πολυπλοκότητας και της κατηγορίας στην οποία το κάθε PR ανήκει, οι αντίστοιχοι ICC παρέμειναν ~83,5% και 92,3%. Παρόμοια υψηλή project-level συγκέντρωση εμφανίστηκε και στα μοντέλα των PCA component scores και Factor Analysis factor scores, ενώ το αποτέλεσμα παρέμεινε σταθερό και μετά τον περιορισμό της ανάλυσης σε projects με πέντε (5) τουλάχιστον έγκυρα PRs.

Η επαναληψιμότητα αυτή καθιστά την project-level ετερογένεια ένα από τα πλέον σταθερά και σαφή ευρήματα της έρευνας.

Το δεύτερο αντίστοιχα ισχυρό εύρημα αφορά τη σύνδεση της πολυπλοκότητας με την ποιότητα και τη συντηρησιμότητα του κώδικα. Στο OLS μοντέλο, η *log_complexity* παρουσίασε θετική σχέση με την πυκνότητα προβλημάτων ποιότητας ($\beta = 0,368$, $p < 0,001$), ενώ στο κύριο LMM η σχέση παρέμεινε θετική και παρόμοιου μεγέθους ($\beta = 0,398$, $p < 0,001$). Αφού παρήλθε ο περιορισμός του δείγματος μόνο σε project τα οποία πληρούν το κριτήριο των ελάχιστων PRs, έχει τεθεί σε πέντε (5) έγκυρα PRs, ο συντελεστής παρέμεινε θετικός και ισχυρός, περίπου 0,369. Η ίδια δομή εμφανίστηκε και στις πολυμεταβλητές αναλύσεις. Τόσο το PC1 της PCA όσο και ο Factor 1 της Factor Analysis χαρακτηρίζονταν από υψηλές φορτίσεις της *complexity*, *cognitive complexity* και *maintainability issues*. Η σύνδεση, επομένως, μεταξύ αυξημένης πολυπλοκότητας και μεγαλύτερης επιβάρυνσης συντηρησιμότητας υποστηρίζεται από διαφορετικές και μεθοδολογικά ανεξάρτητες προσεγγίσεις.

Ένα τρίτο σταθερό εύρημα είναι ότι η κατηγορία του Pull Request δεν αποτέλεσε ισχυρό και συνεπή predictor των τελικών επιπέδων ποιότητας ή ασφάλειας. Στα κύρια OLS και LMM, οι κατηγορίες *security* και *both* δεν διαφοροποιήθηκαν στατιστικά σημαντικά από την κατηγορία *quality*. Αντίστοιχη απουσία σημαντικών διαφορών παρατηρήθηκε στα μοντέλα των PCA component scores και Factor Analysis factor scores, ενώ η προβολή των Pull Requests στον χώρο PC1-PC2 παρουσίασε έντονη αλληλοεπικάλυψη μεταξύ των τριών κατηγοριών. Παρότι που τα Negative Binomial models ανέδειξαν ορισμένα category effects, δεν αναπαράχθηκαν στα κύρια μοντέλα, με αποτέλεσμα να θεωρούνται περισσότερο ευαίσθητα στην εκάστοτε προδιαγραφή. Συνεπώς, η αρχική ταξινόμηση ενός Pull Request ως *quality*, *security* ή *both* δεν φαίνεται να αρκεί για την πρόβλεψη της τελικής κατάστασης του κώδικα.

Σημαντικό, επίσης, εύρημα είναι η οργάνωση των μετρικών ποιότητας και ασφάλειας σε μία πολυδιάστατη και όχι μονοδιάστατη δομή. Η κύρια Factor Analysis κατέληξαν ανεξάρτητα στη διατήρηση τριών διαστάσεων. Και στις δύο αναλύσεις προέκυψε μία ισχυρή διάσταση πολυπλοκότητας και συντηρησιμότητας, ενώ οι *security-related metrics* σχημάτισαν μία περισσότερο διακριτή διάσταση. Η συμφωνία αυτή υποδεικνύει ότι ποιότητα και ασφάλεια δεν μπορούν να περιγραφούν ως δύο πλήρως ταυτόσημες εκφάνσεις ενός και μοναδικού χαρακτηριστικού. Παράλληλα, η μετάβαση από τρεις (3) σε δύο (2) διαστάσεις στις size-adjusted sensitivity analyses δείχνει ότι ο ακριβής αριθμός των dimensions δεν εμφανίζεται απόλυτα σταθερός. Το ισχυρότερο συμπέρασμα, άρα, δεν αφορά την ύπαρξη τριών διαστάσεων αυτή καθ' αυτή, αλλά την ύπαρξη περισσότερων της μίας διακριτών αλλά αλληλένδετων δομών.

Ένα ακόμα ουσιαστικό εύρημα προέκυψε ήδη από το αρχικό στάδιο επεξεργασίας των δεδομένων. Η σύγκριση μεταξύ των καταστάσεων *pr_base* και *pr_closed* έδειξε ότι 94% έως και 99,9% του συνόλου δεδομένων Pull Requests, ανάλογα με την εκάστοτε μετρική, δεν παρουσίαζαν μεταβολή μεταξύ των δύο καταστάσεων. Το αποτέλεσμα αυτό δεν αποτελεί εύρημα για την κατεύθυνση της σχέσης ποιότητας-ασφάλειας, αλλά είναι ιδιαίτερα σημαντικό ως προς τα όρια της

διαθέσιμης εμπειρικής πληροφορίας. Η απουσία επαρκούς within-PR μεταβλητότητας απέκλεισε μία αξιόπιστη before-after προσέγγιση και οδήγησε τη μελέτη σε cross-sectional ανάλυση της τελικής κατάστασης των Pull Requests. Συνεπώς, ένα από τα ισχυρότερα συμπεράσματα της έρευνας είναι και ότι τα συγκεκριμένα δεδομένα επιτρέπουν τη διεξαγωγή περισσότερο σαφών συμπερασμάτων για τις διαφορές μεταξύ PRs και projects παρά για τις μεταβολές κατά τη διάρκεια του ίδιου PR.

Ως εκ των παραπάνω αναφερόμενων, η εμπειρική εικόνα που προκύπτει κρίνεται συνεκτική. Το project αποτελεί κυρίαρχη πηγή διαφοροποίησης, η πολυπλοκότητα σταθερά συνδέεται με την επιβάρυνση της ποιότητας και της συντηρησιμότητας, η κατηγορία του Pull Request παρουσιάζει περιορισμένη και μη σταθερή προβλεπτική αξία, ενώ οι μετρικές ποιότητας και ασφάλειας οργανώνονται σε περισσότερες από μία διακριτές πολυμεταβλητές διαστάσεις. Από την άλλη πλευρά, τα ευρήματα τα οποία αφορούν τις πιο λεπτές σχέσεις της ασφάλειας ή τις category effects απαιτούν πιο προσεκτική ερμηνεία, καθώς εμφανίζουν μεγαλύτερη εξάρτηση από το χρησιμοποιούμενο στατιστικό μοντέλο.

Τα παραπάνω ευρήματα συνοψίζονται στον παρακάτω πίνακα αποτελεσμάτων και ευρημάτων:

Ισχυρό εύρημα	Βασική τεκμηρίωση	Βαθμός σταθερότητας
Ισχυρή project-level ετερογένεια	ICC έως 92,3%, PCA/FA score models, ≥ 5 PR sensitivity	Πολύ υψηλός
Complexity $\Leftrightarrow$ quality / maintainability	OLS, LMM, PCA, FA, sensitivity analysis	Πολύ υψηλός
Περιορισμένη επίδραση της κατηγορίας PR	OLS, LMM, PCA/FA scores, PC1-PC2 overlap	Υψηλός
Πολυδιάστατη δομή ποιότητας-ασφάλειας	PCA και FA, 3 κύριες διαστάσεις	Υψηλός
Ακριβής αριθμός dimensions	3 κύρια $\Rightarrow$ 2 size-adjusted	Μέτριος
Complexity $\Leftrightarrow$ Security	Σημαντική μόνο στο LMM, μικρό effect	Περιορισμένος

Πίνακας 8.9.3 – Ιεράρχηση των κυριότερων εμπειρικών ευρημάτων με βάση τη συνέπεια και τη σταθερότητά τους μεταξύ των διαφορετικών αναλυτικών προσεγγίσεων

8.9.4 Αποτελέσματα ανά ερευνητικό ερώτημα και υπόθεση

Η ολοκλήρωση των επιμέρους αναλύσεων επιτρέπει την αντιστοίχιση των εμπειρικών ευρημάτων με τα ερευνητικά ερωτήματα και τις υποθέσεις που τέθηκαν κατά τον σχεδιασμό της παρούσας έρευνας. Αυτή η αντιστοίχιση κρίνεται ιδιαίτερα σημαντική, καθώς ορισμένα από τα αρχικά ερωτήματα βασίζονται στη σύγκριση μεταξύ των καταστάσεων pr_base και pr_closed. Η πολύ περιορισμένη μεταβλητότητα μεταξύ των δύο snapshots οδήγησε σε cross-sectional αναλυτική στρατηγική και, επομένως, τα ερωτήματα και οι υποθέσεις που αφορούν άμεσα

μεταβολές δεν μπορούν να αξιολογηθούν με τον ίδιο τρόπο όπως αρχικά είχε προβλεφθεί και οριστεί.

Παρακάτω παρουσιάζονται τα αποτελέσματα ανά **ερευνητικό ερώτημα**:

Ως προς το πρώτο ερευνητικό ερώτημα (υπενθυμίζεται, «Υπάρχει στατιστικά σημαντική και ισχυρή σχέση μεταξύ δεικτών ποιότητας και δεικτών ασφάλειας λογισμικού σε Pull Requests;»), τα αποτελέσματα δεν τεκμηριώνουν την ύπαρξη μίας μοναδικής, ισχυρής και ομοιόμορφης σχέσης μεταξύ των δεικτών ποιότητας και ασφάλειας. Η PCA και η Factor Analysis από την αντίθετη πλευρά, δείχνουν ότι οι συγκεκριμένες μετρικές συμμετέχουν σε μία ευρύτερη πολυδιάστατη δομή. Οι δείκτες πολυπλοκότητας και συντηρησιμότητας συγκεντρώνονται κυρίως σε μία κοινή διάσταση, ενώ οι δείκτες ασφάλειας σχηματίζουν μία περισσότερο διακριτή διάσταση. Η ποιότητα και η ασφάλεια, επαγωγικά, δεν εμφανίζονται ως πλήρως ανεξάρτητες έννοιες, αλλά ούτε και ως μία ενιαία διάσταση η οποία δύναται να μεταβληθεί με τον ίδιο τρόπο σε κάθε περίπτωση. Η σχέση τους χαρακτηρίζεται περισσότερο ως σύνθετη και πολυδιάστατη παρά ως μία μοναδική ισχυρή συσχέτιση.

Σχετικά με το δεύτερο ερευνητικό ερώτημα το οποίο και έχει τεθεί («Ποια χαρακτηριστικά του Pull Request, της τελικής κατάστασης του αναλυόμενου κώδικα και του project -όπως η κατηγορία του PR, το μέγεθος του αναλυόμενου κώδικα, η πολυπλοκότητα, η γνωστική πολυπλοκότητα και η διπλοτυπία- σχετίζονται με αυξημένη πυκνότητα προβλημάτων ποιότητας και ασφάλειας;»), η πολυπλοκότητα αποτέλεσε το χαρακτηριστικό με την πιο σταθερή σχέση με τα προβλήματα ποιότητας. Στο κύριο LMM, η $\log_complexity$ συνδέθηκε θετικά, με την πυκνότητα των quality issues ($\beta = 0,398, p < 0,001$), ενώ το $\log_ncloc$, το οποίο εκφράζει το μέγεθος του αναλυόμενου code snapshot και όχι το συνολικό μέγεθος της αλλαγής του Pull Request, παρουσίασε αρνητική σχέση με $\beta = -0,152$ και $p < 0,001$. Για τα security issues, η πολυπλοκότητα εμφάνισε μία πολύ μικρότερη αρνητική σχέση ($\beta = -0,011, p = 0,017$), ενώ το μέγεθος δεν ήταν στατιστικά σημαντικό. Η κατηγορία στην οποία το κάθε Pull Request ανήκει, δεν παρουσίασε σταθερή ανεξάρτητη επίδραση στα κύρια μοντέλα. Παράλληλα, οι πολυμεταβλητές αναλύσεις έδειξαν ότι cognitive complexity, complexity, maintainability issues και duplication συμμετέχουν σε κοινές δομικές διαστάσεις. Η πολυπλοκότητα, επομένως, και το project αποτελούν τα χαρακτηριστικά που εμφανίζουν την ισχυρότερη και περισσότερο συνεπή σύνδεση με τα εξεταζόμενα outcomes.

Το τρίτο επί σειράς ερευνητικό ερώτημα («Σε ποιο βαθμό οι διαφορές μεταξύ projects εξηγούν τη διακύμανση των προβλημάτων ποιότητας και ασφάλειας;») απαντάται ιδιαίτερα σαφώς από την εφαρμογή των Linear Mixed Models. Στα null models, περίπου 87,8% της συνολικής διακύμανσης του $\log_quality_density$ και ~92,2% της διακύμανσης του $\log_security_density$ εντοπίζονταν στο επίπεδο του project. Ακόμη και μετά την εισαγωγή των fixed effects, οι αντίστοιχοι ICC παρέμειναν εξαιρετικά υψηλοί, σε περίπου 83,5% και 92,3%. Οι διαφορές, επομένως, μεταξύ projects αποτελούν μία από τις κυριότερες πηγές ετερογένειας τόσο για την ποιότητα όσο και για την ασφάλεια.

Για το τέταρτο ερευνητικό ερώτημα («Μπορούν οι πολλές επιμέρους μετρικές ποιότητας και ασφάλειας να αναχθούν σε μικρότερο αριθμό υποκείμενων διαστάσεων;»), τα αποτελέσματα υποστηρίζουν σαφώς τη δυνατότητα αναγωγής των επιμέρους μετρικών σε μικρότερο αριθμό ευρύτερων διαστάσεων. Η κύρια PCA διατήρησε τρεις principal components, οι οποίες εξηγούσαν συνολικά ~77,5% της διακύμανσης, ενώ η Factor Analysis ανέδειξε με την σειρά της τρεις (3) factors. Οι διαστάσεις αυτές συνδέθηκαν κυρίως με την πολυπλοκότητα και τη συντηρησιμότητα, την ασφάλεια και την αξιοπιστία. Εντούτοις, οι size-adjusted αναλύσεις οδήγησαν σε δύο διαστάσεις και στις δύο μεθόδους, γεγονός που δείχνει ότι ο ακριβής αριθμός των υποκείμενων διαστάσεων παρουσιάζει κάποια ευαισθησία στον τρόπο προσαρμογής ως προς το μέγεθος του κώδικα.

Το πέμπτο ερευνητικό ερώτημα («Μεταβολές της ποιότητας από την αρχική στην τελική κατάσταση ενός Pull Request συνδέονται με αντίστοιχες μεταβολές της ασφάλειας;»), το οποίο αφορά τη σχέση μεταξύ ποιότητας και μεταβολών ασφάλειας από την αρχική προς την τελική κατάσταση του Pull Request, δεν κατέστη δυνατό να απαντηθεί αξιόπιστα με τα διαθέσιμα δεδομένα. Η σύγκριση pr_base και pr_closed έδειξε ότι 94% έως και 99,9% των παρατηρήσεων, σύμφωνα με την εκάστοτε μετρική, παρουσίαζαν σχεδόν μηδενική ή μηδενική μεταβολή. Η ανεπαρκής διακύμανση των delta variables δεν επέτρεψε την εφαρμογή μιας ουσιαστικής before-after ανάλυσης. Η μη απάντηση του συγκεκριμένου ερωτήματος αποτελεί, επομένως, περιορισμό που προκύπτει από τα χαρακτηριστικά των δεδομένων και όχι ένδειξη απουσίας πραγματικής σχέσης μεταξύ μεταβολών ποιότητας και ασφάλειας.

Ολοκληρώνοντας, ως προς το έκτο και τελευταίο ερευνητικό ερώτημα που τέθηκε («Υπάρχουν κατηγορίες Pull Requests στις οποίες η σχέση ποιότητας-ασφάλειας διαφοροποιείται;»), δεν προέκυψε σταθερή ένδειξη ότι οι κατηγορίες quality, security και both διαφοροποιούν ουσιαστικά τη σχέση μεταξύ ποιότητας και ασφάλειας. Στα κύρια OLS και LMM, καθώς και στα μοντέλα των PCA component scores και Factor Analysis factor scores, οι category effects δεν ήταν στατιστικά σημαντικές. Η σημαντική αλληλοεπικάλυψη των κατηγοριών στον χώρο PC1-PC2 ενισχύει την ίδια εικόνα. Ορισμένες εκ των διαφοροποιήσεων εμφανίστηκαν στα Negative Binomial models, όμως δεν αναπαράχθηκαν στα κύρια μοντέλα και συνεπώς αντιμετωπίζονται ως model-sensitive και όχι ως ένα σταθερό εύρημα.

Έπειτα της ολοκλήρωσης της περιγραφής συσχέτισης των ερευνητικών ερωτημάτων, τοποθετούνται και αξιολογούνται οι **ερευνητικές υποθέσεις** όπως αυτές ορίστηκαν στην έναρξη της παρούσας βιβλιογραφίας:

Με βάση τα παραπάνω αποτελέσματα, η κατάσταση των έξι ερευνητικών υποθέσεων συνοψίζεται παρακάτω ως εξής:

Υπόθεση	Αξιολόγηση	Εμπειρική τεκμηρίωση
H1 – Σχέση μεταβολών ποιότητας και ασφάλειας	Δεν κατέστη δυνατό να ελεγχθεί αξιόπιστα	Οι διαφορές base-closed ήταν μηδενικές περίπου στο 94-99,9% του συνολικού δείγματος των PRs, με αποτέλεσμα ανεπαρκή

		μεταβλητότητα για delta analysis.
H2 – Επίδραση χαρακτηριστικών του PR	Μερικώς υποστηρίζεται υπό την τελική cross-sectional προσέγγιση	Η πολυπλοκότητα και το μέγεθος σχετίστηκαν σημαντικά με το quality density, ενώ η πολυπλοκότητα παρουσίασε μικρή σχέση και με το security density. Η αρχική διατύπωση ως προς τις «μεταβολές» δεν κατέστη δυνατό να ελεγχθεί άμεσα.
H3 – Project-level ετερογένεια	Ισχυρά υποστηρίζεται	Οι ICC των null και των πλήρων random-intercept LMM ήταν ιδιαίτερα υψηλοί και για τα δύο outcomes. Στα null models, ~87,8% της διακύμανσης του log_quality_density και ~92,2% της διακύμανσης του log_security_density εντοπίζονταν στο επίπεδο του project, ενώ στα πλήρη μοντέλα οι αντίστοιχες τιμές παρέμειναν ~83,5% και ~92,3%. Τα αποτελέσματα τεκμηριώνουν έντονη project-level ετερογένεια στα επίπεδα των quality και security outcomes.
H4 – Ύπαρξη λανθανουσών δομών	Υποστηρίζεται επαρκώς	PCA και Factor Analysis ανέδειξαν μικρότερο αριθμό πολυμεταβλητών διαστάσεων με ερμηνεύσιμη δομή.
H5 – Υπεροχή ιεραρχικής μοντελοποίησης	Υποστηρίζεται επαρκώς	Η πολύ υψηλή project-level ομαδοποίηση και η διαφοροποίηση ορισμένων αποτελεσμάτων μεταξύ OLS και LMM δείχνουν ότι η ιεραρχική δομή πρέπει να μοντελοποιείται ρητά
H6 – Συνεργατικές και ανταγωνιστικές μεταβολές	Δεν κατέστη δυνατό να ελεγχθεί επαρκώς και αξιόπιστα	Η σχεδόν πλήρης απουσία μεταβολής μεταξύ pr_base και pr_closed δεν επέτρεψε να ταξινομηθούν αξιόπιστα τα PRs σε συνεργατικές ή ανταγωνιστικές μεταβολές ποιότητας-ασφάλειας.

Η H1 συμπεραίνουμε άρα ότι δεν απορρίπτεται με την έννοια ότι αποδείχθηκε πως δεν υπάρχει σχέση μεταξύ των μεταβολών. Αντίθετα, πρέπει να χαρακτηριστεί ως μη αξιολογήσιμη με επαρκή εμπειρική ισχύ στο συγκεκριμένο dataset. Η διάκριση αυτή είναι σημαντική, καθώς η απουσία μεταβλητότητας στα δεδομένα δεν ισοδυναμεί με εμπειρική απόδειξη απουσίας σχέσης.

Παρόμοια με την H1 αντιμετώπιση απαιτεί και η H2. Η αρχική υπόθεση αναφερόταν στην επίδραση χαρακτηριστικών των Pull Requests στην έκταση των μεταβολών ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού. Αυτό το μέρος δεν μπόρεσε να ελεγχθεί. Μολαταύτα, μετά τη μετάβαση στην cross-sectional προσέγγιση, προέκυψε σαφής τεκμηρίωση ότι χαρακτηριστικά του Pull Request σχετίζονται με τα τελικά επίπεδα πυκνότητας προβλημάτων, ιδιαίτερα στην περίπτωση της πολυπλοκότητας και του quality outcome. Για τον λόγο αυτό, η υπόθεση υποστηρίζεται αποκλειστικά ως προς την αναθεωρημένη επιχειρησιακή της μορφή.

Η H3 υποστηρίζεται έντονα όσον αφορά την ύπαρξη διαφορών μεταξύ projects. Η εξαιρετικά υψηλή between-project διακύμανση εμφανίστηκε σε διαφορετικά outcomes και αναλυτικές προσεγγίσεις. Τα κύρια μοντέλα ήταν, παρ' όλα αυτά, random-intercept και όχι επιτυχώς εκτιμημένα random-slope models. Τα αποτελέσματα αποδεικνύουν ότι τα βασικά επίπεδα ποιότητας και ασφάλειας διαφέρουν σημαντικά μεταξύ projects, αλλά δεν επιτρέπουν εξίσου ισχυρό συμπέρασμα ότι η ίδια η σχέση μεταξύ συγκεκριμένων predictors και outcomes μεταβάλλεται συστηματικά μεταξύ των διαφόρων projects.

Η H4 αποτελεί μία από τις σαφέστερα υποστηριζόμενες υποθέσεις. Τόσο η PCA όσο και η Factor Analysis τεχνικές έδειξαν ότι οι επιμέρους μετρικές δεν λειτουργούν ως ένα σύνολο πλήρως ανεξάρτητων δεικτών, αλλά οργανώνονται σε ευρύτερες πολυμεταβλητές διαστάσεις. Η σύγκλιση των δύο τεχνικών ως προς τη διάσταση πολυπλοκότητας/συντηρησιμότητας και την ύπαρξη ξεχωριστής security-related δομής ενισχύει το συμπέρασμα που έχουμε εξάγει.

Αντίστοιχα, η H5 υποστηρίζεται από τα υπολογισμένα αποτελέσματα. Οι εξαιρετικά υψηλοί ICC δείχνουν ότι η παραδοχή πλήρους ανεξαρτησίας των Pull Requests δεν περιγράφει επαρκώς τη δομή του δείγματος. Η σχέση της πολυπλοκότητας με το security outcome δεν ήταν σημαντική στο OLS μοντέλο αλλά έγινε σημαντική στην υλοποίηση και εκτέλεση του LMM μετά τη ρητή μοντελοποίηση της project-level ετερογένειας. Βάσει των παραπάνω, μπορούμε να συμπεράνουμε ότι τα LMM παρέχουν μία πιο πλήρης αναπαράσταση της δομής των δεδομένων, χωρίς αυτό να σημαίνει ότι τα cluster-robust OLS αποτελέσματα στερούνται χρησιμότητας ως baseline.

Η H6, τέλος, δεν μπορεί να θεωρηθεί ούτε επιβεβαιωμένη ούτε απορριφθείσα. Η διερεύνηση συνεργατικών μεταβολών, όπου ποιότητα και ασφάλεια μεταβάλλονται προς την ίδια κατεύθυνση και ανταγωνιστικών μεταβολών. Αυτές μεταβάλλονται κινούνται προς αντίθετες κατευθύνσεις, απαιτώντας επαρκή before-after μεταβλητότητα. Η συνθήκη αυτή δεν ικανοποιήθηκε στο διαθέσιμο σύνολο δεδομένων. Η συγκεκριμένη υπόθεση (H6) παραμένει ένα σημαντικό αντικείμενο προς μελλοντική έρευνα με δεδομένα που καταγράφουν ουσιαστικότερες μεταβολές κατά τον κύκλο ζωής των Pull Requests.

Κλείνοντας, τα αποτελέσματα παρέχουν ισχυρότερη υποστήριξη στις υποθέσεις που αφορούν την project-level ετερογένεια, την ιεραρχική δομή των δεδομένων και την ύπαρξη ευρύτερων πολυμεταβλητών διαστάσεων. Αντίθετα, οι υποθέσεις οι οποίες βασίζονται στη χρονική μεταβολή μεταξύ pr_base και pr_closed δεν ήταν δυνατό να αξιολογηθούν αξιόπιστα. Το γεγονός αυτό αποτελεί ουσιαστικό

εμπειρικό εύρημα τόσο για τα συμπεράσματα που υποστηρίζονται από τα δεδομένα όσο και για τα όρια μέσα στα οποία αυτά δύνανται να γενικευθούν.

Συζήτηση

Το παρόν κεφάλαιο επικεντρώνεται στην ερμηνεία και σύνθεση των εμπειρικών αποτελεσμάτων που παρουσιάστηκαν στο προηγούμενο κεφάλαιο, με στόχο να αποσαφηνιστεί η σημασία τους σε σχέση με το βασικό ερευνητικό πρόβλημα της αλληλεπίδρασης μεταξύ ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού. Η συζήτηση δεν περιορίζεται στην απλή επανάληψη των στατιστικών ευρημάτων, αλλά επιχειρεί να εξηγήσει τον τρόπο και τον λόγο που προκύπτουν τα παρατηρούμενα πρότυπα και ποια είναι η ευρύτερη σημασία τους.

Έμφαση δίνεται στη σχέση μεταξύ ποιότητας και ασφάλειας, στον ρόλο της πολυπλοκότητας, της συντηρησιμότητας και των χαρακτηριστικών των PRs, καθώς και στη σημαντική διαφοροποίηση που παρατηρείται μεταξύ των διάφορων projects. Ερμηνεύονται, παράλληλα, τα αποτελέσματα των PCA και Factor Analysis, εξετάζονται οι πιθανές πρακτικές επιπτώσεις των ευρημάτων και συζητούνται επίσης οι βασικοί περιορισμοί και οι απειλές ως προς την εγκυρότητα της έρευνας.

Στόχος του παρόντος τμήματος της βιβλιογραφίας είναι να μετατρέψει τα επιμέρους στατιστικά αποτελέσματα σε μία συνεκτική ερμηνευτική εικόνα για το πώς η ποιότητα και η ασφάλεια συνδέονται, διαφοροποιούνται και επηρεάζονται από το τεχνικό και project-level περιβάλλον μέσα στο οποίο εξελίσσεται ο πηγαίος κώδικας.

9.1 Επιστροφή στο ερευνητικό πρόβλημα

Εναρκτήριοις της παρούσας έρευνας στάθηκε το ερώτημα του τρόπου με τον οποίο η ποιότητα και η ασφάλεια του λογισμικού συνδέονται και αλληλοεπιδρούν κατά την εξέλιξη του πηγαίου κώδικα, με το Pull Request να αποτελεί τη βασική μονάδα παρατήρησης. Αν και οι δύο διαστάσεις συνηθίζεται να εξετάζονται ανεξάρτητα, συνυπάρχουν μέσα στο ίδιο τεχνικό περιβάλλον και επηρεάζονται από κοινά χαρακτηριστικά, μεταξύ των οποίων η πολυπλοκότητα, το μέγεθος, η συντηρησιμότητα και το project στο οποίο πραγματοποιείται κάθε αλλαγή. Βασικός στόχος ήταν να διερευνηθεί η πιθανότητα της από κοινού μεταβολής της ποιότητας και της ασφάλειας, το πλήθος των παραγόντων που σχετίζονται με τα επίπεδά τους και κατά πόσο η σχέση αυτή παραμένει σταθερή μεταξύ διαφορετικών έργων λογισμικού.

Η αρχική ερευνητική προσέγγιση είχε σχεδιαστεί με λογική before-after, αξιοποιώντας τις καταστάσεις `pr_base` και `pr_closed` για κάθε μονάδα PR. Τα δεδομένα, ωστόσο, εμφάνισαν τις δύο αυτές καταστάσεις ως σχεδόν πανομοιότυπες για τη μεγάλη πλειονότητα των μετρικών, με περίπου 94 έως και 99,9% των PRs να εμφανίζουν ελάχιστη έως και μηδενική ποσοστιαία μεταβολή ανάλογα με τη μεταβλητή. Η περιορισμένη αυτή μεταβλητότητα δεν επέτρεψε αξιόπιστη ανάλυση των μεταβολών ποιότητας και ασφάλειας. Για τον λόγο αυτό, η έρευνα

προσανατολίστηκε τελικά σε cross-sectional ανάλυση της τελικής κατάστασης pr_closed, εστιάζοντας στις διαφορές μεταξύ Pull Requests και μεταξύ projects.

Τα αποτελέσματα δεν υποστηρίζουν μία απλή/μονοδιάστατη σχέση μεταξύ ποιότητας και ασφάλειας. Αντιθέτως, αναδεικνύουν μία πολυδιάστατη δομή, μέσα στην οποία οι δύο έννοιες παρουσιάζουν κοινά σημεία αλλά και σαφείς μεταξύ τους διαφοροποιήσεις. Η πολυπλοκότητα συνδέθηκε σταθερά με αυξημένη επιβάρυνση της ποιότητας και της συντηρησιμότητας, ενώ η σχέση της με την ασφάλεια ήταν σαφώς ασθενέστερη και περισσότερο εξαρτημένη από τη χρησιμοποιούμενη στατιστική προσέγγιση. Η κατηγορία ενός Pull Request ως quality, security ή both δεν αποτέλεσε σταθερό παράγοντα πρόβλεψης των τελικών επιπέδων quality και security issues.

Ιδιαίτερα σημαντικό εύρημα εδώ αποτέλεσε η έντονη διαφοροποίηση μεταξύ των projects. Τα υψηλά ICC των Linear Mixed Models έδειξαν ότι πολύ μεγάλο μέρος της συνολικής διακύμανσης τόσο στην ποιότητα όσο και στην ασφάλεια εντοπίζεται στο επίπεδο του project. Αυτό το αποτέλεσμα υποδηλώνει ότι η κατάσταση ενός PR δεν μπορεί να αποσυνδεθεί από το ευρύτερο περιβάλλον του repository στο οποίο ανήκει. Χαρακτηριστικά του ίδιου του έργου, τα οποία δεν αποτυπώνονται πλήρως από τις διαθέσιμες PR-level μεταβλητές, φαίνεται να συνδέονται με σημαντικό μέρος της παρατηρούμενης ετερογένειας.

Η ίδια εικόνα ενισχύθηκε και από τις πολυμεταβλητές αναλύσεις. Η PCA και η Factor Analysis ανέδειξαν επαναλαμβανόμενες δομές που σχετίζονται με την πολυπλοκότητα και τη συντηρησιμότητα, την ασφάλεια και την αξιοπιστία, χωρίς να προκύπτει ένας μοναδικός κοινός άξονας που να περιγράφει ταυτόχρονα όλες τις εμφανιζόμενες/εξεταζόμενες διαστάσεις. Τα αποτελέσματα αυτά υποδεικνύουν ότι η σχέση ποιότητας-ασφάλειας είναι περισσότερο σύνθετη από μία απλή θετική ή αρνητική συσχέτιση και ότι η κατανόησή της απαιτεί ταυτόχρονη εξέταση τόσο των επιμέρους τεχνικών χαρακτηριστικών όσο και του project-level context.

Η έρευνα καταλήγει στο συμπέρασμα ότι ποιότητα και ασφάλεια πηγαίου κώδικα και λογισμικού είναι σχετιζόμενες αλλά διακριτές διαστάσεις, οι οποίες επηρεάζονται από κοινά χαρακτηριστικά του κώδικα χωρίς να ακολουθούν τα ίδια πρότυπα. Παράλληλα, αναδεικνύεται ο καθοριστικός ρόλος της πολυπλοκότητας και, κυρίως, του project στο οποίο πραγματοποιείται κάθε αλλαγή. Η εικόνα αυτή μετατοπίζει το ερευνητικό ενδιαφέρον από την αναζήτηση μιας απλής σχέσης μεταξύ δύο ομάδων μετρικών προς την κατανόηση ενός ευρύτερου συστήματος αλληλεπιδράσεων, μέσα στο οποίο η ποιότητα και η ασφάλεια συνδιαμορφώνονται από χαρακτηριστικά τόσο του PR όσο και συνολικά του software project.

9.2 Ερμηνεία της σχέσης ποιότητας και ασφάλειας

Ένα εκ των βασικών ζητημάτων της παρούσας έρευνας έγκειται στο ερώτημα του σημείου όπου η ποιότητα και η ασφάλεια του λογισμικού αποτελούν διαστάσεις που κινούνται παράλληλα ή παρουσιάζουν διαφορετικές, ενδεχομένως και

ανταγωνιστικές μεταξύ τους, συμπεριφορές. Τα αποτελέσματα δεν υποστηρίζουν μία απλή θετική ή αρνητική μεταξύ τους σχέση. Αντίθετα, αναδεικνύουν μία σύνθετη και πολυδιάστατη συσχετιστική δομή, στην οποία ορισμένες μετρικές εμφανίζουν κοινά πρότυπα μεταβλητότητας, ενώ άλλες οργανώνονται σε περισσότερο διακριτές διαστάσεις.

Η εικόνα αυτή αποτυπώνεται ιδιαίτερα στις πολυμεταβλητές αναλύσεις που διεξήχθησαν. Στην PCA, οι μετρικές πολυπλοκότητας, συντηρησιμότητας και διπλοτυπίας συγκεντρώθηκαν κυρίως σε διαφορετική συνιστώσα από τα security issues και security hotspots. Αντίστοιχη διάκριση προέκυψε και από τη Factor Analysis. Ποιότητα και ασφάλεια, άρα, δεν φαίνεται να αποτελούν διαφορετικές εκφράσεις μίας ενιαίας υποκείμενης ιδιότητας του κώδικα, χωρίς όμως αυτό να σημαίνει ότι λειτουργούν πλήρως ανεξάρτητα.

Το project-level context αποτελεί το κύριο κοινό σημείο των δύο διαστάσεων (ποιότητα, ασφάλεια πηγαίου κώδικα και λογισμικού). Η ιδιαίτερα υψηλή μεταξύ-project διακύμανση τόσο στα quality όσο και στα security outcomes δείχνει ότι χαρακτηριστικά του εκάστοτε έργου συνδέονται σημαντικά και με τις δύο διαστάσεις. Οι επιμέρους PR-level predictors δεν επηρεάζουν, αντίθετα, καθολικά τις δύο διαστάσεις με τον ίδιο τρόπο. Χαρακτηριστικό παράδειγμα εδώ αποτελεί η πολυπλοκότητα, η οποία συνδέθηκε σταθερά με αυξημένη πυκνότητα προβλημάτων ποιότητας, ενώ η σχέση της με την ασφάλεια ήταν σαφώς ασθενέστερη και στο κύριο LMM κατείχε μικρή αρνητική κατεύθυνση.

Αυτά τα αποτελέσματα αφήνουν περιθώριο για την ύπαρξη synergies (συνεργειών), καθώς πρακτικές οι οποίες μειώνουν την πολυπλοκότητα και βελτιώνουν τη συντηρησιμότητα μπορούν να δημιουργούν ευνοϊκότερο τεχνικό περιβάλλον και για την ασφάλεια. Τα δεδομένα εδώ, όμως, δεν δείχνουν ότι μία βελτίωση της ποιότητας συνοδεύεται συστηματικά από αντίστοιχη βελτίωση της ασφάλειας. Οι δύο διαστάσεις φαίνεται να επηρεάζονται από κοινούς παράγοντες, αλλά όχι με την ίδια ένταση ή κατεύθυνση.

Ακόμη μεγαλύτερη προσοχή απαιτείται ως προς τα πιθανά εμφανιζόμενα trade-offs. Η αρχική ερευνητική σχεδίαση επέτρεπε θεωρητικά τη διερεύνηση περιπτώσεων όπου η βελτίωση της μίας διάστασης συνοδεύεται από επιδείνωση της άλλης, μέσω της σύγκρισης pr_base και pr_closed. Πρακτικά όμως, περίπου το εύρος μεταξύ 94% και 99,9% των παρατηρήσεων, σύμφωνα με την εκάστοτε μετρική και την φύση της, δεν παρουσίαζαν μεταβολή μεταξύ των δύο snapshots. Η περιορισμένη αυτή μεταβλητότητα δεν επέτρεψε αξιόπιστη ταξινόμηση των Pull Requests σε περιπτώσεις ταυτόχρονης βελτίωσης, ταυτόχρονης επιδείνωσης ή αντιστάθμισης ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού. Που αυτός υλοποιεί.

Κατά συνέπεια, η παρούσα έρευνα δεν παρέχει επαρκή στοιχεία για την επιβεβαίωση ή την απόρριψη της ύπαρξης πραγματικών synergies και trade-offs σε επίπεδο μεταβολών. Παρά τον συγκεκριμένο περιορισμό, τα διαθέσιμα cross-sectional δεδομένα επιτρέπουν να εξεταστεί κατά πόσο οι quality και security indicators παρουσιάζουν διαφορετικά πρότυπα συσχέτισης και κατά πόσο συνδέονται με διαφορετικό τρόπο με τα εξεταζόμενα χαρακτηριστικά του κώδικα και του project-level context.

Η περιορισμένη σημασία που έλαβε ο κατηγορικός χαρακτηρισμός (quality-security-both) στα κύρια στατιστικά μοντέλα ενισχύει αυτή την ερμηνεία. Η αρχική ταξινόμηση ενός Pull Request δεν ήταν αρκετή για να προβλέψει το τελικό του προφίλ ως προς τα quality και security issues όταν λαμβάνονταν υπόψη η πολυπλοκότητα, το μέγεθος και το project. Η αλληλεπίδραση των δύο διαστάσεων φαίνεται συνεπώς να εξαρτάται περισσότερο από την δομή του ίδιου του πηγαίου κώδικα και το περιβάλλον στο οποίο υπάρχει, αναπτύσσεται και εξελίσσεται το έργο, χωρίς να λαμβάνεται υπόψη στο ίδιο βεληνεκές ο δηλωμένος σκοπός της αλλαγής.

Τα αποτελέσματα υποστηρίζουν μία σχέση μερικής αλληλεξάρτησης μεταξύ ποιότητας και ασφάλειας. Οι δύο διαστάσεις μοιράζονται κοινό τεχνικό και project-level περιβάλλον, αλλά δεν ακολουθούν εξ ορισμού τα ίδια πρότυπα. Δεν προκύπτει επομένως ούτε πλήρης σύμπτωση ούτε σαφής ανταγωνισμός μεταξύ τους. Η ακριβέστερη διερεύνηση συνεργατικών και ανταγωνιστικών μεταβολών απαιτεί μελλοντικά δεδομένα με ουσιαστικότερη διαφοροποίηση μεταξύ των καταστάσεων μεταξύ των καταστάσεων πριν και μετά την ολοκλήρωση κάθε Pull Request.

9.3 Ρόλος της πολυπλοκότητας, της συντηρησιμότητας και των χαρακτηριστικών των Pull Requests

Η έννοια της πολυπλοκότητας αποτέλεσε ένα από τα πιο σταθερά χαρακτηριστικά που συνδέθηκαν με την ποιότητα του λογισμικού. Τόσο στα OLS όσο και στα Linear Mixed Models, η log_complexity παρουσίασε θετική και στατιστικά σημαντική σχέση με την πυκνότητα των προβλημάτων ποιότητας, ενώ η ίδια κατεύθυνση διατηρήθηκε και στις sensitivity analyses. Το εύρημα αυτό υποδηλώνει ότι όσο αυξάνεται η δομική πολυπλοκότητα του κώδικα, τόσο αυξάνεται και η επιβάρυνση που σχετίζεται με τη συντηρησιμότητα.

Η ανωτέρω περιγραφόμενη σχέση επιβεβαιώθηκε και από τις πολυμεταβλητές αναλύσεις. Στην PCA, η complexity, η cognitive complexity και τα maintainability issues παρουσίασαν υψηλές φορτίσεις στην ίδια κύρια συνιστώσα, ενώ αντίστοιχη δομή προέκυψε και στη Factor Analysis. Η σύγκλιση αυτή δείχνει ότι η συντηρησιμότητα δεν αποτελεί απομονωμένο χαρακτηριστικό, αλλά εντάσσεται σε μία διάσταση δομικής πολυπλοκότητας και δυσκολίας συντήρησης του κώδικα.

Ο ρόλος του μεγέθους του πηγαίου κώδικα ήταν περισσότερο σύνθετος. Στα κύρια quality models, το log_ncloc παρουσίασε αρνητική σχέση με την πυκνότητα των προβλημάτων ποιότητας. Επειδή το outcome εκφράζεται ως issues ανά KLOC, το αποτέλεσμα δεν σημαίνει ότι μεγαλύτερα codebases περιέχουν μικρότερο απόλυτο αριθμό προβλημάτων, αλλά ότι τα issues δεν αυξάνονται αναλογικά με το μέγεθος του κώδικα. Επιπλέον το ncloc αφορά το μέγεθος του τελικού snapshot που αναλύθηκε από το SonarQube και δεν πρέπει να ερμηνεύεται ως μέτρο του αριθμού γραμμών που προστέθηκαν, διαγράφηκαν ή τροποποιήθηκαν από το συγκεκριμένο Pull Request. Η μετάβαση, παράλληλα, από τρεις σε δύο διαστάσεις στις size-

adjusted PCA και Factor Analysis δείχνει ότι ο τρόπος με τον οποίο ελέγχεται το code size μπορεί να επηρεάσει ουσιαστικά την πολυμεταβλητή εικόνα των δεδομένων.

Στην περίπτωση της εξέτασης ασφάλειας, οι σχέσεις των διαθέσιμων snapshot-level χαρακτηριστικών ήταν αισθητά ασθενέστερες. Το μέγεθος του αναλυόμενου πηγαίου κώδικα δεν αποτέλεσε σημαντικό predictor του security density, ενώ η πολυπλοκότητα εμφάνισε μόνο μία μικρή αρνητική σχέση στο κύριο LMM. Αυτά τα αποτελέσματα δεν επιτρέπουν συμπεράσματα σχετικά με το εάν μεγαλύτερες ή περισσότερες σύνθετες αλλαγές κώδικα είναι ασφαλέστερες ή περισσότερο ευάλωτες, καθώς η έκταση και η πολυπλοκότητα του ίδιου diff (αρχείου αλλαγών ενός Pull Request) δεν αποτέλεσαν τις συγκεκριμένες explanatory variables των κύριων μοντέλων.

Περιορισμένη ερμηνευτική αξία παρουσίασε και η κατηγορία του Pull Request. Στα κύρια OLS και LMM, καθώς και στα μοντέλα των PCA component scores και factor scores, οι κατηγορίες quality, security και both δεν εμφάνισαν σταθερές στατιστικά σημαντικές διαφοροποιήσεις. Η εικόνα αυτή ενισχύθηκε από την αλληλοεπικάλυψη των κατηγοριών στον χώρο των πρώτων principal components. Παρότι τα Negative Binomial models ανέδειξαν ορισμένα category effects, αυτά δεν αναπαράχθηκαν στις κύριες αναλύσεις και επομένως θεωρούνται περισσότερο model-sensitive.

Η πολυπλοκότητα αναδεικνύεται ως το χαρακτηριστικό με τη μεγαλύτερη συνέπεια ως προς τη σύνδεσή του με την ποιότητα και τη συντηρησιμότητα του κώδικα. Η επίδραση, αντίθετα, του μεγέθους φαίνεται να εξαρτάται περισσότερο από τον τρόπο κανονικοποίησης των μετρικών, ενώ η κατηγορία του Pull Request προσφέρει περιορισμένη πρόσθετη προβλεπτική πληροφορία. Τα ευρήματα αυτά δείχνουν ότι η αξιολόγηση ενός Pull Request είναι προτιμότερο να βασίζεται στα πραγματικά τεχνικά χαρακτηριστικά της αλλαγής και στο ευρύτερο πλαίσιο του project, παρά κυρίως στο μέγεθος ή στην αρχική κατηγοριοποίησή του.

9.4 Project-specific effects και ερμηνεία ICC

Ένα από τα πιο ισχυρά ευρήματα της έρευνας αφορά τη σημαντική διαφοροποίηση των αποτελεσμάτων ανάλογα με το project στο οποίο ανήκει κάθε Pull Request. Τα PRs του ίδιου repository τείνουν να εμφανίζουν περισσότερο παρόμοια επίπεδα προβλημάτων ποιότητας και ασφάλειας από ό,τι PRs διαφορετικών projects, γεγονός το οποίο καταδεικνύει ότι το project αποτελεί ουσιαστικό μέρος του πλαισίου μέσα στο οποίο διαμορφώνονται τα χαρακτηριστικά του κώδικα.

Η έντονη αυτή ομαδοποίηση αποτυπώνεται στους Intraclass Correlation Coefficients (ICC) των πλήρων LMM. Ο ICC ήταν περίπου 83,5% για την πυκνότητα προβλημάτων ποιότητας και 92,3% για την πυκνότητα προβλημάτων ασφάλειας. Οι εξαγόμενες τιμές δείχνουν ότι πολύ μεγάλο μέρος της υπολειπόμενης διακύμανσης οργανώνεται στο επίπεδο του project. Δεν σημαίνουν, όμως, ότι το project προκαλεί

αντίστοιχο ποσοστό των προβλημάτων, αλλά ότι τα Pull Requests του ίδιου repository παρουσιάζουν μεταξύ τους έντονη ομοιότητα.

Η παρατηρούμενη ομοιότητα μπορεί να σχετίζεται με κοινά repository-specific χαρακτηριστικά, όπως η αρχιτεκτονική, οι χρησιμοποιούμενες τεχνολογίες, οι πρακτικές ανάπτυξης και ελέγχου, το τεχνικό χρέος ή γενικότερα η κατάσταση της υπάρχουσας βάσης κώδικα. Οι παράγοντες αυτοί δεν μετρήθηκαν όλοι μέσα και επομένως αποτελούν πιθανές ερμηνείες της project-level ετερογένειας και όχι αποδεδειγμένες αιτιώδεις εξηγήσεις.

Ενδιαφέρουσα, επίσης, διαφοροποίηση παρατηρείται μεταξύ ποιότητας και ασφάλειας. Στο quality model, η μεταξύ-project διακύμανση μειώθηκε από περίπου 2,868 στο null model σε 1,825 έπειτα της εισαγωγής των fixed effects. Η συγκεκριμένη μεταβολή δείχνει ότι οι μετρήσιμες ιδιότητες του τελικού code snapshot -ιδίως το μέγεθος και η πολυπλοκότητα του αναλυόμενου κώδικα- σε συνδυασμό με την κατηγορία του Pull Request απορροφούν μέρος της ετερογένειας που αρχικά παρατηρείται μεταξύ των διαφόρων projects. Στο security model, από την άλλη, η project-level variance παρέμεινε σχεδόν αμετάβλητη, ~0,155 στο null και 0,156 στο πλήρες μοντέλο, γεγονός το οποίο δείχνει ότι οι διαθέσιμες explanatory variables αποτυπώνουν πολύ μικρότερο μέρος της μεταξύ-project διαφοροποίησης του security outcome.

Τα αποτελέσματα αυτά αιτιολογούν τη χρήση των Linear Mixed Models ως κύριας αναλυτικής προσέγγισης. Τα LMM ενσωματώνουν άμεσα την ιεραρχική δομή των δεδομένων μέσω του random intercept, επιτρέποντας τον διαχωρισμό της διακύμανσης μεταξύ projects από εκείνη που εμφανίζεται μεταξύ Pull Requests του ίδιου project. Αντίθετα, τα cluster-robust OLS προσαρμόζουν κυρίως την εκτίμηση της αβεβαιότητας για την εξάρτηση των παρατηρήσεων, χωρίς να μοντελοποιούν άμεσα τη μεταξύ-project ετερογένεια. Η σημασία της διάκρισης αυτής έγινε εμφανής στο security outcome, επίσης, όπου η σχέση της πολυπλοκότητας διαφοροποιήθηκε μετά την ενσωμάτωση του project-level random effect.

Η ερμηνεία των μετρικών σε PR-level μετρήσεις, δεν είναι σε θέση να αποσυνδεθεί από το repository στο οποίο πραγματοποιείται κάθε αλλαγή. Τα ευρήματα δείχνουν ότι η ποιότητα και ιδιαίτερα η ασφάλεια εξαρτώνται σε σημαντικό βαθμό από το ευρύτερο project-specific context, γεγονός το οποίο καθιστά απαραίτητη τη συνεκτίμηση τόσο των χαρακτηριστικών του Pull Request όσο και του έργου στο οποίο αυτό εντάσσεται.

9.5 Ερμηνεία των PCA components και latent factors

Οι τεχνικές PCA και Factor Analysis προσφέρουν μία διαφορετική οπτική σε σχέση με τα παλινδρομικά μοντέλα, καθώς επικεντρώνονται όχι στις επιδράσεις μεμονωμένων predictors, αλλά στην υποκείμενη δομή των μετρικών ποιότητας, ασφάλειας και πολυπλοκότητας. Τα αποτελέσματα δείχνουν ότι οι επιμέρους

μετρικές δεν λειτουργούν ως πλήρως ανεξάρτητοι δείκτες, αλλά οργανώνονται σε ευρύτερες και ερμηνεύσιμες διαστάσεις.

Η σταθερότερη από αυτές αφορά τη δομική πολυπλοκότητα και τη συντηρησιμότητα. Στην PCA, η complexity, η cognitive complexity και τα maintainability issues παρουσίασαν υψηλές φορτίσεις στο πρώτο principal component, ενώ παρόμοια δομή προέκυψε και στον πρώτο factor της Factor Analysis. Η σύγκλιση των δύο μεθόδων υποστηρίζει την ύπαρξη της μίας κοινής διάστασης που συνδέει την αυξημένη πολυπλοκότητα με μεγαλύτερη δυσκολία συντήρησης του κώδικα.

Η διάσταση της ασφάλειας εμφανίστηκε επιπλέον ως σχετικά διακριτή διάσταση. Στην PCA, τα security issues και security hotspots συγκεντρώθηκαν κυρίως στο δεύτερο component, ενώ στη Factor Analysis οργανώθηκαν στον τρίτο factor. Η συγκεκριμένη διάσταση ήταν λιγότερο σταθερή στη Factor Analysis, καθώς βασίστηκε σε μικρότερο αριθμό ισχυρών indicators και τα security issues παρουσίασαν υψηλή uniqueness. Η ύπαρξη, επομένως, μιας ξεχωριστής security-related δομής υποστηρίζεται από τα δεδομένα, αλλά η ακριβής μορφή της απαιτεί μεγαλύτερη επιφύλαξη.

Η αξιοπιστία αποτέλεσε μία ακόμη διακριτή, μεταξύ άλλων, διάσταση, χωρίς να οργανώνεται με ακριβώς τον ίδιο τρόπο στις δύο τεχνικές. Στην PCA εκφράστηκε κυρίως μέσω του τρίτου component, ενώ στη Factor Analysis συνδέθηκε περισσότερο με τη συντηρησιμότητα και τη διπλοτυπία. Η διαφοροποίηση αυτή είναι συμβατή με τον διαφορετικό στόχο των δύο μεθόδων και δεν αναιρεί τη γενικότερη συμφωνία τους ότι η δομή των δεδομένων είναι πολυδιάστατη.

Οι size-adjusted sensitivity analyses έδειξαν παράλληλα ότι ο ακριβής αριθμός των διαστάσεων δεν είναι πλήρως σταθερός, καθώς τόσο η PCA όσο και η Factor Analysis μετέβησαν από τρεις σε δύο διατηρούμενες διαστάσεις. Μεγαλύτερη ερμηνευτική βαρύτητα πρέπει επομένως να αποδίδεται στα επαναλαμβανόμενα δομικά πρότυπα και όχι αποκλειστικά στον ακριβή αριθμό των components ή factors. Επιπλέον, τα υψηλά ICC των component και factor score models έδειξαν ότι οι συγκεκριμένες διαστάσεις διαφοροποιούνται έντονα και μεταξύ των projects.

Η συνδυαστική ερμηνεία των δύο αναλύσεων δείχνει ότι οι μετρικές ποιότητας και ασφάλειας αποτυπώνουν σχετιζόμενες αλλά διακριτές τεχνικές διαστάσεις. Η πολυπλοκότητα και η συντηρησιμότητα σχηματίζουν την πιο σταθερή κοινή δομή, ενώ η ασφάλεια και η αξιοπιστία διατηρούν περισσότερο ιδιαίτερα χαρακτηριστικά. Έτσι, η συνολική κατάσταση του λογισμικού δεν μπορεί να συνοψιστεί επαρκώς σε έναν μοναδικό δείκτη, αλλά απαιτεί την ταυτόχρονη εξέταση πολλαπλών και μεταξύ τους διαφορετικών διαστάσεων.

9.6 Σύγκριση με τη βιβλιογραφία και μη αναμενόμενα αποτελέσματα

Τα αποτελέσματα της παρούσας έρευνας παρουσιάζουν σημαντικά σημεία σύγκλισης και συμφωνίας με την ήδη υπάρχουσα βιβλιογραφία, ιδιαίτερα ως προς τη

σχέση της πολυπλοκότητας με τη συντηρησιμότητα. Προηγούμενες μελέτες έχουν συνδέσει την αυξημένη πολυπλοκότητα, την ύπαρξη πλήθους code smells και άλλα αντίστοιχα δομικά χαρακτηριστικά του κώδικα με μεγαλύτερη δυσκολία συντήρησης και αυξημένη πιθανότητα εμφάνισης προβλημάτων. Η σταθερή θετική σχέση της complexity με την πυκνότητα quality issues, καθώς και η κοινή τους εμφάνιση στις PCA και Factor Analysis, ενισχύουν αυτή την εικόνα.

Τα αποτελέσματα υπογραμμίζουν τον καταλυτικό χαρακτήρα του μεγέθους του πηγαίου κώδικα ως παράγοντα που επηρεάζει τόσο τις ίδιες τις μετρικές όσο και την ερμηνεία τους. Η αρνητική σχέση του log_ncloc με το quality density και η μεταβολή της πολυμεταβλητής δομής στις size-adjusted αναλύσεις δείχνουν ότι ο τρόπος κανονικοποίησης ως προς το μέγεθος μπορεί να επηρεάσει ουσιαστικά τα αποτελέσματα.

Περισσότερο μη αναμενόμενη ήταν η σχέση που αναπτύχθηκε μεταξύ πολυπλοκότητας και ασφάλειας. Ενώ προηγούμενες έρευνες έχουν συνδέσει structural metrics και complexity με την εμφάνιση vulnerabilities, στην παρούσα μελέτη η σχέση αυτή κρίθηκε ασθενής. Δεν ήταν σημαντική στο OLS μοντέλο και εμφανίστηκε μόνο ως μικρή αρνητική σχέση στο LMM. Το εύρημα δεν υποδηλώνει ότι η πολυπλοκότητα βελτιώνει την ασφάλεια, αλλά ότι η συγκεκριμένη σχέση εξαρτάται πιθανότατα από τον τρόπο μέτρησης, το επίπεδο ανάλυσης και το project-level context.

Η έντονα εμφανιζόμενη και παρατηρούμενη project-level ετερογένεια βρίσκεται επίσης σε συμφωνία με τη βιβλιογραφία η οποία αποτυπώνει το γεγονός ότι οι μετρικές λογισμικού και τα predictive models μπορούν να συμπεριφέρονται διαφορετικά από project σε project. Τα πολύ υψηλά ICC της παρούσας έρευνας ενισχύουν την άποψη ότι η μεταφορά συμπερασμάτων μεταξύ repositories πρέπει να γίνεται με προσοχή και ότι το project αποτελεί ουσιαστικό μέρος της ερμηνείας των αποτελεσμάτων.

Ιδιαίτερα μη αναμενόμενο ήταν το γεγονός ότι οι περισσότερες μετρικές παρουσίαζαν ελάχιστη ή ακόμα και μηδενική μεταβολή μεταξύ των δύο δημιουργημένων καταστάσεων, pr_base και pr_closed. Παρότι η βιβλιογραφία δείχνει ότι μεμονωμένες αλλαγές ή refactorings δεν οδηγούν καθολικά σε μεγάλες άμεσες διαφοροποιήσεις στις συνολικές μετρικές ποιότητας, η έκταση της αμεταβλητότητας στο σύνολο δεδομένων το οποίο χρησιμοποιήθηκε στην παρούσα έρευνα ήταν ιδιαίτερα υψηλή. Πιθανές εξηγήσεις μπορεί να είναι ότι πολλές αλλαγές ήταν αρκετά τοπικές ώστε να μην επηρεάζουν αισθητά aggregate static-analysis metrics ή ότι οι συγκεκριμένες μετρικές δεν είναι επαρκώς ευαίσθητες για την καταγραφή μικρών within-PR-μεταβολών.

Τέλος, η PCA και η Factor Analysis συμφωνούν με τη γενικότερη θέση της βιβλιογραφίας η οποία αντιπροσωπεύει την επαρκή απόδοση της ασφάλειας και της ποιότητας μέσω μίας μοναδικής μετρικής. Η πολυπλοκότητα, η συντηρησιμότητα, η αξιοπιστία και η ασφάλεια εμφανίζονται ως σχετιζόμενες αλλά διακριτές διαστάσεις, γεγονός το οποίο ενισχύει την ανάγκη πολυμεταβλητής και context-aware προσέγγισης στην αξιολόγηση του λογισμικού.

9.7 Πρακτικές επιπτώσεις

Στη συγκεκριμένη ενότητα εξετάζεται με ποιον τρόπο τα εμπειρικά αποτελέσματα της έρευνας μπορούν να μεταφραστούν σε πρακτικές κατευθύνσεις για τη διαδικασία ανάπτυξης λογισμικού. Ειδικότερα, συζητείται η αξιοποίηση των ευρημάτων για περισσότερο context-aware code review και προτεραιοποίηση του ελέγχου των Pull Requests, η ενσωμάτωσή τους σε πρακτικές DevSecOps και CI/CD, καθώς και η πιθανή χρήση automated quality και security gates για την ανάδειξη περιπτώσεων που, βάσει συνδυασμού μετρικών και πληροφοριών που προέρχεται από το εσωτερικό περιβάλλον του έργου λογισμικού, ενδέχεται να απαιτούν αυξημένη προσοχή.

Η βασική πρακτική κατεύθυνση που προκύπτει είναι ότι η αξιολόγηση ενός Pull Request δεν είναι σκόπιμο να βασίζεται αποκλειστικά σε μία μεμονωμένη ένδειξη, αλλά να λαμβάνει υπόψη τη δομική πολυπλοκότητα της αναλυόμενης κατάστασης του κώδικα, το προφίλ των quality και security metrics και το ιδιαίτερο context του project. Οι πληροφορίες αυτές μπορούν να χρησιμοποιηθούν συμπληρωματικά για την προτεραιοποίηση του ελέγχου, χωρίς να ερμηνεύονται ως άμεση εκτίμηση πραγματικού technical/security risk και χωρίς να αποδίδεται στο συγκεκριμένο Pull Request αιτιώδης ευθύνη για καταγεγραμμένα findings.

9.7.1 Code review και context-aware προτεραιοποίηση Pull Request

Τα αποτελέσματα της έρευνας δείχνουν ότι το code review μπορεί να γίνει αποτελεσματικότερο όταν η ένταση του ελέγχου προσαρμόζεται στα πραγματικά τεχνικά χαρακτηριστικά κάθε Pull Request. Ειδικότερα, η σταθερή σχέση της πολυπλοκότητας του αναλυόμενου code snapshot με την πυκνότητα των maintainability issues δείχνει ότι περιπτώσεις στις οποίες παρατηρείται αυξημένη δομική επιβάρυνση μπορούν να λαμβάνουν μεγαλύτερη προσοχή κατά την αξιολόγηση. Η ερμηνεία αυτή αφορά την τεχνική κατάσταση του κώδικα η οποία σχετίζεται με το Pull Request και δεν σημαίνει ότι η πολυπλοκότητα ή τα καταγεγραμμένα issues δημιουργήθηκαν από την ίδια την αλλαγή.

Η προσέγγιση αυτή μπορεί να υποστηρίξει μία περισσότερο context-aware προτεραιοποίηση του code review, στην οποία η κατανομή της προσοχής δεν βασίζεται αποκλειστικά στη δηλωμένη κατηγορία του Pull Request ή σε μία μεμονωμένη μετρική, αλλά σε συνδυασμό πληροφοριών, όπως η πολυπλοκότητα της αναλυόμενης κατάστασης του κώδικα, τα maintainability issues, τα security-related findings και το ευρύτερο προφίλ του project. Η περιορισμένη και μη σταθερή συσχέτιση των κατηγοριών quality, security, both με τα κύρια outcomes ενισχύει την ανάγκη να μη χρησιμοποιείται η αρχική ταξινόμηση ως αυτόνομο κριτήριο αξιολόγησης.

Οι παραπάνω δείκτες μπορούν να λειτουργήσουν ως ενδείξεις για αυξημένη προσοχή και όχι ως άμεσος υπολογισμός τεχνικού ή security risk. Η παρούσα ανάλυση δεν εκτιμά πιθανότητα vulnerability exploitation, πιθανότητα security incident ή πιθανότητα ένα συγκεκριμένο Pull Request να εισαγάγει νέο πρόβλημα. Εν συνεπεία, οποιαδήποτε μελλοντική χρήση των ευρημάτων σε risk-informed διαδικασία θα απαιτούσε ξεχωριστό μοντέλο, κατάλληλο operational definition του risk και ανεξάρτητη εμπειρική επιβεβαίωση.

Παράλληλα, η έντονη project-level ετερογένεια υποδηλώνει ότι τα κριτήρια αξιολόγησης δεν είναι απαραίτητο να είναι ίδια για όλα τα repositories. Ένα επίπεδο πολυπλοκότητας ή πυκνότητας issues μπορεί να είναι ασυνήθιστο σε ένα project και συνηθισμένο σε ένα άλλο. Η αξιολόγηση, άρα, ενός PR κρίνεται πιο ορθό να γίνεται και σε σχέση με το ιστορικό και το τεχνικό προφίλ του ίδιου project.

Πρακτικά, μία risk-based προσέγγιση μπορεί να χρησιμοποιηθεί για την επιλογή των Pull Request που χρειάζονται περισσότερους reviewers, βαθύτερο έλεγχο, πρόσθετες δοκιμές ή εξειδικευμένο security review. Τα ευρήματα δεν προσδιορίζουν ένα έτοιμο σύστημα βαθμολόγησης κινδύνου, αλλά υποστηρίζουν μία περισσότερο στοχευμένη και context-aware διαδικασία αξιολόγησης του πηγαίου κώδικα, όπου οι διαθέσιμοι πόροι κατευθύνονται στις αλλαγές που εμφανίζουν εντονότερες ενδείξεις τεχνικής επιβάρυνσης ή κινδύνου.

9.7.2 DevSecOps και CI/CD

Τα ευρήματα της παρούσας έρευνας μπορούν να αξιολογηθούν στο πλαίσιο πρακτικών DevSecOps και CI/CD, όπου οι έλεγχοι ποιότητας και ασφάλειας ενσωματώνονται στη διαδικασία ανάπτυξης και εκτελούνται συστηματικά σε κάθε αλλαγή του κώδικα. Η πολυδιάστατη σχέση που παρατηρήθηκε μεταξύ ποιότητας και ασφάλειας υποδεικνύει ότι οι δύο διαστάσεις είναι προτιμότερο να παρακολουθούνται παράλληλα και όχι ως ανεξάρτητες διαδικασίες ελέγχου.

Σημειώνεται, όμως, ότι η έντονη project-level ετερογένεια υποδεικνύει το γεγονός ότι ίδιοι ορισμένοι κανόνες και thresholds δεν είναι απαραίτητα κατάλληλοι για όλα τα repositories, που είτε εξετάζονται είτε αναπτύσσονται από κάποιον οργανισμό. Μία περισσότερο context-aware DevSecOps προσέγγιση μπορεί να λαμβάνει υπόψη τη συνήθη συμπεριφορά και το ιστορικό κάθε project, αποφεύγοντας την εφαρμογή απολύτως ομοιόμορφων κριτηρίων σε συστήματα λογισμικού με διαφορετικά τεχνικά χαρακτηριστικά.

Παράλληλα, η περιορισμένη προβλεπτική αξία της αρχικής κατηγορίας ενός Pull Request δείχνει ότι οι αυτοματοποιημένοι έλεγχοι δεν θα πρέπει να ενεργοποιούνται μόνο για αλλαγές που χαρακτηρίζονται εξ αρχής ως security ή ως quality. Η συνεχής εφαρμογή, αντίθετα, συνδυασμένων quality και security checks σε όλα τα Pull Requests μειώνει την εξάρτηση από την αρχική ταξινόμηση της αλλαγής και επιτρέπει την έγκαιρη αναγνώριση προβλημάτων που διαφορετικά θα μπορούσαν να μην έχουν εντοπιστεί και παρατηρηθεί.

Η ενσωμάτωση αυτής της λογικής σε ένα DevSecOps περιβάλλον μετατρέπει τις διαδικασίες του CI/CD και CI/CD pipeline από έναν μηχανισμό απλής

αυτοματοποίησης build και deployment σε ένα συνεχές σημείο αξιολόγησης της τεχνικής κατάστασης του κώδικα. Τα ευρήματα της παρούσας έρευνας υποστηρίζουν pipelines τα οποία συνδυάζουν ελέγχους ποιότητας και ασφάλειας, προσαρμόζονται στο context του εκάστοτε project και παρέχουν έγκαιρη ανατροφοδότηση πριν την ολοκλήρωση μιας αλλαγής.

9.7.3 Automated quality και security gates

Η χρήση τεχνικών automated quality και security gates μπορεί να αξιοποιήσει με πρακτικό τρόπο τα ευρήματα της έρευνας, ενσωματώνοντας ελέγχους ποιότητας και ασφάλειας πριν από την αποδοχή ή συγχώνευση ενός οποιουδήποτε Pull Request. Αντί για αποκλειστική εξάρτηση από χειροκίνητο review, τα gates μπορούν να λειτουργούν ως ένα πρώτο επίπεδο αυτοματοποιημένου ελέγχου, εντοπίζοντας αλλαγές που παρουσιάζουν αυξημένη τεχνική επιβάρυνση ή πιθανούς κινδύνους.

Η σταθερή σχέση της πολυπλοκότητας με τα προβλήματα ποιότητας υποδεικνύει ότι μετρικές όπως η complexity, cognitive complexity και maintainability issues μπορούν να συμμετέχουν σε κριτήρια αξιολόγησης ενός Pull Request. Security issues και security hotspots μπορούν με τον τρόπο αυτό να ελέγχονται στο ίδιο pipeline, με στόχο η αποδοχή μιας αλλαγής να μην βασίζεται αποκλειστικά σε μία μοναδική διάσταση του λογισμικού.

Τα εξαγόμενα αποτελέσματα δείχνουν έντονα την άποψη ότι η εφαρμογή απόλυτα ταυτόσημων thresholds σε όλα τα projects μπορεί να είναι περιοριστική. Η έντονη project-level ετερογένεια υποστηρίζει περισσότερο μία προσαρμοζόμενη λογική, όπου τα όρια αξιολόγησης λαμβάνουν υπόψη το ιστορικό και τεχνικό διαμορφωμένο προφίλ του εκάστοτε repository. Έτσι, ένα gate μπορεί να αξιολογεί όχι μόνο αν μία μετρική υπερβαίνει ένα καθολικό όριο, αλλά και αν η νέα αλλαγή αποτελεί σημαντική απόκλιση από τη συνήθη κατάσταση του συγκεκριμένου project.

Η περιορισμένη, επιπλέον, προβλεπτική αξία της κατηγορίας του PR υποδεικνύει ότι οι quality και security έλεγχοι είναι προτιμότερο να εφαρμόζονται σε όλα τα PRs, ανεξάρτητα από το αν έχουν χαρακτηριστεί εξ αρχής με quality/security/both. Με τον τρόπο αυτό, μειώνεται ο κίνδυνος να παραλειφθούν ζητήματα που δεν σχετίζονται άμεσα με τον δηλωμένο σκοπό της αλλαγής.

Τα automated gates δεν είναι απαραίτητο να λειτουργούν αποκλειστικά ως μηχανισμός απόρριψης. Μπορούν να υποστηρίζουν μία πιο κλιμακωτή προσέγγιση, όπου ορισμένες αποκλίσεις οδηγούν σε προειδοποίηση, άλλες απαιτούν πρόσθετο review ή testing και μόνο οι σοβαρότερες εξ αυτών εμποδίζουν προσωρινά τη συγχώνευση. Με τον τρόπο αυτό, η αυτοματοποίηση λειτουργεί ως ένα εργαλείο έγκαιρης ανίχνευσης και προτεραιοποίησης κινδύνου, χωρίς όμως σε καμία περίπτωση να αντικαθιστά την κρίση των developers και reviewers.

Συμπεραίνουμε άρα, ότι η πρακτική αξία των gates βρίσκεται στη δυνατότητά τους να συνδυάζουν πολλαπλές μετρικές, να λειτουργούν συνεχώς μέσα στο CI/CD pipeline και να προσαρμόζονται στο context του κάθε project. Τα ευρήματα της έρευνας υποστηρίζουν μία τέτοια πολυδιάστατη και context-aware προσέγγιση, αντί

για απλούς κανόνες που βασίζονται αποκλειστικά σε μοναδική μετρική ή σε σταθερά όρια για όλα τα repositories.

9.8 Threats to Validity και περιορισμοί

Η ορθή ερμηνεία των αποτελεσμάτων της παρούσας έρευνας προϋποθέτει την αναγνώριση των παραγόντων που ενδέχεται να επηρεάζουν την εγκυρότητα και τη δυνατότητα γενίκευσής τους. Οι περιορισμοί αυτοί δεν αναιρούν τα ευρήματα, αλλά προσδιορίζουν το πλαίσιο μέσα στο οποίο μπορούν να αξιολογηθούν και αποτρέπουν την εξαγωγή συμπερασμάτων που υπερβαίνουν τις δυνατότητες των διαθέσιμων δεδομένων και της εφαρμοζόμενης μεθοδολογίας επί του συνόλου δεδομένων.

Οι βασικές απειλές προς την εγκυρότητα οργανώνονται σε τέσσερις κύριες κατηγορίες. Αρχικά, εξετάζονται ζητήματα *construct validity* και *measurement limitations*, τα οποία αφορούν τον τρόπο κατά τον οποίο οι διαστάσεις ποιότητας και ασφάλειας έλαβαν επιχειρησιακό χαρακτήρα μέσω των μετρικών της στατικής ανάλυσης. Στη συνέχεια, συζητούνται η *internal validity* και οι περιορισμοί της *causal inference*, λαμβάνοντας υπόψη τον χαρακτήρα παρατήρησης της έρευνας, πιθανούς συγχυτικούς παράγοντες και τους περιορισμούς της σύγκρισης μεταξύ των καταστάσεων *pr_base* και *pr_closed*.

Στο παρόν κεφάλαιο, ακολουθούν ζητήματα *external validity* και *selection bias*, τα οποία σχετίζονται με τη διαδικασία επιλογής και φιλτραρίσματος των Pull Requests και των projects, καθώς και με τον βαθμό στον οποίο τα αποτελέσματα ή περιβάλλοντα ανάπτυξης. Εξετάζεται, τέλος, η *conclusion validity*, με έμφαση στα ιδιαίτερα χαρακτηριστικά των κατανομών, στην άνιση σύνθεση των *project-level clusters*, στις παραδοχές των στατιστικών μοντέλων και στην ευαισθησία ορισμένων αποτελεσμάτων έναντι διαφορετικών αναλυτικών επιλογών.

Η συστηματική παρουσίαση των παραπάνω περιορισμών επιτρέπει την ακριβέστερη αποτίμηση της ισχύος των συμπερασμάτων και διαχωρίζει τα ευρήματα που εμφανίζουν υψηλή εμπειρική σταθερότητα από εκείνα που απαιτούν περισσότερο επιφυλακτική ερμηνεία.

9.8.1 Construct validity και measurement limitations

Η Construct Validity αφορά τον βαθμό κατά τον οποίο οι μεταβλητές που χρησιμοποιούνται στην ανάλυση αποτυπώνουν πράγματι τις θεωρητικές έννοιες που επιδιώκεται να μετρηθούν ποσοστιαία. Στην παρούσα έρευνα, έννοιες όπως η ποιότητα, ασφάλεια, συντηρησιμότητα και αξιοπιστία έλαβαν επιχειρησιακό χαρακτήρα μέσω μετρικών που προέκυψαν από τη στατιστική ανάλυση του πηγαίου κώδικα με τη χρήση του εργαλείου SonarQube. Οι συγκεκριμένες μετρικές παρέχουν

αντικειμενικά και επαναλήψιμα σημεία αναφοράς, δεν ταυτίζονται όμως πλήρως με τις αντίστοιχες θεωρητικές έννοιες.

Παραδείγματος χάριν, ο αριθμός των maintainability issues χρησιμοποιήθηκε ως βασικός δείκτης προβλημάτων συντηρησιμότητας, ενώ τα security issues, τα συνολικά vulnerabilities και τα security hotspots αξιοποιήθηκαν για την αποτύπωση διαφορετικών πτυχών της ασφάλειας. Οι δείκτες αυτοί περιγράφουν ευρήματα που αναγνωρίζονται από τους κανόνες του εργαλείου και δεν αποτελούν πλήρη μέτρηση της πραγματικής ποιότητας ή ασφάλειας ενός λογισμικού/συστήματος. Ιδιαίτερα θα πρέπει να αποφεύγεται η ερμηνεία των security hotspots ως επιβεβαιωμένες ευπάθειες, αλλά ως σημεία του κώδικα που απαιτούν περαιτέρω αξιολόγηση.

Περιορισμός αντίστοιχος με τον παραπάνω προκύπτει από τη χρήση static-analysis indicators. Η στατική ανάλυση εξετάζει τον πηγαίο κώδικα χωρίς να παρατηρεί τη συμπεριφορά του κατά την εκτέλεση και συνεπώς δεν μπορεί να καταγράψει όλες τις πιθανές πηγές προβλημάτων. Ζητήματα τα οποία εξαρτώνται από runtime conditions, configuration, αλληλεπιδράσεις μεταξύ συστημάτων ή το πραγματικό exploitation μιας αδυναμίας ενδέχεται να μην αποτυπώνονται επαρκώς. Ταυτοχρόνως, τα αποτελέσματα εξαρτώνται από τους διαθέσιμους κανόνες ανάλυσης και είναι δυνατό να περιλαμβάνουν τόσο false positives καθώς και προβλήματα που δεν αναγνωρίζονται από το εκάστοτε χρησιμοποιούμενο εργαλείο στατικής ανάλυσης, εδώ το SonarQube.

Πρόσθετη προσοχή επίσης απαιτείται στην ερμηνεία των density metrics ανά KLOC. Η κανονικοποίηση ως προς το μέγεθος ncloc επιτρέπει δικαιότερες συγκρίσεις μεταξύ κώδικα διαφορετικού μεγέθους, ενώ παράλληλα εισάγει εξάρτηση από τον εκάστοτε παρονομαστή κανονικοποίησης. Μικρές τιμές του ncloc μπορούν να οδηγήσουν σε ιδιαίτερα υψηλές πυκνότητες ακόμη και σε περιπτώσεις όπου ο απόλυτος αριθμός issues είναι περιορισμένος. Για τον λόγο αυτό αποκλείστηκαν παρατηρήσεις στις οποίες το ncloc ήταν μηδενικό, ενώ οι size-adjusted sensitivity analyses χρησιμοποιήθηκαν για να εξεταστεί το μέγεθος της εξάρτησης από τον συγκεκριμένο τρόπο κανονικοποίησης.

Οι μετρήσεις υπόκεινται επίσης σε πιθανό measurement error που συνδέεται με τη διαδικασία παραγωγής snapshots, την ανάλυση του κώδικα και τη διαθεσιμότητα των επιμέρους metrics. Η σχεδόν πλήρης ομοιότητα πλήθους μετρικών μεταξύ των καταστάσεων pr_base και pr_closed σε αρκετά μεγάλο μέγεθος του συνόλου των PRs περιορίζει ακόμα περισσότερο την ικανότητα των συγκεκριμένων μετρήσεων να αποτυπώσουν μικρές μεταβολές σε επίπεδο Pull Request.

Οι παραπάνω περιορισμοί σημαίνουν ότι τα αποτελέσματα πρέπει να ερμηνεύονται ως σχέσεις μεταξύ μετρήσιμων static-analysis proxies της ποιότητας και της ασφάλειας και όχι ως πλήρης αποτύπωση όλων των ιδιοτήτων του λογισμικού. Η χρήση πολλαπλών μετρικών σε συνδυασμό με τα όρια που τέθηκαν, την εφαρμογή των στατιστικών τεχνικών PCA και Factor Analysis και οι συμπληρωματικές sensitivity αναλύσεις περιορίζουν την εξάρτηση της ερμηνείας από έναν μοναδικό δείκτη και επιτρέπουν μία περισσότερο πολυδιάστατη προσέγγιση των δύο εννοιών.

Αντίστοιχη διάκριση απαιτείται και για τη μεταβλητή Category. Η ταξινόμηση ενός Pull Request ως quality, security ή both αποτυπώνει τον τεχνικό χαρακτήρα που αποδόθηκε στην αλλαγή και δεν αποτελεί μέτρηση της πραγματικής παρουσίας maintainability ή security findings στο τελικό code snapshot. Η διάκριση αυτή είναι μεθοδολογικά σημαντική, καθώς η μεταβλητή κατηγορίας και τα κύρια statistical outcomes προέρχονται από διαφορετικά στάδια της ερευνητικής διαδικασίας και δεν ορίζονται το ένα από το άλλο. Συμπερασματικά καταλήγουμε ότι οι συσχετίσεις μεταξύ Category και τον SonarQube scanner outcomes δεν αποτελούν προϊόν κυκλικής κατασκευής των μεταβλητών.

Παράλληλα, η περιορισμένη αντιστοίχιση μεταξύ PR category και τελικής κατανομής των findings αποτελεί και η ίδια εύρημα που χρήζει προσεκτικής ερμηνείας. Δεν αποδεικνύει σφάλμα ταξινόμησης, καθώς ένα Pull Request μπορεί να αφορά μία security-related παρέμβαση χωρίς το αναλυόμενο snapshot να περιέχει θετικό softwareQualitySecurityIssues, ενώ ένα security-oriented PR δύναται να βρίσκεται ταυτοχρόνως σε codebase με maintainability findings. Η Category, επομένως, αντιμετωπίζεται στην παρούσα έρευνα ως χαρακτηριστικό του προσανατολισμού της αλλαγής και όχι ως proxy για την πραγματική κατάσταση ποιότητας ή ασφάλειας του αναλυόμενου κώδικα. Επομένως, οι security-related μεταβλητές της παρούσας έρευνας ερμηνεύονται ως ποσοτικοί δείκτες των ευρημάτων που παράγει η συγκεκριμένη διαδικασία στατιστικής ανάλυσης και όχι ως άμεσες μετρήσεις της πραγματικής δυνατότητας exploitation, της πιθανότητας επιτυχούς επίθεσης ή του συνολικού security risk του αντίστοιχου συστήματος.

Επιπλέον περιορισμός construct validity αφορά τη μεταβλητή Category. Η αρχική ταξινόμηση των Pull Requests σε code_quality, code_security, software_quality και software_security πραγματοποιήθηκε αυτοματοποιημένα μέσω του LLM μοντέλου gpt-oss-120b, με βάση GitHub metadata, πληροφορίες του Pull Request και το πλήρες .diff ή .patch της αλλαγής. Η συγκεκριμένη μεταβλητή είναι ανεξάρτητη από τα SonarQube findings που χρησιμοποιήθηκαν ως statistical outcomes, γεγονός το οποίο αποτρέπει την άμεση κυκλική κατασκευή predictor και outcome. Δεν πραγματοποιήθηκε ανεξάρτητη χειροκίνητη επικύρωση των παραγόμενων ταξινομήσεων. Συνεπώς, δεν κατέστη δυνατό να ποσοτικοποιηθεί η ακρίβεια της αρχικής LLM-based ταξινόμησης ούτε να αποκλειστεί η ύπαρξη περιπτώσεων λανθασμένης ή αμφίσημης κατηγοριοποίησης.

Δεν διατηρήθηκαν πλήρεις πληροφορίες για τις inference parameters της συγκεκριμένης εκτέλεσης του μοντέλου. Ο περιορισμός αυτός δεν επηρεάζει την αναπαραγωγή των μεταγενέστερων στατιστικών αναλύσεων όταν χρησιμοποιούνται τα ήδη αποθηκευμένα classification outputs, περιορίζει όμως τη δυνατότητα αυστηρής end-to-end αναπαραγωγής της αρχικής LLM classification διαδικασίας. Για τον λόγο αυτό, οι κατηγορίες quality, security και both ερμηνεύονται ως operationalized χαρακτηρισμοί του τεχνικού προσανατολισμού των Pull Requests και όχι ως ανεξάρτητα επικυρωμένες ground-truth κατηγορίες.

9.8.2 Internal validity και causal inference

Η internal validity της έρευνας περιορίζεται από τον χαρακτήρα παρατήρησης των δεδομένων. Οι σχέσεις που εντοπίστηκαν μεταξύ των μεγεθών πολυπλοκότητας, μεγέθους πηγαίου κώδικα, ποιότητας και ασφάλειας πρέπει επομένως να ερμηνεύονται ως στατιστικές συσχετίσεις και όχι ως αποδείξεις αιτιώδους επίδρασης.

Σημαντικός περιορισμός αφορά στην πιθανή παρουσία confounding factors και omitted variables. Χαρακτηριστικά όπως η αρχιτεκτονική του συστήματος, η χρησιμοποιούμενη γλώσσα προγραμματισμού, το τεχνικό χρέος, η εμπειρία των προγραμματιστών που συμμετέχουν στην ανάπτυξη και συντήρηση του λογισμικού και οι πρακτικές code review ή testing μπορούν να επηρεάζουν ταυτόχρονα τα χαρακτηριστικά ενός Pull Request και τα επίπεδα ποιότητας ή ασφάλειας, χωρίς να είναι όλα διαθέσιμα στο τελικό dataset. Τα LMM περιορίζουν εν μέρει το εμφανιζόμενο πρόβλημα μέσω της μοντελοποίησης της project-level ετερογένειας, αλλά ο random intercept δεν προσδιορίζει τους μηχανισμούς που δημιουργούν αυτές τις διαφορές ούτε εξαλείφει πλήρως το residual confounding.

Αναφέρεται επίσης ότι η σύγκριση μεταξύ των καταστάσεων pr_base και pr_closed αποτέλεσε πρόσθετο περιορισμό στην εκτέλεση των αναλύσεων. Η πολύ περιορισμένη μεταβλητότητα μεταξύ των δύο snapshots δεν επέτρεψε αξιόπιστη before-after ή delta analysis, με αποτέλεσμα η τελική προσέγγιση να είναι cross-sectional. Κατά συνέπεια, η έρευνα μπορεί να αναγνωρίσει διαφορές και σχέσεις μεταξύ Pull Requests και projects, αλλά όχι να τεκμηριώσει ότι μία συγκεκριμένη αλλαγή στον κώδικα προκάλεσε αντίστοιχη μεταβολή στην ποιότητα ή στην ασφάλεια.

Οι παρατηρούμενες σχέσεις επιδέχονται πρόσθετες εναλλακτικές ερμηνείες. Για παράδειγμα, η σύνδεση της πολυπλοκότητας με τα quality issues μπορεί να επηρεάζεται και από άλλα μη μετρούμενα χαρακτηριστικά, ενώ η μικρή αρνητική σχέση της πολυπλοκότητας με το security density δεν πρέπει να ερμηνεύεται ως ένδειξη ότι η αυξημένη πολυπλοκότητα είναι ικανή να βελτιώσει ποσοστιαία την ασφάλεια.

Συνοψίζοντας, μέσω της παρούσας έρευνας παρέχεται ισχυρή βάση για την αναγνώριση δημιουργημένων συσχετίσεων, προτύπων και project-level διαφορών, αλλά όχι για την επιβεβαίωση αιτιωδών μηχανισμών. Ισχυρότερη causal inference θα απαιτούσε δεδομένα με μεγαλύτερη χρονική μεταβλητότητα, πληρέστερη καταγραφή πιθανών confounders και ερευνητικά σχέδια καταλληλότερα για την συνολική απομόνωση αιτιωδών επιδράσεων.

9.8.3 External validity και selection bias

Η External Validity αφορά στον βαθμό κατά τον οποίο τα αποτελέσματα της έρευνας μπορούν να γενικευθούν πέρα από το συγκεκριμένο δείγμα. Στο πλαίσιο της παρούσας μελέτης, τα δεδομένα προέρχονται από σύνολο Pull Requests και projects του GitHub και συνεπώς τα συμπεράσματα αντανakλούν πρωτίστως τα

χαρακτηριστικά των repositories που τελικά συμπεριλήφθηκαν στην ανάλυση. Δεν μπορεί να θεωρηθεί ότι το δείγμα αντιπροσωπεύει το σύνολο των έργων λογισμικού ή όλων των πιθανών περιβαλλόντων ανάπτυξης.

Πρόσθετη πηγή selection bias αποτελεί η διαδικασία φιλτραρίσματος. Από τα αρχικά 353 εντοπισμένα projects, έπειτα από το «φιλτράρισμα», στο τελικό δείγμα παρέμειναν 242 εξ αυτών, ενώ παράλληλα οι παρατηρήσεις με $ncloc = 0$ αποκλείστηκαν επειδή δεν επέτρεπαν τον ουσιαστικό υπολογισμό των density metrics. Ο συγκεκριμένος αποκλεισμός εγγραφών κρίθηκε απαραίτητος για τη στατιστική ανάλυση, μπορεί όμως να σημαίνει ότι Pull Requests με πολύ μικρές, μη κωδικές ή διαφορετικού τύπου αλλαγές εκπροσωπούνται λιγότερο έως καθόλου στα τελικά αποτελέσματα.

Η δυνατότητα γενίκευσης παρουσιάζεται επίσης από τις διαφορές που εμφανίζονται εκ φύσεως μεταξύ διαφορετικών γλωσσών προγραμματισμού, τεχνολογικών ecosystems, αρχιτεκτονικών και πρακτικών ανάπτυξης. Η ίδια τιμή πολυπλοκότητας, issue density ή security metric δεν είναι απαραίτητο να έχει ακριβώς την ίδια σημασία σε διαφορετικά τεχνολογικά περιβάλλοντα. Η έντονη project-level ετερογένεια που παρατηρήθηκε στην παρούσα έρευνα ενισχύει αυτή την επιφύλαξη, καθώς δείχνει ότι σημαντικό μέρος των αποτελεσμάτων εξαρτάται άμεσα από το context του εκάστοτε repository.

Τα ευρήματα, συνεπώς, είναι καταλληλότερο να ερμηνεύονται ως εμπειρικά πρότυπα που χαρακτηρίζουν το συγκεκριμένο σύνολο GitHub project και όχι ως καθολικοί κανόνες κάθε συστήματος λογισμικού. Η επιβεβαίωσή τους σε διαφορετικές γλώσσες, οικοσυστήματα, τύπους έργων και ιδιωτικά ή βιομηχανικά repositories θα ήταν απαραίτητη για ισχυρότερη γενίκευση των εξαχθέν αποτελεσμάτων.

9.8.4 Conclusion validity και statistical limitations

Η Conclusion Validity της πραγματευόμενης έρευνας επηρεάζεται από ορισμένα χαρακτηριστικά των δεδομένων και των εφαρμοζόμενων μοντέλων. Αρχικά, τα projects περιείχαν άνισο αριθμό Pull Requests, με διάμεσο 8 και εύρος 1-85 PRs, γεγονός που συνεπάγεται διαφορετικό βαθμό πληροφoρίας για την εκτίμηση των project-specific effects. Η χρήση Linear Mixed Models και partial pooling, καθώς και η sensitivity analysis σε projects με πέντε (5) τουλάχιστον PRs, περιόρισαν την επίδραση του συγκεκριμένου προβλήματος, με τα βασικά αποτελέσματα να παραμένουν σταθερά.

Παράλληλα, αρκετές μεταβλητές παρουσίαζαν έντονη ασυμμετρία και υψηλή συγκέντρωση μηδενικών τιμών, ιδιαίτερα οι security-related metrics. Οι λογαριθμικοί μετασχηματισμοί χρησιμοποιήθηκαν προκειμένου τεθεί υπό περιορισμό η ασυμμετρία στις κύριες συνεχείς analytical variables, ενώ τα Negative Binomial models αποτέλεσαν συμπληρωματική προσέγγιση για την άμεση μοντελοποίηση των issue counts και της διασποράς τους. Το GEE, αντίθετα, χρησιμοποιήθηκε για διαφορετικό estimand, δηλαδή για τη population-averaged διερεύνηση της σχετικής σύνθεσης security και maintainability issues.

Η υψηλή συχνότητα μηδενικών τιμών δεν θεωρήθηκε από μόνη της επαρκής ένδειξη zero inflation. Δεν εφαρμόστηκε zero-inflated ή hurdle model ούτε πραγματοποιήθηκε ξεχωριστή εκτίμηση ενός structural-zero process. Η πιθανότητα ύπαρξης διαφορετικών μηχανισμών παραγωγής των μηδενικών παρατηρήσεων παραμένει ως στατιστικός περιορισμός και ως αντικείμενο πιθανής μελλοντικής διερεύνησης, χωρίς να θεωρείται ότι έχει αποδειχθεί από τα διαθέσιμα δεδομένα.

Ένας επιπλέον στατιστικός περιορισμός αφορά τη διαδοχική εφαρμογή της Factor Analysis και των LMM. Η διαδικασία πραγματοποιήθηκε σε δύο σειριακά στάδια. Αρχικά εκτιμήθηκε η παραγοντική δομή και υπολογίστηκαν τα factor scores για κάθε Pull Request και έπειτα, οι εκτιμημένες αυτές βαθμολογίες χρησιμοποιήθηκαν ως εξαρτημένες μεταβλητές στα OLS και LMM μοντέλα. Κατά το δεύτερο στάδιο τα factor scores αντιμετωπίστηκαν ως δεδομένες αριθμητικές τιμές, με αποτέλεσμα η αβεβαιότητα η οποία προκύπτει κατά το πρώτο στάδιο από την εκτίμησή τους να μην μεταφέρεται άμεσα στα standard errors και στις υπόλοιπες εκτιμήσεις αβεβαιότητας των μεταγενέστερων μοντέλων. Τα αποτελέσματα των LMM και των factor scores, επομένως, πρέπει να ερμηνεύονται ως conditional ως προς την πρωτύτερα εκτιμημένη παραγοντική λύση και όχι ως πλήρως ταυτόχρονη εκτίμηση του measurement και του hierarchical model.

Μία, ίσως πιο πλήρης, μελλοντική προσέγγιση θα μπορούσε να χρησιμοποιήσει multilevel Structural Equation Modeling ή άλλο αντίστοιχα κατάλληλο latent-variable framework, με στόχο την πραγματοποίηση της εκτίμησης των λανθανουσών διαστάσεων και των project-level σχέσεων τους μέσα σε ένα ενιαίο μοντέλο. Η συγκεκριμένη παρατήρηση δεν αναιρεί τα αποτελέσματα της παρούσας δύο-βημάτων ανάλυσης, αλλά περιορίζει τον βαθμό βεβαιότητας με τον οποίο μπορούν να ερμηνευτούν οι ακριβείς standard errors και οι inferential quantities των LMM που χρησιμοποιούν factor scores.

Πρόσθετος περιορισμός στο δείγμα προέκυψε από τη μη σύγκλιση του random-slope model. Τα δεδομένα υποστήριξαν με μεγαλύτερη σταθερότητα τη χρήση random-intercept models, χωρίς να επιτρέπουν παράλληλα ασφαλές συμπέρασμα για το κατά πόσο οι επιδράσεις συγκεκριμένων predictors διαφοροποιούνται μεταξύ projects. Αντίστοιχα, στην PCA και τη Factor Analysis η κύρια ανάλυση υποστήριξε τρεις (3) διαστάσεις, ενώ οι size-adjusted sensitivity analyses οδήγησαν σε δύο (2). Η ύπαρξη πολυμεταβλητής δομής, επομένως, εμφανίζεται σταθερή, αλλά ο ακριβής αριθμός των components ή factors είναι ευαίσθητος στις αναλυτικές επιλογές.

Ένας επιπρόσθετος περιορισμός αφορά τη χρήση density και per-KLOC μεταβλητών. Η κανονικοποίηση ως προς το NCLOC βελτιώνει τη συγκρισιμότητα μεταξύ code snapshots διαφορετικού μεγέθους, δεν εξαλείφει όμως πλήρως την επίδραση που επιβάλει το μέγεθος στο τελικό αποτέλεσμα. Επιπλέον, όταν πολλαπλές μεταβλητές μοιράζονται τον ίδιο παρονομαστή, μέρος της παρατηρούμενης συσχετιστικής δομής μπορεί να επηρεάζεται από τη συγκεκριμένη ratio-based κατασκευή (Kim, 1999). Η παράμετρος αυτή είναι ιδιαίτερα σημαντική για τις PCA και FA, καθώς οι κύριες αναλύσεις βασίστηκαν σε αρκετούς per-KLOC indicators.

Κλείνοντας τη συζήτηση περί των αποτελεσμάτων της ερευνητικής δραστηριότητας, η εφαρμογή πολλαπλών μοντέλων και η εξέταση πολλών επιμέρους σχέσεων αυξάνουν τη στατιστική αβεβαιότητα και καθιστούν ανεπαρκή την αξιολόγηση των αποτελεσμάτων αποκλειστικά μέσω μεμονωμένων *p-values*. Για τον λόγο αυτό, μεγαλύτερη βαρύτητα αποδόθηκε στη συνέπεια της κατεύθυνσης και του μεγέθους των εκτιμήσεων, στα *confidence intervals* και στην αναπαραγωγή των ευρημάτων μεταξύ διαφορετικών μοντέλων και *sensitivity analyses*. Ευρήματα που εμφανίστηκαν μόνο σε μία συγκεκριμένη προδιαγραφή αντιμετωπίστηκαν αντίστοιχα με μεγαλύτερη επιφύλαξη.

Συμπεράσματα και μελλοντική έρευνα

Συνοψίζοντας τα σημαντικότερα αποτελέσματα και αποτιμώντας τη συνολική τους σημασία σε σχέση με το αρχικό ερευνητικό πρόβλημα, το παρόν κεφάλαιο ολοκληρώνει την ερευνητική εργασία. Έπειτα της παρουσίασης και της ερμηνείας των εμπειρικών ευρημάτων, το ενδιαφέρον μετατοπίζεται πλέον από τις επιμέρους στατιστικές σχέσεις προς τη συνολική εικόνα που προκύπτει για την αλληλεπίδραση μεταξύ ποιότητας και ασφάλειας του πηγαίου κώδικα και του λογισμικού.

Πραγματοποιείται αρχικά ανακεφαλαίωση της ερευνητικής διαδικασίας και των βασικών ευρημάτων, ενώ στη συνέχεια διατυπώνεται συνολική απάντηση στο κύριο ερευνητικό ερώτημα. Ιδιαίτερη έμφαση δίνεται στο γεγονός ότι η ποιότητα και η ασφάλεια δεν προκύπτουν ως πλήρως ανεξάρτητες ούτε ως ταυτόσημες διαστάσεις, αλλά ως έννοιες που παρουσιάζουν σύνθετες και πολυδιάστατες σχέσεις, οι οποίες επηρεάζονται τόσο από χαρακτηριστικά όσο και από το κάθε project context.

Στη συνέχεια, παρουσιάζεται η θεωρητική, εμπειρική, μεθοδολογική και πρακτική συνεισφορά της έρευνας, καθώς και οι βασικές κατευθύνσεις που μπορούν να ακολουθηθούν σε μελλοντικές μελέτες. Σημαντικό ρόλο καταλαμβάνουν η αξιοποίηση περισσότερο πλούσιων και μεταβαλλόμενων δεδομένων, η ανάλυση σε πιο λεπτομερή επίπεδα του κώδικα και η ενσωμάτωση πρόσθετων πηγών πληροφορίας για την ασφάλεια. Το κεφάλαιο ολοκληρώνεται με τη συνολική αποτίμηση των συμπερασμάτων και της σημασίας τους για τη μελέτη της ποιότητας και της ασφάλειας στην εποχή της σύγχρονης ανάπτυξης λογισμικού.

10.1 Ανακεφαλαίωση της έρευνας και βασικά ευρήματα

Η παρούσα έρευνα επικεντρώθηκε στη μελέτη της σχέσης μεταξύ ποιότητας και ασφάλειας λογισμικού στο πλαίσιο τροποποιήσεων του πηγαίου κώδικα μέσω Pull Requests. Βασικός στόχος ήταν να διερευνηθεί η βαρύτητα σύνδεσης των δύο διαστάσεων μεταξύ τους, ποια χαρακτηριστικά των αλλαγών σχετίζονται με αυξημένη παρουσία προβλημάτων ποιότητας ή ασφάλειας, καθώς και σε ποιο βαθμό το project στο οποίο ανήκει κάθε Pull Request επηρεάζει τις παρατηρούμενες διαφοροποιήσεις.

Το αρχικό σύνολο δεδομένων -εγγραφών Pull Request- περιλάμβανε 20.880 εγγραφές καταστάσεων Pull Requests από 353 projects, με δύο snapshots για κάθε Pull Requests, τις καταστάσεις `pr_base` και `pr_closed`. Μετά τους απαραίτητους ελέγχους και τα στάδια καθαρισμού και φιλτραρίσματος προέκυψαν 9.813 έγκυρα `base-closed pairs`. Η σύγκριση, όμως, των δύο snapshots έδειξε ότι 94 έως και 99,9% των μετρικών παρέμενε αμετάβλητο μεταξύ των δύο χρονικών σημείων, περιορίζοντας σημαντικά τη δυνατότητα αξιόπιστης ανάλυσης των μεταβολών. Μεγάλο μέρος των παρατηρήσεων, παράλληλα, παρουσίαζε μέγεθος `ncloc` = 0. Για

τον λόγο αυτό η τελική analytical strategy μετατοπίστηκε σε cross-sectional προσέγγιση, με μία τελική pr_closed παρατήρηση ανά Pull Request και διατήρηση μόνο των εγγραφών με θετικό ncloc (ncloc > 0). Το τελικό δείγμα περιλάμβανε 3.701 Pull Requests από 242 projects.

Η ανάλυση βασίστηκε σε συνδυασμό συμπληρωματικών στατιστικών μεθόδων. Η OLS regression χρησιμοποιήθηκε ως αρχικό μοντέλο αναφοράς, ενώ τα Linear Mixed Models επέτρεψαν την ενσωμάτωση της ιεραρχικής δομής των δεδομένων, με τα PRs να είναι ομαδοποιημένα μέσα σε projects. Ταυτόχρονα εφαρμόστηκαν οι στατιστικές τεχνικές Principal Component Analysis (PCA) και Factor Analysis (FA) για τη διερεύνηση της πολυμεταβλητής δομής των quality και security metrics, ενώ πρόσθετες robustness και sensitivity analyses χρησιμοποιήθηκαν για την αξιολόγηση της σταθερότητας των βασικών συμπερασμάτων.

Ένα από τα πιο ισχυρά αποτελέσματα ήταν η έντονη project-level ετερογένεια. Οι υψηλές τιμές ICC στα mixed models έδειξαν ότι σημαντικό μέρος της συνολικής διακύμανσης τόσο στα quality όσο και στα security outcomes σχετίζεται με το project στο οποίο και ανήκει το εκάστοτε PR. Το εύρημα αυτό υπογραμμίζει ότι οι μεμονωμένες αλλαγές κώδικα δεν μπορούν να αξιολογούνται ανεξάρτητα από το repository και το τεχνικό περιβάλλον μέσα στο οποίο πραγματοποιούνται.

Η μετρική της πολυπλοκότητας, παράλληλα, παρουσίασε την πιο σταθερή και σαφή σχέση με την ποιότητα και τη συντηρησιμότητα του κώδικα. Η σχέση αυτή εμφανίστηκε τόσο στα regression models όσο και στις πολυμεταβλητές αναλύσεις, όπου οι complexity, cognitive complexity και maintainability-related indicators σχημάτιζαν μία ισχυρή κοινή διάσταση. Οι σχέσεις, αντίθετα, με την διάσταση της ασφάλειας ήταν γενικά ασθενέστερες και περισσότερο model-dependent, γεγονός το οποίο δεν υποστηρίζει μία απλή ή μονοσήμαντη σχέση μεταξύ αυξημένης πολυπλοκότητας και προβλήματα security.

Η αρχική κατηγορία που έλαβε το κάθε Pull Request (quality, security, both) δεν παρουσίασε ισχυρή και σταθερή προβλεπτική αξία στα κύρια μοντέλα. Ταυτόχρονα, τα αποτελέσματα της PCA και της FA έδειξαν ότι οι μετρικές δεν συγκροτούν μία ενιαία διάσταση, αλλά οργανώνονται σε διακριτές, αν και σχετιζόμενες, δομές. Ιδιαίτερα η πολυπλοκότητα και η συντηρησιμότητα εμφανίστηκαν ως σταθερός κοινός άξονας, ενώ οι security-related μεταβλητές παρουσίασαν περισσότερο διακριτή και λιγότερο σταθερή δομή.

Περίληπτικά, η έρευνα ανέδειξε ότι η ποιότητα και η ασφάλεια συγκροτούν μία σύνθετη και πολυδιάστατη εμπειρική δομή, ενώ τα επίπεδα των αντίστοιχων outcomes παρουσιάζουν έντονη εξάρτηση από το project-level context. Η περιορισμένη μεταβλητότητα που εντοπίστηκε μεταξύ των καταστάσεων pr_base και pr_closed δεν επέτρεψε αξιόπιστη αξιολόγηση των πραγματικών μεταβολών ποιότητας και ασφάλειας κατά τη διάρκεια ενός Pull Request. Τα ισχυρότερα, επομένως, συμπεράσματα της παρούσας εργασίας αφορούν τις cross-sectional σχέσεις μεταξύ των χαρακτηριστικών των PRs, των quality/security indicators και του project-level context και όχι αιτιώδεις ή longitudinal μεταβολές.

10.2 Απάντηση στο κύριο ερευνητικό ερώτημα

Το κύριο ερευνητικό ερώτημα το οποίο η παρούσα εργασία επιδιώκει να απαντήσει αφορά τη φύση της σχέσης μεταξύ ποιότητας και ασφάλειας του λογισμικού κατά τη διαδικασία τροποποίησης του πηγαίου κώδικα. Βάσει του συνόλου των εμπειρικών αποτελεσμάτων, η σχέση αυτή δεν μπορεί να περιγραφεί ως απλή, μονοδιάστατη ή σταθερά θετική ή αρνητική. Αντίθετα, η ποιότητα και η ασφάλεια προκύπτουν ως σχετιζόμενες αλλά διακριτές διαστάσεις, οι οποίες εντάσσονται σε μία ευρύτερη πολυδιάστατη δομή χαρακτηριστικών του λογισμικού.

Η ποιότητα και ιδιαίτερα η συντηρησιμότητα παρουσίασαν πιο σαφή και περισσότερο σταθερή σύνδεση με χαρακτηριστικά όπως η complexity και cognitive complexity. Αντίθετα, οι αντίστοιχες σχέσεις με τους δείκτες ασφάλειας ήταν γενικά ασθενέστερες και περισσότερο εξαρτώμενες από τη στατιστική προδιαγραφή. Τα αποτελέσματα των PCA και FA ενίσχυσαν με τη σειρά τους την εικόνα αυτή, καθώς οι quality, complexity και maintainability related μεταβλητές συγκροτούσαν έναν σχετικά σταθερό κοινό άξονα, ενώ οι security-related μεταβλητές εμφάνιζαν περισσότερο διακριτή και λιγότερο σταθερή δομή.

Η έντονη project-level ετερογένεια έδειξε ότι η σχέση των εξεταζόμενων χαρακτηριστικών δεν μπορεί να απομονωθεί από το repository στο οποίο και εμφανίζεται. Το project-specific context αποτελεί επομένως βασικό μέρος της ερμηνείας των συνιστωσών ποιότητας και ασφάλειας, καθώς Pull Requests με παρόμοια χαρακτηριστικά ενδέχεται να εμφανίζουν διαφορετικά αποτελέσματα όταν προέρχονται από διαφορετικά τεχνικά περιβάλλοντα. Αντίστοιχα, η κατηγορία του Pull Request δεν αποδείχθηκε σταθερός διαχωριστικός παράγοντας των quality και security outcomes.

Η έρευνα δεν μπόρεσε όμως να απαντήσει με την ίδια αξιοπιστία στο κατά πόσο μία βελτίωση της ποιότητας οδηγεί χρονικά σε βελτίωση ή επιδείνωση της ασφάλειας, ή το αντίστροφο. Η πολύ περιορισμένη μεταβλητότητα μεταξύ των pr_base και pr_closed snapshots δεν παρείχε επαρκή πληροφορία για αξιόπιστη και πλήρη delta analysis. Δεν κατέστη δυνατό να τεκμηριωθούν από τα συγκεντρωμένα δεδομένα συστηματικά synergies ή trade-offs μεταξύ των μεταβολών ποιότητας και ασφάλειας.

Η συνολική απάντηση στο κύριο ερευνητικό ερώτημα δείχνει ότι η ποιότητα και η ασφάλεια του πηγαίου κώδικα και του λογισμικού αποτελούν σχετιζόμενες αλλά διακριτές διαστάσεις μέσα σε ένα σύνθετο και πολυδιάστατο πλαίσιο. Οι μεταξύ τους σχέσεις διαφοροποιούνται ανάλογα με τις εξεταζόμενες μετρικές και τα χαρακτηριστικά του αναλυόμενου κώδικα και επηρεάζονται έντονα από το project-specific context. Η αξιολόγησή τους απαιτεί τη συνδυασμένη εξέταση πολλαπλών δεικτών και δεν δύναται να περιορίζεται σε έναν ενιαίο δείκτη ή σε μία απλή γραμμική σχέση.

10.3 Συνεισφορά της έρευνας

Η συνεισφορά της παρούσας έρευνας εντοπίζεται στη συνδυασμένη θεωρητική, εμπειρική και μεθοδολογική διερεύνηση της ποιότητας και της ασφάλειας ως σχετιζόμενων αλλά διακριτών και πολυδιάστατων χαρακτηριστικών του λογισμικού και του πηγαίου κώδικα. Τα αποτελέσματα δεν υποστηρίζουν την αναγωγή των δύο εννοιών σε έναν κοινό άξονα ούτε μία απλή σχέση σύμφωνα με την οποία η βελτίωση ή αντίθετα η επιδείνωση της μίας συνεπάγεται αντίστοιχη μεταβολή της άλλης. Αντίθετα, αναδεικνύουν μία περισσότερο σύνθετη δομή, στην οποία διαφορετικοί δείκτες ποιότητας και ασφάλειας παρουσιάζουν διαφορετικά πρότυπα συσχέτισης και διαφορετική σχέση με τα χαρακτηριστικά του αναλυόμενου κώδικα.

Σε εμπειρικό επίπεδο, η εργασία αξιοποίησε ένα τελικό δείγμα 3.701 Pull Requests από 242 διαφορετικά software projects, παρέχοντας τη δυνατότητα παράλληλης εξέτασης PR-level observations και project-level ετερογένειας. Τα αποτελέσματα έδειξαν ότι η πολυπλοκότητα της τελικής κατάστασης του αναλυόμενου κώδικα παρουσιάζει σταθερή και περισσότερο ισχυρή σχέση με την πυκνότητα των προβλημάτων συντηρησιμότητας, ενώ οι αντίστοιχες σχέσεις με τα security-related outcomes είναι σαφώς ασθενέστερες και περισσότερο εξαρτώμενες από τη χρησιμοποιούμενη στατιστική προδιαγραφή. Παράλληλα, η κατηγορία του Pull Request δεν παρουσίασε σταθερή ανεξάρτητη επίδραση στα κύρια μοντέλα. Τα ευρήματα δείχνουν ότι οι παράγοντες οι οποίοι συνδέονται με τις δύο διαστάσεις δεν είναι αναγκαστικά οι ίδιοι και δεν παρουσιάζουν την ίδια ένταση σχέσης.

Ιδιαίτερα σημαντική εμπειρική συνεισφορά αποτελεί η ανάδειξη του project-specific context. Τα Linear Mixed-Effects Models κατέδειξαν ότι πολύ μεγάλο μέρος της μεταβλητότητας τόσο του quality όσο και του security outcome βρίσκεται στο επίπεδο του project. Οι παρατηρήσεις, συνεπώς, που προέρχονται από διαφορετικά repositories δεν μπορούν να αντιμετωπίζονται ως πλήρως ανταλλάξιμες ή ανεξάρτητες από το περιβάλλον στο οποίο και παράγονται. Η έντονη αυτή ομαδοποίηση υποδεικνύει ότι μη παρατηρούμενα χαρακτηριστικά κάθε έργου -όπως η αρχιτεκτονική, οι τεχνολογικές επιλογές, οι πρακτικές ανάπτυξης και άλλες project-specific ιδιότητες- αποτελούν σημαντικό μέρος του πλαισίου μέσα στο οποίο πρέπει να ερμηνεύονται οι μετρικές ποιότητας και ασφάλειας.

Η πολυμεταβλητή ανάλυση προσθέτει μία δεύτερη διάσταση στη συνεισφορά της παρούσας έρευνας. Η Principal Component Analysis και η Factor Analysis έδειξαν ότι οι εξεταζόμενες μετρικές δεν συμπεριφέρονται ως ανεξάρτητες και απομονωμένες μεταβλητές, αλλά οργανώνονται σε ευρύτερες υποκείμενες δομές. Στην κύρια ανάλυση διακρίθηκαν διαστάσεις οι οποίες συνδέονται κυρίως με την πολυπλοκότητα και τη συντηρησιμότητα, με την ασφάλεια και με την αξιοπιστία, ενώ οι sensitive analyses έδειξαν ότι ο ακριβής αριθμός των διαστάσεων επηρεάζεται από τον τρόπο προσαρμογής ως προς το μέγεθος του πηγαίου κώδικα. Συμπεραίνουμε άρα ότι ένα από τα βασικά συμπεράσματα της έρευνας είναι ο

χαρακτηρισμός της σχέσης ποιότητας και ασφάλειας ως πολυδιάστατης έναντι της μονοδιάστατης , ενώ ο ακριβής τρόπος με τον οποίο οι επιμέρους δείκτες οργανώνονται εξαρτάται εν μέρει και από τη μεθοδολογία κανονικοποίησης και ανάλυσής τους.

Σε μεθοδολογικό επίπεδο, η εργασία συνδυάζει διαφορετικές αλλά συμπληρωματικές μεταξύ τους αναλυτικές προσεγγίσεις. Η χρήση OLS ως baseline, LMM για την αποτύπωση της ιεραρχικής δομής, Negative Binomial και GEE models ως συμπληρωματικών robustness analyses και PCA/Factor Analysis για τη διερεύνηση της πολυμεταβλητής δομής επέτρεψε την εξέταση των ίδιων δεδομένων από διαφορετικές στατιστικές οπτικές. Ιδιαίτερη σημασία επίσης λαμβάνει η προσαρμογή της ερευνητικής στρατηγικής στα πραγματικά χαρακτηριστικά του dataset. Η αρχικά προβλεπόμενη delta-based ανάλυση δεν διατηρήθηκε ως κύρια προσέγγιση από τη στιγμή που διαπιστώθηκε ότι άνω του 94% των συνολικών Pull Requests παρουσίαζαν μηδενική μεταβολή στα maintainability-related outcomes και πάνω από 99% στα security-related outcomes. Αντί της εξαγωγής συμπερασμάτων από ανεπαρκώς μεταβαλλόμενα δεδομένα, υιοθετήθηκε μία cross-sectional στρατηγική βασισμένη στο τελικό pr_closed snapshot. Η επιλογή αυτή αποτελεί ουσιαστικό στοιχείο ερευνητικής διαφάνειας και περιορίζει έτσι τον κίνδυνο εξαγωγής συμπερασμάτων που δεν υποστηρίζονται από την πληροφορία του dataset.

Σε πρακτικό επίπεδο, τα αποτελέσματα υποστηρίζουν μία περισσότερο context-aware και πολυμετρική προσέγγιση στην αξιολόγηση του πηγαίου κώδικα. Οι quality και security indicators είναι χρησιμότερο να εξετάζονται συνδυαστικά με τη δομική πολυπλοκότητα και το ιδιαίτερη περιβάλλον κάθε project, παρά ως απομονωμένες μετρικές ή ως ένας ενιαίος δείκτης τεχνικής κατάστασης. Η προσέγγιση αυτή μπορεί να αξιοποιηθεί συμπληρωματικά σε διαδικασίες code review, CI/CD και DevSecOps, χωρίς τα αποτελέσματα της παρούσας να τοποθετούνται ως επικυρωμένο μοντέλο πρόβλεψης πραγματικού security risk ή της πιθανότητας ένα συγκεκριμένο Pull Request να εισαγάγει νέα προβλήματα.

Κλείνοντας, η βασική συνεισφορά της εργασίας έγκειται στην εμπειρική τεκμηρίωση ότι η ποιότητα και η ασφάλεια χρήζουν εξέτασης ως σχετιζόμενες διαστάσεις, των οποίων η συμπεριφορά είναι πολυδιάστατη και έντονα εξαρτώμενη από το project context. Η πολυπλοκότητα αναδεικνύεται ως σημαντικότερος και περισσότερο σταθερός παράγοντας για τη διάσταση της συντηρησιμότητας από ό,τι για τη διάσταση της ασφάλειας, ενώ η υψηλή μεταξύ-project ετερογένεια καταδεικνύει ότι η ερμηνεία των software metrics χωρίς να λαμβάνεται υπόψη το αποθετήριο στο οποίο ανήκουν δύναται να αποκρύπτει σημαντικό μέρος της πραγματικής δομής των δεδομένων. Αντίθετα, η παρούσα έρευνα τεκμηριώνει άμεση χρονική ή αιτιώδη αμφίδρομη επίδραση μεταξύ μεταβολών ποιότητας και ασφάλειας, καθώς τα διαθέσιμα pr_base-pr_closed δεδομένα δεν παρείχαν την απαιτούμενη μεταβλητότητα για έναν τέτοιο έλεγχο.

10.4 Κατευθύνσεις μελλοντικής έρευνας

Τα ευρήματα και οι περιορισμοί της παρούσας μελέτης αναδεικνύουν αρκετές κατευθύνσεις για περαιτέρω διερεύνηση της σχέσης μεταξύ ποιότητας και ασφάλειας λογισμικού. Οι σημαντικότερες από αυτές μπορούν να οργανωθούν σε τέσσερις βασικούς άξονες, οι οποίοι αφορούν στη διερεύνηση του δείγματος, την αύξηση της αναλυτικής λεπτομέρειας, τον εμπλουτισμό των security δεδομένων και την εφαρμογή περισσότερο προηγμένων αναλυτικών μεθόδων.

Ο πρώτος άξονας αφορά τη διεύρυνση και διαφοροποίηση του δείγματος. Μελλοντικές έρευνες μπορούν να συμπεριλάβουν μεγαλύτερο αριθμό repositories, διαφορετικού μεταξύ τους τύπου έργων λογισμικού, επιπλέον τεχνολογικά οικοσυστήματα και περισσότερες γλώσσες προγραμματισμού. Ιδιαίτερο ενδιαφέρον επίσης παρουσιάζουν και οι programming-language-specific analyses, καθώς διαφορετικές γλώσσες και τεχνολογικές στοίβες ενδέχεται να παρουσιάζουν διαφορετικές συσχετίσεις μεταξύ complexity, maintainability και security indicators. Η διεύρυνση αυτή θα επέτρεπε παράλληλα την αξιολόγηση της δυνατότητας γενίκευσης των ευρημάτων πέρα από το συγκεκριμένο σύνολο projects.

Ο δεύτερος άξονας αφορά στη μεγαλύτερη αναλυτική λεπτομέρεια και τη χρονική διάσταση των δεδομένων. Η ανάλυση σε επίπεδο commit, file ή function θα μπορούσε να εντοπίσει τοπικές μεταβολές που ενδέχεται να αποκρύπτονται σε aggregated PR-level μετρικές. Παράλληλα, πραγματικά longitudinal datasets και security patch pairs θα επέτρεπαν ακριβέστερη παρακολούθηση των αλλαγών πριν και μετά από συγκεκριμένες τροποποιήσεις. Η συγκεκριμένη κατεύθυνση είναι ιδιαίτερα σημαντική, καθώς θα μπορούσε να επιτρέψει την ουσιαστική διερεύνηση πιθανών συνεργειών (synergies) και αντισταθμίσεων (trade-offs) μεταξύ των διαστάσεων ποιότητας και ασφάλειας πηγαίου κώδικα και λογισμικού, τα οποία δεν ήταν δυνατό να αξιολογηθούν αξιόπιστα στην παρούσα έρευνα λόγω της περιορισμένης μεταβλητότητας μεταξύ των καταστάσεων pr_base και pr_closed η οποία και εντοπίστηκε μεταξύ των δεδομένων.

Ο τρίτος άξονας αφορά τον εμπλουτισμό των δεδομένων ασφάλειας πέρα από τη στατική ανάλυση. Η συνδυασμένη αξιοποίηση SAST με DAST τεχνικών, Software Composition Analysis, dependency και secret scanning, IaC/configuration scanning και runtime security information είναι επίσης ικανή να προσφέρει πληρέστερη εικόνα της πραγματικής επιφάνειας κινδύνου ενός συστήματος. Επιπλέον, η σύνδεση των ευρημάτων με επιβεβαιωμένες ευπάθειες ή CVEs θα επέτρεπε σαφέστερη διάκριση μεταξύ ενδείξεων στατικής ανάλυσης και προβλημάτων με τεκμηριωμένη πρακτική επίπτωση.

Ο τέταρτος, και τελευταίος, άξονας αφορά την εφαρμογή περισσότερο προηγμένων στατιστικών και υπολογιστικών προσεγγίσεων. Μοντέλα όπως τα Generalized Linear Mixed Models μπορούν να είναι καταλληλότερα για outcomes με counts, υψηλή ασυμμετρία ή μεγάλο αριθμό μηδενικών τιμών, ενώ Bayesian hierarchical models θα μπορούσαν να επιτρέψουν πιο ευέλικτη εκτίμηση της αβεβαιότητας και των project-specific effects. Ερευνητικοί, παράλληλα, σχεδιασμοί καταλληλότεροι για causal inference θα μπορούσαν να εξετάσουν πληρέστερα αν συγκεκριμένες αλλαγές στον κώδικα προκαλούν μεταβολές στην ποιότητα ή στην

ασφάλεια. Τέλος, machine-learning προσεγγίσεις θα μπορούσαν να διερευνήσουν τη δυνατότητα δημιουργίας PR risk prediction models, τα οποία συνδυάζουν πολλαπλές μεταβλητές που να αφορούν την ποιότητα, την ασφάλεια και το ίδιο το context του εκάστοτε έργου καθώς και του συνόλου ανάλυσης για την έγκαιρη αναγνώριση αλλαγών που απαιτούν αυξημένη προσοχή.

Οι παραπάνω κατευθύνσεις δεν αποτελούν απομονωμένες μεταξύ τους προτάσεις, αλλά μπορούν να λειτουργήσουν συμπληρωματικά. Ένα περισσότερο πλούσιο και χρονικά ευαίσθητο σύνολο δεδομένων (dataset), σε συνδυασμό με διαφορετικές πηγές security evidence και πιο προηγμένα hierarchical ή predictive models, θα μπορούσε να οδηγήσει σε πληρέστερη κατανόηση τόσο της σχέσης μεταξύ των συνιστωσών ποιότητας και ασφάλειας όσο και των μηχανισμών μέσω των οποίων αυτή μεταβάλλεται κατά την εξέλιξη του λογισμικού.

10.5 Τελικό συμπέρασμα

Το συμπέρασμα στο οποίο η παρούσα έρευνα συγκλίνει είναι ότι η ποιότητα και η ασφάλεια του πηγαίου κώδικα και του λογισμικού δεν αποτελούν πλήρως ανεξάρτητες και ταυτόσημες διαστάσεις. Η μεταξύ τους σχέση είναι σύνθετη και πολυδιάστατη, καθώς διαφορετικά χαρακτηριστικά του κώδικα συνδέονται με διαφορετικό τρόπο με τα quality και security outcomes. Ιδιαίτερα η πολυπλοκότητα αναδείχθηκε ως σημαντικός παράγοντας για την ποιότητα και τη συντηρησιμότητα, ενώ οι αντίστοιχες σχέσεις με την ασφάλεια εμφανίστηκαν ασθενέστερες και περισσότερο εξαρτώμενες από το αναλυτικό πλαίσιο.

Η έντονη διαφοροποίηση μεταξύ των projects που εμφανίστηκε, παράλληλα, έδειξε ότι η αξιολόγηση της ποιότητας και της ασφάλειας δεν μπορεί να αποσυνδεθεί πλήρως από το project-specific context. Εντούτοις, μία ουσιαστική προσέγγιση απαιτεί συνδυασμένη εξέταση πολλαπλών μετρικών, ιεραρχική κατανόηση των δεδομένων και αποφυγή γενικευμένων συμπερασμάτων που βασίζονται σε έναν μόνο δείκτη ή σε καθολικά thresholds.

Καταλήγοντας, η σχέση των διαστάσεων ποιότητας και ασφάλειας πρέπει να προσεγγίζεται ως ένα context-aware και πολυμετρικό πρόβλημα, τόσο στην ερευνητική ανάλυση όσο και στην πρακτική ανάπτυξη λογισμικού. Μελλοντικές μελέτες με πιο πλούσια -ίσως- longitudinal δεδομένα, λεπτομερέστερη ανάλυση αλλαγών και ευρύτερη ενσωμάτωση security evidence μπορούν να προσφέρουν ακόμη σαφέστερη εικόνα για τον τρόπο με τον οποίο οι δύο εξεταζόμενες διαστάσεις αλληλοεπιδρούν κατά την εξέλιξη πραγματικών συστημάτων λογισμικού.

Βιβλιογραφία

- Abid, C., Kessentini, M., Alizadeh, V., Dhaouadi, M. & Kazman, R. (2022). How does refactoring impact security when improving quality? A security-aware refactoring approach. *IEEE Transactions on Software Engineering*, 48(3), 864–878. <https://doi.org/10.1109/TSE.2020.3005995>
- Alfadel, M., Nagy, N. A., Costa, D. E., Abdalkareem, R. & Shihab, E. (2023). Empirical analysis of security-related code reviews in npm packages. *Journal of Systems and Software*, 203, 111752. <https://doi.org/10.1016/j.jss.2023.111752>
- Almogahed, A., Omar, M. & Zakaria, N. H. (2022). Refactoring codes to improve software security requirements. *Procedia Computer Science*, 204, 108–115. Διαθέσιμο στο: <https://www.sciencedirect.com/science/article/pii/S1877050922007517> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Aloraini, B., Nagappan, M., German, D. M., Hayashi, S. & Higo, Y. (2019). An empirical study of security warnings from static application security testing tools. *Journal of Systems and Software*, 158, 110427. <https://doi.org/10.1016/j.jss.2019.110427>
- Almogahed, A., Omar, M. & Zakaria, N. H. (2022). Refactoring codes to improve software security requirements. *Procedia Computer Science*, 204, 108–115. <https://doi.org/10.1016/j.procs.2022.08.014>
- Antinyan, V., Staron, M. & Sandberg, A. (2017). Evaluating code complexity triggers, use of complexity measures and the influence of code complexity on maintenance time. *Empirical Software Engineering*, 22, 3057–3087. <https://doi.org/10.1007/s10664-017-9508-2>
- Avizienis, A., Laprie, J.-C., Randell, B. & Landwehr, C. (2004). Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1), 11–33. <https://doi.org/10.1109/TDSC.2004.2>
- Bacchelli, A. & Bird, C. (2013). Expectations, outcomes, and challenges of modern code review. In: *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, pp. 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- Bates, D., Mächler, M., Bolker, B. & Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software*, 67(1), 1–48. <https://doi.org/10.18637/jss.v067.i01>
- Bavota, G., De Carluccio, B., De Lucia, A., Di Penta, M., Oliveto, R. & Strollo, O. (2012). When does a refactoring induce bugs? An empirical study. In: *2012 IEEE 12th International Working Conference on Source Code Analysis and Manipulation*, pp. 104–113. <https://doi.org/10.1109/SCAM.2012.20>
- Black, P. E., Guttman, B. & Okun, V. (2021). *Guidelines on Minimum Standards for Developer Verification of Software (NISTIR 8397)*. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.IR.8397>
- Bogaert, J., Loh, W. W., & Rosseel, Y. (2023). A Small Sample Correction for Factor Score Regression. *Educational and Psychological Measurement*, 83(3), 495–519. <https://doi.org/10.1177/00131644221105505>
- Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>
- Burnham, K. P. & Anderson, D. R. (2002). *Model Selection and Multimodel Inference: A Practical Information-Theoretic Approach*, 2nd ed. New York: Springer. <https://doi.org/10.1007/b97636>
- Cameron, A. C. & Miller, D. L. (2015). A practitioner's guide to cluster-robust inference. *Journal of Human Resources*, 50(2), 317–372. <https://doi.org/10.3368/jhr.50.2.317>
- Campbell, G. A. (2018). Cognitive complexity: An overview and evaluation. In: *Proceedings of the 2018 International Conference on Technical Debt (TechDebt '18)*, pp. 57–58. <https://doi.org/10.1145/3194164.3194186>
- Chacon, S. & Straub, B. (2014). *Pro Git*, 2nd ed. New York: Apress. Διαθέσιμο στο: <https://git-scm.com/book/en/v2> (Πρόσβαση: 18 Σεπτεμβρίου 2026).

- Chandramouli, R., Kautz, F. & Torres-Arias, S. (2024). *Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines (NIST SP 800-204D)*. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-204D>
- Cook, C. R. & Roesch, A. (1994). Real-time software metrics. *Journal of Systems and Software*, 24(3), 223–237. [https://doi.org/10.1016/0164-1212\(94\)90065-5](https://doi.org/10.1016/0164-1212(94)90065-5)
- Cunningham, W. (1992). The WyCash portfolio management system. In: *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, pp. 29–30. <https://doi.org/10.1145/157709.157715>
- CVE Program. (χ.χ.). *Common Vulnerabilities and Exposures (CVE)*. Διαθέσιμο στο: <https://www.cve.org/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Delaitre, A., Black, P. E., Cupif, D., Haben, G., Alex-Kevin, L., Okun, V. & Prono, Y. (2023). SATE VI Report: Bug Injection and Collection (NIST SP 500-341). Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.500-341> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Fabrigar, L. R., Wegener, D. T., MacCallum, R. C. & Strahan, E. J. (1999). Evaluating the use of exploratory factor analysis in psychological research. *Psychological Methods*, 4(3), 272–299. <https://doi.org/10.1037/1082-989X.4.3.272> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Field, A. (2018). *Discovering Statistics Using IBM SPSS Statistics*, 5th ed. London: SAGE. Διαθέσιμο στο: <https://uk.sagepub.com/en-gb/eur/discovering-statistics-using-ibm-spss-statistics/book257672> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- FIRST. (χ.χ.). *Common Vulnerability Scoring System Version 4.0: Specification Document, Version 1.2*. Διαθέσιμο στο: <https://www.first.org/cvss/v4.0/specification-document> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Flater, D. W., Black, P. E., Fong, E. N., Kacker, R. N., Okun, V., Wood, S. S. & Kuhn, D. R. (2016). *A Rational Foundation for Software Metrology*. Gaithersburg, MD: National Institute of Standards and Technology. Διαθέσιμο στο: <https://www.nist.gov/publications/rational-foundation-software-metrology> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Fontana, F. A., Milanese, M., Refolli, F. & Trubiani, C. (2026). Impact of refactoring architectural smells on quality, security, and performance metrics. In: *2026 IEEE 23rd International Conference on Software Architecture (ICSA)*, pp. 213–220. <https://doi.org/10.1109/ICSA66085.2026.00027>
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA: Addison-Wesley. Διαθέσιμο στο: <https://martinfowler.com/books/refactoring.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Furia, C. A. & Torkar, R. (2026). Mitigating omitted variable bias in empirical software engineering. *Empirical Software Engineering*. <https://doi.org/10.1007/s10664-026-10851-1>
- Gigante, D., Pecorelli, F., Barletta, V. S., Janes, A., Lenarduzzi, V., Taibi, D. & Baldassarre, M. T. (2023). Resolving security issues via quality-oriented refactoring: A user study. In: *2023 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 82–91. <https://doi.org/10.1109/TechDebt59074.2023.00016>
- Gousios, G., Pinzger, M. & van Deursen, A. (2014). An exploratory study of the pull-based software development model. In: *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pp. 345–355. <https://doi.org/10.1145/2568225.2568260>
- Hilbe, J. M. (2011). *Negative Binomial Regression*, 2nd ed. Cambridge: Cambridge University Press. <https://doi.org/10.1017/CBO9780511973420>
- Hilton, M., Tunnell, T., Huang, K., Marinov, D. & Dig, D. (2016). Usage, costs, and benefits of continuous integration in open-source projects. In: *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pp. 426–437. <https://doi.org/10.1145/2970276.2970358>
- Horn, J. L. (1965). A rationale and test for the number of factors in factor analysis. *Psychometrika*, 30, 179–185. <https://doi.org/10.1007/BF02289447>
- Iannone, E., Codabux, Z., Lenarduzzi, V., De Lucia, A. & Palomba, F. (2023). Rubbing salt in the wound? A large-scale investigation into the effects of refactoring on security. *Empirical Software Engineering*, 28, 89. <https://doi.org/10.1007/s10664-023-10287-x>

- IEEE. (1990). *IEEE Std 610.12-1990: IEEE Standard Glossary of Software Engineering Terminology*. Διαθέσιμο στο: <https://www.nist.gov/document/ieee-standard-glossary-software-engineering-terminology> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2001). *ISO/IEC 9126-1:2001 — Software engineering — Product quality — Part 1: Quality model*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/22749.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2011). *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/35733.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2016). *ISO/IEC 25023:2016 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/35747.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2019). *ISO/IEC 25020:2019 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality measurement framework*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/72117.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2021). *ISO/IEC 5055:2021 — Information technology — Software measurement — Software quality measurement — Automated source code quality measures*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/80623.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2022). *ISO/IEC 27005:2022 — Information security, cybersecurity and privacy protection — Guidance on managing information security risks*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/80585.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC. (2023). *ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/78176.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC/IEEE. (2017a). *ISO/IEC/IEEE 15939:2017 — Systems and software engineering — Measurement process*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/71197.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- ISO/IEC/IEEE. (2017b). *ISO/IEC/IEEE 24765:2017 — Systems and software engineering — Vocabulary*. Geneva: International Organization for Standardization. Διαθέσιμο στο: <https://www.iso.org/standard/71952.html> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Jolliffe, I. T. & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065), 20150202. <https://doi.org/10.1098/rsta.2015.0202> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M. & Damian, D. (2016). The promises and perils of mining GitHub. *Empirical Software Engineering*, 21, 2035–2071. <https://doi.org/10.1007/s10664-015-9393-5>
- Kim, J.-H. (1999). Spurious correlation between ratios with a common divisor. *Statistics & Probability Letters*, 44(4), 383–386. [https://doi.org/10.1016/S0167-7152\(99\)00030-9](https://doi.org/10.1016/S0167-7152(99)00030-9)
- Kim, S. & Lee, H. (2018). Software systems at risk: An empirical study of cloned vulnerabilities in practice. *Computers & Security*, 77, 720–736. <https://doi.org/10.1016/j.cose.2018.02.007>
- Kochhar, P. S., Lo, D., Lawall, J. & Nagappan, N. (2017). Code coverage and postrelease defects: A large-scale study on open source projects. *IEEE Transactions on Reliability*, 66(4), 1213–1228. Διαθέσιμο στο: <https://www.microsoft.com/en-us/research/publication/code-coverage-and-post-release-defects-a-large-scale-study-on-open-source-projects/>, <https://doi.org/10.1109/TR.2017.2727062> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Kuuttila, M., Mäntylä, M., Claes, M., Elovainio, M. & Adams, B. (2021). Individual differences limit

- predicting well-being and productivity using software repositories: A longitudinal industrial study. *Empirical Software Engineering*, 26, 88. <https://doi.org/10.1007/s10664-021-09977-1>
- Lenarduzzi, V., Nikkola, V., Saarimäki, N. & Taibi, D. (2021). Does code quality affect pull request acceptance? An empirical study. *Journal of Systems and Software*, 171, 110806. <https://doi.org/10.1016/j.jss.2020.110806>
- Lenarduzzi, V., Pecorelli, F., Saarimäki, N., Lujan, S. & Palomba, F. (2023). A critical comparison on six static analysis tools: Detection, agreement, and precision. *Journal of Systems and Software*, 198, 111575. <https://doi.org/10.1016/j.jss.2022.111575>
- Liang, K.-Y. & Zeger, S. L. (1986). Longitudinal data analysis using generalized linear models. *Biometrika*, 73(1), 13–22. <https://doi.org/10.1093/biomet/73.1.13>
- Li, Z., Avgeriou, P. & Liang, P. (2015). A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101, 193–220. <https://doi.org/10.1016/j.jss.2014.12.027> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Manadhata, P. K. & Wing, J. M. (2011). An attack surface metric. *IEEE Transactions on Software Engineering*, 37(3), 371–386. <https://doi.org/10.1109/TSE.2010.60>
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- McIntosh, S., Kamei, Y., Adams, B. & Hassan, A. E. (2016). An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering*, 21, 2146–2189. <https://doi.org/10.1007/s10664-015-9381-9>
- Meneely, A. & Williams, L. (2009). Secure open source collaboration: An empirical study of Linux' law. *Proceedings of the 16th ACM Conference on Computer and Communications Security (CCS '09)*, 453–462. <https://doi.org/10.1145/1653662.1653717>
- MITRE. (χ.χ.). *Common Weakness Enumeration (CWE)*. Διαθέσιμο στο: <https://cwe.mitre.org/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Mumtaz, H., Alshayeb, M., Mahmood, S. & Niazi, M. (2018). An empirical study to improve software security through the application of code refactoring. *Information and Software Technology*, 96, 112–125. Διαθέσιμο στο: <https://www.sciencedirect.com/science/article/pii/S0950584916303664>, <https://doi.org/10.1016/j.infsof.2017.11.010> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Nagappan, N. & Ball, T. (2005). Static analysis tools as early indicators of pre-release defect density. In: *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, pp. 580–586. Διαθέσιμο στο: <https://www.microsoft.com/en-us/research/publication/static-analysis-tools-as-early-indicators-of-pre-release-defect-density/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Nagappan, N., Ball, T. & Zeller, A. (2006). Mining metrics to predict component failures. In: *Proceedings of the 28th International Conference on Software Engineering (ICSE)*, pp. 452–461. <https://doi.org/10.1145/1134285.1134349>
- Nakagawa, S., Johnson, P. C. D. & Schielzeth, H. (2017). The coefficient of determination R^2 and intra-class correlation coefficient from generalized linear mixed-effects models revisited and expanded. *Journal of the Royal Society Interface*, 14(134), 20170213. <https://doi.org/10.1098/rsif.2017.0213>
- National Institute of Standards and Technology (NIST). (2004). *Standards for Security Categorization of Federal Information and Information Systems (FIPS PUB 199)*. Gaithersburg, MD: NIST. <https://doi.org/10.6028/NIST.FIPS.199>
- National Institute of Standards and Technology (NIST). (2012). *Guide for Conducting Risk Assessments (NIST SP 800-30 Rev. 1)*. Gaithersburg, MD: NIST. <https://doi.org/10.6028/NIST.SP.800-30r1>
- National Institute of Standards and Technology (NIST). (2025). *Digital Identity Guidelines (NIST SP 800-63-4)*. Gaithersburg, MD: NIST. <https://doi.org/10.6028/NIST.SP.800-63-4>
- Opdyke, W. F. (1992). *Refactoring Object-Oriented Frameworks*. PhD dissertation. University of Illinois at Urbana-Champaign. Διαθέσιμο στο: <https://www.ideals.illinois.edu/items/72240> (Πρόσβαση: 18 Σεπτεμβρίου 2026).

- Ouellet, A. & Badri, M. (2024). Combining object-oriented metrics and centrality measures to predict faults in object-oriented software: An empirical validation. *Journal of Software: Evolution and Process*, 36(4), e2548. <https://doi.org/10.1002/smr.2548>
- OWASP Foundation. (2020). *Software Assurance Maturity Model (SAMM), Version 2.0*. Διαθέσιμο στο: <https://owasp.org/samm/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- OWASP Foundation. (2025). *OWASP Top 10:2025 — The Ten Most Critical Web Application Security Risks*. Διαθέσιμο στο: <https://top10.owasp.org/2025/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- OWASP Foundation. (χ.χ.). *DevSecOps Verification Standard (DSOVS)*. Διαθέσιμο στο: <https://owasp.org/projects/devsecops-verification-standard> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Reis, S., Abreu, R. & Cruz, L. (2021). Fixing vulnerabilities potentially hinders maintainability. *Empirical Software Engineering*, 26, 127. <https://doi.org/10.1007/s10664-021-10019-z>
- Ruland, S., Kulcsár, G., Leblebici, E., Peldszus, S. & Lochau, M. (2018). Controlling the attack surface of object-oriented refactorings. In: *Software Technologies: Applications and Foundations — STAF 2018*. Cham: Springer. https://doi.org/10.1007/978-3-319-89363-1_3
- Roy, C. K., Cordy, J. R. & Koschke, R. (2009). Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7), 470–495. <https://doi.org/10.1016/j.scico.2009.02.007> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Scarfone, K., Souppaya, M., Cody, A. & Orebaugh, A. (2008). *Technical Guide to Information Security Testing and Assessment (NIST SP 800-115)*. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-115> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Shahin, M., Babar, M. A. & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5, 3909–3943. <https://doi.org/10.1109/ACCESS.2017.2685629>
- Shin, Y. & Williams, L. (2008). An empirical model to predict security vulnerabilities using code complexity metrics. In: *Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 315–317. <https://doi.org/10.1145/1414004.1414065>
- Shin, Y., Meneely, A., Williams, L. & Osborne, J. A. (2011). Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE Transactions on Software Engineering*, 37(6), 772–787. <https://doi.org/10.1109/TSE.2010.81>
- Siavvas, M., Tsoukalas, D., Jankovic, M., Kehagias, D. & Tzovaras, D. (2022). Technical debt as an indicator of software security risk: A machine learning approach for software development enterprises. *Enterprise Information Systems*, 16(5), 1824017. <https://doi.org/10.1080/17517575.2020.1824017>
- Siavvas, M., Tsoukalas, D., Janković, M., Kehagias, D., Chatzigeorgiou, A., Tzovaras, D., Aničić, N. & Gelenbe, E. (2019). An empirical evaluation of the relationship between technical debt and software security. In: *Proceedings of the 15th International Conference on Artificial Intelligence Applications and Innovations (AIAI 2019 Workshops)*, pp. 199–203. Διαθέσιμο στο: <https://zenodo.org/records/3374712> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Software Engineering Institute. (1988). *Software Metrics*. Carnegie Mellon University. Διαθέσιμο στο: <https://www.sei.cmu.edu/library/software-metrics/> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Software Engineering Institute. (χ.χ.). *Dynamic Design Analysis*. Carnegie Mellon University. Διαθέσιμο στο: <https://www.sei.cmu.edu/library/dynamic-design-analysis-2/> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- SonarSource. (χ.χ.-a). *Managing Security Hotspots — SonarQube Server*. Διαθέσιμο στο: <https://docs.sonarsource.com/sonarqube-server/2025.5/user-guide/security-hotspots> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- SonarSource. (χ.χ.-b). *Understanding measures and metrics — SonarQube Server 2026.1 LTA*. Διαθέσιμο στο: <https://docs.sonarsource.com/sonarqube-server/2026.1/user-guide/code-metrics/metrics-definition> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- SonarSource. (χ.χ.-c). *Understanding quality gates — SonarQube Server 2026.1 LTA*.

- Διαθέσιμο στο: <https://docs.sonarsource.com/sonarqube-server/2026.1/quality-standards-administration/managing-quality-gates/introduction-to-quality-gates> (Πρόσβαση: 19 Σεπτεμβρίου 2026)
- Souppaya, M., Scarfone, K. & Dodson, D. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218)*. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>
- Thompson, C. & Wagner, D. (2017). A large-scale study of modern code review and security in open source projects. In: *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '17)*, pp. 83-92 Διαθέσιμο στο: <https://escholarship.org/uc/item/86d5w0gn> , <https://doi.org/10.1145/3127005.3127014> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B. & Wesslén, A. (2024). *Experimentation in Software Engineering*, 2nd ed. Berlin: Springer. <https://doi.org/10.1007/978-3-662-69306-3>
- Woody, C., Ellison, R. J. & Nichols, B. (2014). *Predicting Software Assurance Using Quality and Reliability Measures (CMU/SEI-2014-TN-026)*. Pittsburgh, PA: Software Engineering Institute. Διαθέσιμο στο: <https://www.sei.cmu.edu/library/predicting-software-assurance-using-quality-and-reliability-measures-2/> (Πρόσβαση: 18 Σεπτεμβρίου 2026).
- Yamashita, A. & Moonen, L. (2013). To what extent can maintenance problems be predicted by code smell detection? An empirical study. *Information and Software Technology*, 55(12), 2223–2242. <https://doi.org/10.1016/j.infsof.2013.05.002>
- Zhang, X., Yu, Y., Gousios, G. & Rastogi, A. (2023). Pull request decisions explained: An empirical overview. *IEEE Transactions on Software Engineering*, 49(2), 849–871. <https://doi.org/10.1109/TSE.2022.3165056>
- Zimmermann, T. & Nagappan, N. (2008). Predicting defects using network analysis on dependency graphs. In: *Proceedings of the 30th International Conference on Software Engineering (ICSE)*, pp. 531–540. <https://doi.org/10.1145/1368088.1368161>

ΜΕΡΟΣ Δ

ΠΑΡΑΡΤΗΜΑΤΑ

Παράρτημα Α – Data Dictionary

Α.1 Μεταβλητές ταυτοποίησης repository και Pull Request

Μεταβλητή	Τύπος	Περιγραφή	Μονάδα/μορφή	Προέλευση	Μετασχηματισμός	Χρήση στην τελική ανάλυση
pullRequestState	object	Κατάσταση του PR, π.χ. closed	Categorical string	GitHub	Καμία	Έλεγχος / φιλτράρισμα
pullRequestMerged	bool	Ένδειξη εάν το PR συγχωνεύθηκε	Boolean	GitHub	Καμία	Συμπληρωματικό metadata
baseBranchName	object	Branch προς το οποίο κατευθύνεται το PR	String	GitHub	Καμία	Metadata
headBranchName	object	Branch από το οποίο προέρχεται η αλλαγή	String	GitHub	Καμία	Metadata
Category	object	Αρχική multi-label κατηγορία ταξινόμησης και τεχνικού προσανατολισμού του PR	Categorical string	GitHub metadata + πλήρες diff/patch + LLM classification (gpt-oss-120b)	Επανακωδικοποίηση σε category_bucket, χρήση threshold 15%	Categorical predictor

A.2 Μεταβλητές snapshot και commit reconstruction

Μεταβλητή	Τύπος	Περιγραφή	Μονάδα/μορφή	Προέλευση	Μετασχηματισμός	Χρήση στην τελική ανάλυση
analysisRole	object	Προσδιορίζει τον ρόλο του snapshot, pr_base ή pr_closed	Categorical	Snapshot reconstruction pipeline	Επιλογή pr_closed για το τελικό cross-sectional dataset	Κρίσιμη για filtering και αρχική base-closed ανάλυση
commitHash	object	SHA hash του commit που αντιστοιχεί στο συγκεκριμένο snapshot	Git SHA	Git/GitHub	Καμία	Ιχνηλασιμότητα

A.3 SonarQube metadata και πληροφορίες ανάλυσης

Μεταβλητή	Τύπος	Περιγραφή	Μονάδα/μορφή	Προέλευση	Μετασχηματισμός	Χρήση στην τελική ανάλυση
sonarProjectKey	object	Μοναδικό SonarQube project key που αντιστοιχεί στην ανάλυση του PR	String	SonarQube	Καμία	Ιχνηλασιμότητα
sonarProjectName	object	Ανθρώπινα αναγνώσιμο όνομα του SonarQube project	String	SonarQube	Καμία	Metadata
sonarDashboardUrl	object	URL προς το αντίστοιχο SonarQube dashboard	URL	SonarQube	Καμία	Ιχνηλασιμότητα
sonarTaskId	object	Αναγνωριστικό του SonarQube compute task	ID / string	SonarQube	Καμία	Ιχνηλασιμότητα
sonarAnalysisId	object	Αναγνωριστικό της συγκεκριμένης SonarQube analysis	ID / string	SonarQube	Καμία	Ιχνηλασιμότητα
qualityGateStatus	object	Κατάσταση του SonarQube	Categorical string	SonarQube	Καμία	Δεν χρησιμοποιήθηκε

		Quality Gate κατά την ανάλυση				ως κύριος predictor
issuesSummaryJson	object	JSON δομή με αναλυτικές πληροφορίες για issues, severities, types, components και άλλα στοιχεία της SonarQube analysis	JSON ως string	SonarQube API	Δεν μεταφέρθηκε στο τελικό analytical dataset	Χρησιμοποιήθηκε ως raw evidence, όχι στα στατιστικά μοντέλα

Η *issuesSummaryJson* αποτελεί την πλέον λεπτομερή μη δομημένη μεταβλητή του αρχικού dataset. Περιέχει issue-level πληροφορίες. Δεν χρησιμοποιήθηκε απευθείας στις στατιστικές αναλύσεις, καθώς οι βασικές πληροφορίες που απαιτούνταν για την έρευνα είχαν ήδη μετατραπεί σε διακριτές αριθμητικές SonarQube metrics.

A.4 Βασικές μετρικές ποιότητας, ασφάλειας, μεγέθους και πολυπλοκότητας

Μεταβλητή	Τύπος	Περιγραφή	Μονάδα	Προέλευση	Μετασχηματισμός / Παράγωγη χρήση	Τελικός ρόλος
bugs	int64	Αριθμός issues που ταξινομούνται ως bugs σύμφωνα με την παραδοσιακή SonarQube taxonomy	Πλήθος	SonarQube	Διατηρήθηκε raw	Περιγραφική ανάλυση
vulnerabilities	int64	Αριθμός issues που ταξινομούνται ως vulnerabilities	Πλήθος	SonarQube	Διατηρήθηκε raw	Περιγραφική security λύση
codeSmells	int64	Αριθμός code smells	Πλήθος	SonarQube	Διατηρήθηκε raw	Περιγραφική quality λύση
duplicatedLinesDensity	float64	Ποσοστό γραμμών κώδικα που χαρακτηρίζονται ως duplicated	%	SonarQube	Χρησιμοποιήθηκε ως ξεχωριστός quality/structural indicator	Περιγραφικά, PCA, Factor Analysis
ncloc	int64	Non-comment lines of code. Αποτυπώνει το μέγεθος αναλυόμενου κώδικα	Γραμμές κώδικα	SonarQube	$ncloc > 0$, denominator των density metrics, $\log_ncloc = \ln(1 + ncloc)$	Κύριος size predictor και normalization variable

complexity	int64	Συνολική complexity του αναλυόμενου κώδικα	Complexity units	SonarQube	$\log_complexity = \ln(1 + complexity)$ και density/size-adjusted εκδοχές	Κύριος predictor, PCA/FA
cognitiveComplexity	int64	Cognitive Complexity του κώδικα, δηλαδή ένδειξη της δυσκολίας κατανόησης της λογικής ροής	Complexity units	SonarQube	$\log_cognitive = \ln(1 + cognitiveComplexity)$	Περιγραφικά, PCA/FA
softwareQualityReliabilityIssues	int64	Πλήθος issues που αποδίδονται στη διάσταση Reliability της SonarQube software-quality taxonomy	Πλήθος issues	SonarQube	Μετατροπή σε density indicator για PCA/FA	Περιγραφικά, PCA/FA
softwareQualityMaintainabilityIssues	int64	Πλήθος issues που σχετίζονται με Maintainability	Πλήθος issues	SonarQube	Αριθμητής του quality_density	Βασικό quality outcome
softwareQualitySecurityIssues	int64	Πλήθος issues που σχετίζονται με Security	Πλήθος issues	SonarQube	Αριθμητής του security_density και του security_share	Βασικό security outcome

Ιδιαίτερη διάκριση πρέπει να γίνεται μεταξύ των παραδοσιακών μεταβλητών bugs, vulnerabilities, codeSmells και των softwareQualityReliabilityIssues, softwareQualitySecurityIssues και SoftwareQualityMaintainabilityIssues. Οι δύο ομάδες προέρχονται από διαφορετικές ταξινομητικές οπτικές του SonarQube και για τον λόγο αυτό διατηρήθηκαν ως ξεχωριστές μεταξύ τους μεταβλητές.

A.5 Παράγωγες μεταβλητές τελικού analytical dataset

Μεταβλητή	Τύπος	Ορισμός / υπολογισμός	Μονάδα	Ρόλος
category_bucket	object	Ενοποιημένη κατηγορία με τρεις (3) τελικές τιμές: <i>quality</i> , <i>security</i> , <i>both</i>	Category	Categorical predictor
quality_density	float64	softwareQualityMaintainabilityIssues / ncloc * 1000	Maintainability Issues / KLOC	Κύριο quality outcome πριν από log transformation
security_density	float64	softwareQualitySecurityIssues / ncloc * 1000	Security issues / KLOC	Κύριο security outcome πριν από log transformation
total_issues	int64	softwareQualityMaintainabilityIssues +	Πλήθος issues	Ενδιάμεση μεταβλητή

		softwareQualitySecurity Issues		
security_share	float64	Αναλογία των security issues προς το σύνολο maintainability και security issues	Αναλογία [0-1]	Outcome στη συμπληρωματική GEE analysis
log_ncloc	float64	$\ln(1 + \text{ncloc})$	Log scale	Κύριος predictor
log_complexity	float64	$\ln(1 + \text{complexity})$	Log scale	Κύριος predictor
log_cognitive	float64	$\ln(1 + \text{cognitiveComplexity})$	Log scale	Παράγωγος complexity indicator
log_quality_dens ity	float64	$\ln(1 + \text{quality_density})$	Log scale	Κύριο outcome στα quality OLS/LMM
log_security_den sity	float64	$\ln(1 + \text{security_density})$	Log scale	Κύριο outcome στα security OLS/LMM
project	object	Αντίγραφο/αναλυτική αναπαράσταση του repositoryName	Category	Grouping variable στα hierarchical models
n_prs_in_project	int64	Αριθμός PRs του project στο τελικό analytical dataset	Πλήθος PRs	Cluster-size indicator και sensitivity filtering
lmm_fitted_log_ density	float64	Προβλεπόμενη τιμή του LMM για το log-transformed quality density	Log scale	Model diagnostics
lmm_residual	float64	Διαφορά μεταξύ παρατηρούμενου log_quality_density και fitted value	Log scale	Residual diagnostics
cat_quality	int64	Dummy indicator: 1 όταν <i>category_bucket = quality</i> , αλλιώς 0	Binary 0 1	Dummy coding, quality reference category
cat_security	int64	Dummy indicator: 1 όταν <i>category_bucket = security</i> , διαφορετικά 0	Binary 0 1	Category predictor
cat_both	int64	Dummy indicator: 1 όταν <i>category_bucket = both</i> , διαφορετικά 0	Binary 0 1	Category predictor

Οι density μεταβλητές εκφράστηκαν ανά 1.000 γραμμές κώδικα (KLOC) προκειμένου να περιοριστεί η άμεση εξάρτηση των απόλυτων issue counts από το μέγεθος του κώδικα. Οι λογαριθμικοί μετασχηματισμοί της μορφής $\ln(1 + x)$ επέτρεψαν τη διατήρηση των μηδενικών παρατηρήσεων και περιόρισαν την έντονη εμφανιζόμενη δεξιά ασυμμετρία των αρχικών κατανομών.

Η *category_bucket* αποτέλεσε την απλοποιημένη εκδοχή της αρχικής Category. Στα regression models η κατηγορία quality χρησιμοποιήθηκε ως reference category, επομένως οι συντελεστές των security και both εκφράζουν διαφοροποίηση σε σχέση με αυτή.

A.6 Μεταβλητές του normalized snapshot-level dataset

Μεταβλητή	Τύπος	Ορισμός	Ερμηνεία / τελική χρήση
bugsPerLineOfCode	float64	bugs / ncloc	Bugs ανά γραμμή κώδικα, αρχική normalized μετρική
vulnerabilitiesPerLineOfCode	float64	Vulnerabilities / ncloc	Vulnerabilities ανά γραμμή κώδικα
codeSmellsPerLineOfCode	float64	codeSmells / ncloc	Code smells ανά γραμμή κώδικα
securityHotspotsPerLineOfCode	float64	securityHotspots / ncloc	Security hotspots ανά γραμμή κώδικα
duplicatedLinesDensityPerLineOfCode	float64	duplicatedLinesDensity / ncloc	Αρχική normalization του duplication density, δεν αποτελεί την κύρια τελική αναπαράσταση της duplication μετρικής
nclocPerLineOfCode	float64	ncloc / ncloc	Ισούται με 1 για ncloc θετικό, δεν έχει αυτόνομη αναλυτική πληροφορία
complexityPerLineOfCode	float64	complexity / ncloc	Complexity ανά γραμμή κώδικα
cognitiveComplexityPerLineOfCode	float64	cognitiveComplexity / ncloc	Cognitive Complexity ανά γραμμή κώδικα
softwareQualityReliabilityIssuesPerLineOfCode	float64	softwareQualityReliabilityIssues / ncloc	Reliability issue density ανά γραμμή κώδικα
softwareQualityMaintainabilityIssuesPerLineOfCode	float64	softwareQualityMaintainabilityIssues / ncloc	Maintainability issue density ανά γραμμή κώδικα
softwareQualitySecurityIssuesPerLineOfCode	float64	softwareQualitySecurityIssues / ncloc	Security issue density ανά γραμμή κώδικα

A.7 Ρόλος των μεταβλητών στην τελική στατιστική στρατηγική

Αναλυτικός ρόλος	Μεταβλητές
Κύρια quality outcome	quality_density, log_quality_density
Κύρια security outcome	security_density, log_security_density
Count outcomes / robustness	softwareQualityMaintainabilityIssues, softwareQualitySecurityIssues
Security proportion outcome	security_share
Κύριοι continuous predictors	log_ncloc, log_complexity
Συμπληρωματικοί complexity metric	cognitiveComplexity, log_cognitive
Categorical predictor	category_bucket, cat_security, cat_both
Grouping variable	project
Cluster-size / sensitive variable	n_prs_in_project
PCA / Factor analysis indicators	reliability density, maintainability density, security density, hotspot density, complexity

	density, cognitive-complexity density, duplicatedLinesDensity
LMM diagnostics	lmm_fitted_log_density, lmm_residual
Snapshot selection / longitudinal diagnostics	analysisRole
PR identification	pr_key, repositoryName, pullRequestNumber

Παράρτημα B – Data Collection, Snapshot Reconstruction, Data Cleaning και Reproduction

Το παρόν παράρτημα συνοψίζει τη διαδικασία συλλογής, ανακατασκευής, καθαρισμού και προετοιμασίας των δεδομένων που οδήγησε στο τελικό analytical dataset της έρευνας.

B.1 Data Collection Pipeline

Τα δεδομένα συλλέχθηκαν από PRs διαφόρων repositories του GitHub και συνδυάστηκαν με πληροφορίες από Git commits και αποτελέσματα στατικής ανάλυσης με την χρήση του εργαλείου SonarQube. Για κάθε Pull Request καταγράφηκαν metadata, στοιχεία commits, κατηγορία PR και οι βασικές quality, security, size και complexity metrics.

Η μεταβλητή Category δημιουργήθηκε σε ξεχωριστό classification stage πριν από την τελική στατιστική επεξεργασία. Για κάθε Pull Request χρησιμοποιήθηκαν ο τίτλος, το body/description, τα GitHub labels, τα changed files, τα διαθέσιμα comments και reviews και το πλήρες .diff/.patch. Οι πληροφορίες αυτές υποβλήθηκαν στο μοντέλο gpt-oss-120b, το οποίο μπορούσε να αποδώσει έναν ή περισσότερους χαρακτηρισμούς εκ του συνόλου code_quality, code_security, software_quality, software_security. Για τη διατήρηση κάθε χαρακτηρισμού εφαρμόστηκε threshold 15%. Τα παραγόμενα classification outputs αποθηκεύτηκαν στο dataset και στη συνέχεια κωδικοποιήθηκαν ξανά στις κατηγορίες quality, security και both για τις τελικές αναλύσεις. Δεν πραγματοποιήθηκε ανεξάρτητη χειροκίνητη επικύρωση των αρχικών classifications.

Χρησιμοποιημένο Prompt για το LLM που διενέργησε την αρχική ταξινόμηση:

System prompt:

You are a strict JSON-only classifier for GitHub Pull Requests.

GOAL

Given ONLY:

- 1) a public GitHub repository URL, and*
- 2) a GitHub Pull Request URL (open or closed/merged),*
- 3) fetched PR .diff content, and*
- 4) fetched PR .patch content,*

you must classify the PR by using the provided data, with primary focus on .diff/.patch content.

Then you must classify the PR into one or more categories:

- "code_quality"*
- "code_security"*
- "software_quality"*
- "software_security"*

A PR may belong to multiple categories.

You must also recognize when the PR includes changes outside those 4 categories. You will express that implicitly by keeping the sum of affectedPercentages across the 4 categories ≤ 100 , leaving the remaining portion as "other changes".

CRITICAL OUTPUT RULES (NON-NEGOTIABLE)

- 1) Output **MUST** be a single JSON object as plain text.
- 2) Output **MUST** be valid JSON parseable by `JSON.parse` in JavaScript:
 - Use double quotes for ALL keys and ALL string values.
 - No trailing commas.
 - No markdown, no code fences, no commentary, no extra text.
- 3) Output **MUST** strictly match this schema and include ALL keys:

```
{
  "category": ("code_quality" | "code_security" | "software_quality" | "software_security"),
  "percentage": number,
  "affectedCategories": {
    "codeQuality": { "affectedPercentage": number, "percentage": number },
    "codeSecurity": { "affectedPercentage": number, "percentage": number },
    "softwareQuality": { "affectedPercentage": number, "percentage": number },
    "softwareSecurity": { "affectedPercentage": number, "percentage": number }
  }
}
```
- 4) Numbers are integers 0–100. (No decimals.)
- 5) Always include all four affectedCategories entries, even if they are 0.
- 6) Never invent facts. If the provided `.diff/.patch` content is empty or insufficient, return a low-confidence "unknown" result as defined in FAILSAFE.

DATA USAGE BEHAVIOR (NO WEB SEARCH / NO BROWSING)

You are given URLs and pre-fetched raw data. You **MUST NOT** request web search or browsing.
You **MUST** use the provided `.diff/.patch` content as the source of truth for classification.
You may use PR URL/repo URL only as metadata context.

FAILSAFE (WHEN CONTENT IS NOT ACCESSIBLE / INSUFFICIENT)

If the provided content is not enough to classify (e.g., missing/empty/non-meaningful diff and patch), you **MUST** return:

- "category": []
- "percentage": 0
- all affectedPercentage: 0
- all per-category percentage: 0
- another one property inside the json with format "failReason": <HERE YOU SHOULD PUT THE FAIL REASON AS STRING>

This is the **ONLY** acceptable behavior when content is insufficient. Do not ask questions. Do not output errors.
Do not output anything except valid JSON.

DEFINITIONS (WHAT EACH CATEGORY MEANS)

... (UNCHANGED: keep the rest of the system prompt exactly as provided) ...

User prompt:

repo_url: {{repo_url}}
pr_url: {{pr_url}}

Use **ONLY** the following pre-fetched data for your analysis. Do not use web search.

pr_diff_content:
{{pr_diff_content}}

pr_patch_content:
{{pr_patch_content}}

Η συνολική διαδικασία ακολούθησε στην εξής ροή:

Repository Selection $\Rightarrow$ *Pull Request Collection* $\Rightarrow$ *Snapshot Reconstruction* $\Rightarrow$ *Static Analysis* $\Rightarrow$ *Metrics Extraction* $\Rightarrow$ *Data Cleaning* $\Rightarrow$ *Statistical Analysis*

B.2 Snapshot Reconstruction (*pr_base* / *pr_closed*)

Για κάθε Pull Request επιχειρήθηκε η δημιουργία δύο snapshots:

- *pr_base*: κατάσταση αναφοράς πριν από την ολοκλήρωση της αλλαγής
- *pr_closed*: τελική κατάσταση του Pull Request

Η αρχική πρόθεση ήταν η ανάλυση της μεταβολής μεταξύ των δύο snapshots. Το 94-99,9%, όμως, των εξεταζόμενων μετρικών παρέμενε αμετάβλητο, με αποτέλεσμα οι σχετικές deltas να μην παρέχουν επαρκή μεταβλητότητα. Για τον λόγο αυτό, η κύρια ανάλυση μετατράπηκε σε cross-sectional και διατηρήθηκε μία *pr_closed* παρατήρηση ανά Pull Request.

B.3 Data Cleaning και Filtering

Η διαδικασία καθαρισμού περιλάμβανε, με βήματα:

1. διατήρηση μόνο Pull Requests με έγκυρο *pr_base-pr_closed* pair
2. επιλογή του *pr_closed* snapshot
3. αποκλεισμό παρατηρήσεων με *ncloc* = 0
4. ενοποίηση της κατηγορίας PR στις τιμές quality, security και both

Η τελική ροή φιλτραρίσματος και καθαρισμού δεδομένων ήταν:

20.880 εγγραφές | 353 projects $\Rightarrow$ 9.813 valid base-closed PR pairs $\Rightarrow$ 9.813 *pr_closed* snapshots $\Rightarrow$ 3.702 PRs | 242 projects με *ncloc* > 0 $\Rightarrow$ 3.701 PRs | 242 projects στο τελικό δείγμα

B.4 Exclusion Log

Στάδιο	Παρατηρήσεις που παρέμειναν	Αιτία αποκλεισμού
Αρχικό dataset	20.880 εγγραφές	-
Valid pairs	9.813 PRs	298 PRs χωρίς αναμενόμενο pair structure
<i>pr_closed</i>	9.813 PRs	Επιλογή μίας τελικής παρατήρησης ανά PR
<i>ncloc</i> > 0	3.702 PRs	Αποκλεισμός snapshots χωρίς αναλυόμενο κώδικα
Τελική κατηγορία	3.701 PRs	1 PR εκτός αναμενόμενης κατηγορίας (quality-security-both)

B.5 Δημιουργία τελικού analytical dataset

Στο τελικό dataset δημιουργήθηκαν οι βασικές παράγωγες μεταβλητές της ανάλυσης, μεταξύ αυτών:

- quality_density

- security_density
- log_ncloc
- log_complexity
- log_quality_density
- log_security_density
- category_bucket
- project
- n_prs_in_project

Οι density metrics εκφράστηκαν ανά KLOC, ενώ οι βασικές συνεχείς μεταβλητές μετασχηματίστηκαν όπου απαιτούνταν με $\ln(1 + x)$ για τον περιορισμό της έντονης ασυμμετρίας.

B.6 Reproduction Guide

Για την αναπαραγωγή της ανάλυσης απαιτούνται:

- το αρχικό καθαρισμένο dataset
- η ίδια διαδικασία filtering
- οι ίδιοι ορισμοί των derived variables
- τα ίδια statistical scripts και dependencies
- οι ίδιες παράμετροι στις stochastic διαδικασίες, όπου αυτές εμφανίζονται και εφαρμόζονται.

Η ορθή αναπαραγωγή πρέπει να καταλήγει στα βασικά checkpoints, όπως αυτά έχουν οριστεί, των checkpoints των 3.701 Pull Requests από 242 projects και στην ίδια τελική δομή των μεταβλητών που χρησιμοποιήθηκαν στις αναλύσεις OLS, LMM, PCA, Factor Analysis και robustness analyses.

Παράρτημα Γ – Πλήρεις περιγραφικές αναλύσεις και correlation matrices

Γ.1 Περιγραφικά στατιστικά βασικών μεταβλητών

Variable	Mean	SD	P25	Median	P75	P95	Zero %	Min	Max
ncloc	25.517,77	106.512,15	322	2.128	15.190	87.225	0,00 %	1	1.216.504
complexity	2.184,86	8.135,46	0	50	637	7.903	31,10 %	0	76.869
cognitiveComplexity	2.226,38	9.822,02	0	19	411	9.321	39,04 %	0	105.622
duplicatedLinesDensity	4,57	11,67	0	0	2,80	28,50	61,79 %	0	97,80
bugs	43,04	321,62	0	0	4	302	63,77 %	0	13.709
codeSmells	1.292,53	8.265,54	1	13	202	3.319	22,45 %	0	98.685
softwareQualityReliabilityIssues	269,65	2.498,86	0	0	13	535	56,55 %	0	30.810

softwareQualityMaintainabilityIssues	1.288,21	8.265,79	1	12	202	3.256	22,48 %	0	98.678
vulnerabilities	1,23	6,11	0	0	0	6	80,98 %	0	135
softwareQualitySecurityIssues	1,55	6,68	0	0	0	8	78,36 %	0	135
securityHotspots	17,48	104,01	0	0	4	40	54,36 %	0	1.733
quality_density	28,11	53,90	0,41	9,02	31,96	106,43	22,48 %	0	877,55
security_density	0,34	2,51	0	0	0	0,75	78,36 %	0	61,22

Γ.2 Pearson correlations των κύριων αναλυτικών μεταβλητών

	log_ncloc	log_complexity	log_quality_density	log_security_density
log_ncloc	1,000	0,714	0,279	-0,055
log_complexity	0,714	1,000	0,557	-0,064
log_quality_density	0,279	0,557	1,000	0,175
log_security_density	-0,055	-0,064	0,175	1,000

Γ.3 Spearman correlations

	log_ncloc	log_complexity	log_quality_density	log_security_density
log_ncloc	1,000	0,702	0,308	0,364
log_complexity	0,702	1,000	0,577	0,309
log_quality_density	0,308	0,577	1,000	0,323
log_security_density	0,364	0,309	0,323	1,000

Παράρτημα Δ – Πλήρη στατιστικά μοντέλα, diagnostics και robustness analyses

Δ.1 OLS models με cluster-robust standard errors

Quality outcome – log_quality_density

Predictor	β	SE	p-value	95% CI
Intercept	2,049	0,481	< 0,001	[1,106, 2,992]
both	-0,004	0,091	0,968	[-0,183, 0,176]
security	0,008	0,094	0,933	[-0,177, 0,193]
log_ncloc	-0,169	0,073	0,021	[-0,313, -0,025]

log_complexity	0,368	0,046	< 0,001	[0,278, 0,458]
----------------	-------	-------	---------	----------------

$R^2 = 0,339$, adjusted $R^2 = 0,338$ και συνολικό $F = 23,32$, $p < 0,001$

Security outcome – log_security_density

Predictor	β	SE	p-value	95% CI
Intercept	0,145	0,101	0,152	[-0,053, 0,342]
both	0,046	0,028	0,082	[-0,006, 0,104]
security	-0,015	0,021	0,474	[-0,056, 0,026]
log_ncloc	-0,003	0,011	0,768	[-0,025, 0,018]
log_complexity	-0,006	0,011	0,595	[-0,027, 0,016]

To security model παρουσίασε σαφώς χαμηλότερη explanatory capacity, με $R^2 = 0,006$ και adjusted $R^2 = 0,005$.

Δ.2 Linear Mixed Models

Τα κύρια LMM εκτιμήθηκαν με random intercept ανά project και REML. Και τα δύο πλήρη random-intercept models συγκλίνουν επιτυχώς.

Fixed effects

Predictor	Quality β	Quality p	Security β	Security p
both	-0,027	0,368	0,000	0,991
security	-0,038	0,356	-0,008	0,320
log_ncloc	-0,152	< 0,001	-0,004	0,455
log_complexity	0,398	< 0,001	-0,011	0,017

Η πολυπλοκότητα διατήρησε ισχυρή θετική σχέση με το quality outcome, ενώ στο security model η αντίστοιχη σχέση ήταν μικρή και αρνητική. Οι κατηγορίες των Pull Requests δεν παρουσίασαν στατιστικά σημαντικές διαφορές στα κύρια LMM.

Variance components και ICC

Outcome	Μοντέλο	Project variance	Residual variance	ICC
log_quality_density	Null (N)	2,868	0,397	87,8 %
log_quality_density	Full (F)	1,825	0,3616	83,5 %
log_security_density	Null (N)	0,155	0,013	92,2 %
log_security_density	Full (F)	0,156	0,0130	92,3 %

Οι ιδιαίτερα υψηλές τιμές ICC τεκμηριώνουν την έντονη ομαδοποίηση των Pull Requests ανά project. Στο quality model η project-level variance μειώθηκε αφού εισήχθησαν τα fixed effects, ενώ στο security model παρέμεινε πρακτικά αμετάβλητη.

Δ.3 Diagnostics και model convergence

Τα OLS diagnostics κατέδειξαν σαφέστερες αποκλίσεις από την κανονικότητα κυρίως στο security outcome. Για το quality model η skewness ήταν 0,322, ενώ για το security model έφτασε το 5,356, με ιδιαίτερα υψηλό kurtosis. Το αποτέλεσμα είναι συμβατό με την έντονα ασύμμετρη και υψηλή συγκέντρωση παρατηρήσεων στο μηδέν του security outcome και αποτέλεσε έναν από τους λόγους για τους οποίους τα κύρια αποτελέσματα εξετάστηκαν συμπληρωματικά μέσω εναλλακτικών model specifications. Ειδικότερα τα Negative Binomial models χρησιμοποιήθηκαν για την άμεση μοντελοποίηση των issue counts, ενώ το GEE εξέτασε διαφορετικό population-averaged outcome σχετικό με τη σύνθεση των security και maintainability findings.

Τα βασικά random-intercept LMM συγκλίνουν επιτυχώς. Το περισσότερο σύνθετο, αντίθετα, quality model με random slope του log_ncloc ανά project δεν συγκλίνει. Κατά συνέπεια, οι εκτιμήσεις του συγκεκριμένου μοντέλου δεν χρησιμοποιήθηκαν για inferential conclusions και η κύρια ανάλυση παρέμεινε στη πιο σταθερή random-intercept specification.

Δ.4 Robustness analyses

Negative Binomial models

Για την απευθείας μοντελοποίηση των issue counts εφαρμόστηκαν Negative Binomial GLMs με *offset=log(ncloc)* και cluster-robust standard errors.

Outcome	Predictor	β	p-value
Maintainability issues	<i>security category</i>	-0,538	0,003
Maintainability issues	<i>both category</i>	-0,222	0,116
Maintainability issues	<i>log_complexity</i>	0,214	< 0,001
Security issues	<i>security category</i>	0,010	0,970
Security issues	<i>both category</i>	0,568	0,002
Security issues	<i>log_complexity</i>	-0,058	0,441

Τα αποτελέσματα επιβεβαιώνουν τη θετική σχέση της πολυπλοκότητας με τα maintainability issues, ενώ ορισμένες category effects εμφανίζονται μόνο στη Negative Binomial specification. Για τον λόγο αυτό, οι συγκεκριμένες σχέσεις αντιμετωπίζονται ως model-sensitive και όχι ως καθολικά ευρήματα.

GEE security-share model

Το συμπληρωματικό GEE Binomial model εφαρμόστηκε σε 2.877 PRs από 186 projects με τουλάχιστον ένα maintainability ή security issue.

Predictor	β	p-value	95% CI
log_ncloc	-0,092	0,527	[-0,378, 0,193]
log_complexity	-0,280	0,135	[-0,646, 0,087]

Ούτε το μέγεθος ούτε η πολυπλοκότητα παρουσίασαν σημαντική σχέση με την αναλογία security issues προς το σύνολο των security και maintainability issues.

Sensitivity analysis – projects με ≥ 5 PRs

Για να εξεταστεί η επίδραση των πολύ μικρών clusters, το quality LMM επαναλήφθηκε αποκλειστικά στα projects με τουλάχιστον πέντε (5) Pull Requests. Το υποσύνολο περιλάμβανε 3.544 PRs από 149 διαφορετικά projects.

Predictor	β	p-value	95% CI
both	-0,022	0,465	[-0,081, 0,037]
security	-0,021	0,603	[-0,100, 0,058]
log_ncloc	-0,141	< 0,001	[-0,194, -0,088]
log_complexity	-0,369	< 0,001	[0,324, 0,414]

Δ.5 Συνολική αποτίμηση των ελέγχων

Οι εναλλακτικές στατιστικές προδιαγραφές δεν παρήγαγαν πλήρως ταυτόσημα αποτελέσματα, αλλά τα ισχυρότερα ευρήματα παρέμειναν συνεπή. Ιδιαίτερα σταθερές υπήρχαν η σχέση πολυπλοκότητας-quality outcomes και η έντονη project-level ετερογένεια. Αντίθετα, οι περισσότερες σχέσεις που αφορούσαν τα security outcomes και τις κατηγορίες των PRs ήταν ασθενέστερες ή εξαρτώμενες από το μοντέλο και ερμηνεύτηκαν περισσότερο επιφυλακτικά.

Παράρτημα Ε – Πλήρη PCA, Factor Analysis και συμπληρωματικά γραφήματα

Το παρόν παράρτημα συγκεντρώνει τα αναλυτικότερα αποτελέσματα της Principal Component Analysis και της Exploratory Factor Analysis. Οι δύο αυτές στατιστικές μέθοδοι εφαρμόστηκαν στις ίδιες επτά βασικές μεταβλητές, με στόχο να είναι δυνατή η άμεση σύγκριση της πολυμεταβλητής δομής που προκύπτει από καθεμία.

E.1 Principal Component Analysis

Η κύρια PCA εφαρμόστηκε στις επτά (7) standardized per-KLOC/density μεταβλητές. Τόσο η Parallel Analysis όσο και το Kaiser criterion υπέδειξαν τη διατήρηση τριών κύριων συνιστωσών, οι οποίες εξηγούσαν συνολικά το ~77,5% της συνολικής διακύμανσης.

Component	Eigenvalue	Explained variance	Cumulative variance
PC1	2,806	40,1 %	40,1 %
PC2	1,547	22,1 %	62,2%
PC3	1,069	15,3%	77,5%

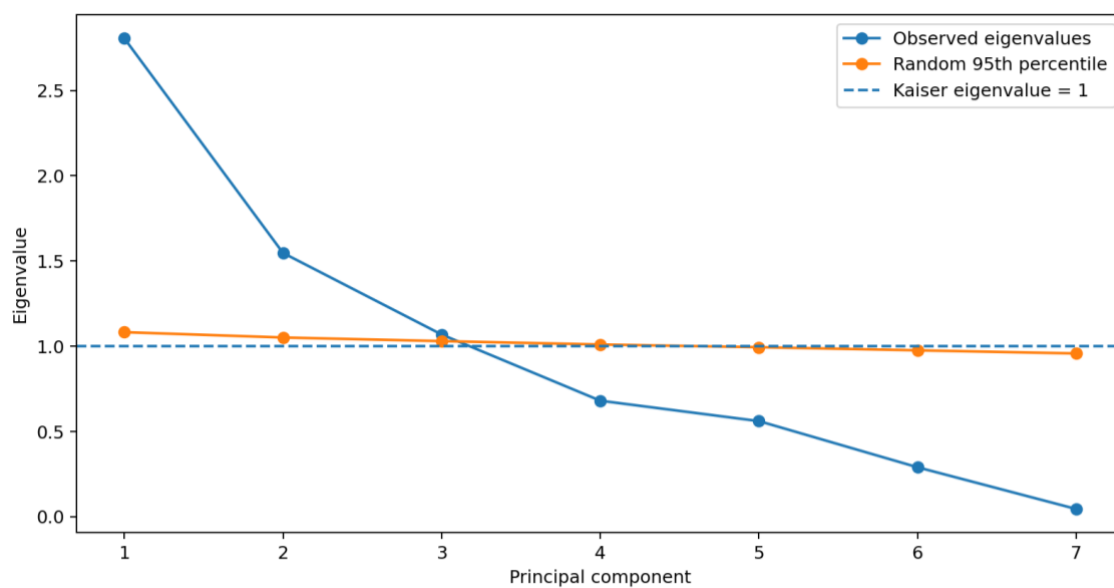
Οι τρεις συνιστώσες παρουσίασαν την ακόλουθη βασική δομή loadings:

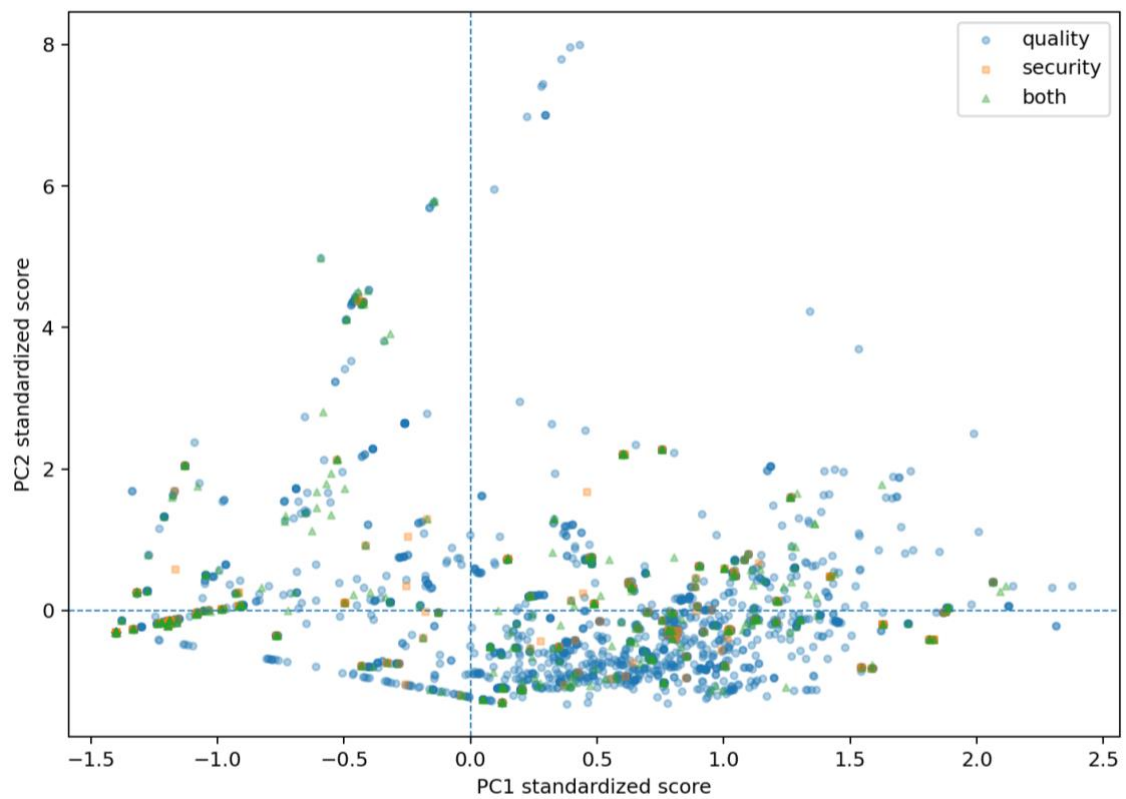
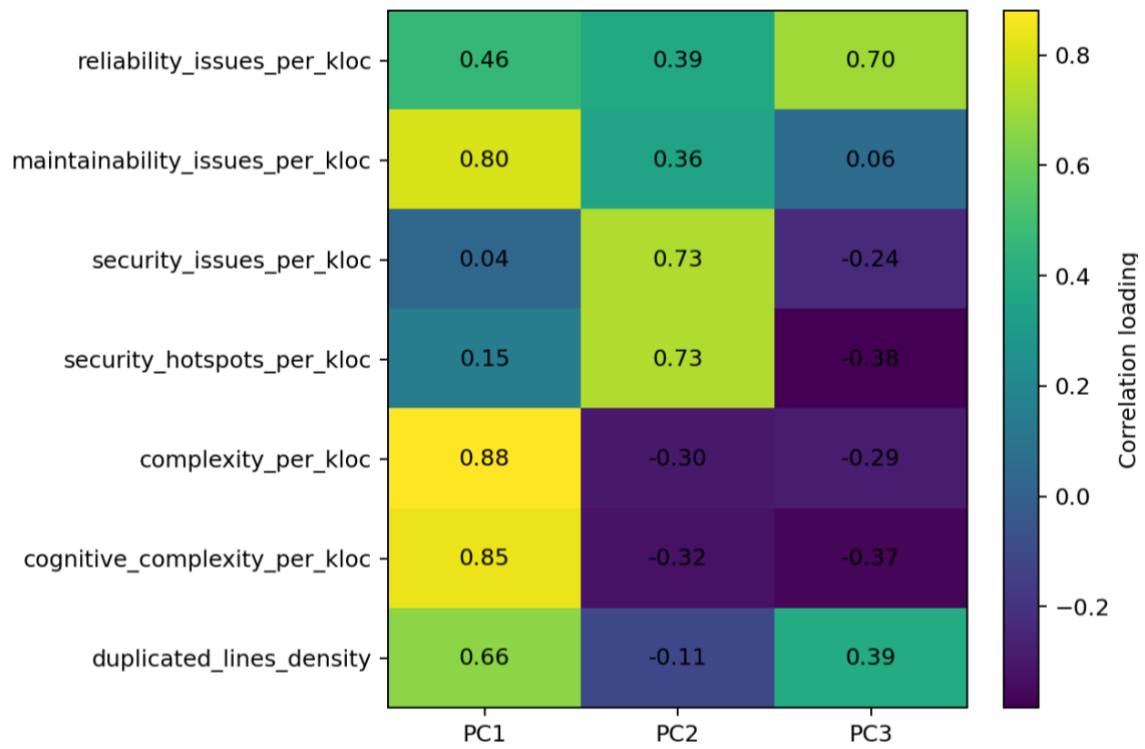
Μεταβλητή	PC1	PC2	PC3
Reliability issues / KLOC	0,458	0,385	0,700

Maintainability issues / KLOC	0,803	0,357	0,062
Security issues / KLOC	0,041	0,762	-0,235
Security hotspots / KLOC	0,152	0,730	-0,384
Complexity / KLOC	0,880	-0,268	-0,289
Cognitive complexity / KLOC	0,845	-0,321	-0,365
Duplicated lines density	0,661	-0,107	0,395

Το PC1 αντιστοιχεί κυρίως σε διάσταση πολυπλοκότητας και συντηρησιμότητας, το PC2 σε περισσότερο διακριτή διάσταση ασφάλειας, ενώ το PC3 σχετίζεται κυρίως με την αξιοπιστία και παρουσιάζει περισσότερο μικτή δομή.

Γραφήματα τα οποία αντιστοιχούν στην ανάλυση:





E.2 Factor Analysis

Ο βαθμός καταλληλότητας των δεδομένων για Factor Analysis αξιολογήθηκε με KMO = 0,631 και στατιστικά σημαντικό Bartlett's test:

$$\begin{aligned}x^2 &= 13.877,232 \\df &= 21 \\p &< 0,001\end{aligned}$$

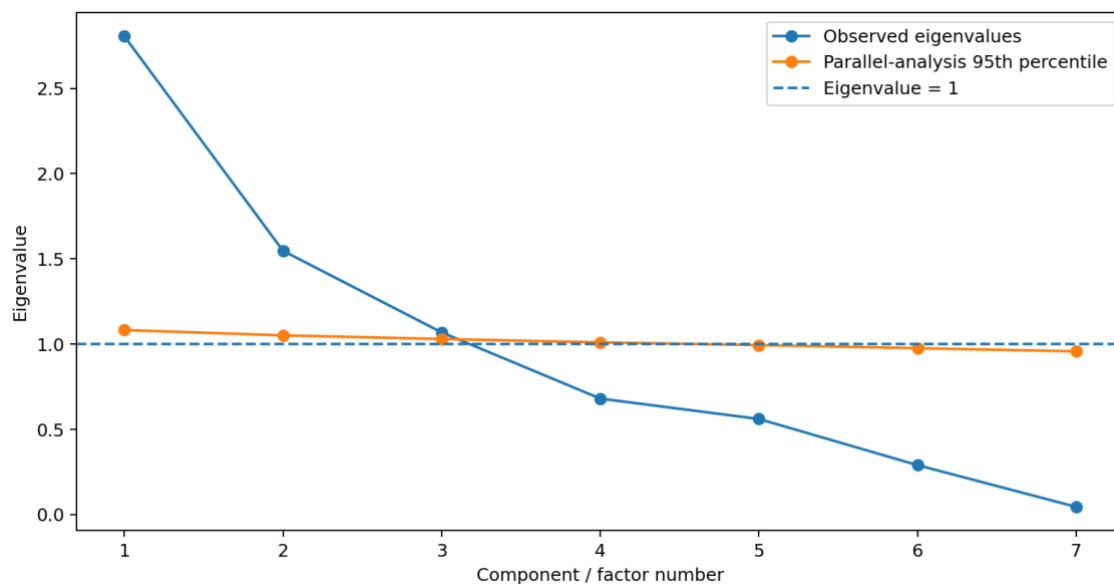
Η Parallel Analysis υπέδειξε τη διατήρηση τριών factors.

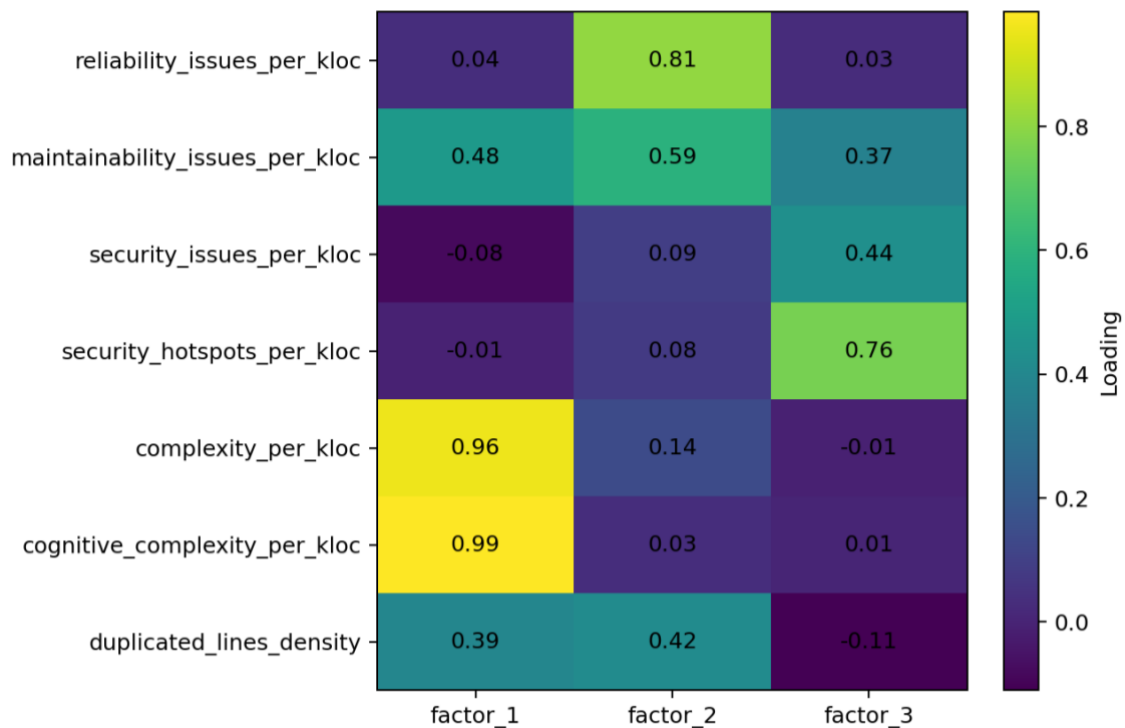
Η Varimax-rotated λύση ήταν:

Μεταβλητή	Factor 1	Factor 2	Factor 3	Communality
Reliability issues / KLOC	0,035	0,807	0,028	0,653
Maintainability issues / KLOC	0,484	0,589	0,373	0,720
Security issues / KLOC	-0,081	0,092	0,436	0,206
Security hotspots / KLOC	-0,007	0,077	0,759	0,582
Complexity / KLOC	0,959	0,139	-0,008	0,940
Cognitive complexity / KLOC	0,985	0,033	0,005	0,972
Duplicated lines density	0,394	0,418	-0,110	0,342

Ο Factor 1 εκφράζει κυρίως δομική πολυπλοκότητα και συντηρησιμότητα, ο Factor 2 αξιοπιστία και συντηρησιμότητα, ενώ ο Factor 3 δημιουργεί μεγαλύτερη σύνδεση με τη διάσταση της ασφάλειας. Ο τρίτος παράγοντας πρέπει να ερμηνεύεται περισσότερο επιφυλακτικά, καθώς στηρίζεται σε δύο salient indicators και χαρακτηρίστηκε από το verified analysis ως περισσότερο εύθραυστος.

Ιδιαίτερα σημαντική είναι η χαμηλή communality των *security_issues_per_kloc* (0,206) και η αντίστοιχα υψηλή uniqueness (0,794), γεγονός το οποίο δείχνει ότι μεγάλο μέρος της διακύμανσής τους δεν εξηγείται από την τριπαραγοντική λύση. Αντίθετα, complexity και cognitive complexity εμφανίζουν πολύ υψηλές communalities, 0,940 και 0,972 αντίστοιχα.





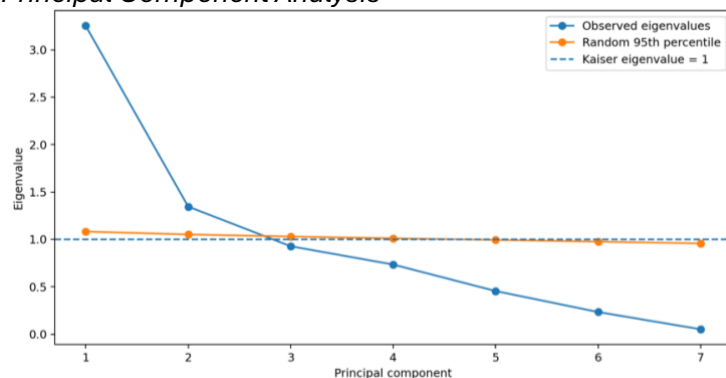
E.3 Size-adjusted sensitivity analyses

Στην PCA, η κύρια λύση των τριών (3) PCs μειώθηκε σε δύο (2) PCs, τα οποία ήταν ικανά να εξηγήσουν περίπου το 65,7% της συνολικής διακύμανσης. Αντίστοιχα, στην Factor Analysis διατήρησε 2 factors αντί για 3.

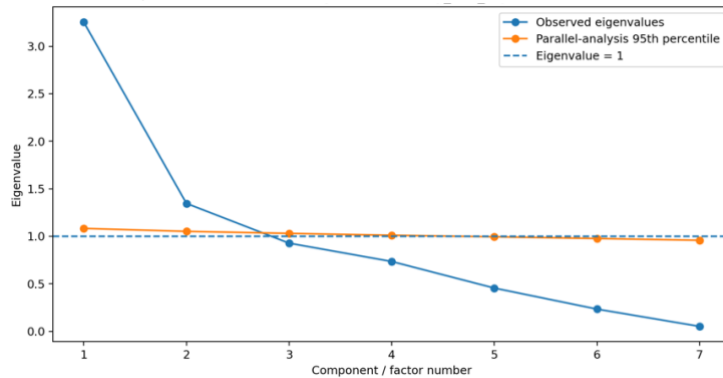
Στατιστική μεθοδολογία	Κύρια λύση	Size-adjusted λύση
PCA	3 PCs	2 PCs
Factor Analysis	3 factors	2 factors

Παρά τη παρατηρούμενη φυσική μεταβολή του ακριβούς αριθμού διαστάσεων, το βασικό πρότυπο παρέμεινε. Η πολυπλοκότητα και η συντηρησιμότητα σχηματίζουν την ισχυρότερη και πιο σταθερή κοινή διάσταση. Η μεταβολή από τρεις (3) σε δύο (2) διαστάσεις δείχνει ότι η ακριβής dimensionality είναι ευαίσθητη στον τρόπο ελέγχου του μεγέθους του κώδικα και δεν θα πρέπει να λαμβάνεται ως οριστική.

Principal Component Analysis



Factor Analysis



Ε.4 Συνολική σύγκριση μεταξύ των PCA και Factor Analysis

Διάσταση	PCA	Factor Analysis
Πολυπλοκότητα / Συντηρησιμότητα	PC1	Factor 1
Ασφάλεια	PC2	Factor 3
Αξιοπιστία	PC3	Factor 2
Κύρια dimensionality	3 PCs	3 factors
Size-adjusted sensitivity	2 PCs	2 factors

Η συμφωνία αυτή υποστηρίζει την ύπαρξη επαναλαμβανόμενων δομικών προτύπων στις μετρικές, χωρίς να συνεπάγεται ότι τρεις κύριες διαστάσεις αποτελούν οριστική ή μοναδική αναπαράσταση των δεδομένων. Η Factor Analysis ειδικότερα δείχνει ότι η security dimension είναι λιγότερο σταθερή, ενώ και οι δύο μέθοδοι εμφανίζουν μεταβολή της dimensionality μετά τον έλεγχο για το μέγεθος του κώδικα.